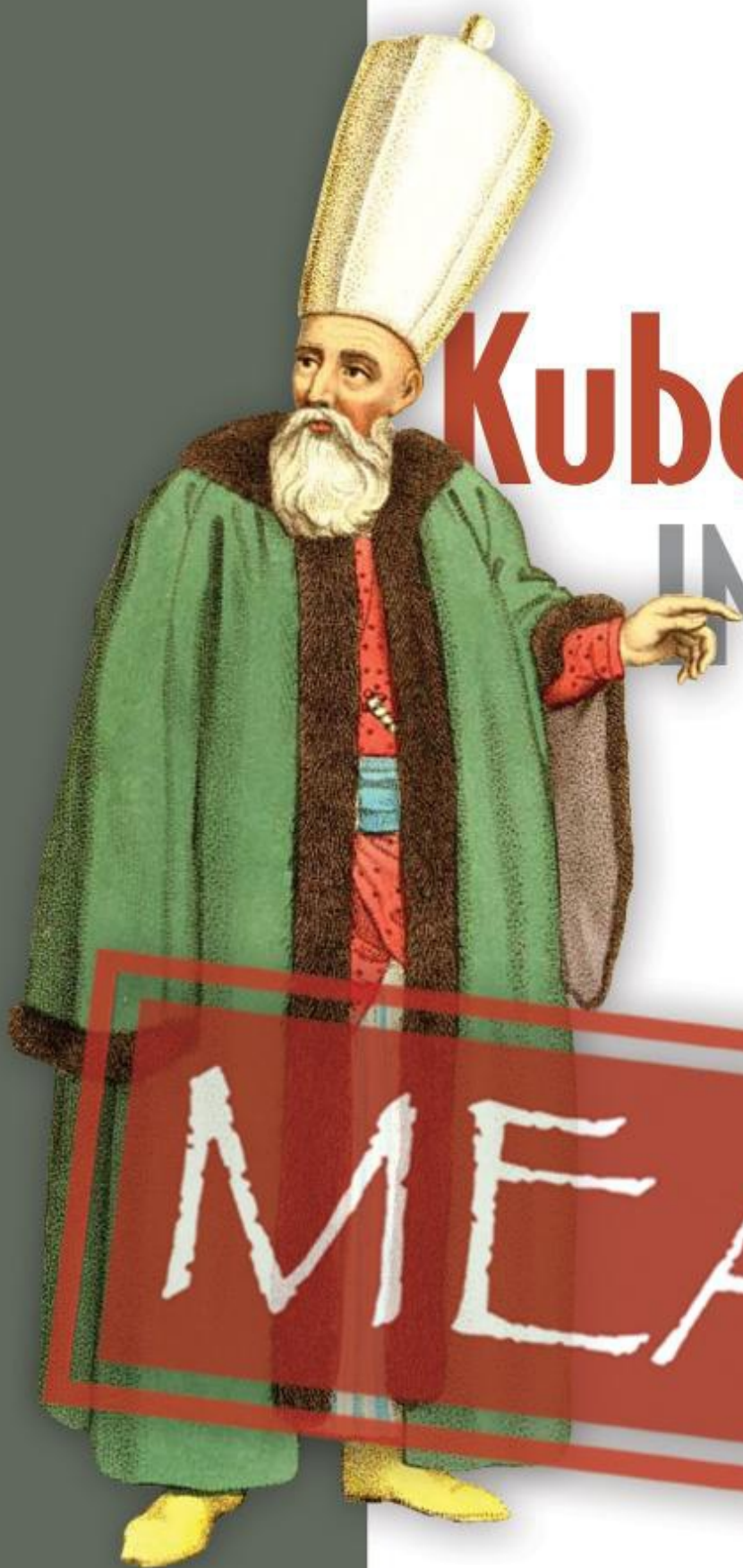




Kubernetes IN ACTION

SECOND EDITION

Marko Lukša

MEAP



Kubernetes IN ACTION

SECOND EDITION

Marko Lukša

MEAP

 MANNING

Kubernetes in Action, Second Edition MEAP V15

1. [Copyright 2023 Manning Publications](#)
2. [welcome](#)
3. [1 Introducing Kubernetes](#)
4. [2 Understanding containers](#)
5. [3 Deploying your first application](#)
6. [4 Introducing Kubernetes API objects](#)
7. [5 Running workloads in Pods](#)
8. [6 Managing the Pod lifecycle](#)
9. [7 Attaching storage volumes to Pods](#)
10. [8 Persisting data in PersistentVolumes](#)
11. [9 Configuration via ConfigMaps, Secrets, and the Downward API](#)
12. [10 Organizing objects using Namespaces and Labels](#)
13. [11 Exposing Pods with Services](#)
14. [12 Exposing Services with Ingress](#)
15. [13 Replicating Pods with ReplicaSets](#)
16. [14 Managing Pods with Deployments](#)
17. [15 Deploying stateful workloads with StatefulSets](#)
18. [16 Deploying node agents and daemons with DaemonSets](#)
19. [17 Running finite workloads with Jobs and CronJobs](#)



MEAP Edition

Manning Early Access Program

Kubernetes in Action

Second edition

Version 15

**Copyright 2023 Manning
Publications**

©Manning Publications Co. We welcome reader comments about anything in the manuscript - other than typos and other simple mistakes.

These will be cleaned up during production of the book by copyeditors and proofreaders.

<https://livebook.manning.com/book/kubernetes-in-action-second-edition/discussion>

For more information on this and other Manning titles go to

manning.com

welcome

Thank you for purchasing the MEAP for *Kubernetes in Action, Second Edition*.

As part of my work at Red Hat, I started using Kubernetes in 2014, even before version 1.0 was released. Those were interesting times. Not many people working in the software industry knew about Kubernetes, and there was no real community yet. There were hardly any blog posts about it and the documentation was still very basic. Kubernetes itself was ridden with bugs. When you combine all these facts, you can imagine that working with Kubernetes was extremely difficult.

In 2015 I was asked by Manning to write the first edition of this book. The originally planned 300-page book grew to over 600 pages full of information. The writing forced me to also research those parts of Kubernetes that I wouldn't have looked at more closely otherwise. I put most of what I learned into the book. Judging by their reviews and comments, readers love a detailed book like this.

The plan for the second edition of the book is to add even more information and to rearrange some of the existing content. The exercises in this book will take you from deploying a trivial application that initially uses only the basic features of Kubernetes to a full-fledged application that incorporates additional features as the book introduces them.

The book is divided into five parts. In the first part, after the introduction of Kubernetes and containers, you'll deploy the application in the simplest way. In the second part you'll learn the main concepts used to describe and deploy your application. After that you'll explore the inner workings of Kubernetes components. This will give you a good foundation to learn the difficult part - how to manage Kubernetes in production. In the last part of the book you'll learn about best practices and how to extend Kubernetes.

I hope you all like this second edition even better than the first, and if you're

reading the book for the first time, your feedback will be even more valuable. If any part of the book is difficult to understand, please post your questions, comments or suggestions in the [liveBook forum](#).

Thank you for helping me write the best book possible.

—Marko Lukša

In this book

[Copyright 2023 Manning Publications](#) [welcome](#) [brief contents](#) [1 Introducing Kubernetes](#) [2 Understanding containers](#) [3 Deploying your first application](#) [4 Introducing Kubernetes API objects](#) [5 Running workloads in Pods](#) [6 Managing the Pod lifecycle](#) [7 Attaching storage volumes to Pods](#) [8 Persisting data in PersistentVolumes](#) [9 Configuration via ConfigMaps, Secrets, and the Downward API](#) [10 Organizing objects using Namespaces and Labels](#) [11 Exposing Pods with Services](#) [12 Exposing Services with Ingress](#) [13 Replicating Pods with ReplicaSets](#) [14 Managing Pods with Deployments](#) [15 Deploying stateful workloads with StatefulSets](#) [16 Deploying node agents and daemons with DaemonSets](#) [17 Running finite workloads with Jobs and CronJobs](#)

1 Introducing Kubernetes

This chapter covers

- Introductory information about Kubernetes and its origins
- Why Kubernetes has seen such wide adoption
- How Kubernetes transforms your data center
- An overview of its architecture and operation
- How and if you should integrate Kubernetes into your own organization

Before you can learn about the ins and outs of running applications with Kubernetes, you must first gain a basic understanding of the problems Kubernetes is designed to solve, how it came about, and its impact on application development and deployment. This first chapter is intended to give a general overview of these topics.

1.1 Introducing Kubernetes

The word *Kubernetes* is Greek for pilot or helmsman, the person who steers the ship - the person standing at the helm (the ship's wheel). A helmsman is not necessarily the same as a captain. A captain is responsible for the ship, while the helmsman is the one who steers it.

After learning more about what Kubernetes does, you'll find that the name hits the spot perfectly. A helmsman maintains the course of the ship, carries out the orders given by the captain and reports back the ship's heading. Kubernetes steers your applications and reports on their status while you - the captain - decide where you want the system to go.

How to pronounce Kubernetes and what is k8s?

The correct Greek pronunciation of Kubernetes, which is *Kie-ver-nee-tees*, is different from the English pronunciation you normally hear in technical conversations. Most often it's *Koo-ber-netties* or *Koo-ber-nay'-tace*, but you

may also hear *Koo-ber-nets*, although rarely.

In both written and oral conversations, it's also referred to as *Kube* or *K8s*, pronounced *Kates*, where the 8 signifies the number of letters omitted between the first and last letter.

1.1.1 Kubernetes in a nutshell

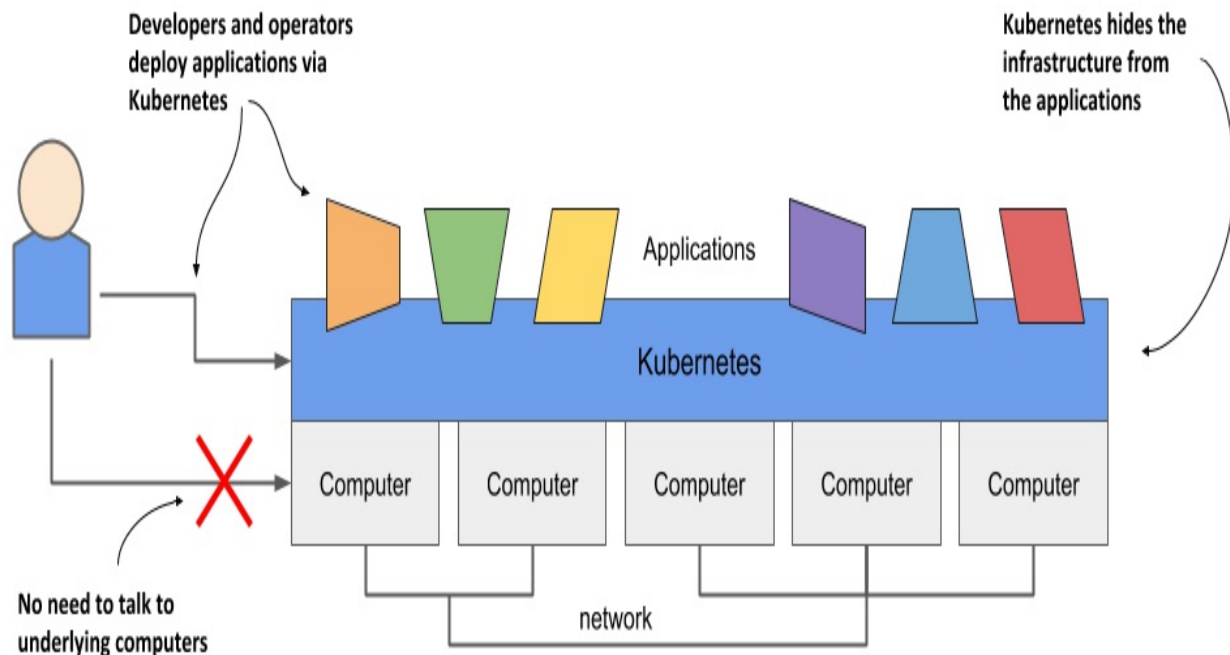
Kubernetes is a software system for automating the deployment and management of complex, large-scale application systems composed of computer processes running in containers. Let's learn what it does and how it does it.

Abstracting away the infrastructure

When software developers or operators decide to deploy an application, they do this through Kubernetes instead of deploying the application to individual computers. Kubernetes provides an abstraction layer over the underlying hardware to both users and applications.

As you can see in the following figure, the underlying infrastructure, meaning the computers, the network and other components, is hidden from the applications, making it easier to develop and configure them.

Figure 1.1 Infrastructure abstraction using Kubernetes



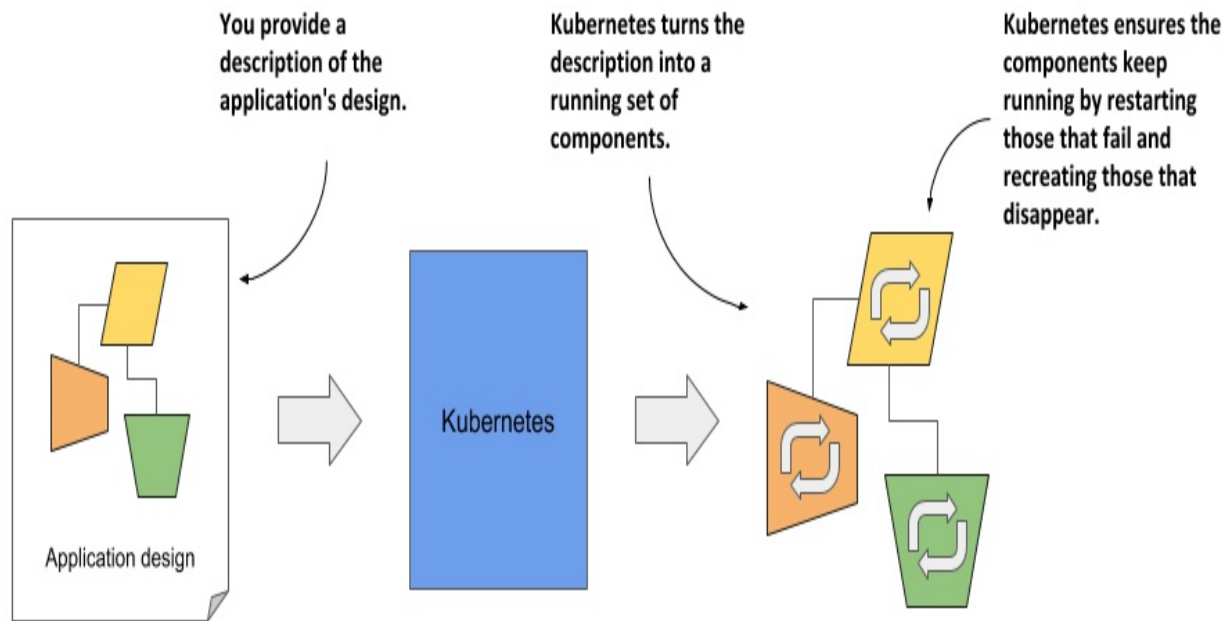
Standardizing how we deploy applications

Because the details of the underlying infrastructure no longer affect the deployment of applications, you deploy applications to your corporate data center in the same way as you do in the cloud. A single manifest that describes the application can be used for local deployment and for deploying on any cloud provider. All differences in the underlying infrastructure are handled by Kubernetes, so you can focus on the application and the business logic it contains.

Deploying applications declaratively

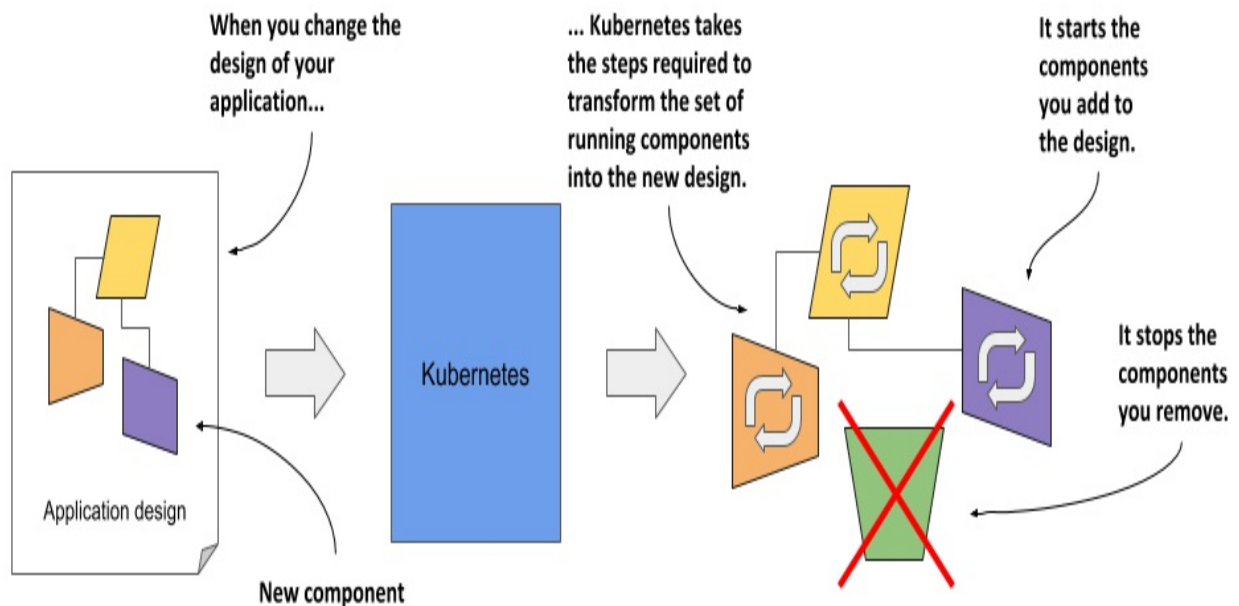
Kubernetes uses a declarative model to define an application, as shown in the next figure. You describe the components that make up your application and Kubernetes turns this description into a running application. It then keeps the application healthy by restarting or recreating parts of it as needed.

Figure 1.2 The declarative model of application deployment



Whenever you change the description, Kubernetes will take the necessary steps to reconfigure the running application to match the new description, as shown in the next figure.

Figure 1.3 Changes in the description are reflected in the running application

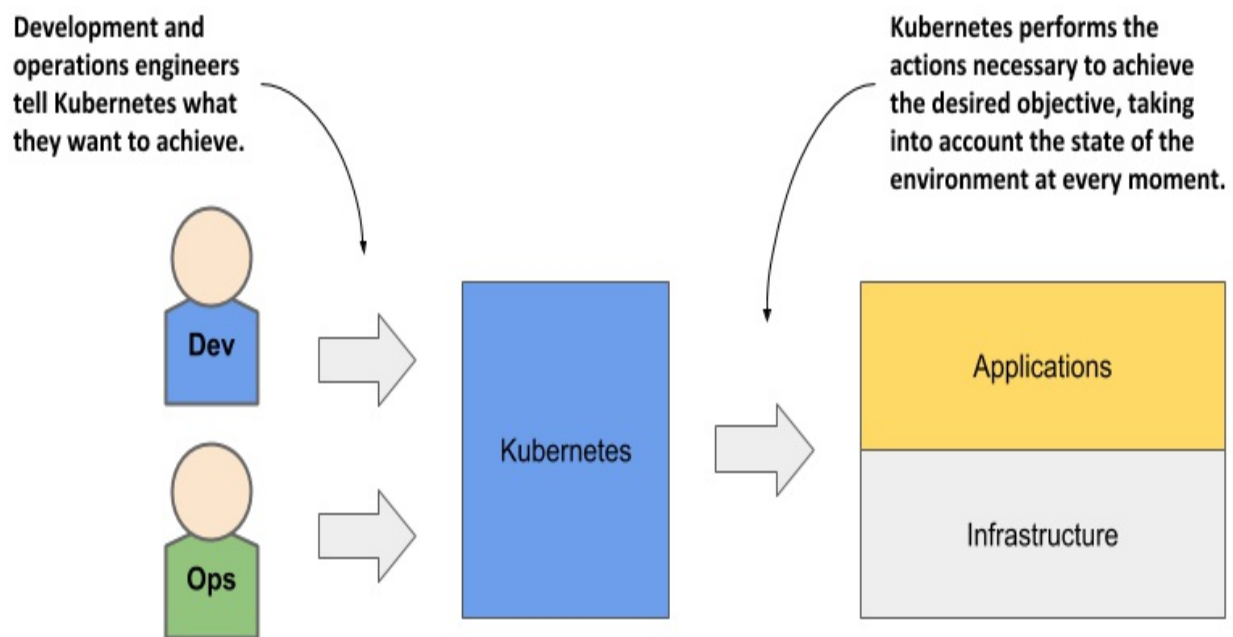


Taking on the daily management of applications

As soon as you deploy an application to Kubernetes, it takes over the daily management of the application. If the application fails, Kubernetes will automatically restart it. If the hardware fails or the infrastructure topology changes so that the application needs to be moved to other machines, Kubernetes does this all by itself. The engineers responsible for operating the system can focus on the big picture instead of wasting time on the details.

To circle back to the sailing analogy: the development and operations engineers are the ship's officers who make high-level decisions while sitting comfortably in their armchairs, and Kubernetes is the helmsman who takes care of the low-level tasks of steering the system through the rough waters your applications and infrastructure sail through.

Figure 1.4 Kubernetes takes over the management of applications



Everything that Kubernetes does and all the advantages it brings requires a longer explanation, which we'll discuss later. Before we do that, it might help you to know how it all began and where the Kubernetes project currently stands.

1.1.2 About the Kubernetes project

Kubernetes was originally developed by Google. Google has practically always run applications in containers. As early as 2014, it was reported that they start two billion containers every week. That's over 3,000 containers per second, and the figure is much higher today. They run these containers on thousands of computers distributed across dozens of data centers around the world. Now imagine doing all this manually. It's clear that you need automation, and at this massive scale, it better be perfect.

About Borg and Omega - the predecessors of Kubernetes

The sheer scale of Google's workload has forced them to develop solutions to make the development and management of thousands of software components manageable and cost-effective. Over the years, Google developed an internal system called *Borg* (and later a new system called *Omega*) that helped both application developers and operators manage these thousands of applications and services.

In addition to simplifying development and management, these systems have also helped them to achieve better utilization of their infrastructure. This is important in any organization, but when you operate hundreds of thousands of machines, even tiny improvements in utilization mean savings in the millions, so the incentives for developing such a system are clear.

Note

Data on Google's energy use suggests that they run around 900,000 servers.

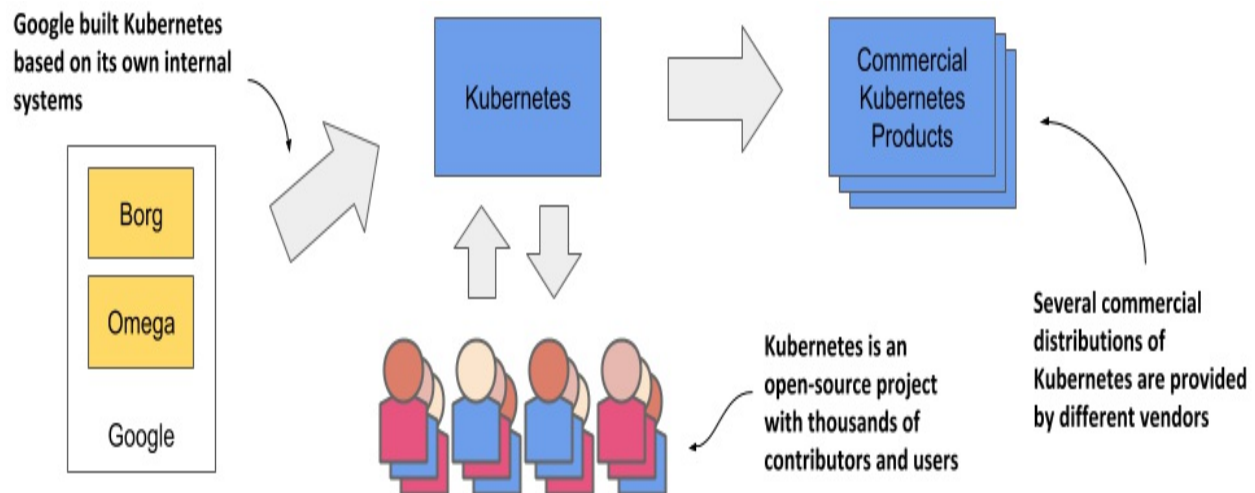
Over time, your infrastructure grows and evolves. Every new data center is state-of-the-art. Its infrastructure differs from those built in the past. Despite the differences, the deployment of applications in one data center should not differ from deployment in another data center. This is especially important when you deploy your application across multiple zones or regions to reduce the likelihood that a regional failure will cause application downtime. To do this effectively, it's worth having a consistent method for deploying your applications.

About Kubernetes - the open-source project - and commercial products

derived from it

Based on the experience they gained while developing Borg, Omega and other internal systems, in 2014 Google introduced Kubernetes, an open-source project that can now be used and further improved by everyone.

Figure 1.5 The origins and state of the Kubernetes open-source project



As soon as Kubernetes was announced, long before version 1.0 was officially released, other companies, such as Red Hat, who has always been at the forefront of open-source software, quickly stepped on board and helped develop the project. It eventually grew far beyond the expectations of its founders, and today is arguably one of the world's leading open-source projects, with dozens of organizations and thousands of individuals contributing to it.

Several companies are now offering enterprise-quality Kubernetes products that are built from the open-source project. These include Red Hat OpenShift, Pivotal Container Service, Rancher and many others.

How Kubernetes grew a whole new cloud-native eco-system

Kubernetes has also spawned many other related open-source projects, most of which are now under the umbrella of the *Cloud Native Computing*

Foundation (CNCF), which is part of the Linux Foundation.

CNCF organizes several KubeCon - CloudNativeCon conferences per year - in North America, Europe and China. In 2019, the total number of attendees exceeded 23,000, with KubeCon North America reaching an overwhelming number of 12,000 participants. These figures show that Kubernetes has had an incredibly positive impact on the way companies around the world deploy applications today. It wouldn't have been so widely adopted if that wasn't the case.

1.1.3 Understanding why Kubernetes is so popular

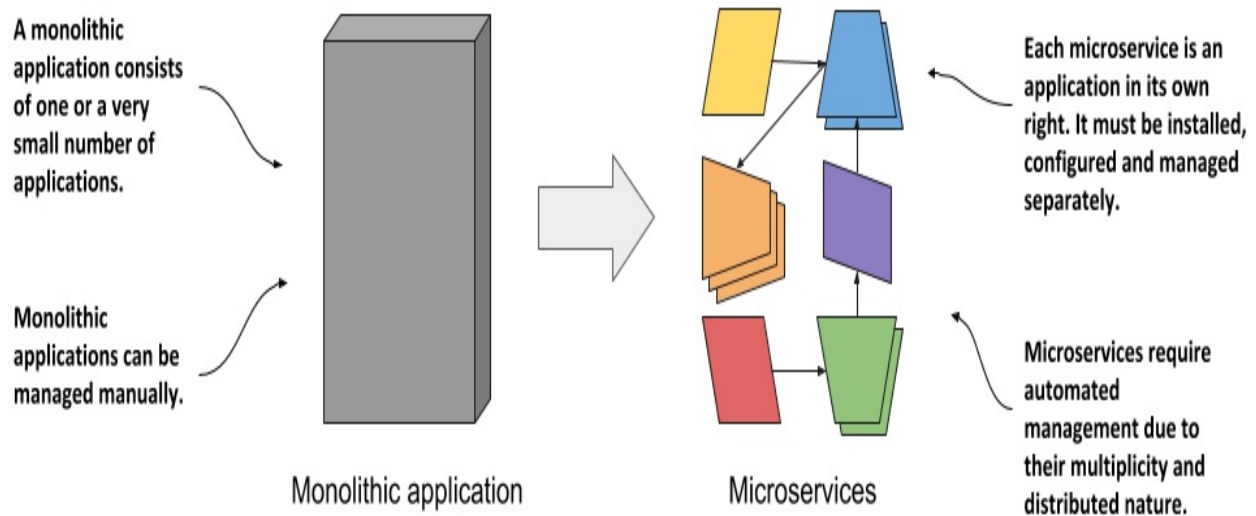
In recent years, the way we develop applications has changed considerably. This has led to the development of new tools like Kubernetes, which in turn have fed back and fuelled further changes in application architecture and the way we develop them. Let's look at concrete examples of this.

Automating the management of microservices

In the past, most applications were large monoliths. The components of the application were tightly coupled, and they all ran in a single computer process. The application was developed as a unit by a large team of developers and the deployment of the application was straightforward. You installed it on a powerful computer and provided the little configuration it required. Scaling the application horizontally was rarely possible, so whenever you needed to increase the capacity of the application, you had to upgrade the hardware - in other words, scale the application vertically.

Then came the microservices paradigm. The monoliths were divided into dozens, sometimes hundreds, of separate processes, as shown in the following figure. This allowed organizations to divide their development departments into smaller teams where each team developed only a part of the entire system - just some of the microservices.

Figure 1.6 Comparing monolithic applications with microservices



Each microservice is now a separate application with its own development and release cycle. The dependencies of different microservices will inevitably diverge over time. One microservice requires one version of a library, while another microservice requires another, possibly incompatible, version of the same library. Running the two applications in the same operating system becomes difficult.

Fortunately, containers alone solve this problem where each microservice requires a different environment, but each microservice is now a separate application that must be managed individually. The increased number of applications makes this much more difficult.

Individual parts of the entire application no longer need to run on the same computer, which makes it easier to scale the entire system, but also means that the applications need to be configured to communicate with each other. For systems with only a handful of components, this can usually be done manually, but it's now common to see deployments with well over a hundred microservices.

When the system consists of many microservices, automated management is crucial. Kubernetes provides this automation. The features it offers make the task of managing hundreds of microservices almost trivial.

Bridging the dev and ops divide

Along with these changes in application architecture, we've also seen changes in the way teams develop and run software. It used to be normal for a development team to build the software in isolation and then throw the finished product over the wall to the operations team, who would then deploy it and manage it from there.

With the advent of the Dev-ops paradigm, the two teams now work much more closely together throughout the entire life of the software product. The development team is now much more involved in the daily management of the deployed software. But that means that they now need to know about the infrastructure on which it's running.

As a software developer, your primary focus is on implementing the business logic. You don't want to deal with the details of the underlying servers. Fortunately, Kubernetes hides these details.

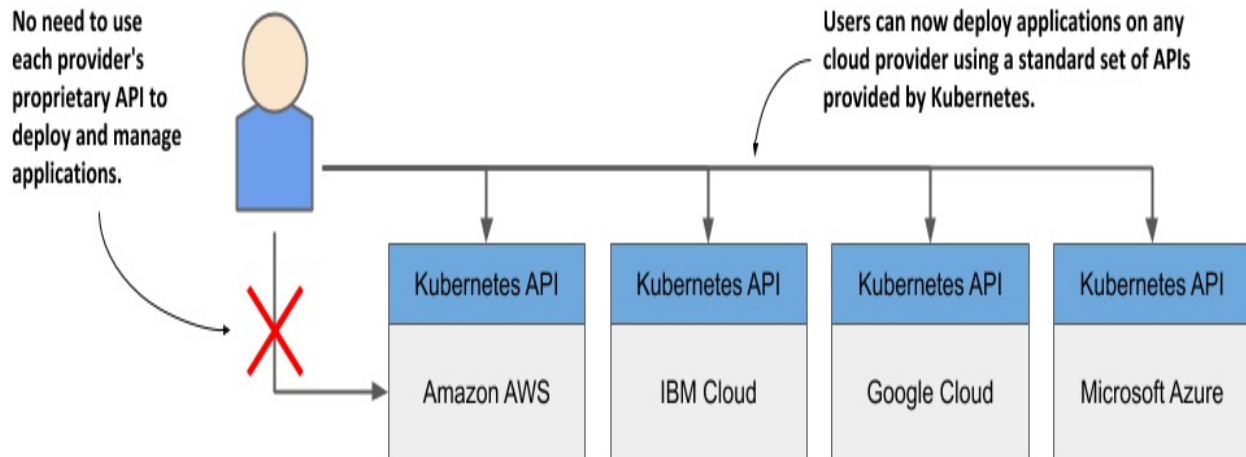
Standardizing the cloud

Over the past decade or two, many organizations have moved their software from local servers to the cloud. The benefits of this seem to have outweighed the fear of being locked-in to a particular cloud provider, which is caused by relying on the provider's proprietary APIs to deploy and manage applications.

Any company that wants to be able to move its applications from one provider to another will have to make additional, initially unnecessary efforts to abstract the infrastructure and APIs of the underlying cloud provider from the applications. This requires resources that could otherwise be focused on building the primary business logic.

Kubernetes has also helped in this respect. The popularity of Kubernetes has forced all major cloud providers to integrate Kubernetes into their offerings. Customers can now deploy applications to any cloud provider through a standard set of APIs provided by Kubernetes.

Figure 1.7 Kubernetes has standardized how you deploy applications on cloud providers



If the application is built on the APIs of Kubernetes instead of directly on the proprietary APIs of a specific cloud provider, it can be transferred relatively easily to any other provider.

1.2 Understanding Kubernetes

The previous section explained the origins of Kubernetes and the reasons for its wide adoption. In this section we'll take a closer look at what exactly Kubernetes is.

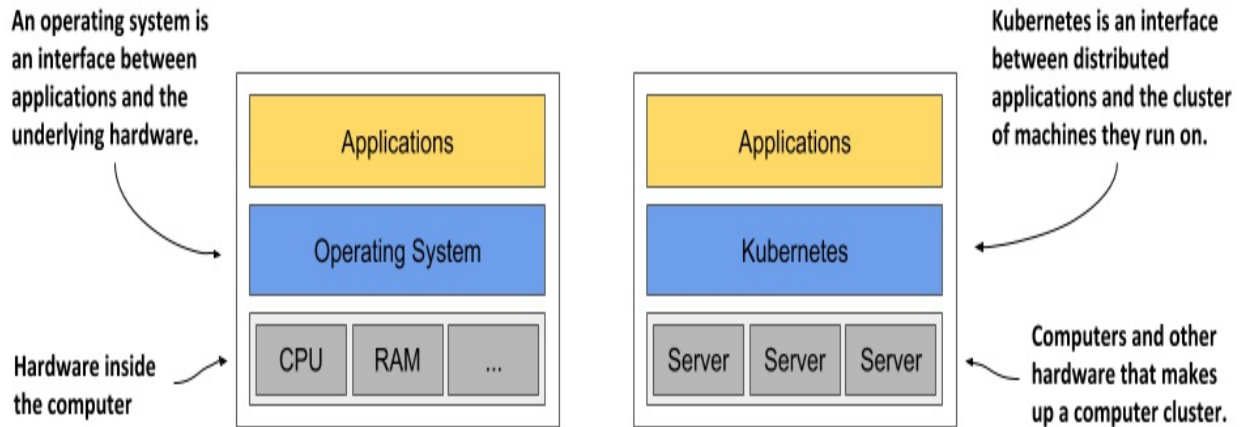
1.2.1 Understanding how Kubernetes transforms a computer cluster

Let's take a closer look at how the perception of the data center changes when you deploy Kubernetes on your servers.

Kubernetes is like an operating system for computer clusters

One can imagine Kubernetes as an operating system for the cluster. The next figure illustrates the analogies between an operating system running on a computer and Kubernetes running on a cluster of computers.

Figure 1.8 Kubernetes is to a computer cluster what an Operating System is to a computer



Just as an operating system supports the basic functions of a computer, such as scheduling processes onto its CPUs and acting as an interface between the application and the computer's hardware, Kubernetes schedules the components of a distributed application onto individual computers in the underlying computer cluster and acts as an interface between the application and the cluster.

It frees application developers from the need to implement infrastructure-related mechanisms in their applications; instead, they rely on Kubernetes to provide them. This includes things like:

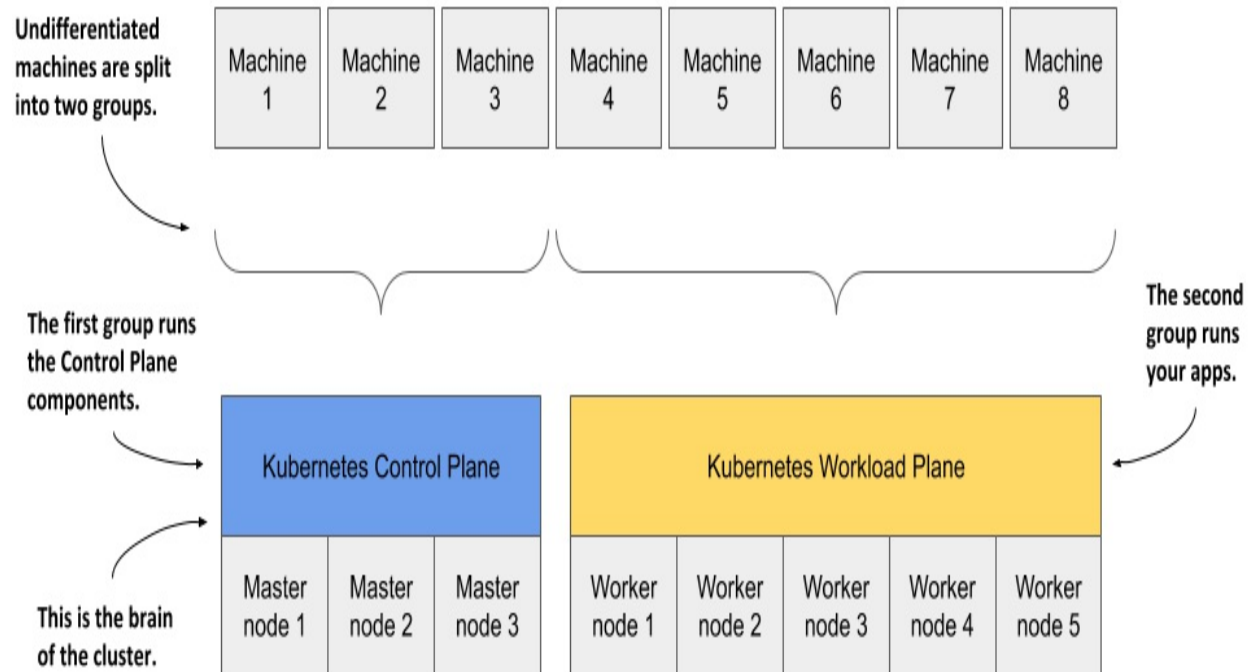
- *service discovery* - a mechanism that allows applications to find other applications and use the services they provide,
- *horizontal scaling* - replicating your application to adjust to fluctuations in load,
- *load-balancing* - distributing load across all the application replicas,
- *self-healing* - keeping the system healthy by automatically restarting failed applications and moving them to healthy nodes after their nodes fail,
- *leader election* - a mechanism that decides which instance of the application should be active while the others remain idle but ready to take over if the active instance fails.

By relying on Kubernetes to provide these features, application developers can focus on implementing the core business logic instead of wasting time integrating applications with the infrastructure.

How Kubernetes fits into a computer cluster

To get a concrete example of how Kubernetes is deployed onto a cluster of computers, look at the following figure.

Figure 1.9 Computers in a Kubernetes cluster are divided into the Control Plane and the Workload Plane



You start with a fleet of machines that you divide into two groups - the master and the worker nodes. The master nodes will run the Kubernetes Control Plane, which represents the brain of your system and controls the cluster, while the rest will run your applications - your workloads - and will therefore represent the Workload Plane.

Note

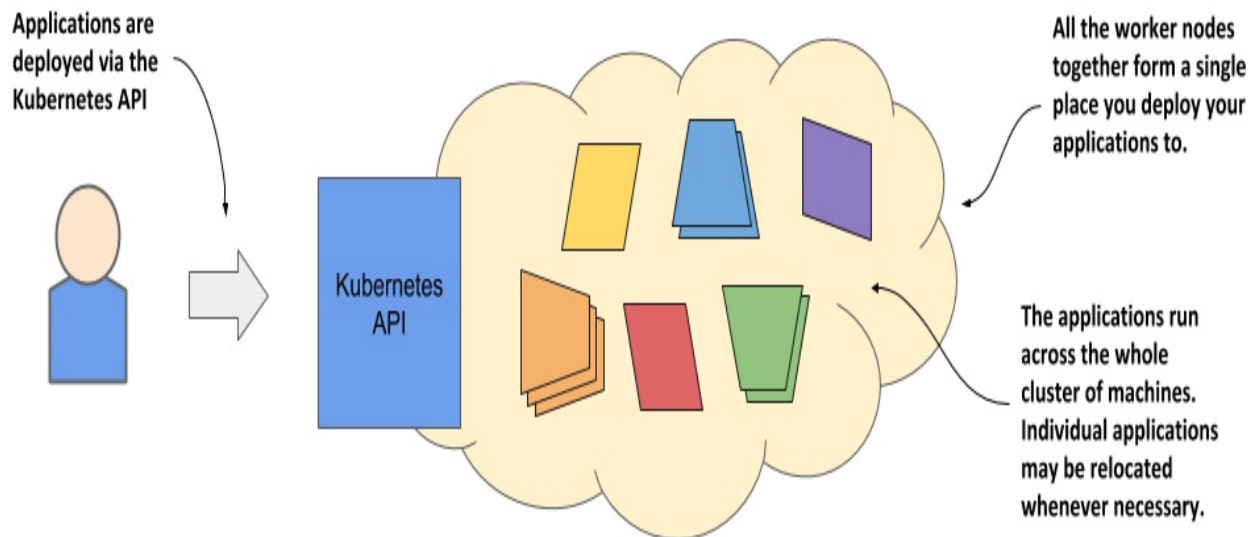
The Workload Plane is sometimes referred to as the Data Plane, but this term could be confusing because the plane doesn't host data but applications. Don't be confused by the term "plane" either - in this context you can think of it as the "surface" the applications run on.

Non-production clusters can use a single master node, but highly available clusters use at least three physical master nodes to host the Control Plane. The number of worker nodes depends on the number of applications you'll deploy.

How all cluster nodes become one large deployment area

After Kubernetes is installed on the computers, you no longer need to think about individual computers when deploying applications. Regardless of the number of worker nodes in your cluster, they all become a single space where you deploy your applications. You do this using the Kubernetes API, which is provided by the Kubernetes Control Plane.

Figure 1.10 Kubernetes exposes the cluster as a uniform deployment area



When I say that all worker nodes become one space, I don't want you to think that you can deploy an extremely large application that is spread across several small machines. Kubernetes doesn't do magic tricks like this. Each application must be small enough to fit on one of the worker nodes.

What I meant was that when deploying applications, it doesn't matter which worker node they end up on. Kubernetes may later even move the application from one node to another. You may not even notice when that happens, and you shouldn't care.

1.2.2 The benefits of using Kubernetes

You've already learned why many organizations across the world have welcomed Kubernetes into their data centers. Now, let's take a closer look at the specific benefits it brings to both development and IT operations teams.

Self-service deployment of applications

Because Kubernetes presents all its worker nodes as a single deployment surface, it no longer matters which node you deploy your application to. This means that developers can now deploy applications on their own, even if they don't know anything about the number of nodes or the characteristics of each node.

In the past, the system administrators were the ones who decided where each application should be placed. This task is now left to Kubernetes. This allows a developer to deploy applications without having to rely on other people to do so. When a developer deploys an application, Kubernetes chooses the best node on which to run the application based on the resource requirements of the application and the resources available on each node.

Reducing costs via better infrastructure utilization

If you don't care which node your application lands on, it also means that it can be moved to any other node at any time without you having to worry about it. Kubernetes may need to do this to make room for a larger application that someone wants to deploy. This ability to move applications allows the applications to be packed tightly together so that the resources of the nodes can be utilized in the best possible way.

Note

In chapter 17 you'll learn more about how Kubernetes decides where to place each application and how you can influence the decision.

Finding optimal combinations can be challenging and time consuming, especially when the number of all possible options is huge, such as when you

have many application components and many server nodes on which they can be deployed. Computers can perform this task much better and faster than humans. Kubernetes does it very well. By combining different applications on the same machines, Kubernetes improves the utilization of your hardware infrastructure so you can run more applications on fewer servers.

Automatically adjusting to changing load

Using Kubernetes to manage your deployed applications also means that the operations team doesn't have to constantly monitor the load of each application to respond to sudden load peaks. Kubernetes takes care of this also. It can monitor the resources consumed by each application and other metrics and adjust the number of running instances of each application to cope with increased load or resource usage.

When you run Kubernetes on cloud infrastructure, it can even increase the size of your cluster by provisioning additional nodes through the cloud provider's API. This way, you never run out of space to run additional instances of your applications.

Keeping applications running smoothly

Kubernetes also makes every effort to ensure that your applications run smoothly. If your application crashes, Kubernetes will restart it automatically. So even if you have a broken application that runs out of memory after running for more than a few hours, Kubernetes will ensure that your application continues to provide the service to its users by automatically restarting it in this case.

Kubernetes is a self-healing system in that it deals with software errors like the one just described, but it also handles hardware failures. As clusters grow in size, the frequency of node failure also increases. For example, in a cluster with one hundred nodes and a MTBF (mean-time-between-failure) of 100 days for each node, you can expect one node to fail every day.

When a node fails, Kubernetes automatically moves applications to the remaining healthy nodes. The operations team no longer needs to manually

move the application and can instead focus on repairing the node itself and returning it to the pool of available hardware resources.

If your infrastructure has enough free resources to allow normal system operation without the failed node, the operations team doesn't even have to react immediately to the failure. If it occurs in the middle of the night, no one from the operations team even has to wake up. They can sleep peacefully and deal with the failed node during regular working hours.

Simplifying application development

The improvements described in the previous section mainly concern application deployment. But what about the process of application development? Does Kubernetes bring anything to their table? It definitely does.

As mentioned previously, Kubernetes offers infrastructure-related services that would otherwise have to be implemented in your applications. This includes the discovery of services and/or peers in a distributed application, leader election, centralized application configuration and others. Kubernetes provides this while keeping the application Kubernetes-agnostic, but when required, applications can also query the Kubernetes API to obtain detailed information about their environment. They can also use the API to change the environment.

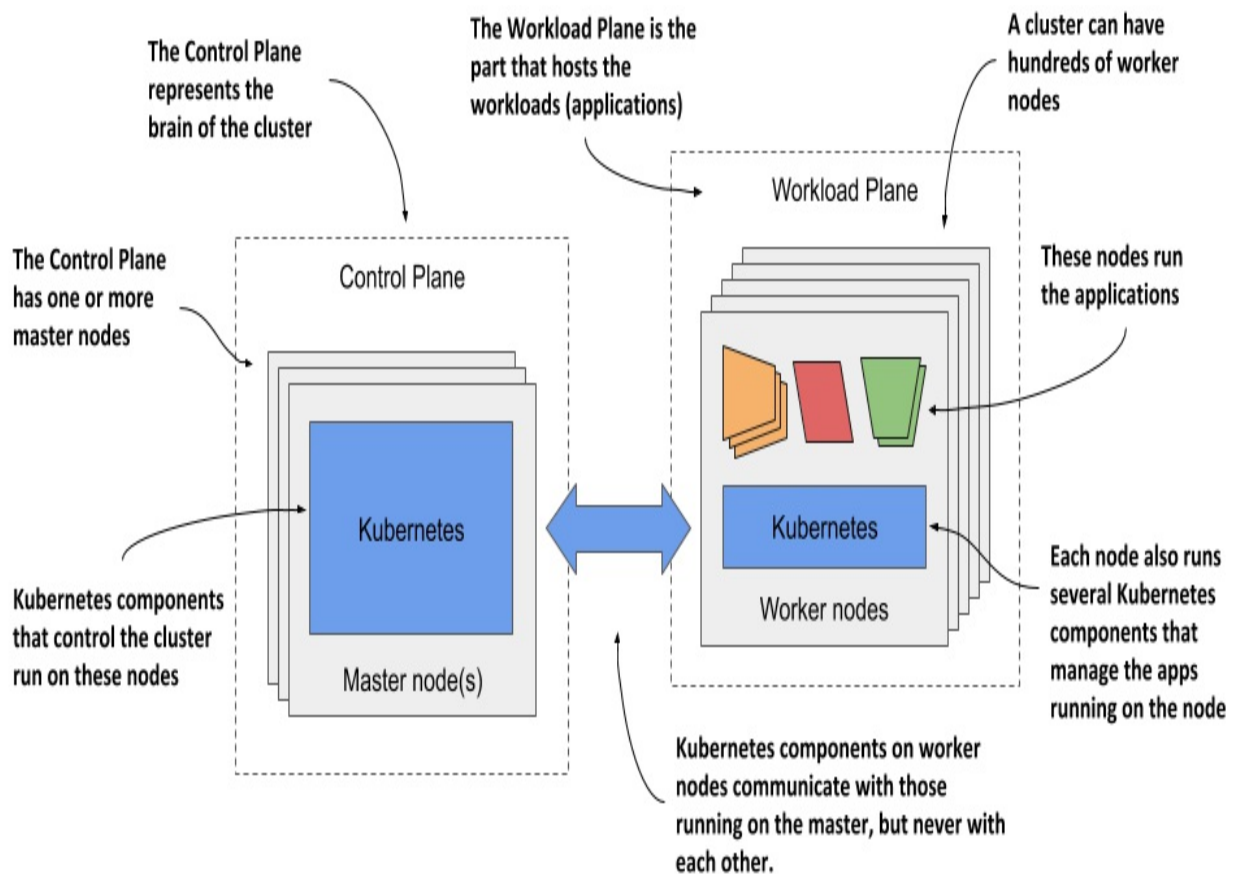
1.2.3 The architecture of a Kubernetes cluster

As you've already learned, a Kubernetes cluster consists of nodes divided into two groups:

- A set of *master nodes* that host the *Control Plane* components, which are the brains of the system, since they control the entire cluster.
- A set of *worker nodes* that form the *Workload Plane*, which is where your workloads (or applications) run.

The following figure shows the two planes and the different nodes they consist of.

Figure 1.11 The two planes that make up a Kubernetes cluster

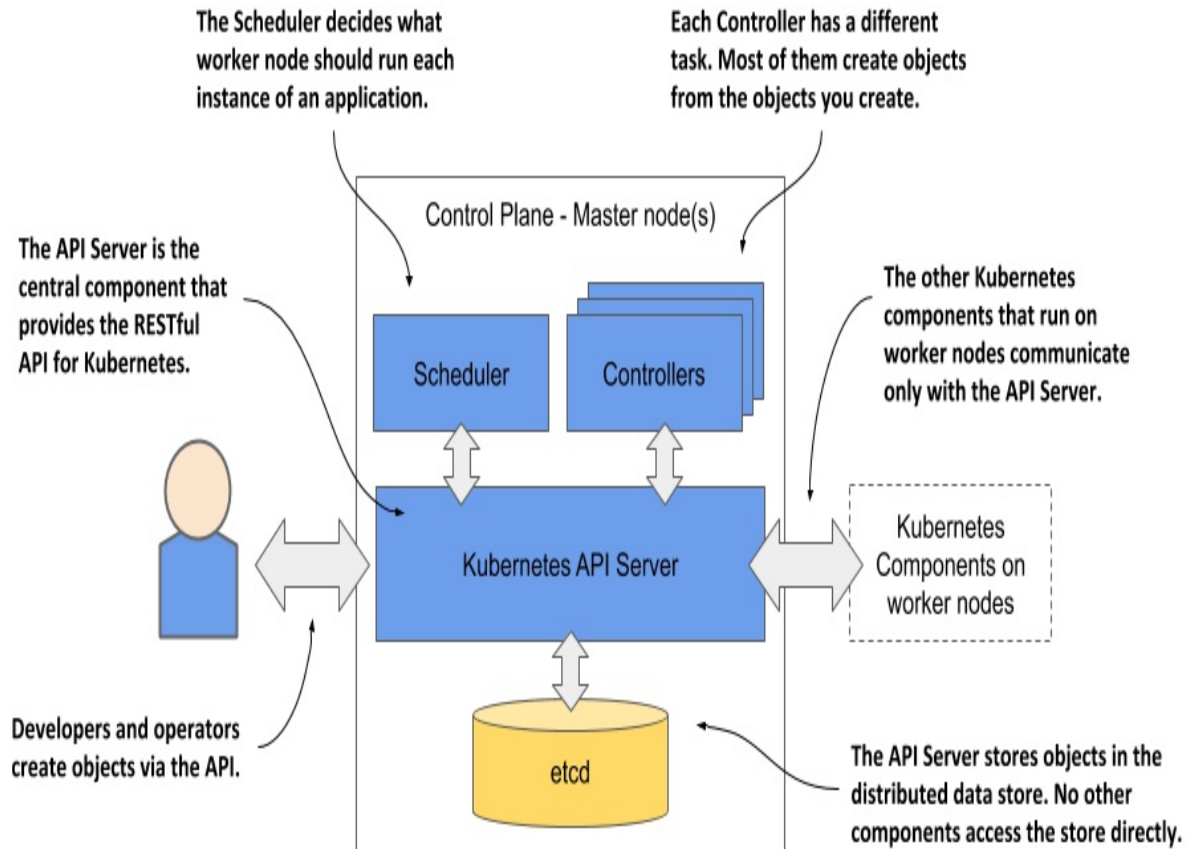


The two planes, and hence the two types of nodes, run different Kubernetes components. The next two sections of the book introduce them and summarize their functions without going into details. These components will be mentioned several times in the next part of the book where I explain the fundamental concepts of Kubernetes. An in-depth look at the components and their internals follows in the third part of the book.

Control Plane components

The Control Plane is what controls the cluster. It consists of several components that run on a single master node or are replicated across multiple master nodes to ensure high availability. The Control Plane's components are shown in the following figure.

Figure 1.12 The components of the Kubernetes Control Plane



These are the components and their functions:

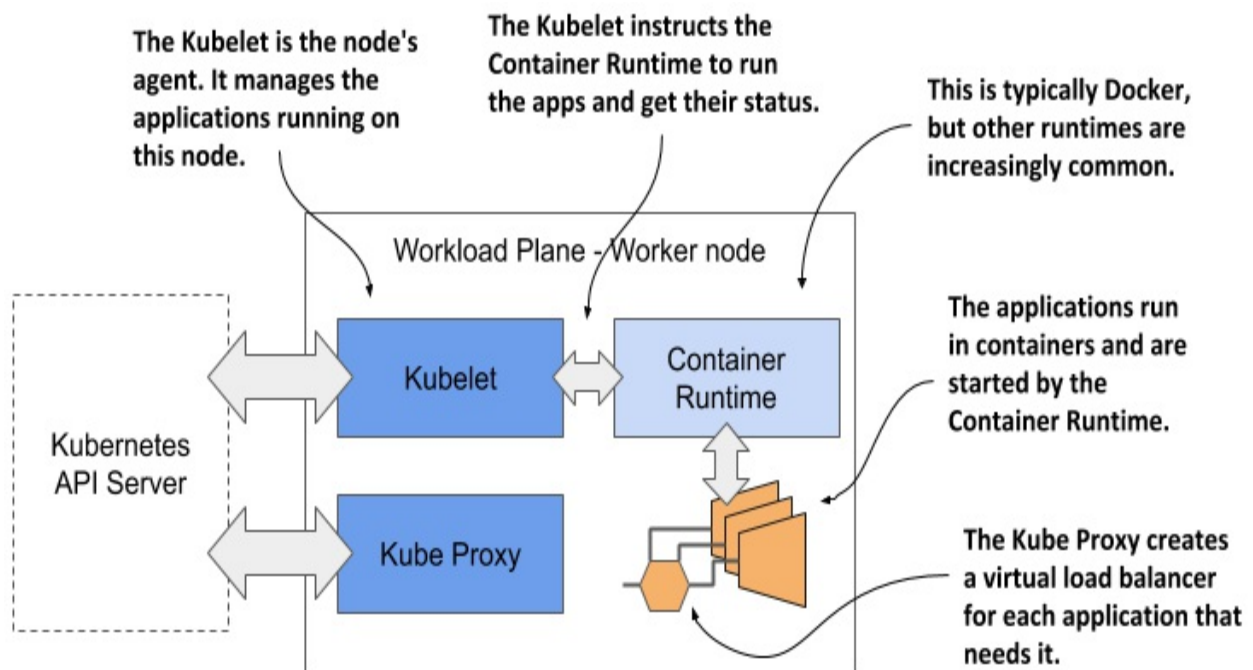
- The *Kubernetes API Server* exposes the RESTful Kubernetes API. Engineers using the cluster and other Kubernetes components create objects via this API.
- The *etcd* distributed datastore persists the objects you create through the API, since the API Server itself is stateless. The Server is the only component that talks to etcd.
- The *Scheduler* decides on which worker node each application instance should run.
- *Controllers* bring to life the objects you create through the API. Most of them simply create other objects, but some also communicate with external systems (for example, the cloud provider via its API).

The components of the Control Plane hold and control the state of the cluster, but they don't run your applications. This is done by the (worker) nodes.

Worker node components

The worker nodes are the computers on which your applications run. They form the cluster's Workload Plane. In addition to applications, several Kubernetes components also run on these nodes. They perform the task of running, monitoring and providing connectivity between your applications. They are shown in the following figure.

Figure 1.13 The Kubernetes components that run on each node



Each node runs the following set of components:

- The *Kubelet*, an agent that talks to the API server and manages the applications running on its node. It reports the status of these applications and the node via the API.
- The *Container Runtime*, which can be Docker or any other runtime compatible with Kubernetes. It runs your applications in containers as instructed by the Kubelet.
- The *Kubernetes Service Proxy (Kube Proxy)* load-balances network traffic between applications. Its name suggests that traffic flows through it, but that's no longer the case. You'll learn why in chapter 14.

Add-on components

Most Kubernetes clusters also contain several other components. This includes a DNS server, network plugins, logging agents and many others. They typically run on the worker nodes but can also be configured to run on the master.

Gaining a deeper understanding of the architecture

For now, I only expect you to be vaguely familiar with the names of these components and their function, as I'll mention them many times throughout the following chapters. You'll learn snippets about them in these chapters, but I'll explain them in more detail in chapter 14.

I'm not a fan of explaining how things work until I first explain *what* something does and teach you how to use it. It's like learning to drive. You don't want to know what's under the hood. At first, you just want to learn how to get from point A to B. Only then will you be interested in how the car makes this possible. Knowing what's under the hood may one day help you get your car moving again after it has broken down and you are stranded on the side of the road. I hate to say it, but you'll have many moments like this when dealing with Kubernetes due to its sheer complexity.

1.2.4 How Kubernetes runs an application

With a general overview of the components that make up Kubernetes, I can finally explain how to deploy an application in Kubernetes.

Defining your application

Everything in Kubernetes is represented by an object. You create and retrieve these objects via the Kubernetes API. Your application consists of several types of these objects - one type represents the application deployment as a whole, another represents a running instance of your application, another represents the service provided by a set of these instances and allows reaching them at a single IP address, and there are many others.

All these types are explained in detail in the second part of the book. At the moment, it's enough to know that you define your application through several types of objects. These objects are usually defined in one or more manifest files in either YAML or JSON format.

Definition

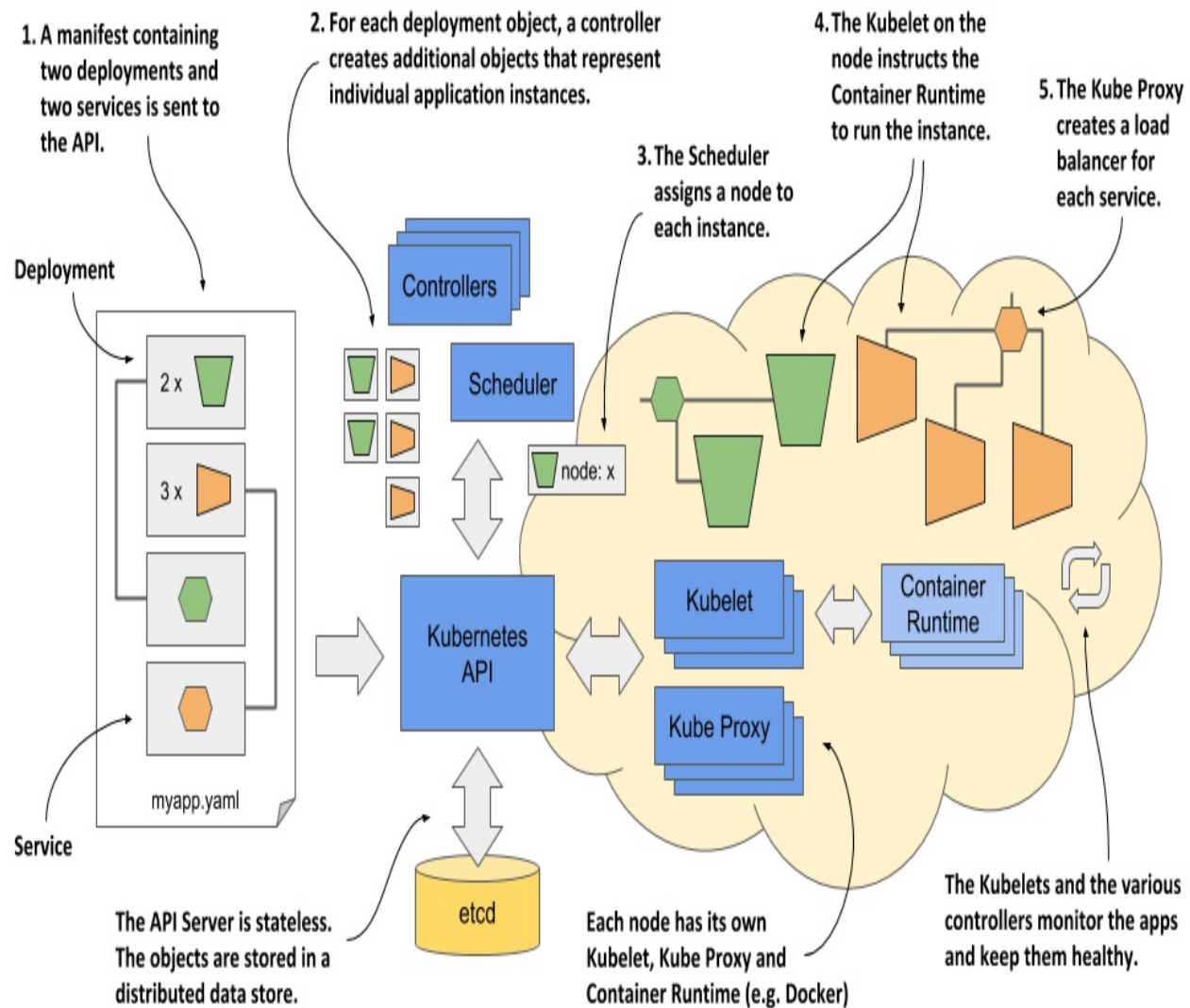
YAML was initially said to mean “Yet Another Markup Language”, but it was later changed to the recursive acronym “YAML Ain't Markup Language”. It's one of the ways to serialize an object into a human-readable text file.

Definition

JSON is short for JavaScript Object Notation. It's a different way of serializing an object, but more suitable for exchanging data between applications.

The following figure shows an example of deploying an application by creating a manifest with two deployments exposed using two services.

Figure 1.14 Deploying an application to Kubernetes



These actions take place when you deploy the application:

1. You submit the application manifest to the Kubernetes API. The API Server writes the objects defined in the manifest to etcd.
2. A controller notices the newly created objects and creates several new objects - one for each application instance.
3. The Scheduler assigns a node to each instance.
4. The Kubelet notices that an instance is assigned to the Kubelet's node. It runs the application instance via the Container Runtime.
5. The Kube Proxy notices that the application instances are ready to accept connections from clients and configures a load balancer for them.
6. The Kubelets and the Controllers monitor the system and keep the applications running.

The procedure is explained in more detail in the following sections, but the complete explanation is given in chapter 14, after you have familiarized yourself with all the objects and controllers involved.

Submitting the application to the API

After you've created your YAML or JSON file(s), you submit the file to the API, usually via the Kubernetes command-line tool called *kubectl*.

Note

Kubectl is pronounced *kube-control*, but the softer souls in the community prefer to call it *kube-cuddle*. Some refer to it as *kube-C-T-L*.

Kubectl splits the file into individual objects and creates each of them by sending an HTTP PUT or POST request to the API, as is usually the case with RESTful APIs. The API Server validates the objects and stores them in the etcd datastore. In addition, it notifies all interested components that these objects have been created. Controllers, which are explained next, are one of these components.

About the controllers

Most object types have an associated controller. A controller is interested in a particular object type. It waits for the API server to notify it that a new object has been created, and then performs operations to bring that object to life. Typically, the controller just creates other objects via the same Kubernetes API. For example, the controller responsible for application deployments creates one or more objects that represent individual instances of the application. The number of objects created by the controller depends on the number of replicas specified in the application deployment object.

About the Scheduler

The scheduler is a special type of controller, whose only task is to schedule application instances onto worker nodes. It selects the best worker node for

each new application instance object and assigns it to the instance - by modifying the object via the API.

About the Kubelet and the Container Runtime

The Kubelet that runs on each worker node is also a type of controller. Its task is to wait for application instances to be assigned to the node on which it is located and run the application. This is done by instructing the Container Runtime to start the application's container.

About the Kube Proxy

Because an application deployment can consist of multiple application instances, a load balancer is required to expose them at a single IP address. The Kube Proxy, another controller running alongside the Kubelet, is responsible for setting up the load balancer.

Keeping the applications healthy

Once the application is up and running, the Kubelet keeps the application healthy by restarting it when it terminates. It also reports the status of the application by updating the object that represents the application instance. The other controllers monitor these objects and ensure that applications are moved to healthy nodes if their nodes fail.

You're now roughly familiar with the architecture and functionality of Kubernetes. You don't need to understand or remember all the details at this moment, because internalizing this information will be easier when you learn about each individual object types and the controllers that bring them to life in the second part of the book.

1.3 Introducing Kubernetes into your organization

To close this chapter, let's see what options are available to you if you decide to introduce Kubernetes in your own IT environment.

1.3.1 Running Kubernetes on-premises and in the cloud

If you want to run your applications on Kubernetes, you have to decide whether you want to run them locally, in your organization's own infrastructure (on-premises) or with one of the major cloud providers, or perhaps both - in a hybrid cloud solution.

Running Kubernetes on-premises

Running Kubernetes on your own infrastructure may be your only option if regulations require you to run applications on site. This usually means that you'll have to manage Kubernetes yourself, but we'll come to that later.

Kubernetes can run directly on your bare-metal machines or in virtual machines running in your data center. In either case, you won't be able to scale your cluster as easily as when you run it in virtual machines provided by a cloud provider.

Deploying Kubernetes in the cloud

If you have no on-premises infrastructure, you have no choice but to run Kubernetes in the cloud. This has the advantage that you can scale your cluster at any time at short notice if required. As mentioned earlier, Kubernetes itself can ask the cloud provider to provision additional virtual machines when the current size of the cluster is no longer sufficient to run all the applications you want to deploy.

When the number of workloads decreases and some worker nodes are left without running workloads, Kubernetes can ask the cloud provider to destroy the virtual machines of these nodes to reduce your operational costs. This elasticity of the cluster is certainly one of the main benefits of running Kubernetes in the cloud.

Using a hybrid cloud solution

A more complex option is to run Kubernetes on-premises, but also allow it to

spill over into the cloud. It's possible to configure Kubernetes to provision additional nodes in the cloud if you exceed the capacity of your own data center. This way, you get the best of both worlds. Most of the time, your applications run locally without the cost of virtual machine rental, but in short periods of peak load that may occur only a few times a year, your applications can handle the extra load by using the additional resources in the cloud.

If your use-case requires it, you can also run a Kubernetes cluster across multiple cloud providers or a combination of any of the options mentioned. This can be done using a single control plane or one control plane in each location.

1.3.2 To manage or not to manage Kubernetes yourself

If you are considering introducing Kubernetes in your organization, the most important question you need to answer is whether you'll manage Kubernetes yourself or use a Kubernetes-as-a-Service type offering where someone else manages it for you.

Managing Kubernetes yourself

If you already run applications on-premises and have enough hardware to run a production-ready Kubernetes cluster, your first instinct is probably to deploy and manage it yourself. If you ask anyone in the Kubernetes community if this is a good idea, you'll usually get a very definite "no".

Figure 1.14 was a very simplified representation of what happens in a Kubernetes cluster when you deploy an application. Even that figure should have scared you. Kubernetes brings with it an enormous amount of additional complexity. Anyone who wants to run a Kubernetes cluster must be intimately familiar with its inner workings.

The management of production-ready Kubernetes clusters is a multi-billion-dollar industry. Before you decide to manage one yourself, it's essential that you consult with engineers who have already done it to learn about the issues most teams run into. If you don't, you may be setting yourself up for failure.

On the other hand, trying out Kubernetes for non-production use-cases or using a managed Kubernetes cluster is much less problematic.

Using a managed Kubernetes cluster in the cloud

Using Kubernetes is ten times easier than managing it. Most major cloud providers now offer Kubernetes-as-a-Service. They take care of managing Kubernetes and its components while you simply use the Kubernetes API like any of the other APIs the cloud provider offers.

The top managed Kubernetes offerings include the following:

- Google Kubernetes Engine (GKE)
- Azure Kubernetes Service (AKS)
- Amazon Elastic Kubernetes Service (EKS)
- IBM Cloud Kubernetes Service
- Red Hat OpenShift Online and Dedicated
- VMware Cloud PKS
- Alibaba Cloud Container Service for Kubernetes (ACK)

The first half of this book focuses on just using Kubernetes. You'll run the exercises in a local development cluster and on a managed GKE cluster, as I find it's the easiest to use and offers the best user experience. The second part of the book gives you a solid foundation for managing Kubernetes, but to truly master it, you'll need to gain additional experience.

1.3.3 Using vanilla or extended Kubernetes

The final question is whether to use a vanilla open-source version of Kubernetes or an extended, enterprise-quality Kubernetes product.

Using a vanilla version of Kubernetes

The open-source version of Kubernetes is maintained by the community and represents the cutting edge of Kubernetes development. This also means that it may not be as stable as the other options. It may also lack good security defaults. Deploying the vanilla version requires a lot of fine tuning to set

everything up for production use.

Using enterprise-grade Kubernetes distributions

A better option for using Kubernetes in production is to use an enterprise-quality Kubernetes distribution such as OpenShift or Rancher. In addition to the increased security and performance provided by better defaults, they offer additional object types in addition to those provided in the upstream Kubernetes API. For example, vanilla Kubernetes does not contain object types that represent cluster users, whereas commercial distributions do. They also provide additional software tools for deploying and managing well-known third-party applications on Kubernetes.

Of course, extending and hardening Kubernetes takes time, so these commercial Kubernetes distributions usually lag one or two versions behind the upstream version of Kubernetes. It's not as bad as it sounds. The benefits usually outweigh the disadvantages.

1.3.4 Should you even use Kubernetes?

I hope this chapter has made you excited about Kubernetes and you can't wait to squeeze it into your IT stack. But to close this chapter properly, we need to say a word or two about when introducing Kubernetes is not a good idea.

Do your workloads require automated management?

The first thing you need to be honest about is whether you need to automate the management of your applications at all. If your application is a large monolith, you definitely don't need Kubernetes.

Even if you deploy microservices, using Kubernetes may not be the best option, especially if the number of your microservices is very small. It's difficult to provide an exact number when the scales tip over, since other factors also influence the decision. But if your system consists of less than five microservices, throwing Kubernetes into the mix is probably not a good idea. If your system has more than twenty microservices, you will most likely

benefit from the integration of Kubernetes. If the number of your microservices falls somewhere in between, other factors, such as the ones described next, should be considered.

Can you afford to invest your engineers' time into learning Kubernetes?

Kubernetes is designed to allow applications to run without them knowing that they are running in Kubernetes. While the applications themselves don't need to be modified to run in Kubernetes, development engineers will inevitably spend a lot of time learning how to use Kubernetes, even though the operators are the only ones that actually need that knowledge.

It would be hard to tell your teams that you're switching to Kubernetes and expect only the operations team to start exploring it. Developers like shiny new things. At the time of writing, Kubernetes is still a very shiny thing.

Are you prepared for increased costs in the interim?

While Kubernetes reduces long-term operational costs, introducing Kubernetes in your organization initially involves increased costs for training, hiring new engineers, building and purchasing new tools and possibly additional hardware. Kubernetes requires additional computing resources in addition to the resources that the applications use.

Don't believe the hype

Although Kubernetes has been around for several years at the time of writing this book, I can't say that the hype phase is over. The initial excitement has just begun to calm down, but many engineers may still be unable to make rational decisions about whether the integration of Kubernetes is as necessary as it seems.

1.4 Summary

In this introductory chapter, you've learned that:

- Kubernetes is Greek for helmsman. As a ship's captain oversees the ship while the helmsman steers it, you oversee your computer cluster, while Kubernetes performs the day-to-day management tasks.
- Kubernetes is pronounced *koo-ber-netties*. Kubectl, the Kubernetes command-line tool, is pronounced *kube-control*.
- Kubernetes is an open-source project built upon Google's vast experience in running applications on a global scale. Thousands of individuals now contribute to it.
- Kubernetes uses a declarative model to describe application deployments. After you provide a description of your application to Kubernetes, it brings it to life.
- Kubernetes is like an operating system for the cluster. It abstracts the infrastructure and presents all computers in a data center as one large, contiguous deployment area.
- Microservice-based applications are more difficult to manage than monolithic applications. The more microservices you have, the more you need to automate their management with a system like Kubernetes.
- Kubernetes helps both development and operations teams to do what they do best. It frees them from mundane tasks and introduces a standard way of deploying applications both on-premises and in any cloud.
- Using Kubernetes allows developers to deploy applications without the help of system administrators. It reduces operational costs through better utilization of existing hardware, automatically adjusts your system to load fluctuations, and heals itself and the applications running on it.
- A Kubernetes cluster consists of master and worker nodes. The master nodes run the *Control Plane*, which controls the entire cluster, while the worker nodes run the deployed applications or workloads, and therefore represent the *Workload Plane*.
- Using Kubernetes is simple, but managing it is hard. An inexperienced team should use a Kubernetes-as-a-Service offering instead of deploying Kubernetes by itself.

So far, you've only observed the ship from the pier. It's time to come aboard. But before you leave the docks, you should inspect the shipping containers it's carrying. You'll do this next.

2 Understanding containers

This chapter covers

- Understanding what a container is
- Differences between containers and virtual machines
- Creating, running, and sharing a container image with Docker
- Linux kernel features that make containers possible

Kubernetes primarily manages applications that run in containers - so before you start exploring Kubernetes, you need to have a good understanding of what a container is. This chapter explains the basics of Linux containers that a typical Kubernetes user needs to know.

2.1 Introducing containers

In Chapter 1 you learned how different microservices running in the same operating system may require different, potentially conflicting versions of dynamically linked libraries or have different environment requirements.

When a system consists of a small number of applications, it's okay to assign a dedicated virtual machine to each application and run each in its own operating system. But as the microservices become smaller and their numbers start to grow, you may not be able to afford to give each one its own VM if you want to keep your hardware costs low and not waste resources.

It's not just a matter of wasting hardware resources - each VM typically needs to be individually configured and managed, which means that running higher numbers of VMs also results in higher staffing requirements and the need for a better, often more complicated automation system. Due to the shift to microservice architectures, where systems consist of hundreds of deployed application instances, an alternative to VMs was needed. Containers are that alternative.

2.1.1 Comparing containers to virtual machines

Instead of using virtual machines to isolate the environments of individual microservices (or software processes in general), most development and operations teams now prefer to use containers. They allow you to run multiple services on the same host computer, while keeping them isolated from each other. Like VMs, but with much less overhead.

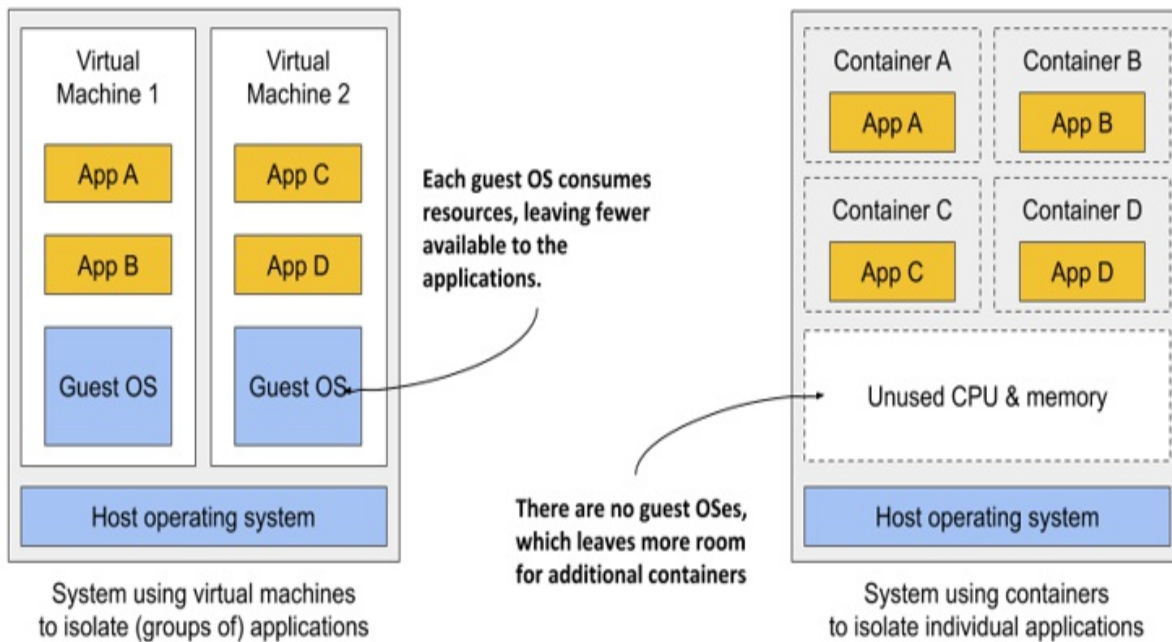
Unlike VMs, which each run a separate operating system with several system processes, a process running in a container runs within the existing host operating system. Because there is only one operating system, no duplicate system processes exist. Although all the application processes run in the same operating system, their environments are isolated, though not as well as when you run them in separate VMs. To the process in the container, this isolation makes it look like no other processes exist on the computer. You'll learn how this is possible in the next few sections, but first let's dive deeper into the differences between containers and virtual machines.

Comparing the overhead of containers and virtual machines

Compared to VMs, containers are much lighter, because they don't require a separate resource pool or any additional OS-level processes. While each VM usually runs its own set of system processes, which requires additional computing resources in addition to those consumed by the user application's own process, a container is nothing more than an isolated process running in the existing host OS that consumes only the resources the app consumes. They have virtually no overhead.

Figure 2.1 shows two bare metal computers, one running two virtual machines, and the other running containers instead. The latter has space for additional containers, as it runs only one operating system, while the first runs three – one host and two guest OSes.

Figure 2.1 Using VMs to isolate groups of applications vs. isolating individual apps with containers



Because of the resource overhead of VMs, you often group multiple applications into each VM. You may not be able to afford dedicating a whole VM to each app. But containers introduce no overhead, which means you can afford to create a separate container for each application. In fact, you should never run multiple applications in the same container, as this makes managing the processes in the container much more difficult. Moreover, all existing software dealing with containers, including Kubernetes itself, is designed under the premise that there's only one application in a container. But as you'll learn in the next chapter, Kubernetes provides a way to run related applications together, yet still keep them in separate containers.

Comparing the start-up time of containers and virtual machines

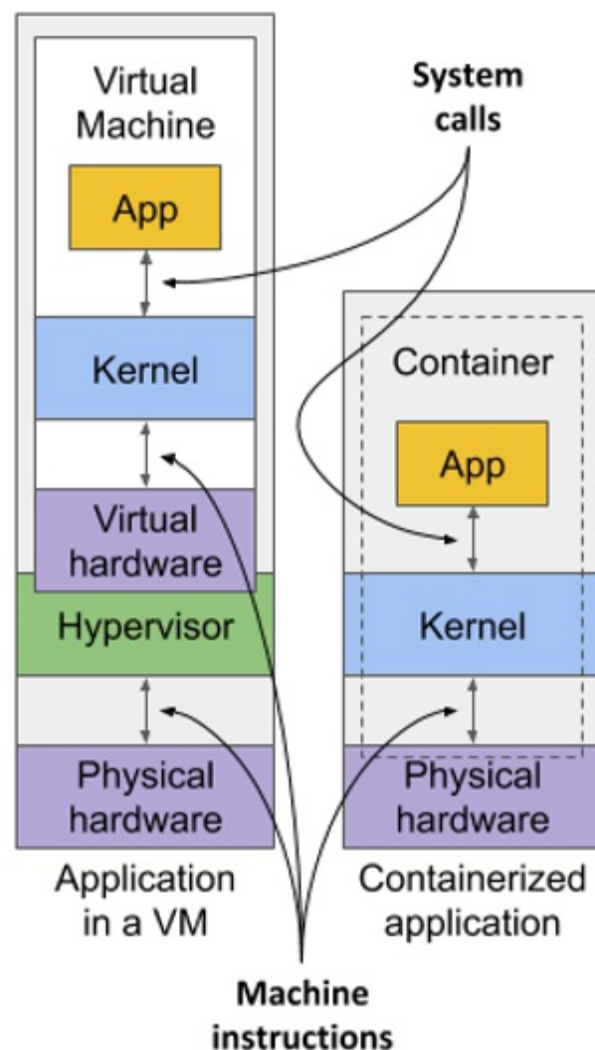
In addition to the lower runtime overhead, containers also start the application faster, because only the application process itself needs to be started. No additional system processes need to be started first, as is the case when booting up a new virtual machine.

Comparing the isolation of containers and virtual machines

You'll agree that containers are clearly better when it comes to the use of

resources, but there's also a disadvantage. When you run applications in virtual machines, each VM runs its own operating system and kernel. Underneath those VMs is the hypervisor (and possibly an additional operating system), which splits the physical hardware resources into smaller sets of virtual resources that the operating system in each VM can use. As figure 2.2 shows, applications running in these VMs make system calls (sys-calls) to the guest OS kernel in the VM, and the machine instructions that the kernel then executes on the virtual CPUs are then forwarded to the host's physical CPU via the hypervisor.

Figure 2.2 How apps use the hardware when running in a VM vs. in a container



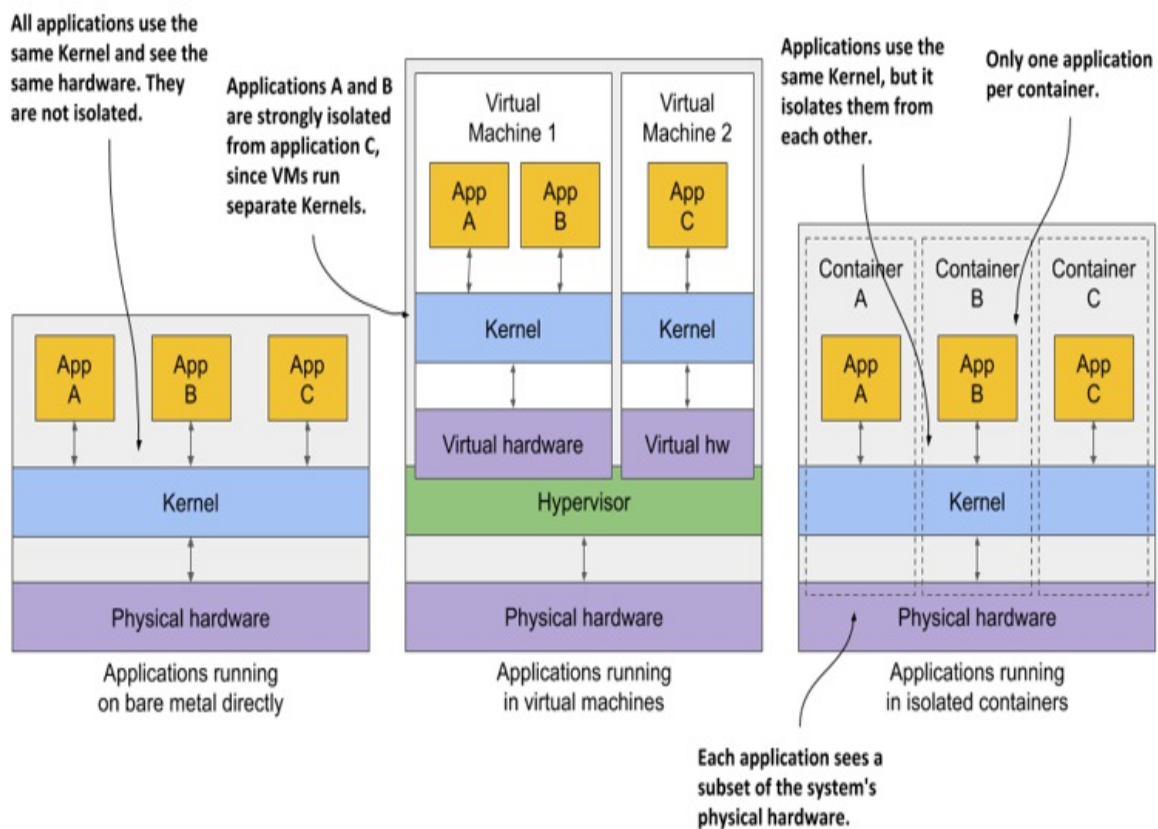
Note

Two types of hypervisors exist. Type 1 hypervisors don't require running a host OS, while type 2 hypervisors do.

Containers, on the other hand, all make system calls on the single kernel running in the host OS. This single kernel is the only one that executes instructions on the host's CPU. The CPU doesn't need to handle any kind of virtualization the way it does with VMs.

Examine the following figure to see the difference between running three applications on bare metal, running them in two separate virtual machines, or running them in three containers.

Figure 2.3 The difference between running applications on bare metal, in virtual machines, and in containers



In the first case, all three applications use the same kernel and aren't isolated

at all. In the second case, applications A and B run in the same VM and thus share the kernel, while application C is completely isolated from the other two, since it uses its own kernel. It only shares the hardware with the first two.

The third case shows the same three applications running in containers. Although they all use the same kernel, they are isolated from each other and completely unaware of the others' existence. The isolation is provided by the kernel itself. Each application sees only a part of the physical hardware and sees itself as the only process running in the OS, although they all run in the same OS.

Understanding the security-implications of container isolation

The main advantage of using virtual machines over containers is the complete isolation they provide, since each VM has its own Linux kernel, while containers all use the same kernel. This can clearly pose a security risk. If there's a bug in the kernel, an application in one container might use it to read the memory of applications in other containers. If the apps run in different VMs and therefore share only the hardware, the probability of such attacks is much lower. Of course, complete isolation is only achieved by running applications on separate physical machines.

Additionally, containers share memory space, whereas each VM uses its own chunk of memory. Therefore, if you don't limit the amount of memory that a container can use, this could cause other containers to run out of memory or cause their data to be swapped out to disk.

Note

This can't happen in Kubernetes, because it requires that swap is disabled on all the nodes.

Understanding what enables containers and what enables virtual machines

While virtual machines are enabled through virtualization support in the CPU

and by virtualization software on the host, containers are enabled by the Linux kernel itself. You'll learn about container technologies later when you can try them out for yourself. You'll need to have Docker installed for that, so let's learn how it fits into the container story.

2.1.2 Introducing the Docker container platform

While container technologies have existed for a long time, they only became widely known with the rise of Docker. Docker was the first container system that made them easily portable across different computers. It simplified the process of packaging up the application and all its libraries and other dependencies - even the entire OS file system - into a simple, portable package that can be used to deploy the application on any computer running Docker.

Introducing containers, images and registries

Docker is a platform for packaging, distributing and running applications. As mentioned earlier, it allows you to package your application along with its entire environment. This can be just a few dynamically linked libraries required by the app, or all the files that are usually shipped with an operating system. Docker allows you to distribute this package via a public repository to any other Docker-enabled computer.

Figure 2.4 The three main Docker concepts are images, registries and containers

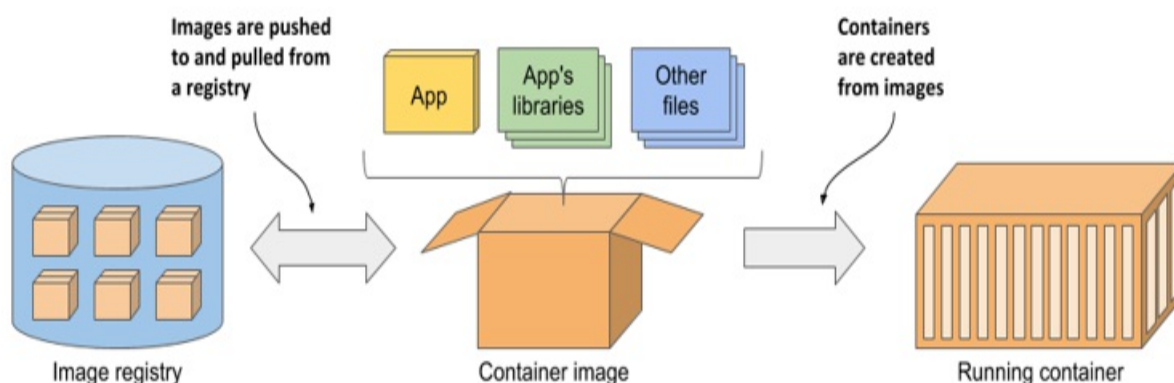


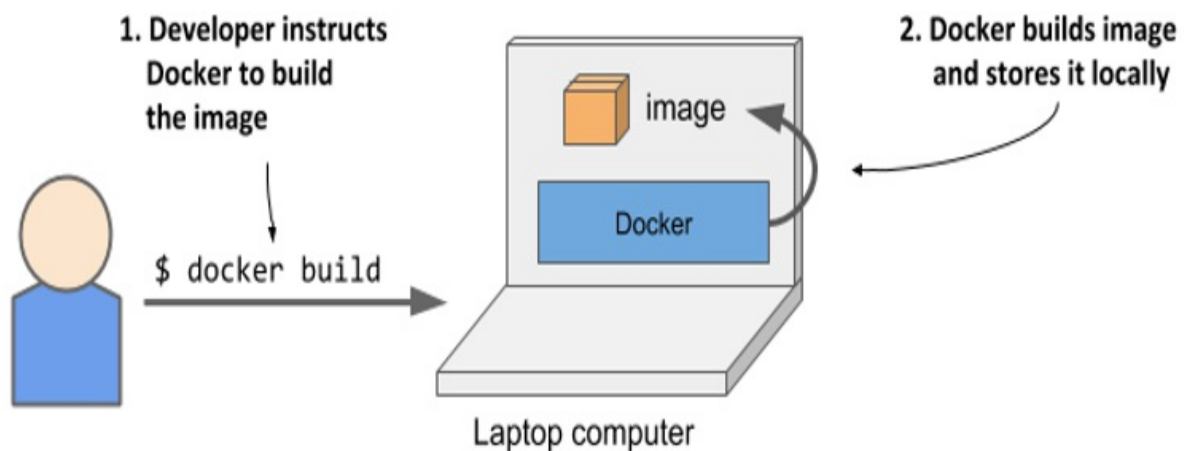
Figure 2.4 shows three main Docker concepts that appear in the process I've just described. Here's what each of them is:

- *Images*—A container image is something you package your application and its environment into. Like a zip file or a tarball. It contains the whole filesystem that the application will use and additional metadata, such as the path to the executable file to run when the image is executed, the ports the application listens on, and other information about the image.
- *Registries*—A registry is a repository of container images that enables the exchange of images between different people and computers. After you build your image, you can either run it on the same computer, or *push* (upload) the image to a registry and then *pull* (download) it to another computer. Certain registries are public, allowing anyone to pull images from it, while others are private and only accessible to individuals, organizations or computers that have the required authentication credentials.
- *Containers*—A container is instantiated from a container image. A running container is a normal process running in the host operating system, but its environment is isolated from that of the host and the environments of other processes. The file system of the container originates from the container image, but additional file systems can also be mounted into the container. A container is usually resource-restricted, meaning it can only access and use the amount of resources such as CPU and memory that have been allocated to it.

Building, distributing, and running a container image

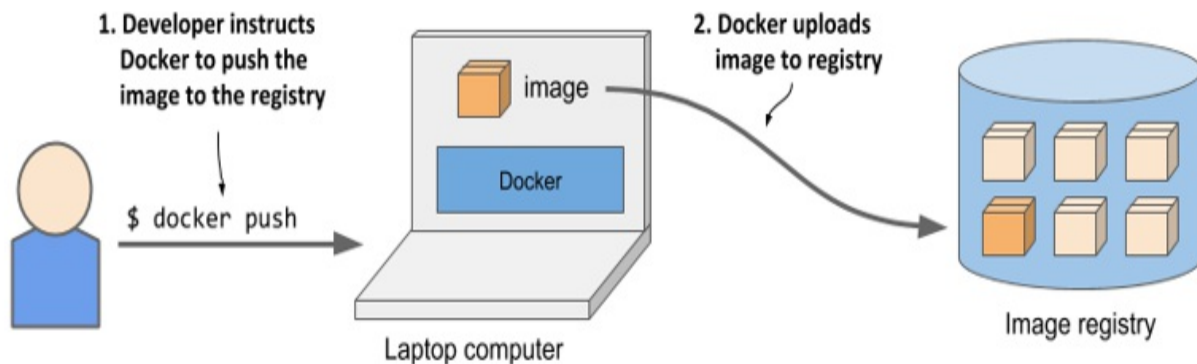
To understand how containers, images and registries relate to each other, let's look at how to build a container image, distribute it through a registry and create a running container from the image. These three processes are shown in figures 2.5 to 2.7.

Figure 2.5 Building a container image



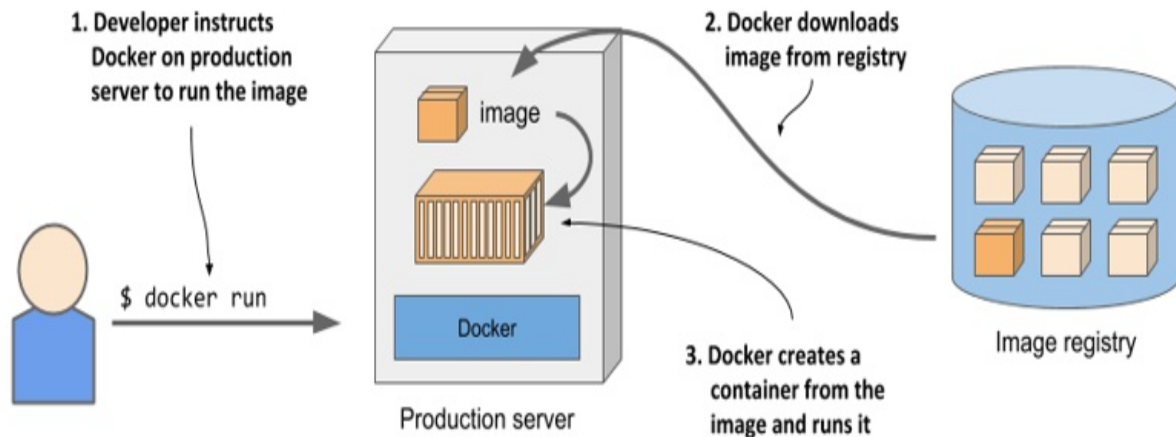
As shown in figure 2.5, the developer first builds an image, and then pushes it to a registry, as shown in figure 2.6. The image is now available to anyone who can access the registry.

Figure 2.6 Uploading a container image to a registry



As the next figure shows, another person can now pull the image to any other computer running Docker and run it. Docker creates an isolated container based on the image and invokes the executable file specified in the image.

Figure 2.7 Running a container on a different computer



Running the application on any computer is made possible by the fact that the environment of the application is decoupled from the environment of the host.

Understanding the environment that the application sees

When you run an application in a container, it sees exactly the file system content you bundled into the container image, as well as any additional file systems you mount into the container. The application sees the same files whether it's running on your laptop or a full-fledged production server, even if the production server uses a completely different Linux distribution. The application typically has no access to the files in the host's operating system, so it doesn't matter if the server has a completely different set of installed libraries than your development computer.

For example, if you package your application with the files of the entire Red Hat Enterprise Linux (RHEL) operating system and then run it, the application will think it's running inside RHEL, whether you run it on your Fedora-based or a Debian-based computer. The Linux distribution installed on the host is irrelevant. The only thing that might be important is the kernel version and the kernel modules it loads. Later, I'll explain why.

This is similar to creating a VM image by creating a new VM, installing an operating system and your app in it, and then distributing the whole VM image so that other people can run it on different hosts. Docker achieves the same effect, but instead of using VMs for app isolation, it uses Linux

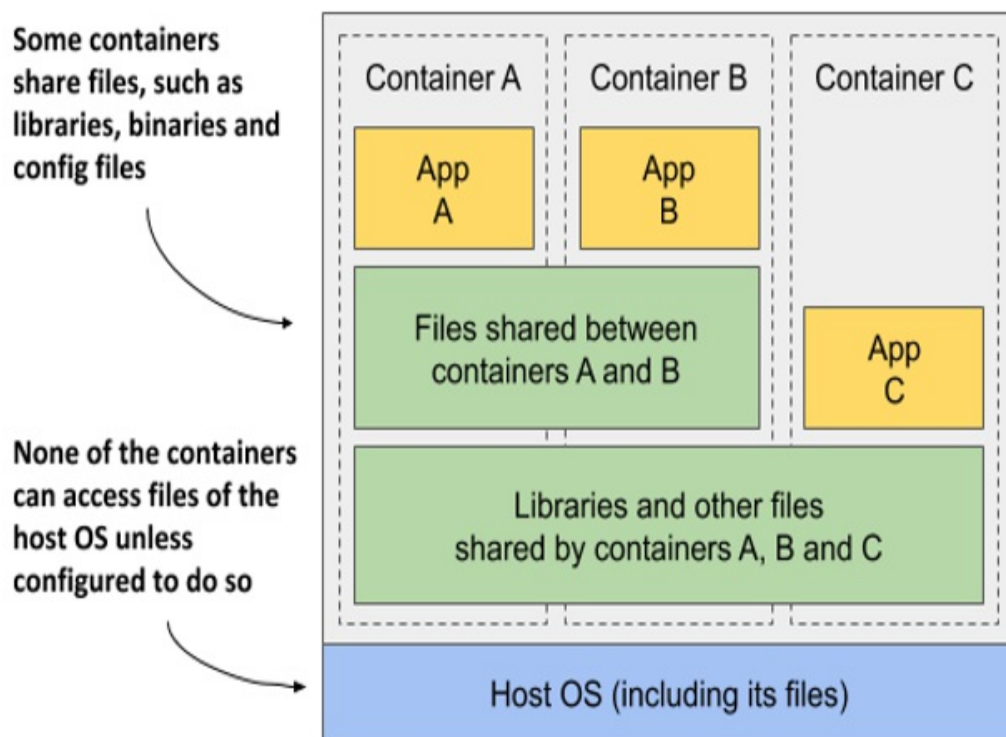
container technologies to achieve (almost) the same level of isolation.

Understanding image layers

Unlike virtual machine images, which are big blobs of the entire filesystem required by the operating system installed in the VM, container images consist of layers that are usually much smaller. These layers can be shared and reused across multiple images. This means that only certain layers of an image need to be downloaded if the rest were already downloaded to the host as part of another image containing the same layers.

Layers make image distribution very efficient but also help to reduce the storage footprint of images. Docker stores each layer only once. As you can see in the following figure, two containers created from two images that contain the same layers use the same files.

Figure 2.8 Containers can share image layers



The figure shows that containers A and B share an image layer, which means that applications A and B read some of the same files. In addition, they also share the underlying layer with container C. But if all three containers have access to the same files, how can they be completely isolated from each other? Are changes that application A makes to a file stored in the shared layer not visible to application B? They aren't. Here's why.

The filesystems are isolated by the Copy-on-Write (CoW) mechanism. The filesystem of a container consists of read-only layers from the container image and an additional read/write layer stacked on top. When an application running in container A changes a file in one of the read-only layers, the entire file is copied into the container's read/write layer and the file contents are changed there. Since each container has its own writable layer, changes to shared files are not visible in any other container.

When you delete a file, it is only marked as deleted in the read/write layer, but it's still present in one or more of the layers below. What follows is that deleting files never reduces the size of the image.

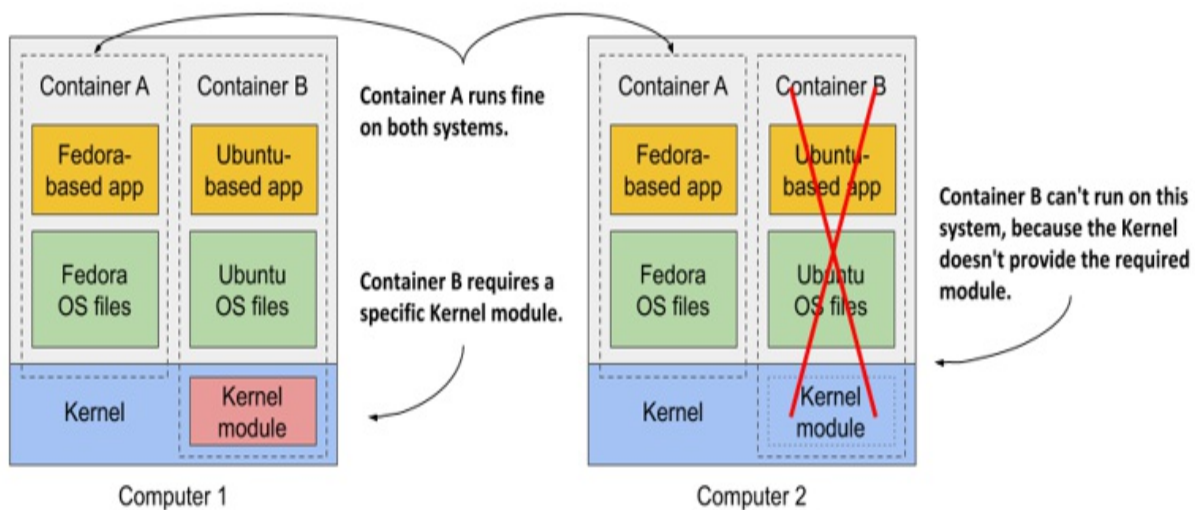
WARNING

Even seemingly harmless operations such as changing permissions or ownership of a file result in a new copy of the entire file being created in the read/write layer. If you perform this type of operation on a large file or many files, the image size may swell significantly.

Understanding the portability limitations of container images

In theory, a Docker-based container image can be run on any Linux computer running Docker, but one small caveat exists, because containers don't have their own kernel. If a containerized application requires a particular kernel version, it may not work on every computer. If a computer is running a different version of the Linux kernel or doesn't load the required kernel modules, the app can't run on it. This scenario is illustrated in the following figure.

Figure 2.9 If a container requires specific kernel features or modules, it may not work everywhere



Container B requires a specific kernel module to run properly. This module is loaded in the kernel in the first computer, but not in the second. You can run the container image on the second computer, but it will break when it tries to use the missing module.

And it's not just about the kernel and its modules. It should also be clear that a containerized app built for a specific hardware architecture can only run on computers with the same architecture. You can't put an application compiled for the x86 CPU architecture into a container and expect to run it on an ARM-based computer just because Docker is available there. For this you would need a VM to emulate the x86 architecture.

2.1.3 Installing Docker and running a Hello World container

You should now have a basic understanding of what a container is, so let's use Docker to run one. You'll install Docker and run a Hello World container.

Installing Docker

Ideally, you'll install Docker directly on a Linux computer, so you won't have to deal with the additional complexity of running containers inside a VM running within your host OS. But, if you're using macOS or Windows and don't know how to set up a Linux VM, the Docker Desktop application

will set it up for you. The Docker command-line (CLI) tool that you'll use to run containers will be installed in your host OS, but the Docker daemon will run inside the VM, as will all the containers it creates.

The Docker Platform consists of many components, but you only need to install Docker Engine to run containers. If you use macOS or Windows, install Docker Desktop. For details, follow the instructions at <http://docs.docker.com/install>.

Note

Docker Desktop for Windows can run either Windows or Linux containers. Make sure that you configure it to use Linux containers, as all the examples in this book assume that's the case.

Running a Hello World container

After the installation is complete, you use the docker CLI tool to run Docker commands. Let's try pulling and running an existing image from Docker Hub, the public image registry that contains ready-to-use container images for many well-known software packages. One of them is the busybox image, which you'll use to run a simple echo "Hello world" command in your first container.

If you're unfamiliar with busybox, it's a single executable file that combines many of the standard UNIX command-line tools, such as echo, ls, gzip, and so on. Instead of the busybox image, you could also use any other full-fledged OS container image like Fedora, Ubuntu, or any other image that contains the echo executable file.

Once you've got Docker installed, you don't need to download or install anything else to run the busybox image. You can do everything with a single docker run command, by specifying the image to download and the command to run in it. To run the Hello World container, the command and its output are as follows:

```
$ docker run busybox echo "Hello World"
Unable to find image 'busybox:latest' locally      #A
```

```
latest: Pulling from library/busybox      #A
7c9d20b9b6cd: Pull complete              #A
Digest: sha256:fe301db49df08c384001ed752dff6d52b4...  #A
Status: Downloaded newer image for busybox:latest      #A
Hello World      #B
```

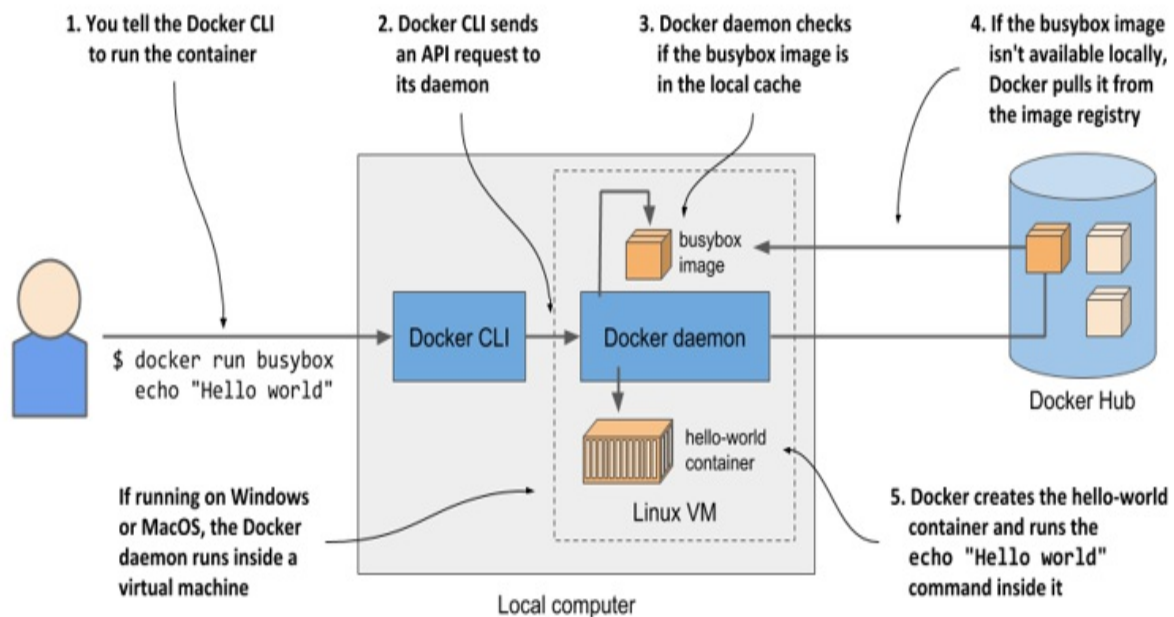
With this single command, you told Docker what image to create the container from and what command to run in the container. This may not look so impressive, but keep in mind that the entire “application” was downloaded and executed with a single command, without you having to install the application or any of its dependencies.

In this example, the application was just a single executable file, but it could also have been a complex application with dozens of libraries and additional files. The entire process of setting up and running the application would be the same. What isn’t obvious is that it ran in a container, isolated from the other processes on the computer. You’ll see that this is true in the remaining exercises in this chapter.

Understanding what happens when you run a container

Figure 2.10 shows exactly what happens when you execute the `docker run` command.

Figure 2.10 Running echo “Hello world” in a container based on the busybox container image



The docker CLI tool sends an instruction to run the container to the Docker daemon, which checks whether the busybox image is already present in its local image cache. If it isn't, the daemon pulls it from the Docker Hub registry.

After downloading the image to your computer, the Docker daemon creates a container from that image and executes the echo command in it. The command prints the text to the standard output, the process then terminates and the container stops.

If your local computer runs a Linux OS, the Docker CLI tool and the daemon both run in this OS. If it runs macOS or Windows, the daemon and the containers run in the Linux VM.

Running other images

Running other existing container images is much the same as running the busybox image. In fact, it's often even simpler, since you don't normally need to specify what command to execute, as with the echo command in the previous example. The command that should be executed is usually written in the image itself, but you can override it when you run it.

For example, if you want to run the Redis datastore, you can find the image name on <http://hub.docker.com> or another public registry. In the case of Redis, one of the images is called `redis:alpine`, so you'd run it like this:

```
$ docker run redis:alpine
```

To stop and exit the container, press Control-C.

Note

If you want to run an image from a different registry, you must specify the registry along with the image name. For example, if you want to run an image from the Quay.io registry, which is another publicly accessible image registry, run it as follows: `docker run quay.io/some/image`.

Understanding image tags

If you've searched for the Redis image on Docker Hub, you've noticed that there are many image *tags* you can choose from. For Redis, the tags are `latest`, `buster`, `alpine`, but also `5.0.7-buster`, `5.0.7-alpine`, and so on.

Docker allows you to have multiple versions or variants of the same image under the same name. Each variant has a unique tag. If you refer to images without explicitly specifying the tag, Docker assumes that you're referring to the special `latest` tag. When uploading a new version of an image, image authors usually tag it with both the actual version number and with `latest`. When you want to run the latest version of an image, use the `latest` tag instead of specifying the version.

Note

The `docker run` command only pulls the image if it hasn't already pulled it before. Using the `latest` tag ensures that you get the latest version when you first run the image. The locally cached image is used from that point on.

Even for a single version, there are usually several variants of an image. For Redis I mentioned `5.0.7-buster` and `5.0.7-alpine`. They both contain the same version of Redis, but differ in the base image they are built on. `5.0.7-`

buster is based on Debian version “Buster”, while 5.0.7-alpine is based on the Alpine Linux base image, a very stripped-down image that is only 5MB in total – it contains only a small set of the installed binaries you see in a typical Linux distribution.

To run a specific version and/or variant of the image, specify the tag in the image name. For example, to run the 5.0.7-alpine tag, you’d execute the following command:

```
$ docker run redis:5.0.7-alpine
```

These days, you can find container images for virtually all popular applications. You can use Docker to run those images using the simple `docker run` single-line command.

2.1.4 Introducing the Open Container Initiative and Docker alternatives

Docker was the first container platform to make containers mainstream. I hope I’ve made it clear that Docker itself is not what provides the process isolation. The actual isolation of containers takes place at the Linux kernel level using the mechanisms it provides. Docker is the tool using these mechanisms to make running container almost trivial. But it’s by no means the only one.

Introducing the Open Container Initiative (OCI)

After the success of Docker, the Open Container Initiative (OCI) was born to create open industry standards around container formats and runtime. Docker is part of this initiative, as are other container runtimes and a number of organizations with interest in container technologies.

OCI members created the *OCI Image Format Specification*, which prescribes a standard format for container images, and the *OCI Runtime Specification*, which defines a standard interface for container runtimes with the aim of standardizing the creation, configuration and execution of containers.

Introducing the Container Runtime Interface (CRI) and its implementation (CRI-O)

This book focuses on using Docker as the container runtime for Kubernetes, as it was initially the only one supported by Kubernetes and is still the most widely used. But Kubernetes now supports many other container runtimes through the Container Runtime Interface (CRI).

One implementation of CRI is CRI-O, a lightweight alternative to Docker that allows you to leverage any OCI-compliant container runtime with Kubernetes. Examples of OCI-compliant runtimes include *rkt* (pronounced Rocket), *runC*, and *Kata Containers*.

2.2 Deploying Kiada—the Kubernetes in Action Demo Application

Now that you’ve got a working Docker setup, you can start building a more complex application. You’ll build a microservices-based application called *Kiada* - the Kubernetes in Action Demo Application.

In this chapter, you’ll use Docker to run this application. In the next and remaining chapters, you’ll run the application in Kubernetes. Over the course of this book, you’ll iteratively expand it and learn about individual Kubernetes features that help you solve the typical problems you face when running applications.

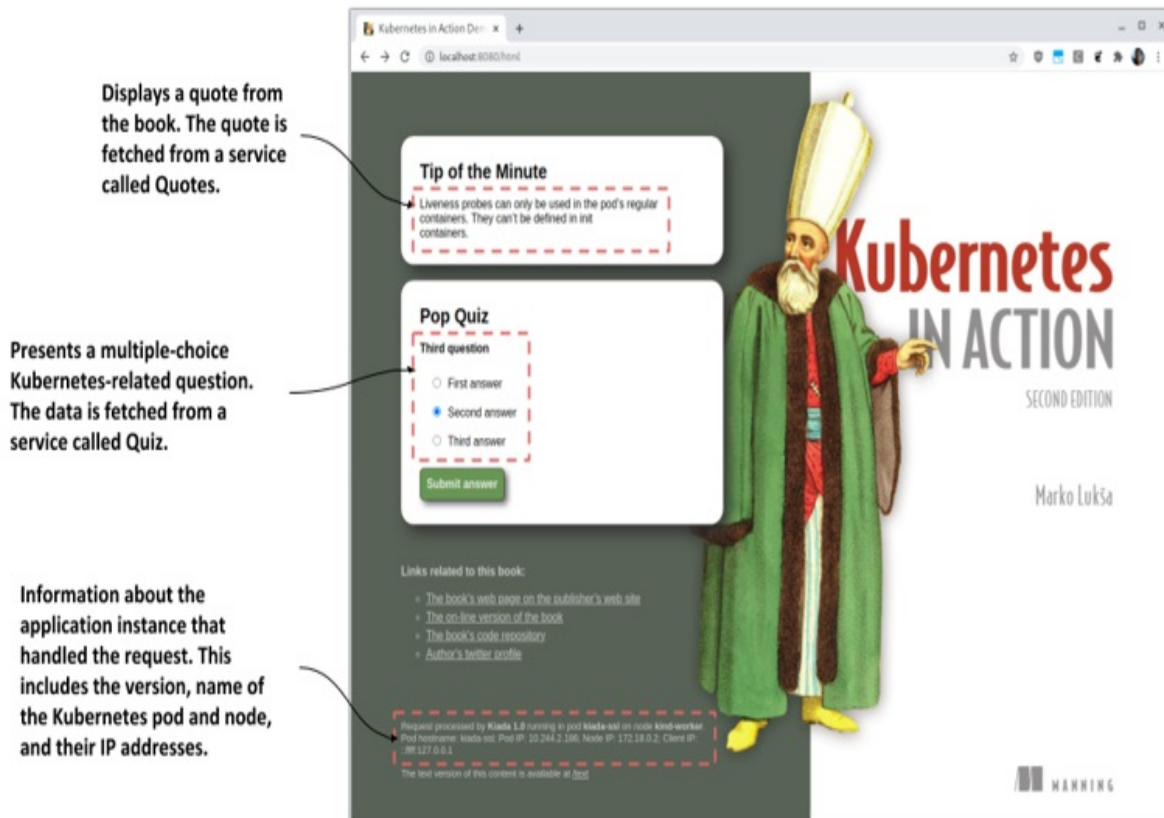
2.2.1 Introducing the Kiada Suite

The Kubernetes in Action Demo Application is a web-based application that shows quotes from this book, asks you Kubernetes-related questions to help you check how your knowledge is progressing, and provides a list of hyperlinks to external websites related to Kubernetes or this book. It also prints out the information about the container that served processed the browser’s request. You’ll soon see why this is important.

The look and operation of the application

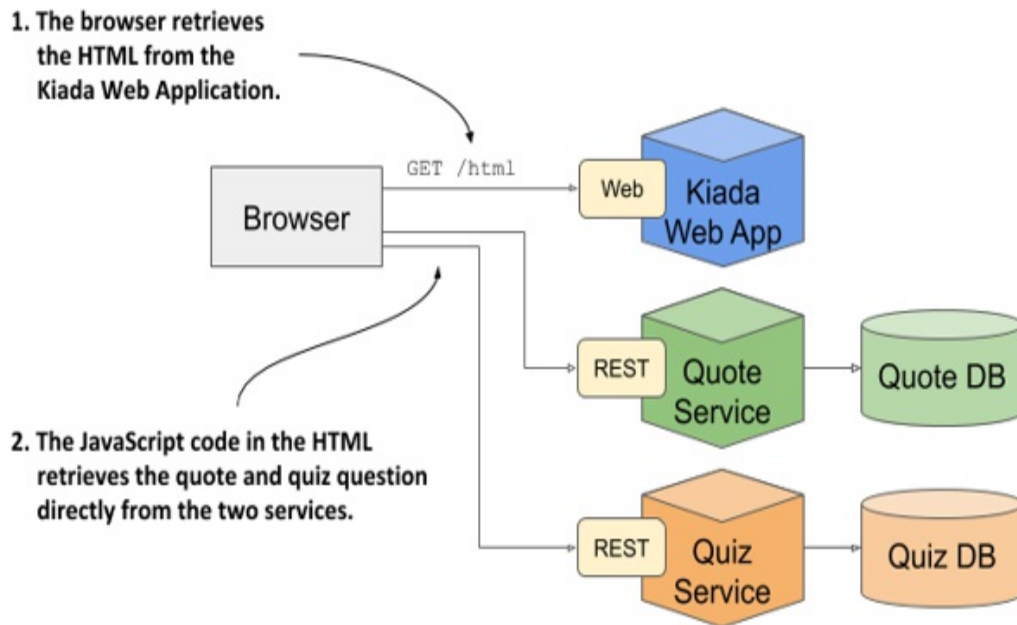
A screenshot of the web application is presented in the following figure.

Figure 2.11 A screenshot of the Kubernetes in Action Demo Application (Kiada)



The architecture of the Kiada application is shown in the next figure. The HTML is served by a web application running in a Node.js server. The client-side JavaScript code then retrieves the quote and question from the Quote and the Quiz RESTful services. The Node.js application and the services comprise the complete *Kiada Suite*.

Figure 2.12 The architecture and operation of the Kiada Suite



The web browser talks directly to three different services. If you're familiar with microservice architectures, you might wonder why no API gateway exists in the system. This is so that I can demonstrate the issues and solutions associated with cases where many different services are deployed in Kubernetes (services that may not belong behind the same API gateway). But chapter 11 will also explain how to introduce Kubernetes-native API gateways into the system.

The look and operation of the plain-text version

You'll spend a lot of time interacting with Kubernetes via a terminal, so you may not want to go back and forth between it and a web browser when you perform the exercises. For this reason, the application can also be used in plain-text mode.

The plain-text mode allows you to use the application directly from the terminal using a tool such as `curl`. In that case, the response sent by the application looks like the following:

==== TIP OF THE MINUTE

Liveness probes can only be used in the pod's regular containers. They can't be defined in init containers.

==== POP QUIZ

Third question

0) First answer

1) Second answer

2) Third answer

Submit your answer to /question/0/answers/<index of answer> using

==== REQUEST INFO

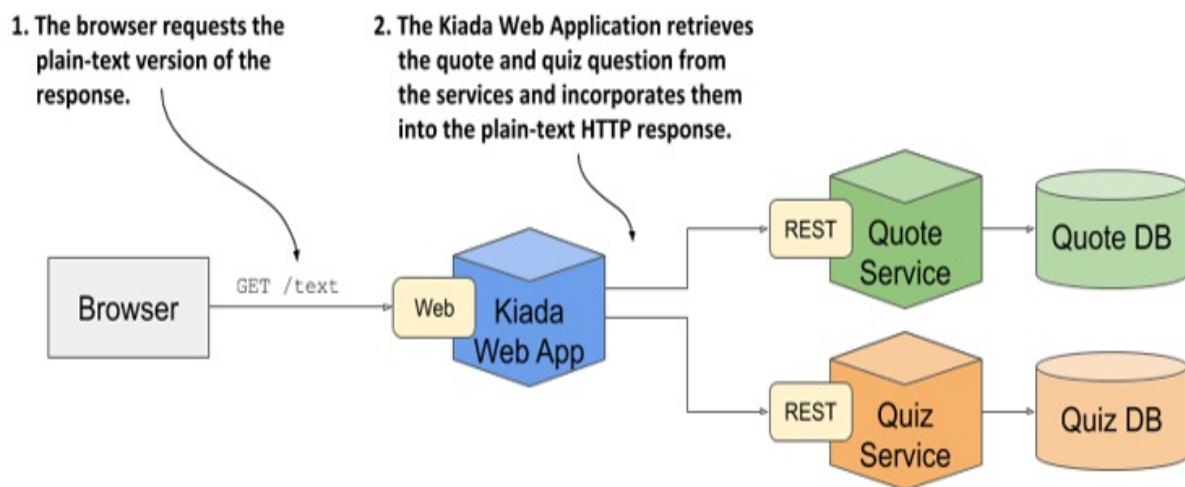
Request processed by Kubelet 1.0 running in pod "kiada-ssl" on node

Pod hostname: kiada-ssl; Pod IP: 10.244.2.188; Node IP: 172.18.0.

The HTML version is accessible at the request URI /html, whereas the text version is at /text. If the client requests the root URI path /, the application inspects the Accept request header to guess whether the client is a graphical web browser, in which case it redirects it to /html, or a text-based tool like curl, in which case it sends the plain-text response.

In this mode of operation, it's the Node.js application that calls the Quote and the Quiz services, as shown in the next figure.

Figure 2.13 The operation when the client requests the text version



From a networking standpoint, this mode of operation is much different than the one described previously. In this case, the Quote and the Quiz service are invoked within the cluster, whereas previously, they were invoked directly

from the browser. To support both operation modes, the services must therefore be exposed both internally and externally.

Note

The initial version of the application will not connect to any services. You'll build and incorporate the services in later chapters.

2.2.2 Building the application

With the general overview of the application behind us, it's time to start building the application. Instead of going straight to the full-blown version of the application, we'll take things slow and build the application iteratively throughout the book.

Introducing the initial version of the application

The initial version of the application that you'll run in this chapter, while supporting both HTML and plain-text modes, will not display the quote and pop quiz, but merely the information about the application and the request. This includes the version of the application, the network hostname of the server that processed the client's request, and the IP of the client. Here's the plain-text response that it sends:

```
Kiada version 0.1. Request processed by "<server-hostname>". Clie
```

The application source code is available in the book's code repository on GitHub. You'll find the code of the initial version in the directory `Chapter02/kiada-0.1`. The JavaScript code is in the `app.js` file and the HTML and other resources are in the `html` subdirectory. The template for the HTML response is in `index.html`. For the plain-text response it's in `index.txt`.

You could now download and install Node.js locally and test the application directly on your computer, but that's not necessary. Since you already have Docker installed, it's easier to package the application into a container image and run it in a container. This way, you don't need to install anything, and

you'll be able to run the same image on any other Docker-enabled host without installing anything there either.

Creating the Dockerfile for the container image

To package your app into an image, you need a file called `Dockerfile`, which contains a list of instructions that Docker performs when building the image. The following listing shows the contents of the file, which you'll find in [Chapter02/kiada-0.1/Dockerfile](#).

Listing 2.1 A minimal Dockerfile for building a container image for your app

```
FROM node:16      #A
COPY app.js /app.js    #B
COPY html/ /html      #C
ENTRYPOINT ["node", "app.js"]    #D
```

The `FROM` line defines the container image that you'll use as the starting point (the base image you're building on top of). The base image used in the listing is the node container image with the tag 12. In the second line, the `app.js` file is copied from your local directory into the root directory of the image. Likewise, the third line copies the `html` directory into the image. Finally, the last line specifies the command that Docker should run when you start the container. In the listing, the command is `node app.js`.

Choosing a base image

You may wonder why use this specific image as your base. Because your app is a Node.js app, you need your image to contain the node binary file to run the app. You could have used any image containing this binary, or you could have even used a Linux distribution base image such as `fedora` or `ubuntu` and installed Node.js into the container when building the image. But since the node image already contains everything needed to run Node.js apps, it doesn't make sense to build the image from scratch. In some organizations, however, the use of a specific base image and adding software to it at build-time may be mandatory.

Building the container image

The Dockerfile, the app.js file, and the files in the html directory is all you need to build your image. With the following command, you'll build the image and tag it as kiada:latest:

```
$ docker build -t kiada:latest .
Sending build context to Docker daemon 3.072kB
Step 1/4 : FROM node:16 #A
12: Pulling from library/node
092586df9206: Pull complete #B
ef599477fae0: Pull complete #B
... #B
89e674ac3af7: Pull complete #B
08df71ec9bb0: Pull complete #B
Digest: sha256:a919d679dd773a56acce15afa0f436055c9b9f20e1f28b4469
Status: Downloaded newer image for node:16
---> e498dabfee1c #C
Step 2/4 : COPY app.js /app.js #D
---> 28d67701d6d9 #D
Step 3/4 : COPY html/ /html #E
---> 1d4de446f0f0 #E
Step 4/4 : ENTRYPOINT ["node", "app.js"] #F
---> Running in a01d42eda116 #F
Removing intermediate container a01d42eda116 #F
---> b0ecc49d7a1d #F
Successfully built b0ecc49d7a1d #G
Successfully tagged kiada:latest #G
```

The -t option specifies the desired image name and tag, and the dot at the end specifies that Dockerfile and the artefacts needed to build the image are in the current directory. This is the so-called build context.

When the build process is complete, the newly created image is available in your computer's local image store. You can see it by listing local images with the following command:

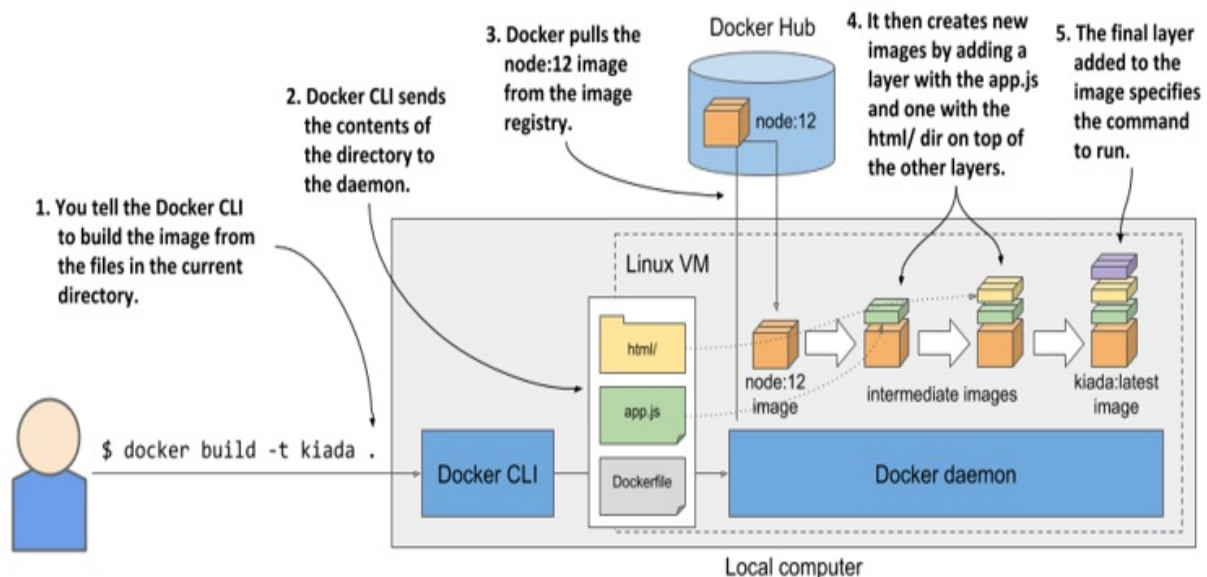
```
$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        VIRT
kiada         latest    b0ecc49d7a1d   1 minute ago   908
...
```

Understanding how the image is built

The following figure shows what happens during the build process. You tell

Docker to build an image called `kiada` based on the contents of the current directory. Docker reads the `Dockerfile` in the directory and builds the image based on the directives in the file.

Figure 2.14 Building a new container image using a Dockerfile



The build itself isn't performed by the docker CLI tool. Instead, the contents of the entire directory are uploaded to the Docker daemon and the image is built by it. You've already learned that the CLI tool and the daemon aren't necessarily on the same computer. If you're using Docker on a non-Linux system such as macOS or Windows, the client is in your host OS, but the daemon runs inside a Linux VM. But it could also run on a remote computer.

Tip

Don't add unnecessary files to the build directory, as they will slow down the build process—especially if the Docker daemon is located on a remote system.

To build the image, Docker first pulls the base image (`node:16`) from the public image repository (Docker Hub in this case), unless the image is already stored locally. It then creates a new container from the image and executes the next directive from the `Dockerfile`. The container's final state

yields a new image with its own ID. The build process continues by processing the remaining directives in the Dockerfile. Each one creates a new image. The final image is then tagged with the tag you specified with the `-t` flag in the `docker build` command.

Understanding the image layers

Some pages ago, you learned that images consist of several layers. One might think that each image consists of only the layers of the base image and a single new layer on top, but that's not the case. When building an image, a new layer is created for each individual directive in the Dockerfile.

During the build of the `kiada` image, after it pulls all the layers of the base image, Docker creates a new layer and adds the `app.js` file into it. It then adds another layer with the files from the `html` directory and finally creates the last layer, which specifies the command to run when the container is started. This last layer is then tagged as `kiada:latest`.

You can see the layers of an image and their size by running `docker history`. The command and its output are shown next (note that the top-most layers are printed first):

```
$ docker history kiada:latest
IMAGE          CREATED        CREATED BY          S
b0ecc49d7a1d   7 min ago     /bin/sh -c #(nop)  ENTRYPOINT ["n...  0
1d4de446f0f0   7 min ago     /bin/sh -c #(nop)  COPY dir:6ecee...   5
28d67701d6d9   7 min ago     /bin/sh -c #(nop)  COPY file:2ed5...   2
e498dabfee1c   2 days ago    /bin/sh -c #(nop)  CMD ["node"]        0
<missing>      2 days ago    /bin/sh -c #(nop)  ENTRYPOINT ["d...   0
<missing>      2 days ago    /bin/sh -c #(nop)  COPY file:2387...   1
<missing>      2 days ago    /bin/sh -c set -ex && for key in...  5
<missing>      2 days ago    /bin/sh -c #(nop)  ENV YARN_VERS...    0
<missing>      2 days ago    /bin/sh -c ARCH= && dpkgArch="$(...  6
<missing>      2 days ago    /bin/sh -c #(nop)  ENV NODE_VERS...    0
<missing>      3 weeks ago   /bin/sh -c groupadd --gid 1000 n...   3
<missing>      3 weeks ago   /bin/sh -c set -ex; apt-get upd...   5
<missing>      3 weeks ago   /bin/sh -c apt-get update && apt...   1
<missing>      3 weeks ago   /bin/sh -c set -ex; if ! comman...   7
<missing>      3 weeks ago   /bin/sh -c apt-get update && apt...   2
<missing>      3 weeks ago   /bin/sh -c #(nop)  CMD ["bash"]      0
<missing>      3 weeks ago   /bin/sh -c #(nop)  ADD file:9788b...   1
```


Most of the layers you see come from the `node:16` image (they also include layers of that image's own base image). The three uppermost layers correspond to the `COPY` and `ENTRYPOINT` directives in the Dockerfile.

As you can see in the `CREATED BY` column, each layer is created by executing a command in the container. In addition to adding files with the `COPY` directive, you can also use other directives in the Dockerfile. For example, the `RUN` directive executes a command in the container during the build. In the listing above, you'll find a layer where the `apt-get update` and some additional `apt-get` commands were executed. `apt-get` is part of the Ubuntu package manager used to install software packages. The command shown in the listing installs some packages onto the image's filesystem.

To learn about `RUN` and other directives you can use in a Dockerfile, refer to the Dockerfile reference at <https://docs.docker.com/engine/reference/builder/>.

Tip

Each directive creates a new layer. I have already mentioned that when you delete a file, it is only marked as deleted in the new layer and is not removed from the layers below. Therefore, deleting a file with a subsequent directive won't reduce the size of the image. If you use the `RUN` directive, make sure that the command it executes deletes all temporary files it creates before it terminates.

2.2.3 Running the container

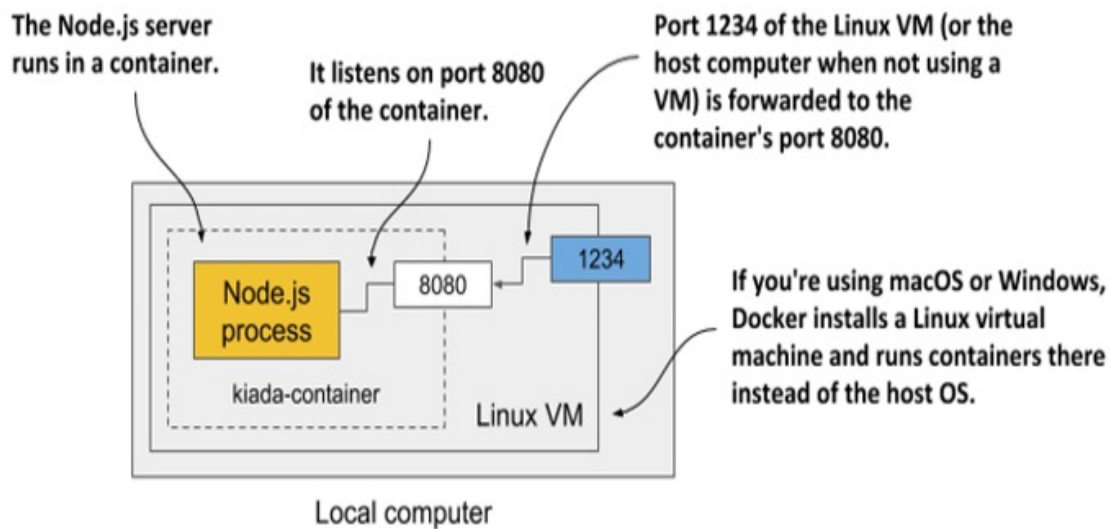
With the image built and ready, you can now run the container with the following command:

```
$ docker run --name kiada-container -p 1234:8080 -d kiada  
9d62e8a9c37e056a82bb1efad57789e947df58669f94adc2006c087a03c54e02
```

This tells Docker to run a new container called `kiada-container` from the `kiada` image. The container is detached from the console (`-d` flag) and runs in the background. Port 1234 on the host computer is mapped to port 8080 in the container (specified by the `-p 1234:8080` option), so you can access the app at <http://localhost:1234>.

The following figure should help you visualize how everything fits together. Note that the Linux VM exists only if you use macOS or Windows. If you use Linux directly, there is no VM and the box depicting port 1234 is at the edge of the local computer.

Figure 2.15 Visualizing your running container



Accessing your app

Now access the application at <http://localhost:1234> using curl or your internet browser:

```
$ curl localhost:1234
Kiada version 0.1. Request processed by "44d76963e8e1". Client IP
```

NOTE

If the Docker Daemon runs on a different machine, you must replace `localhost` with the IP of that machine. You can look it up in the `DOCKER_HOST` environment variable.

If all went well, you should see the response sent by the application. In my case, it returns `44d76963e8e1` as its hostname. In your case, you'll see a

different hexadecimal number. That's the ID of the container. You'll also see it displayed when you list the running containers next.

Listing all running containers

To list all the containers that are running on your computer, run the following command. Its output has been edited to make it more readable—the last two lines of the output are the continuation of the first two.

```
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED
44d76963e8e1   kiada:latest   "node app.js"           6 minutes ago
...
STATUS        PORTS          NAMES
Up 6 minutes   0.0.0.0:1234->8080/tcp   kiada-container
```

For each container, Docker prints its ID and name, the image it uses, and the command it executes. It also shows when the container was created, what status it has, and which host ports are mapped to the container.

Getting additional information about a container

The `docker ps` command shows the most basic information about the containers. To see additional information, you can use `docker inspect`:

```
$ docker inspect kiada-container
```

Docker prints a long JSON-formatted document containing a lot of information about the container, such as its state, config, and network settings, including its IP address.

Inspecting the application log

Docker captures and stores everything the application writes to the standard output and error streams. This is typically the place where applications write their logs. You can use the `docker logs` command to see the output:

```
$ docker logs kiada-container
Kiada - Kubernetes in Action Demo Application
```

```
-----  
Kiada 0.1 starting...  
Local hostname is 44d76963e8e1  
Listening on port 8080  
Received request for / from ::ffff:172.17.0.1
```

You now know the basic commands for executing and inspecting an application in a container. Next, you'll learn how to distribute it.

2.2.4 Distributing the container image

The image you've built is only available locally. To run it on other computers, you must first push it to an external image registry. Let's push it to the public Docker Hub registry, so that you don't need to set up a private one. You can also use other registries, such as Quay.io, which I've already mentioned, or the Google Container Registry.

Before you push the image, you must re-tag it according to Docker Hub's image naming schema. The image name must include your Docker Hub ID, which you choose when you register at <http://hub.docker.com>. I'll use my own ID (luksa) in the following examples, so remember to replace it with your ID when trying the commands yourself.

Tagging an image under an additional tag

Once you have your ID, you're ready to add an additional tag for your image. Its current name is kiada and you'll now tag it also as yourid/kiada:0.1 (replace yourid with your actual Docker Hub ID). This is the command I used:

```
$ docker tag kiada luksa/kiada:0.1
```

Run `docker images` again to confirm that your image now has two names :

```
$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	VIR
luksa/kiada	0.1	b0ecc49d7a1d	About an hour ago	908
kiada	latest	b0ecc49d7a1d	About an hour ago	908
node	12	e498dabfee1c	3 days ago	908
...				

As you can see, both `kiada` and `luksa/kiada:0.1` point to the same image ID, meaning that these aren't two images, but a single image with two names.

Pushing the image to Docker Hub

Before you can push the image to Docker Hub, you must log in with your user ID using the `docker login` command as follows:

```
$ docker login -u yourid docker.io
```

The command will ask you to enter your Docker Hub password. After you're logged in, push the `yourid/kiada:0.1` image to Docker Hub with the following command:

```
$ docker push yourid/kiada:0.1
```

Running the image on other hosts

When the push to Docker Hub is complete, the image is available to all. You can now run the image on any Docker-enabled host by running the following command:

```
$ docker run --name kiada-container -p 1234:8080 -d luksa/kiada:0
```

If the container runs correctly on your computer, it should run on any other Linux computer, provided that the Node.js binary doesn't need any special Kernel features (it doesn't).

2.2.5 Stopping and deleting the container

If you've run the container on the other host, you can now terminate it, as you'll only need the one on your local computer for the exercises that follow.

Stopping a container

Instruct Docker to stop the container with this command:

```
$ docker stop kiada-container
```

This sends a termination signal to the main process in the container so that it can shut down gracefully. If the process doesn't respond to the termination signal or doesn't shut down in time, Docker kills it. When the top-level process in the container terminates, no other process runs in the container, so the container is stopped.

Deleting a container

The container is no longer running, but it still exists. Docker keeps it around in case you decide to start it again. You can see stopped containers by running `docker ps -a`. The `-a` option prints all the containers - those running and those that have been stopped. As an exercise, you can start the container again by running `docker start kiada-container`.

You can safely delete the container on the other host, because you no longer need it. To delete it, run the following `docker rm` command:

```
$ docker rm kiada-container
```

This deletes the container. All its contents are removed and it can no longer be started. The image is still there, though. If you decide to create the container again, the image won't need to be downloaded again. If you also want to delete the image, use the `docker rmi` command:

```
$ docker rmi kiada:latest
```

Alternatively, you can remove all unused images with the `docker image prune` command.

2.3 Understanding containers

You should keep the container running on your local computer so that you can use it in the following exercises, in which you'll examine how containers allow process isolation without using virtual machines. Several features of the Linux kernel make this possible and it's time to get to know them.

2.3.1 Using Namespaces to customize the environment of a

process

The first feature called *Linux Namespaces* ensures that each process has its own view of the system. This means that a process running in a container will only see some of the files, processes and network interfaces on the system, as well as a different system hostname, just as if it were running in a separate virtual machine.

Initially, all the system resources available in a Linux OS, such as filesystems, process IDs, user IDs, network interfaces, and others, are all in the same bucket that all processes see and use. But the Kernel allows you to create additional buckets known as namespaces and move resources into them so that they are organized in smaller sets. This allows you to make each set visible only to one process or a group of processes. When you create a new process, you can specify which namespace it should use. The process only sees resources that are in this namespace and none in the other namespaces.

Introducing the available namespace types

More specifically, there isn't just a single type of namespace. There are in fact several types – one for each resource type. A process thus uses not only one namespace, but one namespace for each type.

The following types of namespaces exist:

- The Mount namespace (mnt) isolates mount points (file systems).
- The Process ID namespace (pid) isolates process IDs.
- The Network namespace (net) isolates network devices, stacks, ports, etc.
- The Inter-process communication namespace (ipc) isolates the communication between processes (this includes isolating message queues, shared memory, and others).
- The UNIX Time-sharing System (UTS) namespace isolates the system hostname and the Network Information Service (NIS) domain name.
- The User ID namespace (user) isolates user and group IDs.
- The Time namespace allows each container to have its own offset to the

system clocks.

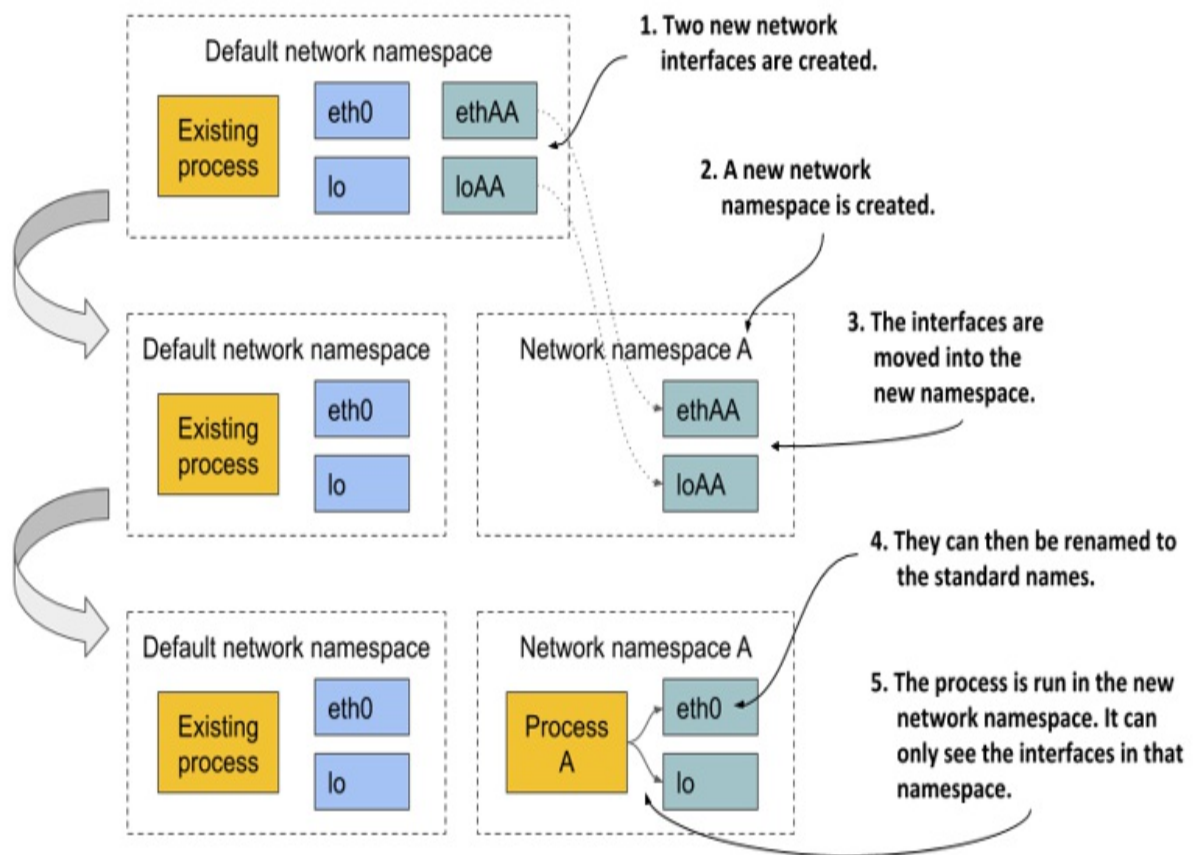
- The Cgroup namespace isolates the Control Groups root directory. You'll learn about cgroups later in this chapter.

Using network namespaces to give a process a dedicated set of network interfaces

The network namespace in which a process runs determines what network interfaces the process can see. Each network interface belongs to exactly one namespace but can be moved from one namespace to another. If each container uses its own network namespace, each container sees its own set of network interfaces.

Examine the following figure for a better overview of how network namespaces are used to create a container. Imagine you want to run a containerized process and provide it with a dedicated set of network interfaces that only that process can use.

Figure 2.16 The network namespace limits which network interfaces a process uses



Initially, only the default network namespace exists. You then create two new network interfaces for the container and a new network namespace. The interfaces can then be moved from the default namespace to the new namespace. Once there, they can be renamed, because names must only be unique in each namespace. Finally, the process can be started in this network namespace, which allows it to only see the two interfaces that were created for it.

By looking solely at the available network interfaces, the process can't tell whether it's in a container or a VM or an OS running directly on a bare-metal machine.

Using the UTS namespace to give a process a dedicated hostname

Another example of how to make it look like the process is running on its own host is to use the UTS namespace. It determines what hostname and

domain name the process running inside this namespace sees. By assigning two different UTS namespaces to two different processes, you can make them see different system hostnames. To the two processes, it looks as if they run on two different computers.

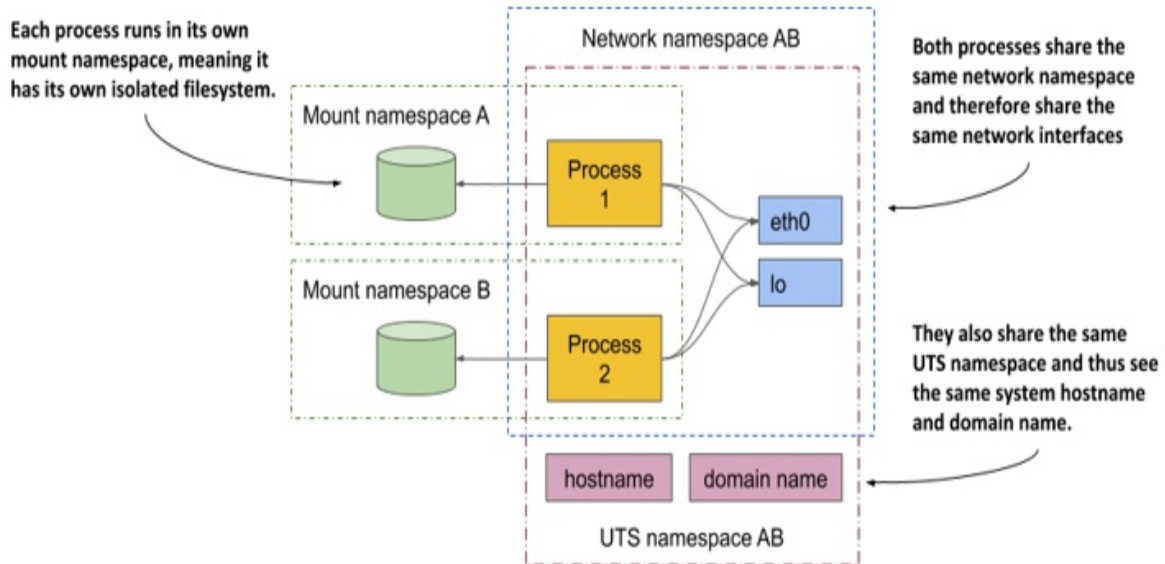
Understanding how namespaces isolate processes from each other

By creating a dedicated namespace instance for all available namespace types and assigning it to a process, you can make the process believe that it's running in its own OS. The main reason for this is that each process has its own environment. A process can only see and use the resources in its own namespaces. It can't use any in other namespaces. Likewise, other processes can't use its resources either. This is how containers isolate the environments of the processes that run within them.

Sharing namespaces between multiple processes

In the next chapter you'll learn that you don't always want to isolate the containers completely from each other. Related containers may want to share certain resources. The following figure shows an example of two processes that share the same network interfaces and the host and domain name of the system, but not the file system.

Figure 2.17 Each process is associated with multiple namespace types, some of which can be shared.



Concentrate on the shared network devices first. The two processes see and use the same two devices (`eth0` and `lo`) because they use the same network namespace. This allows them to bind to the same IP address and communicate through the loopback device, just as they could if they were running on a machine that doesn't use containers. The two processes also use the same UTS namespace and therefore see the same system host name. In contrast, they each use their own mount namespace, which means they have separate file systems.

In summary, processes may want to share some resources but not others. This is possible because separate namespace *types* exist. A process has an associated namespace for each type.

In view of all this, one might ask what is a container at all? A process that runs "in a container" doesn't run in something that resembles a real enclosure like a VM. It's only a process to which several namespaces (one for each type) are assigned. Some are shared with other processes, while others are not. This means that the boundaries between the processes do not all fall on the same line.

In a later chapter, you'll learn how to debug a container by running a new process directly on the host OS, but using the network namespace of an existing container, while using the host's default namespaces for everything

else. This will allow you to debug the container's networking system with tools available on the host that may not be available in the container.

2.3.2 Exploring the environment of a running container

What if you want to see what the environment inside the container looks like? What is the system host name, what is the local IP address, what binaries and libraries are available on the file system, and so on?

To explore these features in the case of a VM, you typically connect to it remotely via ssh and use a shell to execute commands. With containers, you run a shell in the container.

Note

The shell's executable file must be present in the container's file system. This isn't always the case with containers running in production.

Running a shell inside an existing container

The Node.js image on which your image is based provides the bash shell, meaning you can run it in the container with the following command:

```
$ docker exec -it kiada-container bash
root@44d76963e8e1:/#      #A
```

This command runs bash as an additional process in the existing kiada-container container. The process has the same Linux namespaces as the main container process (the running Node.js server). This way you can explore the container from within and see how Node.js and your app see the system when running in the container. The `-it` option is shorthand for two options:

- `-i` tells Docker to run the command in interactive mode.
- `-t` tells it to allocate a pseudo terminal (TTY) so you can use the shell properly.

You need both if you want to use the shell the way you're used to. If you

omit the first, you can't execute any commands, and if you omit the second, the command prompt doesn't appear and some commands may complain that the TERM variable is not set.

Listing running processes in a container

Let's list the processes running in the container by executing the `ps aux` command inside the shell that you ran in the container:

```
root@44d76963e8e1:/# ps aux
USER  PID %CPU %MEM    VSZ   RSS TTY  STAT  START  TIME COMMAND
root    1  0.0  0.1 676380 16504 ?    Sl   12:31   0:00 node app.js
root   10  0.0  0.0  20216   1924 ?    Ss   12:31   0:00 bash
root   19  0.0  0.0  17492   1136 ?    R+   12:38   0:00 ps aux
```

The list shows only three processes. These are the only ones that run in the container. You can't see the other processes that run in the host OS or in other containers because the container runs in its own Process ID namespace.

Seeing container processes in the host's list of processes

If you now open another terminal and list the processes in the host OS itself, you *will* also see the processes that run in the container. This will confirm that the processes in the container are in fact regular processes that run in the host OS. Here's the command and its output:

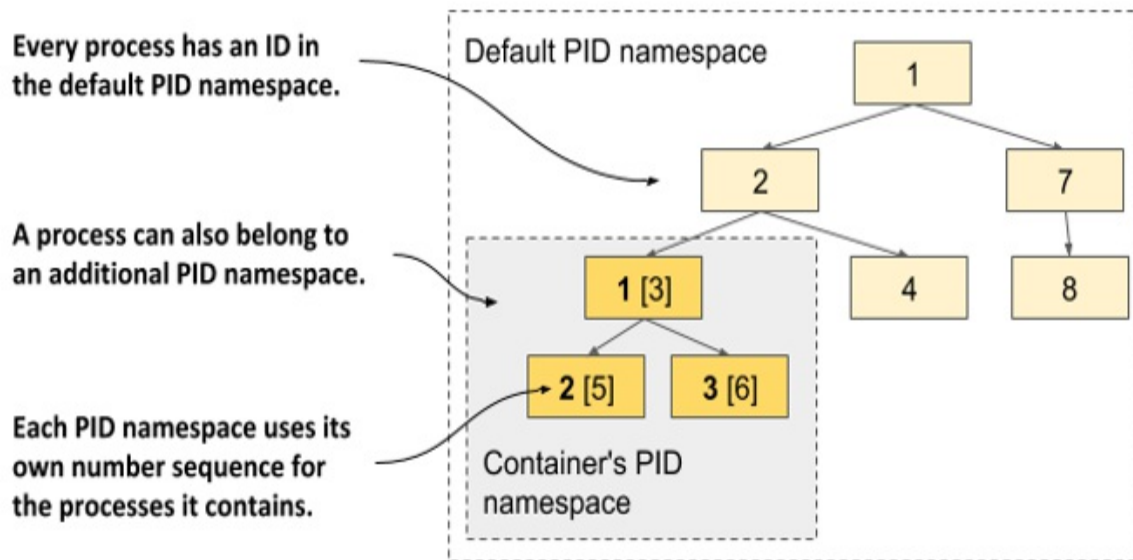
```
$ ps aux | grep app.js
USER  PID %CPU %MEM    VSZ   RSS TTY  STAT  START  TIME COMMAND
root  382  0.0  0.1 676380 16504 ?    Sl   12:31   0:00 node app.js
```

NOTE

If you use macOS or Windows, you must list the processes in the VM that hosts the Docker daemon, as that's where your containers run. In Docker Desktop, you can enter the VM using the command `wsl -d docker-desktop` or with `docker run --net=host --ipc=host --uts=host --pid=host -it --security-opt=seccomp=unconfined --privileged --rm -v /:/host alpine chroot /host`

If you have a sharp eye, you may notice that the process IDs in the container are different from those on the host. Because the container uses its own Process ID namespace it has its own process tree with its own ID number sequence. As the next figure shows, the tree is a subtree of the host's full process tree. Each process thus has two IDs.

Figure 2.18 The PID namespace makes a process sub-tree appear as a separate process tree with its own numbering sequence



Understanding container filesystem isolation from the host and other containers

As with an isolated process tree, each container also has an isolated filesystem. If you list the contents of the container's root directory, only the files in the container are displayed. This includes files from the container image and files created during container operation, such as log files. The next listing shows the files in the kiada container's root file directory:

```
root@44d76963e8e1:/# ls /
app.js  boot  etc   lib   media  opt   root  sbin  sys  usr
bin     dev   home  lib64 mnt    proc  run   srv   tmp  var
```

It contains the `app.js` file and other system directories that are part of the

node:16 base image. You are welcome to browse the container's filesystem. You'll see that there is no way to view files from the host's filesystem. This is great, because it prevents a potential attacker from gaining access to them through vulnerabilities in the Node.js server.

To leave the container, leave the shell by running the `exit` command or pressing Control-D and you'll be returned to your host computer (similar to logging out from an ssh session).

Tip

Entering a running container like this is useful when debugging an app running in a container. When something breaks, the first thing you'll want to investigate is the actual state of the system your application sees.

2.3.3 Limiting a process' resource usage with Linux Control Groups

Linux Namespaces make it possible for processes to access only some of the host's resources, but they don't limit how much of a single resource each process can consume. For example, you can use namespaces to allow a process to access only a particular network interface, but you can't limit the network bandwidth the process consumes. Likewise, you can't use namespaces to limit the CPU time or memory available to a process. You may want to do that to prevent one process from consuming all the CPU time and preventing critical system processes from running properly. For that, we need another feature of the Linux kernel.

Introducing cgroups

The second Linux kernel feature that makes containers possible is called *Linux Control Groups (cgroups)*. It limits, accounts for and isolates system resources such as CPU, memory and disk or network bandwidth. When using cgroups, a process or group of processes can only use the allotted CPU time, memory, and network bandwidth for example. This way, processes cannot occupy resources that are reserved for other processes.

At this point, you don't need to know how Control Groups do all this, but it may be worth seeing how you can ask Docker to limit the amount of CPU and memory a container can use.

Limiting a container's use of the CPU

If you don't impose any restrictions on the container's use of the CPU, it has unrestricted access to all CPU cores on the host. You can explicitly specify which cores a container can use with Docker's `--cpuset-cpus` option. For example, to allow the container to only use cores one and two, you can run the container with the following option:

```
$ docker run --cpuset-cpus="1,2" ...
```

You can also limit the available CPU time using options `--cpus`, `--cpu-period`, `--cpu-quota` and `--cpu-shares`. For example, to allow the container to use only half of a CPU core, run the container as follows:

```
$ docker run --cpus="0.5" ...
```

Limiting a container's use of memory

As with CPU, a container can use all the available system memory, just like any regular OS process, but you may want to limit this. Docker provides the following options to limit container memory and swap usage: `--memory`, `--memory-reservation`, `--kernel-memory`, `--memory-swap`, and `--memory-swappiness`.

For example, to set the maximum memory size available in the container to 100MB, run the container as follows (m stands for megabyte):

```
$ docker run --memory="100m" ...
```

Behind the scenes, all these Docker options merely configure the cgroups of the process. It's the Kernel that takes care of limiting the resources available to the process. See the Docker documentation for more information about the other memory and CPU limit options.

2.3.4 Strengthening isolation between containers

Linux Namespaces and Cgroups separate the containers' environments and prevent one container from starving the other containers of compute resources. But the processes in these containers use the same system kernel, so we can't say that they are really isolated. A rogue container could make malicious system calls that would affect its neighbours.

Imagine a Kubernetes node on which several containers run. Each has its own network devices and files and can only consume a limited amount of CPU and memory. At first glance, a rogue program in one of these containers can't cause damage to the other containers. But what if the rogue program modifies the system clock that is shared by all containers?

Depending on the application, changing the time may not be too much of a problem, but allowing programs to make any system call to the kernel allows them to do virtually anything. Sys-calls allow them to modify the kernel memory, add or remove kernel modules, and many other things that regular containers aren't supposed to do.

This brings us to the third set of technologies that make containers possible. Explaining them fully is outside the scope of this book, so please refer to other resources that focus specifically on containers or the technologies used to secure them. This section provides a brief introduction to these technologies.

Giving containers full privileges to the system

The operating system kernel provides a set of *sys-calls* that programs use to interact with the operating system and underlying hardware. These includes calls to create processes, manipulate files and devices, establish communication channels between applications, and so on.

Some of these sys-calls are safe and available to any process, but others are reserved for processes with elevated privileges only. If you look at the example presented earlier, applications running on the Kubernetes node should be allowed to open their local files, but not change the system clock or

modify the kernel in a way that breaks the other containers.

Most containers should run without elevated privileges. Only those programs that you trust and that actually need the additional privileges should run in privileged containers.

Note

With Docker you create a privileged container by using the `--privileged` flag.

Using Capabilities to give containers a subset of all privileges

If an application only needs to invoke some of the sys-calls that require elevated privileges, creating a container with full privileges is not ideal. Fortunately, the Linux kernel also divides privileges into units called *capabilities*. Examples of capabilities are:

- `CAP_NET_ADMIN` allows the process to perform network-related operations,
- `CAP_NET_BIND_SERVICE` allows it to bind to port numbers less than 1024,
- `CAP_SYS_TIME` allows it to modify the system clock, and so on.

Capabilities can be added or removed (*dropped*) from a container when you create it. Each capability represents a set of privileges available to the processes in the container. Docker and Kubernetes drop all capabilities except those required by typical applications, but users can add or drop other capabilities if authorized to do so.

Note

Always follow the *principle of least privilege* when running containers. Don't give them any capabilities that they don't need. This prevents attackers from using them to gain access to your operating system.

Using seccomp profiles to filter individual sys-calls

If you need even finer control over what sys-calls a program can make, you can use *seccomp* (Secure Computing Mode). You can create a custom seccomp profile by creating a JSON file that lists the sys-calls that the container using the profile is allowed to make. You then provide the file to Docker when you create the container.

Hardening containers using AppArmor and SELinux

And as if the technologies discussed so far weren't enough, containers can also be secured with two additional mandatory access control (MAC) mechanisms: SELinux (Security-Enhanced Linux) and AppArmor (Application Armor).

With SELinux, you attach labels to files and system resources, as well as to users and processes. A user or process can only access a file or resource if the labels of all subjects and objects involved match a set of policies. AppArmor is similar but uses file paths instead of labels and focuses on processes rather than users.

Both SELinux and AppArmor considerably improve the security of an operating system, but don't worry if you are overwhelmed by all these security-related mechanisms. The aim of this section was to shed light on everything involved in the proper isolation of containers, but a basic understanding of namespaces should be more than sufficient for the moment.

2.4 Summary

If you were new to containers before reading this chapter, you should now understand what they are, why we use them, and what features of the Linux kernel make them possible. If you have previously used containers, I hope this chapter has helped to clarify your uncertainties about how containers work, and you now understand that they're nothing more than regular OS processes that the Linux kernel isolates from other processes.

After reading this chapter, you should know that:

- Containers are regular processes, but isolated from each other and the

other processes running in the host OS.

- Containers are much lighter than virtual machines, but because they use the same Linux kernel, they are not as isolated as VMs.
- Docker was the first container platform to make containers popular and the first container runtime supported by Kubernetes. Now, others are supported through the Container Runtime Interface (CRI).
- A container image contains the user application and all its dependencies. It is distributed through a container registry and used to create running containers.
- Containers can be downloaded and executed with a single `docker run` command.
- Docker builds an image from a `Dockerfile` that contains commands that Docker should execute during the build process. Images consist of layers that can be shared between multiple images. Each layer only needs to be transmitted and stored once.
- Containers are isolated by Linux kernel features called Namespaces, Control groups, Capabilities, seccomp, AppArmor and/or SELinux. Namespaces ensure that a container sees only a part of the resources available on the host, Control groups limit the amount of a resource it can use, while other features strengthen the isolation between containers.

After inspecting the containers on this ship, you're now ready to raise the anchor and sail into the next chapter, where you'll learn about running containers with Kubernetes.

3 Deploying your first application

This chapter covers

- Running a local Kubernetes cluster on your laptop
- Setting up a cluster on Google Kubernetes Engine
- Setting up a cluster on Amazon Elastic Kubernetes Service
- Setting up and using the `kubectl` command-line tool
- Deploying an application in Kubernetes and making it available across the globe
- Horizontally scaling the application

The goal of this chapter is to show you how to run a local single-node development Kubernetes cluster or set up a proper, managed multi-node cluster in the cloud. Once your cluster is running, you'll use it to run the container you created in the previous chapter.

Note

You'll find the code files for this chapter at <https://github.com/luksa/kubernetes-in-action-2nd-edition/tree/master/Chapter03>

3.1 Deploying a Kubernetes cluster

Setting up a full-fledged, multi-node Kubernetes cluster isn't a simple task, especially if you're not familiar with Linux and network administration. A proper Kubernetes installation spans multiple physical or virtual machines and requires proper network setup to allow all containers in the cluster to communicate with each other.

You can install Kubernetes on your laptop computer, on your organization's infrastructure, or on virtual machines provided by cloud providers (Google Compute Engine, Amazon EC2, Microsoft Azure, and so on). Alternatively,

most cloud providers now offer managed Kubernetes services, saving you from the hassle of installation and management. Here's a short overview of what the largest cloud providers offer:

- Google offers GKE - Google Kubernetes Engine,
- Amazon has EKS - Amazon Elastic Kubernetes Service,
- Microsoft has AKS – Azure Kubernetes Service,
- IBM has IBM Cloud Kubernetes Service,
- Alibaba provides the Alibaba Cloud Container Service.

Installing and managing Kubernetes is much more difficult than just using it, especially until you're intimately familiar with its architecture and operation. For this reason, we'll start with the easiest ways to obtain a working Kubernetes cluster. You'll learn several ways to run a single-node Kubernetes cluster on your local computer and how to use a hosted cluster running on Google Kubernetes Engine (GKE).

A third option, which involves installing a cluster using the `kubeadm` tool, is explained in Appendix B. The tutorial there will show you how to set up a three-node Kubernetes cluster using virtual machines. But you may want to try that only after you've become familiar with using Kubernetes. Many other options also exist, but they are beyond the scope of this book. Refer to the kubernetes.io website to learn more.

If you've been granted access to an existing cluster deployed by someone else, you can skip this section and go on to section 3.2 where you'll learn how to interact with Kubernetes clusters.

3.1.1 Using the built-in Kubernetes cluster in Docker Desktop

If you use macOS or Windows, you've most likely installed Docker Desktop to run the exercises in the previous chapter. It contains a single-node Kubernetes cluster that you can enable via its Settings dialog box. This may be the easiest way for you to start your Kubernetes journey, but keep in mind that the version of Kubernetes may not be as recent as when using the alternative options described in the next sections.

Note

Although technically not a cluster, the single-node Kubernetes system provided by Docker Desktop should be enough to explore most of the topics discussed in this book. When an exercise requires a multi-node cluster, I will point this out.

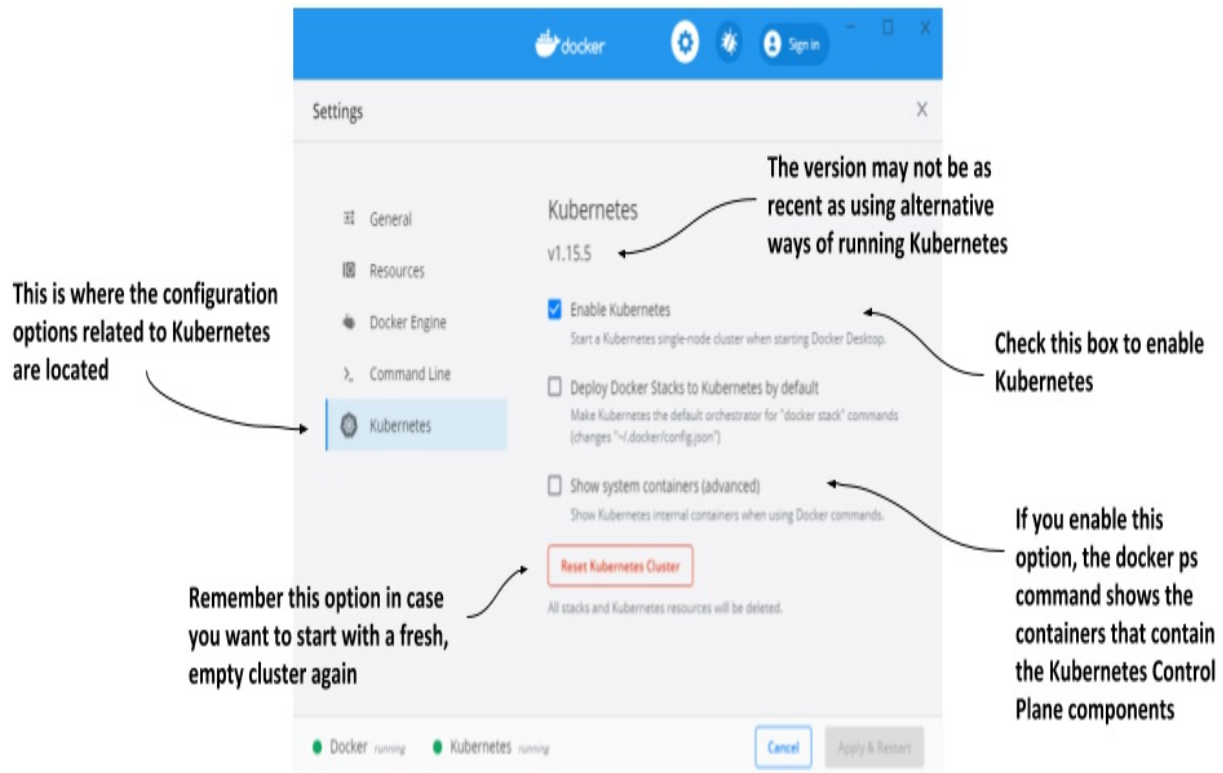
Enabling Kubernetes in Docker Desktop

Assuming Docker Desktop is already installed on your computer, you can start the Kubernetes cluster by clicking the whale icon in the system tray and opening the Settings dialog box. Click the *Kubernetes* tab and make sure the *Enable Kubernetes* checkbox is selected. The components that make up the Control Plane run as Docker containers, but they aren't displayed in the list of running containers when you invoke the `docker ps` command. To display them, select the *Show system containers* checkbox.

Note

The initial installation of the cluster takes several minutes, as all container images for the Kubernetes components must be downloaded.

Figure 3.1 The Settings dialog box in Docker Desktop for Windows

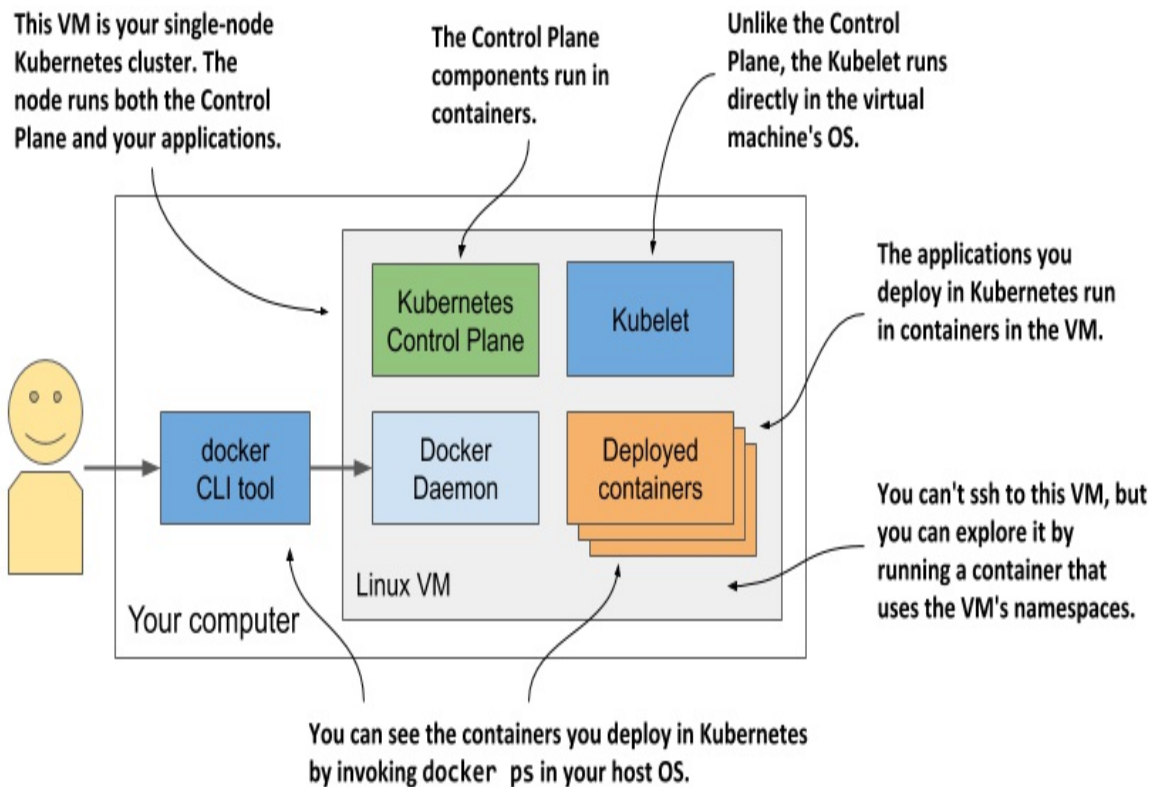


Remember the *Reset Kubernetes Cluster* button if you ever want to reset the cluster to remove all the objects you've deployed in it.

Visualizing the system

To understand where the various components that make up the Kubernetes cluster run in Docker Desktop, look at the following figure.

Figure 3.2 Kubernetes running in Docker Desktop



Docker Desktop sets up a Linux virtual machine that hosts the Docker Daemon and all the containers. This VM also runs the Kubelet - the Kubernetes agent that manages the node. The components of the Control Plane run in containers, as do all the applications you deploy.

To list the running containers, you don't need to log on to the VM because the docker CLI tool available in your host OS displays them.

Exploring the Virtual Machine from the inside

At the time of writing, Docker Desktop provides no command to log into the VM if you want to explore it from the inside. However, you can run a special container configured to use the VM's namespaces to run a remote shell, which is virtually identical to using SSH to access a remote server. To run the container, execute the following command:

```
$ docker run --net=host --ipc=host --uts=host --pid=host --privileged --security-opt=seccomp=unconfined -it --rm -v /:/host alpine ch
```

This long command requires explanation:

- The container is created from the alpine image.
- The `--net`, `--ipc`, `--uts` and `--pid` flags make the container use the host's namespaces instead of being sandboxed, and the `--privileged` and `--security-opt` flags give the container unrestricted access to all sys-calls.
- The `-it` flag runs the container interactive mode and the `--rm` flags ensures the container is deleted when it terminates.
- The `-v` flag mounts the host's root directory to the `/host` directory in the container. The `chroot /host` command then makes this directory the root directory in the container.

After you run the command, you are in a shell that's effectively the same as if you had used SSH to enter the VM. Use this shell to explore the VM - try listing processes by executing the `ps aux` command or explore the network interfaces by running `ip addr`.

3.1.2 Running a local cluster using Minikube

Another way to create a Kubernetes cluster is to use *Minikube*, a tool maintained by the Kubernetes community. The version of Kubernetes that Minikube deploys is usually more recent than the version deployed by Docker Desktop. The cluster consists of a single node and is suitable for both testing Kubernetes and developing applications locally. It normally runs Kubernetes in a Linux VM, but if your computer is Linux-based, it can also deploy Kubernetes directly in your host OS via Docker.

Note

If you configure Minikube to use a VM, you don't need Docker, but you do need a hypervisor like VirtualBox. In the other case you need Docker, but not the hypervisor.

Installing Minikube

Minikube supports macOS, Linux, and Windows. It has a single binary

executable file, which you'll find in the Minikube repository on GitHub (<http://github.com/kubernetes/minikube>). It's best to follow the current installation instructions published there, but roughly speaking, you install it as follows.

On macOS you can install it using the Brew Package Manager, on Windows there's an installer that you can download, and on Linux you can either download a .deb or .rpm package or simply download the binary file and make it executable with the following command:

```
$ curl -LO https://storage.googleapis.com/minikube/releases/latest  
sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

For details on your specific OS, please refer to the installation guide online.

Starting a Kubernetes cluster with Minikube

After Minikube is installed, start the Kubernetes cluster as shown next:

```
$ minikube start  
minikube v1.11.0 on Fedora 31  
Using the virtualbox driver based on user configuration  
Downloading VM boot image ...  
> minikube-v1.11.0.iso.sha256: 65 B / 65 B [-----] 100.00  
> minikube-v1.11.0.iso: 174.99 MiB / 174.99 MiB [] 100.00% 50.16  
Starting control plane node minikube in cluster minikube  
Downloading Kubernetes v1.18.3 preload ...  
> preloaded-images-k8s-v3-v1.18.3-docker-overlay2-amd64.tar.lz4:  
Creating virtualbox VM (CPUs=2, Memory=6000MB, Disk=20000MB) ...  
Preparing Kubernetes v1.18.3 on Docker 19.03.8 ...  
Verifying Kubernetes components...  
Enabled addons: default-storageclass, storage-provisioner  
Done! kubectl is now configured to use "minikube"
```

The process may take several minutes, because the VM image and the container images of the Kubernetes components must be downloaded.

Tip

If you use Linux, you can reduce the resources required by Minikube by creating the cluster without a VM. Use this command: `minikube start --`

```
vm-driver none
```

Checking Minikube's status

When the `minikube start` command is complete, you can check the status of the cluster by running the `minikube status` command:

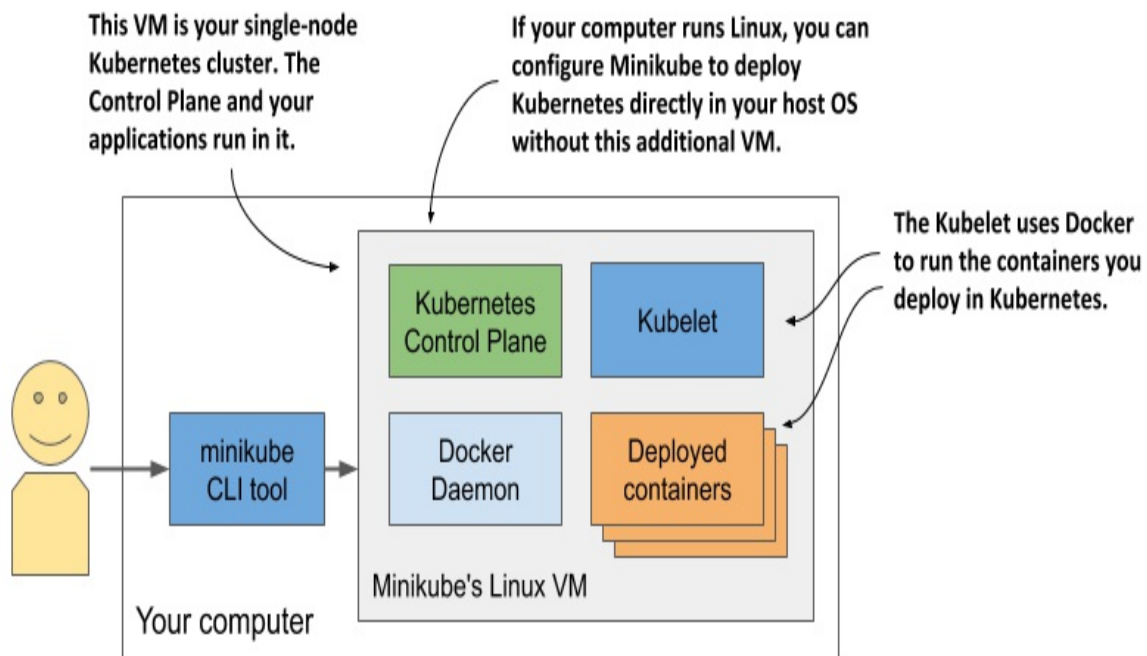
```
$ minikube status
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

The output of the command shows that the Kubernetes host (the VM that hosts Kubernetes) is running, and so are the Kubelet – the agent responsible for managing the node – and the Kubernetes API server. The last line shows that the `kubectl` command-line tool (CLI) is configured to use the Kubernetes cluster that Minikube has provided. Minikube doesn't install the CLI tool, but it does create its configuration file. Installation of the CLI tool is explained in section 3.2.

Visualizing the system

The architecture of the system, which is shown in the next figure, is practically identical to the one in Docker Desktop.

Figure 3.3 Running a single-node Kubernetes cluster using Minikube



The Control Plane components run in containers in the VM or directly in your host OS if you used the `--vm-driver none` option to create the cluster. The Kubelet runs directly in the VM's or your host's operating system. It runs the applications you deploy in the cluster via the Docker Daemon.

You can run `minikube ssh` to log into the Minikube VM and explore it from inside. For example, you can see what's running in the VM by running `ps aux` to list running processes or `docker ps` to list running containers.

Tip

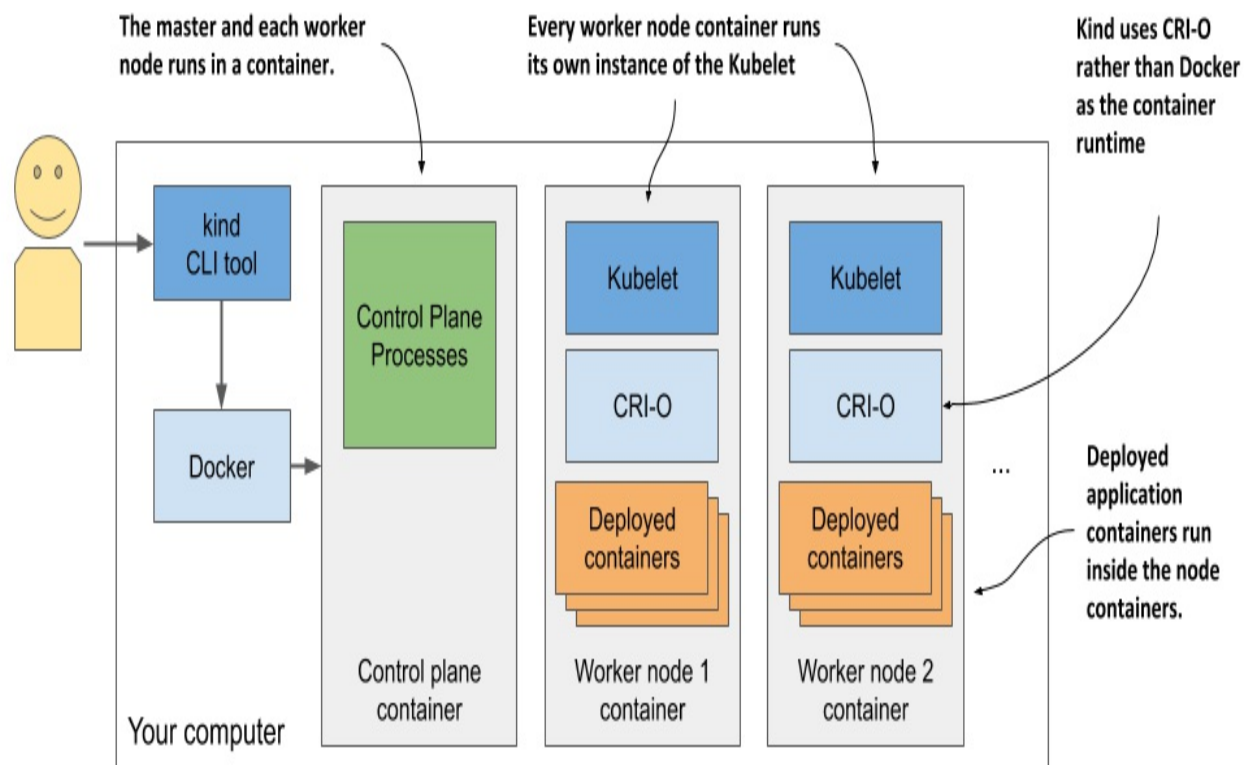
If you want to list containers using your local docker CLI instance, as in the case of Docker Desktop, run the following command: `eval $(minikube docker -env)`

3.1.3 Running a local cluster using kind (Kubernetes in Docker)

An alternative to Minikube, although not as mature, is *kind* (Kubernetes-in-Docker). Instead of running Kubernetes in a virtual machine or directly on

the host, kind runs each Kubernetes cluster node inside a container. Unlike Minikube, this allows it to create multi-node clusters by starting several containers. The actual application containers that you deploy to Kubernetes then run within these node containers. The system is shown in the next figure.

Figure 3.4 Running a multi-node Kubernetes cluster using kind



In the previous chapter I mentioned that a process that runs in a container actually runs in the host OS. This means that when you run Kubernetes using kind, all Kubernetes components run in your host OS. The applications you deploy to the Kubernetes cluster also run in your host OS.

This makes kind the perfect tool for development and testing, as everything

runs locally and you can debug running processes as easily as when you run them outside of a container. I prefer to use this approach when I develop apps on Kubernetes, as it allows me to do magical things like run network traffic analysis tools such as Wireshark or even my web browser inside the containers that run my applications. I use a tool called nsenter that allows me to run these tools in the network or other namespaces of the container.

If you're new to Kubernetes, the safest bet is to start with Minikube, but if you're feeling adventurous, here's how to get started with kind.

Installing kind

Just like Minikube, kind consists of a single binary executable file. To install it, refer to the installation instructions at <https://kind.sigs.k8s.io/docs/user/quick-start/>. On macOS and Linux, the commands to install it are as follows:

```
$ curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.11.1/kind-$(uname)
$ chmod +x ./kind
$ mv ./kind /some-dir-in-your-PATH/kind
```

Check the documentation to see what the latest version is and use it instead of v0.7.0 in the above example. Also, replace /some-dir-in-your-PATH/ with an actual directory in your path.

Note

Docker must be installed on your system to use kind.

Starting a Kubernetes cluster with kind

Starting a new cluster is as easy as it is with Minikube. Execute the following command:

```
$ kind create cluster
```

Like Minikube, kind configures kubectl to use the cluster that it creates.

Starting a multi-node cluster with kind

Kind runs a single-node cluster by default. If you want to run a cluster with multiple worker nodes, you must first create a configuration file. The following listing shows the contents of this file (Chapter03/kind-multi-node.yaml).

Listing 3.1 Config file for running a three-node cluster with the kind tool

```
kind: Cluster
apiVersion: kind.sigs.k8s.io/v1alpha3
nodes:
- role: control-plane
- role: worker
- role: worker
```

With the file in place, create the cluster using the following command:

```
$ kind create cluster --config kind-multi-node.yaml
```

Listing worker nodes

At the time of this writing, kind doesn't provide a command to check the status of the cluster, but you can list cluster nodes using `kind get nodes`:

```
$ kind get nodes
kind-worker2
kind-worker
kind-control-plane
```

Since each node runs as a container, you can also see the nodes by listing the running containers using `docker ps`:

```
$ docker ps
CONTAINER ID   IMAGE                                ...   NAMES
45d0f712eac0   kindest/node:v1.18.2               ...   kind-worker2
d1e88e98e3ae   kindest/node:v1.18.2               ...   kind-worker
4b7751144ca4   kindest/node:v1.18.2               ...   kind-control-plane
```

Logging into cluster nodes provisioned by kind

Unlike Minikube, where you use `minikube ssh` to log into the node if you want to explore the processes running inside it, with `kind` you use `docker exec`. For example, to enter the node called `kind-control-plane`, run:

```
$ docker exec -it kind-control-plane bash
```

Instead of using Docker to run containers, nodes created by `kind` use the CRI-O container runtime, which I mentioned in the previous chapter as a lightweight alternative to Docker. The `crictl` CLI tool is used to interact with CRI-O. Its use is very similar to that of the `docker` tool. After logging into the node, list the containers running in it by running `crictl ps` instead of `docker ps`. Here's an example of the command and its output:

```
root@kind-control-plane:/# crictl ps
```

CONTAINER ID	IMAGE	CREATED	STATE	NAME
c7f44d171fb72	eb516548c180f	15 min ago	Running	coredns
cce9c0261854c	eb516548c180f	15 min ago	Running	coredns
e6522aae66fcc	d428039608992	16 min ago	Running	kube-proxy
6b2dc4bbfee0c	ef97cccdfdb50	16 min ago	Running	kindnet-cn
c3e66dfe44deb	be321f2ded3f3	16 min ago	Running	kube-apiserver

3.1.4 Creating a managed cluster with Google Kubernetes Engine

If you want to use a full-fledged multi-node Kubernetes cluster instead of a local one, you can use a managed cluster, such as the one provided by Google Kubernetes Engine (GKE). This way, you don't have to manually set up all the cluster nodes and networking, which is usually too hard for someone taking their first steps with Kubernetes. Using a managed solution such as GKE ensures that you don't end up with an incorrectly configured cluster.

Setting up Google Cloud and installing the `gcloud` client binary

Before you can set up a new Kubernetes cluster, you must set up your GKE environment. The process may change in the future, so I'll only give you a few general instructions here. For complete instructions, refer to <https://cloud.google.com/container-engine/docs/before-you-begin>.

Roughly, the whole procedure includes

1. Signing up for a Google account if you don't have one already.
2. Creating a project in the Google Cloud Platform Console.
3. Enabling billing. This does require your credit card info, but Google provides a 12-month free trial with a free \$300 credit. And they don't start charging automatically after the free trial is over.
4. Downloading and installing the Google Cloud SDK, which includes the `gcloud` tool.
5. Creating the cluster using the `gcloud` command-line tool.

NOTE

Certain operations (the one in step 2, for example) may take a few minutes to complete, so relax and grab a coffee in the meantime.

Creating a GKE Kubernetes cluster with three nodes

Before you create your cluster, you must decide in which geographical region and zone it should be created. Refer to <https://cloud.google.com/compute/docs/regions-zones> to see the list of available locations. In the following examples, I use the `eu-west3` region based in Frankfurt, Germany. It has three different zones - I'll use the zone `eu-west3-c`. The default zone for all `gcloud` operations can be set with the following command:

```
$ gcloud config set compute/zone eu-west3-c
```

Create the Kubernetes cluster like this:

```
$ gcloud container clusters create kiada --num-nodes 3
Creating cluster kiada in eu-west3-c...
...
kubeconfig entry generated for kiada.
NAME      LOCAT.  MASTER_VER  MASTER_IP  MACH_TYPE  ...  NODES  S
kiada     eu-w3-c  1.13.11...  5.24.21.22  n1-standard-1  ...  3      R
```

Note

I'm creating all three worker nodes in the same zone, but you can also spread them across all zones in the region by setting the `compute/zone` config value

to an entire region instead of a single zone. If you do so, note that `--num-nodes` indicates the number of nodes *per zone*. If the region contains three zones and you only want three nodes, you must set `--num-nodes` to 1.

You should now have a running Kubernetes cluster with three worker nodes. Each node is a virtual machine provided by the Google Compute Engine (GCE) infrastructure-as-a-service platform. You can list GCE virtual machines using the following command:

```
$ gcloud compute instances list
```

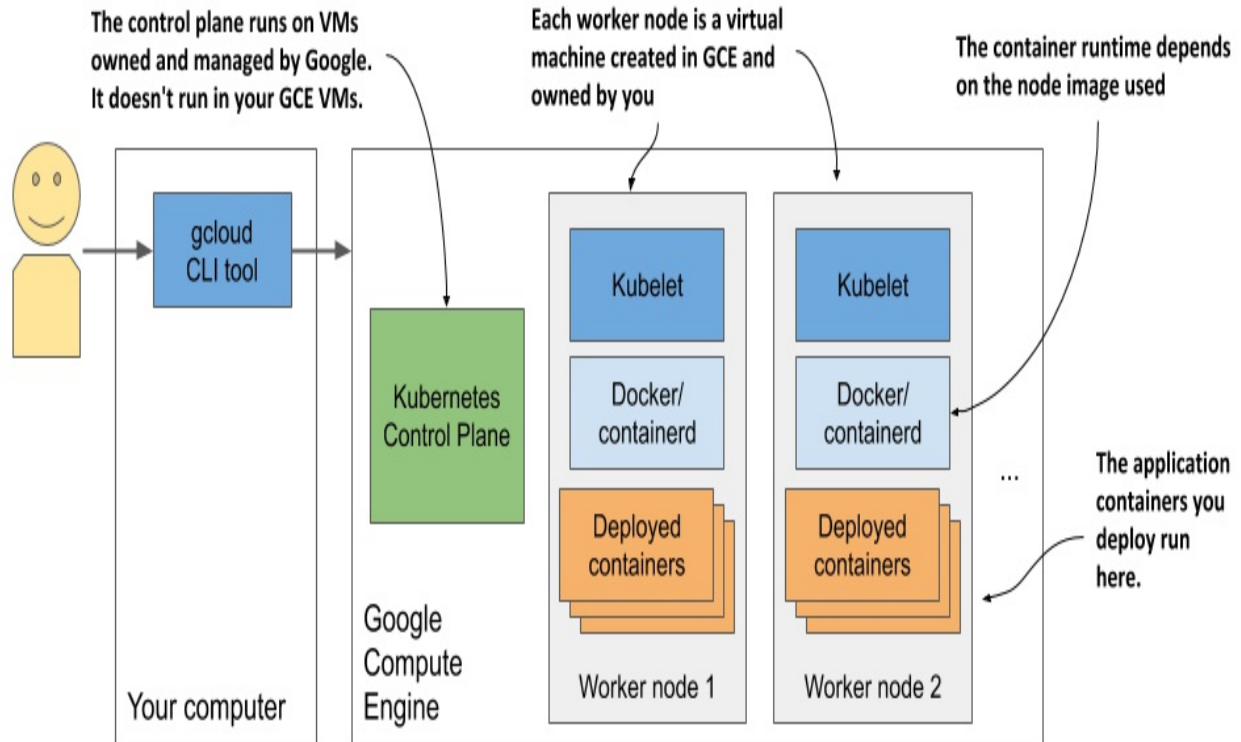
NAME	ZONE	MACHINE_TYPE	INTERNAL_IP	EXTERNAL_IP
...-ctlk	eu-west3-c	n1-standard-1	10.156.0.16	34.89.238.55
...-gj1f	eu-west3-c	n1-standard-1	10.156.0.14	35.242.223.97
...-r01z	eu-west3-c	n1-standard-1	10.156.0.15	35.198.191.189

Tip

Each VM incurs costs. To reduce the cost of your cluster, you can reduce the number of nodes to one, or even to zero while not using it. See next section for details.

The system is shown in the next figure. Note that only your worker nodes run in GCE virtual machines. The control plane runs elsewhere - you can't access the machines hosting it.

Figure 3.5 Your Kubernetes cluster in Google Kubernetes Engine



Scaling the number of nodes

Google allows you to easily increase or decrease the number of nodes in your cluster. For most exercises in this book you can scale it down to just one node if you want to save money. You can even scale it down to zero so that your cluster doesn't incur any costs.

To scale the cluster to zero, use the following command:

```
$ gcloud container clusters resize kiada --size 0
```

The nice thing about scaling to zero is that none of the objects you create in your Kubernetes cluster, including the applications you deploy, are deleted. Granted, if you scale down to zero, the applications will have no nodes to run on, so they won't run. But as soon as you scale the cluster back up, they will be redeployed. And even with no worker nodes you can still interact with the Kubernetes API (you can create, update, and delete objects).

Inspecting a GKE worker node

If you're interested in what's running on your nodes, you can log into them with the following command (use one of the node names from the output of the previous command):

```
$ gcloud compute ssh gke-kiada-default-pool-9bba9b18-4g1f
```

While logged into the node, you can try to list all running containers with `docker ps`. You haven't run any applications yet, so you'll only see Kubernetes system containers. What they are isn't important right now, but you'll learn about them in later chapters.

3.1.5 Creating a cluster using Amazon Elastic Kubernetes Service

If you prefer to use Amazon instead of Google to deploy your Kubernetes cluster in the cloud, you can try the Amazon Elastic Kubernetes Service (EKS). Let's go over the basics.

First, you have to install the `eksctl` command-line tool by following the instructions at <https://docs.aws.amazon.com/eks/latest/userguide/getting-started-eksctl.html>.

Creating an EKS Kubernetes cluster

Creating an EKS Kubernetes cluster using `eksctl` does not differ significantly from how you create a cluster in GKE. All you must do is run the following command:

```
$ eksctl create cluster --name kiada --region eu-central-1 --node
```

This command creates a three-node cluster in the eu-central-1 region. The regions are listed at <https://aws.amazon.com/about-aws/global-infrastructure/regional-product-services/>.

Inspecting an EKS worker node

If you're interested in what's running on those nodes, you can use SSH to connect to them. The `--ssh-access` flag used in the command that creates the cluster ensures that your SSH public key is imported to the node.

As with GKE and Minikube, once you've logged into the node, you can try to list all running containers with `docker ps`. You can expect to see similar containers as in the clusters we covered earlier.

3.1.6 Deploying a multi-node cluster from scratch

Until you get a deeper understanding of Kubernetes, I strongly recommend that you don't try to install a multi-node cluster from scratch. If you are an experienced systems administrator, you may be able to do it without much pain and suffering, but most people may want to try one of the methods described in the previous sections first. Proper management of Kubernetes clusters is incredibly difficult. The installation alone is a task not to be underestimated.

If you still feel adventurous, you can start with the instructions in Appendix B, which explain how to create VMs with VirtualBox and install Kubernetes using the `kubeadm` tool. You can also use those instructions to install Kubernetes on your bare-metal machines or in VMs running in the cloud.

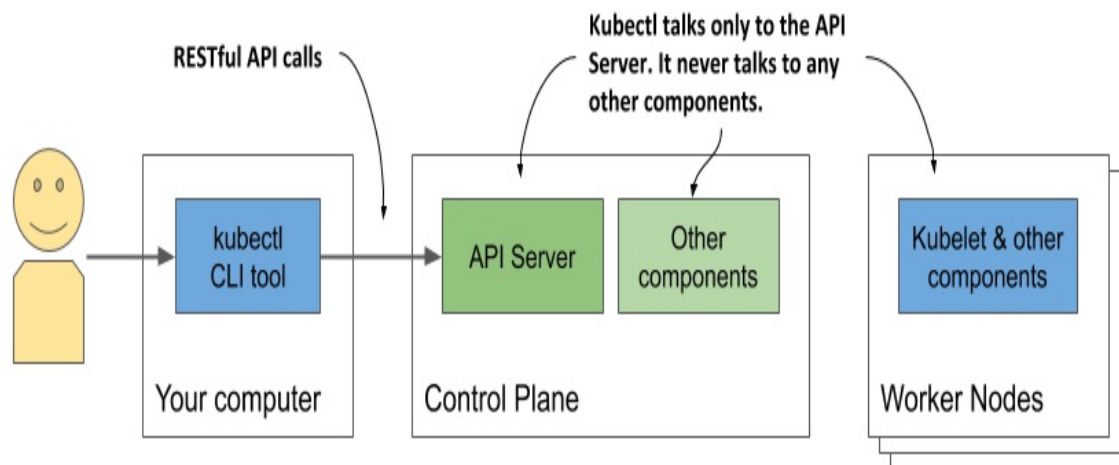
Once you've successfully deployed one or two clusters using `kubeadm`, you can then try to deploy it completely manually, by following Kelsey Hightower's *Kubernetes the Hard Way* tutorial at github.com/kelseyhightower/kubernetes-the-hard-way. Though you may run into several problems, figuring out how to solve them can be a great learning experience.

3.2 Interacting with Kubernetes

You've now learned about several possible methods to deploy a Kubernetes cluster. Now's the time to learn how to use the cluster. To interact with Kubernetes, you use a command-line tool called `kubectl`, pronounced *kube-control*, *kube-C-T-L* or *kube-cuddle*.

As the next figure shows, the tool communicates with the Kubernetes API server, which is part of the Kubernetes Control Plane. The control plane then triggers the other components to do whatever needs to be done based on the changes you made via the API.

Figure 3.6 How you interact with a Kubernetes cluster



3.2.1 Setting up kubectl - the Kubernetes command-line client

Kubectl is a single executable file that you must download to your computer and place into your path. It loads its configuration from a configuration file called *kubeconfig*. To use kubectl, you must both install it and prepare the kubeconfig file so kubectl knows what cluster to talk to.

Downloading and Installing kubectl

The latest stable release for Linux can be downloaded and installed with the following commands:

```
$ curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io)
$ chmod +x kubectl
$ sudo mv kubectl /usr/local/bin/
```

To install `kubectl` on macOS, you can either run the same command, but replace `linux` in the URL with `darwin`, or install the tool via Homebrew by running `brew install kubectl`.

On Windows, download `kubectl.exe` from <https://storage.googleapis.com/kubernetes-release/release/v1.18.2/bin/windows/amd64/kubectl.exe>. To download the latest version, first go to [https://storage.googleapis.com/kubernetes-release/stable.txt](https://storage.googleapis.com/kubernetes-release/release/stable.txt) to see what the latest stable version is and then replace the version number in the first URL with this version. To check if you've installed it correctly, run `kubectl --help`. Note that `kubectl` may or may not yet be configured to talk to your Kubernetes cluster, which means most commands may not work yet.

Tip

You can always append `--help` to any `kubectl` command to get more information.

Setting up a short alias for kubectl

You'll use `kubectl` often. Having to type the full command every time is needlessly time-consuming, but you can speed things up by setting up an alias and tab completion for it.

Most users of Kubernetes use `k` as the alias for `kubectl`. If you haven't used aliases yet, here's how to define it in Linux and macOS. Add the following line to your `~/.bashrc` or equivalent file:

```
alias k=kubectl
```

On Windows, if you use the Command Prompt, define the alias by executing `doskey k=kubectl %*`. If you use PowerShell, execute `set-alias -name k -value kubectl`.

Note

You may not need an alias if you used `gcloud` to set up the cluster. It installs

the `k` binary in addition to `kubectl`.

Configuring tab completion for `kubectl`

Even with a short alias like `k`, you'll still have to type a lot. Fortunately, the `kubectl` command can also output shell completion code for both the `bash` and the `zsh` shell. It enables tab completion of not only command names but also the object names. For example, later you'll learn how to view details of a particular cluster node by executing the following command:

```
$ kubectl describe node gke-kiada-default-pool-9bba9b18-4glf
```

That's a lot of typing that you'll repeat all the time. With tab completion, things are much easier. You just press `TAB` after typing the first few characters of each token:

```
$ kubectl desc<TAB> no<TAB> gke-ku<TAB>
```

To enable tab completion in `bash`, you must first install a package called `bash-completion` and then run the following command (you can also add it to `~/.bashrc` or equivalent):

```
$ source <(kubectl completion bash)
```

Note

This enables completion in `bash`. You can also run this command with a different shell. At the time of writing, the available options are `bash`, `zsh`, `fish`, and `powershell`.

However, this will only complete your commands when you use the full `kubectl` command name. It won't work when you use the `k` alias. To enable completion for the alias, you must run the following command:

```
$ complete -o default -F __start_kubectl k
```

3.2.2 Configuring `kubectl` to use a specific Kubernetes cluster

The kubeconfig configuration file is located at `~/.kube/config`. If you deployed your cluster using Docker Desktop, Minikube or GKE, the file was created for you. If you've been given access to an existing cluster, you should have received the file. Other tools, such as `kind`, may have written the file to a different location. Instead of moving the file to the default location, you can also point `kubectl` to it by setting the `KUBECONFIG` environment variable as follows:

```
$ export KUBECONFIG=/path/to/custom/kubeconfig
```

To learn more about how to manage `kubectl`'s configuration and create a config file from scratch, refer to appendix A.

Note

If you want to use several Kubernetes clusters (for example, both Minikube and GKE), see appendix A for information on switching between different `kubectl` contexts.

3.2.3 Using kubectl

Assuming you've installed and configured `kubectl`, you can now use it to talk to your cluster.

Verifying if the cluster is up and kubectl can talk to it

To verify that your cluster is working, use the `kubectl cluster-info` command:

```
$ kubectl cluster-info
Kubernetes master is running at https://192.168.99.101:8443
KubeDNS is running at https://192.168.99.101:8443/api/v1/namespac
```

This indicates that the API server is active and responding to requests. The output lists the URLs of the various Kubernetes cluster services running in your cluster. The above example shows that besides the API server, the KubeDNS service, which provides domain-name services within the cluster, is another service that runs in the cluster.

Listing cluster nodes

Now use the `kubectl get nodes` command to list all nodes in your cluster. Here's the output that is generated when you run the command in a cluster provisioned by kind:

```
$ kubectl get nodes
NAME             STATUS    ROLES    AGE   VERSION
control-plane    Ready    <none>   12m   v1.18.2
kind-worker      Ready    <none>   12m   v1.18.2
kind-worker2     Ready    <none>   12m   v1.18.2
```

Everything in Kubernetes is represented by an object and can be retrieved and manipulated via the RESTful API. The `kubectl get` command retrieves a list of objects of the specified type from the API. You'll use this command all the time, but it only displays summary information about the listed objects.

Retrieving additional details of an object

To see more detailed information about an object, you use the `kubectl describe` command, which shows much more:

```
$ kubectl describe node gke-kiada-85f6-node-0rrx
```

I omit the actual output of the `describe` command because it's quite wide and would be completely unreadable here in the book. If you run the command yourself, you'll see that it displays the status of the node, information about its CPU and memory usage, system information, containers running on the node, and much more.

If you run the `kubectl describe` command without specifying the resource name, information of all nodes will be printed.

Tip

Executing the `describe` command without specifying the object name is useful when only one object of a certain type exists. You don't have to type or copy/paste the object name.

You'll learn more about the numerous other `kubectl` commands throughout the book.

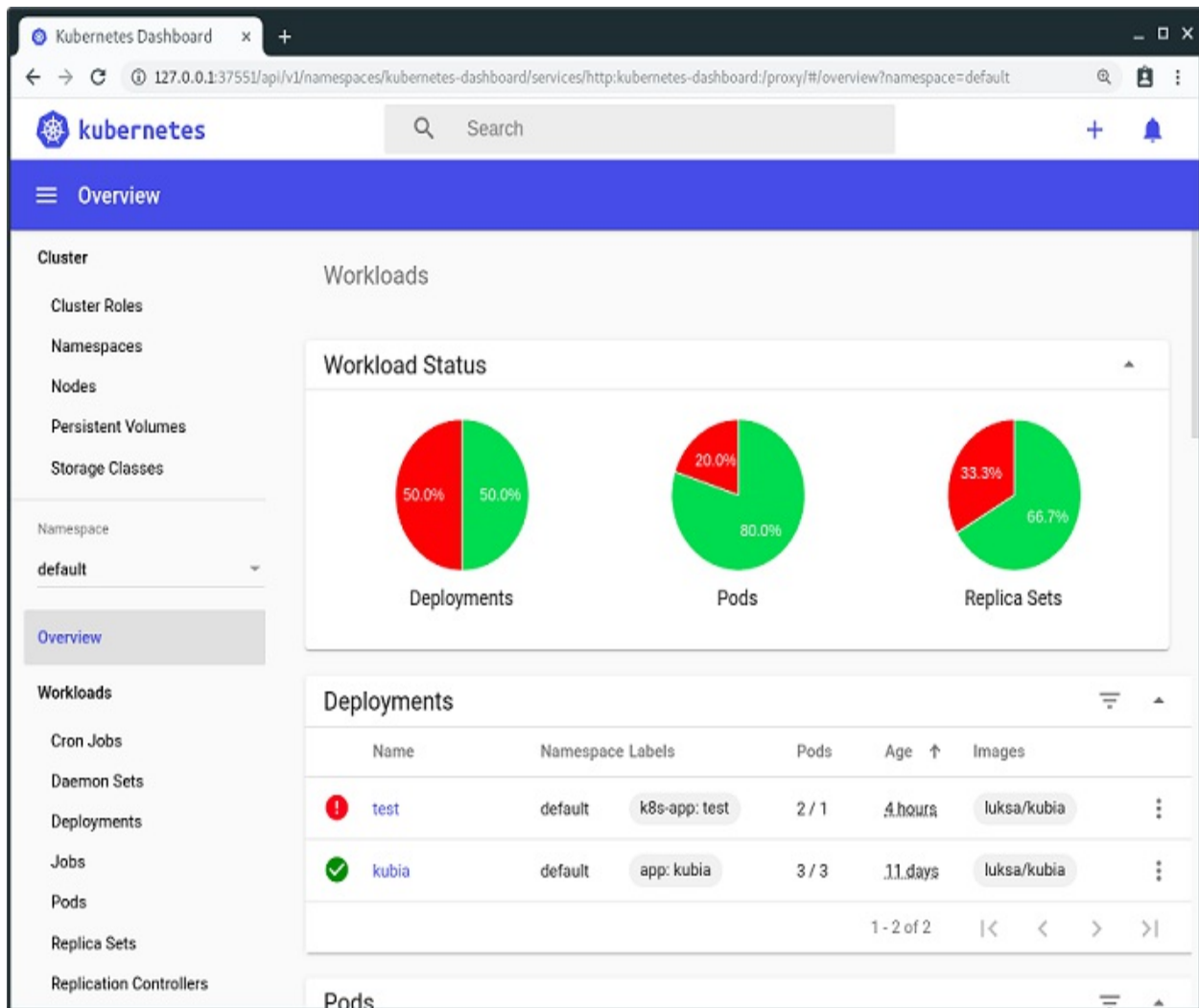
3.2.4 Interacting with Kubernetes through web dashboards

If you prefer using graphical web user interfaces, you'll be happy to hear that Kubernetes also comes with a nice web dashboard. Note, however, that the functionality of the dashboard may lag significantly behind `kubectl`, which is the primary tool for interacting with Kubernetes.

Nevertheless, the dashboard shows different resources in context and can be a good start to get a feel for what the main resource types in Kubernetes are and how they relate to each other. The dashboard also offers the possibility to modify the deployed objects and displays the equivalent `kubectl` command for each action - a feature most beginners will appreciate.

Figure 3.7 shows the dashboard with two workloads deployed in the cluster.

Figure 3.7 Screenshot of the Kubernetes web-based dashboard



Although you won't use the dashboard in this book, you can always open it to quickly see a graphical view of the objects deployed in your cluster after you create them via `kubectl`.

Accessing the dashboard in Docker Desktop

Unfortunately, Docker Desktop does not install the Kubernetes dashboard by default. Accessing it is also not trivial, but here's how. First, you need to install it using the following command:

```
$ kubectl apply -f https://raw.githubusercontent.com/kubernetes/d
```

Refer to github.com/kubernetes/dashboard to find the latest version number. After installing the dashboard, the next command you must run is:

```
$ kubectl proxy
```

This command runs a local proxy to the API server, allowing you to access the services through it. Let the proxy process run and use the browser to open the dashboard at the following URL:

<http://localhost:8001/api/v1/namespaces/kubernetes-dashboard/services/https:kubernetes-dashboard:/proxy/>

You'll be presented with an authentication page. You must then run the following command to retrieve an authentication token.

```
PS C:\> kubectl -n kubernetes-dashboard describe secret $(kubectl
```

Note

This command must be run in Windows PowerShell.

Find the token listed under `kubernetes-dashboard-token-xyz` and paste it into the token field on the authentication page shown in your browser. After you do this, you should be able to use the dashboard. When you're finished using it, terminate the `kubectl proxy` process using `Control-C`.

Accessing the dashboard when using Minikube

If you're using Minikube, accessing the dashboard is much easier. Run the following command and the dashboard will open in your default browser:

```
$ minikube dashboard
```

Accessing the dashboard when running Kubernetes elsewhere

The Google Kubernetes Engine no longer provides access to the open source Kubernetes Dashboard, but it offers an alternative web-based console. The same applies to other cloud providers. For information on how to access the dashboard, please refer to the documentation of the respective provider.

If your cluster runs on your own infrastructure, you can deploy the dashboard

by following the guide at kubernetes.io/docs/tasks/access-application-cluster/web-ui-dashboard.

3.3 Running your first application on Kubernetes

Now is the time to finally deploy something to your cluster. Usually, to deploy an application, you'd prepare a JSON or YAML file describing all the components that your application consists of and apply that file to your cluster. This would be the declarative approach.

Since this may be your first time deploying an application to Kubernetes, let's choose an easier way to do this. We'll use simple, one-line imperative commands to deploy your application.

3.3.1 Deploying your application

The imperative way to deploy an application is to use the `kubectl create deployment` command. As the command itself suggests, it creates a *Deployment* object, which represents an application deployed in the cluster. By using the imperative command, you avoid the need to know the structure of Deployment objects as when you write YAML or JSON manifests.

Creating a Deployment

In the previous chapter, you created a Node.js application called Kiada that you packaged into a container image and pushed to Docker Hub to make it easily distributable to any computer.

Note

If you skipped chapter two because you are already familiar with Docker and containers, you might want to go back and read section 2.2.1 that describes the application that you'll deploy here and in the rest of this book.

Let's deploy the Kiada application to your Kubernetes cluster. Here's the command that does this:

```
$ kubectl create deployment kiada --image=luksa/kiada:0.1
deployment.apps/kiada created
```

In the command, you specify three things:

- You want to create a deployment object.
- You want the object to be called `kiada`.
- You want the deployment to use the container image `luksa/kiada:0.1`.

By default, the image is pulled from Docker Hub, but you can also specify the image registry in the image name (for example, `quay.io/luksa/kiada:0.1`).

Note

Make sure that the image is stored in a public registry and can be pulled without access authorization. You'll learn how to provide credentials for pulling private images in chapter 8.

The Deployment object is now stored in the Kubernetes API. The existence of this object tells Kubernetes that the `luksa/kiada:0.1` container must run in your cluster. You've stated your *desired* state. Kubernetes must now ensure that the *actual* state reflects your wishes.

Listing deployments

The interaction with Kubernetes consists mainly of the creation and manipulation of objects via its API. Kubernetes stores these objects and then performs operations to bring them to life. For example, when you create a Deployment object, Kubernetes runs an application. Kubernetes then keeps you informed about the current state of the application by writing the status to the same Deployment object. You can view the status by reading back the object. One way to do this is to list all Deployment objects as follows:

```
$ kubectl get deployments
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
kiada     0/1     1             0           6s
```

The `kubectl get deployments` command lists all Deployment objects that

currently exist in the cluster. You have only one Deployment in your cluster. It runs one instance of your application as shown in the UP-TO-DATE column, but the AVAILABLE column indicates that the application is not yet available. That's because the container isn't ready, as shown in the READY column. You can see that zero of a total of one container are ready.

You may wonder if you can ask Kubernetes to list all the running containers by running `kubectl get containers`. Let's try this.

```
$ kubectl get containers
error: the server doesn't have a resource type "containers"
```

The command fails because Kubernetes doesn't have a "Container" object type. This may seem odd, since Kubernetes is all about running containers, but there's a twist. A container is not the smallest unit of deployment in Kubernetes. So, what is?

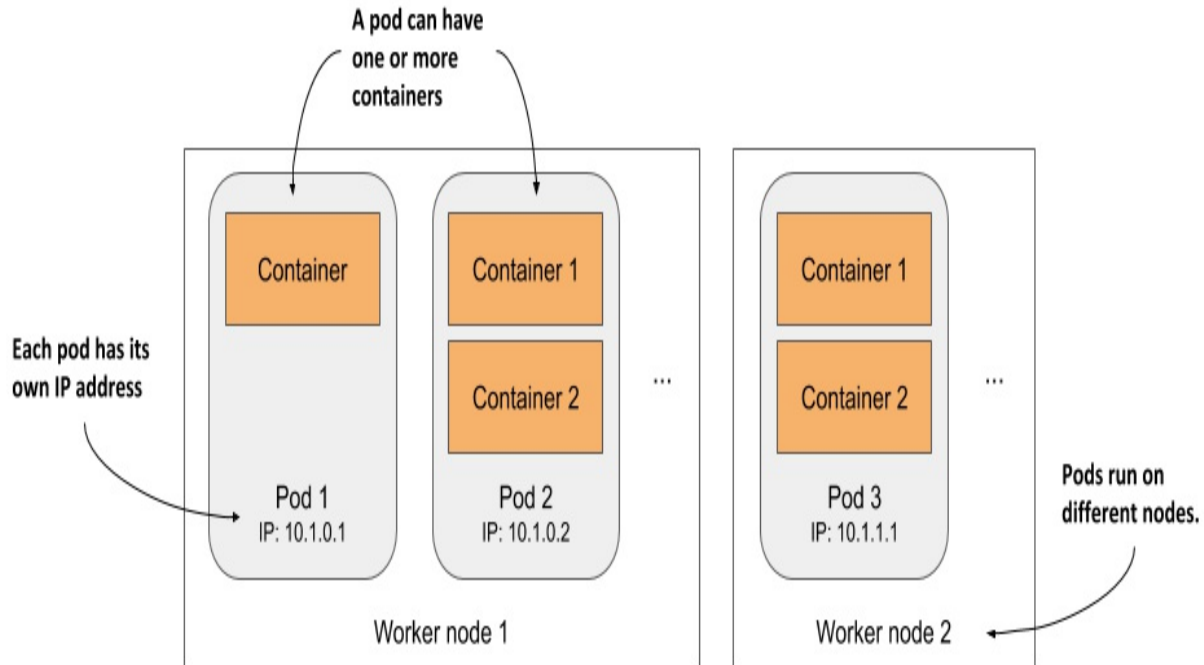
Introducing Pods

In Kubernetes, instead of deploying individual containers, you deploy groups of co-located containers – so-called *Pods*. You know, as in *pod of whales*, or a *pea pod*.

A pod is a group of one or more closely related containers (not unlike peas in a pod) that run together on the same worker node and need to share certain Linux namespaces, so that they can interact more closely than with other pods.

In the previous chapter I showed an example where two processes use the same namespaces. By sharing the network namespace, both processes use the same network interfaces, share the same IP address and port space. By sharing the UTS namespace, both see the same system hostname. This is exactly what happens when you run containers in the same pod. They use the same network and UTS namespaces, as well as others, depending on the pod's spec.

Figure 3.8 The relationship between containers, pods, and worker nodes



As illustrated in figure 3.8, you can think of each pod as a separate logical computer that contains one application. The application can consist of a single process running in a container, or a main application process and additional supporting processes, each running in a separate container. Pods are distributed across all the worker nodes of the cluster.

Each pod has its own IP, hostname, processes, network interfaces and other resources. Containers that are part of the same pod think that they're the only ones running on the computer. They don't see the processes of any other pod, even if located on the same node.

Listing pods

Since containers aren't a top-level Kubernetes object, you can't list them. But you can list pods. As the following output of the `kubectl get pods` command shows, by creating the Deployment object, you've deployed one pod:

```
$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE   #A
kiada-9d785b578-p449x              0/1     Pending   0           1m
```

This is the pod that houses the container running your application. To be precise, since the status is still Pending, the application, or rather the container, isn't running yet. This is also expressed in the READY column, which indicates that the pod has a single container that's not ready.

The reason the pod is pending is because the worker node to which the pod has been assigned must first download the container image before it can run it. When the download is complete, the pod's container is created and the pod enters the Running state.

If Kubernetes can't pull the image from the registry, the `kubectl get pods` command will indicate this in the STATUS column. If you're using your own image, ensure it's marked as public on Docker Hub. Try pulling the image manually with the `docker pull` command on another computer.

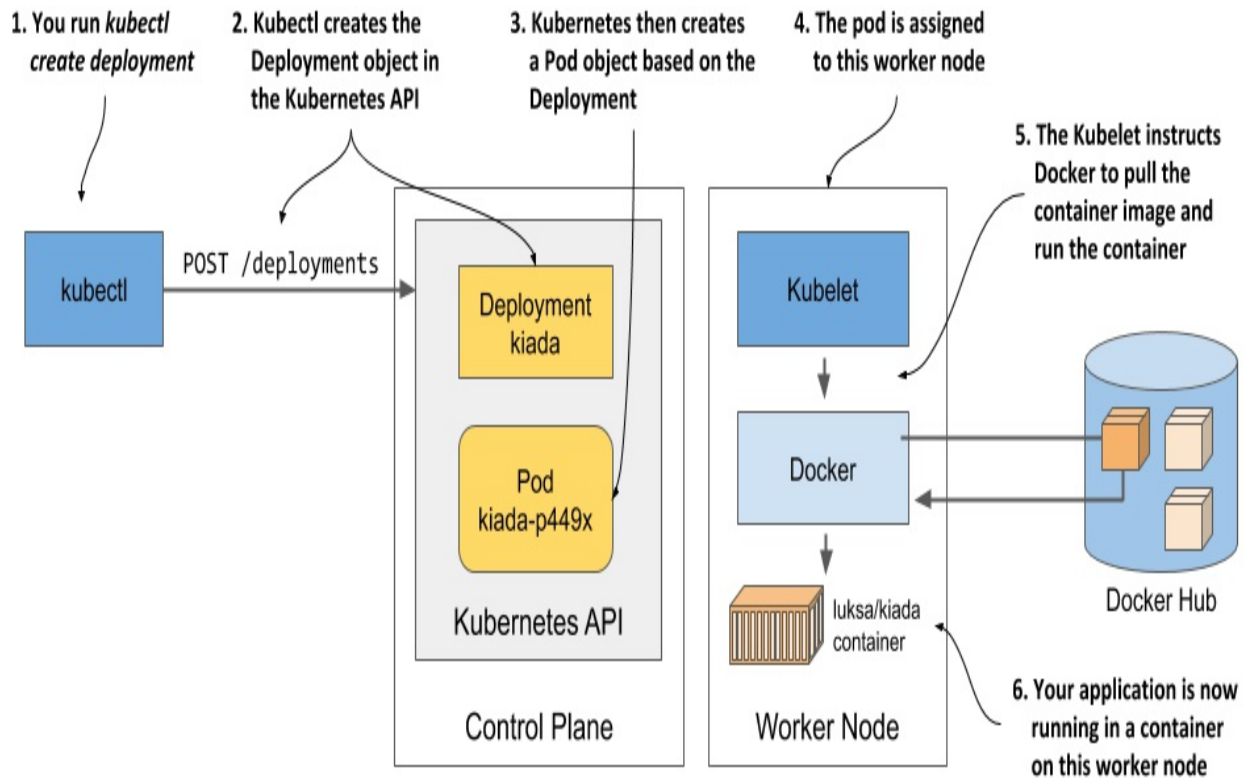
If another issue is causing your pod not to run, or if you simply want to see more information about the pod, you can also use the `kubectl describe pod` command, as you did earlier to see the details of a worker node. If there are any issues with the pod, they should be displayed by this command. Look at the events shown at the bottom of its output. For a running pod, they should be close the following:

Type	Reason	Age	From	Message
----	-----	----	----	-----
Normal	Scheduled	25s	default-scheduler	Successfully ass default/kiada-9d to kind-worker2
Normal	Pulling	23s	kubelet, kind-worker2	Pulling image "l
Normal	Pulled	21s	kubelet, kind-worker2	Successfully pul
Normal	Created	21s	kubelet, kind-worker2	Created containe
Normal	Started	21s	kubelet, kind-worker2	Started containe

Understanding what happens behind the scenes

To help you visualize what happened when you created the Deployment, see figure 3.9.

Figure 3.9 How creating a Deployment object results in a running application container



When you ran the `kubectl create` command, it created a new Deployment object in the cluster by sending an HTTP request to the Kubernetes API server. Kubernetes then created a new Pod object, which was then assigned or *scheduled* to one of the worker nodes. The Kubernetes agent on the worker node (the Kubelet) became aware of the newly created Pod object, saw that it was scheduled to its node, and instructed Docker to pull the specified image from the registry, create a container from the image, and execute it.

DEFInItiON

The term scheduling refers to the assignment of the pod to a node. The pod runs immediately, not at some point in the future. Just like how the CPU scheduler in an operating system selects what CPU to run a process on, the scheduler in Kubernetes decides what worker node should execute each container. Unlike an OS process, once a pod is assigned to a node, it runs only on that node. Even if it fails, this instance of the pod is never moved to other nodes, as is the case with CPU processes, but a new pod instance may be created to replace it.

Depending on what you use to run your Kubernetes cluster, the number of worker nodes in your cluster may vary. The figure shows only the worker node that the pod was scheduled to. In a multi-node cluster, none of the other worker nodes are involved in the process.

3.3.2 Exposing your application to the world

Your application is now running, so the next question to answer is how to access it. I mentioned that each pod gets its own IP address, but this address is internal to the cluster and not accessible from the outside. To make the pod accessible externally, you'll *expose* it by creating a Service object.

Several types of Service objects exist. You decide what type you need. Some expose pods only within the cluster, while others expose them externally. A service with the type `LoadBalancer` provisions an external load balancer, which makes the service accessible via a public IP. This is the type of service you'll create now.

Creating a Service

The easiest way to create the service is to use the following imperative command:

```
$ kubectl expose deployment kiada --type=LoadBalancer --port 8080
service/kiada exposed
```

The `create deployment` command that you ran previously created a Deployment object, whereas the `expose deployment` command creates a Service object. This is what running the above command tells Kubernetes:

- You want to expose all pods that belong to the `kiada` Deployment as a new service.
- You want the pods to be accessible from outside the cluster via a load balancer.
- The application listens on port 8080, so you want to access it via that port.

You didn't specify a name for the Service object, so it inherits the name of

the Deployment.

Listing services

Services are API objects, just like Pods, Deployments, Nodes and virtually everything else in Kubernetes, so you can list them by executing `kubectl get services`:

```
$ kubectl get svc
NAME                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)
kubernetes          ClusterIP           10.19.240.1     <none>           443/TCP
kiada               LoadBalancer       10.19.243.17    <pending>        8080:3083
```

Note

Notice the use of the abbreviation `svc` instead of `services`. Most resource types have a short name that you can use instead of the full object type (for example, `po` is short for pods, `no` for nodes and `deploy` for deployments).

The list shows two services with their types, IPs and the ports they expose. Ignore the `kubernetes` service for now and take a close look at the `kiada` service. It doesn't yet have an external IP address. Whether it gets one depends on how you've deployed the cluster.

Listing the available object types with `kubectl api-resources`

You've used the `kubectl get` command to list various things in your cluster: Nodes, Deployments, Pods and now Services. These are all Kubernetes object types. You can display a list of all supported types by running `kubectl api-resources`. The list also shows the short name for each type and some other information you need to define objects in JSON/YAML files, which you'll learn in the following chapters.

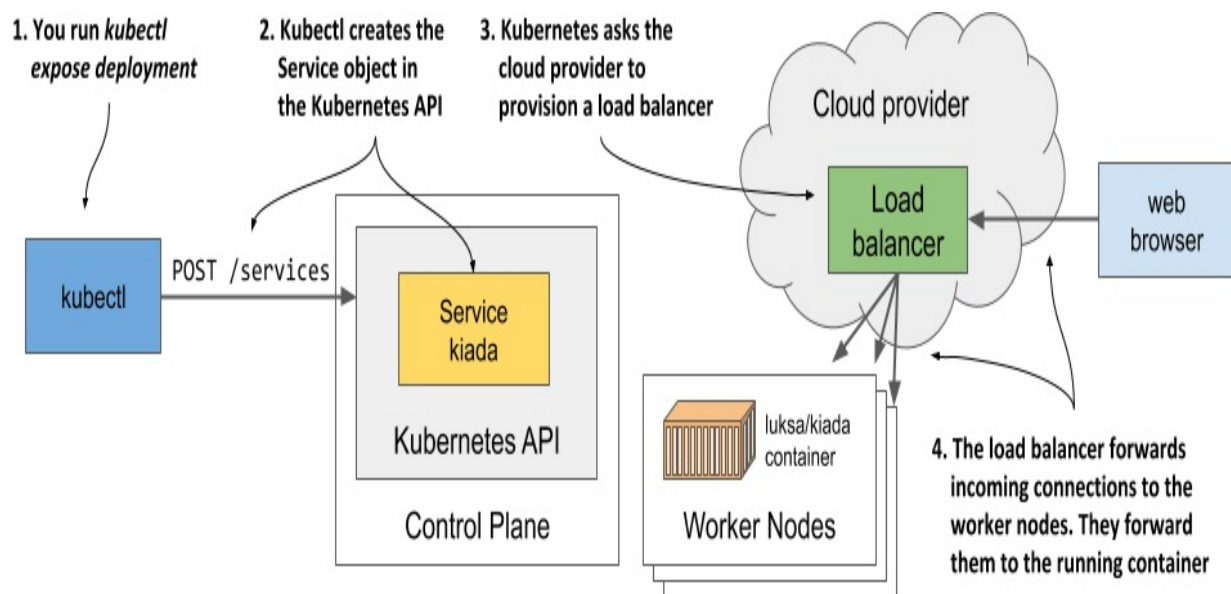
Understanding load balancer services

While Kubernetes allows you to create so-called LoadBalancer services, it doesn't provide the load balancer itself. If your cluster is deployed in the cloud, Kubernetes can ask the cloud infrastructure to provision a load

balancer and configure it to forward traffic into your cluster. The infrastructure tells Kubernetes the IP address of the load balancer and this becomes the external address of your service.

The process of creating the Service object, provisioning the load balancer and how it forwards connections into the cluster is shown in the next figure.

Figure 3.10 What happens when you create a Service object of type LoadBalancer



Provisioning of the load balancer takes some time, so let's wait a few more seconds and check again whether the IP address is already assigned. This time, instead of listing all services, you'll display only the kiada service as follows:

```
$ kubectl get svc kiada
NAME         TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)
kiada        LoadBalancer  10.19.243.17  35.246.179.22  8080:30838
```

The external IP is now displayed. This means that the load balancer is ready to forward requests to your application for clients around the world.

Note

If you deployed your cluster with Docker Desktop, the load balancer's IP

address is shown as localhost, referring to your Windows or macOS machine, not the VM where Kubernetes and the application runs. If you use Minikube to create the cluster, no load balancer is created, but you can access the service in another way. More on this later.

Accessing your application through the load balancer

You can now send requests to your application through the external IP and port of the service:

```
$ curl 35.246.179.22:8080
Kiada version 0.1. Request processed by "kiada-9d785b578-p449x".
```

Note

If you use Docker Desktop, the service is available at localhost:8080 from within your host operating system. Use curl or your browser to access it.

Congratulations! If you use Google Kubernetes Engine, you've successfully published your application to users across the globe. Anyone who knows its IP and port can now access it. If you don't count the steps needed to deploy the cluster itself, only two simple commands were needed to deploy your application:

- `kubectl create deployment` and
- `kubectl expose deployment`.

Accessing your application when a load balancer isn't available

Not all Kubernetes clusters have mechanisms to provide a load balancer. The cluster provided by Minikube is one of them. If you create a service of type LoadBalancer, the service itself works, but there is no load balancer. Kubectl always shows the external IP as <pending> and you must use a different method to access the service.

Several methods of accessing services exist. You can even bypass the service and access individual pods directly, but this is mostly used for

troubleshooting. You'll learn how to do this in chapter 5. For now, let's explore the next easier way to access your service if no load balancer is available.

Minikube can tell you where to access the service if you use the following command:

```
$ minikube service kiada --url  
http://192.168.99.102:30838
```

The command prints out the URL of the service. You can now point `curl` or your browser to that URL to access your application:

```
$ curl http://192.168.99.102:30838  
Kiada version 0.1. Request processed by "kiada-9d785b578-p449x".
```

Tip

If you omit the `--url` option when running the `minikube service` command, your browser opens and loads the service URL.

You may wonder where this IP address and port come from. This is the IP of the Minikube virtual machine. You can confirm this by executing the `minikube ip` command. The Minikube VM is also your single worker node. The port 30838 is the so-called *node port*. It's the port on the worker node that forwards connections to your service. You may have noticed the port in the service's port list when you ran the `kubectl get svc` command:

```
$ kubectl get svc kiada
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
kiada	LoadBalancer	10.19.243.17	<pending>	8080:30838

Your service is accessible via this port number on all your worker nodes, regardless of whether you're using Minikube or any other Kubernetes cluster.

Note

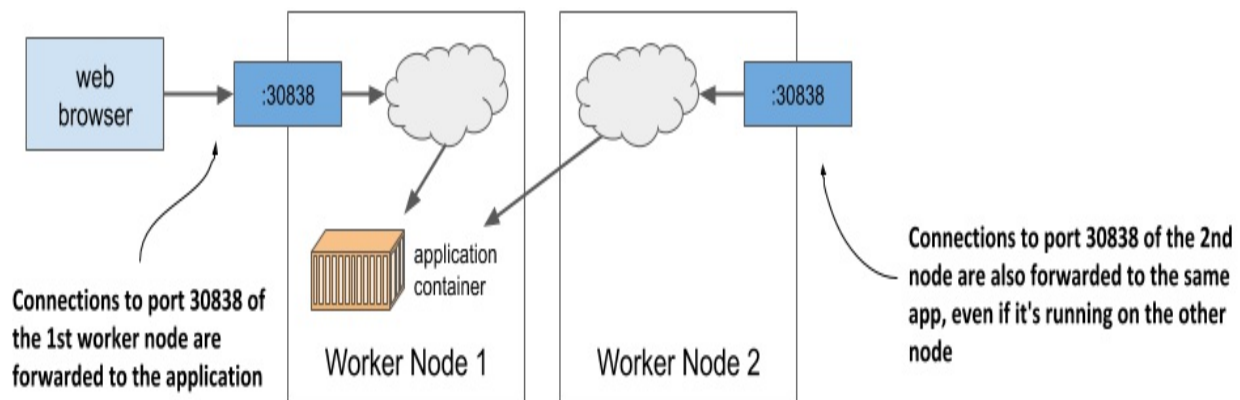
If you use Docker Desktop, the VM running Kubernetes can't be reached from your host OS through the VM's IP. You can access the service through the node port only within the VM by logging into it using the special

container as described in section 3.1.1.

If you know the IP of at least one of your worker nodes, you should be able to access your service through this IP:port combination, provided that firewall rules do not prevent you from accessing the port.

The next figure shows how external clients access the application via the node ports.

Figure 3.11 Connection routing through a service's node port



To connect this to what I mentioned earlier about the load balancer forwarding connections to the nodes and the nodes then forwarding them to the containers: the node ports are exactly where the load balancer sends incoming requests to. Kubernetes then ensures that they are forwarded to the application running in the container. You'll learn how it does this in chapter 10, as we delve deeper into services. Don't lose too much time thinking about it until then. Instead, let's play a little more with our cluster to see what else Kubernetes can do.

3.3.3 Horizontally scaling the application

You now have a running application that is represented by a Deployment and exposed to the world by a Service object. Now let's create some additional magic.

One of the major benefits of running applications in containers is the ease

with which you can scale your application deployments. You're currently running a single instance of your application. Imagine you suddenly see many more users using your application. The single instance can no longer handle the load. You need to run additional instances to distribute the load and provide service to your users. This is known as *scaling out*. With Kubernetes, it's trivial to do.

Increasing the number of running application instances

To deploy your application, you've created a Deployment object. By default, it runs a single instance of your application. To run additional instances, you only need to scale the Deployment object with the following command:

```
$ kubectl scale deployment kiada --replicas=3
deployment.apps/kiada scaled
```

You've now told Kubernetes that you want to run three exact copies or *replicas* of your pod. Note that you haven't instructed Kubernetes what to do. You haven't told it to add two more pods. You just set the new desired number of replicas and let Kubernetes determine what action it must take to reach the new desired state.

This is one of the most fundamental principles in Kubernetes. Instead of telling Kubernetes what to do, you simply set a new desired state of the system and let Kubernetes achieve it. To do this, it examines the current state, compares it with the desired state, identifies the differences and determines what it must do to reconcile them.

Seeing the results of the scale-out

Although it's true that the `kubectl scale deployment` command seems imperative, since it apparently tells Kubernetes to scale your application, what the command actually does is modify the specified Deployment object. As you'll see in a later chapter, you could have simply edited the object instead of giving the imperative command. Let's view the Deployment object again to see how the scale command has affected it:

```
$ kubectl get deploy
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
kiada	3/3	3	3	18m

Three instances are now up to date and available and three of three containers are ready. This isn't clear from the command output, but the three containers are not part of the same pod instance. There are three pods with one container each. You can confirm this by listing pods:

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
kiada-9d785b578-58vhc	1/1	Running	0	17s
kiada-9d785b578-jmnj8	1/1	Running	0	17s
kiada-9d785b578-p449x	1/1	Running	0	18m

As you can see, three pods now exist. As indicated in the READY column, each has a single container, and all the containers are ready. All the pods are Running.

Displaying the pods' host node when listing pods

If you use a single-node cluster, all your pods run on the same node. But in a multi-node cluster, the three pods should be distributed throughout the cluster. To see which nodes the pods were scheduled to, you can use the `-o wide` option to display a more detailed pod list:

```
$ kubectl get pods -o wide
```

NAME	...	IP	NODE	
kiada-9d785b578-58vhc	...	10.244.1.5	kind-worker	#A
kiada-9d785b578-jmnj8	...	10.244.2.4	kind-worker2	#B
kiada-9d785b578-p449x	...	10.244.2.3	kind-worker2	#B

Note

You can also use the `-o wide` output option to see additional information when listing other object types.

The wide output shows that one pod was scheduled to one node, whereas the other two were both scheduled to a different node. The Scheduler usually distributes pods evenly, but it depends on how it's configured. You'll learn more about scheduling in chapter 21.

Does the host node matter?

Regardless of the node they run on, all instances of your application have an identical OS environment, because they run in containers created from the same container image. You may remember from the previous chapter that the only thing that might be different is the OS kernel, but this only happens when different nodes use different kernel versions or load different kernel modules.

In addition, each pod gets its own IP and can communicate in the same way with any other pod - it doesn't matter if the other pod is on the same worker node, another node located in the same server rack or even a completely different data center.

So far, you've set no resource requirements for the pods, but if you had, each pod would have been allocated the requested amount of compute resources. It shouldn't matter to the pod which node provides these resources, as long as the pod's requirements are met.

Therefore, you shouldn't care where a pod is scheduled to. It's also why the default `kubectl get pods` command doesn't display information about the worker nodes for the listed pods. In the world of Kubernetes, it's just not that important.

As you can see, scaling an application is incredibly easy. Once your application is in production and there is a need to scale it, you can add additional instances with a single command without having to manually install, configure and run additional copies.

Note

The app itself must support horizontal scaling. Kubernetes doesn't magically make your app scalable; it merely makes it trivial to replicate it.

Observing requests hitting all three pods when using the service

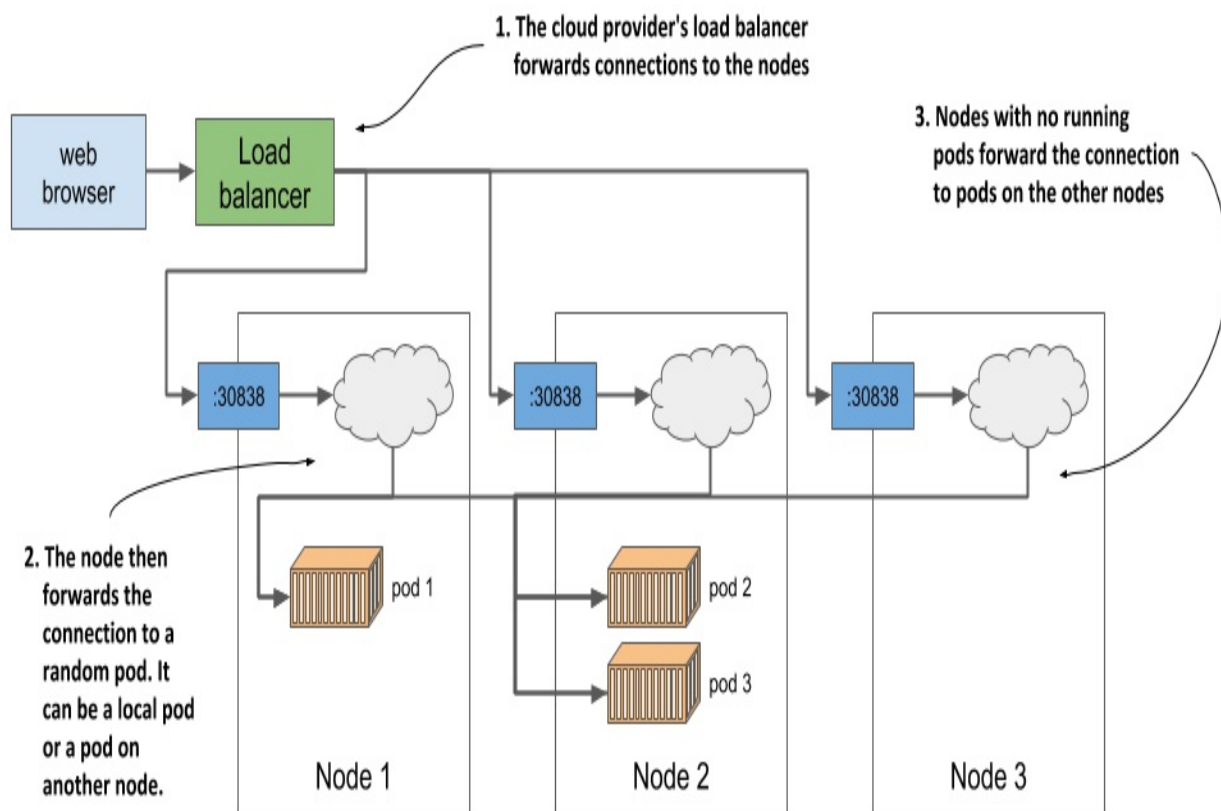
Now that multiple instances of your app are running, let's see what happens when you hit the service URL again. Will the response come from the same

instance every time? Here's the answer:

```
$ curl 35.246.179.22:8080
Kiada version 0.1. Request processed by "kiada-9d785b578-58vhc".
$ curl 35.246.179.22:8080
Kiada version 0.1. Request processed by "kiada-9d785b578-p449x".
$ curl 35.246.179.22:8080
Kiada version 0.1. Request processed by "kiada-9d785b578-jmnj8".
$ curl 35.246.179.22:8080
Kiada version 0.1. Request processed by "kiada-9d785b578-p449x".
```

If you look closely at the responses, you'll see that they correspond to the names of the pods. Each request arrives at a different pod in random order. This is what services in Kubernetes do when more than one pod instance is behind them. They act as load balancers in front of the pods. Let's visualize the system using the following figure.

Figure 3.12 Load balancing across multiple pods backing the same service



As the figure shows, you shouldn't confuse this load balancing mechanism,

which is provided by the Kubernetes service itself, with the additional load balancer provided by the infrastructure when running in GKE or another cluster running in the cloud. Even if you use Minikube and have no external load balancer, your requests are still distributed across the three pods by the service itself. If you use GKE, there are actually *two* load balancers in play. The figure shows that the load balancer provided by the infrastructure distributes requests across the nodes, and the service then distributes requests across the pods.

I know this may be very confusing right now, but it should all become clear in chapter 10.

3.3.4 Understanding the deployed application

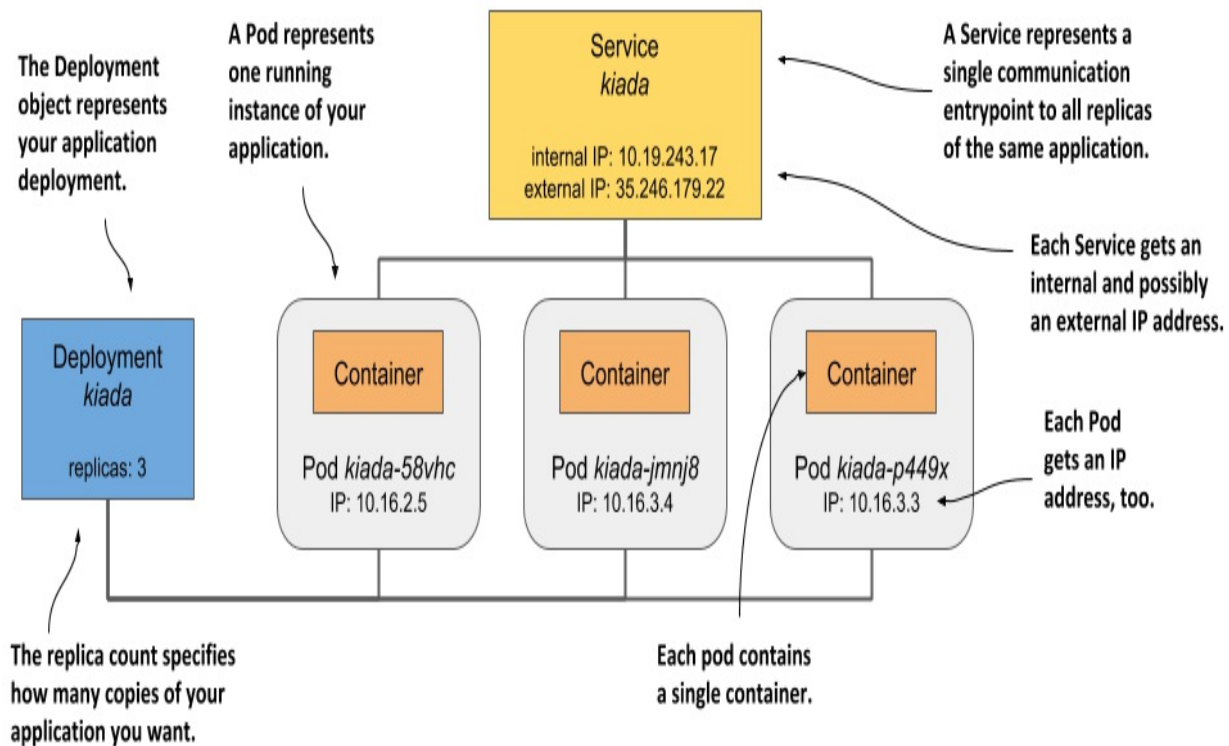
To conclude this chapter, let's review what your system consists of. There are two ways to look at your system – the logical and the physical view. You've just seen the physical view in figure 3.12. There are three running containers that are deployed on three worker nodes (a single node when using Minikube). If you run Kubernetes in the cloud, the cloud infrastructure has also created a load balancer for you. Docker Desktop also creates a type of local load balancer. Minikube doesn't create a load balancer, but you can access your service directly through the node port.

While differences in the physical view of the system in different clusters exist, the logical view is always the same, whether you use a small development cluster or a large production cluster with thousands of nodes. If you're not the one who manages the cluster, you don't even need to worry about the physical view of the cluster. If everything works as expected, the logical view is all you need to worry about. Let's take a closer look at this view.

Understanding the API objects representing your application

The logical view consists of the objects you've created in the Kubernetes API – either directly or indirectly. The following figure shows how the objects relate to each other.

Figure 3.13 Your deployed application consists of a Deployment, several Pods, and a Service.



The objects are as follows:

- the Deployment object you created,
- the Pod objects that were automatically created based on the Deployment, and
- the Service object you created manually.

There are other objects between the three just mentioned, but you don't need to know them yet. You'll learn about them in the following chapters.

Remember when I explained in chapter 1 that Kubernetes abstracts the infrastructure? The logical view of your application is a great example of this. There are no nodes, no complex network topology, no physical load balancers. Just a simple view that only contains your applications and the supporting objects. Let's look at how these objects fit together and what role they play in your small setup.

The Deployment object represents an application deployment. It specifies

which container image contains your application and how many replicas of the application Kubernetes should run. Each replica is represented by a Pod object. The Service object represents a single communication entry point to these replicas.

Understanding the pods

The essential and most important part of your system are the pods. Each pod definition contains one or more containers that make up the pod. When Kubernetes brings a pod to life, it runs all the containers specified in its definition. As long as a Pod object exists, Kubernetes will do its best to ensure that its containers keep running. It only shuts them down when the Pod object is deleted.

Understanding the role of the Deployment

When you first created the Deployment object, only a single Pod object was created. But when you increased the desired number of replicas on the Deployment, Kubernetes created additional replicas. Kubernetes ensures that the actual number of pods always matches the desired number.

If one or more pods disappear or their status is unknown, Kubernetes replaces them to bring the actual number of pods back to the desired number of replicas. A pod disappears when someone or something deletes it, whereas a pod's status is unknown when the node it is running on no longer reports its status due to a network or node failure.

Strictly speaking, a Deployment results in nothing more than the creation of a certain number of Pod objects. You may wonder if you can create Pods directly instead of having the Deployment create them for you. You can certainly do this, but if you wanted to run multiple replicas, you'd have to manually create each pod individually and make sure you give each one a unique name. You'd then also have to keep a constant eye on your pods to replace them if they suddenly disappear or the node on which they run fails. And that's exactly why you almost never create pods directly but use a Deployment instead.

Understanding why you need a service

The third component of your system is the Service object. By creating it, you tell Kubernetes that you need a single communication entry point to your pods. The service gives you a single IP address to talk to your pods, regardless of how many replicas are currently deployed. If the service is backed by multiple pods, it acts as a load balancer. But even if there is only one pod, you still want to expose it through a service. To understand why, you need to learn an important detail about pods.

Pods are ephemeral. A pod may disappear at any time. This can happen when its host node fails, when someone inadvertently deletes the pod, or when the pod is evicted from an otherwise healthy node to make room for other, more important pods. As explained in the previous section, when pods are created through a Deployment, a missing pod is immediately replaced with a new one. This new pod is not the same as the one it replaces. It's a completely new pod, with a new IP address.

If you weren't using a service and had configured your clients to connect directly to the IP of the original pod, you would now need to reconfigure all these clients to connect to the IP of the new pod. This is not necessary when using a service. Unlike pods, services aren't ephemeral. When you create a service, it is assigned a static IP address that never changes during lifetime of the service.

Instead of connecting directly to the pod, clients should connect to the IP of the service. This ensures that their connections are always routed to a healthy pod, even if the set of pods behind the service is constantly changing. It also ensures that the load is distributed evenly across all pods should you decide to scale the deployment horizontally.

3.4 Summary

In this hands-on chapter, you've learned:

- Virtually all cloud providers offer a managed Kubernetes option. They take on the burden of maintaining your Kubernetes cluster, while you

just use its API to deploy your applications.

- You can also install Kubernetes in the cloud yourself, but this has often proven not to be the best idea until you master all aspects of managing Kubernetes.
- You can install Kubernetes locally, even on your laptop, using tools such as Docker Desktop or Minikube, which run Kubernetes in a Linux VM, or kind, which runs the master and worker nodes as Docker containers and the application containers inside those containers.
- Kubectl, the command-line tool, is the usual way you interact with Kubernetes. A web-based dashboard also exists but is not as stable and up to date as the CLI tool.
- To work faster with kubectl, it is useful to define a short alias for it and enable shell completion.
- An application can be deployed using `kubectl create deployment`. It can then be exposed to clients by running `kubectl expose deployment`. Horizontally scaling the application is trivial: `kubectl scale deployment` instructs Kubernetes to add new replicas or removes existing ones to reach the number of replicas you specify.
- The basic unit of deployment is not a container, but a pod, which can contain one or more related containers.
- Deployments, Services, Pods and Nodes are Kubernetes objects/resources. You can list them with `kubectl get` and inspect them with `kubectl describe`.
- The Deployment object deploys the desired number of Pods, while the Service object makes them accessible under a single, stable IP address.
- Each service provides internal load balancing in the cluster, but if you set the type of service to `LoadBalancer`, Kubernetes will ask the cloud infrastructure it runs in for an additional load balancer to make your application available at a publicly accessible address.

You've now completed your first guided tour around the bay. Now it's time to start learning the ropes, so that you'll be able to sail independently. The next part of the book focuses on the different Kubernetes objects and how/when to use them. You'll start with the most important one – the Pod.

4 Introducing Kubernetes API objects

This chapter covers

- Managing a Kubernetes cluster and the applications it hosts via its API
- Understanding the structure of Kubernetes API objects
- Retrieving and understanding an object's YAML or JSON manifest
- Inspecting the status of cluster nodes via Node objects
- Inspecting cluster events through Event objects

The previous chapter introduced three fundamental objects that make up a deployed application. You created a Deployment object that spawned multiple Pod objects representing individual instances of your application and exposed them to the world by creating a Service object that deployed a load balancer in front of them.

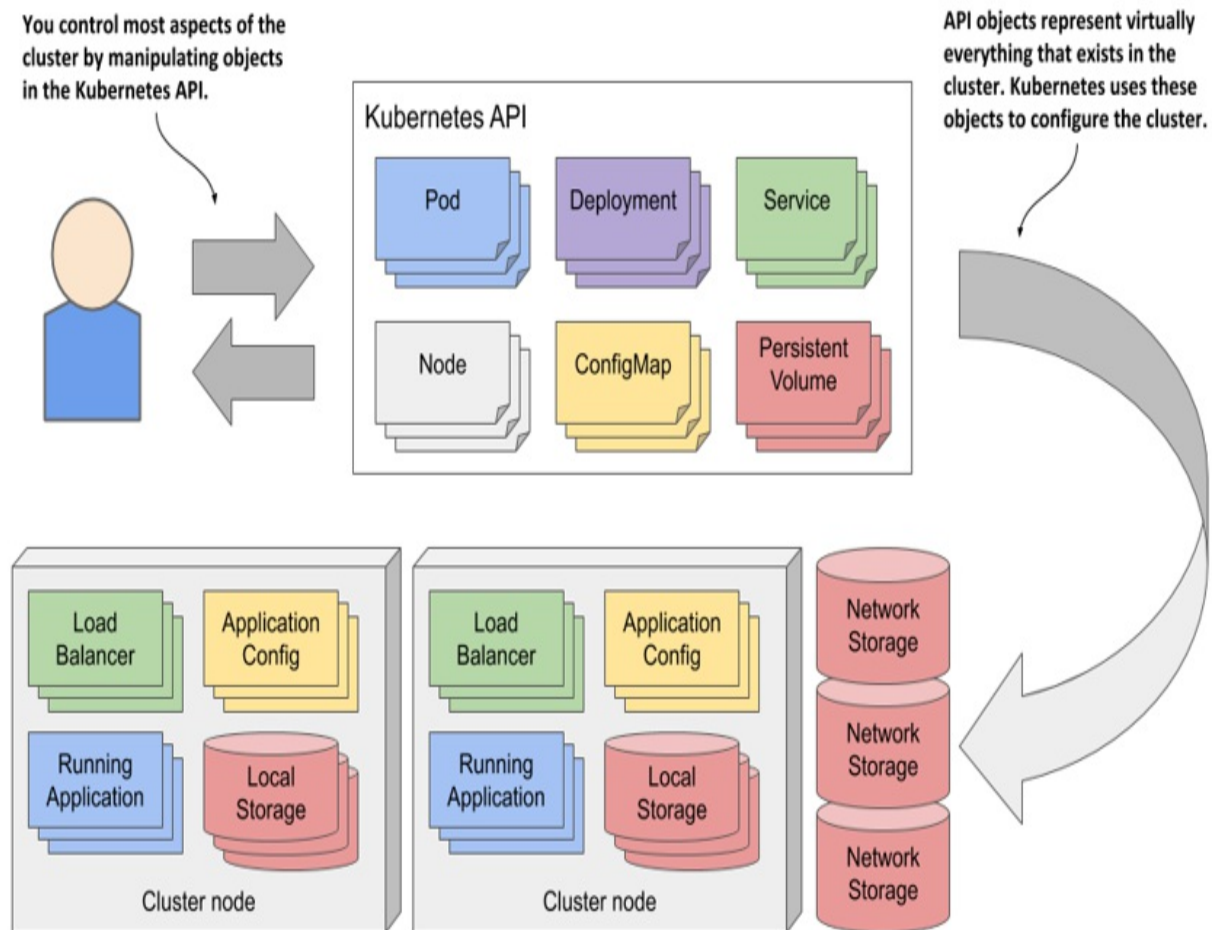
The chapters in the second part of this book explain these and other object types in detail. In this chapter, the common features of Kubernetes objects are presented using the example of Node and Event objects.

4.1 Getting familiar with the Kubernetes API

In a Kubernetes cluster, both users and Kubernetes components interact with the cluster by manipulating objects through the Kubernetes API, as shown in figure 4.1.

These objects represent the configuration of the entire cluster. They include the applications running in the cluster, their configuration, the load balancers through which they are exposed within the cluster or externally, the underlying servers and the storage used by these applications, the security privileges of users and applications, and many other details of the infrastructure.

Figure 4.1 A Kubernetes cluster is configured by manipulating objects in the Kubernetes API



4.1.1 Introducing the API

The Kubernetes API is the central point of interaction with the cluster, so much of this book is dedicated to explaining this API. The most important API objects are described in the following chapters, but a basic introduction to the API is presented here.

Understanding the architectural style of the API

The Kubernetes API is an HTTP-based RESTful API where the state is represented by *resources* on which you perform CRUD operations (Create, Read, Update, Delete) using standard HTTP methods such as POST, GET, PUT/PATCH or DELETE.

Definition

REST is Representational State Transfer, an architectural style for implementing interoperability between computer systems via web services using stateless operations, described by Roy Thomas Fielding in his doctoral dissertation. To learn more, read the dissertation at <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>.

It is these resources (or objects) that represent the configuration of the cluster. Cluster administrators and engineers who deploy applications into the cluster therefore influence the configuration by manipulating these objects.

In the Kubernetes community, the terms “resource” and “object” are used interchangeably, but there are subtle differences that warrant an explanation.

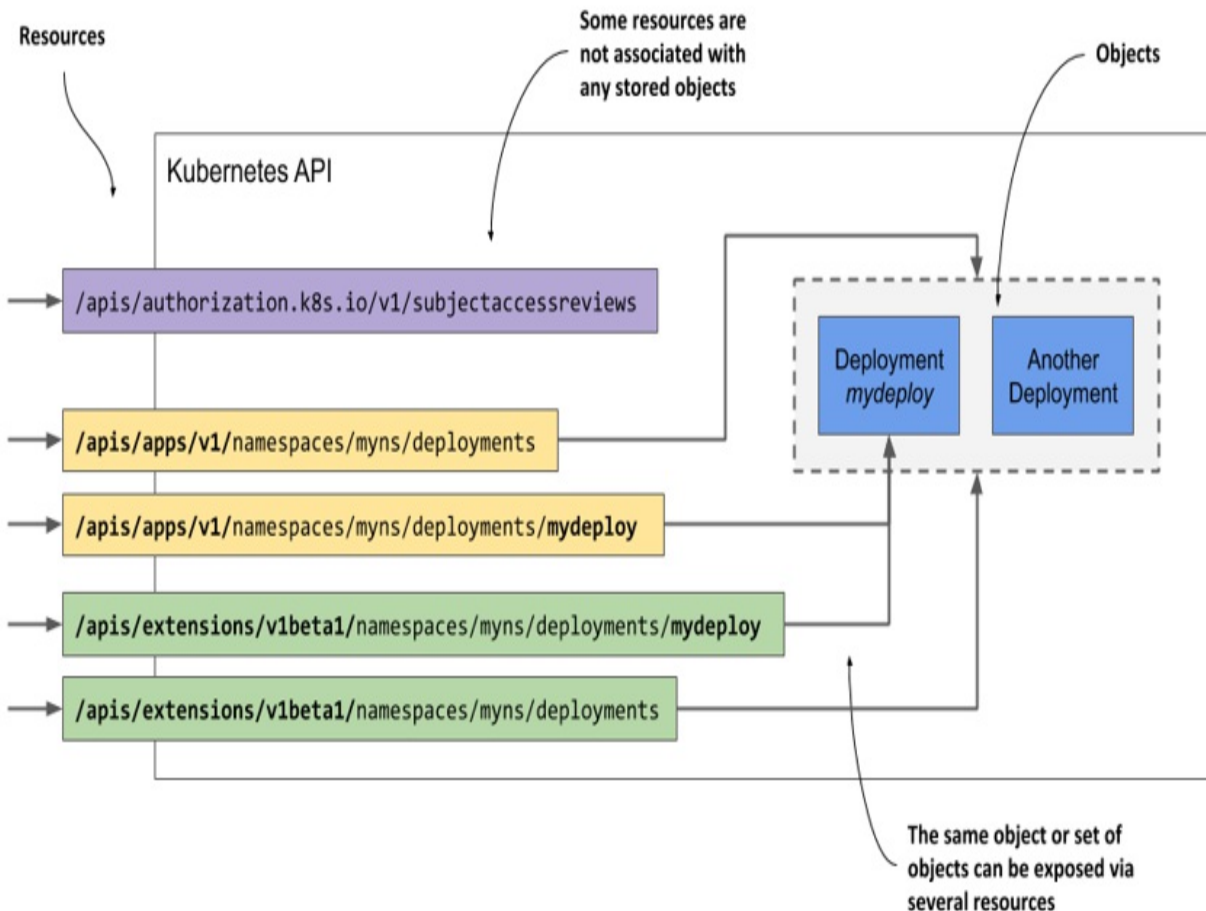
Understanding the difference between resources and objects

The essential concept in RESTful APIs is the resource, and each resource is assigned a URI or Uniform Resource Identifier that uniquely identifies it. For example, in the Kubernetes API, application deployments are represented by deployment resources.

The collection of all deployments in the cluster is a REST resource exposed at `/api/v1/deployments`. When you use the GET method to send an HTTP request to this URI, you receive a response that lists all deployment instances in the cluster.

Each individual deployment instance also has its own unique URI through which it can be manipulated. The individual deployment is thus exposed as another REST resource. You can retrieve information about the deployment by sending a GET request to the resource URI and you can modify it using a PUT request.

Figure 4.2 A single object can be exposed by two or more resources



An object can therefore be exposed through more than one resource. As shown in figure 4.2, the Deployment object instance named `mydeploy` is returned both as an element of a collection when you query the `deployments` resource and as a single object when you query the individual resource URI directly.

In addition, a single object instance can also be exposed via multiple resources if multiple API versions exist for an object type. Up to Kubernetes version 1.15, two different representations of Deployment objects were exposed by the API. In addition to the `apps/v1` version, exposed at `/apis/apps/v1/deployments`, an older version, `extensions/v1beta1`, exposed at `/apis/extensions/v1beta1/deployments` was available in the API. These two resources didn't represent two different sets of Deployment objects, but a single set that was represented in two different ways - with small differences in the object schema. You could create an instance of a Deployment object via the first URI and then read it back using the second.

In some cases, a resource doesn't represent any object at all. An example of this is the way the Kubernetes API allows clients to verify whether a subject (a person or a service) is authorized to perform an API operation. This is done by submitting a POST request to the `/apis/authorization.k8s.io/v1/subjectaccessreviews` resource. The response indicates whether the subject is authorized to perform the operation specified in the request body. The key thing here is that no object is created by the POST request.

The examples described above show that a resource isn't the same as an object. If you are familiar with relational database systems, you can compare resources and object types with views and tables. Resources are views through which you interact with objects.

Note

Because the term “resource” can also refer to compute resources, such as CPU and memory, to reduce confusion, the term “objects” is used in this book to refer to API resources.

Understanding how objects are represented

When you make a GET request for a resource, the Kubernetes API server returns the object in structured text form. The default data model is JSON, but you can also tell the server to return YAML instead. When you update the object using a POST or PUT request, you also specify the new state with either JSON or YAML.

The individual fields in an object's manifest depend on the object type, but the general structure and many fields are shared by all Kubernetes API objects. You'll learn about them next.

4.1.2 Understanding the structure of an object manifest

Before you are confronted with the complete manifest of a Kubernetes object, let me first explain its major parts, because this will help you to find your way through the sometimes hundreds of lines it is composed of.

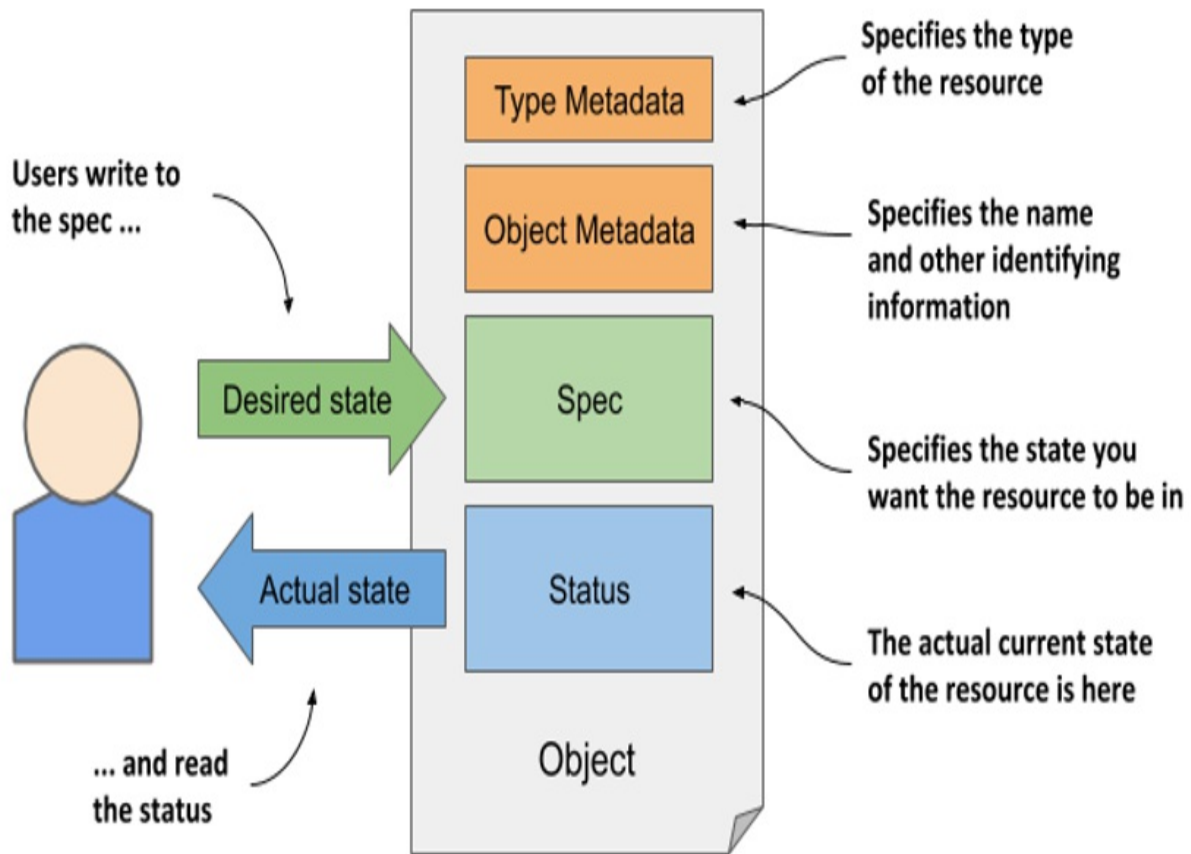
Introducing the main parts of an object

The manifest of most Kubernetes API objects consists of the following four sections:

- *Type Metadata* contains information about the type of object this manifest describes. It specifies the object type, the group to which the type belongs, and the API version.
- *Object Metadata* holds the basic information about the object instance, including its name, time of creation, owner of the object, and other identifying information. The fields in the Object Metadata are the same for all object types.
- *Spec* is the part in which you specify the desired state of the object. Its fields differ between different object types. For pods, this is the part that specifies the pod's containers, storage volumes and other information related to its operation.
- *Status* contains the current actual state of the object. For a pod, it tells you the condition of the pod, the status of each of its containers, its IP address, the node it's running on, and other information that reveals what's happening to your pod.

A visual representation of an object manifest and its four sections is shown in the next figure.

Figure 4.3 The main sections of a Kubernetes API object.



Note

Although the figure shows that users write to the object's Spec section and read its Status, the API server always returns the entire object when you perform a GET request; to update the object, you also send the entire object in the PUT request.

You'll see an example later to see which fields exist in these sections but let me first explain the Spec and Status sections, as they represent the flesh of the object.

Understanding the spec and status sections

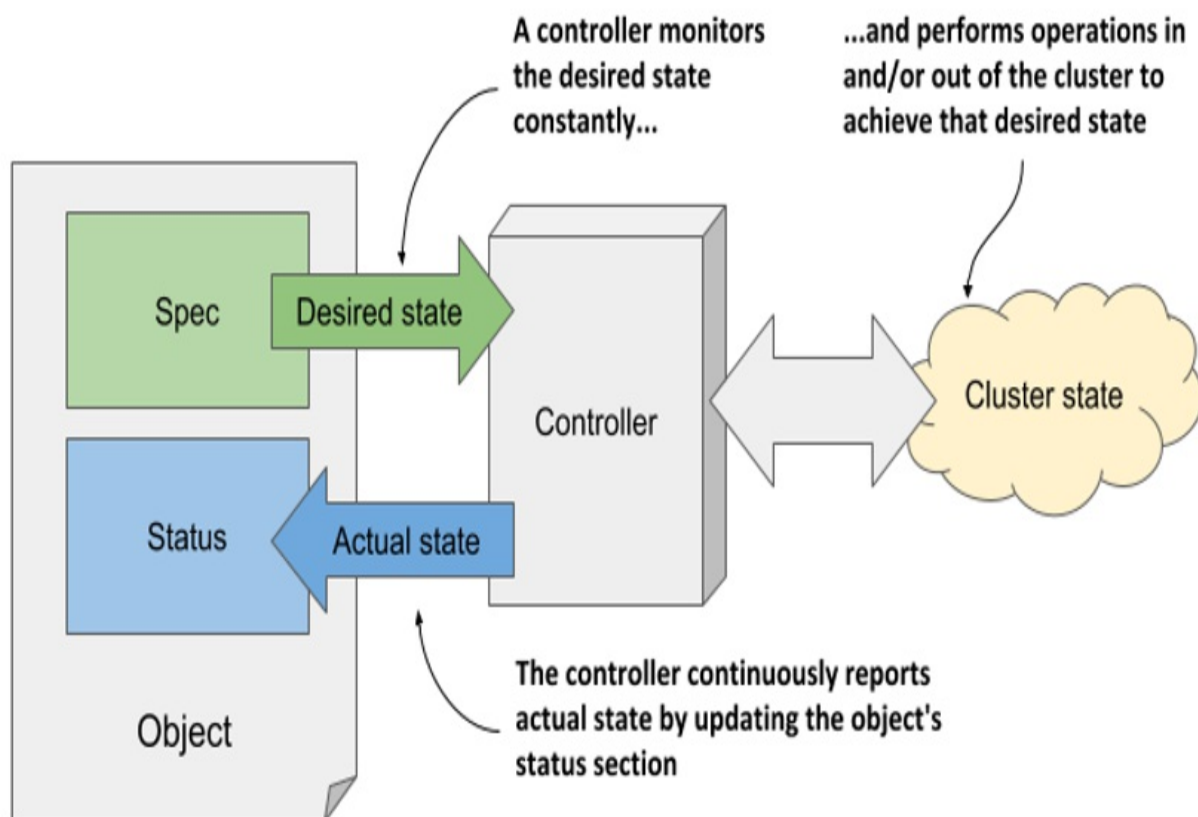
As you may have noticed in the previous figure, the two most important parts of an object are the Spec and Status sections. You use the Spec to specify the desired state of the object and read the actual state of the object from the

Status section. So, you are the one who writes the Spec and reads the Status, but who or what reads the Spec and writes the Status?

The Kubernetes Control Plane runs several components called *controllers* that manage the objects you create. Each controller is usually only responsible for one object type. For example, the *Deployment controller* manages Deployment objects.

As shown in figure 4.4, the task of a controller is to read the desired object state from the object's Spec section, perform the actions required to achieve this state, and report back the actual state of the object by writing to its Status section.

Figure 4.4 How a controller manages an object



Essentially, you tell Kubernetes what it has to do by creating and updating API objects. Kubernetes controllers use the same API objects to tell you what

they have done and what the status of their work is.

You'll learn more about the individual controllers and their responsibilities in chapter 13. For now, just remember that a controller is associated with most object types and that the controller is the thing that reads the Spec and writes the Status of the object.

Not all objects have the spec and status sections

All Kubernetes API objects contain the two metadata sections, but not all have the Spec and Status sections. Those that don't, typically contain just static data and don't have a corresponding controller, so it is not necessary to distinguish between the desired and the actual state of the object.

An example of such an object is the Event object, which is created by various controllers to provide additional information about what is happening with an object that the controller is managing. The Event object is explained in section 4.3.

You now understand the general outline of an object, so the next section of this chapter can finally explore the individual fields of an object.

4.2 Examining an object's individual properties

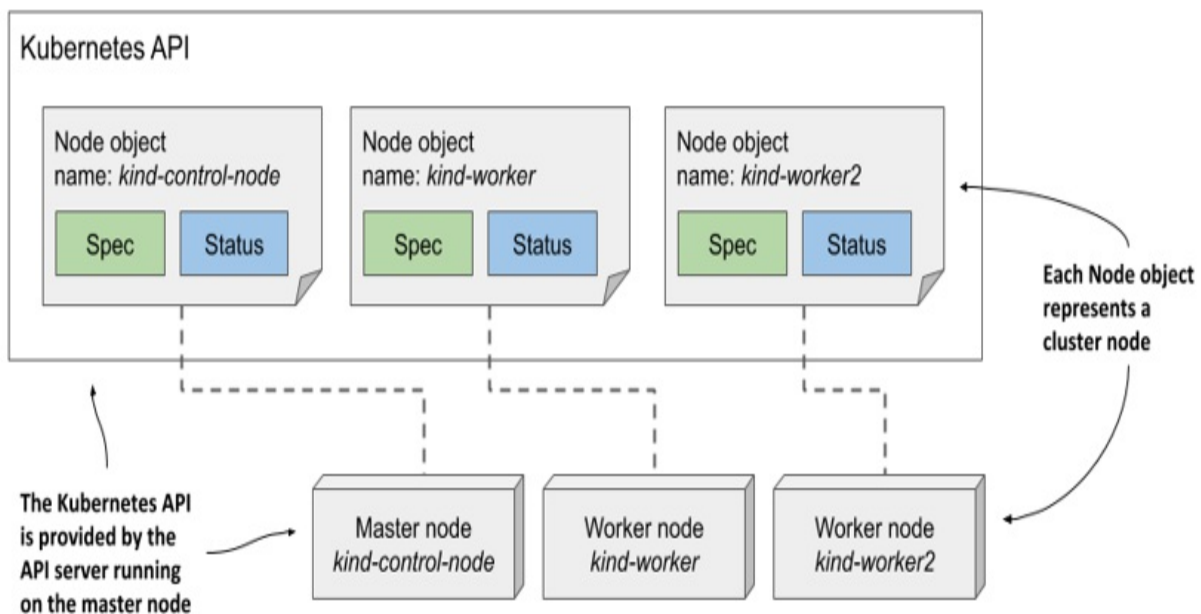
To examine Kubernetes API objects up close, we'll need a concrete example. Let's take the Node object, which should be easy to understand because it represents something you might be relatively familiar with - a computer in the cluster.

My Kubernetes cluster provisioned by the kind tool has three nodes - one master and two workers. They are represented by three Node objects in the API. I can query the API and list these objects using `kubectl get nodes`:

```
$ kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
kind-control-plane  Ready    master   1h    v1.18.2
kind-worker          Ready    <none>   1h    v1.18.2
kind-worker2         Ready    <none>   1h    v1.18.2
```

The following figure shows the three Node objects and the actual cluster machines that make up the cluster. Each Node object instance represents one host. In each instance, the Spec section contains (part of) the configuration of the host, and the Status section contains the state of the host.

Figure 4.5 Cluster nodes are represented by Node objects



Note

Node objects are slightly different from other objects because they are usually created by the Kubelet - the node agent running on the cluster node - rather than by users. When you add a machine to the cluster, the Kubelet registers the node by creating a Node object that represents the host. Users can then edit (some of) the fields in the Spec section.

4.2.1 Exploring the full manifest of a Node object

Let's take a close look at one of the Node objects. List all Node objects in your cluster by running the `kubectl get nodes` command and select one you want to inspect. Then, execute the `kubectl get node <node-name> -o yaml` command, where you replace `<node-name>` with the name of the node, as shown here:

```
$ kubectl get node kind-control-plane -o yaml
apiVersion: v1                #A
kind: Node                    #A
metadata:
  annotations: ...
  creationTimestamp: "2020-05-03T15:09:17Z"
  labels: ...
  name: kind-control-plane
  resourceVersion: "3220054"
  selfLink: /api/v1/nodes/kind-control-plane
  uid: 16dc1e0b-8d34-4cfb-8ade-3b0e91ec838b
spec:
  podCIDR: 10.244.0.0/24
  podCIDRs:
  - 10.244.0.0/24
  taints:
  - effect: NoSchedule
    key: node-role.kubernetes.io/master
status:
  addresses:
  - address: 172.18.0.2
    type: InternalIP
  - address: kind-control-plane
    type: Hostname
  allocatable: ...
  capacity:
    cpu: "8"
    ephemeral-storage: 401520944Ki
    hugepages-1Gi: "0"
    hugepages-2Mi: "0"
    memory: 32720824Ki
    pods: "110"
  conditions:
  - lastHeartbeatTime: "2020-05-17T12:28:41Z"
    lastTransitionTime: "2020-05-03T15:09:17Z"
    message: kubelet has sufficient memory available
    reason: KubeletHasSufficientMemory
    status: "False"
    type: MemoryPressure
    ...
  daemonEndpoints:
    kubeletEndpoint:
      Port: 10250
  images:
  - names:
    - k8s.gcr.io/etcd:3.4.3-0
    sizeBytes: 289997247
    ...
```

```
nodeInfo:
  architecture: amd64
  bootID: 233a359f-5897-4860-863d-06546130e1ff
  containerRuntimeVersion: containerd://1.3.3-14-g449e9269
  kernelVersion: 5.5.10-200.fc31.x86_64
  kubeProxyVersion: v1.18.2
  kubeletVersion: v1.18.2
  machineID: 74b74e389bb246e99abdf731d145142d
  operatingSystem: linux
  osImage: Ubuntu 19.10
  systemUUID: 8749f818-8269-4a02-bdc2-84bf5fa21700
```

Note

Use the `-o json` option to display the object in JSON instead of YAML.

In the YAML manifest, the four main sections of the object definition and the more important properties of the node are annotated to help you distinguish between the more and less important fields. Some lines have been omitted to reduce the length of the manifest.

Accessing the API directly

You may be interested in trying to access the API directly instead of through `kubectl`. As explained earlier, the Kubernetes API is web based, so you can use a web browser or the `curl` command to perform API operations, but the API server uses TLS and you typically need a client certificate or token for authentication. Fortunately, `kubectl` provides a special proxy that takes care of this, allowing you to talk to the API through the proxy using plain HTTP.

To run the proxy, execute the command:

```
$ kubectl proxy
Starting to serve on 127.0.0.1:8001
```

You can now access the API using HTTP at 127.0.0.1:8001. For example, to retrieve the node object, open the URL `http://127.0.0.1:8001/api/v1/nodes/kind-control-plane` (replace `kind-control-plane` with one of your nodes' names).

Now let's take a closer look at the fields in each of the four main sections.

The Type Metadata fields

As you can see, the manifest starts with the `apiVersion` and `kind` fields, which specify the API version and type of the object that this object manifest specifies. The API version is the schema used to describe this object. As mentioned before, an object type can be associated with more than one schema, with different fields in each schema being used to describe the object. However, usually only one schema exists for each type.

The `apiVersion` in the previous manifest is `v1`, but you'll see in the following chapters that the `apiVersion` in other object types contains more than just the version number. For Deployment objects, for example, the `apiVersion` is `apps/v1`. Whereas the field was originally used only to specify the API version, it is now also used to specify the API group to which the resource belongs. Node objects belong to the core API group, which is conventionally omitted from the `apiVersion` field.

The type of object defined in the manifest is specified by the field `kind`. The object kind in the previous manifest is `Node`. In the previous chapters, you created objects of kind `Deployment`, `Service`, and `Pod`.

Fields in the Object Metadata section

The metadata section contains the metadata of this object instance. It contains the name of the instance, along with additional attributes such as labels and annotations, which are explained in chapter 9, and fields such as `resourceVersion`, `managedFields`, and other low-level fields, which are explained at depth in chapter 12.

Fields in the Spec section

Next comes the `spec` section, which is specific to each object kind. It is relatively short for Node objects compared to what you find for other object kinds. The `podCIDR` fields specify the pod IP range assigned to the node. Pods running on this node are assigned IPs from this range. The `taints` field is not important at this point, but you'll learn about it in chapter 18.

Typically, an object's spec section contains many more fields that you use to configure the object.

Fields in the Status section

The status section also differs between the different kinds of object, but its purpose is always the same - it contains the last observed state of the thing the object represents. For Node objects, the status reveals the node's IP address(es), host name, capacity to provide compute resources, the current conditions of the node, the container images it has already downloaded and which are now cached locally, and information about its operating system and the version of Kubernetes components running on it.

4.2.2 Understanding individual object fields

To learn more about individual fields in the manifest, you can refer to the API reference documentation at <http://kubernetes.io/docs/reference/> or use the `kubectl explain` command as described next.

Using `kubectl explain` to explore API object fields

The `kubectl` tool has a nice feature that allows you to look up the explanation of each field for each object type (kind) from the command line. Usually, you start by asking it to provide the basic description of the object kind by running `kubectl explain <kind>`, as shown here:

```
$ kubectl explain nodes
```

```
KIND:      Node
```

```
VERSION:   v1
```

```
DESCRIPTION:
```

```
Node is a worker node in Kubernetes. Each node will have a u  
identifier in the cache (i.e. in etcd).
```

```
FIELDS:
```

```
  apiVersion    <string>
```

```
  APIVersion defines the versioned schema of this representati  
  object. Servers should convert recognized schemas to the lat
```

```

kind <string>
    Kind is a string value representing the REST resource this o
    represents. Servers may infer this from the endpoint the cli

metadata    <Object>
    Standard object's metadata. More info: ...

spec <Object>
    Spec defines the behavior of a node...

status      <Object>
    Most recently observed status of the node. Populated by the
    Read-only. More info: ...

```

The command prints the explanation of the object and lists the top-level fields that the object can contain.

Drilling deeper into an API object's structure

You can then drill deeper to find subfields under each specific field. For example, you can use the following command to explain the node's spec field:

```

$ kubectl explain node.spec
KIND:      Node
VERSION:   v1

RESOURCE: spec <Object>

DESCRIPTION:
    Spec defines the behavior of a node.

    NodeSpec describes the attributes that a node is created wit

FIELDS:
    configSource <Object>
        If specified, the source to get node configuration from The
        DynamicKubeletConfig feature gate must be enabled for the Ku

    externalID   <string>
        Deprecated. Not all kubelets will set this field...

    podCIDR      <string>
        PodCIDR represents the pod IP range assigned to the node.

```

Please note the API version given at the top. As explained earlier, multiple versions of the same kind can exist. Different versions can have different fields or default values. If you want to display a different version, specify it with the `--api-version` option.

Note

If you want to see the complete structure of an object (the complete hierarchical list of fields without the descriptions), try `kubectl explain pods --recursive`.

4.2.3 Understanding an object's status conditions

The set of fields in both the `spec` and `status` sections is different for each object kind, but the `conditions` field is found in many of them. It gives a list of conditions the object is currently in. They are very useful when you need to troubleshoot an object, so let's examine them more closely. Since the `Node` object is used as an example, this section also teaches you how to easily identify problems with a cluster node.

Introducing the node's status conditions

Let's print out the YAML manifest of the one of the node objects again, but this time we'll only focus on the `conditions` field in the object's `status`. The command to run and its output are as follows:

```
$ kubectl get node kind-control-plane -o yaml
...
status:
  ...
  conditions:
  - lastHeartbeatTime: "2020-05-17T13:03:42Z"
    lastTransitionTime: "2020-05-03T15:09:17Z"
    message: kubelet has sufficient memory available
    reason: KubeletHasSufficientMemory
    status: "False"
    type: MemoryPressure
  - lastHeartbeatTime: "2020-05-17T13:03:42Z"
    lastTransitionTime: "2020-05-03T15:09:17Z"
    message: kubelet has no disk pressure
```

```

    reason: KubeletHasNoDiskPressure
    status: "False"                                #B
    type: DiskPressure                             #B
- lastHeartbeatTime: "2020-05-17T13:03:42Z"
  lastTransitionTime: "2020-05-03T15:09:17Z"
  message: kubelet has sufficient PID available
  reason: KubeletHasSufficientPID
  status: "False"                                #C
  type: PIDPressure                             #C
- lastHeartbeatTime: "2020-05-17T13:03:42Z"
  lastTransitionTime: "2020-05-03T15:10:15Z"
  message: kubelet is posting ready status
  reason: KubeletReady
  status: "True"                                #D
  type: Ready                                   #D

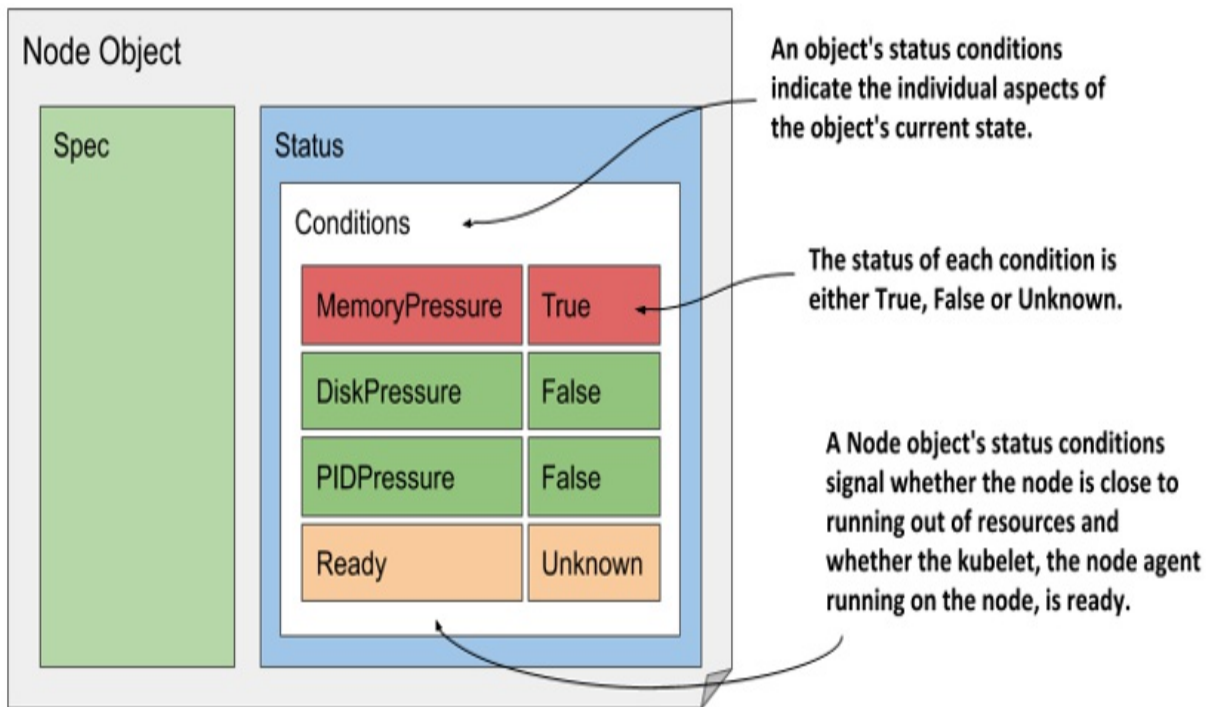
```

Tip

The jq tool is very handy if you want to see only a part of the object's structure. For example, to display the node's status conditions, you can run `kubectl get node <name> -o json | jq .status.conditions`. The equivalent tool for YAML is `yq`.

There are four conditions that reveal the state of the node. Each condition has a type and a status field, which can be True, False or Unknown, as shown in the figure 4.6. A condition can also specify a machine-facing reason for the last transition of the condition and a human-facing message with details about the transition. The `lastTransitionTime` field indicates when the condition moved from one status to another, whereas the `lastHeartbeatTime` field reveals the last time the controller received an update on the given condition.

Figure 4.6 The status conditions indicating the state of a Node object



Although it's the last condition in the list, the Ready condition is probably the most important, as it signals whether the node is ready to accept new workloads (pods). The other conditions (MemoryPressure, DiskPressure and PIDPressure) signal whether the node is running out of resources. Remember to check these conditions if a node starts to behave strangely - for example, if the applications running on it start running out of resources and/or crash.

Understanding conditions in other object kinds

A condition list such as that in Node objects is also used in many other object kinds. The conditions explained earlier are a good example of why the state of most objects is represented by multiple conditions instead of a single field.

Note

Conditions are usually orthogonal, meaning that they represent unrelated aspects of the object.

If the state of an object were represented as a single field, it would be very difficult to subsequently extend it with new values, as this would require

updating all clients that monitor the state of the object and perform actions based on it. Some object kinds originally used such a single field, and some still do, but most now use a list of conditions instead.

Since the focus of this chapter is to introduce the common features of the Kubernetes API objects, we've focused only on the conditions field, but it is far from being the only field in the status of the Node object. To explore the others, use the `kubectl explain` command as described in the previous sidebar. The fields that are not immediately easy for you to understand should become clear to you after reading the remaining chapters in this part of the book.

Note

As an exercise, use the command `kubectl get <kind> <name> -o yaml` to explore the other objects you've created so far (deployments, services, and pods).

4.2.4 Inspecting objects using the `kubectl describe` command

To give you a correct impression of the entire structure of the Kubernetes API objects, it was necessary to show you the complete YAML manifest of an object. While I personally often use this method to inspect an object, a more user-friendly way to inspect an object is the `kubectl describe` command, which typically displays the same information or sometimes even more.

Understanding the `kubectl describe` output for a Node object

Let's try running the `kubectl describe` command on a Node object. To keep things interesting, let's use it to describe one of the worker nodes instead of the master. This is the command and its output:

```
$ kubectl describe node kind-worker-2
Name:                kind-worker2
Roles:               <none>
Labels:              beta.kubernetes.io/arch=amd64
                   beta.kubernetes.io/os=linux
```

```

kubernetes.io/arch=amd64
kubernetes.io/hostname=kind-worker2
kubernetes.io/os=linux
Annotations:  kubeadm.alpha.kubernetes.io/cri-socket: /run/
               node.alpha.kubernetes.io/ttl: 0
               volumes.kubernetes.io/controller-managed-atta
CreationTimestamp:  Sun, 03 May 2020 17:09:48 +0200
Taints:          <none>
Unschedulable:   false
Lease:
  HolderIdentity:  kind-worker2
  AcquireTime:     <unset>
  RenewTime:       Sun, 17 May 2020 16:15:03 +0200
Conditions:
  Type            Status    ...   Reason                                     Mess
  ----            -
  MemoryPressure  False    ...   KubeletHasSufficientMemory               ...
  DiskPressure    False    ...   KubeletHasNoDiskPressure                 ...
  PIDPressure     False    ...   KubeletHasSufficientPID                  ...
  Ready           True      ...   KubeletReady                             ...
Addresses:
  InternalIP:  172.18.0.4
  Hostname:    kind-worker2
Capacity:
  cpu:          8
  ephemeral-storage:  401520944Ki
  hugepages-1Gi:    0
  hugepages-2Mi:    0
  memory:           32720824Ki
  pods:            110
Allocatable:
...
System Info:
...
PodCIDR:          10.244.1.0/24
PodCIDRs:         10.244.1.0/24
Non-terminated Pods:  (2 in total)
  Namespace       Name                               CPU Requests  CPU Limits    ...
  -----
  kube-system     kindnet-4xmjh                     100m (1%)     100m (1%)     ...
  kube-system     kube-proxy-dgkfm                   0 (0%)        0 (0%)        ...
Allocated resources:
(Total limits may be over 100 percent, i.e., overcommitted.)
Resource          Requests      Limits
-----
cpu               100m (1%)    100m (1%)
memory           50Mi (0%)    50Mi (0%)

```

```

ephemeral-storage 0 (0%)    0 (0%)
hugepages-1Gi     0 (0%)    0 (0%)
hugepages-2Mi     0 (0%)    0 (0%)
Events:
  Type            Reason                                     Age      From
  ----            -
Normal          Starting                                3m50s    kubelet, kind-worker2
Normal          NodeAllocatableEnforced                3m50s    kubelet, kind-worker2
Normal          NodeHasSufficientMemory                3m50s    kubelet, kind-worker2
Normal          NodeHasNoDiskPressure                  3m50s    kubelet, kind-worker2
Normal          NodeHasSufficientPID                   3m50s    kubelet, kind-worker2
Normal          Starting                                3m49s    kube-proxy, kind-worker

```

As you can see, the `kubectl describe` command displays all the information you previously found in the YAML manifest of the Node object, but in a more readable form. You can see the name, IP address, and hostname, as well as the conditions and available capacity of the node.

Inspecting other objects related to the Node

In addition to the information stored in the Node object itself, the `kubectl describe` command also displays the pods running on the node and the total amount of compute resources allocated to them. Below is also a list of events related to the node.

This additional information isn't found in the Node object itself but is collected by the `kubectl` tool from other API objects. For example, the list of pods running on the node is obtained by retrieving Pod objects via the `pods` resource.

If you run the `describe` command yourself, no events may be displayed. This is because only events that have occurred recently are shown. For Node objects, unless the node has resource capacity issues, you'll only see events if you've recently (re)started the node.

Virtually every API object kind has events associated with it. Since they are crucial for debugging a cluster, they warrant a closer look before you start exploring other objects.

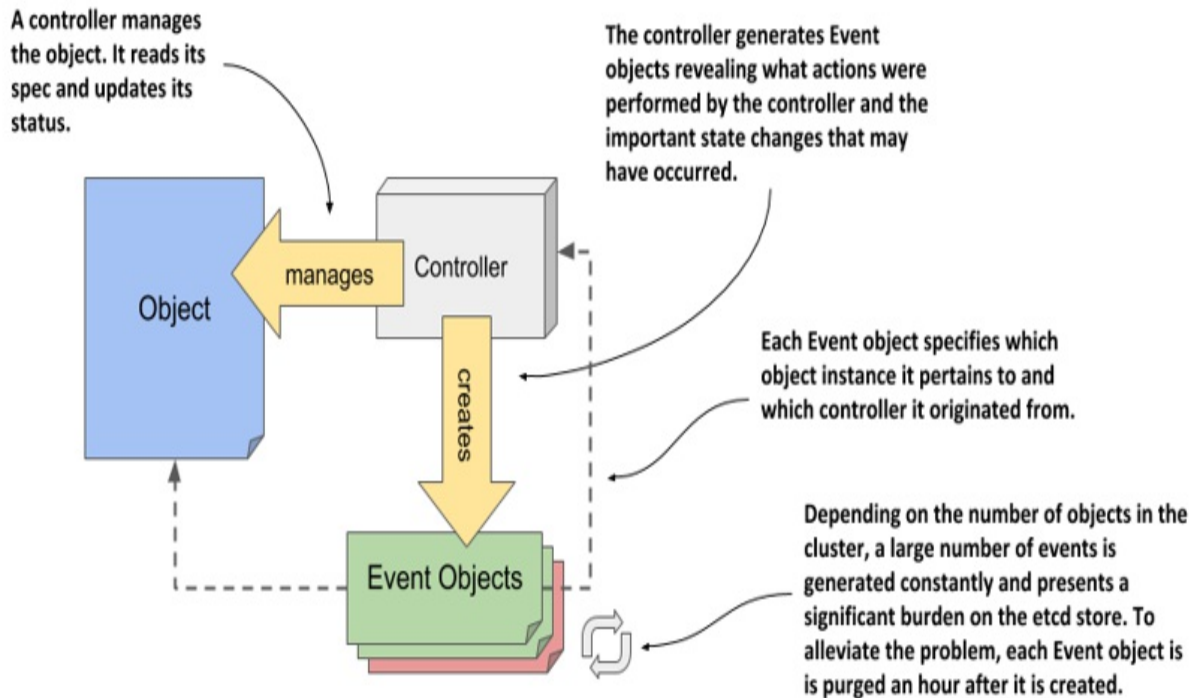
4.3 Observing cluster events via Event objects

As controllers perform their task of reconciling the actual state of an object with the desired state, as specified in the object's spec field, they generate events to reveal what they have done. Two types of events exist: Normal and Warning. Events of the latter type are usually generated by controllers when something prevents them from reconciling the object. By monitoring this type of events, you can be quickly informed of any problems that the cluster encounters.

4.3.1 Introducing the Event object

Like everything else in Kubernetes, events are represented by Event objects that are created and read via the Kubernetes API. As the following figure shows, they contain information about what happened to the object and what the source of the event was. Unlike other objects, each Event object is deleted one hour after its creation to reduce the burden on etcd, the data store for Kubernetes API objects.

Figure 4.7 The relationship between Event objects, controllers, and other API objects.



Note

The amount of time to retain events is configurable via the API server's command-line options.

Listing events using `kubectl get events`

The events displayed by `kubectl describe` refer to the object you specify as the argument to the command. Due to their nature and the fact that many events can be created for an object in a short time, they aren't part of the object itself. You won't find them in the object's YAML manifest, as they exist on their own, just like Nodes and the other objects you've seen so far.

Note

If you want to follow the exercises in this section in your own cluster, you may need to restart one of the nodes to ensure that the events are recent enough to still be present in etcd. If you can't do this, don't worry, and just skip doing these exercises yourself, as you'll also be generating and inspecting events in the exercises in the next chapter.

Because Events are standalone objects, you can list them using `kubectl get events`:

```
$ kubectl get ev
LAST
SEEN   TYPE      REASON                                OBJECT                                MESSAGE
48s    Normal    Starting                             node/kind-worker2                    Startin
48s    Normal    NodeAllocatableEnforced              node/kind-worker2                    Updated
48s    Normal    NodeHasSufficientMemory              node/kind-worker2                    Node ki
48s    Normal    NodeHasNoDiskPressure                node/kind-worker2                    Node ki
48s    Normal    NodeHasSufficientPID                 node/kind-worker2                    Node ki
47s    Normal    Starting                             node/kind-worker2                    Startin
```

Note

The previous listing uses the short name `ev` in place of `events`.

You'll notice that some events displayed in the listing match the status conditions of the Node. This is often the case, but you'll also find additional events. The two events with the reason `Starting` are two such examples. In the case at hand, they signal that the Kubelet and the Kube Proxy components have been started on the node. You don't need to worry about these components yet. They are explained in the third part of the book.

Understanding what's in an Event object

As with other objects, the `kubectl get` command only outputs the most important object data. To display additional information, you can enable additional columns by executing the command with the `-o wide` option:

```
$ kubectl get ev -o wide
```

The output of this command is extremely wide and is not listed here in the book. Instead, the information that is displayed is explained in the following table.

Table 4.1 Properties of the Event object

Property	Description
----------	-------------

Name	The name of this Event object instance. Useful only if you want to retrieve the given object from the API.
Type	The type of the event. Either Normal or Warning.
Reason	The machine-facing description why the event occurred.
Source	The component that reported this event. This is usually a controller.
Object	The object instance to which the event refers. For example, node/xyz.
Sub-object	The sub-object to which the event refers. For example, what container of the pod.
Message	The human-facing description of the event.
First seen	The first time this event occurred. Remember that each Event object is deleted after a while, so this may not be the first time that the event actually occurred.
Last seen	Events often occur repeatedly. This field indicates when this event last occurred.
Count	The number of times this event has occurred.

Tip

As you complete the exercises throughout this book, you may find it useful to run the `kubectl get events` command each time you make changes to one of your objects. This will help you learn what happens beneath the surface.

Displaying only warning events

Unlike the `kubectl describe` command, which only displays events related to the object you're describing, the `kubectl get events` command displays all events. This is useful if you want to check if there are events that you should be concerned about. You may want to ignore events of type `Normal` and focus only on those of type `Warning`.

The API provides a way to filter objects through a mechanism called field selectors. Only objects where the specified field matches the specified selector value are returned. You can use this to display only `Warning` events. The `kubectl get` command allows you to specify the field selector with the `--field-selector` option. To list only events that represent warnings, you execute the following command:

```
$ kubectl get ev --field-selector type=Warning
No resources found in default namespace.
```

If the command does not print any events, as in the above case, no warnings have been recorded in your cluster recently.

You may wonder how I knew the exact name of the field to be used in the field selector and what its exact value should be (perhaps it should have been lower case, for example). Hats off if you guessed that this information is provided by the `kubectl explain events` command. Since events are regular API objects, you can use it to look up documentation on the event objects' structure. There you'll learn that the `type` field can have two values: either `Normal` or `Warning`.

4.3.2 Examining the YAML of the Event object

To inspect the events in your cluster, the commands `kubectl describe` and `kubectl get events` should be sufficient. Unlike other objects, you'll probably never have to display the complete YAML of an Event object. But I'd like to take this opportunity to show you an annoying thing about Kubernetes object manifests that the API returns.

Event objects have no spec and status sections

If you use the `kubectl explain` to explore the structure of the Event object, you'll notice that it has no spec or status sections. Unfortunately, this means that its fields are not as nicely organized as in the Node object, for example.

Inspect the following YAML and see if you can easily find the object's kind, metadata, and other fields.

```
apiVersion: v1
count: 1
eventTime: null
firstTimestamp: "2020-05-17T18:16:40Z"
involvedObject:
  kind: Node
  name: kind-worker2
  uid: kind-worker2
kind: Event
lastTimestamp: "2020-05-17T18:16:40Z"
message: Starting kubelet.
metadata:
  creationTimestamp: "2020-05-17T18:16:40Z"
  name: kind-worker2.160fe38fc0bc3703
  namespace: default
  resourceVersion: "3528471"
  selfLink: /api/v1/namespaces/default/events/kind-worker2.160f..
  uid: da97e812-d89e-4890-9663-091fd1ec5e2d
reason: Starting
reportingComponent: ""
reportingInstance: ""
source:
  component: kubelet
  host: kind-worker2
type: Normal
```

You will surely agree that the YAML manifest in the listing is disorganized. The fields are listed alphabetically instead of being organized into coherent

groups. This makes it difficult for us humans to read. It looks so chaotic that it's no wonder that many people hate to deal with Kubernetes YAML or JSON manifests, since both suffer from this problem.

In contrast, the earlier YAML manifest of the Node object was relatively easy to read, because the order of the top-level fields is what one would expect: `apiVersion`, `kind`, `metadata`, `spec`, and `status`. You'll notice that this is simply because the alphabetical order of the five fields just happens to make sense. But the fields under those fields suffer from the same problem, as they are also sorted alphabetically.

YAML is supposed to be easy for people to read, but the alphabetical field order in Kubernetes YAML breaks this. Fortunately, most objects contain the `spec` and `status` sections, so at least the top-level fields in these objects are well organized. As for the rest, you'll just have to accept this unfortunate aspect of dealing with Kubernetes manifests.

4.4 Summary

In this chapter, you've learned:

- Kubernetes provides a RESTful API for interaction with a cluster. API Objects map to actual components that make up the cluster, including applications, load balancers, nodes, storage volumes, and many others.
- An object instance can be represented by many resources. A single object type can be exposed through several resources that are just different representations of the same thing.
- Kubernetes API objects are described in YAML or JSON manifests. Objects are created by posting a manifest to the API. The status of the object is stored in the object itself and can be retrieved by requesting the object from the API with a GET request.
- All Kubernetes API objects contain Type and Object Metadata, and most have a `spec` and `status` sections. A few object types don't have these two sections, because they only contain static data.
- Controllers bring objects to life by constantly watching for changes in their `spec`, updating the cluster state and reporting the current state via the object's `status` field.

- As controllers manage Kubernetes API objects, they emit events to reveal what actions they have performed. Like everything else, events are represented by Event objects and can be retrieved through the API. Events signal what is happening to a Node or other object. They show what has recently happened to the object and can provide clues as to why it is broken.
- The `kubectl explain` command provides a quick way to look up documentation on a specific object kind and its fields from the command line.
- The status in a Node object contains information about the node's IP address and hostname, its resource capacity, conditions, cached container images and other information about the node. Pods running on the node are not part of the node's status, but the `kubectl describe node` command gets this information from the pods resource.
- Many object types use status conditions to signal the state of the component that the object represents. For nodes, these conditions are `MemoryPressure`, `DiskPressure` and `PIDPressure`. Each condition is either `True`, `False`, or `Unknown` and has an associated reason and message that explain why the condition is in the specified state.

You should now be familiar with the general structure of the Kubernetes API objects. In the next chapter, you'll learn about the Pod object, the fundamental building block which represents one running instance of your application.

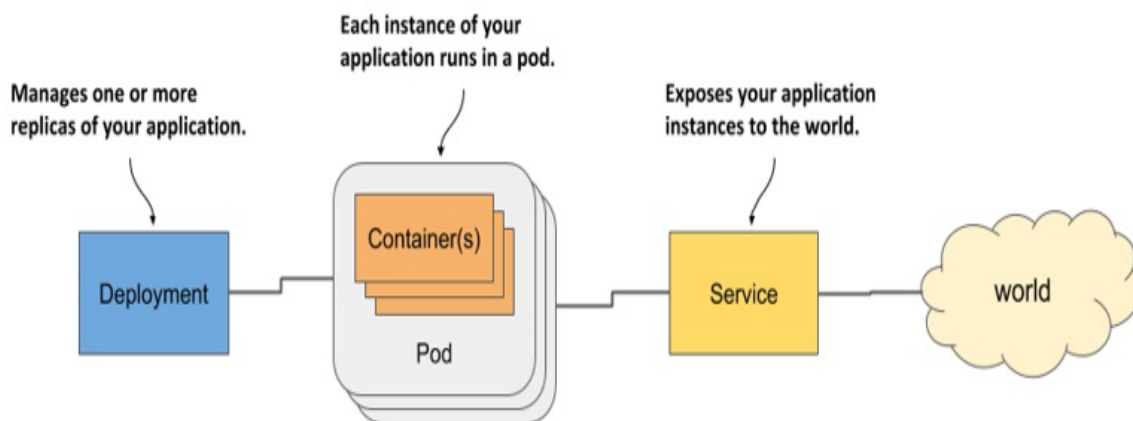
5 Running workloads in Pods

This chapter covers

- Understanding how and when to group containers
- Running an application by creating a Pod object from a YAML file
- Communicating with an application, viewing its logs, and exploring its environment
- Adding a sidecar container to extend the pod's main container
- Initializing pods by running init containers at pod startup

Let me refresh your memory with a diagram that shows the three types of objects you created in chapter 3 to deploy a minimal application on Kubernetes. Figure 5.1 shows how they relate to each other and what functions they have in the system.

Figure 5.1 Three basic object types comprising a deployed application



You now have a basic understanding of how these objects are exposed via the Kubernetes API. In this and the following chapters, you'll learn about the specifics of each of them and many others that are typically used to deploy a full application. Let's start with the Pod object, as it represents the central, most important concept in Kubernetes - a running instance of your application.

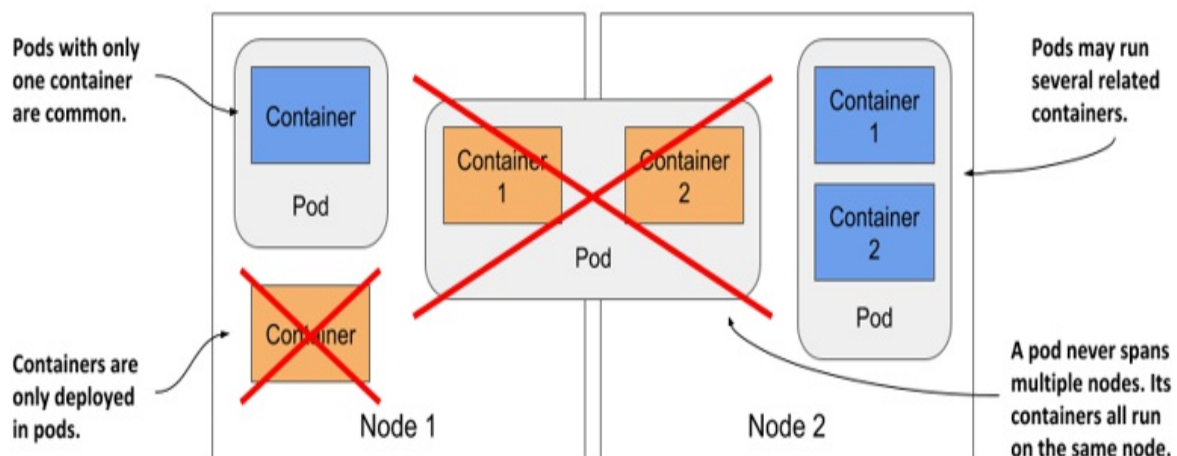
Note

You'll find the code files for this chapter at <https://github.com/luksa/kubernetes-in-action-2nd-edition/tree/master/Chapter05>

5.1 Understanding pods

You've already learned that a pod is a co-located group of containers and the basic building block in Kubernetes. Instead of deploying containers individually, you deploy and manage a group of containers as a single unit — a pod. Although pods may contain several, it's not uncommon for a pod to contain just a single container. When a pod has multiple containers, all of them run on the same worker node — a single pod instance never spans multiple nodes. Figure 5.2 will help you visualize this information.

Figure 5.2 All containers of a pod run on the same node. A pod never spans multiple nodes.



5.1.1 Understanding why we need pods

Let's discuss why we need to run multiple containers together, as opposed to, for example, running multiple processes in the same container.

Understanding why one container shouldn't contain multiple processes

Imagine an application that consists of several processes that communicate with each other via *IPC* (Inter-Process Communication) or shared files, which requires them to run on the same computer. In chapter 2, you learned that each container is like an isolated computer or virtual machine. A computer typically runs several processes; containers can also do this. You can run all the processes that make up an application in just one container, but that makes the container very difficult to manage.

Containers are *designed* to run only a single process, not counting any child processes that it spawns. Both container tooling and Kubernetes were developed around this fact. For example, a process running in a container is expected to write its logs to standard output. Docker and Kubernetes commands that you use to display the logs only show what has been captured from this output. If a single process is running in the container, it's the only writer, but if you run multiple processes in the container, they all write to the same output. Their logs are therefore intertwined, and it's difficult to tell which process each line belongs to.

Another indication that containers should only run a single process is the fact that the container runtime only restarts the container when the container's root process dies. It doesn't care about any child processes created by this root process. If it spawns child processes, it alone is responsible for keeping all these processes running.

To take full advantage of the features provided by the container runtime, you should consider running only one process in each container.

Understanding how a pod combines multiple containers

Since you shouldn't run multiple processes in a single container, it's evident you need another higher-level construct that allows you to run related processes together even when divided into multiple containers. These processes must be able to communicate with each other like processes in a normal computer. And that is why pods were introduced.

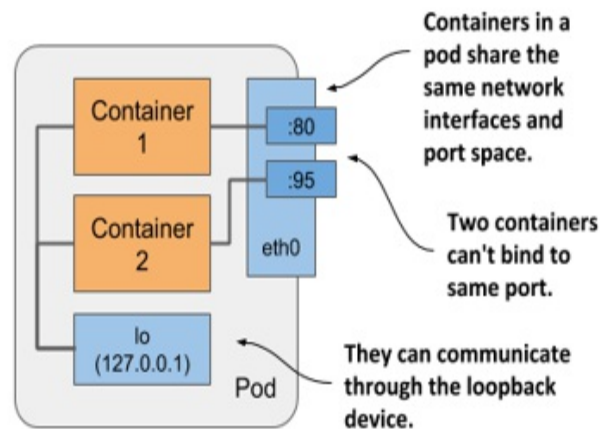
With a pod, you can run closely related processes together, giving them (almost) the same environment as if they were all running in a single

container. These processes are somewhat isolated, but not completely - they share some resources. This gives you the best of both worlds. You can use all the features that containers offer, but also allow processes to work together. A pod makes these interconnected containers manageable as one unit.

In the second chapter, you learned that a container uses its own set of Linux namespaces, but it can also share some with other containers. This sharing of namespaces is exactly how Kubernetes and the container runtime combine containers into pods.

As shown in figure 5.3, all containers in a pod share the same Network namespace and thus the network interfaces, IP address(es) and port space that belong to it.

Figure 5.3 Containers in a pod share the same network interfaces



Because of the shared port space, processes running in containers of the same pod can't be bound to the same port numbers, whereas processes in other pods have their own network interfaces and port spaces, eliminating port conflicts between different pods.

All the containers in a pod also see the same system hostname, because they share the UTS namespace, and can communicate through the usual IPC mechanisms because they share the IPC namespace. A pod can also be configured to use a single PID namespace for all its containers, which makes them share a single process tree, but you must explicitly enable this for each

pod individually.

Note

When containers of the same pod use separate PID namespaces, they can't see each other or send process signals like `SIGTERM` or `SIGINT` between them.

It's this sharing of certain namespaces that gives the processes running in a pod the impression that they run together, even though they run in separate containers.

In contrast, each container always has its own Mount namespace, giving it its own file system, but when two containers must share a part of the file system, you can add a *volume* to the pod and mount it into both containers. The two containers still use two separate Mount namespaces, but the shared volume is mounted into both. You'll learn more about volumes in chapter 7.

5.1.2 Organizing containers into pods

You can think of each pod as a separate computer. Unlike virtual machines, which typically host multiple applications, you typically run only one application in each pod. You never need to combine multiple applications in a single pod, as pods have almost no resource overhead. You can have as many pods as you need, so instead of stuffing all your applications into a single pod, you should divide them so that each pod runs only closely related application processes.

Let me illustrate this with a concrete example.

Splitting a multi-tier application stack into multiple pods

Imagine a simple system composed of a front-end web server and a back-end database. I've already explained that the front-end server and the database shouldn't run in the same container, as all the features built into containers were designed around the expectation that not more than one process runs in a container. If not in a single container, should you then run them in separate containers that are all in the same pod?

Although nothing prevents you from running both the front-end server and the database in a single pod, this isn't the best approach. I've explained that all containers of a pod always run co-located, but do the web server and the database have to run on the same computer? The answer is obviously no, as they can easily communicate over the network. Therefore you shouldn't run them in the same pod.

If both the front-end and the back-end are in the same pod, both run on the same cluster node. If you have a two-node cluster and only create this one pod, you are using only a single worker node and aren't taking advantage of the computing resources available on the second node. This means wasted CPU, memory, disk storage and bandwidth. Splitting the containers into two pods allows Kubernetes to place the front-end pod on one node and the back-end pod on the other, thereby improving the utilization of your hardware.

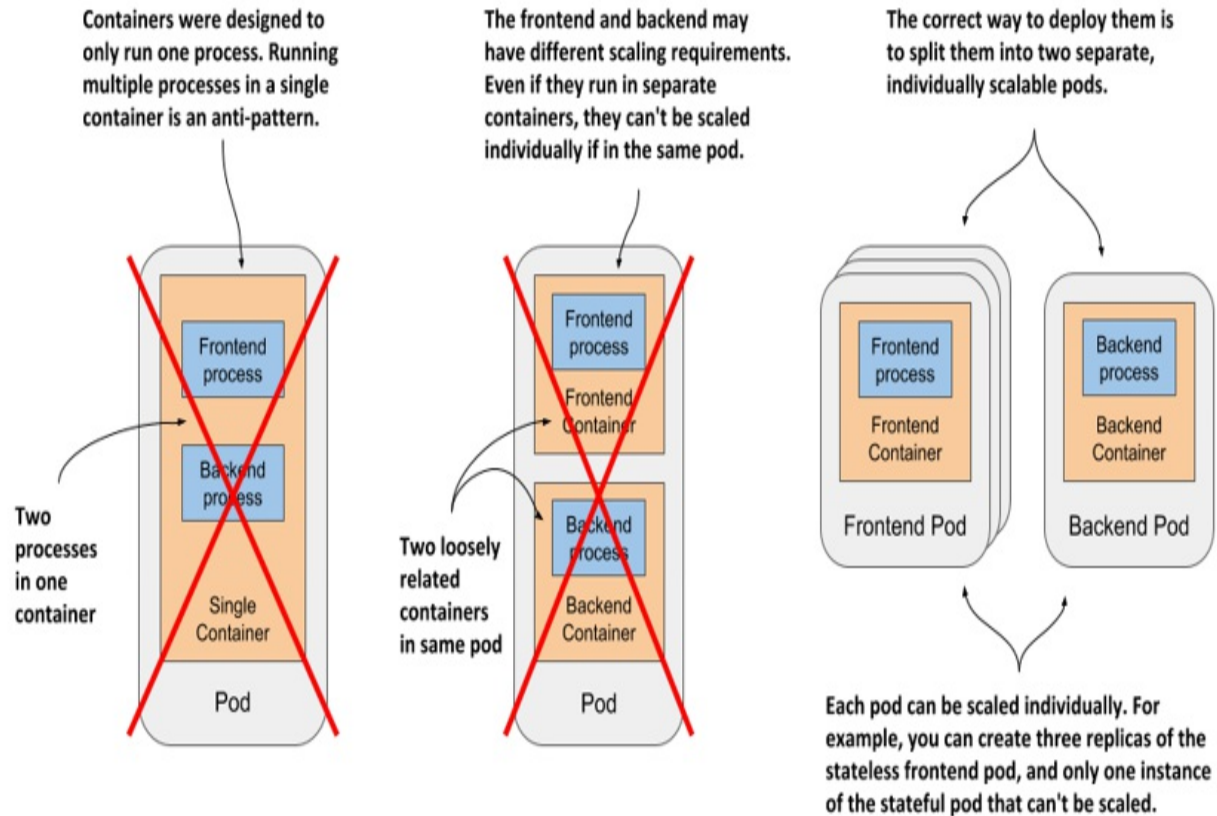
Splitting into multiple pods to enable individual scaling

Another reason not to use a single pod has to do with horizontal scaling. A pod is not only the basic unit of deployment, but also the basic unit of scaling. In chapter 2 you scaled the Deployment object and Kubernetes created additional pods – additional replicas of your application. Kubernetes doesn't replicate containers within a pod. It replicates the entire pod.

Front-end components usually have different scaling requirements than back-end components, so we typically scale them individually. When your pod contains both the front-end and back-end containers and Kubernetes replicates it, you end up with multiple instances of both the front-end and back-end containers, which isn't always what you want. Stateful back-ends, such as databases, usually can't be scaled. At least not as easily as stateless front ends. If a container has to be scaled separately from the other components, this is a clear indication that it must be deployed in a separate pod.

The following figure illustrates what was just explained.

Figure 5.4 Splitting an application stack into pods

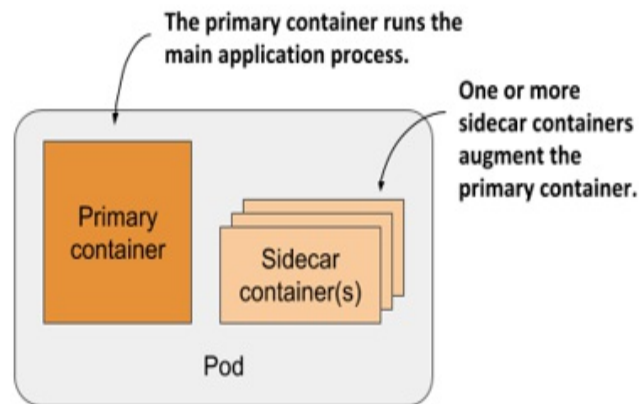


Splitting application stacks into multiple pods is the correct approach. But then, when does one run multiple containers in the same pod?

Introducing sidecar containers

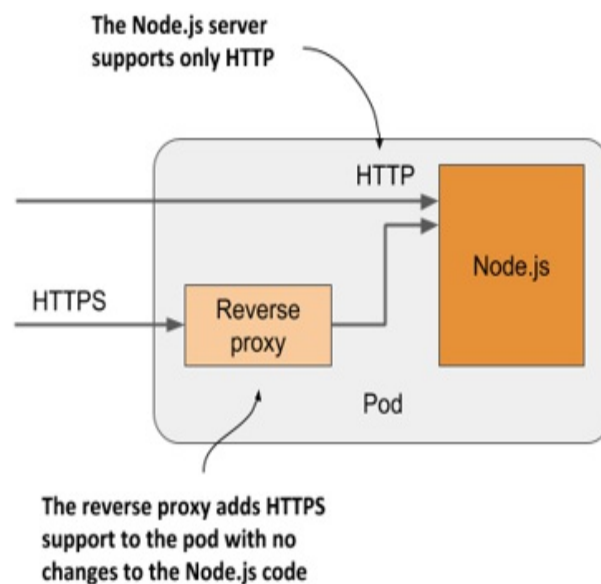
Placing several containers in a single pod is only appropriate if the application consists of a primary process and one or more processes that complement the operation of the primary process. The container in which the complementary process runs is called a *sidecar container* because it's analogous to a motorcycle sidecar, which makes the motorcycle more stable and offers the possibility of carrying an additional passenger. But unlike motorcycles, a pod can have more than one sidecar, as shown in figure 5.5.

Figure 5.5 A pod with a primary and sidecar container(s)



It's difficult to imagine what constitutes a complementary process, so I'll give you some examples. In chapter 2, you deployed pods with one container that runs a Node.js application. The Node.js application only supports the HTTP protocol. To make it support HTTPS, we could add a bit more JavaScript code, but we can also do it without changing the existing application at all - by adding an additional container to the pod – a reverse proxy that converts HTTPS traffic to HTTP and forwards it to the Node.js container. The Node.js container is thus the primary container, whereas the container running the proxy is the sidecar container. Figure 5.6 shows this example.

Figure 5.6 A sidecar container that converts HTTPS traffic to HTTP

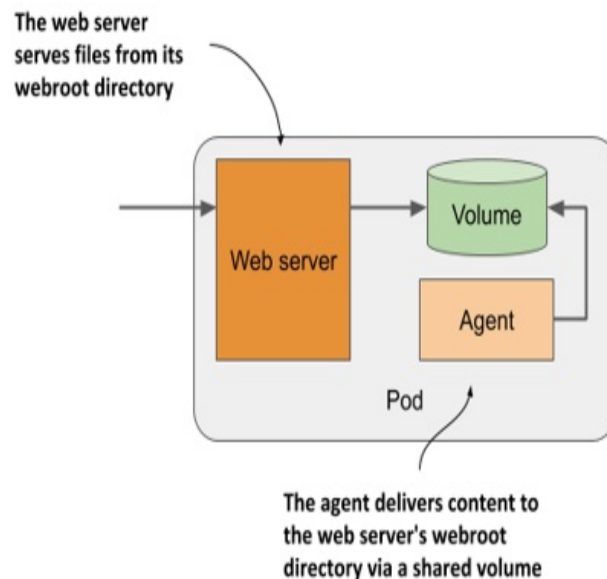


Note

You'll create this pod in section 5.4.

Another example, shown in figure 5.7, is a pod where the primary container runs a web server that serves files from its webroot directory. The other container in the pod is an agent that periodically downloads content from an external source and stores it in the web server's webroot directory. As I mentioned earlier, two containers can share files by sharing a volume. The webroot directory would be located on this volume.

Figure 5.7 A sidecar container that delivers content to the web server container via a volume



Note

You'll create this pod in the chapter 7.

Other examples of sidecar containers are log rotators and collectors, data processors, communication adapters, and others.

Unlike changing the application's existing code, adding a sidecar increases the pod's resources requirements because an additional process must run in the pod. But keep in mind that adding code to legacy applications can be very

difficult. This could be because its code is difficult to modify, it's difficult to set up the build environment, or the source code itself is no longer available. Extending the application by adding an additional process is sometimes a cheaper and faster option.

How to decide whether to split containers into multiple pods

When deciding whether to use the sidecar pattern and place containers in a single pod, or to place them in separate pods, ask yourself the following questions:

- Do these containers have to run on the same host?
- Do I want to manage them as a single unit?
- Do they form a unified whole instead of being independent components?
- Do they have to be scaled together?
- Can a single node meet their combined resource needs?

If the answer to all these questions is yes, put them all in the same pod. As a rule of thumb, always place containers in separate pods unless a specific reason requires them to be part of the same pod.

5.2 Creating pods from YAML or JSON files

With the information you learned in the previous sections, you can now start creating pods. In chapter 3, you created them using the imperative command `kubectl create`, but pods and other Kubernetes objects are usually created by creating a JSON or YAML manifest file and posting it to the Kubernetes API, as you've already learned in the previous chapter.

Note

The decision whether to use YAML or JSON to define your objects is yours. Most people prefer to use YAML because it's slightly more human-friendly and allows you to add comments to the object definition.

By using YAML files to define the structure of your application, you don't need shell scripts to make the process of deploying your applications

repeatable, and you can keep a history of all changes by storing these files in a VCS (Version Control System). Just like you store code.

In fact, the application manifests of the exercises in this book are all stored in a VCS. You can find them on GitHub at github.com/luksa/kubernetes-in-action-2nd-edition.

5.2.1 Creating a YAML manifest for a pod

In the previous chapter you learned how to retrieve and examine the YAML manifests of existing API objects. Now you'll create an object manifest from scratch.

You'll start by creating a file called `pod.kiada.yaml` on your computer, in a location of your choosing. You can also find the file in the book's code archive in the `Chapter05/` directory. The following listing shows the contents of the file.

Listing 5.1 A basic pod manifest file

```
apiVersion: v1      #A
kind: Pod           #B
metadata:
  name: kiada       #C
spec:
  containers:
    - name: kiada    #D
      image: luksa/kiada:0.1    #E
      ports:
        - containerPort: 8080    #F
```

I'm sure you'll agree that this pod manifest is much easier to understand than the mammoth of a manifest representing the Node object, which you saw in the previous chapter. But once you post this pod object manifest to the API and then read it back, it won't be much different.

The manifest in listing 5.1 is short only because it does not yet contain all the fields that a pod object gets after it is created through the API. For example, you'll notice that the metadata section contains only a single field and that the status section is completely missing. Once you create the object from

this manifest, this will no longer be the case. But we'll get to that later.

Before you create the object, let's examine the manifest in detail. It uses version v1 of the Kubernetes API to describe the object. The object kind is Pod and the name of the object is kiada. The pod consists of a single container also called kiada, based on the luksa/kiada:0.1 image. The pod definition also specifies that the application in the container listens on port 8080.

Tip

Whenever you want to create a pod manifest from scratch, you can also use the following command to create the file and then edit it to add more fields: `kubect1 run kiada --image=luksa/kiada:0.1 --dry-run=client -o yaml > mypod.yaml`. The `--dry-run=client` flag tells kubect1 to output the definition instead of actually creating the object via the API.

The fields in the YAML file are self-explanatory, but if you want more information about each field or want to know what additional fields you can add, remember to use the `kubect1 explain pods` command.

5.2.2 Creating the Pod object from the YAML file

After you've prepared the manifest file for your pod, you can now create the object by posting the file to the Kubernetes API.

Creating objects by applying the manifest file to the cluster

When you post the manifest to the API, you are directing Kubernetes to *apply* the manifest to the cluster. That's why the kubect1 sub-command that does this is called `apply`. Let's use it to create the pod:

```
$ kubect1 apply -f pod.kiada.yaml
pod "kiada" created
```

Updating objects by modifying the manifest file and re-applying it

The `kubectl apply` command is used for creating objects as well as for making changes to existing objects. If you later decide to make changes to your pod object, you can simply edit the `pod.kiada.yaml` file and run the `apply` command again. Some of the pod's fields aren't mutable, so the update may fail, but you can always delete the pod and re-create it. You'll learn how to delete pods and other objects at the end of this chapter.

Retrieving the full manifest of a running pod

The pod object is now part of the cluster configuration. You can now read it back from the API to see the full object manifest with the following command:

```
$ kubectl get po kiada -o yaml
```

If you run this command, you'll notice that the manifest has grown considerably compared to the one in the `pod.kiada.yaml` file. You'll see that the metadata section is now much bigger, and the object now has a status section. The spec section has also grown by several fields. You can use `kubectl explain` to learn more about these new fields, but most of them will be explained in this and the following chapters.

5.2.3 Checking the newly created pod

Let's use the basic `kubectl` commands to see how the pod is doing before we start interacting with the application running inside it.

Quickly checking the status of a pod

Your Pod object has been created, but how do you know if the container in the pod is actually running? You can use the `kubectl get` command to see a summary of the pod:

```
$ kubectl get pod kiada
NAME      READY   STATUS    RESTARTS   AGE
kiada     1/1     Running   0           32s
```

You can see that the pod is running, but not much else. To see more, you can

try the `kubectl get pod -o wide` or the `kubectl describe` command that you learned in the previous chapter.

Using `kubectl describe` to see pod details

To display a more detailed view of the pod, use the `kubectl describe` command:

```
$ kubectl describe pod kiada
Name:          kiada
Namespace:     default
Priority:       0
Node:          worker2/172.18.0.4
Start Time:    Mon, 27 Jan 2020 12:53:28 +0100
...
```

The listing doesn't show the entire output, but if you run the command yourself, you'll see virtually all information that you'd see if you print the complete object manifest using the `kubectl get -o yaml` command.

Inspecting events to see what happens beneath the surface

As in the previous chapter where you used the `describe node` command to inspect a Node object, the `describe pod` command should display several events related to the pod at the bottom of the output.

If you remember, these events aren't part of the object itself, but are separate objects. Let's print them to learn more about what happens when you create the pod object. These are the events that were logged after the pod was created:

```
$ kubectl get events
LAST SEEN   TYPE      REASON      OBJECT      MESSAGE
<unknown>   Normal    Scheduled    pod/kiada    Successfully assigne
5m          Normal    Pulling      pod/kiada    Pulling image luksa/
5m          Normal    Pulled       pod/kiada    Successfully pulled
5m          Normal    Created      pod/kiada    Created container ki
5m          Normal    Started      pod/kiada    Started container ki
```

These events are printed in chronological order. The most recent event is at the bottom. You see that the pod was first assigned to one of the worker nodes, then the container image was pulled, then the container was created and finally started.

No warning events are displayed, so everything seems to be fine. If this is not the case in your cluster, you should read section 5.4 to learn how to troubleshoot pod failures.

5.3 Interacting with the application and the pod

Your container is now running. In this section, you'll learn how to communicate with the application, inspect its logs, and execute commands in the container to explore the application's environment. Let's confirm that the application running in the container responds to your requests.

5.3.1 Sending requests to the application in the pod

In chapter 2, you used the `kubectl expose` command to create a service that provisioned a load balancer so you could talk to the application running in your pod(s). You'll now take a different approach. For development, testing and debugging purposes, you may want to communicate directly with a specific pod, rather than using a service that forwards connections to randomly selected pods.

You've learned that each pod is assigned its own IP address where it can be accessed by every other pod in the cluster. This IP address is typically internal to the cluster. You can't access it from your local computer, except when Kubernetes is deployed in a specific way – for example, when using `kind` or `Minikube` without a VM to create the cluster.

In general, to access pods, you must use one of the methods described in the following sections. First, let's determine the pod's IP address.

Getting the pod's IP address

You can get the pod's IP address by retrieving the pod's full YAML and

searching for the podIP field in the status section. Alternatively, you can display the IP with `kubectl describe`, but the easiest way is to use `kubectl get` with the wide output option:

```
$ kubectl get pod kiada -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP           NODE
kiada     1/1     Running   0           35m   10.244.2.4   worker2
```

As indicated in the IP column, my pod's IP is 10.244.2.4. Now I need to determine the port number the application is listening on.

Getting the port number used by the application

If I wasn't the author of the application, it would be difficult for me to find out which port the application listens on. I could inspect its source code or the Dockerfile of the container image, as the port is usually specified there, but I might not have access to either. If someone else had created the pod, how would I know which port it was listening on?

Fortunately, you can specify a list of ports in the pod definition itself. It isn't necessary to specify any ports, but it is a good idea to always do so. See sidebar for details.

Why specify container ports in pod definitions

Specifying ports in the pod definition is purely informative. Their omission has no effect on whether clients can connect to the pod's port. If the container accepts connections through a port bound to its IP address, anyone can connect to it, even if the port isn't explicitly specified in the pod spec or if you specify an incorrect port number.

Despite this, it's a good idea to always specify the ports so that anyone who has access to your cluster can see which ports each pod exposes. By explicitly defining ports, you can also assign a name to each port, which is very useful when you expose pods via services.

The pod manifest says that the container uses port 8080, so you now have everything you need to talk to the application.

Connecting to the pod from the worker nodes

The Kubernetes network model dictates that each pod is accessible from any other pod and that each *node* can reach any pod on any node in the cluster.

Because of this, one way to communicate with your pod is to log into one of your worker nodes and talk to the pod from there. You've already learned that the way you log on to a node depends on what you used to deploy your cluster. If you're using `kind`, run `docker exec -it kind-worker bash`, or `minikube ssh` if you're using Minikube. On GKE use the `gcloud compute ssh` command. For other clusters refer to their documentation.

Once you have logged into the node, use the `curl` command with the pod's IP and port to access your application. My pod's IP is 10.244.2.4 and the port is 8080, so I run the following command:

```
$ curl 10.244.2.4:8080
Kiada version 0.1. Request processed by "kiada". Client IP: ::ffff
```

Normally you don't use this method to talk to your pods, but you may need to use it if there are communication issues and you want to find the cause by first trying the shortest possible communication route. In this case, it's best to log into the node where the pod is located and run `curl` from there. The communication between it and the pod takes place locally, so this method always has the highest chances of success.

Connecting from a one-off client pod

The second way to test the connectivity of your application is to run `curl` in another pod that you create specifically for this task. Use this method to test if other pods will be able to access your pod. Even if the network works perfectly, this may not be the case. In chapter 24, you'll learn how to lock down the network by isolating pods from each other. In such a system, a pod can only talk to the pods it's allowed to.

To run `curl` in a one-off pod, use the following command:

```
$ kubectl run --image=curlimages/curl -it --restart=Never --rm cl
```

```
Kiada version 0.1. Request processed by "kiada". Client IP: ::ffff
pod "client-pod" deleted
```

This command runs a pod with a single container created from the `curlimages/curl` image. You can also use any other image that provides the `curl` binary executable. The `-it` option attaches your console to the container's standard input and output, the `--restart=Never` option ensures that the pod is considered Completed when the `curl` command and its container terminate, and the `--rm` options removes the pod at the end. The name of the pod is `client-pod` and the command executed in its container is `curl 10.244.2.4:8080`.

Note

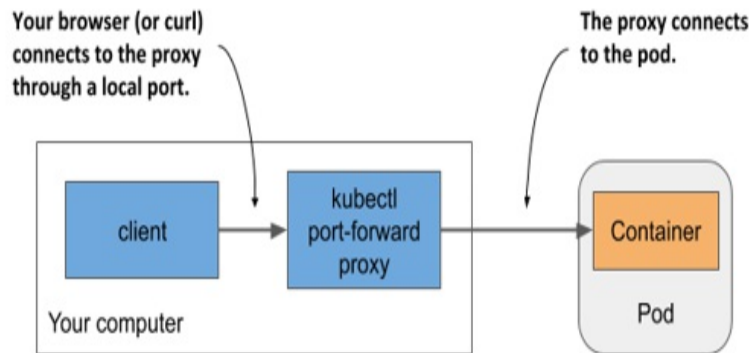
You can also modify the command to run the bash shell in the client pod and then run `curl` from the shell.

Creating a pod just to see if it can access another pod is useful when you're specifically testing pod-to-pod connectivity. If you only want to know if your pod is responding to requests, you can also use the method explained in the next section.

Connecting to pods via kubectl port forwarding

During development, the easiest way to talk to applications running in your pods is to use the `kubectl port-forward` command, which allows you to communicate with a specific pod through a proxy bound to a network port on your local computer, as shown in the next figure.

Figure 5.8 Connecting to a pod through the kubectl port-forward proxy



To open a communication path with a pod, you don't even need to look up the pod's IP, as you only need to specify its name and the port. The following command starts a proxy that forwards your computer's local port 8080 to the kiada pod's port 8080:

```
$ kubectl port-forward kiada 8080
... Forwarding from 127.0.0.1:8080 -> 8080
... Forwarding from [::1]:8080 -> 8080
```

The proxy now waits for incoming connections. Run the following `curl` command in another terminal:

```
$ curl localhost:8080
Kiada version 0.1. Request processed by "kiada". Client IP: ::ffff
```

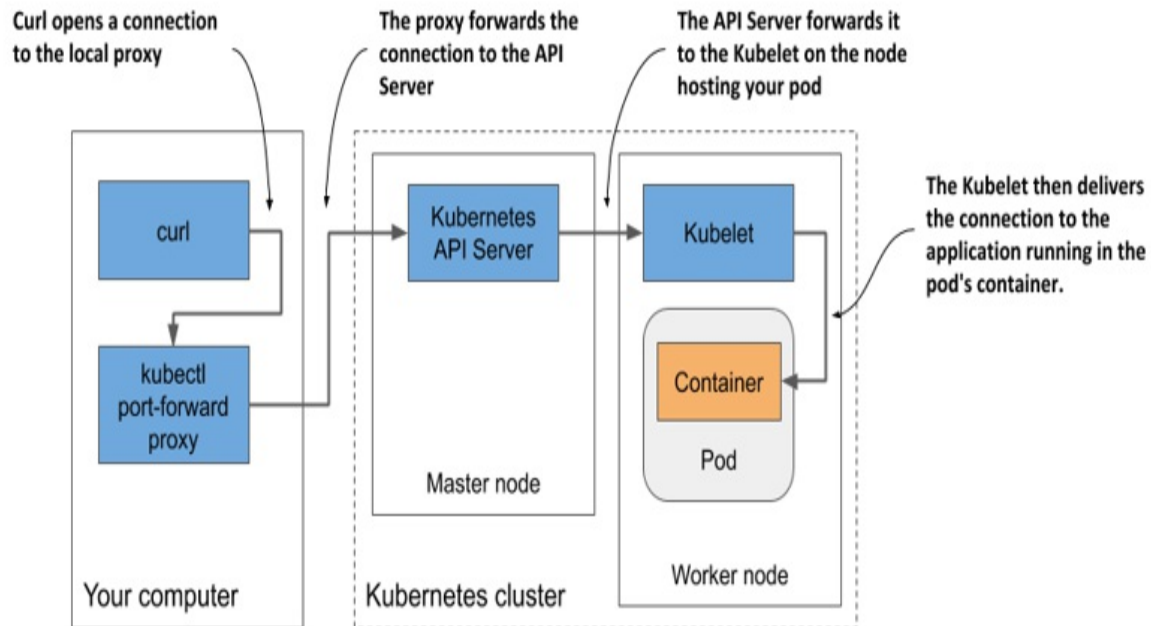
As you can see, `curl` has connected to the local proxy and received the response from the pod. While the `port-forward` command is the easiest method for communicating with a specific pod during development and troubleshooting, it's also the most complex method in terms of what happens underneath. Communication passes through several components, so if anything is broken in the communication path, you won't be able to talk to the pod, even if the pod itself is accessible via regular communication channels.

Note

The `kubectl port-forward` command can also forward connections to services instead of pods and has several other useful features. Run `kubectl port-forward --help` to learn more.

Figure 5.9 shows how the network packets flow from the `curl` process to your application and back.

Figure 5.9 The long communication path between `curl` and the container when using port forwarding



As shown in the figure, the `curl` process connects to the proxy, which connects to the API server, which then connects to the Kubelet on the node that hosts the pod, and the Kubelet then connects to the container through the pod's loopback device (in other words, through the `localhost` address). I'm sure you'll agree that the communication path is exceptionally long.

Note

The application in the container must be bound to a port on the loopback device for the Kubelet to reach it. If it listens only on the pod's `eth0` network interface, you won't be able to reach it with the `kubectl port-forward` command.

5.3.2 Viewing application logs

Your Node.js application writes its log to the standard output stream. Instead

of writing the log to a file, containerized applications usually log to the standard output (*stdout*) and standard error streams (*stderr*). This allows the container runtime to intercept the output, store it in a consistent location (usually `/var/log/containers`) and provide access to the log without having to know where each application stores its log files.

When you run an application in a container using Docker, you can display its log with `docker logs <container-id>`. When you run your application in Kubernetes, you could log into the node that hosts the pod and display its log using `docker logs`, but Kubernetes provides an easier way to do this with the `kubectl logs` command.

Retrieving a pod's log with `kubectl logs`

To view the log of your pod (more specifically, the container's log), run the following command:

```
$ kubectl logs kiada
Kiada - Kubernetes in Action Demo Application
-----
Kiada 0.1 starting...
Local hostname is kiada
Listening on port 8080
Received request for / from ::ffff:10.244.2.1      #A
Received request for / from ::ffff:10.244.2.5      #B
Received request for / from ::ffff:127.0.0.1      #C
```

Streaming logs using `kubectl logs -f`

If you want to stream the application log in real-time to see each request as it comes in, you can run the command with the `--follow` option (or the shorter version `-f`):

```
$ kubectl logs kiada -f
```

Now send some additional requests to the application and have a look at the log. Press `ctrl-C` to stop streaming the log when you're done.

Displaying the timestamp of each logged line

You may have noticed that we forgot to include the timestamp in the log statement. Logs without timestamps have limited usability. Fortunately, the container runtime attaches the current timestamp to every line produced by the application. You can display these timestamps by using the `--timestamps=true` option as follows:

```
$ kubectl logs kiada --timestamps=true
2020-02-01T09:44:40.954641934Z Kiada - Kubernetes in Action Demo
2020-02-01T09:44:40.954843234Z -----
2020-02-01T09:44:40.955032432Z Kiada 0.1 starting...
2020-02-01T09:44:40.955123432Z Local hostname is kiada
2020-02-01T09:44:40.956435431Z Listening on port 8080
2020-02-01T09:50:04.978043089Z Received request for / from ...
2020-02-01T09:50:33.640897378Z Received request for / from ...
2020-02-01T09:50:44.781473256Z Received request for / from ...
```

Tip

You can display timestamps by only typing `--timestamps` without the value. For boolean options, merely specifying the option name sets the option to true. This applies to all `kubectl` options that take a Boolean value and default to false.

Displaying recent logs

The previous feature is great if you run third-party applications that don't include the timestamp in their log output, but the fact that each line is timestamped brings us another benefit: filtering log lines by time. `Kubectl` provides two ways of filtering the logs by time.

The first option is when you want to only display logs from the past several seconds, minutes or hours. For example, to see the logs produced in the last two minutes, run:

```
$ kubectl logs kiada --since=2m
```

The other option is to display logs produced after a specific date and time using the `--since-time` option. The time format to be used is RFC3339. For example, the following command is used to print logs produced after February 1st, 2020 at 9:50 a.m.:

```
$ kubectl logs kiada --since-time=2020-02-01T09:50:00Z
```

Displaying the last several lines of the log

Instead of using time to constrain the output, you can also specify how many lines from the end of the log you want to display. To display the last ten lines, try:

```
$ kubectl logs kiada --tail=10
```

Note

Kubectl options that take a value can be specified with an equal sign or with a space. Instead of `--tail=10`, you can also type `--tail 10`.

Understanding the availability of the pod's logs

Kubernetes keeps a separate log file for each container. They are usually stored in `/var/log/containers` on the node that runs the container. A separate file is created for each container. If the container is restarted, its logs are written to a new file. Because of this, if the container is restarted while you're following its log with `kubectl logs -f`, the command will terminate, and you'll need to run it again to stream the new container's logs.

The `kubectl logs` command displays only the logs of the current container. To view the logs from the previous container, use the `--previous` (or `-p`) option.

Note

Depending on your cluster configuration, the log files may also be rotated when they reach a certain size. In this case, `kubectl logs` will only display the current log file. When streaming the logs, you must restart the command to switch to the new file when the log is rotated.

When you delete a pod, all its log files are also deleted. To make pods' logs available permanently, you need to set up a central, cluster-wide logging

system. Chapter 23 explains how.

What about applications that write their logs to files?

If your application writes its logs to a file instead of stdout, you may be wondering how to access that file. Ideally, you'd configure the centralized logging system to collect the logs so you can view them in a central location, but sometimes you just want to keep things simple and don't mind accessing the logs manually. In the next two sections, you'll learn how to copy log and other files from the container to your computer and in the opposite direction, and how to run commands in running containers. You can use either method to display the log files or any other file inside the container.

5.3.3 Copying files to and from containers

Sometimes you may want to add a file to a running container or retrieve a file from it. Modifying files in running containers isn't something you normally do - at least not in production - but it can be useful during development.

Kubectl offers the `cp` command to copy files or directories from your local computer to a container of any pod or from the container to your computer. For example, if you'd like to modify the HTML file that the `kiada` pod serves, you can use the following command to copy it to your local file system:

```
$ kubectl cp kiada:html/index.html /tmp/index.html
```

This command copies the `/html/index.html` file from the pod named `kiada` to the `/tmp/index.html` file on your computer. You can now edit the file locally. Once you're happy with the changes, copy the file back to the container with the following command:

```
$ kubectl cp /tmp/index.html kiada:html/
```

Hitting refresh **in** your browser should now include the changes you've made.

Note

The `kubectl cp` command requires the `tar` binary to be present in your container, but this requirement may change in the future.

5.3.4 Executing commands in running containers

When debugging an application running in a container, it may be necessary to examine the container and its environment from the inside. `Kubectl` provides this functionality, too. You can execute any binary file present in the container's file system using the `kubectl exec` command.

Invoking a single command in the container

For example, you can list the processes running in the container in the `kiada` pod by running the following command:

```
$ kubectl exec kiada -- ps aux
USER  PID %CPU %MEM    VSZ   RSS TTY  STAT  START  TIME  COMMAND
root    1  0.0  1.3 812860 27356 ?    Ssl   11:54   0:00 node app.js
root  120  0.0  0.1  17500  2128 ?     Rs    12:22   0:00 ps aux
```

This is the Kubernetes equivalent of the `Docker` command you used to explore the processes in a running container in chapter 2. It allows you to remotely run a command in any pod without having to log in to the node that hosts the pod. If you've used `ssh` to execute commands on a remote system, you'll see that `kubectl exec` is not much different.

In section 5.3.1 you executed the `curl` command in a one-off client pod to send a request to your application, but you can also run the command inside the `kiada` pod itself:

```
$ kubectl exec kiada -- curl -s localhost:8080
Kiada version 0.1. Request processed by "kiada". Client IP: ::1
```

Why use a double dash in the `kubectl exec` command?

The double dash (`--`) in the command delimits `kubectl` arguments from the command to be executed in the container. The use of the double dash isn't necessary if the command has no arguments that begin with a dash. If you omit the double dash in the previous example, the `-s` option is interpreted as

an option for `kubectl exec` and results in the following misleading error:

```
$ kubectl exec kiada curl -s localhost:8080
The connection to the server localhost:8080 was refused - did you
```

This may look like the Node.js server is refusing to accept the connection, but the issue lies elsewhere. The `curl` command is never executed. The error is reported by `kubectl` itself when it tries to talk to the Kubernetes API server at `localhost:8080`, which isn't where the server is. If you run the `kubectl options` command, you'll see that the `-s` option can be used to specify the address and port of the Kubernetes API server. Instead of passing that option to `curl`, `kubectl` adopted it as its own. Adding the double dash prevents this.

Fortunately, to prevent scenarios like this, newer versions of `kubectl` are set to return an error if you forget the double dash.

Running an interactive shell in the container

The two previous examples showed how a single command can be executed in the container. When the command completes, you are returned to your shell. If you want to run several commands in the container, you can run a shell in the container as follows:

```
$ kubectl exec -it kiada -- bash
root@kiada:/#
```

The `-it` is short for two options: `-i` and `-t`, which indicate that you want to execute the `bash` command interactively by passing the standard input to the container and marking it as a terminal (TTY).

You can now explore the inside of the container by executing commands in the shell. For example, you can view the files in the container by running `ls -la`, view its network interfaces with `ip link`, or test its connectivity with `ping`. You can run any tool available in the container.

Not all containers allow you to run shells

The container image of your application contains many important debugging

tools, but this isn't the case with every container image. To keep images small and improve security in the container, most containers used in production don't contain any binary files other than those required for the container's primary process. This significantly reduces the attack surface, but also means that you can't run shells or other tools in production containers. Fortunately, a new Kubernetes feature called *ephemeral containers* allows you to debug running containers by attaching a debug container to them.

Note to MEAP readers

Ephemeral containers are currently an alpha feature, which means they may change or even be removed at any time. This is also why they are currently not explained in this book. If they graduate to beta before the book goes into production, a section explaining them will be added.

5.3.5 Attaching to a running container

The `kubectl attach` command is another way to interact with a running container. It attaches itself to the standard input, output and error streams of the main process running in the container. Normally, you only use it to interact with applications that read from the standard input.

Using `kubectl attach` to see what the application prints to standard output

If the application doesn't read from standard input, the `kubectl attach` command is no more than an alternative way to stream the application logs, as these are typically written to the standard output and error streams, and the `attach` command streams them just like the `kubectl logs -f` command does.

Attach to your `kiada` pod by running the following command:

```
$ kubectl attach kiada
Defaulting container name to kiada.
Use 'kubectl describe pod/kiada -n default' to see all of the con
If you don't see a command prompt, try pressing enter.
```

Now, when you send new HTTP requests to the application using `curl` in another terminal, you'll see the lines that the application logs to standard output also printed in the terminal where the `kubectl attach` command is executed.

Using `kubectl attach` to write to the application's standard input

The Kiada application version 0.1 doesn't read from the standard input stream, but you'll find the source code of version 0.2 that does this in the book's code archive. This version allows you to set a status message by writing it to the standard input stream of the application. This status message will be included in the application's response. Let's deploy this version of the application in a new pod and use the `kubectl attach` command to set the status message.

You can find the artifacts required to build the image in the `kiada-0.2/` directory. You can also use the pre-built image `docker.io/luksa/kiada:0.2`. The pod manifest is in the file `Chapter05/pod.kiada-stdin.yaml` and is shown in the following listing. It contains one additional line compared to the previous manifest (this line is highlighted in the listing).

Listing 5.2 Enabling standard input for a container

```
apiVersion: v1
kind: Pod
metadata:
  name: kiada-stdin      #A
spec:
  containers:
    - name: kiada
      image: luksa/kiada:0.2      #B
      stdin: true                #C
      ports:
        - containerPort: 8080
```

As you can see in the listing, if the application running in a pod wants to read from standard input, you must indicate this in the pod manifest by setting the `stdin` field in the container definition to `true`. This tells Kubernetes to

allocate a buffer for the standard input stream, otherwise the application will always receive an EOF when it tries to read from it.

Create the pod from this manifest file with the `kubectl apply` command:

```
$ kubectl apply -f pod.kiada-stdin.yaml
pod/kiada-stdin created
```

To enable communication with the application, use the `kubectl port-forward` command again, but because the local port 8080 is still being used by the previously executed `port-forward` command, you must either terminate it or choose a different local port to forward to the new pod. You can do this as follows:

```
$ kubectl port-forward kiada-stdin 8888:8080
Forwarding from 127.0.0.1:8888 -> 8080
Forwarding from [::1]:8888 -> 8080
```

The command-line argument `8888:8080` instructs the command to forward local port 8888 to the pod's port 8080.

You can now reach the application at <http://localhost:8888>:

```
$ curl localhost:8888
Kiada version 0.2. Request processed by "kiada-stdin". Client IP:
```

Let's set the status message by using `kubectl attach` to write to the standard input stream of the application. Run the following command:

```
$ kubectl attach -i kiada-stdin
```

Note the use of the additional option `-i` in the command. It instructs `kubectl` to pass its standard input to the container.

Note

Like the `kubectl exec` command, `kubectl attach` also supports the `--tty` or `-t` option, which indicates that the standard input is a terminal (TTY), but the container must be configured to allocate a terminal through the `tty` field in the container definition.

You can now enter the status message into the terminal and press the ENTER key. For example, type the following message:

```
This is my custom status message.
```

The application prints the new message to the standard output:

```
Status message set to: This is my custom status message.
```

To see if the application now includes the message in its responses to HTTP requests, re-execute the `curl` command or refresh the page in your web browser:

```
$ curl localhost:8888
Kiada version 0.2. Request processed by "kiada-stdin". Client IP:
This is my custom status message.      #A
```

You can change the status message again by typing another line in the terminal running the `kubectl attach` command. To exit the attach command, press Control-C or the equivalent key.

Note

An additional field in the container definition, `stdinOnce`, determines whether the standard input channel is closed when the attach session ends. It's set to `false` by default, which allows you to use the standard input in every `kubectl attach` session. If you set it to `true`, standard input remains open only during the first session.

5.4 Running multiple containers in a pod

The Kiada application you deployed in section 5.2 only supports HTTP. Let's add TLS support so it can also serve clients over HTTPS. You could do this by adding code to the `app.js` file, but an easier option exists where you don't need to touch the code at all.

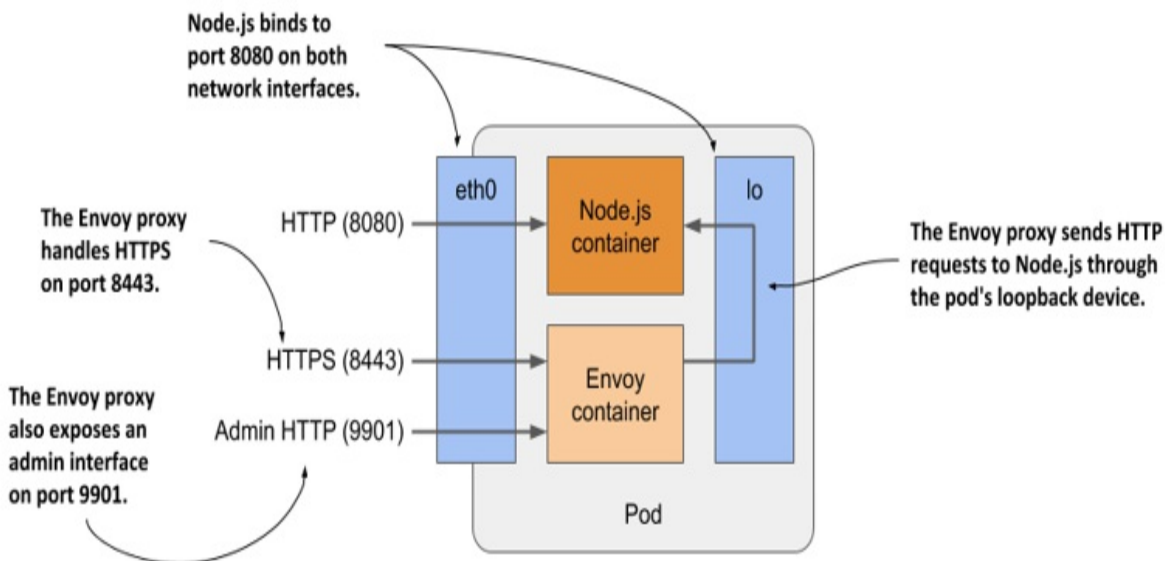
You can run a reverse proxy alongside the Node.js application in a sidecar container, as explained in section 5.1.2, and let it handle HTTPS requests on behalf of the application. A very popular software package that can provide

this functionality is *Envoy*. The Envoy proxy is a high-performance open source service proxy originally built by Lyft that has since been contributed to the Cloud Native Computing Foundation. Let's add it to your pod.

5.4.1 Extending the Kiada Node.js application using the Envoy proxy

Let me briefly explain what the new architecture of the application will look like. As shown in the next figure, the pod will have two containers - the Node.js and the new Envoy container. The Node.js container will continue to handle HTTP requests directly, but the HTTPS requests will be handled by Envoy. For each incoming HTTPS request, Envoy will create a new HTTP request that it will then send to the Node.js application via the local loopback device (via the localhost IP address).

Figure 5.10 Detailed view of the pod's containers and network interfaces



Envoy also provides a web-based administration interface that will prove handy in some of the exercises in the next chapter.

It's obvious that if you implement TLS support within the Node.js application itself, the application will consume less computing resources and have lower latency because no additional network hop is required, but adding

the Envoy proxy could be a faster and easier solution. It also provides a good starting point from which you can add many other features provided by Envoy that you would probably never implement in the application code itself. Refer to the Envoy proxy documentation at envoyproxy.io to learn more.

5.4.2 Adding Envoy proxy to the pod

You'll create a new pod with two containers. You've already got the Node.js container, but you also need a container that will run Envoy.

Creating the Envoy container image

The authors of the proxy have published the official Envoy proxy container image at Docker Hub. You could use this image directly, but you would need to somehow provide the configuration, certificate, and private key files to the Envoy process in the container. You'll learn how to do this in chapter 7. For now, you'll use an image that already contains all three files.

I've already created the image and made it available at `docker.io/luksa/kiada-ssl-proxy:0.1`, but if you want to build it yourself, you can find the files in the `kiada-ssl-proxy-image` directory in the book's code archive.

The directory contains the `Dockerfile`, as well as the private key and certificate that the proxy will use to serve HTTPS. It also contains the `envoy.conf` config file. In it, you'll see that the proxy is configured to listen on port 8443, terminate TLS, and forward requests to port 8080 on `localhost`, which is where the Node.js application is listening. The proxy is also configured to provide an administration interface on port 9901, as explained earlier.

Creating the pod manifest

After building the image, you must create the manifest for the new pod. The following listing shows the contents of the pod manifest file `pod.kiada-ssl.yaml`.

Listing 5.3 Manifest of pod kiada-ssl

```
apiVersion: v1
kind: Pod
metadata:
  name: kiada-ssl
spec:
  containers:
    - name: kiada                                #A
      image: luksa/kiada:0.2                     #A
      ports:                                      #A
        - name: http                             #A
          containerPort: 8080                     #A
    - name: envoy                                #B
      image: luksa/kiada-ssl-proxy:0.1           #B
      ports:                                      #B
        - name: https                             #B
          containerPort: 8443                     #B
        - name: admin                             #B
          containerPort: 9901                     #B
```

The name of this pod is kiada-ssl. It has two containers: kiada and envoy. The manifest is only slightly more complex than the manifest in section 5.2.1. The only new fields are the port names, which are included so that anyone reading the manifest can understand what each port number stands for.

Creating the pod

Create the pod from the manifest using the command `kubectl apply -f pod.kiada-ssl.yaml`. Then use the `kubectl get` and `kubectl describe` commands to confirm that the pod's containers were successfully launched.

5.4.3 Interacting with the two-container pod

When the pod starts, you can start using the application in the pod, inspect its logs and explore the containers from within.

Communicating with the application

As before, you can use the `kubectl port-forward` to enable communication with the application in the pod. Because it exposes three different ports, you

enable forwarding to all three ports as follows:

```
$ kubectl port-forward kiada-ssl 8080 8443 9901
Forwarding from 127.0.0.1:8080 -> 8080
Forwarding from [::1]:8080 -> 8080
Forwarding from 127.0.0.1:8443 -> 8443
Forwarding from [::1]:8443 -> 8443
Forwarding from 127.0.0.1:9901 -> 9901
Forwarding from [::1]:9901 -> 9901
```

First, confirm that you can communicate with the application via HTTP by opening the URL <http://localhost:8080> in your browser or by using curl:

```
$ curl localhost:8080
Kiada version 0.2. Request processed by "kiada-ssl". Client IP: :
```

If this works, you can also try to access the application over HTTPS at <https://localhost:8443>. With curl you can do this as follows:

```
$ curl https://localhost:8443 --insecure
Kiada version 0.2. Request processed by "kiada-ssl". Client IP: :
```

Success! The Envoy proxy handles the task perfectly. Your application now supports HTTPS using a sidecar container.

Why use the --insecure option?

There are two reasons to use the --insecure option when accessing the service. The certificate used by the Envoy proxy is self-signed and was issued for the domain name `example.com`. You're accessing the service through `localhost`, where the local `kubectl` proxy process is listening. Therefore, the hostname doesn't match the name in the server certificate.

To make the names match, you can tell curl to send the request to `example.com`, but resolve it to `127.0.0.1` with the --resolve flag. This will ensure that the certificate matches the requested URL, but since the server's certificate is self-signed, curl will still not accept it as valid. You can fix the problem by telling curl the certificate to use to verify the server with the --cacert flag. The whole command then looks like this:

```
$ curl https://example.com:8443 --resolve example.com:8443:127.0.
```

That's a lot of typing. That's why I prefer to use the `--insecure` option or the shorter `-k` variant.

Displaying logs of pods with multiple containers

The `kiada-ssl` pod contains two containers, so if you want to display the logs, you must specify the name of the container using the `--container` or `-c` option. For example, to view the logs of the `kiada` container, run the following command:

```
$ kubectl logs kiada-ssl -c kiada
```

The Envoy proxy runs in the container named `envoy`, so you display its logs as follows:

```
$ kubectl logs kiada-ssl -c envoy
```

Alternatively, you can display the logs of both containers with the `--all-containers` option:

```
$ kubectl logs kiada-ssl --all-containers
```

You can also combine these commands with the other options explained in section 5.3.2.

Running commands in containers of multi-container pods

If you'd like to run a shell or another command in one of the pod's containers using the `kubectl exec` command, you also specify the container name using the `--container` or `-c` option. For example, to run a shell inside the `envoy` container, run the following command:

```
$ kubectl exec -it kiada-ssl -c envoy -- bash
```

Note

If you don't provide the name, `kubectl exec` defaults to the first container specified in the pod manifest.

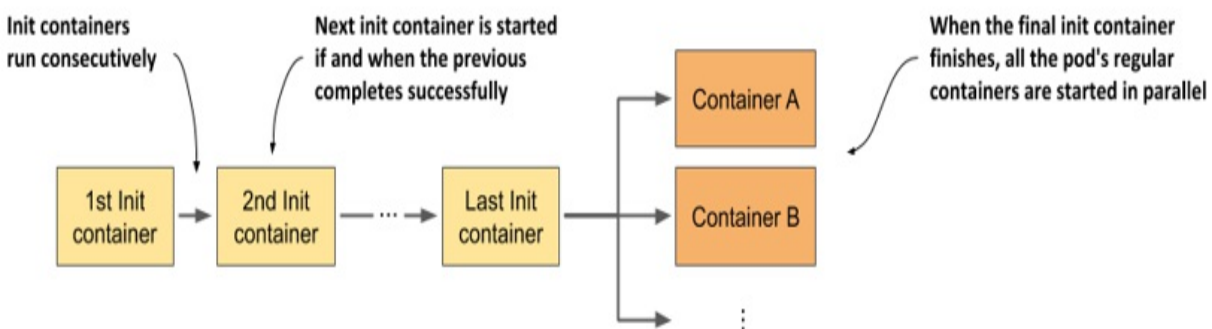
5.5 Running additional containers at pod startup

When a pod contains more than one container, all the containers are started in parallel. Kubernetes doesn't yet provide a mechanism to specify whether a container depends on another container, which would allow you to ensure that one is started before the other. However, Kubernetes allows you to run a sequence of containers to initialize the pod before its main containers start. This special type of container is explained in this section.

5.5.1 Introducing init containers

A pod manifest can specify a list of containers to run when the pod starts and before the pod's normal containers are started. These containers are intended to initialize the pod and are appropriately called *init containers*. As the following figure shows, they run one after the other and must all finish successfully before the main containers of the pod are started.

Figure 5.11 Time sequence showing how a pod's init and regular containers are started



Init containers are like the pod's regular containers, but they don't run in parallel - only one init container runs at a time.

Understanding what init containers can do

Init containers are typically added to pods to achieve the following:

- Initialize files in the volumes used by the pod's main containers. This

includes retrieving certificates and private keys used by the main container from secure certificate stores, generating config files, downloading data, and so on.

- Initialize the pod's networking system. Because all containers of the pod share the same network namespaces, and thus the network interfaces and configuration, any changes made to it by an init container also affect the main container.
- Delay the start of the pod's main containers until a precondition is met. For example, if the main container relies on another service being available before the container is started, an init container can block until this service is ready.
- Notify an external service that the pod is about to start running. In special cases where an external system must be notified when a new instance of the application is started, an init container can be used to deliver this notification.

You could perform these operations in the main container itself but using an init container is sometimes a better option and can have other advantages. Let's see why.

Understanding when moving initialization code to init containers makes sense

Using an init container to perform initialization tasks doesn't require the main container image to be rebuilt and allows a single init container image to be reused with many different applications. This is especially useful if you want to inject the same infrastructure-specific initialization code into all your pods. Using an init container also ensures that this initialization is complete before any of the (possibly multiple) main containers start.

Another important reason is security. By moving tools or data that could be used by an attacker to compromise your cluster from the main container to an init container, you reduce the pod's attack surface.

For example, imagine that the pod must be registered with an external system. The pod needs some sort of secret token to authenticate against this system. If the registration procedure is performed by the main container, this

secret token must be present in its filesystem. If the application running in the main container has a vulnerability that allows an attacker to read arbitrary files on the filesystem, the attacker may be able to obtain this token. By performing the registration from an init container, the token must be available only in the filesystem of the init container, which an attacker can't easily compromise.

5.5.2 Adding init containers to a pod

In a pod manifest, init containers are defined in the `initContainers` field in the `spec` section, just as regular containers are defined in its `containers` field.

Defining init containers in a pod manifest

Let's look at an example of adding two init containers to the `kiada` pod. The first init container emulates an initialization procedure. It runs for 5 seconds, while printing a few lines of text to standard output.

The second init container performs a network connectivity test by using the `ping` command to check if a specific IP address is reachable from within the pod. The IP address is configurable via a command-line argument which defaults to `1.1.1.1`.

Note

An init container that checks if specific IP addresses are reachable could be used to block an application from starting until the services it depends on become available.

You'll find the `Dockerfiles` and other artifacts for both images in the book's code archive, if you want to build them yourself. Alternatively, you can use the images that I've built.

A pod manifest file containing these two init containers is `pod.kiada-init.yaml`. Its contents are shown in the following listing.

Listing 5.4 Defining init containers in a pod manifest

```

apiVersion: v1
kind: Pod
metadata:
  name: kiada-init
spec:
  initContainers:
    - name: init-demo
      image: luksa/init-demo:0.1
    - name: network-check
      image: luksa/network-connectivity-checker:0.1
  containers:
    - name: kiada
      image: luksa/kiada:0.2
      stdin: true
      ports:
        - name: http
          containerPort: 8080
    - name: envoy
      image: luksa/kiada-ssl-proxy:0.1
      ports:
        - name: https
          containerPort: 8443
        - name: admin
          containerPort: 9901

```

As you can see, the definition of an init container is almost trivial. It's sufficient to specify only the name and image for each container.

Note

Container names must be unique within the union of all init and regular containers.

Deploying a pod with init containers

Before you create the pod from the manifest file, run the following command in a separate terminal so you can see how the pod's status changes as the init and regular containers start:

```
$ kubectl get pods -w
```

You'll also want to watch events in another terminal using the following command:

```
$ kubectl get events -w
```

When ready, create the pod by running the apply command:

```
$ kubectl apply -f pod.kiada-init.yaml
```

Inspecting the startup of a pod with init containers

As the pod starts up, inspect the events that are shown by the `kubectl get events -w` command:

TYPE	REASON	MESSAGE	
Normal	Scheduled	Successfully assigned pod to worker2	
Normal	Pulling	Pulling image "luksa/init-demo:0.1"	#A
Normal	Pulled	Successfully pulled image	#A
Normal	Created	Created container init-demo	#A
Normal	Started	Started container init-demo	#A
Normal	Pulling	Pulling image "luksa/network-connec..."	#B
Normal	Pulled	Successfully pulled image	#B
Normal	Created	Created container network-check	#B
Normal	Started	Started container network-check	#B
Normal	Pulled	Container image "luksa/kiada:0.1"	#C
		already present on machine	#C
Normal	Created	Created container kiada	#C
Normal	Started	Started container kiada	#C
Normal	Pulled	Container image "luksa/kiada-ssl-proxy:0.1"	#C
		already present on machine	#C
Normal	Created	Created container envoy	#C
Normal	Started	Started container envoy	#C

The listing shows the order in which the containers are started. The `init-demo` container is started first. When it completes, the `network-check` container is started, and when it completes, the two main containers, `kiada` and `envoy`, are started.

Now inspect the transitions of the pod's status in the other terminal. They should look like this:

NAME	READY	STATUS	RESTARTS	AGE	
kiada-init	0/2	Pending	0	0s	
kiada-init	0/2	Pending	0	0s	
kiada-init	0/2	Init:0/2	0	0s	#A
kiada-init	0/2	Init:0/2	0	1s	#A

kiada-init	0/2	Init:1/2	0	6s	#B
kiada-init	0/2	PodInitializing	0	7s	#C
kiada-init	2/2	Running	0	8s	#D

As the listing shows, when the init containers run, the pod's status shows the number of init containers that have completed and the total number. When all init containers are done, the pod's status is displayed as `PodInitializing`. At this point, the images of the main containers are pulled. When the containers start, the status changes to `Running`.

5.5.3 Inspecting init containers

As with regular containers, you can run additional commands in a running init container using `kubectl exec` and display the logs using `kubectl logs`.

Displaying the logs of an init container

The standard and error output, into which each init container can write, are captured exactly as they are for regular containers. The logs of an init container can be displayed using the `kubectl logs` command by specifying the name of the container with the `-c` option either while the container runs or after it has completed. To display the logs of the `network-check` container in the `kiada-init` pod, run the next command:

```
$ kubectl logs kiada-init -c network-check
Checking network connectivity to 1.1.1.1 ...
Host appears to be reachable
```

The logs show that the `network-check` init container ran without errors. In the next chapter, you'll see what happens if an init container fails.

Entering a running init container

You can use the `kubectl exec` command to run a shell or a different command inside an init container the same way you can with regular containers, but you can only do this before the init container terminates. If you'd like to try this yourself, create a pod from the `pod.kiada-init-slow.yaml` file, which makes the `init-demo` container run for 60 seconds.

When the pod starts, run a shell in the container with the following command:

```
$ kubectl exec -it kiada-init-slow -c init-demo -- sh
```

You can use the shell to explore the container from the inside, but only for a short time. When the container's main process exits after 60 seconds, the shell process is also terminated.

You typically enter a running init container only when it fails to complete in time, and you want to find the cause. During normal operation, the init container terminates before you can run the `kubectl exec` command.

5.6 Deleting pods and other objects

If you've tried the exercises in this chapter and in chapter 2, several pods and other objects now exist in your cluster. To close this chapter, you'll learn various ways to delete them. Deleting a pod will terminate its containers and remove them from the node. Deleting a Deployment object causes the deletion of its pods, whereas deleting a LoadBalancer-typed Service deprovisions the load balancer if one was provisioned.

5.6.1 Deleting a pod by name

The easiest way to delete an object is to delete it by name.

Deleting a single pod

Use the following command to remove the `kiada` pod from your cluster:

```
$ kubectl delete po kiada
pod "kiada" deleted
```

By deleting a pod, you state that you no longer want the pod or its containers to exist. The Kubelet shuts down the pod's containers, removes all associated resources, such as log files, and notifies the API server after this process is complete. The Pod object is then removed.

Tip

By default, the `kubectl delete` command waits until the object no longer exists. To skip the wait, run the command with the `--wait=false` option.

While the pod is in the process of shutting down, its status changes to Terminating:

```
$ kubectl get po kiada
NAME      READY   STATUS    RESTARTS   AGE
kiada     1/1     Terminating    0          35m
```

Knowing exactly how containers are shut down is important if you want your application to provide a good experience for its clients. This is explained in the next chapter, where we dive deeper into the life cycle of the pod and its containers.

Note

If you're familiar with Docker, you may wonder if you can stop a pod and start it again later, as you can with Docker containers. The answer is no. With Kubernetes, you can only remove a pod completely and create it again later.

Deleting multiple pods with a single command

You can also delete multiple pods with a single command. If you ran the `kiada-init` and the `kiada-init-slow` pods, you can delete them both by specifying their names separated by a space, as follows:

```
$ kubectl delete po kiada-init kiada-init-slow
pod "kiada-init" deleted
pod "kiada-init-slow" deleted
```

5.6.2 Deleting objects defined in manifest files

Whenever you create objects from a file, you can also delete them by passing the file to the `delete` command instead of specifying the name of the pod.

Deleting objects by specifying the manifest file

You can delete the `kiada-ssl` pod, which you created from the `pod.kiada-ssl.yaml` file, with the following command:

```
$ kubectl delete -f pod.kiada-ssl.yaml
pod "kiada-ssl" deleted
```

In your case, the file contains only a single pod object, but you'll typically come across files that contain several objects of different types that represent a complete application. This makes deploying and removing the application as easy as executing `kubectl apply -f app.yaml` and `kubectl delete -f app.yaml`, respectively.

Deleting objects from multiple manifest files

Sometimes, an application is defined in several manifest files. You can specify multiple files by separating them with a comma. For example:

```
$ kubectl delete -f pod.kiada.yaml,pod.kiada-ssl.yaml
```

Note

You can also apply several files at the same time using this syntax (for example: `kubectl apply -f pod.kiada.yaml,pod.kiada-ssl.yaml`).

I've never actually used this approach in the many years I've been using Kubernetes, but I often deploy all the manifest files from a file directory by specifying the directory name instead of the names of individual files. For example, you can deploy all the pods you created in this chapter again by running the following command in the base directory of this book's code archive:

```
$ kubectl apply -f Chapter05/
```

This applies to all files in the directory that have the correct file extension (`.yaml`, `.json`, and similar). You can then delete the pods using the same method:

```
$ kubectl delete -f Chapter05/
```

Note

If your manifest files are stored in subdirectories, you must use the `--recursive` flag (or `-R`).

5.6.3 Deleting all pods

You've now removed all pods except `kiada-stdin` and the pods you created in chapter 3 using the `kubectl create deployment` command. Depending on how you've scaled the deployment, some of these pods should still be running:

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
kiada-stdin	1/1	Running	0	10m
kiada-9d785b578-58vhc	1/1	Running	0	1d
kiada-9d785b578-jmnj8	1/1	Running	0	1d

Instead of deleting these pods by name, we can delete them all using the `--all` option:

```
$ kubectl delete po --all
pod "kiada-stdin" deleted
pod "kiada-9d785b578-58vhc" deleted
pod "kiada-9d785b578-jmnj8" deleted
```

Now confirm that no pods exist by executing the `kubectl get pods` command again:

```
$ kubectl get po
```

NAME	READY	STATUS	RESTARTS	AGE
kiada-9d785b578-cc6tk	1/1	Running	0	13s
kiada-9d785b578-h4gml	1/1	Running	0	13s

That was unexpected! Two pods are still running. If you look closely at their names, you'll see that these aren't the two you've just deleted. The `AGE` column also indicates that these are *new* pods. You can try to delete them as well, but you'll see that no matter how often you delete them, new pods are created to replace them.

The reason why these pods keep popping up is because of the Deployment

object. The controller responsible for bringing Deployment objects to life must ensure that the number of pods always matches the desired number of replicas specified in the object. When you delete a pod associated with the Deployment, the controller immediately creates a replacement pod.

To delete these pods, you must either scale the Deployment to zero or delete the object altogether. This would indicate that you no longer want this deployment or its pods to exist in your cluster.

5.6.4 Deleting objects using the “all” keyword

You can delete everything you’ve created so far - including the deployment, its pods, and the service - with the following command:

```
$ kubectl delete all --all
pod "kiada-9d785b578-cc6tk" deleted
pod "kiada-9d785b578-h4gml" deleted
service "kubernetes" deleted
service "kiada" deleted
deployment.apps "kiada" deleted
replicaset.apps "kiada-9d785b578" deleted
```

The first `all` in the command indicates that you want to delete objects of all types. The `--all` option indicates that you want to delete all instances of each object type. You used this option in the previous section when you tried to delete all pods.

When deleting objects, `kubectl` prints the type and name of each deleted object. In the previous listing, you should see that it deleted the pods, the deployment, and the service, but also a so-called replica set object. You’ll learn what this is in chapter 11, where we take a closer look at deployments.

You’ll notice that the delete command also deletes the built-in `kubernetes` service. Don’t worry about this, as the service is automatically recreated after a few moments.

Certain objects aren’t deleted when using this method, because the keyword `all` does not include all object kinds. This is a precaution to prevent you from accidentally deleting objects that contain important information. The Event

object kind is one example of this.

Note

You can specify multiple object types in the `delete` command. For example, you can use `kubectl delete events,all --all` to delete events along with all object kinds included in `all`.

5.7 Summary

In this chapter, you've learned:

- Pods run one or more containers as a co-located group. They are the unit of deployment and horizontal scaling. A typical container runs only one process. Sidecar containers complement the primary container in the pod.
- Containers should only be part of the same pod if they must run together. A frontend and a backend process should run in separate pods. This allows them to be scaled individually.
- When a pod starts, its init containers run one after the other. When the last init container completes, the pod's main containers are started. You can use an init container to configure the pod from within, delay startup of its main containers until a precondition is met, or notify an external service that the pod is about to start running.
- The `kubectl` tool is used to create pods, view their logs, copy files to/from their containers, execute commands in those containers and enable communication with individual pods during development.

In the next chapter, you'll learn about the lifecycle of the pod and its containers.

6 Manging the Pod lifecycle

This chapter covers

- Inspecting the pod's status
- Keeping containers healthy using liveness probes
- Using lifecycle hooks to perform actions at container startup and shutdown
- Understanding the complete lifecycle of the pod and its containers

After reading the previous chapter, you should be able to deploy, inspect and communicate with pods containing one or more containers. In this chapter, you'll gain a much deeper understanding of how the pod and its containers operate.

Note

You'll find the code files for this chapter at <https://github.com/luksa/kubernetes-in-action-2nd-edition/tree/master/Chapter06>

6.1 Understanding the pod's status

After you create a pod object and it runs, you can see what's going on with the pod by reading the pod object back from the API. As you've learned in chapter 4, the pod object manifest, as well as the manifests of most other kinds of objects, contain a section, which provides the status of the object. A pod's status section contains the following information:

- the IP addresses of the pod and the worker node that hosts it
- when the pod was started
- the pod's quality-of-service (QoS) class
- what phase the pod is in,
- the conditions of the pod, and

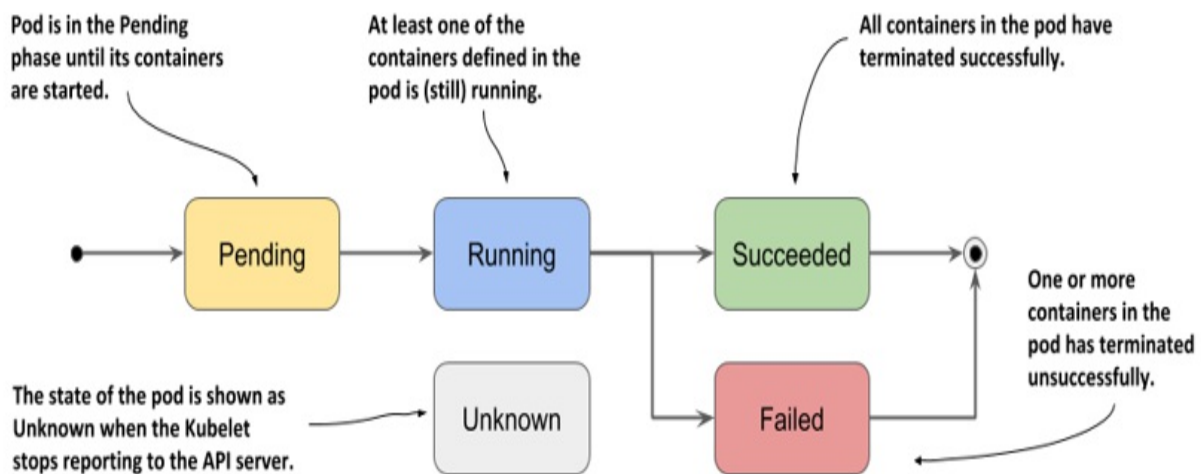
- the state of its individual containers.

The IP addresses and the start time don't need any further explanation, and the QoS class isn't relevant now - you'll learn about it in chapter 19. However, the phase and conditions of the pod, as well as the states of its containers are important for you to understand the pod lifecycle.

6.1.1 Understanding the pod phase

In any moment of the pod's life, it's in one of the five phases shown in the following figure.

Figure 6.1 The phases of a Kubernetes pod



The meaning of each phase is explained in the following table.

Table 6.1 List of phases a pod can be in

Pod Phase	Description
Pending	After you create the Pod object, this is its initial phase. Until the pod is scheduled to a node and the images of its containers are pulled and started, it remains in this phase.

Running	At least one of the pod's containers is running.
Succeeded	Pods that aren't intended to run indefinitely are marked as Succeeded when all their containers complete successfully.
Failed	When a pod is not configured to run indefinitely and at least one of its containers terminates unsuccessfully, the pod is marked as Failed.
Unknown	The state of the pod is unknown because the Kubelet has stopped reporting communicating with the API server. Possibly the worker node has failed or has disconnected from the network.

The pod's phase provides a quick summary of what's happening with the pod. Let's deploy the kiada pod again and inspect its phase. Create the pod by applying the manifest file to your cluster again, as in the previous chapter (you'll find it in `Chapter06/pod.kiada.yaml`):

```
$ kubectl apply -f pod.kiada.yaml
```

Displaying a pod's phase

The pod's phase is one of the fields in the pod object's status section. You can see it by displaying its manifest and optionally grepping the output to search for the field:

```
$ kubectl get po kiada -o yaml | grep phase
phase: Running
```

Tip

Remember the `jq` tool? You can use it to print out the value of the phase field like this: `kubectl get po kiada -o json | jq .status.phase`

You can also see the pod's phase using `kubectl describe`. The pod's status is shown close to the top of the output.

```
$ kubectl describe po kiada
Name:          kiada
Namespace:     default
...
Status:        Running
...
```

Although it may appear that the `STATUS` column displayed by `kubectl get pods` also shows the phase, this is only true for pods that are healthy:

```
$ kubectl get po kiada
NAME      READY   STATUS    RESTARTS   AGE
kiada     1/1     Running   0           40m
```

For unhealthy pods, the `STATUS` column indicates what's wrong with the pod. You'll see this later in this chapter.

6.1.2 Understanding pod conditions

The phase of a pod says little about the condition of the pod. You can learn more by looking at the pod's list of conditions, just as you did for the node object in chapter 4. A pod's conditions indicate whether a pod has reached a certain state or not, and why that's the case.

In contrast to the phase, a pod has several conditions at the same time. Four condition *types* are known at the time of writing. They are explained in the following table.

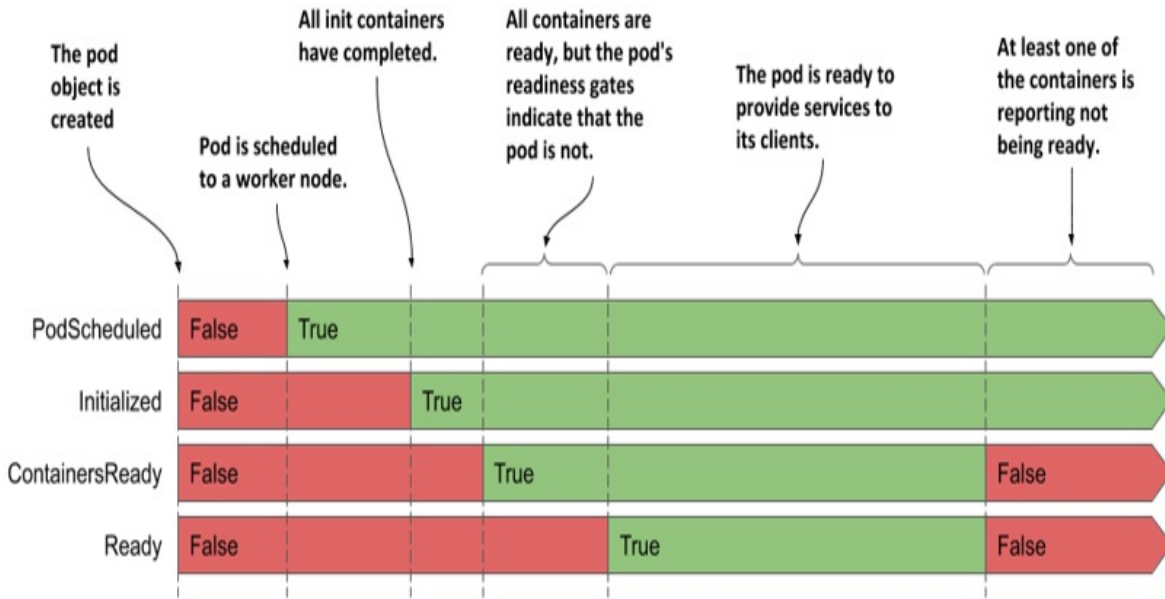
Table 6.2 List of pod conditions

Pod Condition	Description

PodScheduled	Indicates whether or not the pod has been scheduled to a node.
Initialized	The pod's init containers have all completed successfully.
ContainersReady	All containers in the pod indicate that they are ready. This is a necessary but not sufficient condition for the entire pod to be ready.
Ready	The pod is ready to provide services to its clients. The containers in the pod and the pod's readiness gates are all reporting that they are ready. Note: this is explained in chapter 10.

Each condition is either fulfilled or not. As you can see in the following figure, the PodScheduled and Initialized conditions start as unfulfilled, but are soon fulfilled and remain so throughout the life of the pod. In contrast, the Ready and ContainersReady conditions can change many times during the pod's lifetime.

Figure 6.2 The transitions of the pod's conditions during its lifecycle



Do you remember the conditions you can find in a node object? They are `MemoryPressure`, `DiskPressure`, `PIDPressure` and `Ready`. As you can see, each object has its own set of condition types, but many contain the generic `Ready` condition, which typically indicates whether everything is fine with the object.

Inspecting the pod's conditions

To see the conditions of a pod, you can use `kubectl describe` as shown here:

```
$ kubectl describe po kiada
...
Conditions:
  Type                Status
  Initialized          True    #A
  Ready               True    #B
  ContainersReady     True    #B
  PodScheduled        True    #C
...
```

The `kubectl describe` command shows only whether each condition is true or not. To find out why a condition is false, you must look for the `status.conditions` field in the pod manifest as follows:

```
$ kubectl get po kiada -o json | jq .status.conditions
[
  {
    "lastProbeTime": null,
    "lastTransitionTime": "2020-02-02T11:42:59Z",
    "status": "True",
    "type": "Initialized"
  },
  ...
]
```

Each condition has a `status` field that indicates whether the condition is `True`, `False` or `Unknown`. In the case of the `kiada` pod, the status of all conditions is `True`, which means they are all fulfilled. The condition can also contain a `reason` field that specifies a machine-facing reason for the last change of the condition's status, and a `message` field that explains the change in detail. The `lastTransitionTime` field shows when the change occurred, while the `lastProbeTime` indicates when this condition was last checked.

6.1.3 Understanding the container status

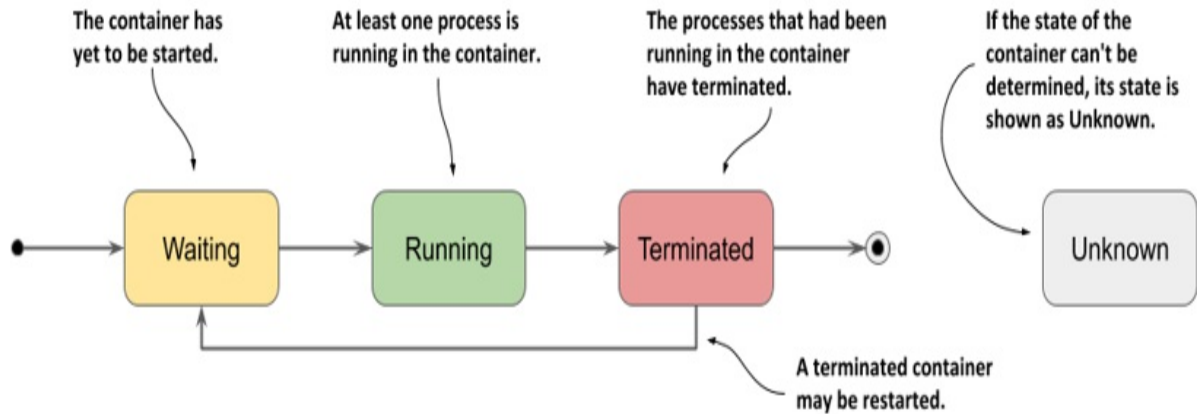
Also contained in the status of the pod is the status of each of its containers. Inspecting the status provides better insight into the operation of each individual container.

The status contains several fields. The `state` field indicates the container's current state, whereas the `lastState` field shows the state of the previous container after it has terminated. The container status also indicates the internal ID of the container (`containerID`), the `image` and `imageID` the container is running, whether the container is ready or not and how often it has been restarted (`restartCount`).

Understanding the container state

The most important part of a container's status is its state. A container can be in one of the states shown in the following figure.

Figure 6.3 The possible states of a container



Individual states are explained in the following table.

Table 6.3 Possible container states

Container State	Description
Waiting	The container is waiting to be started. The reason and message fields indicate why the container is in this state.
Running	The container has been created and processes are running in it. The <code>startedAt</code> field indicates the time at which this container was started.
Terminated	The processes that had been running in the container have terminated. The <code>startedAt</code> and <code>finishedAt</code> fields indicate when the container was started and when it terminated. The exit code with which the main process terminated is in the <code>exitCode</code> field.
Unknown	The state of the container couldn't be determined.

Displaying the status of the pod's containers

The pod list displayed by `kubectl get pods` shows only the number of containers in each pod and how many of them are ready. To see the status of individual containers, you can use `kubectl describe`:

```
$ kubectl describe po kiada
...
Containers:
  kiada:
    Container ID:   docker://c64944a684d57faacfced0be1af44686...
    Image:          luksa/kiada:0.1
    Image ID:       docker-pullable://luksa/kiada@sha256:3f28...
    Port:          8080/TCP
    Host Port:      0/TCP
    State:          Running      #A
      Started:      Sun, 02 Feb 2020 12:43:03 +0100      #A
    Ready:          True        #B
    Restart Count:  0          #C
    Environment:    <none>
  ...
```

Focus on the annotated lines in the listing, as they indicate whether the container is healthy. The kiada container is Running and is Ready. It has never been restarted.

Tip

You can also display the container status(es) using `jq` like this: `kubectl get po kiada -o json | jq .status.containerStatuses`

Inspecting the status of an init container

In the previous chapter, you learned that in addition to regular containers, a pod can also have init containers that run when the pod starts. As with regular containers, the status of these containers is available in the status section of the pod object manifest, but in the `initContainerStatuses` field.

Inspecting the status of the kiada-init pod

As an additional exercise you can try on your own, create the `kiada-init` pod from the previous chapter and inspect its phase, conditions and the status of its two regular and two init containers. Use the `kubectl describe` command and the `kubectl get po kiada-init -o json | jq .status` command to find the information in the object definition.

6.2 Keeping containers healthy

The pods you created in the previous chapter ran without any problems. But what if one of the containers dies? What if all the containers in a pod die? How do you keep the pods healthy and their containers running? That's the focus of this section.

6.2.1 Understanding container auto-restart

When a pod is scheduled to a node, the Kubelet on that node starts its containers and from then on keeps them running for as long as the pod object exists. If the main process in the container terminates for any reason, the Kubelet restarts the container. If an error in your application causes it to crash, Kubernetes automatically restarts it, so even without doing anything special in the application itself, running it in Kubernetes automatically gives it the ability to heal itself. Let's see this in action.

Observing a container failure

In the previous chapter, you created the `kiada-ssl` pod, which contains the `Node.js` and the `Envoy` containers. Create the pod again and enable communication with the pod by running the following two commands:

```
$ kubectl apply -f pod.kiada-ssl.yaml
$ kubectl port-forward kiada-ssl 8080 8443 9901
```

You'll now cause the `Envoy` container to terminate to see how Kubernetes deals with the situation. Run the following command in a separate terminal so you can see how the pod's status changes when one of its containers terminates:

```
$ kubectl get pods -w
```

You'll also want to watch events in another terminal using the following command:

```
$ kubectl get events -w
```

You could emulate a crash of the container's main process by sending it the `KILL` signal, but you can't do this from inside the container because the Linux Kernel doesn't let you kill the root process (the process with PID 1). You would have to SSH to the pod's host node and kill the process from there. Fortunately, Envoy's administration interface allows you to stop the process via its HTTP API.

To terminate the envoy container, open the URL <http://localhost:9901> in your browser and click the *quitquitquit* button or run the following `curl` command in another terminal:

```
$ curl -X POST http://localhost:9901/quitquitquit
OK
```

To see what happens with the container and the pod it belongs to, examine the output of the `kubectl get pods -w` command you ran earlier. This is its output:

```
$ kubectl get po -w
```

NAME	READY	STATUS	RESTARTS	AGE
kiada-ssl	2/2	Running	0	1s
kiada-ssl	1/2	NotReady	0	9m33s
kiada-ssl	2/2	Running	1	9m34s

The listing shows that the pod's `STATUS` changes from `Running` to `NotReady`, while the `READY` column indicates that only one of the two containers is ready. Immediately thereafter, Kubernetes restarts the container and the pod's status returns to `Running`. The `RESTARTS` column indicates that one container has been restarted.

Note

If one of the pod's containers fails, the other containers continue to run.

Now examine the output of the `kubectl get events -w` command you ran earlier. Here's the command and its output:

```
$ kubectl get ev -w
LAST SEEN   TYPE      REASON      OBJECT          MESSAGE
0s          Normal    Pulled       pod/kiada-ssl   Container imag
0s          Normal    Created      pod/kiada-ssl   present on mac
0s          Normal    Started      pod/kiada-ssl   Created contai
0s          Normal    Started      pod/kiada-ssl   Started contai
```

The events show that the new envoy container has been started. You should be able to access the application via HTTPS again. Please confirm with your browser or `curl`.

The events in the listing also expose an important detail about how Kubernetes restarts containers. The second event indicates that the entire envoy container has been recreated. Kubernetes never restarts a container, but instead discards it and creates a new container. Regardless, we call this *restarting* a container.

Note

Any data that the process writes to the container's filesystem is lost when the container is recreated. This behavior is sometimes undesirable. To persist data, you must add a storage volume to the pod, as explained in the next chapter.

Note

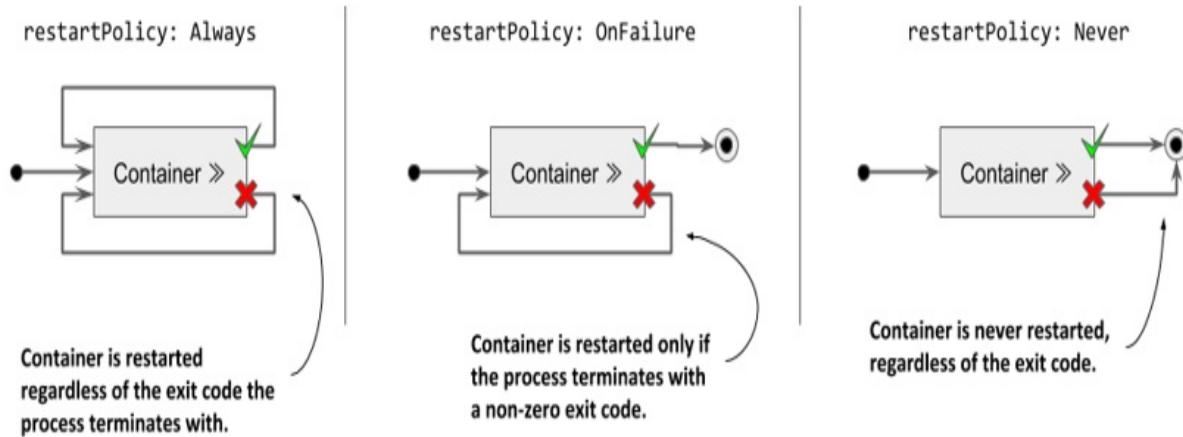
If init containers are defined in the pod and one of the pod's regular containers is restarted, the init containers are not executed again.

Configuring the pod's restart policy

By default, Kubernetes restarts the container regardless of whether the process in the container exits with a zero or non-zero exit code - in other words, whether the container completes successfully or fails. This behavior can be changed by setting the `restartPolicy` field in the pod's spec.

Three restart policies exist. They are explained in the following figure.

Figure 6.4 The pod's restartPolicy determines whether its containers are restarted or not



The following table describes the three restart policies.

Table 6.4 Pod restart policies

Restart Policy	Description
Always	Container is restarted regardless of the exit code the process in the container terminates with. This is the default restart policy.
OnFailure	The container is restarted only if the process terminates with a non-zero exit code, which by convention indicates failure.
Never	The container is never restarted - not even when it fails.

Note

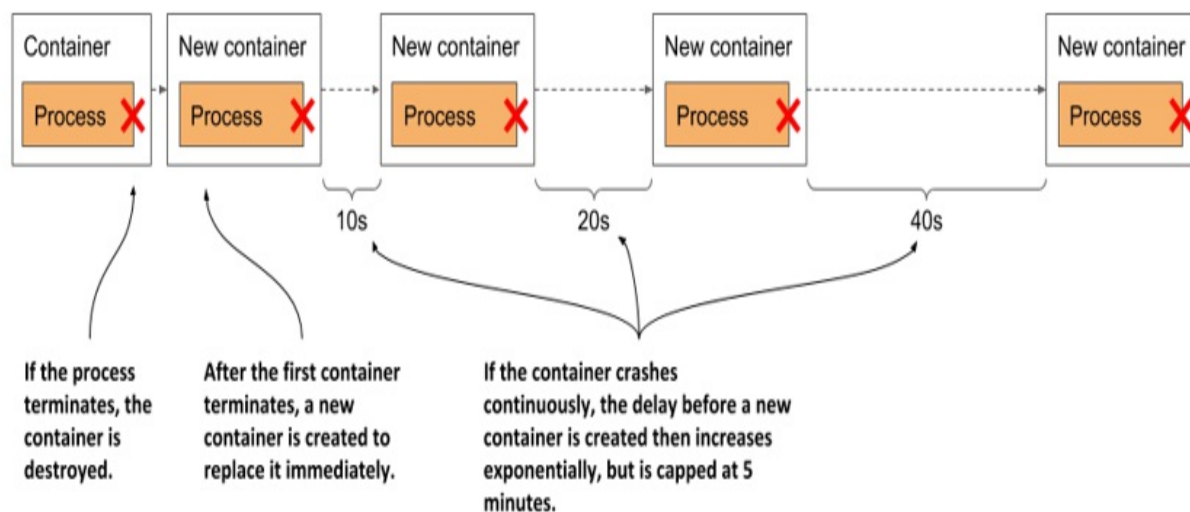
Surprisingly, the restart policy is configured at the pod level and applies to all its containers. It can't be configured for each container individually.

Understanding the time delay inserted before a container is restarted

If you call Envoy's `/quitquitquit` endpoint several times, you'll notice that each time it takes longer to restart the container after it terminates. The pod's status is displayed as either `NotReady` or `CrashLoopBackOff`. Here's what it means.

As shown in the following figure, the first time a container terminates, it is restarted immediately. The next time, however, Kubernetes waits ten seconds before restarting it again. This delay is then doubled to 20, 40, 80 and then to 160 seconds after each subsequent termination. From then on, the delay is kept at five minutes. This delay that doubles between attempts is called exponential back-off.

Figure 6.5 Exponential back-off between container restarts



In the worst case, a container can therefore be prevented from starting for up to five minutes.

Note

The delay is reset to zero when the container has run successfully for 10 minutes. If the container must be restarted later, it is restarted immediately.

Check the container status in the pod manifest as follows:

```
$ kubectl get po kiada-ssl -o json | jq .status.containerStatuses
...
"state": {
  "waiting": {
    "message": "back-off 40s restarting failed container=envoy ..
    "reason": "CrashLoopBackOff"
```

As you can see in the output, while the container is waiting to be restarted, its state is `waiting`, and the reason is `CrashLoopBackOff`. The message field tells you how long it will take for the container to be restarted.

Note

When you tell Envoy to terminate, it terminates with exit code zero, which means it hasn't crashed. The `CrashLoopBackOff` status can therefore be misleading.

6.2.2 Checking the container's health using liveness probes

In the previous section, you learned that Kubernetes keeps your application healthy by restarting it when its process terminates. But applications can also become unresponsive without terminating. For example, a Java application with a memory leak eventually starts spewing out `OutOfMemoryErrors`, but its JVM process continues to run. Ideally, Kubernetes should detect this kind of error and restart the container.

The application could catch these errors by itself and immediately terminate, but what about the situations where your application stops responding because it gets into an infinite loop or deadlock? What if the application can't detect this? To ensure that the application is restarted in such cases, it may be necessary to check its state from the outside.

Introducing liveness probes

Kubernetes can be configured to check whether an application is still alive by defining a *liveness probe*. You can specify a liveness probe for each container in the pod. Kubernetes runs the probe periodically to ask the application if it's still alive and well. If the application doesn't respond, an error occurs, or the response is negative, the container is considered unhealthy and is terminated. The container is then restarted if the restart policy allows it.

Note

Liveness probes can only be used in the pod's regular containers. They can't be defined in init containers.

Types of liveness probes

Kubernetes can probe a container with one of the following three mechanisms:

- An *HTTP GET* probe sends a GET request to the container's IP address, on the network port and path you specify. If the probe receives a response, and the response code doesn't represent an error (in other words, if the HTTP response code is 2xx or 3xx), the probe is considered successful. If the server returns an error response code, or if it doesn't respond in time, the probe is considered to have failed.
- A *TCP Socket* probe attempts to open a TCP connection to the specified port of the container. If the connection is successfully established, the probe is considered successful. If the connection can't be established in time, the probe is considered failed.
- An *Exec* probe executes a command inside the container and checks the exit code it terminates with. If the exit code is zero, the probe is successful. A non-zero exit code is considered a failure. The probe is also considered to have failed if the command fails to terminate in time.

Note

In addition to a liveness probe, a container may also have a *startup* probe, which is discussed in section 6.2.6, and a *readiness* probe, which is explained in chapter 10.

6.2.3 Creating an HTTP GET liveness probe

Let's look at how to add a liveness probe to each of the containers in the `kiada-ssl` pod. Because they both run applications that understand HTTP, it makes sense to use an HTTP GET probe in each of them. The Node.js application doesn't provide any endpoints to explicitly check the health of the application, but the Envoy proxy does. In real-world applications, you'll encounter both cases.

Defining liveness probes in the pod manifest

The following listing shows an updated manifest for the pod, which defines a liveness probe for each of the two containers, with different levels of configuration (file `pod.kiada-liveness.yaml`).

Listing 6.1 Adding a liveness probe to a pod

```
apiVersion: v1
kind: Pod
metadata:
  name: kiada-liveness
spec:
  containers:
    - name: kiada
      image: luksa/kiada:0.1
      ports:
        - name: http
          containerPort: 8080
      livenessProbe: #A
        httpGet: #A
          path: / #A
          port: 8080 #A
    - name: envoy
      image: luksa/kiada-ssl-proxy:0.1
      ports:
        - name: https
          containerPort: 8443
        - name: admin
          containerPort: 9901
      livenessProbe: #B
        httpGet: #B
          path: /ready #B
```



```
port: admin      #B
initialDelaySeconds: 10    #B
periodSeconds: 5      #B
timeoutSeconds: 2       #B
failureThreshold: 3      #B
```

These liveness probes are explained in the next two sections.

Defining a liveness probe using the minimum required configuration

The liveness probe for the kiada container is the simplest version of a probe for HTTP-based applications. The probe simply sends an HTTP GET request for the path / on port 8080 to determine if the container can still serve requests. If the application responds with an HTTP status between 200 and 399, the application is considered healthy.

The probe doesn't specify any other fields, so the default settings are used. The first request is sent 10s after the container starts and is repeated every 5s. If the application doesn't respond within two seconds, the probe attempt is considered failed. If it fails three times in a row, the container is considered unhealthy and is terminated.

Understanding liveness probe configuration options

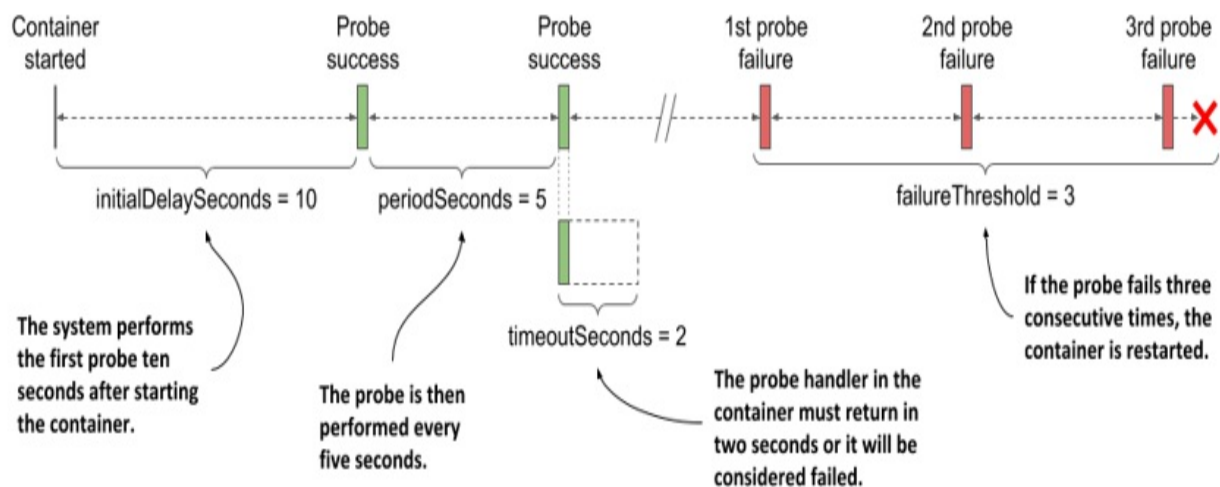
The administration interface of the Envoy proxy provides the special endpoint /ready through which it exposes its health status. Instead of targeting port 8443, which is the port through which Envoy forwards HTTPS requests to Node.js, the liveness probe for the envoy container targets this special endpoint on the admin port, which is port number 9901.

Note

As you can see in the envoy container's liveness probe, you can specify the probe's target port by name instead of by number.

The liveness probe for the envoy container also contains additional fields. These are best explained with the following figure.

Figure 6.6 The configuration and operation of a liveness probe



The parameter `initialDelaySeconds` determines how long Kubernetes should delay the execution of the first probe after starting the container. The `periodSeconds` field specifies the amount of time between the execution of two consecutive probes, whereas the `timeoutSeconds` field specifies how long to wait for a response before the probe attempt counts as failed. The `failureThreshold` field specifies how many times the probe must fail for the container to be considered unhealthy and potentially restarted.

6.2.4 Observing the liveness probe in action

To see Kubernetes restart a container when its liveness probe fails, create the pod from the `pod.kiada-liveness.yaml` manifest file using `kubectl apply`, and run `kubectl port-forward` to enable communication with the pod. You'll need to stop the `kubectl port-forward` command still running from the previous exercise. Confirm that the pod is running and is responding to HTTP requests.

Observing a successful liveness probe

The liveness probes for the pod's containers starts firing soon after the start of each individual container. Since the processes in both containers are healthy, the probes continuously report success. As this is the normal state,

the fact that the probes are successful is not explicitly indicated anywhere in the status of the pod nor in its events.

The only indication that Kubernetes is executing the probe is found in the container logs. The Node.js application in the kiada container prints a line to the standard output every time it handles an HTTP request. This includes the liveness probe requests, so you can display them using the following command:

```
$ kubectl logs kiada-liveness -c kiada -f
```

The liveness probe for the envoy container is configured to send HTTP requests to Envoy's administration interface, which doesn't log HTTP requests to the standard output, but to the file `/tmp/envoy.admin.log` in the container's filesystem. To display the log file, you use the following command:

```
$ kubectl exec kiada-liveness -c envoy -- tail -f /tmp/envoy.admi
```

Observing the liveness probe fail

A successful liveness probe isn't interesting, so let's cause Envoy's liveness probe to fail. To see what will happen behind the scenes, start watching events by executing the following command in a separate terminal:

```
$ kubectl get events -w
```

Using Envoy's administration interface, you can configure its health check endpoint to succeed or fail. To make it fail, open URL <http://localhost:9901> in your browser and click the *healthcheck/fail* button, or use the following curl command:

```
$ curl -X POST localhost:9901/healthcheck/fail
```

Immediately after executing the command, observe the events that are displayed in the other terminal. When the probe fails, a warning event is recorded, indicating the error and the HTTP status code returned:

```
Warning   Unhealthy   Liveness probe failed: HTTP probe failed with
```

Because the probe's `failureThreshold` is set to three, a single failure is not enough to consider the container unhealthy, so it continues to run. You can make the liveness probe succeed again by clicking the *healthcheck/ok* button in Envoy's admin interface, or by using `curl` as follows:

```
$ curl -X POST localhost:9901/healthcheck/ok
```

If you are fast enough, the container won't be restarted.

Observing the liveness probe reach the failure threshold

If you let the liveness probe fail multiple times, the `kubectl get events -w` command should print the following events (note that some columns are omitted due to page width constraints):

```
$ kubectl get events -w
TYPE          REASON      MESSAGE
Warning       Unhealthy   Liveness probe failed: HTTP probe failed with
Warning       Unhealthy   Liveness probe failed: HTTP probe failed with
Warning       Unhealthy   Liveness probe failed: HTTP probe failed with
Normal        Killing     Container envoy failed liveness probe, will b
Normal        Pulled      Container image already present on machine
Normal        Created     Created container envoy
Normal        Started     Started container envoy
```

Remember that the probe failure threshold is set to 3, so when the probe fails three times in a row, the container is stopped and restarted. This is indicated by the events in the listing.

The `kubectl get pods` command shows that the container has been restarted:

```
$ kubectl get po kiada-liveness
NAME                READY   STATUS    RESTARTS   AGE
kiada-liveness      2/2     Running   1           5m
```

The `RESTARTS` column shows that one container restart has taken place in the pod.

Understanding how a container that fails its liveness probe is restarted

If you're wondering whether the main process in the container was gracefully stopped or killed forcibly, you can check the pod's status by retrieving the full manifest using `kubectl get` or using `kubectl describe`:

```
$ kubectl describe po kiada-liveness
Name:                kiada-liveness
...
Containers:
  envoy:
    ...
    State:           Running      #A
      Started:       Sun, 31 May 2020 21:33:13 +0200      #A
    Last State:      Terminated   #B
      Reason:        Completed     #B
      Exit Code:     0             #B
      Started:       Sun, 31 May 2020 21:16:43 +0200      #B
      Finished:      Sun, 31 May 2020 21:33:13 +0200      #B
    ...
```

The exit code zero shown in the listing implies that the application process gracefully exited on its own. If it had been killed, the exit code would have been 137.

Note

Exit code $128+n$ indicates that the process exited due to external signal n . Exit code 137 is $128+9$, where 9 represents the `KILL` signal. You'll see this exit code whenever the container is killed. Exit code 143 is $128+15$, where 15 is the `TERM` signal. You'll typically see this exit code when the container runs a shell that has terminated gracefully.

Examine Envoy's log to confirm that it caught the `TERM` signal and has terminated by itself. You must use the `kubectl logs` command with the `--container` or the shorter `-c` option to specify what container you're interested in.

Also, because the container has been replaced with a new one due to the restart, you must request the log of the previous container using the `--previous` or `-p` flag. Here's the command to use and the last four lines of its output:

```
$ kubectl logs kiada-liveness -c envoy -p
...
...[warning][main] [source/server/server.cc:493] caught SIGTERM
...[info][main] [source/server/server.cc:613] shutting down serve
...[info][main] [source/server/server.cc:560] main dispatch loop
...[info][main] [source/server/server.cc:606] exiting
```

The log confirms that Kubernetes sent the TERM signal to the process, allowing it to shut down gracefully. Had it not terminated by itself, Kubernetes would have killed it forcibly.

After the container is restarted, its health check endpoint responds with HTTP status 200 OK again, indicating that the container is healthy.

6.2.5 Using the exec and the tcpSocket liveness probe types

For applications that don't expose HTTP health-check endpoints, the tcpSocket or the exec liveness probes should be used.

Adding a tcpSocket liveness probe

For applications that accept non-HTTP TCP connections, a tcpSocket liveness probe can be configured. Kubernetes tries to open a socket to the TCP port and if the connection is established, the probe is considered a success, otherwise it's considered a failure.

An example of a tcpSocket liveness probe is shown here:

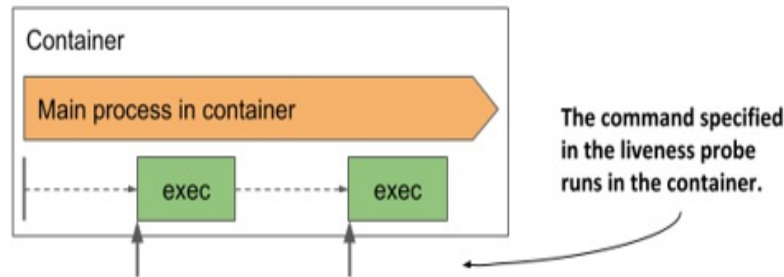
```
livenessProbe:
  tcpSocket:    #A
    port: 1234  #A
  periodSeconds: 2    #B
  failureThreshold: 1  #C
```

The probe in the listing is configured to check if the container's network port 1234 is open. An attempt to establish a connection is made every two seconds and a single failed attempt is enough to consider the container as unhealthy.

Adding an exec liveness probe

Applications that do not accept TCP connections may provide a command to check their status. For these applications, an exec liveness probe is used. As shown in the next figure, the command is executed inside the container and must therefore be available on the container's file system.

Figure 6.7 The exec liveness probe runs the command inside the container



The following is an example of a probe that runs `/usr/bin/healthcheck` every two seconds to determine if the application running in the container is still alive:

```
livenessProbe:
  exec:
    command:
      - /usr/bin/healthcheck
  periodSeconds: 2
  timeoutSeconds: 1
  failureThreshold: 1
```

If the command returns exit code zero, the container is considered healthy. If it returns a non-zero exit code or fails to complete within one second as specified in the `timeoutSeconds` field, the container is terminated immediately, as configured in the `failureThreshold` field, which indicates that a single probe failure is sufficient to consider the container as unhealthy.

6.2.6 Using a startup probe when an application is slow to start

The default liveness probe settings give the application between 20 and 30 seconds to start responding to liveness probe requests. If the application takes longer to start, it is restarted and must start again. If the second start also takes as long, it is restarted again. If this continues, the container never

reaches the state where the liveness probe succeeds and gets stuck in an endless restart loop.

To prevent this, you can increase the `initialDelaySeconds`, `periodSeconds` or `failureThreshold` settings to account for the long start time, but this will have a negative effect on the normal operation of the application. The higher the result of `periodSeconds * failureThreshold`, the longer it takes to restart the application if it becomes unhealthy. For applications that take minutes to start, increasing these parameters enough to prevent the application from being restarted prematurely may not be a viable option.

Introducing startup probes

To deal with the discrepancy between the start and the steady-state operation of an application, Kubernetes also provides *startup probes*.

If a startup probe is defined for a container, only the startup probe is executed when the container is started. The startup probe can be configured to consider the slow start of the application. When the startup probe succeeds, Kubernetes switches to using the liveness probe, which is configured to quickly detect when the application becomes unhealthy.

Adding a startup probe to a pod's manifest

Imagine that the Kiada Node.js application needs more than a minute to warm up, but you want it to be restarted within 10 seconds when it becomes unhealthy during normal operation. The following listing shows how you configure the startup and liveness probes (you can find it in the file `pod.kiada-startup-probe.yaml`).

Listing 6.2 Using a combination of a startup and a liveness probe

```
...
containers:
- name: kiada
  image: luksa/kiada:0.1
  ports:
  - name: http
```

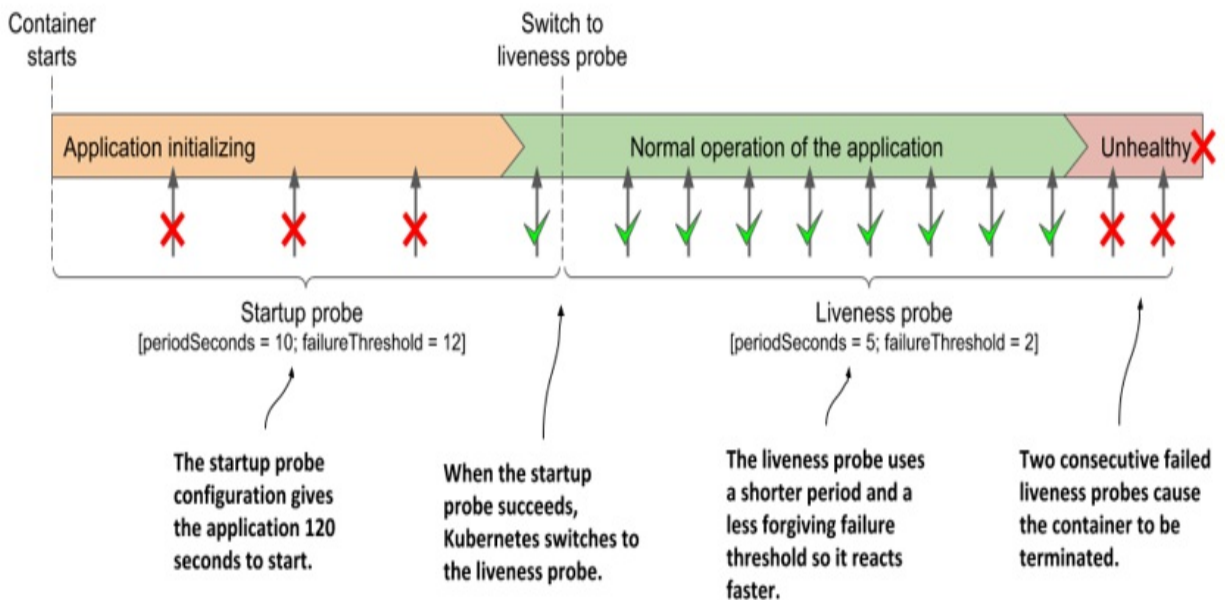


```
    containerPort: 8080
  startupProbe:
    httpGet:
      path: /      #A
      port: http   #A
      periodSeconds: 10    #B
      failureThreshold: 12  #B
  livenessProbe:
    httpGet:
      path: /      #A
      port: http   #A
      periodSeconds: 5    #C
      failureThreshold: 2  #C
```

When the container defined in the listing starts, the application has 120 seconds to start responding to requests. Kubernetes performs the startup probe every 10 seconds and makes a maximum of 12 attempts.

As shown in the following figure, unlike liveness probes, it's perfectly normal for a startup probe to fail. A failure only indicates that the application hasn't yet been completely started. A successful startup probe indicates that the application has started successfully, and Kubernetes should switch to the liveness probe. The liveness probe is then typically executed using a shorter period of time, which allows for faster detection of non-responsive applications.

Figure 6.8 Fast detection of application health problems using a combination of startup and liveness probe



Note

If the startup probe fails often enough to reach the `failureThreshold`, the container is terminated as if the liveness probe had failed.

Usually, the startup and liveness probes are configured to use the same HTTP endpoint, but different endpoints can be used. You can also configure the startup probe as an `exec` or `tcpSocket` probe instead of an `httpGet` probe.

6.2.7 Creating effective liveness probe handlers

You should define a liveness probe for all your pods. Without one, Kubernetes has no way of knowing whether your app is still alive or not, apart from checking whether the application process has terminated.

Causing unnecessary restarts with badly implemented liveness probe handlers

When you implement a handler for the liveness probe, either as an HTTP endpoint in your application or as an additional executable command, be very careful to implement it correctly. If a poorly implemented probe returns a

negative response even though the application is healthy, the application will be restarted unnecessarily. Many Kubernetes users learn this the hard way. If you can make sure that the application process terminates by itself when it becomes unhealthy, it may be safer not to define a liveness probe.

What a liveness probe should check

The liveness probe for the kiada container isn't configured to call an actual health-check endpoint, but only checks that the Node.js server responds to simple HTTP requests for the root URI. This may seem overly simple, but even such a liveness probe works wonders, because it causes a restart of the container if the server no longer responds to HTTP requests, which is its main task. If no liveness probe were defined, the pod would remain in an unhealthy state where it doesn't respond to any requests and would have to be restarted manually. A simple liveness probe like this is better than nothing.

To provide a better liveness check, web applications typically expose a specific health-check endpoint, such as `/healthz`. When this endpoint is called, the application performs an internal status check of all the major components running within the application to ensure that none of them have died or are no longer doing what they should.

Tip

Make sure that the `/healthz` HTTP endpoint doesn't require authentication or the probe will always fail, causing your container to be restarted continuously.

Make sure that the application checks only the operation of its internal components and nothing that is influenced by an external factor. For example, the health-check endpoint of a frontend service should never respond with failure when it can't connect to a backend service. If the backend service fails, restarting the frontend will not solve the problem. Such a liveness probe will fail again after the restart, so the container will be restarted repeatedly until the backend is repaired. If many services are interdependent in this way, the failure of a single service can result in cascading failures across the entire system.

Keeping probes light

The handler invoked by a liveness probe shouldn't use too much computing resources and shouldn't take too long to complete. By default, probes are executed relatively often and only given one second to complete.

Using a handler that consumes a lot of CPU or memory can seriously affect the main process of your container. Later in the book you'll learn how to limit the CPU time and total memory available to a container. The CPU and memory consumed by the probe handler invocation count towards the resource quota of the container, so using a resource-intensive handler will reduce the CPU time available to the main process of the application.

Tip

When running a Java application in your container, you may want to use an HTTP GET probe instead of an exec liveness probe that starts an entire JVM. The same applies to commands that require considerable computing resources.

Avoiding retry loops in your probe handlers

You've learned that the failure threshold for the probe is configurable. Instead of implementing a retry loop in your probe handlers, keep it simple and instead set the `failureThreshold` field to a higher value so that the probe must fail several times before the application is considered unhealthy. Implementing your own retry mechanism in the handler is a waste of effort and represents another potential point of failure.

6.3 Executing actions at container start-up and shutdown

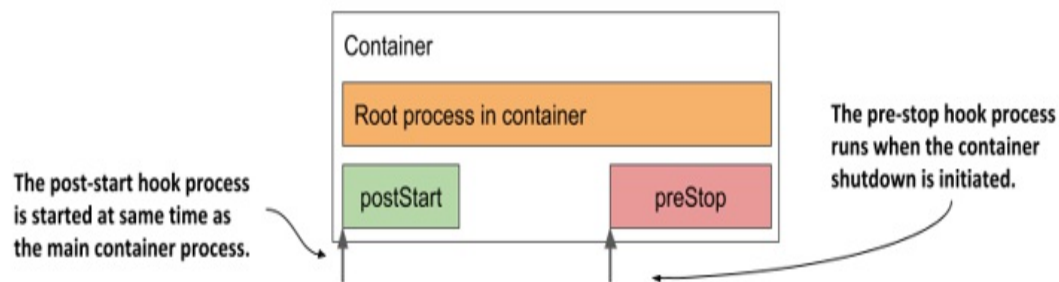
In the previous chapter you learned that you could use init containers to run containers at the start of the pod lifecycle. You may also want to run additional processes every time a container starts and just before it stops. You can do this by adding *lifecycle hooks* to the container. Two types of hooks are

currently supported:

- *Post-start* hooks, which are executed when the container starts, and
- *Pre-stop* hooks, which are executed shortly before the container stops.

These lifecycle hooks are specified per container, as opposed to init containers, which are specified at the pod level. The next figure should help you visualize how lifecycle hooks fit into the lifecycle of a container.

Figure 6.9 How the post-start and pre-stop hook fit into the container's lifecycle



Like liveness probes, lifecycle hooks can be used to either

- execute a command inside the container, or
- send an HTTP GET request to the application in the container.

Note

The same as with liveness probes, lifecycle hooks can only be applied to regular containers and not to init containers. Unlike probes, lifecycle hooks do not support tcpSocket handlers.

Let's look at the two types of hooks individually to see what you can use them for.

6.3.1 Using post-start hooks to perform actions when the container starts

The post-start lifecycle hook is invoked immediately after the container is

created. You can use the `exec` type of the hook to execute an additional process as the main process starts, or you can use the `httpGet` hook to send an HTTP request to the application running in the container to perform some type of initialization or warm-up procedure.

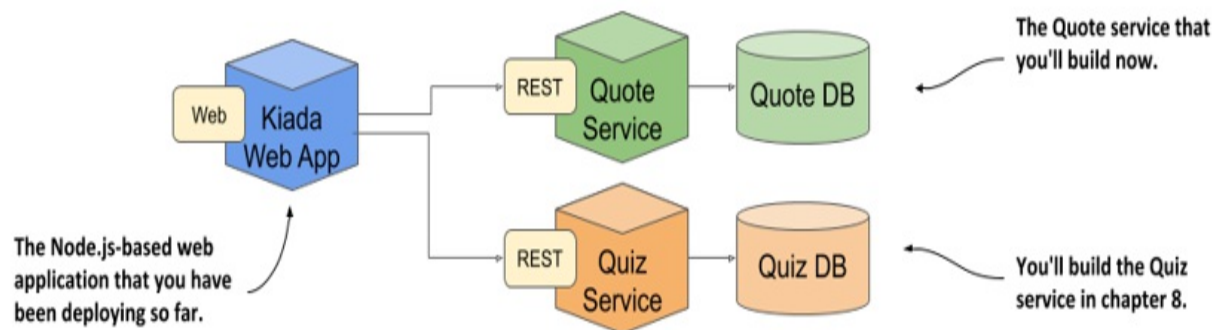
If you're the author of the application, you could perform the same operation within the application code itself, but if you need to add it to an existing application that you didn't create yourself, you may not be able to do so. A post-start hook provides a simple alternative that doesn't require you to change the application or its container image.

Let's look at an example of how a post-start hook can be used in a new service you'll create.

Introducing the Quote service

You may remember from section 2.2.1 that the final version of the Kubernetes in Action Demo Application (Kiada) Suite will contain the Quote and Quiz services in addition to the Node.js application. The data from those two services will be used to show a random quote from the book and a multiple-choice pop quiz to help you test your Kubernetes knowledge. To refresh your memory, the following figure shows the three components that make up the Kiada Suite.

Figure 6.10 The Kubernetes in Action Demo Application Suite



During my first steps with Unix in the 1990s, one of the things I found most amusing was the random, sometimes funny message that the fortune

command displayed every time I logged into our high school's Sun Ultra server. Nowadays, you'll rarely see the `fortune` command installed on Unix/Linux systems anymore, but you can still install it and run it whenever you're bored. Here's an example of what it may display:

```
$ fortune
Dinner is ready when the smoke alarm goes off.
```

The command gets the quotes from files that are packaged with it, but you can also use your own file(s). So why not use `fortune` to build the Quote service? Instead of using the default files, I'll provide a file with quotes from this book.

But one caveat exists. The `fortune` command prints to the standard output. It can't serve the quote over HTTP. However, this isn't a hard problem to solve. We can combine the `fortune` program with a web server such as Nginx to get the result we want.

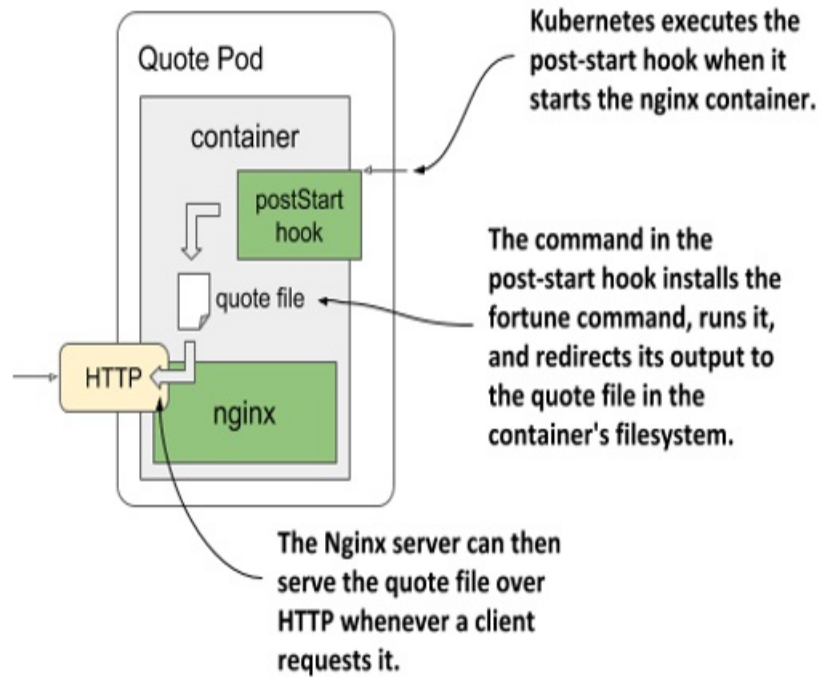
Using a post-start container lifecycle hook to run a command in the container

For the first version of the service, the container will run the `fortune` command when it starts up. The output will be redirected to a file in Nginx' web-root directory, so that it can serve it. Although this means that the same quote is returned in every request, this is a perfectly good start. You'll later improve the service iteratively.

The Nginx web server is available as a container image, so let's use it. Because the `fortune` command is not available in the image, you'd normally build a new image that uses that image as the base and installs the `fortune` package on top of it. But we'll keep things even simpler for now.

Instead of building a completely new image you'll use a post-start hook to install the `fortune` software package, download the file containing the quotes from this book, and finally run the `fortune` command and write its output to a file that Nginx can serve. The operation of the quote-poststart pod is presented in the following figure.

Figure 6.11 The operation of the quote-poststart pod



The following listing shows how to define the hook (file `pod.quote-poststart.yaml`).

Listing 6.3 Pod with a post-start lifecycle hook

```
apiVersion: v1
kind: Pod
metadata:
  name: quote-poststart      #A
spec:
  containers:
    - name: nginx            #B
      image: nginx:alpine    #B
      ports:                  #C
        - name: http         #C
          containerPort: 80   #C
      lifecycle:              #D
        postStart:            #D
          exec:                #D
            command:           #D
              - sh             #E
              - -c             #F
```



```

- |      #G
  apk add fortune && \      #H
  curl -O https://luksa.github.io/kiada/book-quotes.txt
  curl -O https://luksa.github.io/kiada/book-quotes.txt
  fortune book-quotes.txt > /usr/share/nginx/html/quote

```

The YAML in the listing is not simple, so let me make sense of it. First, the easy parts. The pod is named `quote-poststart` and contains a single container based on the `nginx:alpine` image. A single port is defined in the container. A `postStart` lifecycle hook is also defined for the container. It specifies what command to run when the container starts. The tricky part is the definition of this command, but I'll break it down for you.

It's a list of commands that are passed to the `sh` command as an argument. The reason this needs to be so is because you can't define multiple commands in a lifecycle hook. The solution is to invoke a shell as the main command and letting it run the list of commands by specifying them in the command string:

```
sh -c "the command string"
```

In the previous listing, the third argument (the command string) is rather long, so it must be specified over multiple lines to keep the YAML legible. Multi-line string values in YAML can be defined by typing a pipeline character and following it with properly indented lines. The command string in the previous listing is therefore as follows:

```

apk add fortune && \
curl -O https://luksa.github.io/kiada/book-quotes.txt && \
curl -O https://luksa.github.io/kiada/book-quotes.txt.dat && \
fortune book-quotes.txt > /usr/share/nginx/html/quote

```

As you can see, the command string consists of four commands. Here's what they do:

1. The `apk add fortune` command runs the Alpine Linux package management tool, which is part of the image that `nginx:alpine` is based on, to install the `fortune` package in the container.
2. The first `curl` command downloads the `book-quotes.txt` file.
3. The second `curl` command downloads the `book-quotes.txt.dat` file.

4. The `fortune` command selects a random quote from the `book-quotes.txt` file and prints it to standard output. That output is redirected to the `/usr/share/nginx/html/quote` file.

The lifecycle hook command runs parallel to the main process. The `postStart` name is somewhat misleading, because the hook isn't executed after the main process is fully started, but as soon as the container is created, at around the same time the main process starts.

When the `postStart` hook in this example completes, the quote produced by the `fortune` command is stored in the `/usr/share/nginx/html/quote` file and can be served by Nginx.

Use the `kubectl apply` command to create the pod from the `pod.quote-poststart.yaml` file, and you should then be able to use `curl` or your browser to get the quote at URI `/quote` on port 80 of the `quote-poststart` pod. You've already learned how to use the `kubectl port-forward` command to open a tunnel to the container, but you may want to refer to the sidebar because a caveat exists.

Accessing the `quote-poststart` pod

To retrieve the quote from the `quote-poststart` pod, you must first run the `kubectl port-forward` command, which may fail as shown here:

```
$ kubectl port-forward quote-poststart 80
Unable to listen on port 80: Listeners failed to create with the
error: unable to listen on any of the requested ports: [{80 80}]
```

The command fails if your operating system doesn't allow you to run processes that bind to port numbers 0-1023. To fix this, you must use a higher local port number as follows:

```
$ kubectl port-forward quote-poststart 1080:80
```

The last argument tells `kubectl` to use port 1080 locally and forward it to port 80 of the pod. You can now access the Quote service at <http://localhost:1080/quote>.

If everything works as it should, the Nginx server will return a random quote from this book as in the following example:

```
$ curl localhost:1080/quote
```

The same as with liveness probes, lifecycle hooks can only be applied to init containers. Unlike probes, lifecycle hooks do not support

The first version of the Quote service is now done, but you'll improve it in the next chapter. Now let's learn about the caveats of using post-start hooks before we move on.

Understanding how a post-start hook affects the container

Although the post-start hook runs asynchronously with the main container process, it affects the container in two ways.

First, the container remains in the waiting state with the reason `ContainerCreating` until the hook invocation is completed. The phase of the pod is `Pending`. If you run the `kubectl logs` command at this point, it refuses to show the logs, even though the container is running. The `kubectl port-forward` command also refuses to forward ports to the pod.

If you want to see this for yourself, deploy the `pod.quote-poststart-slow.yaml` pod manifest file. It defines a post-start hook that takes 60 seconds to complete. Immediately after the pod is created, inspect its state, and display the logs with the following command:

```
$ kubectl logs quote-poststart-slow
Error from server (BadRequest): container "nginx" in pod "quote-p
```

The error message returned implies that the container hasn't started yet, which isn't the case. To prove this, use the following command to list processes in the container:

```
$ kubectl exec quote-poststart-slow -- ps x
PID    USER      TIME  COMMAND
   1   root         0:00 nginx: master process nginx -g daemon off;
   7   root         0:00 sh -c apk add fortune && \ sleep 60 && \ curl.
  13  nginx       0:00 nginx: worker process
...
```

```
20 nginx      0:00 nginx: worker process
21 root        0:00 sleep 60
22 root        0:00 ps x
```

The other way a post-start hook could affect the container is if the command used in the hook can't be executed or returns a non-zero exit code. If this happens, the entire container is restarted. To see an example of a post-start hook that fails, deploy the pod manifest `pod.quote-poststart-fail.yaml`.

If you watch the pod's status using `kubectl get pods -w`, you'll see the following status:

```
quote-poststart-fail    0/1      PostStartHookError: command 'sh -c
```

It shows the command that was executed and the code with which it terminated. When you review the pod events, you'll see a `FailedPostStartHook` warning event that indicates the exit code and what the command printed to the standard or error output. This is the event:

```
Warning  FailedPostStartHook  Exec lifecycle hook ([sh -c ...]) f
```

The same information is also contained in the `containerStatuses` field in the pod's status field, but only for a short time, as the container status changes to `CrashLoopBackOff` shortly afterwards.

Tip

Because the state of a pod can change quickly, inspecting just its status may not tell you everything you need to know. Rather than inspecting the state at a particular moment in time, reviewing the pod's events is usually a better way to get the full picture.

Capturing the output produced by the process invoked via a post-start hook

As you've just learned, the output of the command defined in the post-start hook can be inspected if it fails. In cases where the command completes successfully, the output of the command is not logged anywhere. To see the output, the command must log to a file instead of the standard or error output.

You can then view the contents of the file with a command like the following:

```
$ kubectl exec my-pod -- cat logfile.txt
```

Using an HTTP GET post-start hook

In the previous example, you configured the post-start hook to execute a command inside the container. Alternatively, you can have Kubernetes send an HTTP GET request when it starts the container by using an `httpGet` post-start hook.

Note

You can't specify both an `exec` and an `httpGet` post-start hook for a container. They are exclusive.

You can configure the lifecycle hook to send the request to a process running in the container itself, a different container in the pod, or a different host altogether.

For example, you can use an `httpGet` post-start hook to tell another service about your pod. The following listing shows an example of a post-start hook definition that does this. You'll find it in file `pod.poststart-httpget.yaml`.

Listing 6.4 Using an `httpGet` post-start hook to warm up a web server

```
lifecycle:    #A
  postStart:  #A
    httpGet:  #A
      host: myservice.example.com    #B
      port: 80                      #B
      path: /container-started      #C
```

The example in the listing shows an `httpGet` post-start hook that calls the following URL when the container starts:

`http://myservice.example.com/container-started`.

In addition to the `host`, `port`, and `path` fields shown in the listing, you can

also specify the scheme (HTTP or HTTPS) and the `httpHeaders` to be sent in the request. The `host` field defaults to the pod IP. Don't set it to `localhost` unless you want to send the request to the node hosting the pod. That's because the request is sent from the host node, not from within the container.

As with command-based post-start hooks, the HTTP GET post-start hook is executed at the same time as the container's main process. And this is what makes these types of lifecycle hooks applicable only to a limited set of use-cases.

If you configure the hook to send the request to the container its defined in, you'll be in trouble if the container's main process isn't yet ready to accept requests. In that case, the post-start hook fails, which then causes the container to be restarted. On the next run, the same thing happens. The result is a container that keeps being restarted.

To see this for yourself, try creating the pod defined in `pod.poststart-httpget-slow.yaml`. I've made the container wait one second before starting the web server. This ensures that the post-start hook never succeeds. But the same thing could also happen if the pause didn't exist. There is no guarantee that the web server will always start up fast enough. It might start fast on your own computer or a server that's not overloaded, but on a production system under considerable load, the container may never start properly.

Warning

Using an HTTP GET post-start hook might cause the container to enter an endless restart loop. Never configure this type of lifecycle hook to target the same container or any other container in the same pod.

Another problem with HTTP GET post-start hooks is that Kubernetes doesn't treat the hook as failed if the HTTP server responds with status code such as `404 Not Found`. Make sure you specify the correct URI in your HTTP GET hook, otherwise you might not even notice that the post-start hook missed its mark.

6.3.2 Using pre-stop hooks to run a process just before the

container terminates

Besides executing a command or sending an HTTP request at container startup, Kubernetes also allows the definition of a *pre-stop* hook in your containers.

A pre-stop hook is executed immediately before a container is terminated. To terminate a process, the `TERM` signal is usually sent to it. This tells the application to finish what it's doing and shut down. The same happens with containers. Whenever a container needs to be stopped or restarted, the `TERM` signal is sent to the main process in the container. Before this happens, however, Kubernetes first executes the pre-stop hook, if one is configured for the container. The `TERM` signal is not sent until the pre-stop hook completes unless the process has already terminated due to the invocation of the pre-stop hook handler itself.

Note

When container termination is initiated, the liveness and other probes are no longer invoked.

A pre-stop hook can be used to initiate a graceful shutdown of the container or to perform additional operations without having to implement them in the application itself. As with post-start hooks, you can either execute a command within the container or send an HTTP request to the application running in it.

Using a pre-stop lifecycle hook to shut down a container gracefully

The Nginx web server used in the quote pod responds to the `TERM` signal by immediately closing all open connections and terminating the process. This is not ideal, as the client requests that are being processed at this time aren't allowed to complete.

Fortunately, you can instruct Nginx to shut down gracefully by running the command `nginx -s quit`. When you run this command, the server stops accepting new connections, waits until all in-flight requests have been

processed, and then quits.

When you run Nginx in a Kubernetes pod, you can use a pre-stop lifecycle hook to run this command and ensure that the pod shuts down gracefully. The following listing shows the definition of this pre-stop hook (you'll find it in the file `pod.quote-prestop.yaml`).

Listing 6.5 Defining a pre-stop hook for Nginx

```
lifecycle:      #A
  preStop:      #A
    exec:       #B
      command:   #B
      - nginx    #C
      - -s       #C
      - quit     #C
```

Whenever a container using this pre-stop hook is terminated, the command `nginx -s quit` is executed in the container before the main process of the container receives the `TERM` signal.

Unlike the post-start hook, the container is terminated regardless of the result of the pre-stop hook - a failure to execute the command or a non-zero exit code does not prevent the container from being terminated. If the pre-stop hook fails, you'll see a `FailedPreStopHook` warning event among the pod events, but you might not see any indication of the failure if you are only monitoring the status of the pod.

Tip

If successful completion of the pre-stop hook is critical to the proper operation of your system, make sure that it runs successfully. I've experienced situations where the pre-stop hook didn't run at all, but the engineers weren't even aware of it.

Like post-start hooks, you can also configure the pre-stop hook to send an HTTP GET request to your application instead of executing commands. The configuration of the HTTP GET pre-stop hook is the same as for a post-start hook. For more information, see section 6.3.1.

Why doesn't my application receive the TERM signal?

Many developers make the mistake of defining a pre-stop hook just to send a TERM signal to their applications in the pre-stop hook. They do this when they find that their application never receives the TERM signal. The root cause is usually not that the signal is never sent, but that it is swallowed by something inside the container. This typically happens when you use the *shell* form of the ENTRYPOINT or the CMD directive in your Dockerfile. Two forms of these directives exist.

The *exec* form is: `ENTRYPOINT ["/myexecutable", "1st-arg", "2nd-arg"]`

The *shell* form is: `ENTRYPOINT /myexecutable 1st-arg 2nd-arg`

When you use the *exec* form, the executable file is called directly. The process it starts becomes the root process of the container. When you use the *shell* form, a shell runs as the root process, and the shell runs the executable as its child process. In this case, the shell process is the one that receives the TERM signal. Unfortunately, it doesn't pass this signal to the child process.

In such cases, instead of adding a pre-stop hook to send the TERM signal to your app, the correct solution is to use the *exec* form of ENTRYPOINT or CMD.

Note that the same problem occurs if you use a shell script in your container to run the application. In this case, you must either intercept and pass signals to the application or use the *exec* shell command to run the application in your script.

Pre-stop hooks are only invoked when the container is requested to terminate, either because it has failed its liveness probe or because the pod has to shut down. They are not called when the process running in the container terminates by itself.

Understanding that lifecycle hooks target containers, not pods

As a final consideration on the post-start and pre-stop hooks, I would like to emphasize that these lifecycle hooks apply to containers and not to pods. You shouldn't use a pre-stop hook to perform an action that needs to be performed

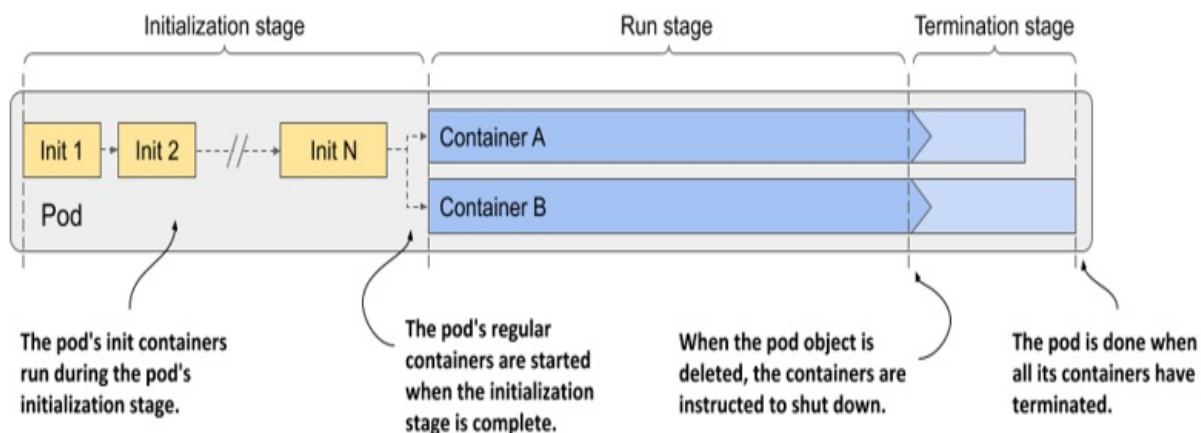
when the entire pod is shut down, because pre-stop hooks run every time the container needs to terminate. This can happen several times during the pod's lifetime, not just when the pod shuts down.

6.4 Understanding the pod lifecycle

So far in this chapter you've learned a lot about how the containers in a pod run. Now let's take a closer look at the entire lifecycle of a pod and its containers.

When you create a pod object, Kubernetes schedules it to a worker node that then runs its containers. The pod's lifecycle is divided into the three stages shown in the next figure:

Figure 6.12 The three stages of the pod's lifecycle



The three stages of the pod's lifecycle are:

1. The initialization stage, during which the pod's init containers run.
2. The run stage, in which the regular containers of the pod run.
3. The termination stage, in which the pod's containers are terminated.

Let's see what happens in each of these stages.

6.4.1 Understanding the initialization stage

As you’ve already learned, the pod’s init containers run first. They run in the order specified in the `initContainers` field in the pod’s spec. Let me explain everything that unfolds.

Pulling the container image

Before each init container is started, its container image is pulled to the worker node. The `imagePullPolicy` field in the container definition in the pod specification determines whether the image is pulled every time, only the first time, or never.

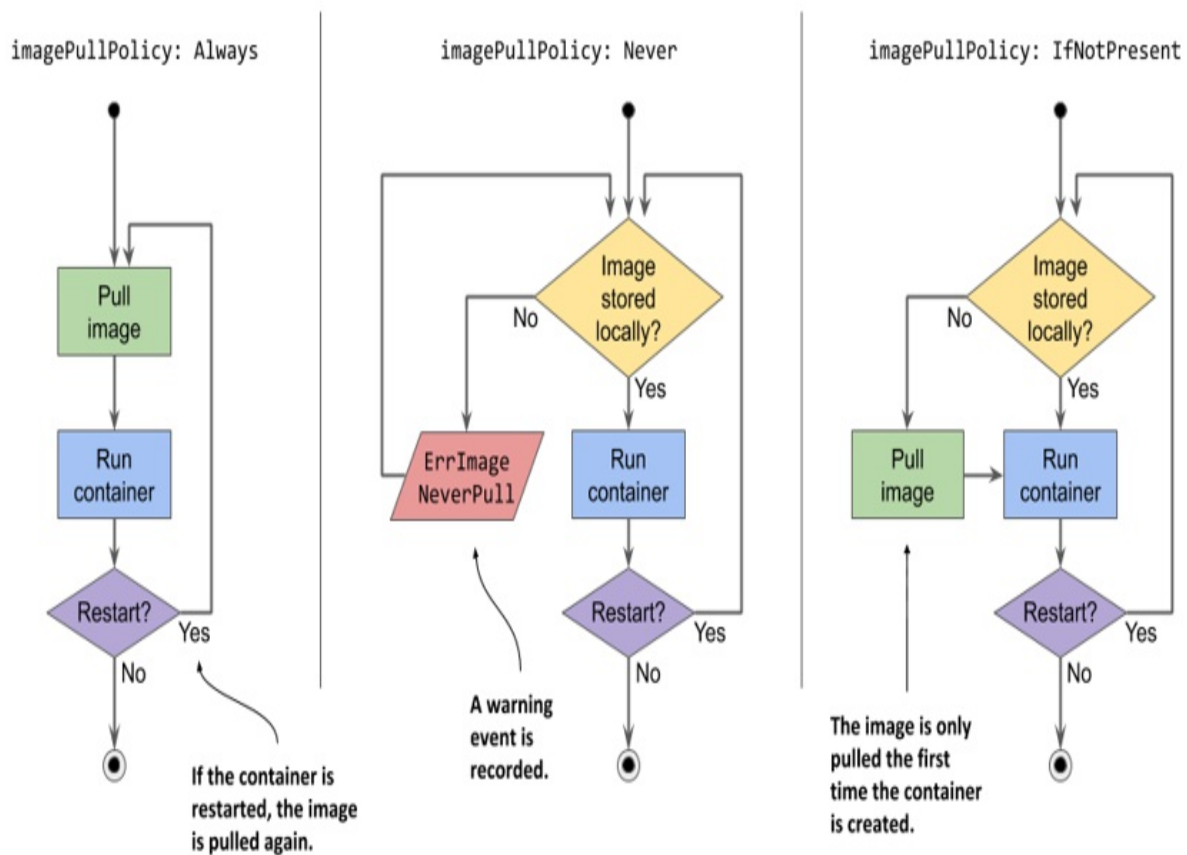
Table 6.5 List of image-pull policies

Image pull policy	Description
Not specified	If the <code>imagePullPolicy</code> is not explicitly specified, it defaults to <code>Always</code> if the <code>:latest</code> tag is used in the image. For other image tags, it defaults to <code>IfNotPresent</code> .
Always	The image is pulled every time the container is (re)started. If the locally cached image matches the one in the registry, it is not downloaded again, but the registry still needs to be contacted.
Never	The container image is never pulled from the registry. It must exist on the worker node beforehand. Either it was stored locally when another container with the same image was deployed, or it was built on the node itself, or simply downloaded by someone or something else.
	Image is pulled if it is not already present on the worker

`IfExists` node. This ensures that the image is only pulled the first time it's required.

The image-pull policy is also applied every time the container is restarted, so a closer look is warranted. Examine the following figure to understand the behavior of these three policies.

Figure 6.13 An overview of the three different image-pull policies



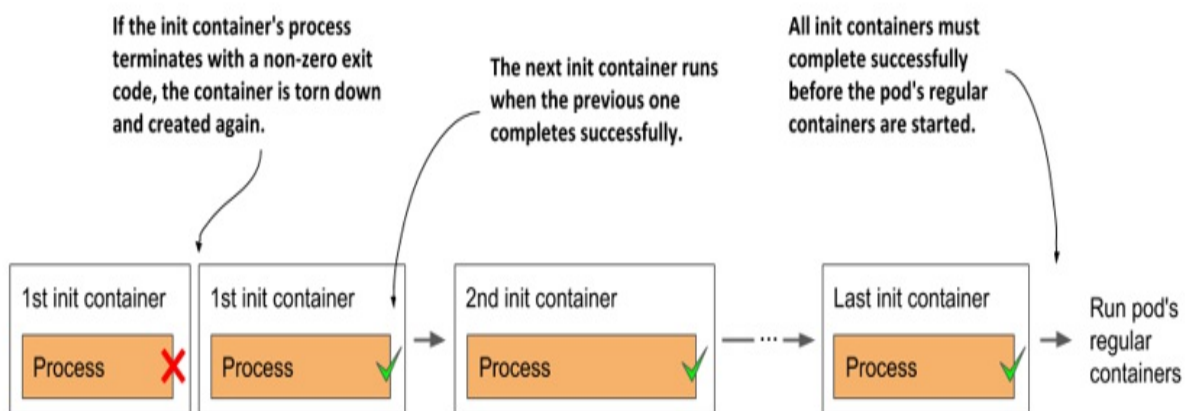
Warning

If the `imagePullPolicy` is set to `Always` and the image registry is offline, the container will not run even if the same image is already stored locally. A registry that is unavailable may therefore prevent your application from (re)starting.

Running the containers

When the first container image is downloaded to the node, the container is started. When the first init container is complete, the image for the next init container is pulled and the container is started. This process is repeated until all init containers are successfully completed. Containers that fail might be restarted, as shown in the following figure.

Figure 6.14 All init containers must run to completion before the regular containers can start



Restarting failed init containers

If an init container terminates with an error and the pod's restart policy is set to `Always` or `OnFailure`, the failed init container is restarted. If the policy is set to `Never`, the subsequent init containers and the pod's regular containers are never started. The pod's status is displayed as `Init:Error` indefinitely. You must then delete and recreate the pod object to restart the application. To try this yourself, deploy the file `pod.kiada-init-fail-norestart.yaml`.

Note

If the container needs to be restarted and `imagePullPolicy` is set to `Always`, the container image is pulled again. If the container had terminated due to an error and you push a new image with the same tag that fixes the error, you don't need to recreate the pod, as the updated container image will be pulled

before the container is restarted.

Re-executing the pod's init containers

Init containers are normally only executed once. Even if one of the pod's main containers is terminated later, the pod's init containers are not re-executed. However, in exceptional cases, such as when Kubernetes must restart the entire pod, the pod's init containers might be executed again. This means that the operations performed by your init containers must be idempotent.

6.4.2 Understanding the run stage

When all init containers are successfully completed, the pod's regular containers are all created in parallel. In theory, the lifecycle of each container should be independent of the other containers in the pod, but this is not quite true. See sidebar for more information.

A container's post-start hook blocks the creation of the subsequent container

The Kubelet doesn't start all containers of the pod at the same time. It creates and starts the containers synchronously in the order they are defined in the pod's spec. If a post-start hook is defined for a container, it runs asynchronously with the main container process, but the execution of the post-start hook handler blocks the creation and start of the subsequent containers.

This is an implementation detail that might change in the future.

In contrast, the termination of containers is performed in parallel. A long-running pre-stop hook does block the shutdown of the container in which it is defined, but it does not block the shutdown of other containers. The pre-stop hooks of the containers are all invoked at the same time.

The following sequence runs independently for each container. First, the container image is pulled, and the container is started. When the container terminates, it is restarted, if this is provided for in the pod's restart policy.

The container continues to run until the termination of the pod is initiated. A more detailed explanation of this sequence is presented next.

Pulling the container image

Before the container is created, its image is pulled from the image registry, following the pod's `imagePullPolicy`. Once the image is pulled, the container is created.

Note

Even if a container image can't be pulled, the other containers in the pod are started nevertheless.

Warning

Containers don't necessarily start at the same moment. If pulling the image takes time, the container may start long after all the others have already started. Consider this if a containers depends on others.

Running the container

The container starts when the main container process starts. If a post-start hook is defined in the container, it is invoked in parallel with the main container process. The post-start hook runs asynchronously and must be successful for the container to continue running.

Together with the main container and the potential post-start hook process, the startup probe, if defined for the container, is started. When the startup probe is successful, or if the startup probe is not configured, the liveness probe is started.

Terminating and restarting the container on failures

If the startup or the liveness probe fails so often that it reaches the configured failure threshold, the container is terminated. As with init containers, the

pod's `restartPolicy` determines whether the container is then restarted or not.

Perhaps surprisingly, if the restart policy is set to `Never` and the startup hook fails, the pod's status is shown as `Completed` even though the post-start hook failed. You can see this for yourself by creating the pod defined in the file `pod.quote-poststart-fail-norestart.yaml`.

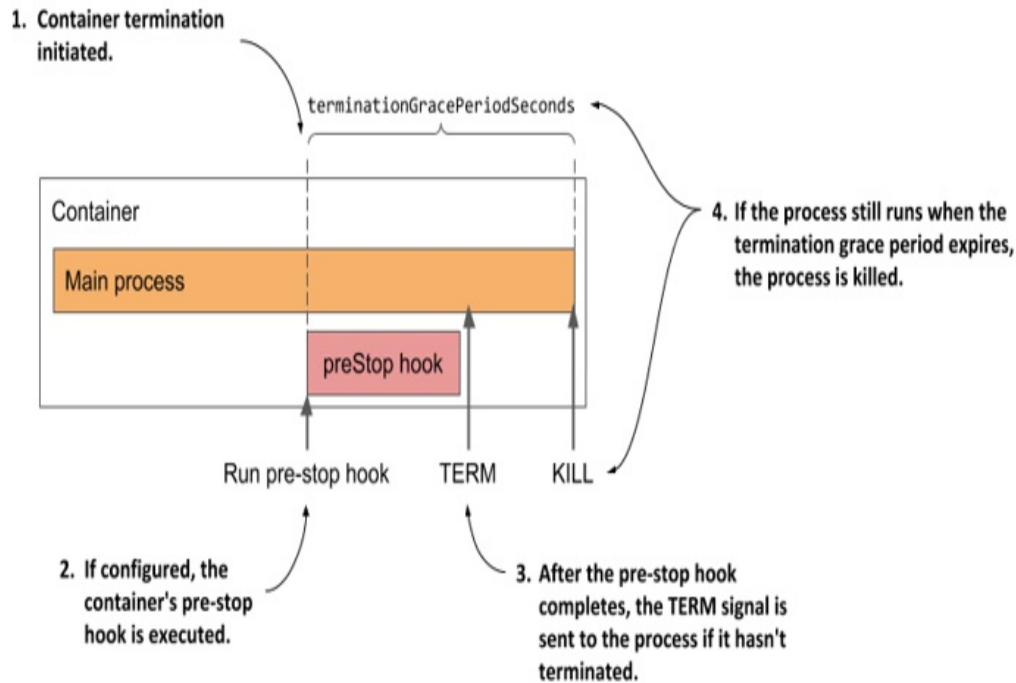
Introducing the termination grace period

If a container must be terminated, the container's pre-stop hook is called so that the application can shut down gracefully. When the pre-stop hook is completed, or if no pre-stop hook is defined, the `TERM` signal is sent to the main container process. This is another hint to the application that it should shut down.

The application is given a certain amount of time to terminate. This time can be configured using the `terminationGracePeriodSeconds` field in the pod's spec and defaults to 30 seconds. The timer starts when the pre-stop hook is called or when the `TERM` signal is sent if no hook is defined. If the process is still running after the termination grace period has expired, it's terminated by force via the `KILL` signal. This terminates the container.

The following figure illustrates the container termination sequence.

Figure 6.15 A container's termination sequence



After the container has terminated, it will be restarted if the pod's restart policy allows it. If not, the container will remain in the `Terminated` state, but the other containers will continue running until the entire pod is shut down or until they fail as well.

6.4.3 Understanding the termination stage

The pod's containers continue to run until you finally delete the pod object. When this happens, termination of all containers in the pod is initiated and its status is changed to `Terminating`.

Introducing the deletion grace period

The termination of each container at pod shutdown follows the same sequence as when the container is terminated because it has failed its liveness probe, except that instead of the termination grace period, the pod's *deletion grace period* determines how much time is available to the containers to shut down on their own.

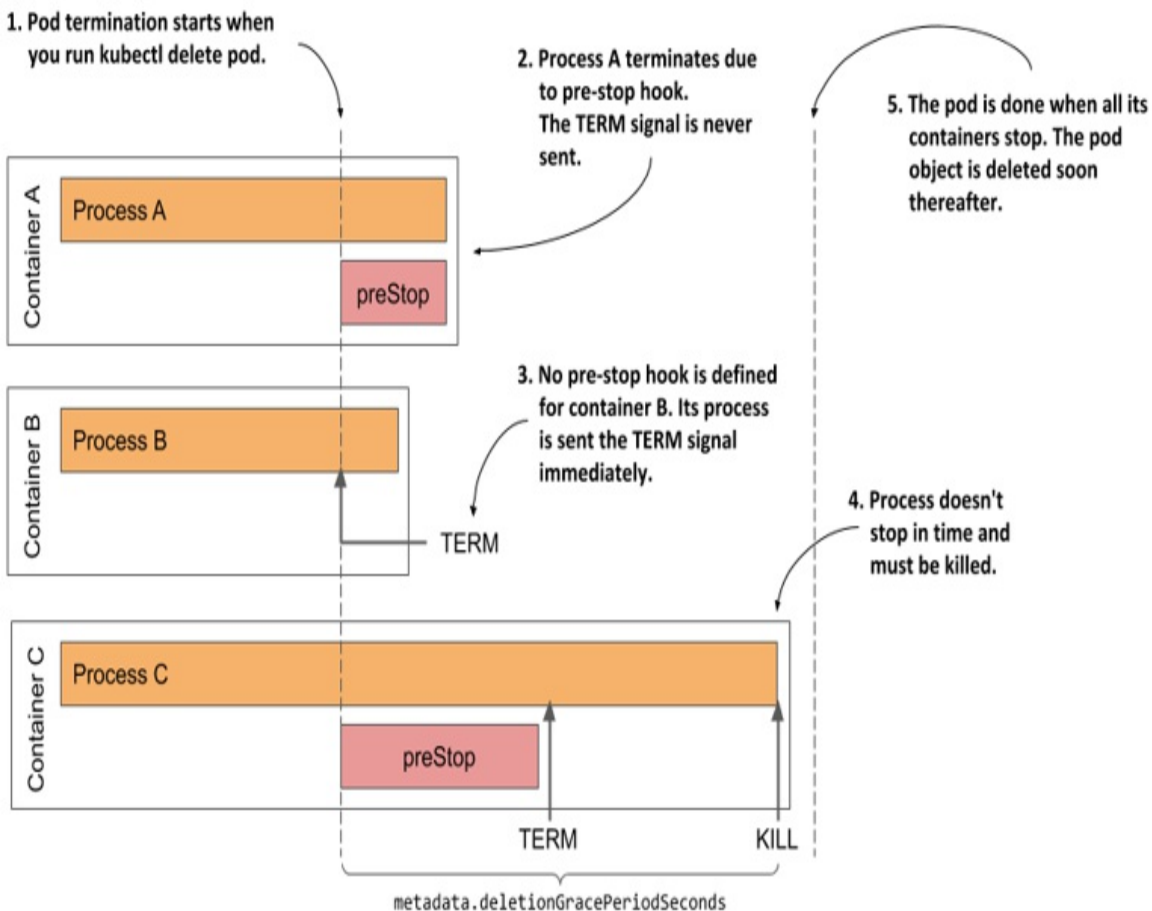
This grace period is defined in the pod's

`metadata.deletionGracePeriodSeconds` field, which gets initialized when you delete the pod. By default, it gets its value from the `spec.terminationGracePeriodSeconds` field, but you can specify a different value in the `kubectl delete` command. You'll see how to do this later.

Understanding how the pod's containers are terminated

As shown in the next figure, the pod's containers are terminated in parallel. For each of the pod's containers, the container's pre-stop hook is called, the `TERM` signal is then sent to the main container process, and finally the process is terminated using the `KILL` signal if the deletion grace period expires before the process stops by itself. After all the containers in the pod have stopped running, the pod object is deleted.

Figure 6.16 The termination sequence inside a pod



Inspecting the slow shutdown of a pod

Let's look at this last stage of the pod's life on one of the pods you created previously. If the `kiada-ssl` pod doesn't run in your cluster, please create it again. Now delete the pod by running `kubectl delete pod kiada-ssl`.

It takes surprisingly long to delete the pod, doesn't it? I counted at least 30 seconds. This is neither normal nor acceptable, so let's fix it.

Considering what you've learned in this section, you may already know what's causing the pod to take so long to finish. If not, let me help you analyze the situation.

The `kiada-ssl` pod has two containers. Both must stop before the pod object can be deleted. Neither container has a pre-stop hook defined, so both containers should receive the `TERM` signal immediately when you delete the pod. The 30s I mentioned earlier match the default termination grace period value, so it looks like one of the containers, if not both, doesn't stop when it receives the `TERM` signal, and is killed after the grace period expires.

Changing the termination grace period

You can try setting the pod's `terminationGracePeriodSeconds` field to a lower value to see if it terminates sooner. The following manifest shows how to the field in the pod manifest (file `pod.kiada-ssl-shortgraceperiod.yaml`).

Listing 6.6 Setting a lower `terminationGracePeriodSeconds` for faster pod shutdown

```
apiVersion: v1
kind: Pod
metadata:
  name: kiada-ssl-shortgraceperiod
spec:
  terminationGracePeriodSeconds: 5    #A
  containers:
  ...
```

In the listing above, the pod's `terminationGracePeriodSeconds` is set to 5.

If you create and then delete this pod, you'll see that its containers are terminated within 5s of receiving the TERM signal.

Tip

A reduction of the termination grace period is rarely necessary. However, it is advisable to extend it if the application usually needs more time to shut down gracefully.

Specifying the deletion grace period when deleting the pod

Any time you delete a pod, the pod's `terminationGracePeriodSeconds` determines the amount of time the pod is given to shut down, but you can override this time when you execute the `kubectl delete` command using the `--grace-period` command line option.

For example, to give the pod 10s to shut down, you run the following command:

```
$ kubectl delete po kiada-ssl --grace-period 10
```

Note

If you set this grace period to zero, the pod's pre-stop hooks are not executed.

Fixing the shutdown behavior of the Kiada application

Considering that the shortening of the grace period leads to a faster shutdown of the pod, it's clear that at least one of the two containers doesn't terminate by itself after it receives the TERM signal. To see which one, recreate the pod, then run the following commands to stream the logs of each container before deleting the pod again:

```
$ kubectl logs kiada-ssl -c kiada -f
$ kubectl logs kiada-ssl -c envoy -f
```

The logs show that the Envoy proxy catches the signal and immediately terminates, whereas the Node.js application doesn't respond to the signal. To

fix this, you need to add the code in the following listing to the end of your `app.js` file. You'll find the updated file in `Chapter06/kiada-0.3/app.js`.

Listing 6.7 Handling the TERM signal in the kiada application

```
process.on('SIGTERM', function () {  
  console.log("Received SIGTERM. Server shutting down...");  
  server.close(function () {  
    process.exit(0);  
  });  
});
```

After you make the change to the code, create a new container image with the tag `:0.3`, push it to your image registry, and deploy a new pod that uses the new image. You can also use the image `docker.io/luksa/kiada:0.3` that I've built. To create the pod, apply the manifest file `pod.kiada-ssl-0.3.yaml`.

If you delete this new pod, you'll see that it shuts down considerably faster. From the logs of the `kiada` container, you can see that it begins to shut down as soon as it receives the `TERM` signal.

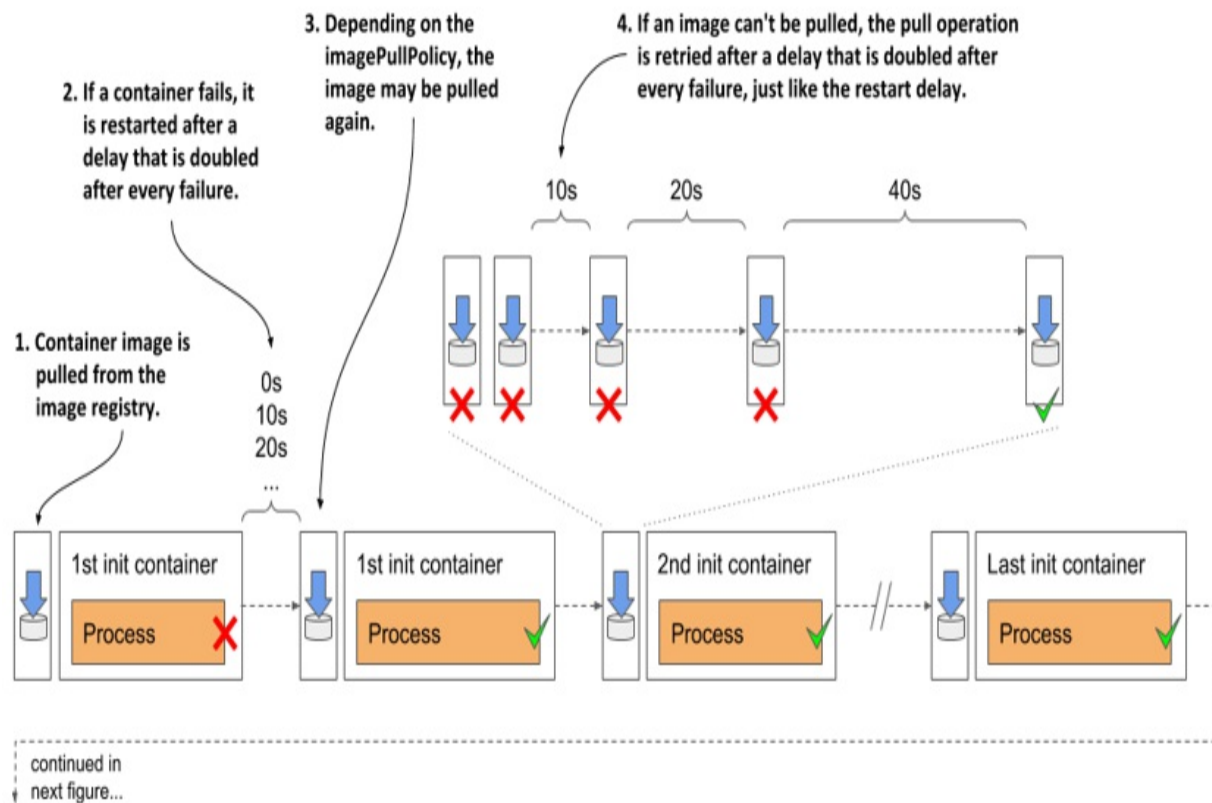
TIP

Don't forget to ensure that your init containers also handle the `TERM` signal so that they shut down immediately if you delete the pod object while it's still being initialized.

6.4.4 Visualizing the full lifecycle of the pod's containers

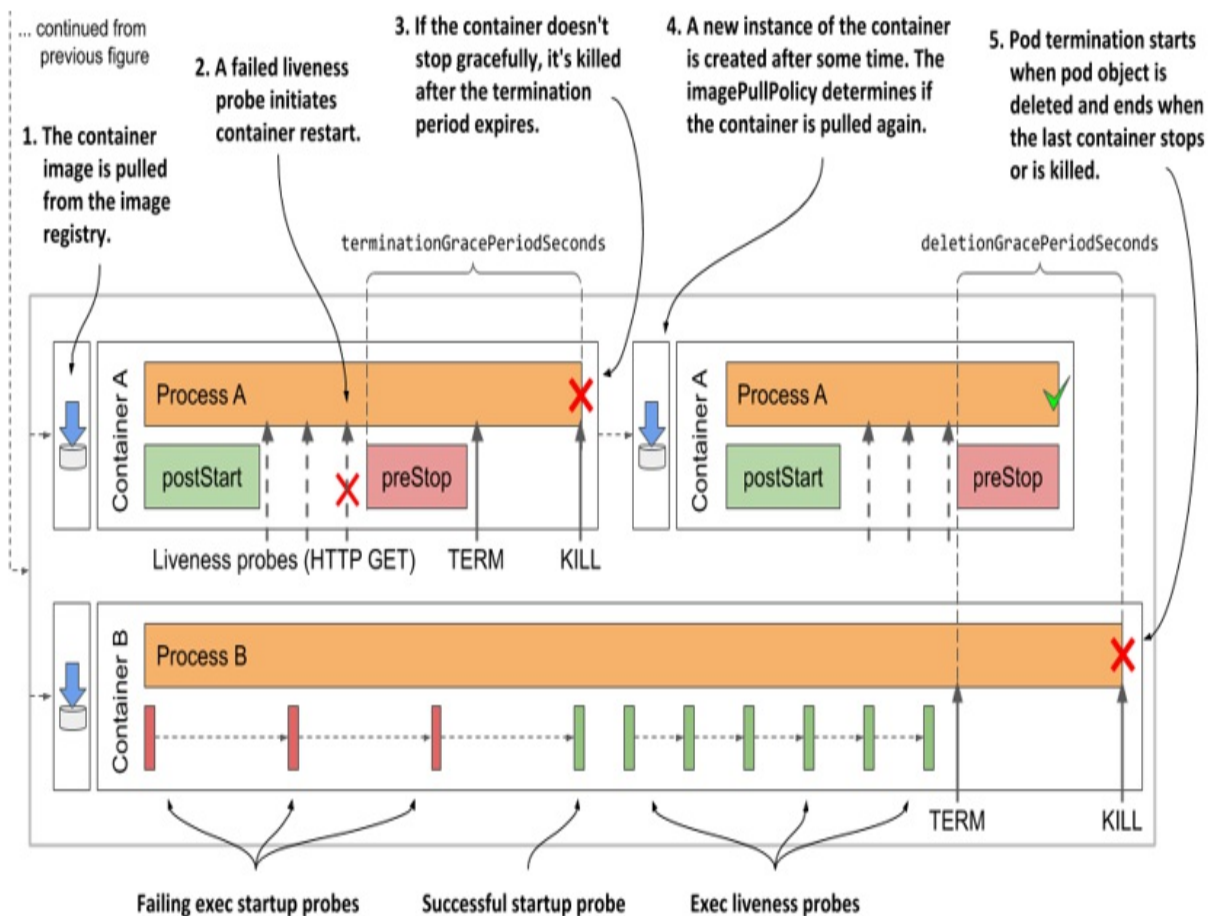
To conclude this chapter on what goes on in a pod, I present a final overview of everything that happens during the life of a pod. The following two figures summarize everything that has been explained in this chapter. The initialization of the pod is shown in the next figure.

Figure 6.17 Complete overview of the pod's initialization stage



When initialization is complete, normal operation of the pod's containers begins. This is shown in the next figure.

Figure 6.18 Complete overview of the pod's normal operation



6.5 Summary

In this chapter, you've learned:

- The status of the pod contains information about the phase of the pod, its conditions, and the status of each of its containers. You can view the status by running the `kubectl describe` command or by retrieving the full pod manifest using the command `kubectl get -o yaml`.
- Depending on the pod's restart policy, its containers can be restarted after they are terminated. In reality, a container is never actually restarted. Instead, the old container is destroyed, and a new container is created in its place.
- If a container is repeatedly terminated, an exponentially increasing delay is inserted before each restart. There is no delay for the first restart, then

the delay is 10 seconds and then doubles before each subsequent restart. The maximum delay is 5 minutes and is reset to zero when the container has been running properly for at least twice this time.

- An exponentially increasing delay is also used after each failed attempt to download a container image.
- Adding a liveness probe to a container ensures that the container is restarted when it stops responding. The liveness probe checks the state of the application via an HTTP GET request, by executing a command in the container, or opening a TCP connection to one of the network ports of the container.
- If the application needs a long time to start, a startup probe can be defined with settings that are more forgiving than those in the liveness probe to prevent premature restarting of the container.
- You can define lifecycle hooks for each of the pod's main containers. A post-start hook is invoked when the container starts, whereas a pre-stop hook is invoked when the container must shut down. A lifecycle hook is configured to either send an HTTP GET request or execute a command within the container.
- If a pre-stop hook is defined in the container and the container must terminate, the hook is invoked first. The TERM signal is then sent to the main process in the container. If the process doesn't stop within `terminationGracePeriodSeconds` after the start of the termination sequence, the process is killed.
- When you delete a pod object, all its containers are terminated in parallel. The pod's `deletionGracePeriodSeconds` is the time given to the containers to shut down. By default, it's set to the termination grace period, but can be overridden with the `kubectl delete` command.
- If shutting down a pod takes a long time, it is likely that one of the processes running in it doesn't handle the TERM signal. Adding a TERM signal handler is a better solution than shortening the termination or deletion grace period.

You now understand everything about the operation of containers in pods. In the next chapter you'll learn about the other important component of pods - storage volumes.

7 Attaching storage volumes to Pods

This chapter covers

- Persisting files across container restarts
- Sharing files between containers of the same pod
- Sharing files between pods
- Attaching network storage to pods
- Accessing the host node filesystem from within a pod

The previous two chapters focused on the pod's containers, but they are only half of what a pod typically contains. They are typically accompanied by storage volumes that allow a pod's containers to store data for the lifetime of the pod or beyond, or to share files with the other containers of the pod. This is the focus of this chapter.

Note

You'll find the code files for this chapter at <https://github.com/luksa/kubernetes-in-action-2nd-edition/tree/master/Chapter07>

7.1 Introducing volumes

A pod is like a small logical computer that runs a single application. This application can consist of one or more containers that run the application processes. These processes share computing resources such as CPU, RAM, network interfaces, and others. In a typical computer, the processes use the same filesystem, but this isn't the case with containers. Instead, each container has its own isolated filesystem provided by the container image.

When a container starts, the files in its filesystem are those that were added to its container image during build time. The process running in the container can then modify those files or create new ones. When the container is

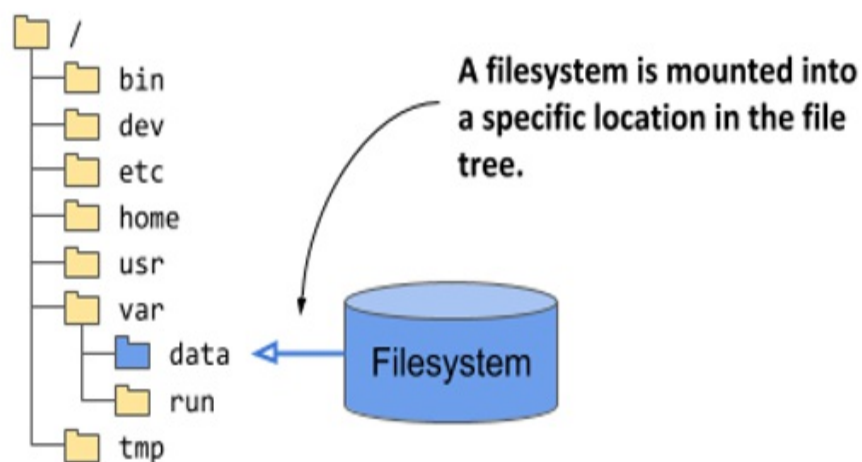
terminated and restarted, all changes it made to its files are lost, because the previous container is not really restarted, but completely replaced, as explained in the previous chapter. Therefore, when a containerized application is restarted, it can't continue from the point where it was when it stopped. Although this may be okay for some types of applications, others may need the entire filesystem or at least part of it to be preserved on restart.

This is achieved by adding a *volume* to the pod and *mounting* it into the container.

Definition

Mounting is the act of attaching the filesystem of some storage device or volume into a specific location in the operating system's file tree, as shown in figure 7.1. The contents of the volume are then available at that location.

Figure 7.1 Mounting a filesystem into the file tree



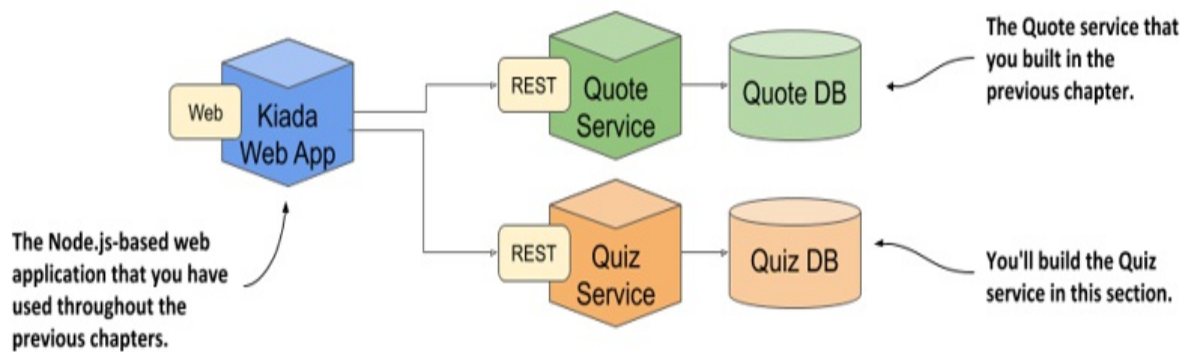
7.1.1 Demonstrating the need for volumes

In this chapter, you'll build a new service that requires its data to be persisted. To do this, the pod that runs the service will need to contain a volume. But before we get to that, let me tell you about this service, and allow you to experience first-hand why it can't work without a volume.

Introducing the Quiz service

The first 14 chapters of this book aim to teach you about the main Kubernetes concepts by showing you how to deploy the Kubernetes in Action Demo Application Suite. You already know the three components that comprise it. If not, the following figure should refresh your memory.

Figure 7.2 How the Quiz service fits into the architecture of the Kiada Suite

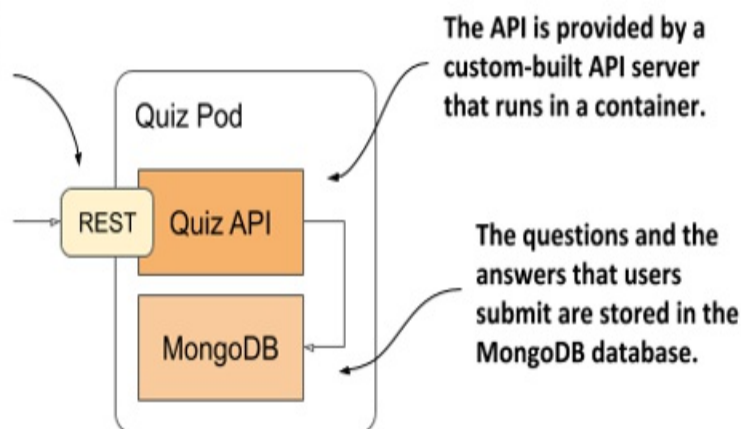


You've already built the initial version of the Kiada web application and the Quote service. Now you'll create the Quiz Service. It will provide the multiple-choice questions that the Kiada web application displays and store your answers to those questions.

The Quiz service consists of a RESTful API frontend and a MongoDB database as the backend. Initially, you'll run these two components in separate containers of the same pod, as shown in the following figure.

Figure 7.3 The Quiz API and the MongoDB database run in the same pod

The Quiz pod provides a RESTful API endpoint that clients use to retrieve questions and post answers.



As I explained in the pod introduction in chapter 5, creating pods like this is not the best idea, as it doesn't allow for the containers to be scaled individually. The reason we'll use a single pod is because you haven't yet learned the correct way to make pods communicate with each other. You'll learn this in chapter 11. That's when you'll split the two containers into separate pods.

Building the Quiz API container

The source code and the artefacts for the container image for the Quiz API component are in the `Chapter07/quiz-api-0.1/` directory. The code is written in Go and built using a container. This may need further explanation for some readers. Instead of having to install the Go environment on your own computer to build the binary file from the Go source code, you build it in a container that already contains the Go environment. The result of the build is the `quiz-api` binary executable file that is written to the `Chapter07/quiz-api-0.1/app/bin/` directory.

This file is then packaged into the `quiz-api:0.1` container image with a separate `docker build` command. If you wish, you can try building the binary and the container image yourself, but you can also use the image that I've built. It's available at `docker.io/luksa/quiz-api:0.1`.

Running the Quiz service in a pod without a volume

The following listing shows the YAML manifest of the quiz pod. You can find it in the file `Chapter07/pod.quiz.novolume.yaml`.

Listing 7.1 The Quiz pod with no volume

```
apiVersion: v1
kind: Pod
metadata:
  name: quiz
spec:      #A
  containers:
    - name: quiz-api      #B
      image: luksa/quiz-api:0.1      #B
      ports:
        - name: http      #C
          containerPort: 8080      #C
    - name: mongo      #C
      image: mongo      #C
```

The listing shows that two containers are defined in the pod. The `quiz-api` container runs the Quiz API component explained earlier, and the `mongo` container runs the MongoDB database that the API component uses to store data.

Create the pod from the manifest and use `kubectl port-forward` to open a tunnel to the pod's port 8080 so that you can talk to the Quiz API. To get a random question, send a GET request to the `/questions/random` URI as follows:

```
$ curl localhost:8080/questions/random
ERROR: Question random not found
```

The database is still empty. You need to add questions to it.

Adding questions to the database

The Quiz API doesn't provide a way to add questions to the database, so you'll have to insert it directly. You can do this via the mongo shell that's available in the mongo container. Use `kubectl exec` to run the shell like this:

```
$ kubectl exec -it quiz -c mongo -- mongo
```

```
MongoDB shell version v4.4.2
connecting to: mongodb://127.0.0.1:27017/...
Implicit session: session { "id" : UUID("42671520-0cf7-...") }
MongoDB server version: 4.4.2
Welcome to the MongoDB shell.
...
```

The Quiz API reads the questions from the questions collection in the kiada database. To add a question to that collection, type the following two commands (printed in bold):

```
> use kiada
switched to db kiada
> db.questions.insert({
... id: 1,
... text: "What does k8s mean?",
... answers: ["Kates", "Kubernetes", "Kooba Dooba Doo!"],
... correctAnswerIndex: 1})
WriteResult({ "nInserted" : 1 })
```

Note

Instead of typing all these commands, you can simply run the Chapter07/insert-question.sh shell script on your local computer to insert the question.

Feel free to add additional questions. When you're done, exit the shell by pressing Control-D or typing the exit command.

Reading questions from the database and the Quiz API

To confirm that the questions that you've just inserted are now stored in the database, run the following command:

```
> db.questions.find()
{ "_id" : ObjectId("5fc249ac18d1e29fed666ab7"), "id" : 1, "text"
"answers" : [ "Kates", "Kubernetes", "Kooba Dooba Doo!" ], "corre
```

Now try to retrieve a random question through the Quiz API:

```
$ curl localhost:8080/questions/random
{"id":1,"text":"What does k8s mean?","correctAnswerIndex":1,
```

```
"answers":["Kates","Kubernetes","Kooba Dooba Doo!"]}]}
```

Good. It looks like the quiz pod provides the service we need for the Kiada Suite. But is that always the case?

Restarting the MongoDB database

Because the MongoDB database writes its files to the container's filesystem, they are lost every time the container is restarted. You can confirm this by telling the database to shut down with the following command:

```
$ kubectl exec -it quiz -c mongo -- mongo admin --eval "db.shutdown"
```

When the database shuts down, the container stops, and Kubernetes starts a new one in its place. Because this is now a new container, with a fresh filesystem, it doesn't contain the questions you entered earlier. You can confirm this is true with the following command:

```
$ kubectl exec -it quiz -c mongo -- mongo kiada --quiet --eval "d
0      #A
```

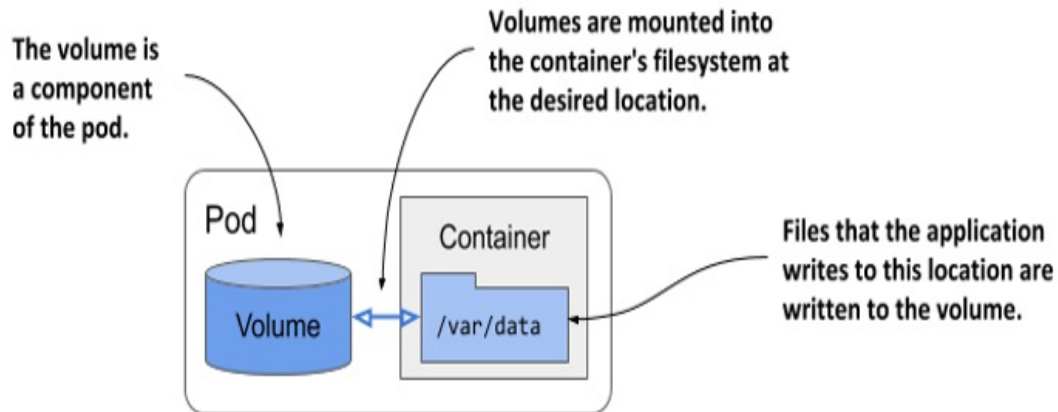
Keep in mind that the quiz pod is still the same pod as before. The quiz-api container has been running fine this whole time. Only the mongo container was restarted. To be perfectly accurate, it was re-created, not restarted. You caused this by shutting down MongoDB, but it could happen for any reason. You'll agree that it's not acceptable that a simple restart causes data to be lost.

To ensure that the data is persisted, it needs to be stored outside of the container - in a volume.

7.1.2 Understanding how volumes fit into pods

Like containers, volumes aren't top-level resources like pods or nodes, but are a component within the pod and thus share its lifecycle. As the following figure shows, a volume is defined at the pod level and then mounted at the desired location in the container.

Figure 7.4 Volumes are defined at the pod level and mounted in the pod's containers



The lifecycle of a volume is tied to the lifecycle of the entire pod and is independent of the lifecycle of the container in which it is mounted. Due to this fact, volumes are also used to persist data across container restarts.

Persisting files across container restarts

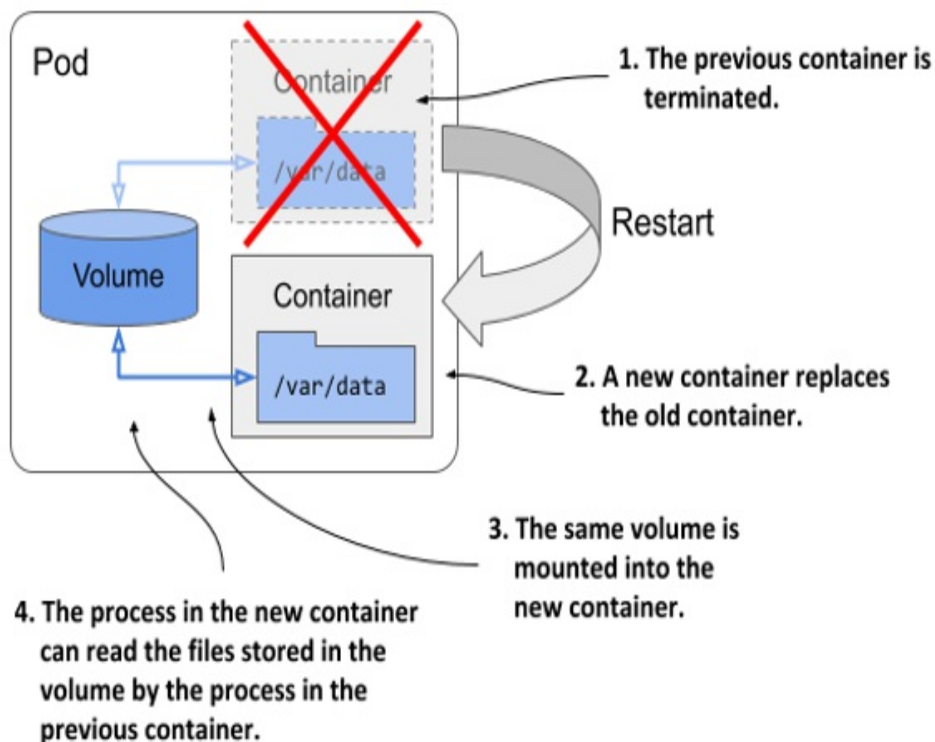
All volumes in a pod are created when the pod is set up - before any of its containers are started. They are torn down when the pod is shut down.

Each time a container is (re)started, the volumes that the container is configured to use are mounted in the container's filesystem. The application running in the container can read from the volume and write to it if the volume and mount are configured to be writable.

A typical reason for adding a volume to a pod is to persist data across container restarts. If no volume is mounted in the container, the entire filesystem of the container is ephemeral. Since a container restart replaces the entire container, its filesystem is also re-created from the container image. As a result, all files written by the application are lost.

If, on the other hand, the application writes data to a volume mounted inside the container, as shown in the following figure, the application process in the new container can access the same data after the container is restarted.

Figure 7.5 Volumes ensure that part of the container's filesystem is persisted across restarts



It is up to the author of the application to determine which files must be retained on restart. Normally you want to preserve data representing the application's state, but you may not want to preserve files that contain the application's locally cached data, as this prevents the container from starting fresh when it's restarted. Starting fresh every time may allow the application to heal itself when corruption of the local cache causes it to crash. Just restarting the container and using the same corrupted files could result in an endless crash loop.

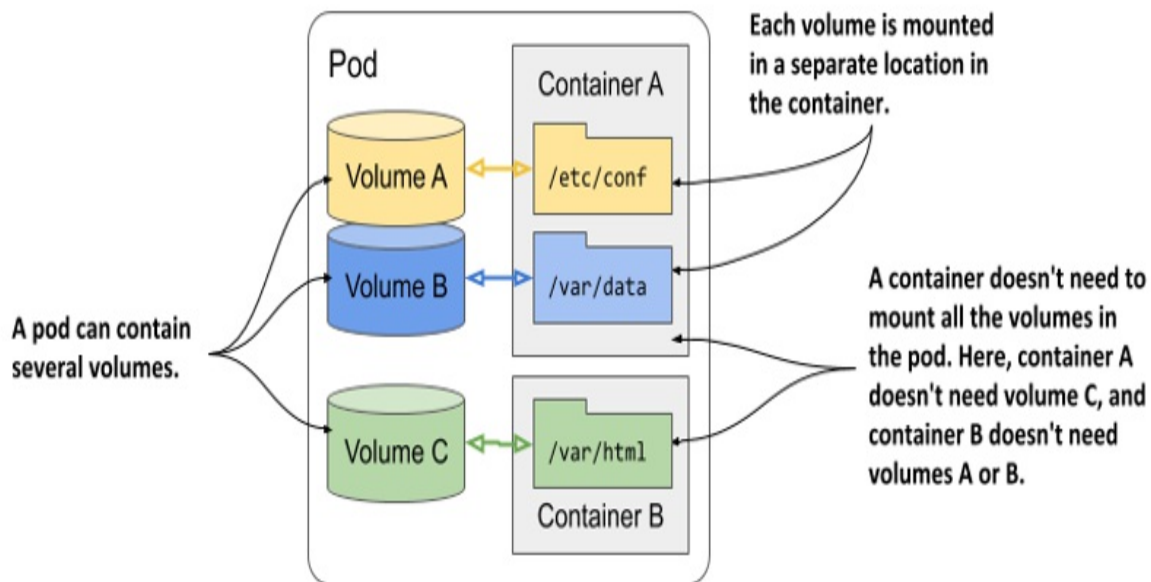
Tip

Before you mount a volume in a container to preserve files across container restarts, consider how this affects the container's self-healing capability.

Mounting multiple volumes in a container

A pod can have multiple volumes and each container can mount zero or more of these volumes in different locations, as shown in the following figure.

Figure 7.6 A pod can contain multiple volumes and a container can mount multiple volumes



The reason why you might want to mount multiple volumes in one container is that these volumes may serve different purposes and can be of different types with different performance characteristics.

In pods with more than one container, some volumes can be mounted in some containers but not in others. This is especially useful when a volume contains sensitive information that should only be accessible to some containers.

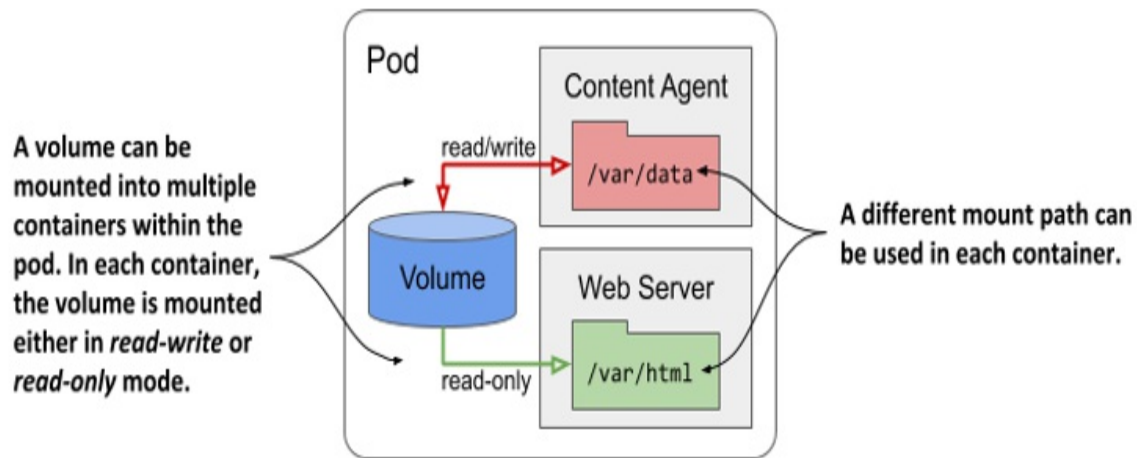
Sharing files between multiple containers

A volume can be mounted in more than one container so that applications running in these containers can share files. As discussed in chapter 5, a pod can combine a main application container with sidecar containers that extend the behavior of the main application. In some cases, the containers must read or write the same files.

For example, you could create a pod that combines a web server running in one container with a content-producing agent running in another container. The content agent container generates the static content that the web server then delivers to its clients. Each of the two containers performs a single task

that has no real value on its own. However, as the next figure shows, if you add a volume to the pod and mount it in both containers, you enable these containers to become a complete system that provides a valuable service and is more than the sum of its parts.

Figure 7.7 A volume can be mounted into more than one container



The same volume can be mounted at different places in each container, depending on the needs of the container itself. If the content agent writes content to `/var/data`, it makes sense to mount the volume there. Since the web server expects the content to be in `/var/html`, the container running it has the volume mounted at this location.

In the figure you'll also notice that the volume mount in each container can be configured either as *read/write* or as *read-only*. Because the content agent needs to write to the volume whereas the web server only reads from it, the two mounts are configured differently. In the interest of security, it's advisable to prevent the web server from writing to the volume, since this could allow an attacker to compromise the system if the web server software has a vulnerability that allows attackers to write arbitrary files to the filesystem and execute them.

Other examples of using a single volume in two containers are cases where a sidecar container runs a tool that processes or rotates the web server logs or when an init container creates configuration files for the main application

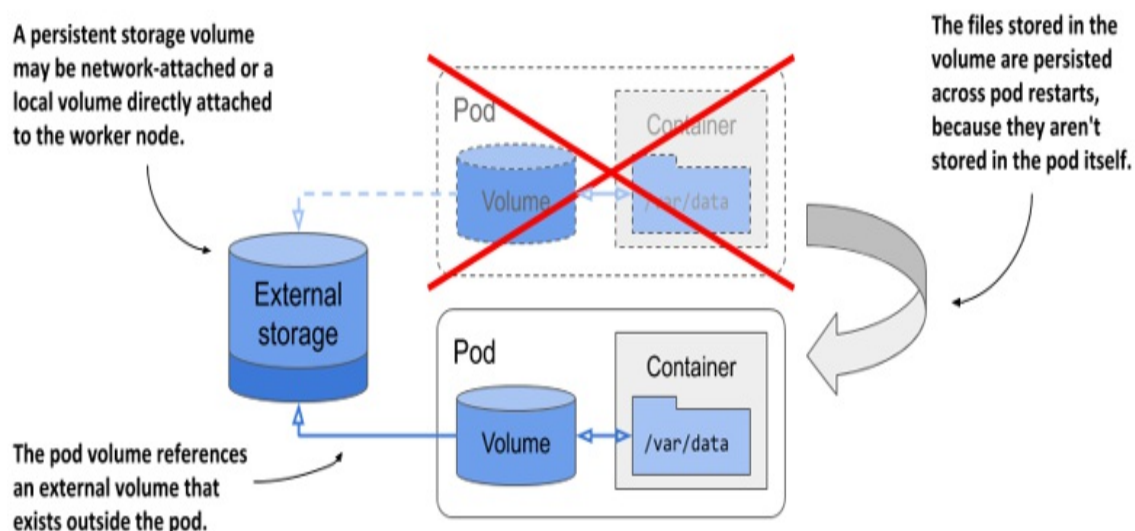
container.

Persisting data across pod instances

A volume is tied to the lifecycle of the pod and only exists for as long as the pod exists, but depending on the volume type, the files in the volume can remain intact after the pod and volume disappear and can later be mounted into a new volume.

As the following figure shows, a pod volume can map to persistent storage outside the pod. In this case, the file directory representing the volume isn't a local file directory that persists data only for the duration of the pod, but is instead a volume mount to an existing, typically network-attached storage volume (NAS) whose lifecycle isn't tied to any pod. The data stored in the volume is thus persistent and can be used by the application even after the pod it runs in is replaced with a new pod running on a different worker node.

Figure 7.8 Pod volumes can also map to storage volumes that persist across pod restarts

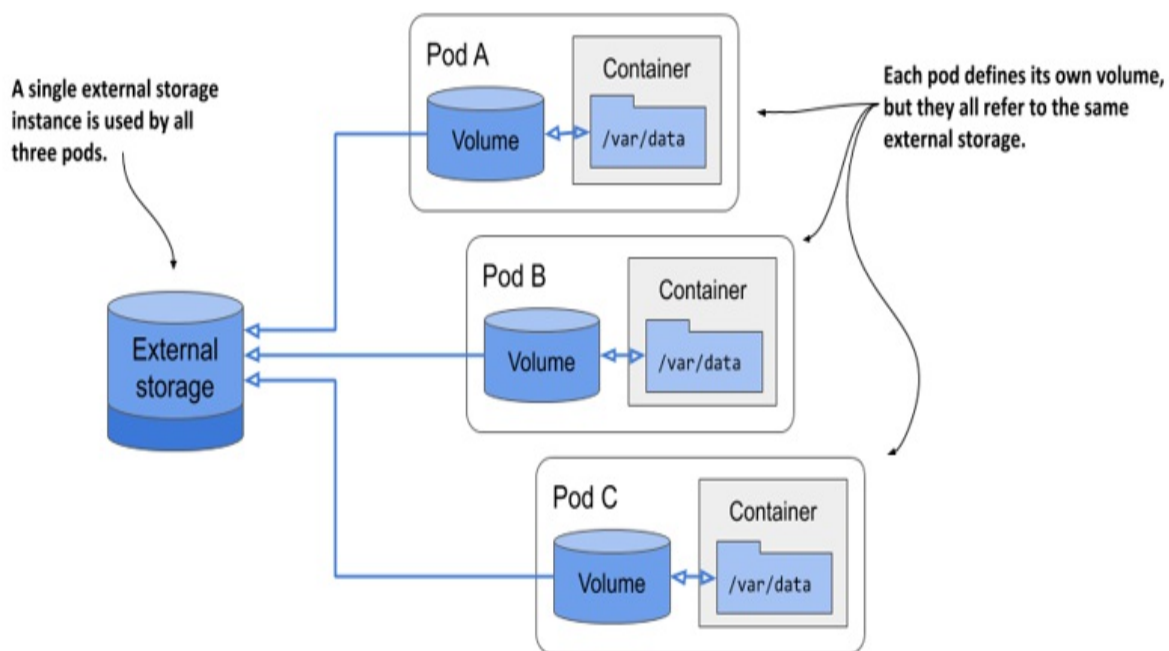


If the pod is deleted and a new pod is created to replace it, the same network-attached storage volume can be attached to the new pod instance so that it can access the data stored there by the previous instance.

Sharing data between pods

Depending on the technology that provides the external storage volume, the same external volume can be attached to multiple pods simultaneously, allowing them to share data. The following figure shows a scenario where three pods each define a volume that is mapped to the same external persistent storage volume.

Figure 7.9 Using volumes to share data between pods



In the simplest case, the persistent storage volume could be a simple local directory on the worker node's filesystem, and the three pods have volumes that map to that directory. If all three pods are running on the same node, they can share files through this directory.

If the persistent storage is a network-attached storage volume, the pods may be able to use it even when they are deployed to different nodes. However, this depends on whether the underlying storage technology supports concurrently attaching the network volume to multiple computers.

While technologies such as Network File System (NFS) allow you to attach

the volume in read/write mode on multiple computers, other technologies typically available in cloud environments, such as the Google Compute Engine Persistent Disk, allow the volume to be used either in read/write mode on a single cluster node, or in read-only mode on many nodes.

Introducing the available volume types

When you add a volume to a pod, you must specify the volume type. A wide range of volume types is available. Some are generic, while others are specific to the storage technologies used underneath. Here's a non-exhaustive list of the supported volume types:

- `emptyDir`—A simple directory that allows the pod to store data for the duration of its life cycle. The directory is created just before the pod starts and is initially empty - hence the name. The `gitRepo` volume, which is now deprecated, is similar, but is initialized by cloning a Git repository. Instead of using a `gitRepo` volume, it is recommended to use an `emptyDir` volume and initialize it using an `init` container.
- `hostPath`—Used for mounting files from the worker node's filesystem into the pod.
- `nfs`—An NFS share mounted into the pod.
- `gcePersistentDisk` (Google Compute Engine Persistent Disk), `awsElasticBlockStore` (Amazon Web Services Elastic Block Store), `azureFile` (Microsoft Azure File Service), `azureDisk` (Microsoft Azure Data Disk)—Used for mounting cloud provider-specific storage.
- `cephfs`, `cinder`, `fc`, `flexVolume`, `flocker`, `glusterfs`, `iscsi`, `portworxVolume`, `quobyte`, `rbd`, `scaleIO`, `storageos`, `photonPersistentDisk`, `vsphereVolume`—Used for mounting other types of network storage.
- `configMap`, `secret`, `downwardAPI`, and the projected volume type—Special types of volumes used to expose information about the pod and other Kubernetes objects through files. They are typically used to configure the application running in the pod. You'll learn about them in chapter 9.
- `persistentVolumeClaim`—A portable way to integrate external storage into pods. Instead of pointing directly to an external storage volume, this volume type points to a `PersistentVolumeClaim` object that points to a

PersistentVolume object that finally references the actual storage. This volume type requires a separate explanation, which you'll find in the next chapter.

- **csi**—A pluggable way of adding storage via the Container Storage Interface. This volume type allows anyone to implement their own storage driver that is then referenced in the `csi` volume definition. During pod setup, the CSI driver is called to attach the volume to the pod.

These volume types serve different purposes. The following sections cover the most representative volume types and help you to gain a general understanding of volumes.

7.2 Using an `emptyDir` volume

The simplest volume type is `emptyDir`. As its name suggests, a volume of this type starts as an empty directory. When this type of volume is mounted in a container, files written by the application to the path where the volume is mounted are preserved for the duration of the pod's existence.

This volume type is used in single-container pods when data must be preserved even if the container is restarted. It's also used when the container's filesystem is marked read-only, and you want part of it to be writable. In pods with two or more containers, an `emptyDir` volume is used to share data between them.

7.2.1 Persisting files across container restarts

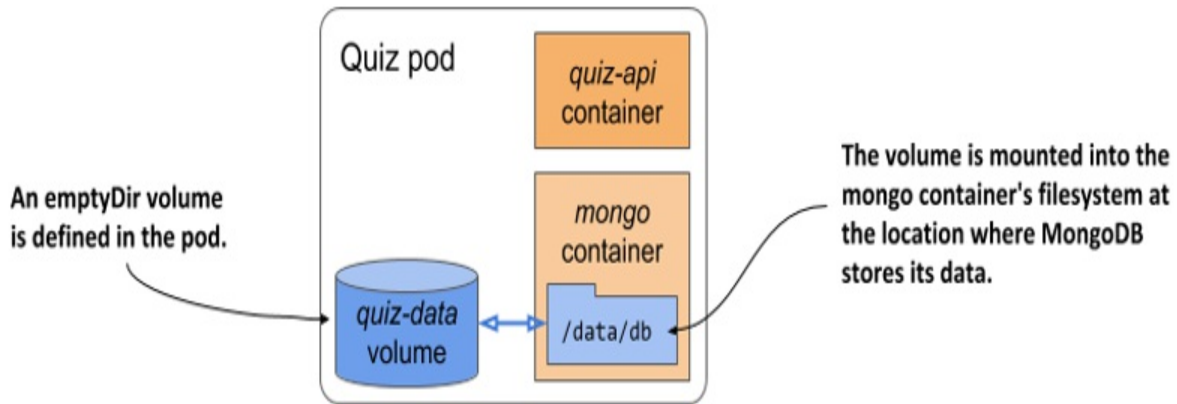
Let's add an `emptyDir` volume to the quiz pod from section 7.1.1 to ensure that its data isn't lost when the MongoDB container restarts.

Adding an `emptyDir` volume to a pod

You'll modify the definition of the quiz pod so that the MongoDB process writes its files to the volume instead of the filesystem of the container it runs in, which is perishable. A visual representation of the pod is given in the next

figure.

Figure 7.10 The quiz pod with an emptyDir volume for storing MongoDB data files



Two changes to the pod manifest are required to achieve this:

1. An emptyDir volume must be added to the pod.
2. The volume must be mounted into the container.

The following listing shows the new pod manifest with these two changes highlighted in bold. You'll find the manifest in the file `pod.quiz.emptydir.yaml`.

Listing 7.2 The quiz pod with an emptyDir volume for the mongo container

```
apiVersion: v1
kind: Pod
metadata:
  name: quiz
spec:
  volumes:      #A
  - name: quiz-data      #A
    emptyDir: {}      #A
  containers:
  - name: quiz-api
    image: luksa/quiz-api:0.1
    ports:
    - name: http
      containerPort: 8080
  - name: mongo
```



```
image: mongo
volumeMounts:    #B
- name: quiz-data    #B
  mountPath: /data/db    #B
```

The listing shows that an `emptyDir` volume named `quiz-data` is defined in the `spec.volumes` array of the pod manifest and that it is mounted into the `mongo` container's filesystem at the location `/data/db`. The following two sections explain more about the volume and the volume mount definitions.

Configuring the `emptyDir` volume

In general, each volume definition must include a name and a type, which is indicated by the name of the nested field (for example: `emptyDir`, `gcePersistentDisk`, `nfs`, and so on). This field typically contains several sub-fields that allow you to configure the volume. The set of sub-fields that you set depends on the volume type.

For example, the `emptyDir` volume type supports two fields for configuring the volume. They are explained in the following table.

Table 7.1 Configuration options for an `emptyDir` volume

Field	Description
<code>medium</code>	The type of storage medium to use for the directory. If left empty, the default medium of the host node is used (the directory is created on one of the node's disks). The only other supported option is <code>Memory</code> , which causes the volume to use <code>tmpfs</code> , a virtual memory filesystem where the files are kept in memory instead of on the hard disk.
<code>sizeLimit</code>	The total amount of local storage required for the directory, whether on disk or in memory. For example, to set the maximum size to ten mebibytes, you set this field to <code>10Mi</code> .

Note

The `emptyDir` field in the volume definition defines neither of these properties. The curly braces `{}` have been added to indicate this explicitly, but they can be omitted.

Mounting the volume in a container

Defining a volume in the pod is only half of what you need to do to make it available in a container. The volume must also be mounted in the container. This is done by referencing the volume by name in the `volumeMounts` array in the container definition.

In addition to the name, a volume mount definition must also include the `mountPath` - the path within the container where the volume should be mounted. In listing 7.2, the volume is mounted at `/data/db` because that's where MongoDB stores its files. You want these files to be written to the volume instead of the container's filesystem, which is ephemeral.

The full list of supported fields in a volume mount definition is presented in the following table.

Table 7.2 Configuration options for a volume mount

Field	Description
<code>name</code>	The name of the volume to mount. This must match one of the volumes defined in the pod.
<code>mountPath</code>	The path within the container at which to mount the volume.

readOnly	Whether to mount the volume as read-only. Defaults to false.
mountPropagation	Specifies what should happen if additional filesystem volumes are mounted inside the volume. Defaults to <code>None</code>, which means that the container won't receive any mounts that are mounted by the host, and the host won't receive any mounts that are mounted by the container. <code>HostToContainer</code> means that the container will receive all mounts that are mounted into this volume by the host, but not the other way around. <code>Bidirectional</code> means that the container will receive mounts added by the host, and the host will receive mounts added by the container.
subPath	Defaults to "" which indicates that the entire volume is to be mounted into the container. When set to a non-empty string, only the specified subPath within the volume is mounted into the container.
subPathExpr	Just like subPath but can have environment variable references using the syntax <code>\$(ENV_VAR_NAME)</code> . Only environment variables that are explicitly defined in the container definition are applicable. Implicit variables such as <code>HOSTNAME</code> will not be resolved. You'll learn how to specify environment variables in chapter 9.

In most cases, you only specify the name, mountPath and whether the mount should be readOnly. The mountPropagation option comes into play for

advanced use-cases where additional mounts are added to the volume's file tree later, either from the host or from the container. The `subPath` and `subPathExpr` options are useful when you want to use a single volume with multiple directories that you want to mount to different containers instead of using multiple volumes.

The `subPathExpr` option is also used when a volume is shared by multiple pod replicas. In chapter 9, you'll learn how to use the Downward API to inject the name of the pod into an environment variable. By referencing this variable in `subPathExpr`, you can configure each replica to use its own subdirectory based on its name.

Understanding the lifespan of an `emptyDir` volume

If you replace the quiz pod with the one in listing 7.2 and insert questions into the database, you'll notice that the questions you add to the database remain intact, regardless of how often the container is restarted. This is because the volume's lifecycle is tied to that of the pod.

To see this is the case, insert the question(s) into the MongoDB database as you did in section 7.1.1. I suggest using the shell script in the file `Chapter07/insert-question.sh` so that you don't have to type the entire question JSON again. After you add the question, count the number of questions in the database as follows:

```
$ kubectl exec -it quiz -c mongo -- mongo kiada --quiet --eval "d
1      #A
```

Now shut down the MongoDB server:

```
$ kubectl exec -it quiz -c mongo -- mongo admin --eval "db.shutdo
```

Check that the mongo container was restarted:

```
$ kubectl get po quiz
NAME    READY   STATUS    RESTARTS   AGE    #A
quiz    2/2     Running   1          10m
```

After the container restarts, recheck the number of questions in the database:

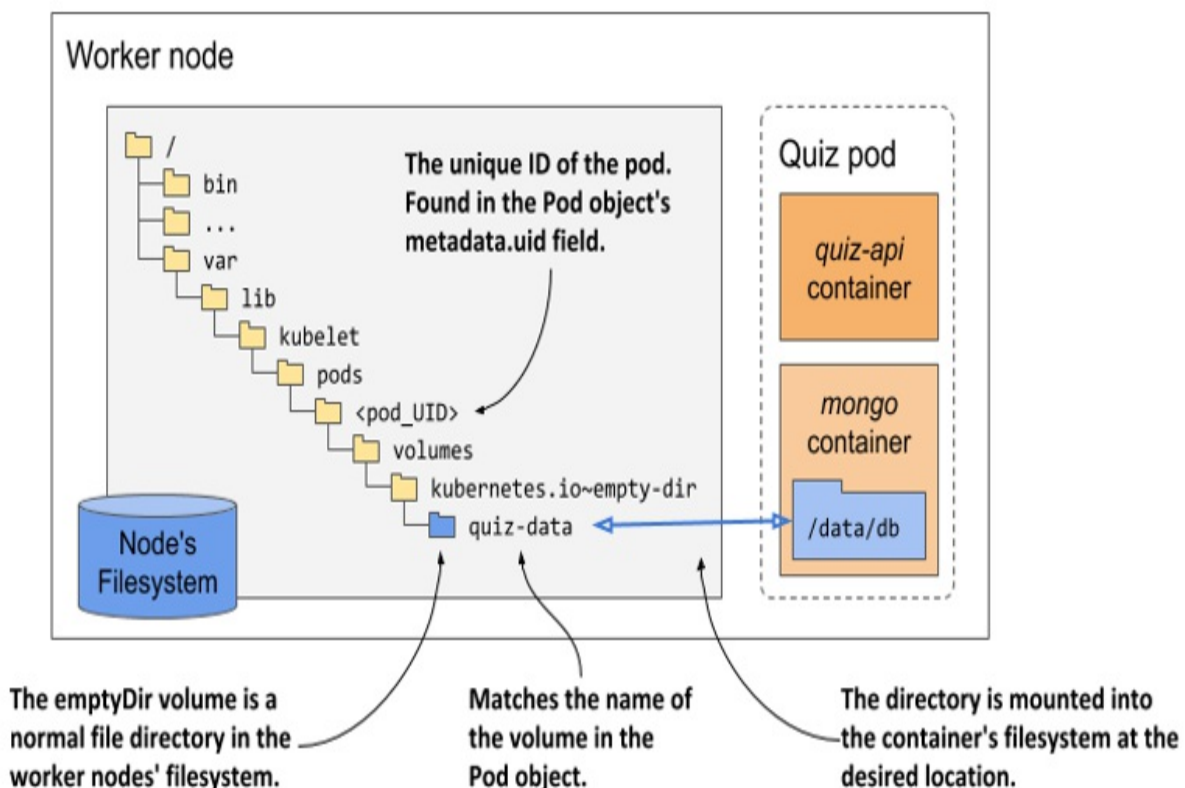
```
$ kubectl exec -it quiz -c mongo -- mongo kiada --quiet --eval "d
1      #A
```

Restarting the container no longer causes the files to disappear because they no longer reside in the container's filesystem. They are stored in the volume. But where exactly is that? Let's find out.

Understanding where the files in an emptyDir volume are stored

As you can see in the following figure, the files in an emptyDir volume are stored in a directory in the host node's filesystem. It's nothing but a normal file directory. This directory is mounted into the container at the desired location.

Figure 7.11 The emptyDir is a normal file directory in the node's filesystem that's mounted into the container



The directory is typically located at the following location in the node's filesystem:

```
/var/lib/kubelet/pods/<pod_UID>/volumes/kubernetes.io~empty-dir/<
```

The `pod_UID` is the unique ID of the pod, which you'll find the Pod object's metadata section. If you want to see the directory for yourself, run the following command to get the `pod_UID`:

```
$ kubectl get po quiz -o json | jq .metadata.uid
"4f49f452-2a9a-4f70-8df3-31a227d020a1"
```

The `volume_name` is the name of the volume in the pod manifest - in the quiz pod, the name is `quiz-data`.

To get the name of the node that runs the pod, use `kubectl get po quiz -o wide` or the following alternative:

```
$ kubectl get po quiz -o json | jq .spec.nodeName
```

Now you have everything you need. Try to log into the node and inspect the contents of the directory. You'll notice that the files match those in the mongo container's `/data/db` directory.

If you delete the pod, the directory is deleted. This means that the data is lost once again. You'll learn how to persist it properly by using external storage volumes in section 7.3.

Creating the `emptyDir` volume in memory

The `emptyDir` volume in the previous example created a directory on the actual drive of the worker node that runs your pod, so its performance depends on the type of drive installed on the node. If you want the I/O operations on the volume to be as fast as possible, you can instruct Kubernetes to create the volume using the *tmpfs* filesystem, which keeps files in memory. To do this, set the `emptyDir`'s `medium` field to `Memory` as in the following snippet:

```
volumes:
- name: content
  emptyDir:
    medium: Memory    #A
```

Creating the `emptyDir` volume in memory is also a good idea whenever it's used to store sensitive data. Because the data is not written to disk, there is less chance that the data will be compromised and persisted longer than desired. As you'll learn in chapter 9, Kubernetes uses the same in-memory approach when it exposes the data from the *Secret* object kind in the container.

Specifying the size limit for the `emptyDir` volume

The size of an `emptyDir` volume can be limited by setting the `sizeLimit` field. Setting this field is especially important for in-memory volumes when the overall memory usage of the pod is limited by so-called *resource limits*. You'll learn about this in chapter 20.

Next, let's see how an `emptyDir` volume is used to share files between containers of the same pod.

7.2.2 Populating an `emptyDir` volume with data using an init container

Every time you create the quiz pod from the previous section, the MongoDB database is empty, and you have to insert the questions manually. Let's improve the pod by automatically populating the database when the pod starts.

Many ways of doing this exist. You could run the MongoDB container locally, insert the data, commit the container state into a new image and use that image in your pod. But then you'd have to repeat the process every time a new version of the MongoDB container image is released.

Fortunately, the MongoDB container image provides a mechanism to populate the database the first time it's started. On start-up, if the database is empty, it invokes any `.js` and `.sh` files that it finds in the `/docker-entrypoint-initdb.d` directory. All you need to do is get the file into that location. Again, you could build a new MongoDB image with the file in that location, but you'd run into the same problem as described previously. An alternative solution is to use a volume to inject the file into that location of the

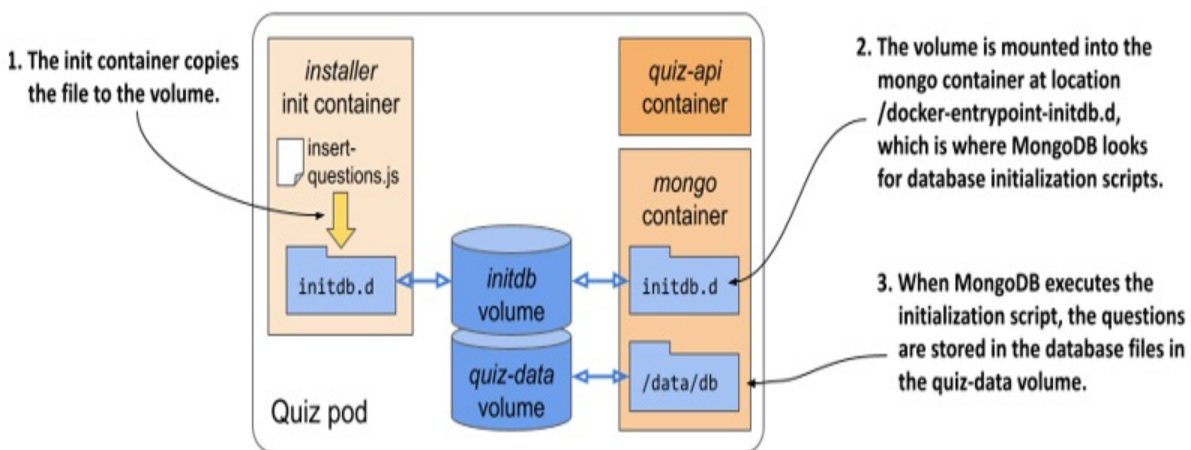
MongoDB container's filesystem. But how do you get the file into the volume in the first place?

Kubernetes provides a special type of volume that is initialized by cloning a Git repository - the `gitRepo` volume. However, this type of volume is now deprecated. The proposed alternative is to use an `emptyDir` volume that you initialize with an `init` container that executes the `git clone` command. You could use this approach, but this would mean that the pod must make a network call to fetch the data.

Another, more generic way of populating an `emptyDir` volume, is to package the data into a container image and copy the data files from the container to the volume when the container starts. This removes the dependency on any external systems and allows the pod to run regardless of the network connectivity status.

To help you visualize the pod, look at the following figure.

Figure 7.12 Using an init container to initialize an `emptyDir` volume



When the pod starts, first the volumes and then the init container is created. The `initdb` volume is mounted into this init container. The container image contains the `insert-questions.js` file, which the container copies to the volume when it runs. Then the copy operation is complete, the init container finishes and the pod's main containers are started. The `initdb` volume is mounted into the mongo container at the location where MongoDB looks for database

initialization scripts. On first start-up, MongoDB executes the `insert-questions.js` script. This inserts the questions into the database. As in the previous version of the pod, the database files are stored in the `quiz-data` volume to allow the data to survive container restarts.

You'll find the `insert-questions.js` file and the Dockerfile required to build `init` container image in the book's code repository. The following listing shows part of the `insert-questions.js` file.

Listing 7.3 The contents of the `insert-questions.js` file

```
db.getSiblingDB("kiada").questions.insertMany(    #A
[
  {    #B
    "id": 1,    #B
    "text": "What is kubect1?",    #B
    ...    #B
```

The Dockerfile for the container image is shown in the next listing.

Listing 7.4 Dockerfile for the `quiz-initdb-script-installer:0.1` container image

```
FROM busybox
COPY insert-questions.js /    #A
CMD cp /insert-questions.js /initdb.d/ \    #B
    && echo "Successfully copied insert-questions.js to /initdb.d"
    || echo "Error copying insert-questions.js to /initdb.d"    #
```

Use these two files to build the image or use the image that I've built. You'll find it at `docker.io/luksa/quiz-initdb-script-installer:0.1`.

After you've got the container image, modify the pod manifest from the previous section so its contents match the next listing (the resulting file is `pod.quiz.emptydir.init.yaml`). The lines that you must add are highlighted in bold font.

Listing 7.5 Using an `init` container to initialize an `emptyDir` volume

```
apiVersion: v1
kind: Pod
metadata:
  name: quiz
```

```

spec:
  volumes:
  - name: initdb      #A
    emptyDir: {}      #A
  - name: quiz-data
    emptyDir: {}
  initContainers:
  - name: installer    #B
    image: luksa/quiz-initdb-script-installer:0.1    #B
    volumeMounts:      #B
    - name: initdb      #B
      mountPath: /initdb.d    #B
  containers:
  - name: quiz-api
    image: luksa/quiz-api:0.1
    ports:
    - name: http
      containerPort: 8080
  - name: mongo
    image: mongo
    volumeMounts:
    - name: quiz-data
      mountPath: /data/db
    - name: initdb      #C
      mountPath: /docker-entrypoint-initdb.d/    #C
      readOnly: true    #C

```

The listing shows that the initdb volume is mounted into the init container. After this container copies the insert-questions.js file to the volume, it terminates and allows the mongo and quiz-api containers to start. Because the initdb volume is mounted in the /docker-entrypoint-initdb.d directory in the mongo container, MongoDB executes the .js file, which populates the database with questions.

You can delete the old quiz pod and deploy this new version of the pod. You'll see that the database gets populated every time you deploy the pod.

7.2.3 Sharing files between containers

As you saw in the previous section, an emptyDir volume can be initialized with an init container and then used by one of the pod's main containers. But a volume can also be used by multiple main containers concurrently. The quiz-api and the mongo containers that are in the quiz pod don't need to share

files, so you'll use a different example to learn how volumes are shared between containers.

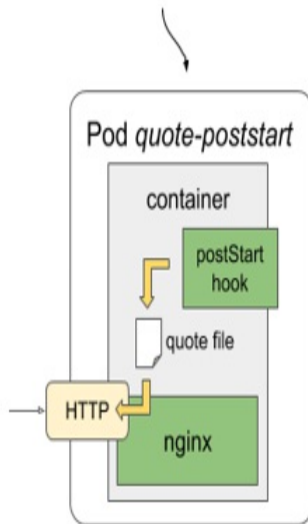
Remember the quote pod from the previous chapter? The one that uses a post-start hook to run the `fortune` command. The command writes a quote from this book into a file that is then served by the Nginx web server. The quote pod currently serves the same quote throughout the lifetime of the pod. This isn't that interesting. Let's build a new version of the pod, where a new quote is served every 60 seconds.

You'll retain Nginx as the web server but will replace the post-start hook with a container that periodically runs the `fortune` command to update the file where the quote is stored. Let's call this container `quote-writer`. The Nginx server will continue to be in the `nginx` container.

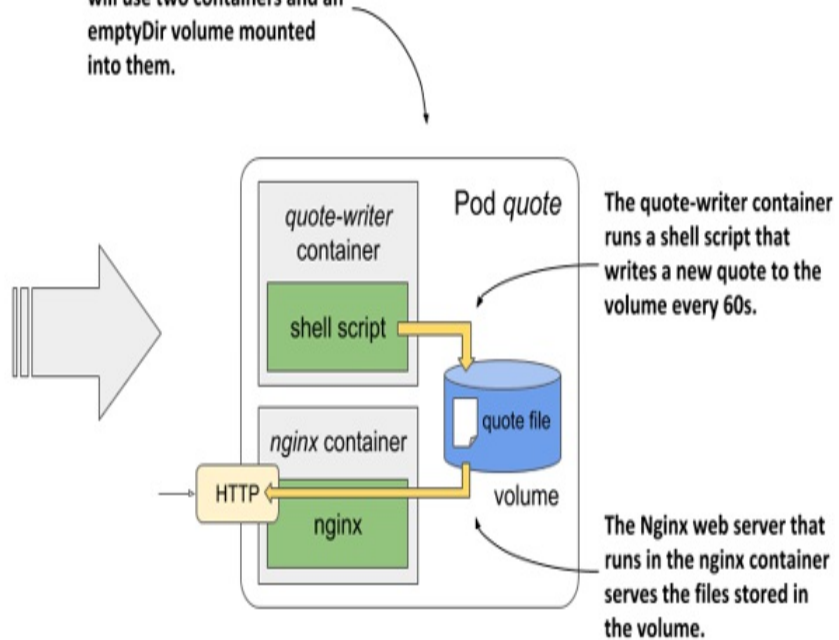
As visualized in the following figure, the pod now has two containers instead of one. To allow the `nginx` container to see the file that the `quote-writer` creates, a volume must be defined in the pod and mounted into both containers.

Figure 7.13 The new version of the Quote service uses two containers and a shared volume

The Quote service was previously implemented as a single-container pod that uses a post-start hook to write the quote file.



The new version of the service will use two containers and an emptyDir volume mounted into them.



Creating a pod with two containers and a shared volume

The image for the quote-writer container is available at `docker.io/luksa/quote-writer:0.1`, but you can also build it yourself from the files in the `Chapter07/quote-writer-0.1` directory. The nginx container will continue to use the existing `nginx:alpine` image.

The pod manifest for the new quote pod is shown in the next listing. You can find it in file `pod.quote.yaml`.

Listing 7.6 A pod with two containers that share a volume

```
apiVersion: v1
kind: Pod
metadata:
  name: quote
spec:
  volumes:      #A
  - name: shared      #A
    emptyDir: {}      #A
  containers:
  - name: quote-writer      #B
```

```

    image: luksa/quote-writer:0.1      #B
    volumeMounts:      #C
    - name: shared      #C
      mountPath: /var/local/output      #C
- name: nginx          #D
  image: nginx:alpine   #D
  volumeMounts:        #E
  - name: shared        #E
    mountPath: /usr/share/nginx/html    #E
    readOnly: true      #E
  ports:
  - name: http
    containerPort: 80

```

The pod consists of two containers and a single volume, which is mounted in both containers, but at a different location in each container. The reason for this is that the quote-writer container writes the quote file to the `/var/local/output` directory, whereas the nginx container serves files from the `/usr/share/nginx/html` directory.

Note

Since the two containers start at the same time, there can be a short period where nginx is already running, but the quote hasn't been generated yet. One way of making sure this doesn't happen is to generate the initial quote using an init container, as explained in section 7.2.3.

Running the pod

When you create the pod from the manifest, the two containers start and continue running until the pod is deleted. The quote-writer container writes a new quote to the file every 60 seconds, and the nginx container serves this file. After you create the pod, use the `kubectl port-forward` command to open a communication tunnel to the pod:

```
$ kubectl port-forward quote 1080:80
```

In another terminal, verify that the server responds with a different quote every 60 seconds by running the following command several times:

```
$ curl localhost:1080/quote
```

Alternatively, you can also display the contents of the file using either of the following two commands:

```
$ kubectl exec quote -c quote-writer -- cat /var/local/output/quote
$ kubectl exec quote -c nginx -- cat /usr/share/nginx/html/quote
```

As you can see, one of them prints the contents of the file from within the `quote-writer` container, whereas the other command prints the contents from within the `nginx` container. Because the two paths point to the same `quote` file on the shared volume, the output of the commands is identical.

7.3 Using external storage in pods

An `emptyDir` volume is a dedicated directory created specifically for and used exclusively by the pod in which the volume is defined. When the pod is deleted, the volume and its contents are deleted. However, other types of volumes don't create a new directory, but instead mount an existing external directory in the filesystem of the container. The contents of this volume can survive multiple instantiations of the same pod and can even be shared by multiple pods. These are the types of volumes we'll explore next.

To learn how external storage is used in a pod, you'll create a pod that runs the document-oriented database MongoDB. To ensure that the data stored in the database is persisted, you'll add a volume to the pod and mount it in the container at the location where MongoDB writes its data files.

The tricky part of this exercise is that the type of persistent volumes available in your cluster depends on the environment in which the cluster is running. At the beginning of this book, you learned that Kubernetes could reschedule a pod to another node at any time. To ensure that the quiz pod can still access its data, it should use network-attached storage instead of the worker node's local drive.

Ideally, you should use a proper Kubernetes cluster, such as GKE, for the following exercises. Unfortunately, clusters provisioned with Minikube or `kind` don't provide any kind of network storage volume out of the box. So, if you're using either of these tools, you'll need to resort to using node-local storage provided by the so-called `hostPath` volume type, but this volume

type is not explained until section 7.4.

7.3.1 Using a Google Compute Engine Persistent Disk as a volume

If you use Google Kubernetes Engine to run the exercises in this book, your cluster nodes run on Google Compute Engine (GCE). In GCE, persistent storage is provided via GCE Persistent Disks. Kubernetes supports adding them to your pods via the `gcePersistentDisk` volume type.

Note

To adapt this exercise for use with other cloud providers, use the appropriate volume type supported by the cloud provider. Consult the documentation provided by the cloud provider to determine how to create the storage volume and how to mount it into the pod.

Creating a GCE Persistent Disk

Before you can use the GCE Persistent Disk volume in your pod, you must create the disk itself. It must reside in the same zone as your Kubernetes cluster. If you don't remember in which zone you created the cluster, you can see it by listing your Kubernetes clusters using the `gcloud` command as follows:

```
$ gcloud container clusters list
NAME      ZONE          MASTER_VERSION  MASTER_IP      ...
kiada     europe-west3-c  1.14.10-gke.42  104.155.84.137  ...
```

In my case, the command output indicates that the cluster is in zone `europe-west3-c`, so I have to create the GCE Persistent Disk there. Create the disk in the correct zone as follows:

```
$ gcloud compute disks create --size=10GiB --zone=europe-west3-c
WARNING: You have selected a disk size of under [200GB]. This may
For more information, see: https://developers.google.com/compute/
Created [https://www.googleapis.com/.../zones/europe-west3-c/disk
NAME      ZONE          SIZE_GB  TYPE          STATUS
quiz-data europe-west3-c  10       pd-standard   READY
```

This command creates a GCE Persistent Disk called quiz-data with 10GiB of space. You can freely ignore the disk size warning, because it doesn't affect the exercises you're about to run. You may also see an additional warning that the disk is not yet formatted. You can ignore that, too, because formatting is done automatically when you use the disk in your pod.

Creating a pod with a gcePersistentDisk volume

Now that you have set up your physical storage, you can use it in a volume inside your quiz pod. You'll create the pod from the YAML in the following listing (file pod.quiz.gcepd.yaml). The highlighted lines are the only difference from the pod.quiz.emptydir.yaml file that you deployed in section 7.2.1.

Listing 7.7 Using a gcePersistentDisk volume in the quiz pod

```
apiVersion: v1
kind: Pod
metadata:
  name: quiz
spec:
  volumes:
    - name: quiz-data
      gcePersistentDisk:      #A
        pdName: quiz-data   #B
        fsType: ext4        #C
  containers:
    - name: quiz-api
      image: luksa/quiz-api:0.1
      ports:
        - name: http
          containerPort: 8080
    - name: mongo
      image: mongo
      volumeMounts:
        - name: quiz-data
          mountPath: /data/db
```

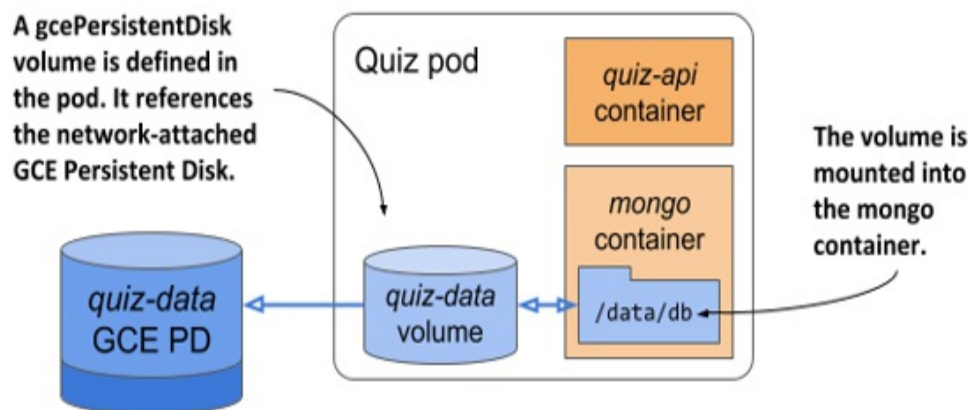
Note

If you created your cluster with Minikube or kind, you can't use a GCE

Persistent Disk. Use the file `pod.quiz.hostpath.yaml`, which uses a `hostPath` volume in place of the GCE PD. This type of volume uses node-local instead of network storage, so you must ensure that the pod is always deployed to the same node. This is always true in Minikube because it creates a single node cluster. However, if you're using `kind`, create the pod from the file `pod.quiz.hostpath.kind.yaml` to ensure that the pod is always deployed to the same node.

The pod is visualized in the following figure. It contains a single volume that refers to the GCE Persistent Disk you created earlier. The volume is mounted in the `mongo` container at `/data/db`. This ensures that MongoDB writes its files to the persistent disk.

Figure 7.14 A GCE Persistent Disk referenced in a pod volume and mounted into the mongo container



Verifying that the GCE Persistent Disk persists data

Use the shell script in the file `Chapter07/insert-question.sh` to add a question to the MongoDB database. Confirm that the question is stored by using the following command:

```
$ kubectl exec -it quiz -c mongo -- mongo kiada --quiet --eval "d
1      #A
```

Okay, the database has the data. MongoDB's data files are stored in the `/data/db` directory, which is where you mounted the GCE Persistent Disk.

Therefore, the data files should be stored on the GCE PD.

You can now safely delete the quiz pod and recreate it:

```
$ kubectl delete pod quiz
pod "quiz" deleted
$ kubectl apply -f pod.quiz.gcepd.yaml
pod "quiz" created
```

Since the new pod is an exact replica of the previous, it points to the same GCE Persistent Disk as the previous pod did. The mongo container should see the files that it wrote earlier, even if the new pod is scheduled to another node.

Tip

You can see what node a pod is scheduled to by running `kubectl get po -o wide`.

Note

If you use a kind-provisioned cluster, the pod is always scheduled to the same node.

After the pod starts, recheck the number of questions in the database:

```
$ kubectl exec -it quiz -c mongo -- mongo kiada --quiet --eval "d
1      #A"
```

As expected, the data still exists even though you deleted and recreated the pod. This confirms that you can use a GCE Persistent Disk to persist data across multiple instantiations of the same pod. To be perfectly precise, it isn't the same pod. These are two pods whose volumes point to the same underlying persistent storage volume.

You might wonder if you can use the same persistent disk in two or more pods at the same time. The answer to this question is not straightforward, because it requires the understanding of how external volumes are mounted in pods. I'll explain this in section 7.3.3. Before I do that, I need to explain

how use external storage when your cluster doesn't run on Google's infrastructure.

7.3.2 Using other persistent volume types

In the previous exercise, I explained how to add persistent storage to a pod running in Google Kubernetes Engine. If you run your cluster elsewhere, you should use whatever volume type is supported by the underlying infrastructure.

For example, if your Kubernetes cluster runs on Amazon's AWS EC2, you can use an `awsElasticBlockStore` volume. If your cluster runs on Microsoft Azure, you can use the `azureFile` or the `azureDisk` volume. I won't go into detail about how to do this, but it's practically the same as in the previous example. You first need to create the actual underlying storage and then set the right fields in the volume definition.

Using an AWS Elastic Block Store volume

For example, if you want to use an AWS Elastic Block Store volume instead of the GCE Persistent Disk, you only need to change the volume definition as shown in the following listing (file `pod.quiz.aws.yaml`).

Listing 7.8 Using an `awsElasticBlockStore` volume in the quiz pod

```
apiVersion: v1
kind: Pod
metadata:
  name: quiz
spec:
  volumes:
    - name: quiz-data
      awsElasticBlockStore:      #A
        volumeID: quiz-data    #B
        fsType: ext4           #C
  containers:
    - ...
```

Using an NFS volume

If your cluster runs on your own servers, you have a range of other supported options for adding external storage to your pods. For example, to mount an NFS share, you specify the NFS server address and the exported path, as shown in the following listing (file `pod.quiz.nfs.yaml`).

Listing 7.9 Using an nfs volume in the quiz pod

```
...
volumes:
- name: quiz-data
  nfs:    #A
    server: 1.2.3.4    #B
    path: /some/path   #C
...
```

Note

Although Kubernetes supports nfs volumes, the operating system running on the worker nodes provisioned by Minikube or kind might not support mounting nfs volumes.

Using other storage technologies

Other supported options are `iscsi` for mounting an iSCSI disk resource, `glusterfs` for a GlusterFS mount, `rbd` for a RADOS Block Device, `flexVolume`, `cinder`, `cephfs`, `flocker`, `fc` (Fibre Channel), and others. You don't need to understand all these technologies. They're mentioned here to show you that Kubernetes supports a wide range of these technologies, and you can use the technologies that are available in your environment or that you prefer.

For details on the properties that you need to set for each of these volume types, you can either refer to the Kubernetes API definitions in the Kubernetes API reference or look up the information by running `kubectl explain pod.spec.volumes`. If you're already familiar with a particular storage technology, you should be able to use the `explain` command to easily find out how to configure the correct volume type (for example, for iSCSI you can see the configuration options by running `kubectl explain pod.spec.volumes.iscsi`).

Why does Kubernetes force software developers to understand low-level storage?

If you're a software developer and not a system administrator, you might wonder if you really need to know all this low-level information about storage volumes? As a developer, should you have to deal with infrastructure-related storage details when writing the pod definition, or should this be left to the cluster administrator?

At the beginning of this book, I explained that Kubernetes abstracts away the underlying infrastructure. The configuration of storage volumes explained earlier clearly contradicts this. Furthermore, including infrastructure-related information, such as the NFS server hostname directly in a pod manifest means that this manifest is tied to this specific Kubernetes cluster. You can't use the same manifest without modification to deploy the pod in another cluster.

Fortunately, Kubernetes offers another way to add external storage to your pods. One that divides the responsibility for configuring and using the external storage volume into two parts. The low-level part is managed by cluster administrators, while software developers only specify the high-level storage requirements for their applications. Kubernetes then connects the two parts.

You'll learn about this in the next chapter, but first you need a basic understanding of pod volumes. You've already learned most of it, but I still need to explain some details.

7.3.3 Understanding how external volumes are mounted

To understand the limitations of using external volumes in your pods, whether a pod references the volume directly or indirectly, as explained in the next chapter, you must be aware of the caveats associated with the way network storage volumes are actually attached to the pods.

Let's return to the issue of using the same network storage volume in multiple pods at the same time. What happens if you create a second pod and

point it to the same GCE Persistent Disk?

I've prepared a manifest for a second MongoDB pod that uses the same GCE Persistent Disk. The manifest can be found in the file `pod.quiz2.gcepd.yaml`. If you use it to create the second pod, you'll notice that it never runs. It never gets past the `ContainerCreating` status:

```
$ kubectl get po
NAME      READY   STATUS             RESTARTS   AGE
quiz      2/2     Running            0           10m
quiz2     0/2     ContainerCreating  0           2m
```

Note

If your GKE cluster has a single worker node and the pod's status is `Pending`, the reason could be that there isn't enough unallocated CPU for the pod to fit on the node. Resize the cluster to at least two nodes with the command `gcloud container clusters resize <cluster-name> --size <number-of-nodes>`.

You can see why this is the case with the `kubectl describe pod quiz2` command. At the very bottom, you see a `FailedAttachVolume` event generated by the `attachdetach-controller`. The event has the following message:

```
AttachVolume.Attach failed for volume "quiz-data" : googleapi: Er
RESOURCE_IN_USE_BY_ANOTHER_RESOURCE -      #A
The disk resource
'projects/kiada/zones/europe-west3-c/disks/quiz-data' is already
'projects/kiada/zones/europe-west3-c/instances/gke-kiada-default-
```

The message indicates that the node hosting the `quiz2` pod can't attach the external volume because it's already in use by another node. If you check where the two pods are scheduled, you'll see that they are not on the same node:

```
$ kubectl get po -o wide
NAME      READY   STATUS             ... NODE
quiz      2/2     Running            ... gke-kiada-default-pool-xyz-
quiz2     0/2     ContainerCreating  ... gke-kiada-default-pool-xyz-
```

The quiz pod runs on node xyz-1b27, whereas quiz2 is on node xyz-gqbj. As is typically the case in cloud environments, you can't mount the same GCE Persistent Disk on multiple hosts simultaneously in read/write mode. You can only mount it on multiple hosts if you use the read-only mode.

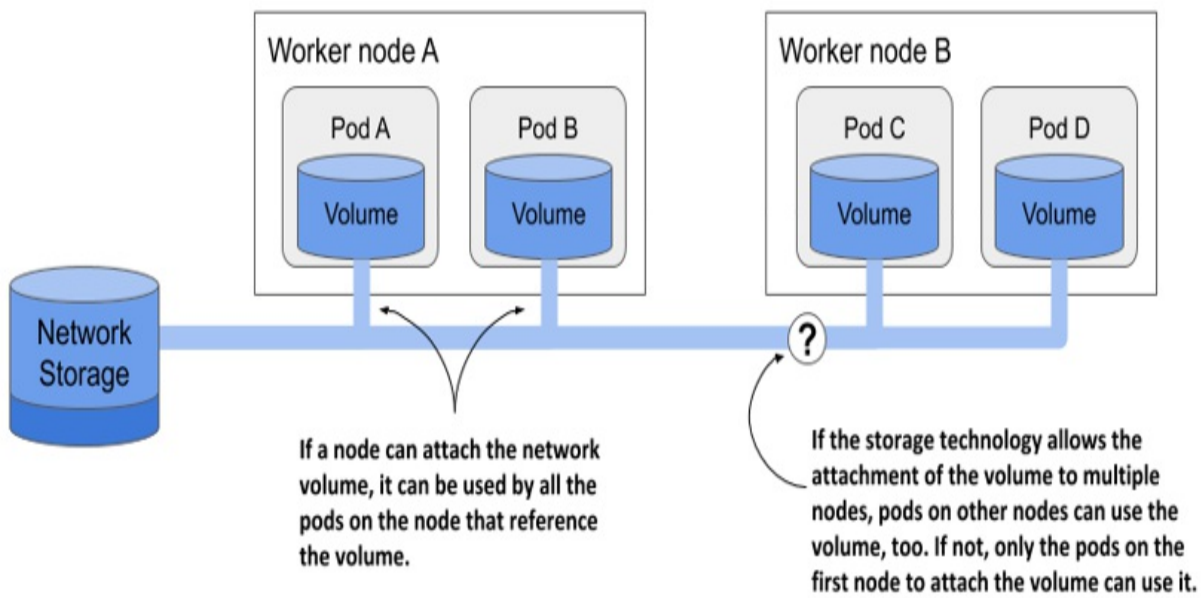
Interestingly, the error message doesn't say that the disk is being used by the quiz pod, but by the node hosting the pod. This is an often overlooked detail about how external volumes are mounted into pods.

Tip

Use the following command to see which network volumes that are attached to a node: `kubectl get node <node-name> -o json | jq .status.volumesAttached`.

As the following figure shows, a network volume is mounted by the host node, and then the pod is given access to the mount point. The underlying storage technology may not allow a volume to be attached to more than one node at a time in read/write mode, but multiple pods on the same node *can* all use the volume in read/write mode.

Figure 7.15 Network volumes are mounted by the host node and then exposed in pods



For most storage technologies available in the cloud, you can typically use the same network volume on multiple nodes simultaneously if you mount them in read-only mode. For example, pods scheduled to different nodes can use the same GCE Persistent Disk if it is mounted in read-only mode, as shown in the next listing.

Listing 7.10 Mounting a GCE Persistent Disk in read-only mode

```
kind: Pod
spec:
  volumes:
  - name: my-volume
    gcePersistentDisk:
      pdName: my-volume
      fsType: ext4
      readOnly: true      #A
```

It is important to consider this network storage limitation when designing the architecture of your distributed application. Replicas of the same pod typically can't use the same network volume in read/write mode. Fortunately, Kubernetes takes care of this, too. In chapter 13, you'll learn how to deploy stateful applications, where each pod instance gets its own network storage volume.

You're now done playing with these two quiz pods, so you can delete them. But don't delete the underlying GCE Persistent Disk yet. You'll use it again in the next chapter.

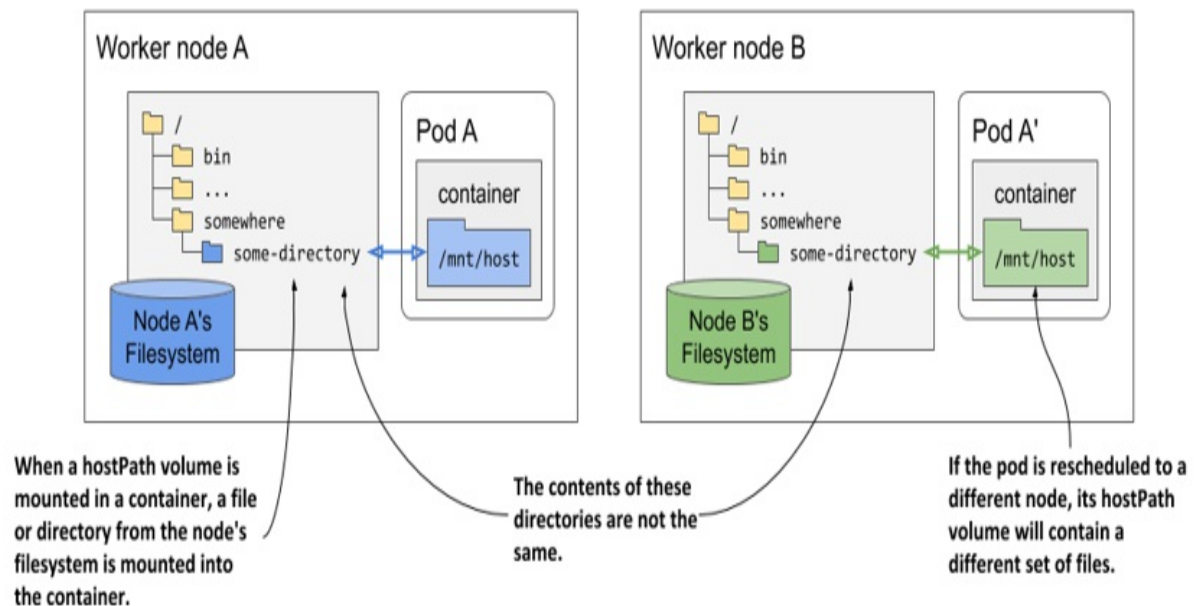
7.4 Accessing files on the worker node's filesystem

Most pods shouldn't care which host node they are running on, and they shouldn't access any files on the node's filesystem. System-level pods are the exception. They may need to read the node's files or use the node's filesystem to access the node's devices or other components via the filesystem. Kubernetes makes this possible through the `hostPath` volume type. I already mentioned it in the previous section, but this is where you'll learn when to actually use it.

7.4.1 Introducing the `hostPath` volume

A `hostPath` volume points to a specific file or directory in the filesystem of the host node, as shown in the next figure. Pods running on the same node and using the same path in their `hostPath` volume have access to the same files, whereas pods on other nodes do not.

Figure 7.16 A `hostPath` volume mounts a file or directory from the worker node's filesystem into the container.



A `hostPath` volume is not a good place to store the data of a database unless you ensure that the pod running the database always runs on the same node. Because the contents of the volume are stored on the filesystem of a specific node, the database pod will not be able to access the data if it gets rescheduled to another node.

Typically, a `hostPath` volume is used in cases where the pod needs to read or write files in the node's filesystem that the processes running on the node read or generate, such as system-level logs.

The `hostPath` volume type is one of the most dangerous volume types in Kubernetes and is usually reserved for use in privileged pods only. If you allow unrestricted use of the `hostPath` volume, users of the cluster can do anything they want on the node. For example, they can use it to mount the Docker socket file (typically `/var/run/docker.sock`) in their container and

then run the Docker client within the container to run any command on the host node as the root user. You'll learn how to prevent this in chapter 24.

7.4.2 Using a hostPath volume

To demonstrate how dangerous hostPath volumes are, let's deploy a pod that allows you to explore the entire filesystem of the host node from within the pod. The pod manifest is shown in the following listing.

Listing 7.11 Using a hostPath volume to gain access to the host node's filesystem

```
apiVersion: v1
kind: Pod
metadata:
  name: node-explorer
spec:
  volumes:
    - name: host-root          #A
      hostPath:                #A
        path: /                #A
  containers:
    - name: node-explorer
      image: alpine
      command: ["sleep", "9999999999"]
      volumeMounts:
        - name: host-root      #B
          mountPath: /host      #B
```

As you can see in the listing, a hostPath volume must specify the path on the host that it wants to mount. The volume in the listing will point to the root directory on the node's filesystem, providing access to the entire filesystem of the node the pod is scheduled to.

After creating the pod from this manifest using `kubectl apply`, run a shell in the pod with the following command:

```
$ kubectl exec -it node-explorer -- sh
```

You can now navigate to the root directory of the node's filesystem by running the following command:

```
/ # cd /host
```

From here, you can explore the files on the host node. Since the container and the shell command are running as root, you can modify any file on the worker node. Be careful not to break anything.

Note

If your cluster has more than one worker node, the pod runs on a randomly selected one. If you'd like to deploy the pod on a specific node, edit the file `node-explorer.specific-node.pod.yaml`, which you'll find in the book's code archive, and set the `.spec.nodeName` field to the name of the node you'd like to run the pod on. You'll learn about scheduling pods to a specific node or a set of nodes in later chapters.

Now imagine you're an attacker that has gained access to the Kubernetes API and are able to deploy this type of pod in a production cluster. Unfortunately, at the time of writing, Kubernetes doesn't prevent regular users from using `hostPath` volumes in their pods and is therefore totally unsecure. As already mentioned, you'll learn how to secure the cluster from this type of attack in chapter 24.

Specifying the type for a `hostPath` volume

In the previous example, you only specified the path for the `hostPath` volume, but you can also specify the type to ensure that the path represents what the process in the container expects (a file, a directory, or something else).

The following table explains the supported `hostPath` types:

Table 7.3 Supported `hostPath` volume types

Type	Description
<code><empty></code>	Kubernetes performs no checks before it mounts the volume.

Directory	Kubernetes checks if a directory exists at the specified path. You use this type if you want to mount a pre-existing directory into the pod and want to prevent the pod from running if the directory doesn't exist.
DirectoryOrCreate	Same as <code>Directory</code> , but if nothing exists at the specified path, an empty directory is created.
File	The specified path must be a file.
FileOrCreate	Same as <code>File</code> , but if nothing exists at the specified path, an empty file is created.
BlockDevice	The specified path must be a block device.
CharDevice	The specified path must be a character device.
Socket	The specified path must be a UNIX socket.

If the specified path doesn't match the type, the pod's containers don't run. The pod's events explain why the `hostPath` type check failed.

Note

When the type is `FileOrCreate` or `DirectoryOrCreate` and Kubernetes needs to create the file/directory, its file permissions are set to 644 (`rw-r--r--`) and 755 (`rw-xr-x`), respectively. In either case, the file/directory is owned by the user and group used to run the Kubelet.

7.5 Summary

This chapter has explained the basics of adding volumes to pods, but this was only the beginning. You'll learn more about this topic in the next chapter. So far, you've learned the following:

- Pods consist of containers and volumes. Each volume can be mounted at the desired location in the container's filesystem.
- Volumes are used to persist data across container restarts, share data between the containers in the pod, and even share data between the pods.
- Many volume types exist. Some are generic and can be used in any cluster regardless of the cluster environment, while others, such as the `gcePersistentDisk`, can only be used if the cluster runs on a specific cloud provider's infrastructure.
- An `emptyDir` volume is used to store data for the duration of the pod. It starts as an empty directory just before the pod's containers are started and is deleted when the pod terminates.
- The `gitRepo` volume is a deprecated volume type that is initialized by cloning a Git repository. Alternatively, an `emptyDir` volume can be used in combination with an `init` container that initializes the volume from Git or any other source.
- Network volumes are typically mounted by the host node and then exposed to the pod(s) on that node.
- Depending on the underlying storage technology, you may or may not be able to mount a network storage volume in read/write mode on multiple nodes simultaneously.
- By using a proprietary volume type in a pod manifest, the pod manifest is tied to a specific Kubernetes cluster. The manifest must be modified before it can be used in another cluster. Chapter 8 explains how to avoid this issue.
- The `hostPath` volume allows a pod to access any path in filesystem of the worker node. This volume type is dangerous because it allows users to make changes to the configuration of the worker node and run any process they want on the node.

In the next chapter, you'll learn how to abstract the underlying storage

technology away from the pod manifest and make the manifest portable to any other Kubernetes cluster.

8 Persisting data in PersistentVolumes

This chapter covers

- Using PersistentVolume objects to represent persistent storage
- Claiming persistent volumes with PersistentVolumeClaim objects
- Dynamic provisioning of persistent volumes
- Using node-local persistent storage

The previous chapter taught you how to mount a network storage volume into your pods. However, the experience was not ideal because you needed to understand the environment your cluster was running in to know what type of volume to add to your pod. For example, if your cluster runs on Google's infrastructure, you must define a `gcePersistentDisk` volume in your pod manifest. You can't use the same manifest to run your application on Amazon because GCE Persistent Disks aren't supported in their environment. To make the manifest compatible with Amazon, one must modify the volume definition in the manifest before deploying the pod.

You may remember from chapter 1 that Kubernetes is supposed to standardize application deployment between cloud providers. Using proprietary storage volume types in pod manifests goes against this premise.

Fortunately, there is a better way to add persistent storage to your pods. One where you don't refer to a specific storage technology within the pod. This chapter explains this improved approach.

Note

You'll find the code files for this chapter at <https://github.com/luksa/kubernetes-in-action-2nd-edition/tree/master/Chapter08>

8.1 Decoupling pods from the underlying storage technology

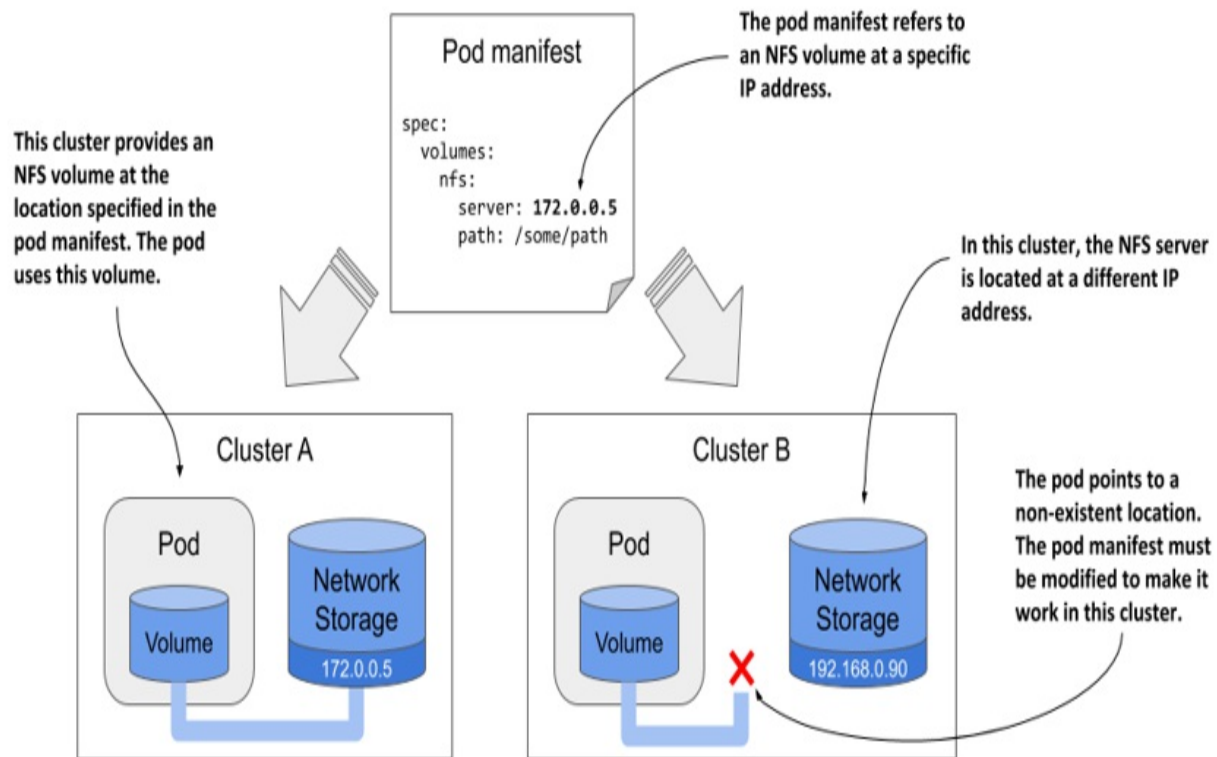
Ideally, a developer who deploys their applications on Kubernetes shouldn't need to know what storage technology the cluster provides, just as they don't need to know the characteristics of the physical servers used to run the pods. Details of the infrastructure should be handled by the people who run the cluster.

For this reason, when you deploy an application to Kubernetes, you typically don't refer directly to the external storage in the pod manifest, as you did in the previous chapter. Instead, you use an indirect approach that is explained in the following section.

One of the examples in the previous chapter shows how to use an NFS file share in a pod. The volume definition in the pod manifest contains the IP address of the NFS server and the file path exported by that server. This ties the pod definition to a specific cluster and prevents it from being used elsewhere.

As illustrated in the following figure, if you were to deploy this pod to a different cluster, you would typically need to change at least the NFS server IP. This means that the pod definition isn't portable across clusters. It must be modified each time you deploy it in a new Kubernetes cluster.

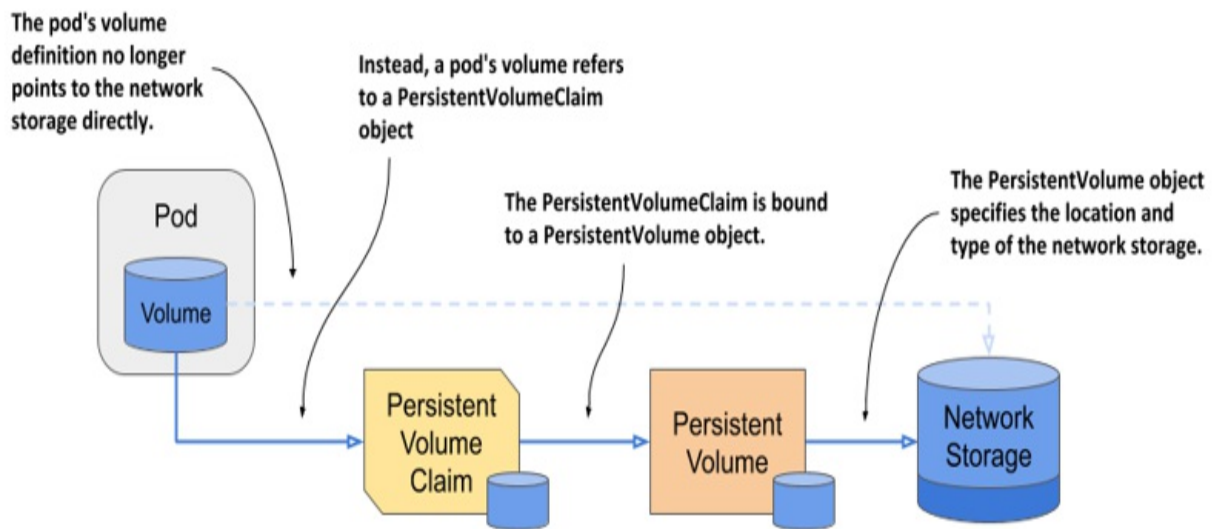
Figure 8.1 A pod manifest with infrastructure-specific volume information is not portable to other clusters



8.1.1 Introducing persistent volumes and claims

To make pod manifests portable across different cluster environments, the environment-specific information about the actual storage volume is moved to a `PersistentVolume` object, as shown in the next figure. A `PersistentVolumeClaim` object connects the pod to this `PersistentVolume` object.

Figure 8.2 Using persistent volumes and persistent volume claims to attach network storage to pods



These two objects are explained next.

Introducing persistent volumes

As the name suggests, a PersistentVolume object represents a storage volume that is used to persist application data. As shown in the previous figure, the PersistentVolume object stores the information about the underlying storage and decouples this information from the pod.

When this infrastructure-specific information isn't in the pod manifest, the same manifest can be used to deploy pods in different clusters. Of course, each cluster must now contain a PersistentVolume object with this information. I agree that this approach doesn't seem to solve anything, since we've only moved information into a different object, but you'll see later that this new approach enables things that weren't possible before.

Introducing persistent volume claims

A pod doesn't refer directly to the PersistentVolume object. Instead, it points to a PersistentVolumeClaim object, which then points to the PersistentVolume.

As its name suggests, a PersistentVolumeClaim object represents a user's

claim on the persistent volume. Because its lifecycle is not tied to that of the pod, it allows the ownership of the persistent volume to be decoupled from the pod. Before a user can use a persistent volume in their pods, they must first claim the volume by creating a `PersistentVolumeClaim` object. After claiming the volume, the user has exclusive rights to it and can use it in their pods. They can delete the pod at any time, and they won't lose ownership of the persistent volume. When the volume is no longer needed, the user releases it by deleting the `PersistentVolumeClaim` object.

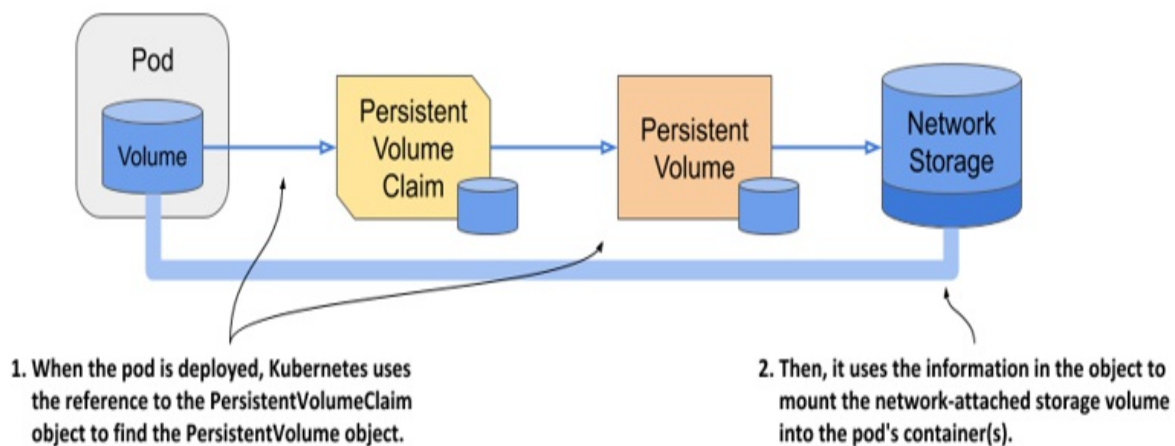
Using a persistent volume claim in a pod

To use the persistent volume in a pod, in its manifest you simply refer to the name of the persistent volume claim that the volume is bound to.

For example, if you create a persistent volume claim that gets bound to a persistent volume that represents an NFS file share, you can attach the NFS file share to your pod by adding a volume definition that points to the `PersistentVolumeClaim` object. The volume definition in the pod manifest only needs to contain the name of the persistent volume claim and no infrastructure-specific information, such as the IP address of the NFS server.

As the following figure shows, when this pod is scheduled to a worker node, Kubernetes finds the persistent volume that is bound to the claim referenced in the pod, and uses the information in the `PersistentVolume` object to mount the network storage volume in the pod's container.

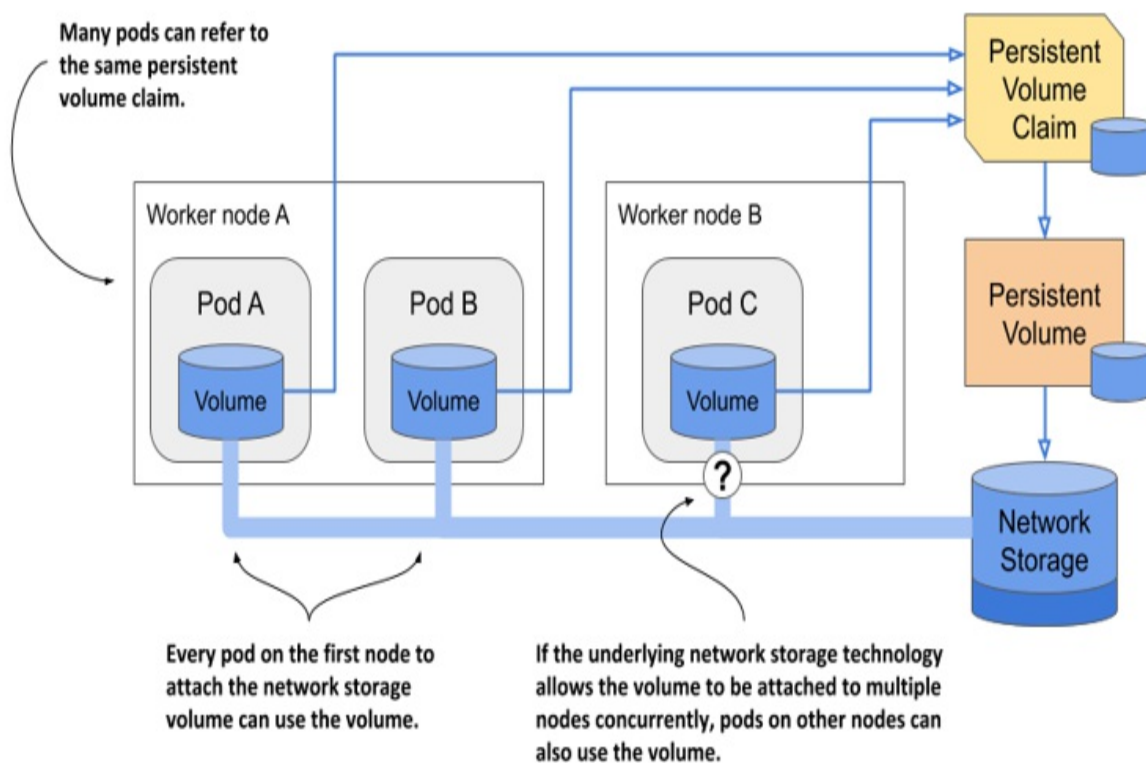
Figure 8.3 Mounting a persistent volume into the pod's container(s)



Using a claim in multiple pods

Multiple pods can use the same storage volume if they refer to the same persistent volume claim and therefore transitively to the same persistent volume, as shown in the following figure.

Figure 8.4 Using the same persistent volume claim in multiple pods



Whether these pods must all run on the same cluster node or can access the underlying storage from different nodes depends on the technology that provides that storage. If the underlying storage technology supports attaching the storage to many nodes concurrently, it can be used by pods on different nodes. If not, the pods must all be scheduled to the node that attached the storage volume first.

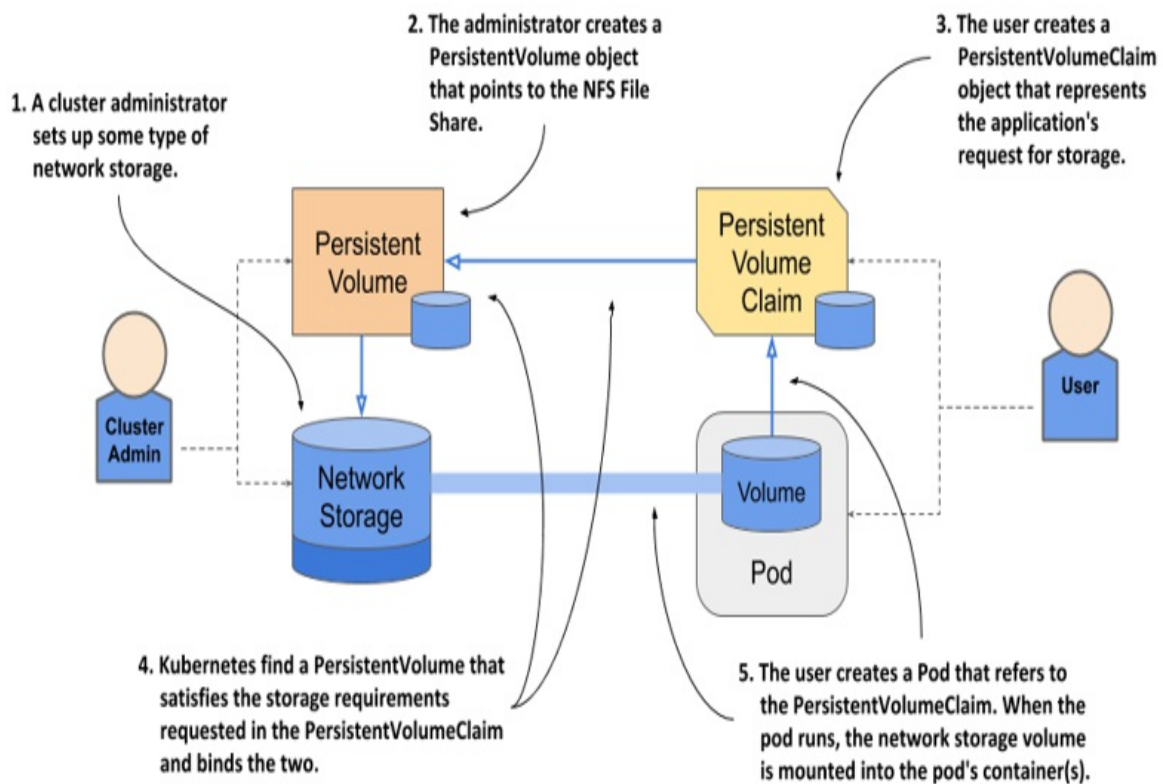
8.1.2 Understanding the benefits of using persistent volumes and claims

A system where you must use two additional objects to let a pod use a storage volume is more complex than the simple approach explained in the previous chapter, where the pod simply referred to the storage volume directly. Why is this new approach better?

The biggest advantage of using persistent volumes and claims is that the infrastructure-specific details are now decoupled from the application represented by the pod. Cluster administrators, who know the data center better than anyone else, can create the `PersistentVolume` objects with all their infrastructure-related low-level details, while software developers focus solely on describing the applications and their needs via the `Pod` and `PersistentVolumeClaim` objects.

The following figure shows how the two user roles and the objects they create fit together.

Figure 8.5 Persistent volumes are provisioned by cluster admins and consumed by pods through persistent volume claims.



Instead of the developer adding a technology-specific volume to their pod, the cluster administrator sets up the underlying storage and then registers it in Kubernetes by creating a `PersistentVolume` object through the Kubernetes API.

When a cluster user needs persistent storage in one of their pods, they first create a `PersistentVolumeClaim` object in which they either refer to a specific persistent volume by name, or specify the minimum volume size and access mode required by the application, and let Kubernetes find a persistent volume that meets these requirements. In both cases, the persistent volume is then bound to the claim and is given exclusive access. The claim can then be referenced in a volume definition within one or more pods. When the pod runs, the storage volume configured in the `PersistentVolume` object is attached to the worker node and mounted into the pod's containers.

It's important to understand that the application developer can create the manifests for the Pod and the `PersistentVolumeClaim` objects without knowing anything about the infrastructure on which the application will run.

Similarly, the cluster administrator can provision a set of storage volumes of varying sizes in advance without knowing much about the applications that will use them.

Furthermore, by using dynamic provisioning of persistent volumes, as discussed later in this chapter, administrators don't need to pre-provision volumes at all. If an automated volume provisioner is installed in the cluster, the physical storage volume and the `PersistentVolume` object are created on demand for each `PersistentVolumeClaim` object that users create.

8.2 Creating persistent volumes and claims

Now that you have a basic understanding of persistent volumes and claims and their relationship to the pods, let's revisit the quiz pod from the previous chapter. You may remember that this pod contains a `gcePersistentDisk` volume. You'll modify that pod's manifest to make it use the GCE Persistent Disk via a `PersistentVolume` object.

As explained earlier, there are usually two different types of Kubernetes users involved in the provisioning and use of persistent volumes. In the following exercises, you will first take on the role of the cluster administrator and create some persistent volumes. One of them will point to the existing GCE Persistent Disk. Then you'll take on the role of a regular user to create a persistent volume claim to get ownership of that volume and use it in the quiz pod.

8.2.1 Creating a `PersistentVolume` object

Imagine being the cluster administrator. The development team has asked you to provide two persistent volumes for their applications. One will be used to store the data files used by MongoDB in the quiz pod, and the other will be used for something else.

If you use Google Kubernetes Engine to run these examples, you'll create persistent volumes that point to GCE Persistent Disks (GCE PD). For the quiz data files, you can use the GCE PD that you provisioned in the previous chapter.

Note

If you use a different cloud provider, consult the provider's documentation to learn how to create the physical volume in their environment. If you use Minikube, kind, or any other type of cluster, you don't need to create volumes because you'll use a persistent volume that refers to a local directory on the worker node.

Creating a persistent volume with GCE Persistent Disk as the underlying storage

If you don't have the quiz-data GCE Persistent Disk set up from the previous chapter, create it again using the `gcloud compute disks create quiz-data` command. After the disk is created, you must create a manifest file for the PersistentVolume object, as shown in the following listing. You'll find the file in `Chapter08/pv.quiz-data.gcepd.yaml`.

Listing 8.1 A persistent volume manifest referring to a GCE Persistent Disk

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: quiz-data      #A
spec:
  capacity:            #B
    storage: 1Gi       #B
  accessModes:         #C
    - ReadWriteOnce    #C
    - ReadOnlyMany     #C
  gcePersistentDisk:    #D
    pdName: quiz-data  #D
    fsType: ext4       #D
```

The spec section in a PersistentVolume object specifies the storage capacity of the volume, the access modes it supports, and the underlying storage technology it uses, along with all the information required to use the underlying storage. In the case of GCE Persistent Disks, this includes the name of the PD resource in Google Compute Engine, the filesystem type, the name of the partition in the volume, and more.

Now create another GCE Persistent Disk named `other-data` and an accompanying `PersistentVolume` object. Create a new file from the manifest in listing 8.1 and make the necessary changes. You'll find the resulting manifest in the file `pv.other-data.gcepd.yaml`.

Creating persistent volumes backed by other storage technologies

If your Kubernetes cluster runs on a different cloud provider, you should be able to easily change the persistent volume manifest to use something other than a GCE Persistent Disk, as you did in the previous chapter when you directly referenced the volume within the pod manifest.

If you used Minikube or the `kind` tool to provision your cluster, you can create a persistent volume that uses a local directory on the worker node instead of network storage by using the `hostPath` field in the `PersistentVolume` manifest. The manifest for the `quiz-data` persistent volume is shown in the next listing (`pv.quiz-data.hostpath.yaml`). The manifest for the `other-data` persistent volume is in `pv.other-data.hostpath.yaml`.

Listing 8.2 A persistent volume using a local directory

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: quiz-data
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
    - ReadOnlyMany
  hostPath:      #A
    path: /var/quiz-data    #A
```

You'll notice that the two persistent volume manifests in this and the previous listing differ only in the part that specifies which underlying storage method to use. The `hostPath`-backed persistent volume stores data in the `/var/quiz-data` directory in the worker node's filesystem.

Note

To list all other supported technologies that you can use in a persistent volume, run `kubectl explain pv.spec`. You can then drill further down to see the individual configuration options for each technology. For example, for GCE Persistent Disks, run `kubectl explain pv.spec.gcePersistentDisk`.

I will not bore you with the details of how to configure the persistent volume for each available storage technology, but I do need to explain the `capacity` and `accessModes` fields that you must set in each persistent volume.

Specifying the volume capacity

The capacity of the volume indicates the size of the underlying volume. Each persistent volume must specify its capacity so that Kubernetes can determine whether a particular persistent volume can meet the requirements specified in the persistent volume claim before it can bind them.

Specifying volume access modes

Each persistent volume must specify a list of `accessModes` it supports. Depending on the underlying technology, a persistent volume may or may not be mounted by multiple worker nodes simultaneously in read/write or read-only mode. Kubernetes inspects the persistent volume's access modes to determine if it meets the requirements of the claim.

Note

The access mode determines how many *nodes*, not pods, can attach the volume at a time. Even if a volume can only be attached to a single node, it can be mounted in many pods if they all run on that single node.

Three access modes exist. They are explained in the following table along with their abbreviated form displayed by `kubectl`.

Table 8.1 Persistent volume access modes

Access Mode	Abbr.	Description
ReadWriteOnce	RWO	The volume can be mounted by a single worker node in read/write mode. While it's mounted to the node, other nodes can't mount the volume.
ReadOnlyMany	ROX	The volume can be mounted on multiple worker nodes simultaneously in read-only mode.
ReadWriteMany	RWX	The volume can be mounted in read/write mode on multiple worker nodes at the same time.

Note

The `ReadOnlyOnce` option doesn't exist. If you use a `ReadWriteOnce` volume in a pod that doesn't need to write to it, you can mount the volume in read-only mode.

Using persistent volumes as block devices

A typical application uses persistent volumes with a formatted filesystem. However, a persistent volume can also be configured so that the application can directly access the underlying block device without using a filesystem. This is configured on the `PersistentVolume` object using the `spec.volumeMode` field. The supported values for the field are explained in the next table.

Table 8.2 Configuring the volume mode for the persistent volume

Volume Mode	Description
-------------	-------------

Filesystem	When the persistent volume is mounted in a container, it is mounted to a directory in the file tree of the container. If the underlying storage is an unformatted block device, Kubernetes formats the device using the filesystem specified in the volume definition (for example, in the field <code>gcePersistentDisk.fsType</code>) before it is mounted in the container. This is the default volume mode.
Block	When a pod uses a persistent volume with this mode, the volume is made available to the application in the container as a raw block device (without a filesystem). This allows the application to read and write data without any filesystem overhead. This mode is typically used by special types of applications, such as database systems.

The manifests for the `quiz-data` and `other-data` persistent volumes do not specify a `volumeMode` field, which means that the default mode is used, namely `Filesystem`.

Creating and inspecting the persistent volume

You can now create the `PersistentVolume` objects by posting the manifests to the Kubernetes API using the now well-known command `kubectl apply`. Then use the `kubectl get` command to list the persistent volumes in your cluster:

```
$ kubectl get pv
NAME          CAPACITY   ACCESS MODES   ...   STATUS   CLAIM
other-data    10Gi       RWO,ROX        ...   Available
quiz-data     10Gi       RWO,ROX        ...   Available
```

Tip

Use `pv` as the shorthand for `PersistentVolume`.

The `STATUS` column indicates that both persistent volumes are `Available`. This is expected because they aren't yet bound to any persistent volume claim, as indicated by the empty `CLAIM` column. Also displayed are the volume capacity and access modes, which are shown in abbreviated form, as explained in table 8.1.

The underlying storage technology used by the persistent volume isn't displayed by the `kubectl get pv` command because it's less important. What is important is that each persistent volume represents a certain amount of storage space available in the cluster that applications can access with the specified modes. The technology and the other parameters configured in each persistent volume are implementation details that typically don't interest users who deploy applications. If someone needs to see these details, they can use `kubectl describe` or print the full definition of the `PersistentVolume` object as in the following command:

```
$ kubectl get pv quiz-data -o yaml
```

8.2.2 Claiming a persistent volume

Your cluster now contains two persistent volumes. Before you can use the `quiz-data` volume in the `quiz` pod, you need to claim it. This section explains how to do this.

Creating a `PersistentVolumeClaim` object

To claim a persistent volume, you create a `PersistentVolumeClaim` object in which you specify the requirements that the persistent volume must meet. These include the minimum capacity of the volume and the required access modes, which are usually dictated by the application that will use the volume. For this reason, persistent volume claims should be created by the author of the application and not by cluster administrators, so take off your administrator hat now and put on your developer hat.

Tip

As an application developer, you should never include persistent volume

definitions in your application manifests. You should include persistent volume claims because they specify the storage requirements of your application.

To create a `PersistentVolumeClaim` object, create a manifest file with the contents shown in the following listing. You'll also find the file in `pvc.quiz-data.static.yaml`.

Listing 8.3 A `PersistentVolumeClaim` object manifest

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: quiz-data      #A
spec:
  resources:
    requests:          #B
      storage: 1Gi      #B
  accessModes:         #C
    - ReadWriteOnce     #C
  storageClassName: ""   #D
  volumeName: quiz-data  #E
```

The persistent volume claim defined in the listing requests that the volume is at least 1GiB in size and can be mounted on a single node in read/write mode. The field `storageClassName` is used for dynamic provisioning of persistent volumes, which you'll learn about later in the chapter. The field must be set to an empty string if you want Kubernetes to bind a pre-provisioned persistent volume to this claim instead of provisioning a new one.

In this exercise, you want to claim the `quiz-data` persistent volume, so you must indicate this with the `volumeName` field. In your cluster, two matching persistent volumes exist. If you don't specify this field, Kubernetes could bind your claim to the other `-data` persistent volume.

If the cluster administrator creates a bunch of persistent volumes with non-descript names, and you don't care which one you get, you can skip the `volumeName` field. In that case, Kubernetes will randomly choose one of the persistent volumes whose capacity and access modes match the claim.

Note

Like persistent volumes, claims can also specify the required `volumeMode`. As you learned in section 8.2.1, this can be either `Filesystem` or `Block`. If left unspecified, it defaults to `Filesystem`. When Kubernetes checks whether a volume can satisfy the claim, the `volumeMode` of the claim and the volume is also considered.

To create the `PersistentVolumeClaim` object, apply its manifest file with `kubectl apply`. After the object is created, Kubernetes soon binds a volume to the claim. If the claim requests a specific persistent volume by name, that's the volume that is bound, if it also matches the other requirements. Your claim requires 1GiB of disk space and the `ReadWriteOnce` access mode. The persistent volume `quiz-data` that you created earlier meets both requirements and this allows it to be bound to the claim.

Listing persistent volume claims

If all goes well, your claim should now be bound to the `quiz-data` persistent volume. Use the `kubectl get` command to see if this is the case:

```
$ kubectl get pvc
NAME          STATUS    VOLUME          CAPACITY   ACCESS MODES   STORAGECLASS
quiz-data     Bound    quiz-data       10Gi       RWX,ROX        standard
```

Tip

Use `pvc` as a shorthand for `persistentvolumeclaim`.

The output of the `kubectl` command shows that the claim is now bound to your persistent volume. It also shows the capacity and access modes of this volume. Even though the claim requested only 1GiB, it has 10GiB of storage space available, because that's the capacity of the volume. Similarly, although the claim requested only the `ReadWriteOnce` access mode, it is bound to a volume that supports both the `ReadWriteOnce` (RWX) and the `ReadOnlyMany` (ROX) access modes.

If you put your cluster admin hat back on for a moment and list the persistent

volumes in your cluster, you'll see that it too is now displayed as Bound:

```
$ kubectl get pv
NAME          CAPACITY   ACCESS MODES   ...   STATUS   CLAIM
quiz-data     10Gi       RWX,RWO        ...   Bound    default/quiz
```

Any cluster admin can see which claim each persistent volume is bound to. In your case, the volume is bound to the claim `default/quiz-data`.

Note

You may wonder what the word `default` means in the claim name. This is the *namespace* in which the `PersistentVolumeClaim` object is located. Namespaces allow objects to be organized into disjoint sets. You'll learn about them in chapter 10.

By claiming the persistent volume, you and your pods now have the exclusive right to use the volume. No one else can claim it until you release it by deleting the `PersistentVolumeClaim` object.

8.2.3 Using a claim and volume in a single pod

In this section, you'll learn the ins and outs of using a persistent volume in a single pod at a time.

Using a persistent volume in pod

To use a persistent volume in a pod, you define a volume within the pod in which you refer to the `PersistentVolumeClaim` object. To try this, modify the quiz pod from the previous chapter and make it use the `quiz-data` claim. The changes to the pod manifest are highlighted in the next listing. You'll find the file in `pod.quiz.pvc.yaml`.

Listing 8.4 A pod using a `PersistentVolumeClaim` volume

```
apiVersion: v1
kind: Pod
metadata:
  name: quiz
```



```
spec:
  volumes:
  - name: quiz-data
    persistentVolumeClaim:    #A
      claimName: quiz-data    #A
  containers:
  - name: quiz-api
    image: luksa/quiz-api:0.1
    ports:
    - name: http
      containerPort: 8080
  - name: mongo
    image: mongo
    volumeMounts:    #B
    - name: quiz-data    #B
      mountPath: /data/db    #B
```

As you can see in the listing, you don't define the volume as a `gcePersistentDisk`, `awsElasticBlockStore`, `nfs` or `hostPath` volume, but as a `persistentVolumeClaim` volume. The pod will use whatever persistent volume is bound to the `quiz-data` claim. In your case, that should be the `quiz-data` persistent volume.

Create and test this pod now. Before the pod starts, the GCE PD volume is attached to the node and mounted into the pod's container(s). If you use GKE and have configured the persistent volume to use the GCE Persistent Disk from the previous chapter, which already contains data, you should be able to retrieve the quiz questions you stored earlier by running the following command:

```
$ kubectl exec -it quiz -c mongo -- mongo kiada --quiet --eval "d
{ \"_id\" : ObjectId(\"5fc3a4890bc9170520b22452\"), \"id\" : 1, \"text\"
\"answers\" : [ \"Kates\", \"Kubernetes\", \"Kooba Dooba Doo!\" ], \"corre
```

If your GCE PD has no data, add it now by running the shell script `Chapter08/insert-question.sh`.

Re-using the claim in a new pod instance

When you delete a pod that uses a persistent volume via a persistent volume claim, the underlying storage volume is detached from the worker node

(assuming that it was the only pod that was using it on that node). The persistent volume object remains bound to the claim. If you create another pod that refers to this claim, this new pod gets access to the volume and its files.

Try deleting the quiz pod and recreating it. If you run the `db.questions.find()` query in this new pod instance, you'll see that it returns the same data as the previous one. If the persistent volume uses network-attached storage such as GCE Persistent Disks, the pod sees the same data regardless of what node it's scheduled to. If you use a kind-provisioned cluster and had to resort to using a `hostPath`-based persistent volume, this isn't the case. To access the same data, you must ensure that the new pod instance is scheduled to the node to which the original instance was scheduled, as the data is stored in that node's filesystem.

Releasing a persistent volume

When you no longer plan to deploy pods that will use this claim, you can delete it. This releases the persistent volume. You might wonder if you can then recreate the claim and access the same volume and data. Let's find out. Delete the pod and the claim as follows to see what happens:

```
$ kubectl delete pod quiz
pod "quiz" deleted

$ kubectl delete pvc quiz-data
persistentvolumeclaim "quiz-data" deleted
```

Now check the status of the persistent volume:

```
$ kubectl get pv quiz-data
```

NAME	...	RECLAIM POLICY	STATUS	CLAIM
quiz-data	...	Retain	Released	default/quiz-data

The `STATUS` column shows the volume as `Released` rather than `Available`, as was the case initially. The `CLAIM` column still shows the `quiz-data` claim to which it was previously bound, even if the claim no longer exists. You'll understand why in a minute.

Binding to a released persistent volume

What happens if you create the claim again? Is the persistent volume bound to the claim so that it can be reused in a pod? Run the following commands to see if this is the case.

```
$ kubectl apply -f pvc.quiz-data.static.yaml
persistentvolumeclaim/quiz-data created
```

```
$ kubectl get pvc
NAME          STATUS    VOLUME          CAPACITY   ACCESSMODES   STORAGECLA
quiz-data     Pending
```

The claim isn't bound to the volume and its status is Pending. When you created the claim earlier, it was immediately bound to the persistent volume, so why not now?

The reason behind this is that the volume has already been used and might contain data that should be erased before another user claims the volume. This is also the reason why the status of the volume is Released instead of Available and why the claim name is still shown on the persistent volume, as this helps the cluster administrator to know if the data can be safely deleted.

Making a released persistent volume available for re-use

To make the volume available again, you must delete and recreate the PersistentVolume object. But will this cause the data stored in the volume to be lost?

Imagine if you had accidentally deleted the pod and the claim and caused a loss of service to the Kiada application. You need to restore the service as soon as possible, with all data intact. If you think that deleting the PersistentVolume object would delete the data, that sounds like the last thing you should do but is actually completely safe.

With a pre-provisioned persistent volume like the one at hand, deleting the object is equivalent to deleting a data pointer. The PersistentVolume object merely *points* to a GCE Persistent Disk. It doesn't store the data. If you delete

and recreate the object, you end up with a new pointer to the same GCE PD and thus the same data. You'll confirm this is the case in the next exercise.

```
$ kubectl delete pv quiz-data
persistentvolume "quiz-data" deleted
```

```
$ kubectl apply -f pv.quiz-data.gcepd.yaml
persistentvolume/quiz-data created
```

```
$ kubectl get pv quiz-data
```

NAME	...	RECLAIM POLICY	STATUS	CLAIM	...
quiz-data	...	Retain	Available		...

Note

An alternative way of making a persistent volume available again is to edit the PersistentVolume object and remove the `claimRef` from the spec section.

The persistent volume is displayed as `Available` again. Let me remind you that you created a claim for the volume earlier. Kubernetes has been waiting for a volume to bind to the claim. As you might expect, the volume you've just created will be bound to this claim in a few seconds. List the volumes again to confirm:

```
$ kubectl get pv quiz-data
```

NAME	...	RECLAIM POLICY	STATUS	CLAIM
quiz-data	...	Retain	Bound	default/quiz-data

The output shows that the persistent volume is again bound to the claim. If you now deploy the quiz pod and query the database again with the following command, you'll see that the data in underlying GCE Persistent Disk has not been lost:

```
$ kubectl exec -it quiz -c mongo -- mongo kiada --quiet --eval "d
{ \"_id\" : ObjectId(\"5fc3a4890bc9170520b22452\"), \"id\" : 1, \"text\"
\"answers\" : [ \"Kates\", \"Kubernetes\", \"Kooba Dooba Doo!\" ], \"corre
```

Configuring the reclaim policy on persistent volumes

What happens to a persistent volume when it is released is determined by the volume's reclaim policy. When you used the `kubectl get pv` command to

list persistent volumes, you may have noticed that the `quiz-data` volume's policy is `Retain`. This policy is configured using the field `.spec.persistentVolumeReclaimPolicy` in the `PersistentVolume` object.

The field can have one of the three values explained in the following table.

Table 8.3 Persistent volume reclaim policies

Reclaim policy	Description
Retain	When the persistent volume is released (this happens when you delete the claim that's bound to it), Kubernetes <i>retains</i> the volume. The cluster administrator must manually reclaim the volume. This is the default policy for manually created persistent volumes.
Delete	The <code>PersistentVolume</code> object and the underlying storage are automatically deleted upon release. This is the default policy for dynamically provisioned persistent volumes, which are discussed in the next section.
Recycle	This option is deprecated and shouldn't be used as it may not be supported by the underlying volume plugin. This policy typically causes all files on the volume to be deleted and makes the persistent volume available again without the need to delete and recreate it.

Tip

You can change the reclaim policy of an existing `PersistentVolume` at any time. If it's initially set to `Delete`, but you don't want to lose your data when deleting the claim, change the volume's policy to `Retain` before doing so.

Warning

If a persistent volume is Released and you subsequently change its reclaim policy from Retain to Delete, the PersistentVolume object and the underlying storage will be deleted immediately. However, if you instead delete the object manually, the underlying storage remains intact.

Deleting a persistent volume while it's bound

You're done playing with the quiz pod, the quiz-data persistent volume claim, and the quiz-data persistent volume, so you'll now delete them. You'll learn one more thing in the process.

Have you wondered what happens if a cluster administrator deletes a persistent volume while it's in use (while it's bound to a claim)? Let's find out. Delete the persistent volume like so:

```
$ kubectl delete pv quiz-data
persistentvolume "quiz-data" deleted    #A
```

This command tells the Kubernetes API to delete the PersistentVolume object and then waits for Kubernetes controllers to complete the process. But this can't happen until you release the persistent volume from the claim by deleting the PersistentVolumeClaim object.

You can cancel the wait by pressing Control-C. However, this doesn't cancel the deletion, as its already underway. You can confirm this as follows:

```
$ kubectl get pv quiz-data
```

NAME	CAPACITY	ACCESS MODES	STATUS	CLAIM
quiz-data	10Gi	RWO, ROX	Terminating	default/quiz-

As you can see, the persistent volume's status shows that it's being terminated. But it's still bound to the persistent volume claim. You need to delete the claim for the volume deletion to complete.

Deleting a persistent volume claim while a pod is using it

The claim is still being used by the quiz pod, but let's try deleting it anyway:

```
$ kubectl delete pvc quiz-data
persistentvolumeclaim "quiz-data" deleted    #A
```

Like the `kubectl delete pv` command, this command also doesn't complete immediately. As before, the command waits for the claim deletion to complete. You can interrupt the execution of the command, but this won't cancel the deletion, as you can see with the following command:

```
$ kubectl get pvc quiz-data
NAME          STATUS          VOLUME          CAPACITY   ACCESS MODES   S
quiz-data     Terminating    quiz-data        10Gi       RWX,ROX
```

The deletion of the claim is blocked by the pod. Unsurprisingly, deleting a persistent volume or a persistent volume claim has no immediate effect on the pod that's using it. The application running in the pod continues to run unaffected. Kubernetes never kills pods just because the cluster administrator wants their disk space back.

To allow the termination of the persistent volume claim and the persistent volume to complete, delete the quiz pod with `kubectl delete po quiz`.

Deleting the underlying storage

As you learned in the previous section, deleting the persistent volume does not delete the underlying storage, such as the `quiz-data` GCE Persistent Disk if you use Google Kubernetes Engine to perform these exercises, or the `/var/quiz-data` directory on the worker node if you use Minikube or kind.

You no longer need the data files and can safely delete them. If you use Minikube or kind, you don't need to delete the data directory, as it doesn't cost you anything. However, a GCE Persistent Disk does. You can delete it with the following command:

```
$ gcloud compute disks delete quiz-data
```

You might remember that you also created another GCE Persistent Disk called `other-data`. Don't delete that one just yet. You'll use it in the next

section's exercise.

8.2.4 Using a claim and volume in multiple pods

So far, you used a persistent volume in only one pod instance at a time. You used the persistent volume in the so-called `ReadWriteOnce` (`RWO`) access mode because it was attached to a single node and allowed both read and write operations. You may remember that two other modes exist, namely `ReadWriteMany` (`RWX`) and `ReadOnlyMany` (`ROX`). The volume's access modes indicate whether it can concurrently be attached to one or many cluster nodes and whether it can only be read from or also written to.

The `ReadWriteOnce` mode doesn't mean that only a single pod can use it, but that a single *node* can attach the volume. As this is something that confuses a lot of users, it warrants a closer look.

Binding a claim to a randomly selected persistent volume

This exercise requires the use of a GKE cluster. Make sure it has at least two nodes. First, create a persistent volume claim for the other-data persistent volume that you created earlier. You'll find the manifest in the file `pvc.other-data.yaml`. It's shown in the following listing.

Listing 8.5 A persistent volume claim requesting both `ReadWriteOnce` and `ReadOnlyMany` access

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: other-data
spec:
  resources:
    requests:
      storage: 1Gi
  accessModes: #A
    - ReadWriteOnce #A
    - ReadOnlyMany #A
  storageClassName: "" #B
```

You'll notice that unlike in the previous section, this persistent volume claim

does not specify the `volumeName`. This means that the persistent volume for this claim will be selected at random among all the volumes that can provide at least 1Gi of space and support both the `ReadWriteOnce` and the `ReadOnlyMany` access modes.

Your cluster should currently contain only the `other-data` persistent volume. Because it matches the requirements in the claim, this is the volume that will be bound to it.

Using a `ReadWriteOnce` volume in multiple pods

The persistent volume bound to the claim supports both `ReadWriteOnce` and `ReadOnlyMany` access modes. First, you'll use it in `ReadWriteOnce` mode, as you'll deploy pods that write to it.

You'll create several replicas of a data-writer pod from a single pod manifest. The manifest is shown in the following listing. You'll find it in `pod.data-writer.yaml`.

Listing 8.6 A pod that writes a file to a shared persistent volume

```
apiVersion: v1
kind: Pod
metadata:
  generateName: data-writer-      #A
spec:
  volumes:
  - name: other-data
    persistentVolumeClaim:      #B
      claimName: other-data      #B
  containers:
  - name: writer
    image: busybox
    command:
    - sh
    - -c
    - |
      echo "A writer pod wrote this." > /other-data/${HOSTNAME} &
      echo "I can write to /other-data/${HOSTNAME}." ;      #C
      sleep 9999      #C
  volumeMounts:
  - name: other-data
```

```
    mountPath: /other-data
resources:    #D
  requests:   #D
    cpu: 1m   #D
```

Use the following command to create the pod from this manifest:

```
$ kubectl create -f pod.data-writer.yaml    #A
pod/data-writer-6mbjg created              #B
```

Notice that you aren't using the `kubectl apply` this time. Because the pod manifest uses the `generateName` field instead of specifying the pod name, `kubectl apply` won't work. You must use `kubectl create`, which is similar, but is only used to create and not update objects.

Repeat the command several times so that you create two to three times as many writer pods as there are cluster nodes to ensure that at least two pods are scheduled to each node. Confirm that this is the case by listing the pods with the `-o wide` option and inspecting the `NODE` column:

```
$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE
data-writer-6mbjg	1/1	Running	0	5m
data-writer-97t9j	0/1	ContainerCreating	0	5m
data-writer-d9f2f	1/1	Running	0	5m
data-writer-dfd8h	0/1	ContainerCreating	0	5m
data-writer-f867j	1/1	Running	0	5m

Note

I've shortened the node names for clarity.

If all your pods are located on the same node, create a few more. Then look at the `STATUS` of these pods. You'll notice that all the pods scheduled to the first node run fine, whereas the pods on the other node are all stuck in the status `ContainerCreating`. Even waiting for several minutes doesn't change anything. Those pods will never run.

If you use `kubectl describe` to display the events related to one of these pods, you'll see that it doesn't run because the persistent volume can't be attached to the node that the pod is on:

```
$ kubectl describe po data-writer-97t9j
...
Warning FailedAttachVolume ... attachdetach-controller At
for volume "other-data" : googleapi: Error 400: RESOURCE_IN_USE_B
The disk resource 'projects/.../disks/other-data' is already bein
'projects/.../instances/gkdp-r6j4'      #A
```

The reason the volume can't be attached is because it's already attached to the first node in read-write mode. The volume supports `ReadWriteOnce` and `ReadOnlyMany` but doesn't support `ReadWriteMany`. This means that only a single node can attach the volume in read-write mode. When the second node tries to do the same, the operation fails.

All the pods on the first node run fine. Check their logs to confirm that they were all able to write a file to the volume. Here's the log of one of them:

```
$ kubectl logs other-data-writer-6mbjg
I can write to /other-data/other-data-writer-6mbjg.
```

You'll find that all the pods on the first node successfully wrote their files to the volume. You don't need `ReadWriteMany` for multiple pods to write to the volume if they are on the same node. As explained before, the word "Once" in `ReadWriteOnce` refers to nodes, not pods.

Using a combination of read-write and read-only pods with a `ReadWriteOnce` and `ReadOnlyMany` volume

You'll now deploy a group of reader pods alongside the data-writer pods. They will use the persistent volume in read-only mode. The following listing shows the pod manifest for these data-reader pods. You'll find it in `pod.data-reader.yaml`.

Listing 8.7 A pod that mounts a shared persistent volume in read-only mode

```
apiVersion: v1
kind: Pod
metadata:
  generateName: data-reader-
spec:
  volumes:
  - name: other-data
```

```

    persistentVolumeClaim:
      claimName: other-data      #A
      readOnly: true           #B
  containers:
  - name: reader
    image: busybox
    imagePullPolicy: Always
    command:
    - sh
    - -c
    - |
      echo "The files in the persistent volume and their contents
      grep ^ /other-data/* ;      #C
      sleep 9999      #C
  volumeMounts:
  - name: other-data
    mountPath: /other-data
  ...

```

Use the `kubectl create` command to create as many of these reader pods as necessary to ensure that each node runs at least two instances. Use the `kubectl get po -o wide` command to see how many pods are on each node.

As before, you'll notice that only those reader pods that are scheduled to the first node are running. The pods on the second node are stuck in `ContainerCreating`, just like the writer pods. Here's a list of just the reader pods (the writer pods are still there, but aren't shown):

```

$ kubectl get pods -o wide | grep reader
NAME                READY    STATUS                RESTARTS   AGE
data-reader-6594s   1/1     Running               0          2m
data-reader-lqwkv   1/1     Running               0          2m
data-reader-mr5mk   0/1     ContainerCreating    0          2m
data-reader-npk24   1/1     Running               0          2m
data-reader-qbpt5   0/1     ContainerCreating    0          2m

```

These pods use the volume in read-only mode. The claim's (and volume's) access modes are both `ReadWriteOnce (RWO)` and `ReadOnlyMany (ROX)`, as you can see by running `kubectl get pvc`:

```

$ kubectl get pvc other-data
NAME          STATUS    VOLUME          CAPACITY   ACCESS MODES   STORAGECLASS
other-data    Bound    other-data       10Gi       RWO, ROX       standard

```

If the claim supports access mode `ReadOnlyMany`, why can't both nodes attach the volume and run the reader pods? This is caused by the writer pods. The first node attached the persistent volume in read-write mode. This prevents other nodes from attaching the volume, even in read-only mode.

Wonder what happens if you delete all the writer pods? Does that allow the second node to attach the volume in read-only mode and run its pods? Delete the writer pods one by one or use the following command to delete them all if you use a shell that supports the following syntax:

```
$ kubectl delete $(kubectl get po -o name | grep writer)
```

Now list the pods again. The status of the reader pods that are on the second node is still ContainerCreating. Even if you give it enough time, the pods on that node never run. Can you figure out why that is so?

It's because the volume is still being used by the reader pods on the first node. The volume is attached in read-write mode because that was the mode requested by the writer pods, which you deployed first. Kubernetes can't detach the volume or change the mode in which it is attached while it's being used by pods.

In the next section, you'll see what happens if you deploy reader pods without first deploying the writers. Before moving on, delete all the pods as follows:

```
$ kubectl delete po --all
```

Give Kubernetes some time to detach the volume from the node. Then go to the next exercise.

Using a ReadOnlyMany volume in multiple pods

Create several reader pods again by repeating the `kubectl create -f pod.data-reader.yaml` command several times. This time, all the pods run, even if they are on different nodes:

```
$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP
------	-------	--------	----------	-----	----

data-reader-9xs5q	1/1	Running	0	27s	10.0.10.34
data-reader-b9b25	1/1	Running	0	29s	10.0.10.32
data-reader-cbnp2	1/1	Running	0	16s	10.0.9.12
data-reader-fjx6t	1/1	Running	0	21s	10.0.9.11

All these pods specify the `readOnly: true` field in the `persistentVolumeClaim` volume definition. This causes the node that runs the first pod to attach the persistent volume in read-only mode. The same thing happens on the second node. They can both attach the volume because they both attach it in read-only mode and the persistent volume supports `ReadOnlyMany`.

The `ReadOnlyMany` access mode doesn't need further explanation. If no pod mounts the volume in read-write mode, any number of pods can use the volume, even on many different nodes.

Can you guess what happens if you deploy a writer pod now? Can it write to the volume? Create the pod and check its status. This is what you'll see:

```
$ kubectl get po -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP
...					
data-writer-dj6w5	1/1	Running	0	3m33s	10.0.10.

This pod is shown as `Running`. Does that surprise you? It did surprise me. I thought it would be stuck in `ContainerCreating` because the node couldn't mount the volume in read-write mode because it's already mounted in read-only mode. Does that mean that the node was able to upgrade the mount point from read-only to read-write without detaching the volume?

Let's check the pod's log to confirm that it could write to the volume:

```
$ kubectl logs data-writer-dj6w5
sh: can't create /other-data/data-writer-dj6w5: Read-only file sy
```

Ahh, there's your answer. The pod is unable to write to the volume because it's read-only. The pod was started even though the volume isn't mounted in read-write mode as the pod requests. This might be a bug. If you try this yourself and the pod doesn't run, you'll know that the bug was fixed after the book was published.

You can now delete all the pods, the persistent volume claim and the underlying GCE Persistent Disk, as you're done using them.

Using a ReadWriteMany volume in multiple pods

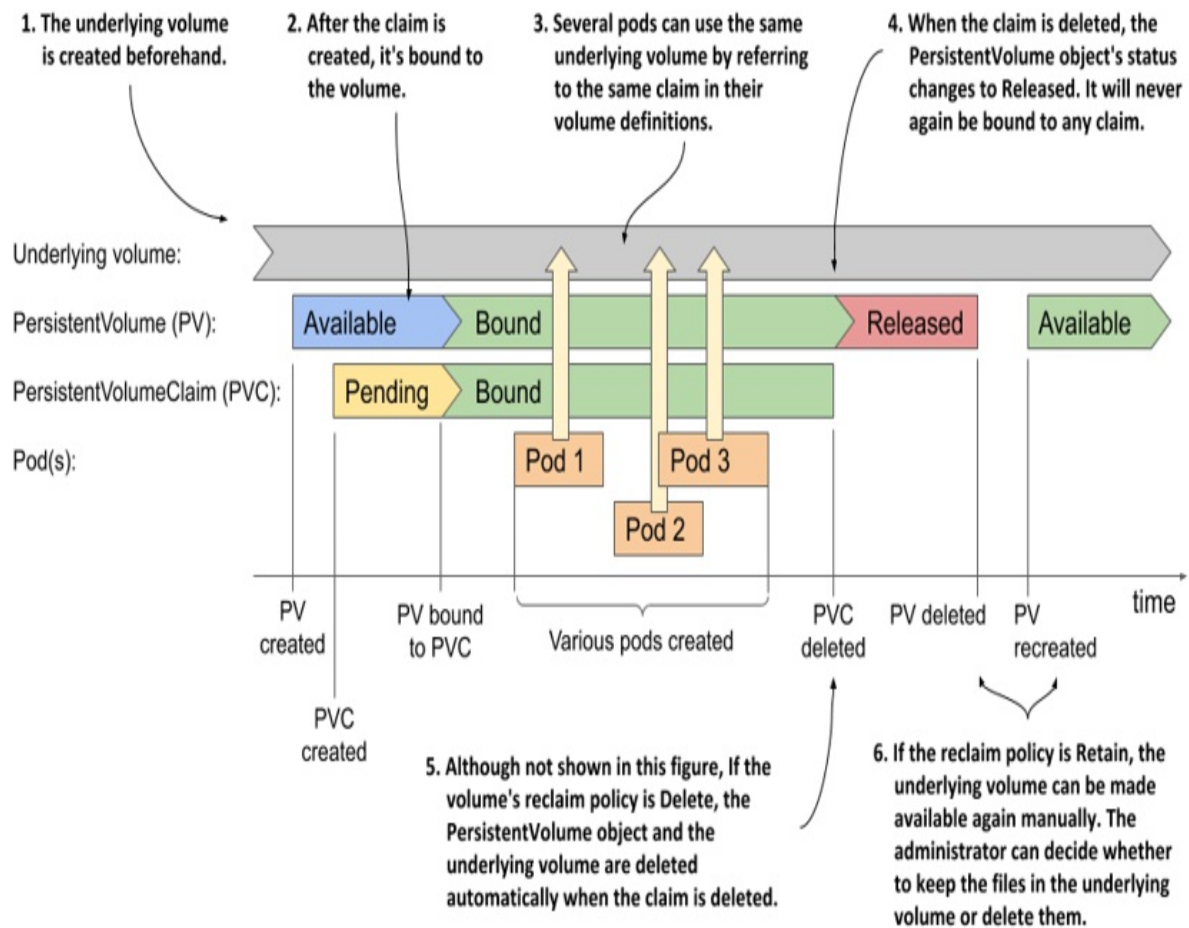
GCE Persistent Disks don't support the ReadWriteMany access mode. However, network-attached volumes available in other cloud environments do support it. As the name of the ReadWriteMany access mode indicates, volumes that support this mode can be attached to many cluster nodes concurrently, yet still allow both read and write operations to be performed on the volume.

As this mode has no restrictions on the number of nodes or pods that can use the persistent volume in either read-write or read-only mode, it doesn't need any further explanation. If you'd like to play with them anyhow, I suggest you deploy the writer and the reader pods as in the previous exercise, but this time use the ReadWriteMany access mode in both the persistent volume and the persistent volume claim definitions.

8.2.5 Understanding the lifecycle of manually provisioned persistent volumes

You used the same GCE Persistent Disk throughout several exercises in this chapter, but you created multiple volumes, claims, and pods that used the same GCE PD. To understand the lifecycles of these four objects, take a look at the following figure.

Figure 8.6 The lifecycle of statically provisioned persistent volumes, claims and the pods that use them



When using manually provisioned persistent volumes, the lifecycle of the underlying storage volume is not coupled to the lifecycle of the PersistentVolume object. Each time you create the object, its initial status is `Available`. When a PersistentVolumeClaim object appears, the persistent volume is bound to it, if it meets the requirements set forth in the claim. Until the claim is bound to the volume, it has the status `Pending`; then both the volume and the claim are displayed as `Bound`.

At this point, one or many pods may use the volume by referring to the claim. When each pod runs, the underlying volume is mounted in the pod's containers. After all the pods are finished with the claim, the PersistentVolumeClaim object can be deleted.

When the claim is deleted, the volume's reclaim policy determines what happens to the PersistentVolume object and the underlying volume. If the

policy is `Delete`, both the object and the underlying volume are deleted. If it's `Retain`, the `PersistentVolume` object and the underlying volume are preserved. The object's status changes to `Released` and the object can't be bound until additional steps are taken to make it `Available` again.

If you delete the `PersistentVolume` object manually, the underlying volume and its files remain intact. They can be accessed again by creating a new `PersistentVolume` object that references the same underlying volume.

Note

The sequence of events described in this section applies to the use of statically provisioned volumes that exist before the claims are created. When persistent volumes are dynamically provisioned, as described in the next section, the situation is different. Look for a similar diagram at the end of the next section.

8.3 Dynamic provisioning of persistent volumes

So far in this chapter you've seen how developers can claim pre-provisioned persistent volumes as a place for their pods to store data persistently without having to deal with the details of the underlying storage technology.

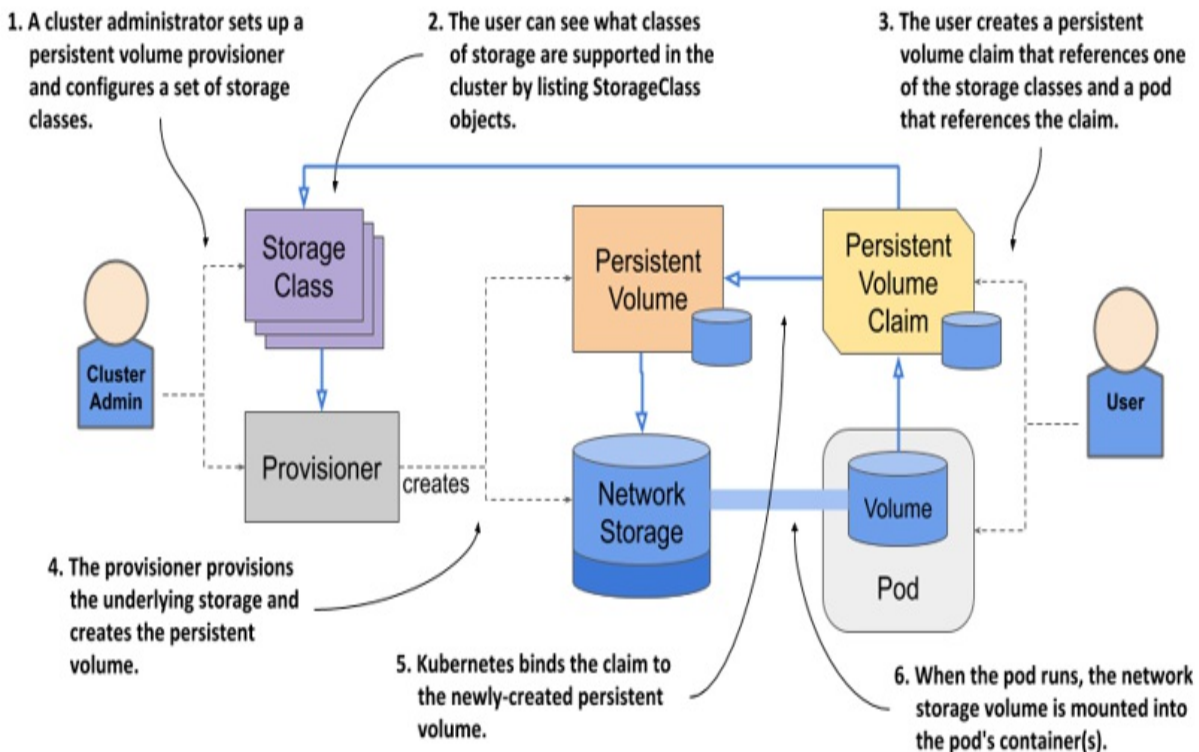
However, a cluster administrator must pre-provision the physical volumes and create a `PersistentVolume` object for each of these volumes. Then each time the volume is bound and released, the administrator must manually delete the data on the volume and recreate the object.

To keep the cluster running smoothly, the administrator may need to pre-provision dozens, if not hundreds, of persistent volumes, and constantly keep track of the number of available volumes to ensure the cluster never runs out. All this manual work contradicts the basic idea of Kubernetes, which is to automate the management of large clusters. As one might expect, a better way to manage volumes exists. It's called *dynamic provisioning of persistent volumes*.

With dynamic provisioning, instead of provisioning persistent volumes in advance (and manually), the cluster admin deploys a persistent volume

provisioner to automate the just-in-time provisioning process, as shown in the following figure.

Figure 8.7 Dynamic provisioning of persistent volumes



In contrast to static provisioning, the order in which the claim and the volume arise is reversed. When a user creates a persistent volume claim, the dynamic provisioner provisions the underlying storage and creates the `PersistentVolume` object for that particular claim. The two objects are then bound.

If your Kubernetes cluster is managed by a cloud provider, it probably already has a persistent volume provisioner configured. If you are running Kubernetes on-premises, you'll need to deploy a custom provisioner, but this is outside the scope of this chapter. Clusters that are provisioned with Minikube or kind usually also come with a provisioner out of the box.

8.3.1 Introducing the `StorageClass` object

The persistent volume claim definition you created in the previous section specifies the minimum size and the required access modes of the volume, but it also contains a field named `storageClassName`, which wasn't discussed yet.

A Kubernetes cluster can run multiple persistent volume provisioners, and a single provisioner may support several different types of storage volumes. When creating a claim, you use the `storageClassName` field to specify which storage class you want.

Listing storage classes

The storage classes available in the cluster are represented by *StorageClass* API objects. You can list them with the `kubectl get sc` command. In a GKE cluster, this is the result:

```
$ kubectl get sc
NAME                                PROVISIONER                AGE
standard (default)                 kubernetes.io/gce-pd       1d    #A
```

Note

The shorthand for `storageclass` is `sc`.

In a kind-provisioned cluster, the result is similar:

```
$ kubectl get sc
NAME                                PROVISIONER                RECLAIMPOLICY    ...
standard (default)                 rancher.io/local-path      Delete           ...
```

Clusters created with Minikube also provide a storage class with the same name:

```
$ kubectl get sc
NAME                                PROVISIONER                RECLAIMPOLICY    V
standard (default)                 k8s.io/minikube-hostpath  Delete           I
```

In many clusters, as in these three examples, only one storage class called `standard` is configured. It's also marked as the default, which means that this is the class that is used to provision the persistent volume when the persistent

volume claim doesn't specify the storage class.

Note

Remember that omitting the `storageClassName` field causes the default storage class to be used, whereas explicitly setting the field to "" disables dynamic provisioning and causes an existing persistent volume to be selected and bound to the claim.

Inspecting the default storage class

Let's get to know the `StorageClass` object kind by inspecting the YAML definition of the standard storage class with the `kubectl get` command. In GKE, you'll find the following definition:

```
$ kubectl get sc standard -o yaml      #A
allowVolumeExpansion: true
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"      #B
  name: standard
  ...
parameters:          #C
  type: pd-standard   #C
provisioner: kubernetes.io/gce-pd      #D
reclaimPolicy: Delete      #E
volumeBindingMode: Immediate      #F
```

The storage class definition in a kind-provisioned cluster is not much different. The main differences are highlighted in bold:

```
$ kubectl get sc standard -o yaml      #A
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"      #B
  name: standard
  ...
provisioner: rancher.io/local-path      #C
```

```
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer #D
```

In clusters created with Minikube, the standard storage class looks as follows:

```
$ kubectl get sc standard -o yaml #A
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  annotations:
    storageclass.kubernetes.io/is-default-class: "true" #B
  name: standard #A
  ...
provisioner: k8s.io/minikube-hostpath #C
reclaimPolicy: Delete #D
volumeBindingMode: Immediate #E
```

Note

You'll notice that StorageClass objects have no spec or status sections. This is because the object only contains static information. Since the object's fields aren't organized in the two sections, the YAML manifest may be more difficult to read. This is also compounded by the fact that fields in YAML are typically sorted in alphabetical order, which means that some fields may appear above the apiVersion, kind or metadata fields. Be careful not to overlook these.

If you look closely at the top of the storage class definitions, you'll see that they all include an annotation that marks the storage class as default.

Note

You'll learn what an object annotation is in chapter 10.

As specified in GKE's storage class definition, when you create a persistent volume claim that references the standard class in GKE, the provisioner `kubernetes.io/gce-pd` is called to provision the persistent volume. In kind-provisioned clusters, the provisioner is `rancher.io/local-path`, whereas in Minikube it's `k8s.io/minikube-hostpath`. GKE's default storage class also specifies a parameter that is provided to the provisioner.

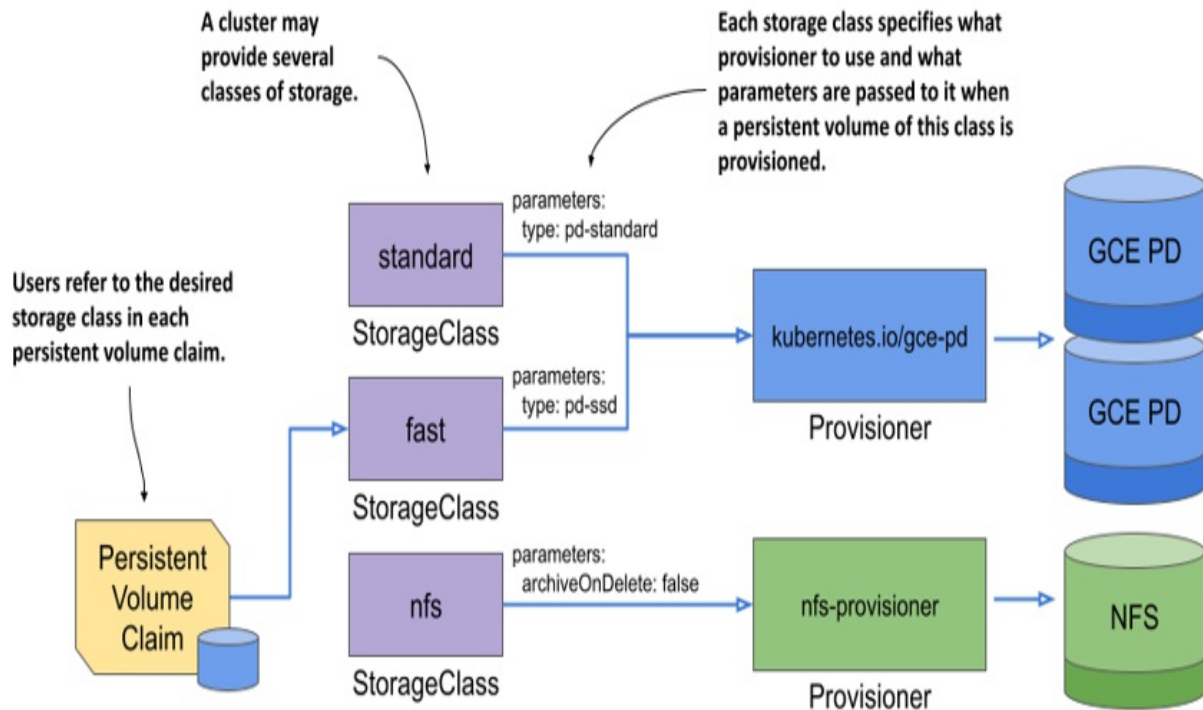
Regardless of what provisioner is used, the volume's reclaim policy is set to whatever is specified in the storage class, which in all of the previous examples is `Delete`. As you have already learned, this means that the volume is deleted when you release it by deleting the claim.

The last field in the storage class definition is `volumeBindingMode`. Both GKE and Minikube use the volume binding mode `Immediate`, whereas `kind` uses `WaitForFirstConsumer`. You'll learn what the difference is later in this chapter.

`StorageClass` objects also support several other fields that are not shown in the above listing. You can use `kubectl explain` to see what they are. You'll learn about some of them in the following sections.

In summary, a `StorageClass` object represents a class of storage that can be dynamically provisioned. As shown in the following figure, each storage class specifies what provisioner to use and the parameters that should be passed to it when provisioning the volume. The user decides which storage class to use for each of their persistent volume claims.

Figure 8.8 The relationship between storage classes, persistent volume claims and dynamic volume provisioners



8.3.2 Dynamic provisioning using the default storage class

You've previously used a statically provisioned persistent volume for the quiz pod. Now you'll use dynamic provisioning to achieve the same result, but with much less manual work. And most importantly, you can use the same pod manifest, regardless of whether you use GKE, Minikube, kind, or any other tool to run your cluster, assuming that a default storage class exists in the cluster.

Creating a claim with dynamic provisioning

To dynamically provision a persistent volume using the storage class from the previous section, you can create a `PersistentVolumeClaim` object with the `storageClassName` field set to `standard` or with the field omitted altogether.

Let's use the latter approach, as this makes the manifest as minimal as possible. You can find the manifest in the `pvc.quiz-data-default.yaml` file. Its contents are shown in the following listing.

Listing 8.8 A minimal PVC definition that uses the default storage class

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: quiz-data-default
spec:
  #A
  resources:
    #B
    requests:
      #B
      storage: 1Gi      #B
  accessModes:
    #C
    - ReadWriteOnce    #C
```

This PersistentVolumeClaim manifest contains only the storage size request and the desired access mode, but no storageClassName field, so the default storage class is used.

After you create the claim with `kubectl apply`, you can see which storage class it's using by inspecting the claim with `kubectl get`. This is what you'll see if you use GKE:

```
$ kubectl get pvc quiz-data-default
NAME                STATUS    VOLUME                CAPACITY   ACCESS
quiz-data-default   Bound    pvc-ab623265-...     1Gi        RWO
```

As expected, and as indicated in the STORAGECLASS column, the claim you just created uses the standard storage class.

In GKE and Minikube, the persistent volume is created immediately and bound to the claim. However, if you create the same claim in a kind-provisioned cluster, that's not the case:

```
$ kubectl get pvc quiz-data-default
NAME                STATUS    VOLUME    CAPACITY   ACCESS MODES
quiz-data-default   Pending
```

In a kind-provisioned cluster, and possibly other clusters, too, the persistent volume claim you just created is not bound immediately and its status is Pending.

In one of the previous sections, you learned that this happens when no persistent volume matches the claim, either because it doesn't exist or

because it's not available for binding. However, you are now using dynamic provisioning, where the volume should be created after you create the claim, and specifically for this claim. Is your claim pending because the cluster needs more time to provision the volume?

No, the reason for the pending status lies elsewhere. Your claim will remain in the Pending state until you create a pod that uses this claim. I'll explain why later. For now, let's just create the pod.

Using the persistent volume claim in a pod

Create a new pod manifest file from the `pod.quiz.pvc.yaml` file that you created earlier. Change the name of the pod to `quiz-default` and the value of the `claimName` field to `quiz-data-default`. You can find the resulting manifest in the file `pod.quiz-default.yaml`. Use it to create the pod.

If you use a kind-provisioned cluster, the status of the persistent volume claim should change to Bound within moments of creating the pod:

```
$ kubectl get pvc quiz-data-default
NAME                STATUS    VOLUME                CAPACITY    ACCESS
quiz-data-default   Bound    pvc-c71fb2c2-...     1Gi         RWO
```

This implies that the persistent volume has been created. List persistent volumes to confirm (the following output has been reformatted to make it easier to read):

```
$ kubectl get pv
NAME                CAPACITY    ACCESS MODES    RECLAIM POLICY    STAT
pvc-c71fb2c2...    1Gi         RWO              Delete             Boun

...    STATUS    CLAIM                                STORAGECLASS    REASON
...    Bound     default/quiz-data-default           standard
```

As you can see, because the volume was created on demand, its properties perfectly match the requirements specified in the claim and the storage class it references. The volume capacity is 1Gi and the access mode is RWO.

Understanding when a dynamically provisioned volume is actually

provisioned

Why is the volume in a kind-provisioned cluster created and bound to the claim only after you deploy the pod? In an earlier example that used a manually pre-provisioned persistent volume, the volume was bound to the claim as soon as you created the claim. Is this a difference between static and dynamic provisioning? Because in both GKE and Minikube, the volume was dynamically provisioned and bound to the claim immediately, it's clear that dynamic provisioning alone is not responsible for this behavior.

The system behaves this way because of how the storage class in a kind-provisioned cluster is configured. You may remember that this storage class was the only one that has `volumeBindingMode` set to `WaitForFirstConsumer`. This causes the system to wait until the first pod, or the *consumer* of the claim, exists before the claim is bound. The persistent volume is also not provisioned before that.

Some types of volumes require this type of behavior, because the system needs to know *where* the pod is scheduled *before* it can provision the volume. This is the case with provisioners that create node-local volumes, such as the one you find in clusters created with the kind tool. You may remember that the provisioner referenced in the storage class had the word “local” in its name (`rancher.io/local-path`). Minikube also provisions a local volume (the provisioner it uses is called `k8s.io/minikube-hostpath`), but because there's only one node in the cluster, there's no need to wait for the pod to be created in order to know which node the persistent volume needs to be created on.

Note

Refer to the documentation of your chosen provisioner to determine whether it requires the volume binding mode to be set to `WaitForFirstConsumer`.

The alternative to `WaitForFirstConsumer` is the `Immediate` volume binding mode. The two modes are explained in the following table.

Table 8.4 Supported volume binding modes

Volume binding mode	Description
Immediate	The provision and binding of the persistent volume takes place immediately after the claim is created. Because the consumer of the claim is unknown at this point, this mode is only applicable to volumes that are can be accessed from any cluster node.
WaitForFirstConsumer	The volume is provisioned and bound to the claim when the first pod that uses this claim is created. This mode is used for topology-constrained volume types.

8.3.3 Creating a storage class and provisioning volumes of that class

As you saw in the previous sections, most Kubernetes clusters contain a single storage class named `standard`, but use different provisioners. A full-blown cluster such as the one you find in GKE can surely provide more than just a single type of persistent volume. So how does one create other types of volumes?

Inspecting the default storage class in GKE

Let's look at the default storage class in GKE more closely. I've rearranged the fields since the original alphabetical ordering makes the YAML definition more difficult to understand. The storage class definition follows:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: standard
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
```

```

...
provisioner: kubernetes.io/gce-pd      #A
parameters:      #B
  type: pd-standard      #B
volumeBindingMode: Immediate
allowVolumeExpansion: true
reclaimPolicy: Delete

```

If you create a persistent volume claim that references this storage class, the provisioner `kubernetes.io/gce-pd` is called to create the volume. In this call, the provisioner receives the parameters defined in the storage class. In the case of the default storage class in GKE, the parameter `type: pd-standard` is passed to the provisioner. This tells the provisioner what type of GCE Persistent Disk to create.

You can create additional storage class objects and specify a different value for the `type` parameter. You'll do this next.

Note

The availability of GCE Persistent Disk types depends on the zone in which your cluster is deployed. To view the list of types for each availability zone, run `gcloud compute disk-types list`.

Creating a new storage class to enable the use of SSD persistent disks in GKE

One of the disk types supported in most GCE zones is the `pd-ssd` type, which provisions a network-attached SSD. Let's create a storage class called `fast` and configure it so that the provisioner creates a disk of type `pd-ssd` when you request this storage class in your claim. The storage class manifest is shown in the next listing (file `sc.fast.gcepd.yaml`).

Listing 8.9 A custom storage class definition

```

apiVersion: storage.k8s.io/v1      #A
kind: StorageClass                  #A
metadata:
  name: fast                        #B
provisioner: kubernetes.io/gce-pd  #C

```

```
parameters:
  type: pd-ssd                                #D
```

Note

If you're using another cloud provider, check their documentation to find the name of the provisioner and the parameters you need to pass in. If you're using Minikube or kind, and you'd like to run this example, set the provisioner and parameters to the same values as in the default storage class. For this exercise, it doesn't matter if the provisioned volume doesn't actually use an SSD.

Create the StorageClass object by applying this manifest to your cluster and list the available storage classes to confirm that more than one is now available. You can now use this storage class in your claims. Let's conclude this section on dynamic provisioning by creating a persistent volume claim that will allow your Quiz pod to use an SSD disk.

Claiming a volume of a specific storage class

The following listing shows the updated YAML definition of the quiz-data claim, which requests the storage class fast that you've just created instead of using the default class. You'll find the manifest in the file `pvc.quiz-data-fast.yaml`.

Listing 8.10 A persistent volume claim requesting a specific storage class

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: quiz-data-fast
spec:
  storageClassName: fast      #A
  resources:
    requests:
      storage: 1Gi
  accessModes:
    - ReadWriteOnce
```

Rather than just specify the size and access modes and let the system use the

default storage class to provision the persistent volume, this claim specifies that the storage class `fast` be used for the volume. When you create the claim, the persistent volume is created by the provisioner referenced in this storage class, using the specified parameters.

You can now use this claim in a new instance of the Quiz pod. Apply the file `pod.quiz-fast.yaml`. If you run this example on GKE, the pod will use an SSD volume.

Note

If a persistent volume claim refers to a non-existent storage class, the claim remains `Pending` until the storage class is created. Kubernetes attempts to bind the claim at regular intervals, generating a `ProvisioningFailed` event each time. You can see the event if you execute the `kubectl describe` command on the claim.

8.3.4 Resizing persistent volumes

If the cluster supports dynamic provisioning, a cluster user can self-provision a storage volume with the properties and size specified in the claim and referenced storage class. If the user later needs a different storage class for their volume, they must, as you might expect, create a new persistent volume claim that references the other storage class. Kubernetes does not support changing the storage class name in an existing claim. If you try to do so, you receive the following error message:

```
* spec: Forbidden: is immutable after creation except resources.r
```

The error indicates that the majority of the claim's specification is immutable. The part that is mutable is `spec.resources.requests`, which is where you indicate the desired size of the volume.

In the previous MongoDB examples you requested 1GiB of storage space. Now imagine that the database grows near this size. Can the volume be resized without restarting the pod and application? Let's find out.

Requesting a larger volume in an existing persistent volume claim

If you use dynamic provisioning, you can generally change the size of a persistent volume simply by requesting a larger capacity in the associated claim. For the next exercise, you'll increase the size of the volume by modifying the `quiz-data-default` claim, which should still exist in your cluster.

To modify the claim, either edit the manifest file or create a copy and then edit it. Set the `spec.resources.requests.storage` field to `10Gi` as shown in the following listing. You can find this manifest in the book's GitHub repository (file `pvc.quiz-data-default. 10gib.pvc.yaml`).

Listing 8.11 Requesting a larger volume

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: quiz-data-default      #A
spec:
  resources:                   #B
    requests:                  #B
      storage: 10Gi           #B
  accessModes:
    - ReadWriteOnce
```

When you apply this file with the `kubectl apply` command, the existing `PersistentVolumeClaim` object is updated. Use the `kubectl get pvc` command to see if the volume's capacity has increased:

```
$ kubectl get pvc quiz-data-default
NAME                STATUS    VOLUME             CAPACITY   ACCESS MOD
quiz-data-default   Bound    pvc-ed36b...       1Gi        RWO
```

You may recall that when claims are listed, the `CAPACITY` column displays the size of the bound volume and not the size requirement specified in the claim. According to the output, this means that the size of the volume hasn't changed. Let's find out why.

Determining why the volume hasn't been resized

To find out why the size of the volume has remained the same regardless of

the change you made to the claim, the first thing you might do is inspect the claim using `kubectl describe`. If this is the case, you've already got the hang of debugging objects in Kubernetes. You'll find that one of the claim's conditions clearly explains why the volume was not resized:

```
$ kubectl describe pvc quiz-data-default
...
Conditions:
  Type              Status    ... Message
  ----              -
  FileSystemResizePending  True      ... Waiting for user to (re-)
                                         pod to finish file system
                                         volume on node.
```

To resize the persistent volume, you may need to delete and recreate the pod that uses the claim. After you do this, the claim and the volume will display the new size:

```
$ kubectl get pvc quiz-data-default
NAME              STATUS    VOLUME          CAPACITY   ACCESS M
quiz-data-default  Bound    pvc-ed36b...    10Gi      RWO
```

Allowing and disallowing volume expansion in the storage class

The previous example shows that cluster users can increase the size of the bound persistent volume by changing the storage requirement in the persistent volume claim. However, this is only possible if it's supported by the provisioner and the storage class.

When the cluster administrator creates a storage class, they can use the `spec.allowVolumeExpansion` field to indicate whether volumes of this class can be resized. If you attempt to expand a volume that you're not supposed to expand, the API server immediately rejects the update operation on the claim.

8.3.5 Understanding the benefits of dynamic provisioning

This section on dynamic provisioning should convince you that automating the provisioning of persistent volumes benefits both the cluster administrator and anyone who uses the cluster to deploy applications. By setting up the dynamic volume provisioner and configuring several storage classes with

different performance or other features, the administrator gives cluster users the ability to provision as many persistent volumes of any type as they want. Each developer decides which storage class is best suited for each claim they create.

Understanding how storage classes allow claims to be portable

Another great thing about storage classes is that claims refer to them by name. If the storage classes are named appropriately, such as `standard`, `fast`, and so on, the persistent volume claim manifests are portable across different clusters.

Note

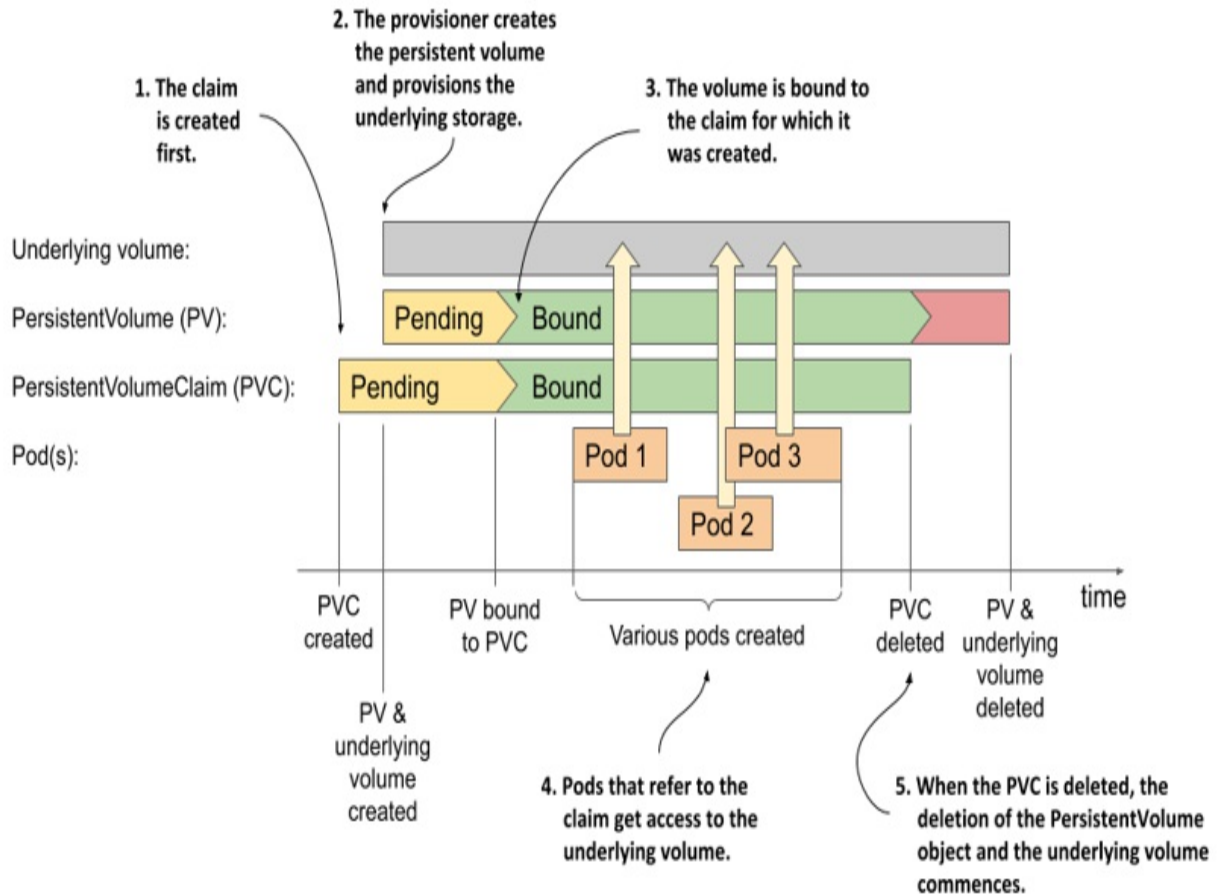
Remember that persistent volume claims are usually part of the application manifest and are written by application developers.

If you used GKE to run the previous examples, you can now try to deploy the same claim and pod manifests in a non-GKE cluster, such as a cluster created with Minikube or kind. In this way, you can see this portability for yourself. The only thing you need to ensure is that all your clusters use the storage class names.

8.3.6 Understanding the lifecycle of dynamically provisioned persistent volumes

To conclude this section on dynamic provisions, let's take one final look at the lifecycles of the underlying storage volume, the `PersistentVolume` object, the associated `PersistentVolumeClaim` object, and the pods that use them, like we did in the previous section on statically provisioned volumes.

Figure 8.9 The lifecycle of dynamically provisioned persistent volumes, claims and the pods using them



Unlike statically provisioned persistent volumes, the sequence of events when using dynamic provisioning begins with the creation of the PersistentVolumeClaim object. As soon as one such object appears, Kubernetes instructs the dynamic provisioner configured in the storage class referenced in this claim to provision a volume for it. The provisioner creates both the underlying storage, typically through the cloud provider's API, and the PersistentVolume object that references the underlying volume.

The underlying volume is typically provisioned asynchronously. When the process completes, the status of the PersistentVolume object changes to Available; at this point, the volume is bound to the claim.

Users can then deploy pods that refer to the claim to gain access to the underlying storage volume. When the volume is no longer needed, the user deletes the claim. This typically triggers the deletion of both the

PersistentVolume object and the underlying storage volume.

This entire process is repeated for each new claim that the user creates. A new PersistentVolume object is created for each claim, which means that the cluster can never run out of them. Obviously, the datacentre itself can run out of available disk space, but at least there is no need for the administrator to keep recycling old PersistentVolume objects.

8.4 Node-local persistent volumes

In the previous sections of this chapter, you've used persistent volumes and claims to provide network-attached storage volumes to your pods, but this type of storage is too slow for some applications. To run a production-grade database, you should probably use an SSD connected directly to the node where the database is running.

In the previous chapter, you learned that you can use a hostPath volume in a pod if you want the pod to access part of the host's filesystem. Now you'll learn how to do the same with persistent volumes. You might wonder why I need to teach you another way to do the same thing, but it's really not the same.

You might remember that when you add a hostPath volume to a pod, the data that the pod sees depends on which node the pod is scheduled to. In other words, if the pod is deleted and recreated, it might end up on another node and no longer have access to the same data.

If you use a local persistent volume instead, this problem is resolved. The Kubernetes scheduler ensures that the pod is always scheduled on the node to which the local volume is attached.

Note

Local persistent volumes are also better than hostPath volumes because they offer much better security. As explained in the previous chapter, you don't want to allow regular users to use hostPath volumes at all. Because persistent volumes are managed by the cluster administrator, regular users

can't use them to access arbitrary paths on the host node.

8.4.1 Creating local persistent volumes

Imagine you are a cluster administrator and you have just connected a fast SSD directly to one of the worker nodes. Because this is a new class of storage in the cluster, it makes sense to create a new StorageClass object that represents it.

Creating a storage class to represent local storage

Create a new storage class manifest as shown in the following listing.

Listing 8.12 Defining the local storage class

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: local                                #A
provisioner: kubernetes.io/no-provisioner    #B
volumeBindingMode: WaitForFirstConsumer     #C
```

As I write this, locally attached persistent volumes need to be provisioned manually, so you need to set the provisioner as shown in the listing. Because this storage class represents locally attached volumes that can only be accessed within the nodes to which they are physically connected, the volumeBindingMode is set to WaitForFirstConsumer, so the binding of the claim is delayed until the pod is scheduled.

Attaching a disk to a cluster node

I assume that you're using a Kubernetes cluster created with the kind tool to run this exercise. Let's emulate the installation of the SSD in the node called kind-worker. Run the following command to create an empty directory at the location /mnt/ssd1 in the node's filesystem:

```
$ docker exec kind-worker mkdir /mnt/ssd1
```

Creating a PersistentVolume object for the new disk

After attaching the disk to one of the nodes, you must tell Kubernetes that this node now provides a local persistent volume by creating a PersistentVolume object. The manifest for the persistent volume is shown in the following listing.

Listing 8.13 Defining a local persistent volume

```
kind: PersistentVolume
apiVersion: v1
metadata:
  name: local-ssd-on-kind-worker          #A
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: local                  #B
  capacity:
    storage: 10Gi
  local:                                  #C
    path: /mnt/ssd1                        #C
  nodeAffinity:                           #D
    required:                              #D
      nodeSelectorTerms:                  #D
        - matchExpressions:              #D
            - key: kubernetes.io/hostname #D
              operator: In                 #D
              values:                      #D
                - kind-worker              #D
```

Because this persistent volume represents a local disk attached to the kind-worker node, you give it a name that conveys this information. It refers to the local storage class that you created previously. Unlike previous persistent volumes, this volume represents storage space that is directly attached to the node. You therefore specify that it is a local volume. Within the local volume configuration, you also specify the path where the SSD is mounted (/mnt/ssd1).

At the bottom of the manifest, you'll find several lines that indicate the volume's node affinity. A volume's node affinity defines which nodes can access this volume.

Note

You'll learn more about node affinity and selectors in later chapters. Although it looks complicated, the node affinity definition in the listing simply defines that the volume is accessible from nodes whose hostname is `kind-worker`. This is obviously exactly one node.

Okay, as a cluster administrator, you've now done everything you needed to do to enable cluster users to deploy applications that use locally attached persistent volumes. Now it's time to put your application developer hat back on again.

8.4.2 Claiming and using local persistent volumes

As an application developer, you can now deploy your pod and its associated persistent volume claim.

Creating the pod

The pod definition is shown in the following listing.

Listing 8.14 Pod using a locally attached persistent volume

```
apiVersion: v1
kind: Pod
metadata:
  name: mongodb-local
spec:
  volumes:
  - name: mongodb-data
    persistentVolumeClaim:
      claimName: quiz-data-local      #A
  containers:
  - image: mongo
    name: mongodb
    volumeMounts:
    - name: mongodb-data
      mountPath: /data/db
```

There should be no surprises in the pod manifest. You already know all this.

Creating the persistent volume claim for a local volume

As with the pod, creating the claim for a local persistent volume is no different than creating any other persistent volume claim. The manifest is shown in the next listing.

Listing 8.15 Persistent volume claim using the local storage class

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: quiz-data-local
spec:
  storageClassName: local #A
  resources:
    requests:
      storage: 1Gi
  accessModes:
    - ReadWriteOnce
```

No surprises here either. Now on to creating these two objects.

Creating the pod and the claim

After you write the pod and claim manifests, you can create the two objects by applying the manifests in any order you want. If you create the pod first, since the pod requires the claim to exist, it simply remains in the Pending state until you create the claim.

After both the pod and the claim are created, the following events take place:

1. The claim is bound to the persistent volume.
2. The scheduler determines that the volume bound to the claim that is used in the pod can only be accessed from the kind-worker node, so it schedules the pod to this node.
3. The pod's container is started on this node, and the volume is mounted in it.

You can now use the MongoDB shell again to add documents to it. Then check the /mnt/ssd1 directory on the kind-worker node to see if the files are

stored there.

Recreating the pod

If you delete and recreate the pod, you'll see that it's always scheduled on the kind-worker node. The same happens if multiple nodes can provide a local persistent volume when you deploy the pod for the first time. At this point, the scheduler selects one of them to run your MongoDB pod. When the pod runs, the claim is bound to the persistent volume on that particular node. If you then delete and recreate the pod, it is always scheduled on the same node, since that is where the volume that is bound to the claim referenced in the pod is located.

8.5 Summary

This chapter explained the details of adding persistent storage for your applications. You've learned that:

- Infrastructure-specific information about storage volumes doesn't belong in pod manifests. Instead, it should be specified in the `PersistentVolume` object.
- A `PersistentVolume` object represents a portion of the disk space that is available to applications within the cluster.
- Before an application can use a `PersistentVolume`, the user who deploys the application must claim the `PersistentVolume` by creating a `PersistentVolumeClaim` object.
- A `PersistentVolumeClaim` object specifies the minimum size and other requirements that the `PersistentVolume` must meet.
- When using statically provisioned volumes, Kubernetes finds an existing persistent volume that meets the requirements set forth in the claim and binds it to the claim.
- When the cluster provides dynamic provisioning, a new persistent volume is created for each claim. The volume is created based on the requirements specified in the claim.
- A cluster administrator creates `StorageClass` objects to specify the storage classes that users can request in their claims.

- A user can change the size of the persistent volume used by their application by modifying the minimum volume size requested in the claim.
- Local persistent volumes are used when applications need to access disks that are directly attached to nodes. This affects the scheduling of the pods, since the pod must be scheduled to one of the nodes that can provide a local persistent volume. If the pod is subsequently deleted and recreated, it will always be scheduled to the same node.

In the next chapter, you'll learn how to pass configuration data to your applications using command-line arguments, environment variables, and files. You'll learn how to specify this data directly in the pod manifest and other Kubernetes API objects.

9 Configuration via ConfigMaps, Secrets, and the Downward API

This chapter covers

- Setting the command and arguments for the container's main process
- Setting environment variables
- Storing configuration in config maps
- Storing sensitive information in secrets
- Using the Downward API to expose pod metadata to the application
- Using configMap, secret, downwardAPI and projected volumes

You've now learned how to use Kubernetes to run an application process and attach file volumes to it. In this chapter, you'll learn how to configure the application - either in the pod manifest itself, or by referencing other API objects within it. You'll also learn how to inject information about the pod itself into the application running inside it.

Note

You'll find the code files for this chapter at <https://github.com/luksa/kubernetes-in-action-2nd-edition/tree/master/Chapter09>

9.1 Setting the command, arguments, and environment variables

Like regular applications, containerized applications can be configured using command-line arguments, environment variables, and files.

You learned that the command that is executed when a container starts is typically defined in the container image. The command is configured in the container's Dockerfile using the `ENTRYPOINT` directive, while the arguments

are typically specified using the `CMD` directive. Environment variables can also be specified using the `ENV` directive in the `Dockerfile`. If the application is configured using configuration files, these can be added to the container image using the `COPY` directive. You've seen several examples of this in the previous chapters.

Let's take the `kiada` application and make it configurable via command-line arguments and environment variables. The previous versions of the application all listen on port 8080. This will now be configurable via the `--listen-port` command line argument. Also, the application will read the initial status message from the environment variable `INITIAL_STATUS_MESSAGE`. Instead of just returning the hostname, the application now also returns the pod name and IP address, as well as the name of the cluster node on which it is running. The application obtains this information through environment variables. You can find the updated code in the book's code repository. The container image for this new version is available at `docker.io/luksa/kiada:0.4`.

The updated `Dockerfile`, which you can also find in the code repository, is shown in the following listing.

Listing 9.1 A sample `Dockerfile` using several application configuration methods

```
FROM node:12
COPY app.js /app.js
COPY html/ /html

ENV INITIAL_STATUS_MESSAGE="This is the default status message"

ENTRYPOINT ["node", "app.js"]
CMD ["--listen-port", "8080"]
```

Hardcoding the configuration into the container image is the same as hardcoding it into the application source code. This is not ideal because you must rebuild the image every time you change the configuration. Also, you should never include sensitive configuration data such as security credentials or encryption keys in the container image because anyone who has access to it can easily extract them.

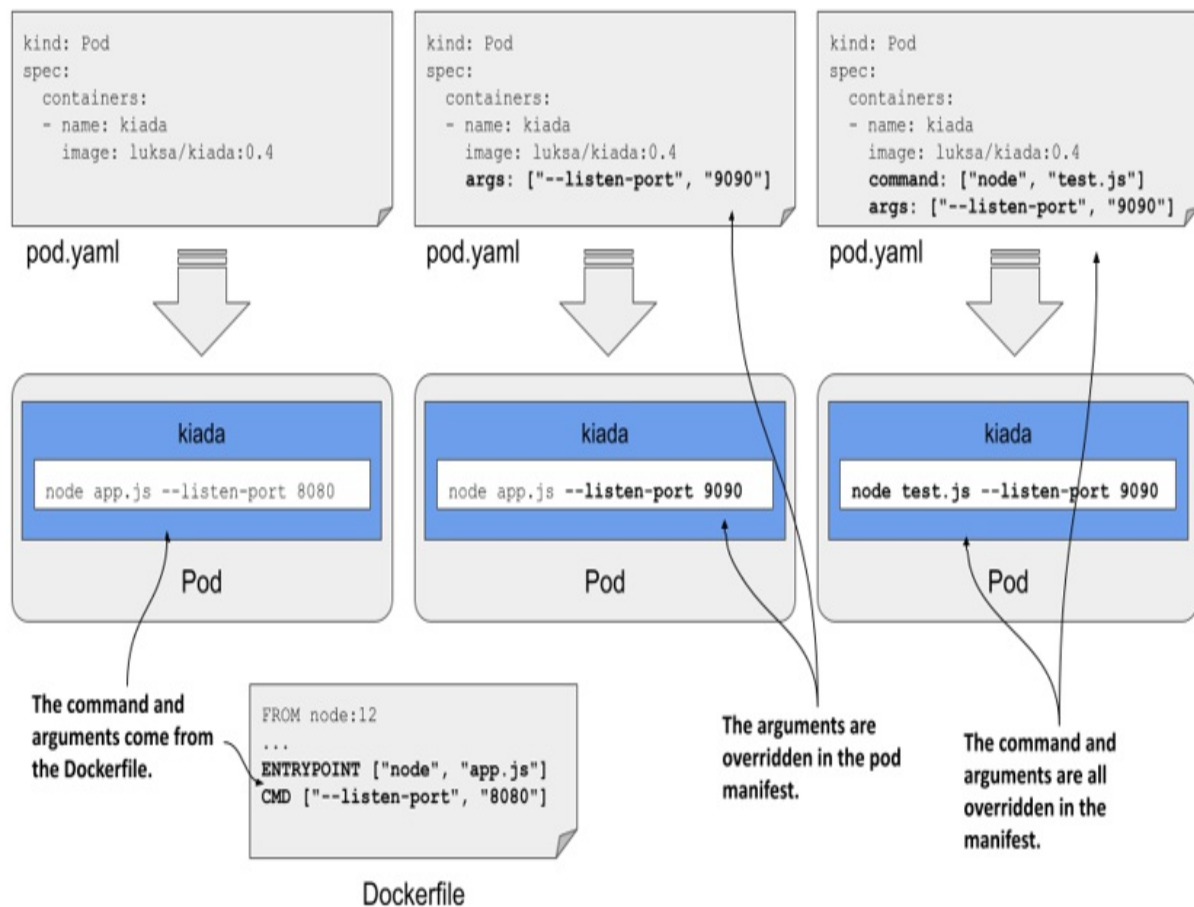
Instead, it's much safer to store these files in a volume that you mount in the container. As you learned in the previous chapter, one way to do this is to store the files in a persistent volume. Another way is to use an `emptyDir` volume and an `init` container that fetches the files from secure storage and writes them to the volume. You should know how to do this if you've read the previous chapters, but there's a better way. In this chapter, you'll learn how to use special volume types to achieve the same result without using `init` containers. But first, let's learn how to change the command, arguments, and environment variables without recreating the container image.

9.1.1 Setting the command and arguments

When creating a container image, the command and its arguments are specified using the `ENTRYPOINT` and `CMD` directives in the `Dockerfile`. Since both directives accept array values, you can specify both the command and its arguments with one of these directives or split them between the two. When the container is executed, the two arrays are concatenated to produce the full command.

Kubernetes provides two fields that are analogous to Docker's `ENTRYPOINT` and `CMD` directives. The two fields are called `command` and `args`, respectively. You specify these fields in the container definition in your pod manifest. As with Docker, the two fields accept array values, and the resulting command executed in the container is derived by concatenating the two arrays.

Figure 9.1 Overriding the command and arguments in the pod manifest



Normally, you use the `ENTRYPOINT` directive to specify the bare command, and the `CMD` directive to specify the arguments. This allows you to override the arguments in the pod manifest without having to specify the command again. If you want to override the command, you can still do so. And you can do it without overriding the arguments.

The following table shows the equivalent pod manifest field for each of the two Dockerfile directives.

Table 9.1 Specifying the command and arguments in the Dockerfile vs the pod manifest

Dockerfile	Pod manifest	Description

ENTRYPOINT	command	The executable file that runs in the container. This may contain arguments in addition to the executable.
CMD	args	Additional arguments passed to the command specified with the ENTRYPOINT directive or the command field.

Let's look at two examples of setting the command and args fields.

Setting the command

Imagine you want to run the Kiada application with CPU and heap profiling enabled. With Node.JS, you can enable profiling by passing the `--cpu-prof` and `--heap-prof` arguments to the node command. Instead of modifying the Dockerfile and rebuilding the image, you can do this by modifying the pod manifest, as shown in the following listing.

Listing 9.2 A container definition with the command specified

```
kind: Pod
spec:
  containers:
  - name: kiada
    image: luksa/kiada:0.4
    command: ["node", "--cpu-prof", "--heap-prof", "app.js"]      #
```

When you deploy the pod in the listing, the `node --cpu-prof --heap-prof app.js` command is run instead of the default command specified in the Dockerfile, which is `node app.js`.

As you can see in the listing, the `command` field, just like its Dockerfile counterpart, accepts an array of strings representing the command to be executed. The array notation used in the listing is great when the array contains only a few elements, but becomes difficult to read as the number of elements increases. In this case, you're better off using the following notation:

command:

- node
- --cpu-prof
- --heap-prof
- app.js

Tip

Values that the YAML parser might interpret as something other than a string must be enclosed in quotes. This includes numeric values such as 1234, and Boolean values such as true and false. Some other special strings must also be quoted, otherwise they would also be interpreted as Boolean or other types. These include the values true, false, yes, no, on, off, y, n, t, f, null, and others.

Setting command arguments

Command line arguments can be overridden with the args field, as shown in the following listing.

Listing 9.3 A container definition with the args fields set

```
kind: Pod
spec:
  containers:
  - name: kiada
    image: luksa/kiada:0.4
    args: ["--listen-port", "9090"]    #A
```

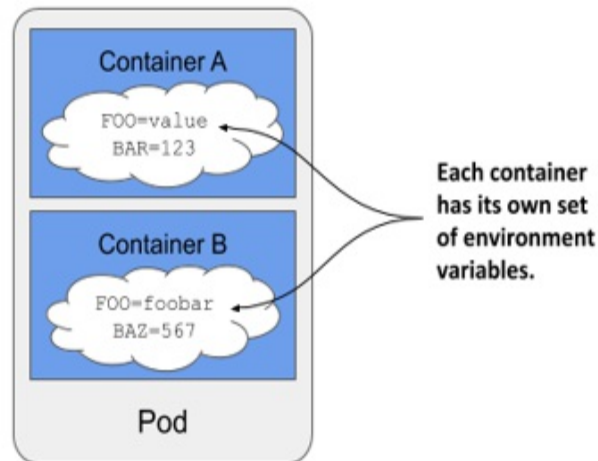
The pod manifest in the listing overrides the default `--listen-port 8080` arguments set in the Dockerfile with `--listen-port 9090`. When you deploy this pod, the full command that runs in the container is `node app.js --listen-port 9090`. The command is a concatenation of the ENTRYPOINT in the Dockerfile and the args field in the pod manifest.

9.1.2 Setting environment variables in a container

Containerized applications are often configured using environment variables. Just like the command and arguments, you can set environment variables for

each of the pod's containers, as shown in figure 9.2.

Figure 9.2 Environment variables are set per container.



Note

As I write this, environment variables can only be set for each container individually. It isn't possible to set a global set of environment variables for the entire pod and have them inherited by all its containers.

You can set an environment variable to a literal value, have it reference another environment variable, or obtain the value from an external source. Let's see how.

Setting a literal value to an environment variable

Version 0.4 of the Kiada application displays the name of the pod, which it reads from the environment variable `POD_NAME`. It also allows you to set the status message using the environment variable `INITIAL_STATUS_MESSAGE`. Let's set these two variables in the pod manifest.

To set the environment variable, you could add the `ENV` directive to the Dockerfile and rebuild the image, but the faster way is to add the `env` field to the container definition in the pod manifest, as I've done in the following listing (file `pod.kiada.env-value.yaml`).

Listing 9.4 Setting environment variables in the pod manifest

```
kind: Pod
metadata:
  name: kiada
spec:
  containers:
  - name: kiada
    image: luksa/kiada:0.4
    env:
      - name: POD_NAME                      #A
        value: kiada                       #B
      - name: INITIAL_STATUS_MESSAGE       #C
        value: This status message is set in the pod spec. #C
    ...
```

As you can see in the listing, the `env` field takes an array of values. Each entry in the array specifies the name of the environment variable and its value.

Note

Since environment variables values must be strings, you must enclose values that aren't strings in quotes to prevent the YAML parser from treating them as anything other than a string. As explained in section 9.1.1, this also applies to strings such as `yes`, `no`, `true`, `false`, and so on.

When you deploy the pod in the listing and send an HTTP request to the application, you should see the pod name and status message that you specified using environment variables. You can also run the following command to examine the environment variables in the container. You'll find the two environment variables in the following output:

```
$ kubectl exec kiada -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=kiada
NODE_VERSION=12.19.1
YARN_VERSION=1.22.5
POD_NAME=kiada
INITIAL_STATUS_MESSAGE=This status message is set in the pod spec
KUBERNETES_SERVICE_HOST=10.96.0.1
...
KUBERNETES_SERVICE_PORT=443
```

As you can see, there are a few other variables set in the container. They come from different sources - some are defined in the container image, some are added by Kubernetes, and the rest come from elsewhere. While there is no way to know where each of the variables comes from, you'll learn to recognize some of them. For example, the ones added by Kubernetes relate to the Service object, which is covered in chapter 11. To determine where the rest come from, you can inspect the pod manifest and the Dockerfile of the container image.

Using variable references in environment variable values

In the previous example, you set a fixed value for the environment variable `INITIAL_STATUS_MESSAGE`, but you can also reference other environment variables in the value by using the syntax `$(VAR_NAME)`.

For example, you can reference the variable `POD_NAME` within the status message variable as in the following listing, which shows part of the file `pod.kiada.env-value-ref.yaml`.

Listing 9.5 Referring to an environment variable in another variable

```
env:
- name: POD_NAME
  value: kiada
- name: INITIAL_STATUS_MESSAGE
  value: My name is $(POD_NAME). I run NodeJS version $(NODE_VERSION)
```

Notice that one of the references points to the environment variable `POD_NAME` defined above, whereas the other points to the variable `NODE_VERSION` set in the container image. You saw this variable when you ran the `env` command in the container earlier. When you deploy the pod, the status message it returns is the following:

```
My name is kiada. I run NodeJS version $(NODE_VERSION).
```

As you can see, the reference to `NODE_VERSION` isn't resolved. This is because you can only use the `$(VAR_NAME)` syntax to refer to variables defined in the same manifest. The referenced variable must be defined *before* the variable that references it. Since `NODE_VERSION` is defined in the NodeJS image's

Dockerfile and not in the pod manifest, it can't be resolved.

Note

If a variable reference can't be resolved, the reference string remains unchanged.

Note

When you want a variable to contain the literal string `$(VAR_NAME)` and don't want Kubernetes to resolve it, use a double dollar sign as in `$$$(VAR_NAME)`. Kubernetes will remove one of the dollar signs and skip resolving the variable.

Using variable references in the command and arguments

You can refer to environment variables defined in the manifest not only in other variables, but also in the command and args fields you learned about in the previous section. For example, the file `pod.kiada.env-value-ref-in-args.yaml` defines an environment variable named `LISTEN_PORT` and references it in the args field. The following listing shows the relevant part of this file.

Listing 9.6 Referring to an environment variable in the args field

```
spec:
  containers:
  - name: kiada
    image: luksa/kiada:0.4
    args:
    - --listen-port
    - $(LISTEN_PORT)           #A
    env:
    - name: LISTEN_PORT
      value: "8080"
```

This isn't the best example, since there's no good reason to use a variable reference instead of just specifying the port number directly. But later you'll learn how to get the environment variable value from an external source. You

can then use a reference as shown in the listing to inject that value into the container's command or arguments.

Referring to environment variables that aren't in the manifest

Just like using references in environment variables, you can only use the `$(VAR_NAME)` syntax in the `command` and `args` fields to reference variables that are defined in the pod manifest. You can't reference environment variables defined in the container image, for example.

However, you can use a different approach. If you run the command through a shell, you can have the shell resolve the variable. If you are using the `bash` shell, you can do this by referring to the variable using the syntax `$VAR_NAME` or `${VAR_NAME}` instead of `$(VAR_NAME)`.

For example, the command in the following listing correctly prints the value of the `HOSTNAME` environment variable even though it's not defined in the pod manifest but is initialized by the operating system. You can find this example in the file `pod.env-var-references-in-shell.yaml`.

Listing 9.7 Referring to environment variables in a shell command

```
containers:
- name: main
  image: alpine
  command:
  - sh      #A
  - -c      #A
  - 'echo "Hostname is $HOSTNAME."; sleep infinity'  #B
```

Setting the pod's fully qualified domain name

While we're on the subject of the pod's hostname, this is a good time to explain that the pod's hostname and subdomain are configurable in the pod manifest. By default, the hostname is the same as the pod's name, but you can override it using the `hostname` field in the pod's spec. You can also set the `subdomain` field so that the fully qualified domain name (FQDN) of the pod is as follows:

```
<hostname>.<subdomain>.<pod namespace>.svc.<cluster domain>
```

This is only the internal FQDN of the pod. It isn't resolvable via DNS without additional steps, which are explained in chapter 11. You can find a sample pod that specifies a custom hostname for the pod in the file `pod.kiada.hostname.yaml`.

9.2 Using a config map to decouple configuration from the pod

In the previous section, you learned how to hardcode configuration directly into your pod manifests. While this is much better than hard-coding in the container image, it's still not ideal because it means you might need a separate version of the pod manifest for each environment you deploy the pod to, such as your development, staging, or production cluster.

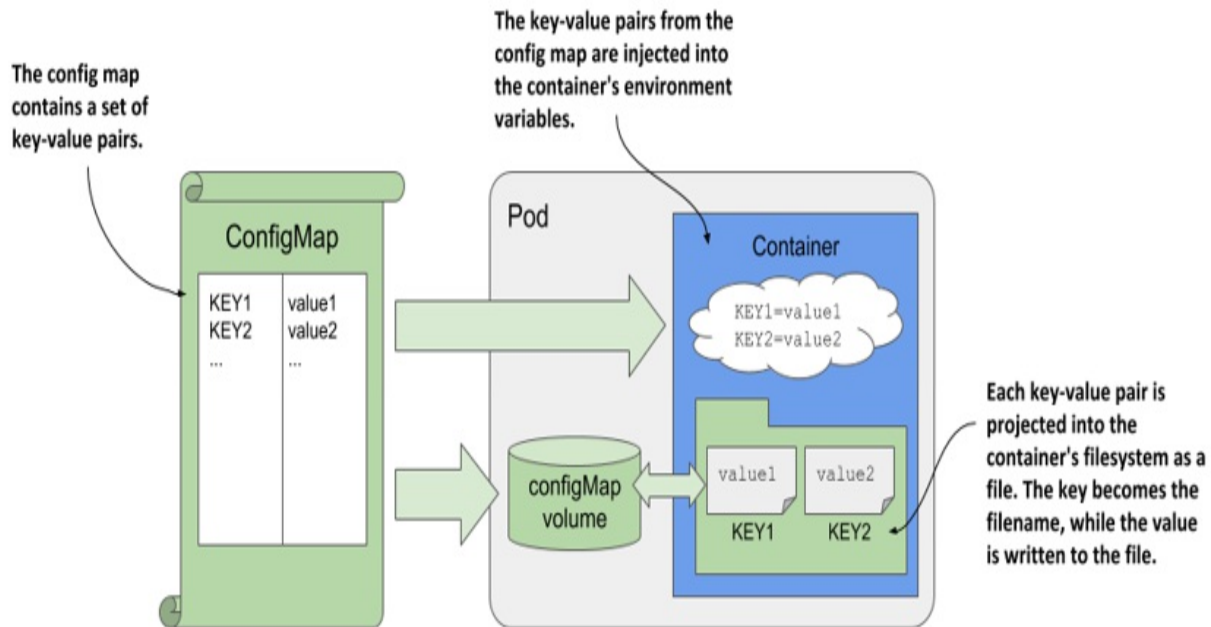
To reuse the same pod definition in multiple environments, it's better to decouple the configuration from the pod manifest. One way to do this is to move the configuration into a ConfigMap object, which you then reference in the pod manifest. This is what you'll do next.

9.2.1 Introducing ConfigMaps

A ConfigMap is a Kubernetes API object that simply contains a list of key/value pairs. The values can range from short strings to large blocks of structured text that you typically find in an application configuration file. Pods can reference one or more of these key/value entries in the config map. A pod can refer to multiple config maps, and multiple pods can use the same config map.

To keep applications Kubernetes-agnostic, they typically don't read the ConfigMap object via the Kubernetes REST API. Instead, the key/value pairs in the config map are passed to containers as environment variables or mounted as files in the container's filesystem via a configMap volume, as shown in the following figure.

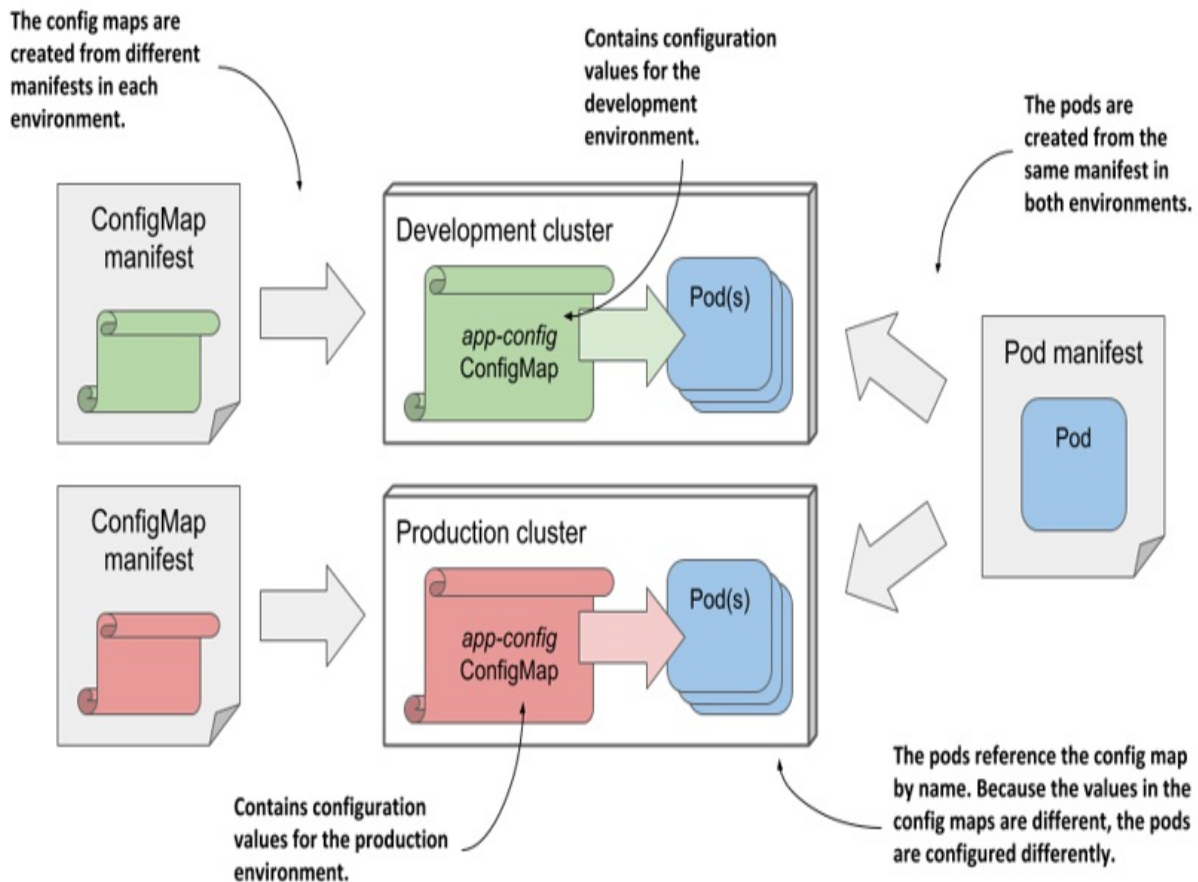
Figure 9.3 Pods use config maps through environment variables and configMap volumes.



In the previous section you learned how to reference environment variables in command-line arguments. You can use this technique to pass a config map entry that you've exposed as an environment variable into a command-line argument.

Regardless of how an application consumes config maps, storing the configuration in a separate object instead of the pod allows you to keep the configuration separate for different environments by simply keeping separate config map manifests and applying each to the environment for which it is intended. Because pods reference the config map by name, you can deploy the same pod manifest across all your environments and still have a different configuration for each environment by using the same config map name, as shown in the following figure.

Figure 9.4 Deploying the same pod manifest and different config map manifests in different environments



9.2.2 Creating a ConfigMap object

Let's create a config map and use it in a pod. The following is a simple example where the config map contains a single entry used to initialize the environment variable `INITIAL_STATUS_MESSAGE` for the kiada pod.

Creating a config map with the `kubectl create configmap` command

As with pods, you can create the ConfigMap object from a YAML manifest, but a faster way is to use the `kubectl create configmap` command as follows:

```
$ kubectl create configmap kiada-config --from-literal status-mes  
configmap "kiada-config" created
```

Note

Keys in a config map may only consist of alphanumeric characters, dashes, underscores, or dots. Other characters are not allowed.

Running this command creates a config map called `kiada-config` with a single entry. The key and value are specified with the `--from-literal` argument.

In addition to `--from-literal`, the `kubectl create configmap` command also supports sourcing the key/value pairs from files. The following table explains the available methods.

Table 9.2 Options for creating config map entries using `kubectl create configmap`

Option	Description
<code>--from-literal</code>	Inserts a key and a literal value into the config map. Example: <code>--from-literal mykey=myvalue</code> .
<code>--from-file</code>	Inserts the contents of a file into the config map. The behavior depends on the argument that comes after <code>--from-file</code>: If only the filename is specified (example: <code>--from-file myfile.txt</code>), the base name of the file is used as the key and the entire contents of the file are used as the value. If <code>key=file</code> is specified (example: <code>--from-file mykey=myfile.txt</code>), the contents of the file are stored under the specified key. If the filename represents a directory, each file contained in the directory is included as a separate entry. The base name of the file is used as the key, and the contents of the file are used as the value. Subdirectories, symbolic links, devices, pipes, and files whose base name isn't a valid

	config map key are ignored.
--from-env-file	Inserts each line of the specified file as a separate entry (example: --from-env-file myfile.env). The file must contain lines with the following format: key=value

Config maps usually contain more than one entry. To create a config map with multiple entries, you can use multiple arguments --from-literal, --from-file, and --from-env-file, or a combination thereof.

Creating a config map from a YAML manifest

Alternatively, you can create the config map from a YAML manifest file. The following listing shows the contents of an equivalent manifest file named `cm.kiada-config.yaml`, which is available in the code repository You can create the config map by applying this file using `kubectl apply`.

Listing 9.8 A config map manifest file

```
apiVersion: v1    #A
kind: ConfigMap  #A
metadata:
  name: kiada-config  #B
data:
  status-message: This status message is set in the kiada-config  #C
```

Listing config maps and displaying their contents

Config maps are Kubernetes API objects that live alongside pods, nodes, persistent volumes, and the others you’ve learned about so far. You can use various `kubectl` commands to perform CRUD operations on them. For example, you can list config maps with:

```
$ kubectl get cm
```

Note

The shorthand for configmaps is cm.

You can display the entries in the config map by instructing kubectl to print its YAML manifest:

```
$ kubectl get cm kiada-config -o yaml
```

Note

Because YAML fields are output in alphabetical order, you'll find the data field at the top of the output.

Tip

To display only the key/value pairs, combine kubectl with jq. For example: `kubectl get cm kiada-config -o json | jq .data`. Display the value of a given entry as follows: `kubectl... | jq '.data["status-message"]'`.

9.2.3 Injecting config map values into environment variables

In the previous section, you created the kiada-config config map. Let's use it in the kiada pod.

Injecting a single config map entry

To inject the single config map entry into an environment variable, you just need to replace the `value` field in the environment variable definition with the `valueFrom` field and refer to the config map entry. The following listing shows the relevant part of the pod manifest. The full manifest can be found in the file `pod.kiada.env-valueFrom.yaml`.

Listing 9.9 Setting an environment variable from a config map entry

```
kind: Pod
...
spec:
  containers:
  - name: kiada
    env:    #A
```

```

- name: INITIAL_STATUS_MESSAGE    #A
  valueFrom:    #B
    configMapKeyRef:    #B
      name: kiada-config    #C
      key: status-message    #D
      optional: true    #E
  volumeMounts:
- ...

```

Let me break down the definition of the environment variable that you see in the listing. Instead of specifying a fixed value for the variable, you declare that the value should be obtained from a config map. The name of the config map is specified using the name field, whereas the key field specifies the key within that map.

Create the pod from this manifest and inspect its environment variables using the following command:

```

$ kubectl exec kiada -- env
...
INITIAL_STATUS_MESSAGE=This status message is set in the kiada-co
...

```

The status message should also appear in the pod's response when you access it via curl or your browser.

Marking a reference optional

In the previous listing, the reference to the config map key is marked as optional so that the container can be executed even if the config map or key is missing. If that's the case, the environment variable isn't set. You can mark the reference as optional because the Kiada application will run fine without it. You can delete the config map and deploy the pod again to confirm this.

Note

If a config map or key referenced in the container definition is missing and not marked as optional, the pod will still be scheduled normally. The other containers in the pod are started normally. The container that references the missing config map key is started as soon as you create the config map with

the referenced key.

Injecting the entire config map

The `env` field in a container definition takes an array of values, so you can set as many environment variables as you need. However, if you want to set more than a few variables, it can become tedious and error prone to specify them one at a time. Fortunately, by using the `envFrom` instead of the `env` field, you can inject all the entries that are in the config map without having to specify each key individually.

The downside to this approach is that you lose the ability to transform the key to the environment variable name, so the keys must already have the proper form. The only transformation that you can do is to prepend a prefix to each key.

For example, the Kiada application reads the environment variable `INITIAL_STATUS_MESSAGE`, but the key you used in the config map is `status-message`. You must change the config map key to match the expected environment variable name if you want it to be read by the application when you use the `envFrom` field to inject the entire config map into the pod. I've already done this in the `cm.kiada-config.envFrom.yaml` file. In addition to the `INITIAL_STATUS_MESSAGE` key, it contains two other keys to demonstrate that they will all be injected into the container's environment.

Replace the config map with the one in the file by running the following command:

```
$ kubectl replace -f cm.kiada-config.envFrom.yaml
```

The pod manifest in the `pod.kiada.envFrom.yaml` file uses the `envFrom` field to inject the entire config map into the pod. The following listing shows the relevant part of the manifest.

Listing 9.10 Using `envFrom` to inject the entire config map into environment variables

```
kind: Pod
...
```

```
spec:
  containers:
  - name: kiada
    envFrom:
      - configMapRef:
          name: kiada-config
          optional: true
```

Instead of specifying both the config map name and the key as in the previous example, only the config map name is specified. If you create the pod from this manifest and inspect its environment variables, you'll see that it contains the environment variable `INITIAL_STATUS_MESSAGE` as well as the other two keys defined in the config map.

As before, you can mark the config map reference as `optional`, allowing the container to run even if the config map doesn't exist. By default, this isn't the case. Containers that reference config maps are prevented from starting until the referenced config maps exist.

Injecting multiple config maps

Listing 9.10 shows that the `envFrom` field takes an array of values, which means you can combine entries from multiple config maps. If two config maps contain the same key, the last one takes precedence. You can also combine the `envFrom` field with the `env` field if you wish to inject all entries of one config map and particular entries of another.

Note

When an environment variable is configured in the `env` field, it takes precedence over environment variables set in the `envFrom` field.

Prefixing keys

Regardless of whether you inject a single config map or multiple config maps, you can set an optional `prefix` for each config map. When their entries are injected into the container's environment, the prefix is prepended to each key to yield the environment variable name.

9.2.4 Injecting config map entries into containers as files

Environment variables are typically used to pass small single-line values to the application, while multiline values are usually passed as files. Config map entries can also contain larger blocks of data that can be projected into the container using the special `configMap` volume type.

Note

The amount of information that can fit in a config map is dictated by etcd, the underlying data store used to store API objects. At this point, the maximum size is on the order of one megabyte.

A `configMap` volume makes the config map entries available as individual files. The process running in the container gets the entry's value by reading the contents of the file. This mechanism is most often used to pass large config files to the container, but can also be used for smaller values, or combined with the `env` or `envFrom` fields to pass large entries as files and others as environment variables.

Creating config map entries from files

In chapter 4, you deployed the `kiada` pod with an Envoy sidecar that handles TLS traffic for the pod. Because volumes weren't explained at that point, the configuration file, TLS certificate, and private key that Envoy uses were built into the container image. It would be more convenient if these files were stored in a config map and injected into the container. That way you could update them without having to rebuild the image. But since the security considerations of these files are different, we must handle them differently. Let's focus on the config file first.

You've already learned how to create a config map from a literal value using the `kubectl create configmap` command. This time, instead of creating the config map directly in the cluster, you'll create a YAML manifest for the config map so that you can store it in a version control system alongside your pod manifest.

Instead of writing the manifest file by hand, you can create it using the same `kubectl create` command that you used to create the object directly. The following command creates the YAML file for a config map named `kiada-envoy-config`:

```
$ kubectl create configmap kiada-envoy-config \
  --from-file=envoy.yaml \
  --from-file=dummy.bin \
  --dry-run=client -o yaml > cm.kiada-envoy-config.yaml
```

The config map will contain two entries that come from the files specified in the command. One is the `envoy.yaml` configuration file, while the other is just some random data to demonstrate that binary data can also be stored in a config map.

When using the `--dry-run` option, the command doesn't create the object in the Kubernetes API server, but only generates the object definition. The `-o yaml` option prints the YAML definition of the object to standard output, which is then redirected to the `cm.kiada-envoy-config.yaml` file. The following listing shows the contents of this file.

Listing 9.11 A config map manifest containing a multi-line value

```
apiVersion: v1
binaryData:
  dummy.bin: n2VW39IEkyQ6Jxo+rdo5J06Vi7cz5...  #A
data:
  envoy.yaml: |  #B
    admin:  #B
      access_log_path: /tmp/envoy.admin.log  #B
      address:  #B
        socket_address:  #B
          protocol: TCP  #B
          address: 0.0.0.0  #B
        ...  #B
kind: ConfigMap
metadata:
  creationTimestamp: null
  name: kiada-envoy-config  #C
```

As you can see in the listing, the binary file ends up in the `binaryData` field, whereas the envoy config file is in the `data` field, which you know from the

previous sections. If a config map entry contains non-UTF-8 byte sequences, it must be defined in the `binaryData` field. The `kubectl create configmap` command automatically determines where to put the entry. The values in this field are Base64 encoded, which is how binary values are represented in YAML.

In contrast, the contents of the `envoy.yaml` file are clearly visible in the `data` field. In YAML, you can specify multi-line values using a pipeline character and appropriate indentation. See the YAML specification on [YAML.org](https://yaml.org) for more ways to do this.

Mind your whitespace hygiene when creating config maps

When creating config maps from files, make sure that none of the lines in the file contain trailing whitespace. If any line ends with whitespace, the value of the entry in the manifest is formatted as a quoted string with the newline character escaped. This makes the manifest incredibly hard to read and edit.

Compare the formatting of the two values in the following config map:

```
$ kubectl create configmap whitespace-demo \
--from-file=envoy.yaml \
--from-file=envoy-trailingspace.yaml \
--dry-run=client -o yaml
apiVersion: v1
data:
  envoy-trailingspace.yaml: "admin: \n access_log_path: /tmp/envoy.
\ address:\n socket_address:\n protocol: TCP\n address: 0.0.0.0\n
\ port_value: 9901\nstatic_resources:\n listeners:\n - name: list
envoy.yaml: | #B
admin: #B
access_log_path: /tmp/envoy.admin.log #B
address: #B
socket_address:... #B
```

Notice that the `envoy-trailingspace.yaml` file contains a space at the end of the first line. This causes the config map entry to be presented in a not very human-friendly format. In contrast, the `envoy.yaml` file contains no trailing whitespace and is presented as an unescaped multi-line string, which makes it easy to read and modify in place.

Don't apply the config map manifest file to the Kubernetes cluster yet. You'll first create the pod that refers to the config map. This way you can see what happens when a pod points to a config map that doesn't exist.

Using a configMap volume in a pod

To make config map entries available as files in the container's filesystem, you define a configMap volume in the pod and mount it in the container, as in the following listing, which shows the relevant parts of the `pod.kiada-ssl.configmap-volume.yaml` file.

Listing 9.12 Defining a configMap volume in a pod

```
apiVersion: v1
kind: Pod
metadata:
  name: kiada-ssl
spec:
  volumes:
    - name: envoy-config    #A
      configMap:            #A
        name: kiada-envoy-config    #A
    ...
  containers:
    ...
    - name: envoy
      image: luksa/kiada-ssl-proxy:0.1
      volumeMounts:        #B
        - name: envoy-config    #B
          mountPath: /etc/envoy    #B
    ...
```

If you've read the previous two chapters, the definitions of the `volume` and `volumeMount` in this listing should be clear. As you can see, the volume is a configMap volume that points to the `kiada-envoy-config` config map, and it's mounted in the `envoy` container under `/etc/envoy`. The volume contains the `envoy.yaml` and `dummy.bin` files that match the keys in the config map.

Create the pod from the manifest file and check its status. Here's what you'll see:

```
$ kubectl get po
NAME          READY   STATUS             RESTARTS   AGE
Kiada-ssl    0/2     ContainerCreating    0           2m
```

Because the pod's configMap volume references a config map that doesn't exist, and the reference isn't marked as optional, the container can't run.

Marking a configMap volume as optional

Previously, you learned that if a container contains an environment variable definition that refers to a config map that doesn't exist, the container is prevented from starting until you create that config map. You also learned that this doesn't prevent the other containers from starting. What about the case at hand where the missing config map is referenced in a volume?

Because all of the pod's volumes must be set up before the pod's containers can be started, referencing a missing config map in a volume prevents all the containers in the pod from starting, not just the container in which the volume is mounted. An event is generated indicating the problem. You can display it with the `kubectl describe pod` or `kubectl get events` command, as explained in the previous chapters.

Note

A configMap volume can be marked as optional by adding the line `optional: true` to the volume definition. If a volume is optional and the config map doesn't exist, the volume is not created, and the container is started without mounting the volume.

To enable the pod's containers to start, create the config map by applying the `cm.kiada-envoy-config.yaml` file you created earlier. Use the `kubectl apply` command. After doing this, the pod should start, and you should be able to confirm that both config map entries are exposed as files in the container by listing the contents of the `/etc/envoy` directory as follows:

```
$ kubectl exec kiada-ssl -c envoy -- ls /etc/envoy
dummy.bin
envoy.yaml
```

Projecting only specific config map entries

Envoy doesn't need the `dummy.bin` file, but imagine that it's needed by another container or pod and you can't remove it from the config map. But having this file appear in `/etc/envoy` is not ideal, so let's do something about it.

Fortunately, `configMap` volumes let you specify which config map entries to project into files. The following listing shows how.

Listing 9.13 Specifying which config map entries to include into a `configMap` volume

```
volumes:
- name: envoy-config
  configMap:
    name: kiada-envoy-config
    items: #A
    - key: envoy.yaml #B
      path: envoy.yaml #B
```

The `items` field specifies the list of config map entries to include in the volume. Each item must specify the key and the file name in the `path` field. Entries not listed here aren't included in the volume. In this way, you can have a single config map for a pod with some entries showing up as environment variables and others as files.

Setting file permissions in a `configMap` volume

By default, the file permissions in a `configMap` volume are set to `rw-r--r--` or `0644` in octal form.

Note

If you aren't familiar with Unix file permissions, `0644` in the octal number system is equivalent to `110,100,100` in the binary system, which maps to the permissions triplet `rw-, r--, r--`. The first element refers to the file owner's permissions, the second to the owning group, and the third to all other users. The owner can read (r) and write (w) the file but can't execute it (- instead of

x), while the owning group and other users can read, but not write or execute the file (r--).

You can set the default permissions for the files in a `configMap` volume by setting the `defaultMode` field in the volume definition. In YAML, the field takes either an octal or decimal value. For example, to set permissions to `rwxr-----`, add `defaultMode: 0740` to the `configMap` volume definition. To set permissions for individual files, set the `mode` field next to the item's key and path.

When specifying file permissions in YAML manifests, make sure you never forget the leading zero, which indicates that the value is in octal form. If you omit the zero, the value will be treated as decimal, which may cause the file to have permissions that you didn't intend.

Important

When you use `kubectl get -o yaml` to display the YAML definition of a pod, note that the file permissions are represented as decimal values. For example, you'll regularly see the value `420`. This is the decimal equivalent of the octal value `0644`, which is the default file permissions.

Before you move on to setting file permissions and checking them in the container, you should know that the files you find in the `configMap` volume are symbolic links (section 9.2.6 explains why). To see the permissions of the actual file, you must follow these links, because they themselves have no permissions and are always shown as `rwxrwxrwx`.

9.2.5 Updating and deleting config maps

As with most Kubernetes API objects, you can update a config map at any time by modifying the manifest file and reapplying it to the cluster using `kubectl apply`. There's also a quicker way, which you'll mostly use during development.

In-place editing of API objects using `kubectl edit`

When you want to make a quick change to an API object, such as a ConfigMap, you can use the `kubectl edit` command. For example, to edit the `kiada-envoy-config` config map, run the following command:

```
$ kubectl edit configmap kiada-envoy-config
```

This opens the object manifest in your default text editor, allowing you to change the object directly. When you close the editor, `kubectl` posts your changes to the Kubernetes API server.

Configuring `kubectl edit` to use a different text editor

You can tell `kubectl` to use a text editor of your choice by setting the `KUBE_EDITOR` environment variable. For example, if you'd like to use `nano` for editing Kubernetes resources, execute the following command (or put it into your `~/.bashrc` or an equivalent file):

```
export KUBE_EDITOR="/usr/bin/nano"
```

If the `KUBE_EDITOR` environment variable isn't set, `kubectl edit` falls back to using the default editor, usually configured through the `EDITOR` environment variable.

What happens when you modify a config map

When you update a config map, the files in the `configMap` volume are automatically updated.

Note

It can take up to a minute for the files in a `configMap` volume to be updated after you change the config map.

Unlike files, environment variables can't be updated while the container is running. However, if the container is restarted for some reason (because it crashed or because it was terminated externally due to a failed liveness probe), Kubernetes will use the new config map values when it sets up the environment variables for the new container. The question is whether you

want it to do that at all.

Understanding the consequences of updating a config map

One of the most important properties of containers is their immutability, which allows you to be sure that there are no differences between multiple instances of the same container (or pod). Shouldn't the config maps from which these instances get their configuration also be immutable?

Let's think about this for a moment. What happens if you change a config map used to inject environment variables into an application? What if the application is configured via config files, but it doesn't automatically reload them when they are modified? The changes you make to the config map don't affect any of these running application instances. However, if some of these instances are restarted or if you create additional instances, they *will* use the new configuration.

A similar scenario occurs even with applications that can reload their configuration. Kubernetes updates configMap volumes asynchronously. Some application instances may see the changes sooner than others. And because the update process may take dozens of seconds, the files in individual pod instances can be out of sync for a considerable amount of time.

In both scenarios, you get instances that are configured differently. This may cause parts of your system to behave differently than the rest. You need to take this into account when deciding whether to allow changes to a config map while it's being used by running pods.

Preventing a config map from being updated

To prevent users from changing the values in a config map, you can mark the config map as immutable, as shown in the following listing.

Listing 9.14 Creating an immutable config map

```
apiVersion: v1
```

```
kind: ConfigMap
metadata:
  name: my-immutable-configmap
data:
  mykey: myvalue
  another-key: another-value
immutable: true                                #A
```

If someone tries to change the data or binaryData fields in an immutable config map, the API server will prevent it. This ensures that all pods using this config map use the same configuration values. If you want to run a set of pods with a different configuration, you typically create a new config map and point them to it.

Immutable config maps prevent users from accidentally changing application configuration, but also help improve the performance of your Kubernetes cluster. When a config map is marked as immutable, the Kubelets on the worker nodes that use it don't have to be notified of changes to the ConfigMap object. This reduces the load on the API server.

Deleting a config map

ConfigMap objects can be deleted with the `kubectl delete` command. The running pods that reference the config map continue to run unaffected, but only until their containers must be restarted. If the config map reference in the container definition isn't marked as optional, the container will fail to run.

9.2.6 Understanding how configMap volumes work

Before you start using configMap volumes in your own pods, it's important that you understand how they work, or you'll spend a lot of time fighting them.

You might think that when you mount a configMap volume in a directory in the container, Kubernetes merely creates some files in that directory, but things are more complicated than that. There are two caveats that you need to keep in mind. One is how volumes are mounted in general, and the other is how Kubernetes uses symbolic links to ensure that files are updated atomically.

Mounting a volume hides existing files in the file directory

If you mount any volume to a directory in the container's filesystem, the files that are in the container image in that directory can no longer be accessed. For example, if you mount a `configMap` volume into the `/etc` directory, which in a Unix system contains important configuration files, the applications running in the container will only see the files defined in the config map. This means that all other files that should be in `/etc` are no longer present and the application may not run. However, this problem can be mitigated by using the `subPath` field when mounting the volume.

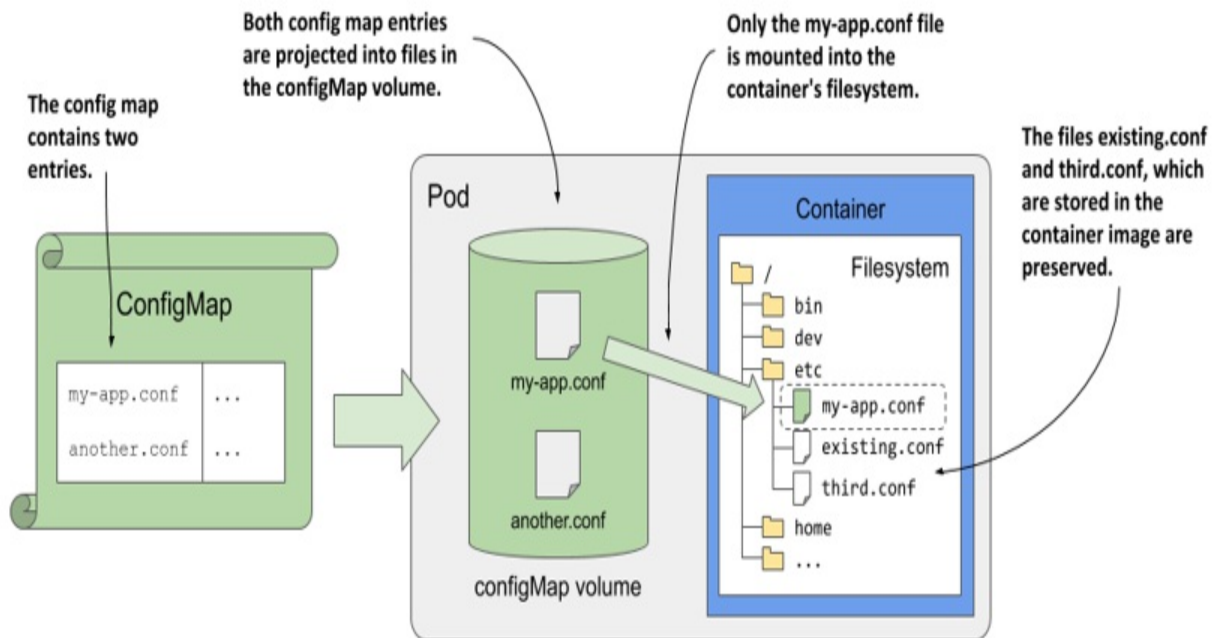
Imagine you have a `configMap` volume that contains the file `my-app.conf`, and you want to add it to the `/etc` directory without losing any existing files in that directory. Instead of mounting the entire volume in `/etc`, you mount only the specific file using a combination of the `mountPath` and `subPath` fields, as shown in the following listing.

Listing 9.15 Mounting an individual file into a container

```
spec:
  containers:
    - name: my-container
      volumeMounts:
        - name: my-volume
          subPath: my-app.conf           #A
          mountPath: /etc/my-app.conf    #B
```

To make it easier to understand how all this works, inspect the following figure.

Figure 9.5 Using `subPath` to mount a single file from the volume



The `subPath` property can be used when mounting any type of volume, but when you use it with a `configMap` volume, please note the following warning:

Warning

If you use the `subPath` field to mount individual files instead of the entire `configMap` volume, the file won't be updated when you modify the config map.

To get around this problem, you can mount the entire volume in another directory and create a symbolic link in the desired location pointing to the file in the other directory. You can create this symbolic link beforehand in the container image itself.

ConfigMap volumes use symbolic links to provide atomic updates

Some applications watch for changes to their configuration files and reload them when this happens. However, if the application is using a large file or multiple files, the application may detect that a file has changed before all file updates are complete. If the application reads the partially updated files, it

may not function properly.

To prevent this, Kubernetes ensures that all files in a configMap volume are updated atomically, meaning that all updates are done instantaneously. This is achieved with the use of symbolic file links, as you can see if you list all the files in the /etc/envoy directory:

```
$ kubectl exec kiada-ssl -c envoy -- ls -lA /etc/envoy
total 4
drwxr-xr-x    ...    ..2020_11_14_11_47_45.728287366    #A
lrwxrwxrwx    ...    ..data -> ..2020_11_14_11_47_45.728287366    #B
lrwxrwxrwx    ...    envoy.yaml -> ..data/envoy.yaml    #C
```

As you can see in the listing, the config map entries that are projected as files into the volume are symbolic links that point to file paths within the directory named `..data`, which is also a symbolic link. It points to a directory whose name clearly represents a timestamp. So the file paths that the application reads point to actual files via two successive symbolic links.

This may look unnecessary, but it allows you to update all files atomically. Every time you change the config map, Kubernetes creates a new timestamped directory, writes the files to it, and then associates the `..data` symbolic link with this new directory, replacing all files instantaneously.

Note

If you use `subPath` in your volume mount definition, this mechanism isn't used. Instead, the file is written directly to the target directory and the file isn't updated when you modify the config map.

9.3 Using Secrets to pass sensitive data to containers

In the previous section, you learned how to store configuration data in ConfigMap objects and make it available to the application via environment variables or files. You may think that you can also use config maps to also store sensitive data such as credentials and encryption keys, but this isn't the best option. For any data that needs to be kept secure, Kubernetes provides another type of object - *Secrets*. They will be covered next.

9.3.1 Introducing Secrets

Secrets are remarkably similar to config maps. Just like config maps, they contain key-value pairs and can be used to inject environment variables and files into containers. So why do we need secrets at all?

Kubernetes supported secrets even before config maps were added. Originally, secrets were not user-friendly when it came to storing plain-text data. For this reason, config maps were then introduced. Over time, both the secrets and config maps evolved to support both types of values. The functions provided by these two types of object converged. If they were added now, they would certainly be introduced as a single object type. However, because they each evolved gradually, there are some differences between them.

Differences in fields between config maps and secrets

The structure of a secret is slightly different from that of a config map. The following table shows the fields in each of the two object types.

Table 9.3 Differences in the structure of secrets and config maps

Secret	ConfigMap	Description
data	binaryData	A map of key-value pairs. The values are Base64-encoded strings.
stringData	data	A map of key-value pairs. The values are plain text strings. The stringData field in secrets is write-only.
immutable	immutable	A boolean value indicating whether the data stored in the object can be updated or not.

type	N/A	A string indicating the type of secret. Can be any string value, but several built-in types have special requirements.
------	-----	--

As you can see in the table, the data field in secrets corresponds to the `binaryData` field in config maps. It can contain binary values as Base64-encoded strings. The `stringData` field in secrets is equivalent to the data field in config maps and is used to store plain text values. This `stringData` field in secrets is write-only. You can use it to add plaintext values to the secret without having to encode them manually. When you read back the `Secret` object, any values you added to `stringData` will be included in the data field as Base64-encoded strings.

This is different from the behavior of the `data` and `binaryData` fields in config maps. Whatever key-value pair you add to one of these fields will remain in that field when you read the `ConfigMap` object back from the API.

Like config maps, secrets can be marked immutable by setting the `immutable` field to `true`.

Secrets have a field that config maps do not. The `type` field specifies the type of the secret and is mainly used for programmatic handling of the secret. You can set the type to any value you want, but there are several built-in types with specific semantics.

Understanding built-in secret types

When you create a secret and set its type to one of the built-in types, it must meet the requirements defined for that type, because they are used by various Kubernetes components that expect them to contain values in specific formats under specific keys. The following table explains the built-in secret types that exist at the time of writing this.

Table 9.4 Types of secrets

Built-in secret type	Description
Opaque	This type of secret can contain secret data stored under arbitrary keys. If you create a secret with no type field, an Opaque secret is created.
<code>bootstrap.kubernetes.io/token</code>	This type of secret is used for tokens that are used when bootstrapping new cluster nodes.
<code>kubernetes.io/basic-auth</code>	This type of secret stores the credentials required for basic authentication. It must contain the username and password keys.
<code>kubernetes.io/dockercfg</code>	This type of secret stores the credentials required for accessing a Docker image registry. It must contain a key called <code>.dockercfg</code> , where the value is the contents of the <code>~/.dockercfg</code> configuration file used by legacy versions of Docker.
	Like above, this type of secret stores the credentials for accessing a Docker registry, but uses the newer Docker configuration file format. The

kubernetes.io/dockerconfigjson	secret must contain a key called .dockerconfigjson. The value must be the contents of the ~/.docker/config.json file used by Docker.
kubernetes.io/service-account-token	This type of secret stores a token that identifies a Kubernetes service account. You'll learn about service accounts and this token in chapter 23.
kubernetes.io/ssh-auth	This type of secret stores the private key used for SSH authentication. The private key must be stored under the key ssh-privatekey in the secret.
kubernetes.io/tls	This type of secrets stores a TLS certificate and the associated private key. They must be stored in the secret under the key tls.crt and tls.key, respectively.

Understanding how Kubernetes stores secrets and config maps

In addition to the small differences in the names of the fields supported by config maps or secrets, Kubernetes treats them differently. When it comes to secrets, you need to remember that they are handled in specific ways in all Kubernetes components to ensure their security. For example, Kubernetes ensures that the data in a secret is distributed only to the node that runs the

pod that needs the secret. Also, secrets on the worker nodes themselves are always stored in memory and never written to physical storage. This makes it less likely that sensitive data will leak out.

For this reason, it's important that you store sensitive data only in secrets and not config maps.

9.3.2 Creating a secret

In section 9.2, you used a config map to inject the configuration file into the Envoy sidecar container. In addition to the file, Envoy also requires a TLS certificate and private key. Because the key represents sensitive data, it should be stored in a secret.

In this section, you'll create a secret to store the certificate and key, and project it into the container's filesystem. With the config, certificate and key files all sourced from outside the container image, you can replace the custom `kiada-ssl-proxy` image with the generic `envoyproxy/envoy` image. This is a considerable improvement, as removing custom images from the system is always a good thing, since you no longer need to maintain them.

First, you'll create the secret. The files for the certificate and private key are provided in the book's code repository, but you can also create them yourself.

Creating a TLS secret

Like for config maps, `kubectl` also provides a command for creating different types of secrets. Since you are creating a secret that will be used by your own application rather than Kubernetes, it doesn't matter whether the secret you create is of type `Opaque` or `kubernetes.io/tls`, as described in table 9.4. However, since you are creating a secret with a TLS certificate and a private key, you should use the built-in secret type `kubernetes.io/tls` to standardize things.

To create the secret, run the following command:

```
$ kubectl create secret tls kiada-tls \           #A
  --cert example-com.crt \                       #B
```

```
--key example-com.key #C
```

This command instructs `kubectl` to create a `tls` secret named `kiada-tls`. The certificate and private key are read from the file `example-com.crt` and `example-com.key`, respectively.

Creating a generic (opaque) secret

Alternatively, you could use `kubectl` to create a generic secret. The items in the resulting secret would be the same, the only difference would be its type. Here's the command to create the secret:

```
$ kubectl create secret generic kiada-tls \    #A
    --from-file tls.crt=example-com.crt \    #B
    --from-file tls.key=example-com.key      #C
```

In this case, `kubectl` creates a generic secret. The contents of the `example-com.crt` file are stored under the `tls.crt` key, while the contents of the `example-com.key` file are stored under `tls.key`.

Note

Like config maps, the maximum size of a secret is approximately 1MB.

Creating secrets from YAML manifests

The `kubectl create secret` command creates the secret directly in the cluster. Previously, you learned how to create a YAML manifest for a config map. What about secrets?

For obvious reasons, it's not the best idea to create YAML manifests for your secrets and store them in your version control system, as you do with config maps. However, if you need to create a YAML manifest instead of creating the secret directly, you can again use the `kubectl create --dry-run=client -o yaml` trick.

Suppose you want to create a secret YAML manifest containing user credentials under the keys `user` and `pass`. You can use the following

command to create the YAML manifest:

```
$ kubectl create secret generic my-credentials \      #A
  --from-literal user=my-username \                #B
  --from-literal pass=my-password \                 #B
  --dry-run=client -o yaml                          #C
apiVersion: v1
data:
  pass: bXktdGFzc3dvcmQ=                           #D
  user: bXktdXNlcm5hbWU=                           #D
kind: Secret
metadata:
  creationTimestamp: null
  name: my-credentials
```

Creating the manifest using the `kubectl create` trick as shown here is much easier than creating it from scratch and manually entering the Base64-encoded credentials. Alternatively, you could avoid encoding the entries by using the `stringData` field as explained next.

Using the `stringData` field

Since not all sensitive data is in binary form, Kubernetes also allows you to specify plain text values in secrets by using `stringData` instead of the `data` field. The following listing shows how you'd create the same secret that you created in the previous example.

Listing 9.16 Adding plain text entries to a secret using the `stringData` field

```
apiVersion: v1
kind: Secret
stringData:
  user: my-username      #A
  pass: my-password      #B
```

The `stringData` field is write-only and can only be used to set values. If you create this secret and read it back with `kubectl get -o yaml`, the `stringData` field is no longer present. Instead, any entries you specified in it will be displayed in the `data` field as Base64-encoded values.

Tip

Since entries in a secret are always represented as Base64-encoded values, working with secrets (especially reading them) is not as human-friendly as working with config maps, so use config maps wherever possible. But never sacrifice security for the sake of comfort.

Okay, let's return to the TLS secret you created earlier. Let's use it in a pod.

9.3.3 Using secrets in containers

As explained earlier, you can use secrets in containers the same way you use config maps - you can use them to set environment variables or create files in the container's filesystem. Let's look at the latter first.

Using a secret volume to project secret entries into files

In one of the previous sections, you created a secret called `kiada-tls`. Now you will project the two entries it contains into files using a secret volume. A secret volume is analogous to the `configMap` volume used before, but points to a secret instead of a config map.

To project the TLS certificate and private key into the `envoy` container of the `kiada-ssl` pod, you need to define a new volume and a new `volumeMount`, as shown in the next listing, which contains the important parts of the `pod.kiada-ssl.secret-volume.yaml` file.

Listing 9.17 Using a secret volume in a pod

```
apiVersion: v1
kind: Pod
metadata:
  name: kiada-ssl
spec:
  volumes:
    - name: cert-and-key    #A
      secret:               #A
        secretName: kiada-tls  #A
        items:               #B
          - key: tls.crt      #B
            path: example-com.crt  #B
          - key: tls.key      #B
```

```

        path: example-com.key    #B
        mode: 0600    #C
    ...
containers:
- name: kiada
    ...
- name: envoy
  image: envoyproxy/envoy:v1.14.1
  volumeMounts:    #D
  - name: cert-and-key    #D
    mountPath: /etc/certs    #D
    readOnly: true    #D
    ...
  ports:
    ...

```

If you’ve read section 9.2 on config maps, the definitions of the volume and volumeMount in this listing should be straightforward since they contain the same fields as you’d find if you were using a config map. The only two differences are that the volume type is secret instead of configMap, and that the name of the referenced secret is specified in the secretName field, whereas in a configMap volume definition the config map is specified in the name field.

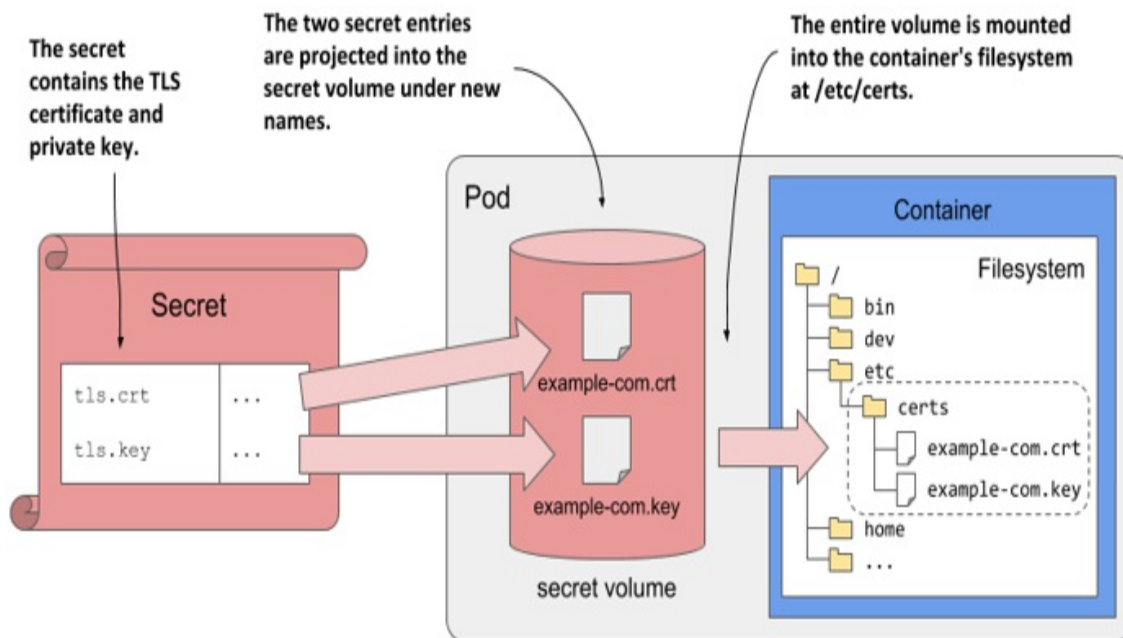
Note

As with configMap volumes, you can set the file permissions on secret volumes with the defaultMode and mode fields. Also, you can set the optional field to true if you want the pod to start even if the referenced secret doesn’t exist. If you omit the field, the pod won’t start until you create the secret.

Given the sensitive nature of the example-com.key file, the mode field is used to set the file permissions to 0600 or rw- - - - - . The file example-com.crt is given the default permissions.

To illustrate the pod, its secret volume and the referenced secret and its entries, see the following figure.

Figure 9.6 Projecting a secret’s entries into the container’s filesystem via a secret volume



Reading the files in the secret volume

After you deploy the pod from the previous listing, you can use the following command to inspect the certificate file in the secret volume:

```
$ kubectl exec kiada-ssl -c envoy -- cat /etc/certs/example-com.c
-----BEGIN CERTIFICATE-----
MIIFkzCCA3ugAwIBAgIUQhQiuFP7vEp1CBG167ICGxg4q0EwDQYJKoZIhvcNAQEL
BQAwWDElMAkGA1UEBhMCWFgxFtATBgNVBACMDERlZmF1bHQgQ210eTEcMBoGA1UE
...
```

As you can see, when you project the entries of a secret into a container via a secret volume, the projected file is not Base64-encoded. The application doesn't need to decode the file. The same is true if the secret entries are injected into environment variables.

Note

The files in a secret volume are stored in an in-memory filesystem (tmpfs), so they are less likely to be compromised.

Injecting secrets into environment variables

As with config maps, you can also inject secrets into the container's environment variables. For example, you can inject the TLS certificate into the `TLS_CERT` environment variable as if the certificate were stored in a config map. The following listing shows how you'd do this.

Listing 9.18 Exposing a secret's key-value pair as an environment variable

```
containers:
  - name: my-container
    env:
      - name: TLS_CERT
        valueFrom:           #A
          secretKeyRef:      #A
            name: kiada-tls  #B
            key: tls.crt     #C
```

This is not unlike setting the `INITIAL_STATUS_MESSAGE` environment variable, except that you're now referring to a secret by using `secretKeyRef` instead of `configMapKeyRef`.

Instead of using `env.valueFrom`, you could also use `envFrom` to inject the entire secret instead of injecting its entries individually, as you did in section 9.2.3. Instead of `configMapRef`, you'd use the `secretRef` field.

Should you inject secrets into environment variables?

As you can see, injecting secrets into environment variables is no different from injecting config maps. But even if Kubernetes allows you to expose secrets in this way, it may not be the best idea, as it can pose a security risk. Applications typically output environment variables in error reports or even write them to the application log at startup, which can inadvertently expose secrets if you inject them into environment variables. Also, child processes inherit all environment variables from the parent process. So, if your application calls a third-party child process, you don't know where your secrets end up.

Tip

Instead of injecting secrets into environment variables, project them into the

container as files in a secret volume. This reduces the likelihood that the secrets will be inadvertently exposed to attackers.

9.4 Passing pod metadata to the application via the Downward API

So far in this chapter, you’ve learned how to pass configuration data to your application. But that data was always static. The values were known before you deployed the pod, and if you deployed multiple pod instances, they would all use the same values.

But what about data that isn’t known until the pod is created and scheduled to a cluster node, such as the IP of the pod, the name of the cluster node, or even the name of the pod itself? And what about data that is already specified elsewhere in the pod manifest, such as the amount of CPU and memory that is allocated to the container? Good engineers never want to repeat themselves.

Note

You’ll learn how to specify the container’s CPU and memory limits in chapter 20.

9.4.1 Introducing the Downward API

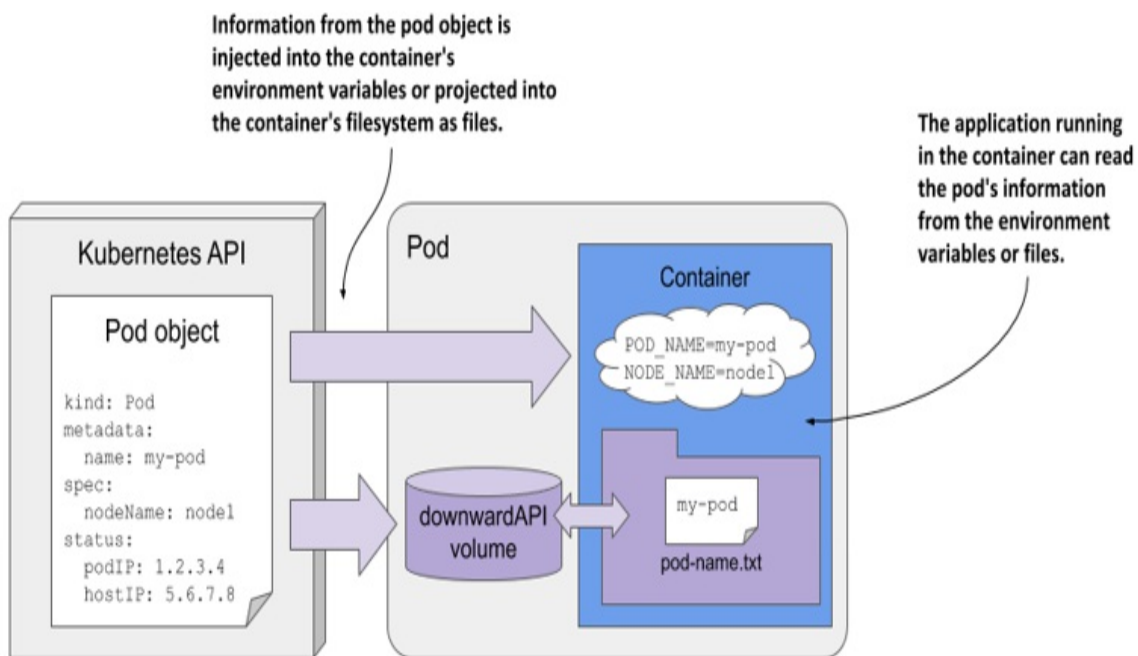
In the remaining chapters of the book, you’ll learn about many additional configuration options that you can set in your pods. There are cases where you need to pass the same information to your application. You could repeat this information when defining the container’s environment variable, but a better option is to use what’s called the Kubernetes *Downward API*, which allows you to expose pod and container metadata via environment variables or files.

Understanding what the Downward API is

The Downward API isn’t a REST endpoint that your app needs to call to get

the data. It's simply a way to inject values from the pod's metadata, spec, or status fields down into the container. Hence the name. An illustration of the Downward API is shown in the following figure.

Figure 9.7 The Downward API exposes pod metadata through environment variables or files.



As you can see, this is no different from setting environment variables or projecting files from config maps and secrets, except that the values come from the pod object itself.

Understanding how the metadata is injected

Earlier in the chapter, you learned that you initialize environment variables from external sources using the `valueFrom` field. To get the value from a config map, use the `configMapKeyRef` field, and to get it from a secret, use `secretKeyRef`. To instead use the Downward API to source the value from the pod object itself, use either the `fieldRef` or the `resourceFieldRef` field, depending on what information you want to inject. The former is used to refer to the pod's general metadata, whereas the latter is used to refer to the container's compute resource constraints.

Alternatively, you can project the pod's metadata as files into the container's filesystem by adding a downwardAPI volume to the pod, just as you'd add a configMap or secret volume. You'll learn how to do this soon, but first let's see what information you can inject.

Understanding what metadata can be injected

You can't use the Downward API to inject any field from the pod object. Only certain fields are supported. The following table shows the fields you can inject via `fieldRef`, and whether they can only be exposed via environment variables, files, or both.

Table 9.5 Downward API fields injected via the `fieldRef` field

Field	Description	Allowed in env	Allowed in volume
<code>metadata.name</code>	The pod's name.	Yes	Yes
<code>metadata.namespace</code>	The pod's namespace.	Yes	Yes
<code>metadata.uid</code>	The pod's UID.	Yes	Yes
<code>metadata.labels</code>	All the pod's labels, one label per line, formatted as <code>key="value"</code> .	No	Yes
<code>metadata.labels['key']</code>	The value of the specified label.	Yes	Yes

<code>metadata.annotations</code>	All the pod's annotations, one per line, formatted as <code>key="value"</code> .	No	Yes
<code>metadata.annotations['key']</code>	The value of the specified annotation.	Yes	Yes
<code>spec.nodeName</code>	The name of the worker node the pod runs on.	Yes	No
<code>spec.serviceAccountName</code>	The name of the pod's service account.	Yes	No
<code>status.podIP</code>	The pod's IP address.	Yes	No
<code>status.hostIP</code>	The worker node's IP address.	Yes	No

You may not know most of these fields yet, but you will in the remaining chapters of this book. As you can see, some fields can only be injected into environment variables, whereas others can only be projected into files. Some allow doing both.

Information about the container's computational resource constraints is injected via the `resourceFieldRef` field. They can all be injected into environment variables and via a downwardAPI volume. The following table lists them.

Table 9.6 Downward API resource fields injected via the `resourceFieldRef` field

Resource field	Description	Allowed in env	Allowed in vol
<code>requests.cpu</code>	The container's CPU request.	Yes	Yes
<code>requests.memory</code>	The container's memory request.	Yes	Yes
<code>requests.ephemeral-storage</code>	The container's ephemeral storage request.	Yes	Yes
<code>limits.cpu</code>	The container's CPU limit.	Yes	Yes
<code>limits.memory</code>	The container's memory limit.	Yes	Yes
<code>limits.ephemeral-storage</code>	The container's ephemeral storage limit.	Yes	Yes

You'll learn what resource requests and limits are in chapter 20, which explains how to constrain the compute resources available to a container.

The book's code repository contains the file `pod.downward-api-test.yaml`, which defines a pod that uses the Downward API to inject each supported field into both environment variables and files. You can deploy the pod and then look in its container log to see what was injected.

A practical example of using the Downward API in the Kiada application is presented next.

9.4.2 Injecting pod metadata into environment variables

At the beginning of this chapter, a new version of the Kiada application was introduced. The application now includes the pod and node names and their IP addresses in the HTTP response. You'll make this information available to the application through the Downward API.

Injecting pod object fields

The application expects the pod's name and IP, as well as the node name and IP, to be passed in via the environment variables `POD_NAME`, `POD_IP`, `NODE_NAME`, and `NODE_IP`, respectively. You can find a pod manifest that uses the Downward API to provide these variables to the container in the `pod.kiada-ssl.downward-api.yaml` file. The contents of this file are shown in the following listing.

Listing 9.19 Using the Downward API to set environment variables

```
apiVersion: v1
kind: Pod
metadata:
  name: kiada-ssl
spec:
  ...
  containers:
  - name: kiada
    image: luksa/kiada:0.4
    env:
      #A
      - name: POD_NAME                #B
        valueFrom:                    #B
          fieldRef:                    #B
            fieldPath: metadata.name  #B
      - name: POD_IP                  #C
        valueFrom:                    #C
          fieldRef:                    #C
            fieldPath: status.podIP   #C
      - name: NODE_NAME               #D
        valueFrom:                    #D
```

```

        fieldRef:                #D
          fieldPath: spec.nodeName #D
-   name: NODE_IP                #E
    valueFrom:                   #E
      fieldRef:                  #E
        fieldPath: status.hostIP #E
ports:
...

```

After you create this pod, you can examine its log using `kubectl logs`. The application prints the values of the three environment variables at startup. You can also send a request to the application and you should get a response like the following:

```
Request processed by Kiada 0.4 running in pod "kiada-ssl" on node
Pod hostname: kiada-ssl; Pod IP: 10.244.2.15; Node IP: 172.18.0.4
```

Compare the values in the response with the field values in the YAML definition of the Pod object by running the command `kubectl get po kiada-ssl -o yaml`. Alternatively, you can compare them with the output of the following commands:

```
$ kubectl get po kiada-ssl -o wide
NAME      READY   STATUS    RESTARTS   AGE      IP             NODE
kiada     1/1     Running   0           7m41s    10.244.2.15    kind-w
```

```
$ kubectl get node kind-worker -o wide
NAME           STATUS    ROLES    AGE   VERSION   INTERNAL-IP   ...
kind-worker    Ready     <none>    26h   v1.19.1   172.18.0.4    ...
```

You can also inspect the container's environment by running `kubectl exec kiada-ssl -- env`.

Injecting container resource fields

Even if you haven't yet learned how to constrain the compute resources available to a container, let's take a quick look at how to pass those constraints to the application when it needs them.

Chapter 20 explains how to set the number of CPU cores and the amount of memory a container may consume. These settings are called CPU and

memory resource *limits*. Kubernetes ensures that the container can't use more than the allocated amount.

Some applications need to know how much CPU time and memory they have been given to run optimally within the given constraints. That's another thing the Downward API is for. The following listing shows how to expose the CPU and memory limits in environment variables.

Listing 9.20 Using the Downward API to inject the container's compute resource limits

```
env:
  - name: MAX_CPU_CORES          #A
    valueFrom:                   #A
      resourceFieldRef:          #A
        resource: limits.cpu     #A
  - name: MAX_MEMORY_KB          #B
    valueFrom:                   #B
      resourceFieldRef:          #B
        resource: limits.memory #B
        divisor: 1k              #B
```

To inject container resource fields, the field `resourceFieldRef` is used. The resource field specifies the resource value that is injected.

Each `resourceFieldRef` can also specify a divisor. It specifies which unit to use for the value. In the listing, the divisor is set to `1k`. This means that the memory limit value is divided by 1000 and the result is then stored in the environment variable. So, the memory limit value in the environment variable will use kilobytes as the unit. If you don't specify a divisor, as is the case in the `MAX_CPU_CORES` variable definition in the listing, the value defaults to 1.

The divisor for memory limits/requests can be `1` (byte), `1k` (kilobyte) or `1Ki` (kibibyte), `1M` (megabyte) or `1Mi` (mebibyte), and so on. The default divisor for CPU is `1`, which is a whole core, but you can also set it to `1m`, which is a milli core or a thousandth of a core.

Because environment variables are defined within a container definition, the resource constraints of the enclosing container are used by default. In cases where a container needs to know the resource limits of another container in the pod, you can specify the name of the other container using the

containerName field within resourceFieldRef.

9.4.3 Using a downwardAPI volume to expose pod metadata as files

As with config maps and secrets, pod metadata can also be projected as files into the container's filesystem using the downwardAPI volume type.

Suppose you want to expose the name of the pod in the /pod-metadata/pod-name file inside the container. The following listing shows the volume and volumeMount definitions you'd add to the pod.

Listing 9.21 Injecting pod metadata into the container's filesystem

```
...
  volumes:                                     #A
  - name: pod-meta                             #A
    downwardAPI:                               #A
      items:                                   #B
      - path: pod-name.txt                     #B
        fieldRef:                             #B
          fieldPath: metadata.name             #B
  containers:
  - name: foo
    ...
    volumeMounts:                             #C
    - name: pod-meta                           #C
      mountPath: /pod-metadata                 #C
```

The pod manifest in the listing contains a single volume of type downwardAPI. The volume definition contains a single file named pod-name.txt, which contains the name of the pod read from the metadata.name field of the pod object. This volume is mounted in the container's filesystem at /pod-metadata.

As with environment variables, each item in a downwardAPI volume definition uses either fieldRef to refer to the pod object's fields, or resourceFieldRef to refer to the container's resource fields. For resource fields, the containerName field must be specified because volumes are defined at the pod level and it isn't obvious which container's resources are

being referenced. As with environment variables, a divisor can be specified to convert the value into the expected unit.

As with `configMap` and `secret` volumes, you can set the default file permissions using the `defaultMode` field or per-file using the `mode` field, as explained earlier.

9.5 Using projected volumes to combine volumes into one

In this chapter, you learned how to use three special volume types to inject values from config maps, secrets, and the Pod object itself. Unless you use the `subPath` field in your `volumeMount` definition, you can't inject the files from these different sources, or even multiple sources of the same type, into the same file directory. For example, you can't combine the keys from different secrets into a single volume and mount them into a single file directory. While the `subPath` field allows you to inject individual files from multiple volumes, it isn't the final solution because it prevents the files from being updated when the source values change.

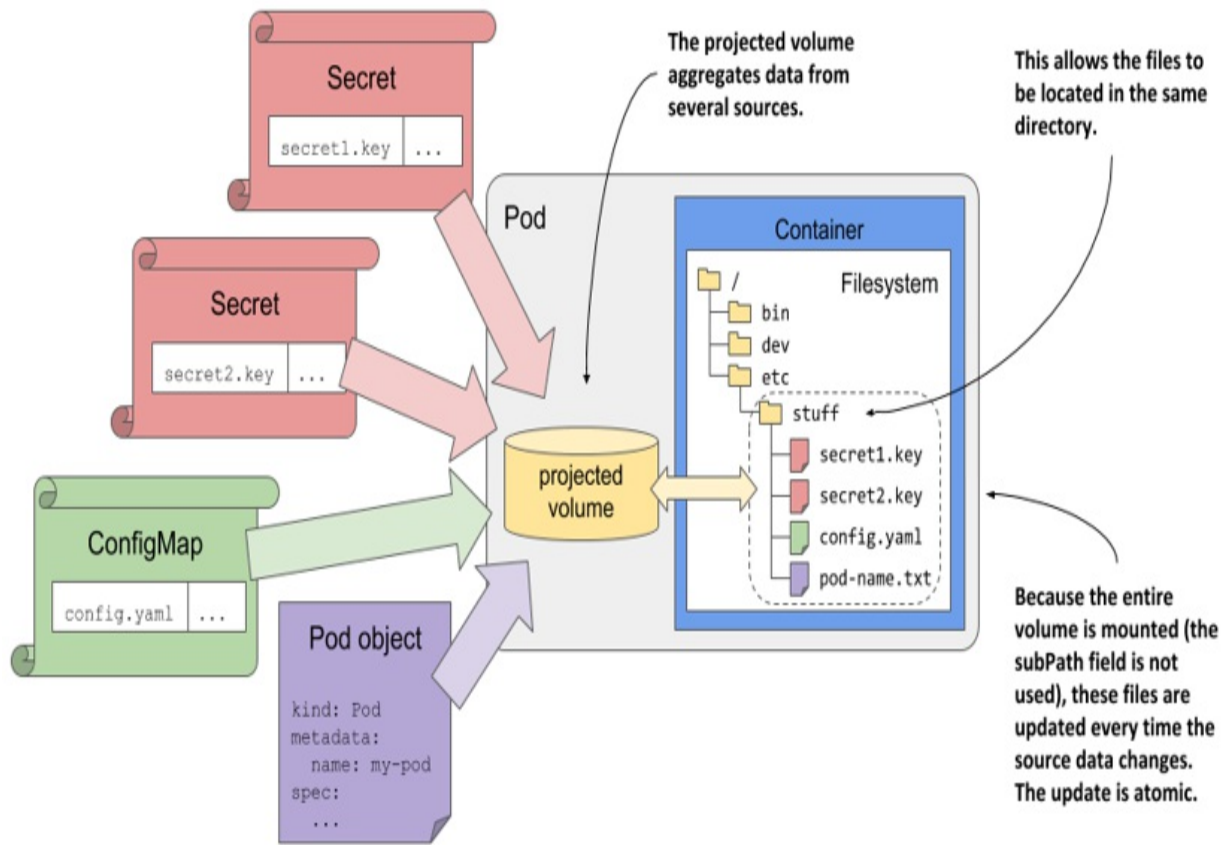
If you need to populate a single volume from multiple sources, you can use the projected volume type.

9.5.1 Introducing the projected volume type

Projected volumes allow you to combine information from multiple config maps, secrets, and the Downward API into a single pod volume that you can then mount in the pod's containers. They behave exactly like the `configMap`, `secret`, and `downwardAPI` volumes you learned about in the previous sections of this chapter. They provide the same features and are configured in almost the same way as the other volume types.

The following figure shows a visualization of a projected volume.

Figure 9.8 Using a projected volume with several sources



In addition to the three volume types described earlier, you can also use projected volumes to expose the service account token to your application. You'll learn what those are in chapter 23.

9.5.2 Using a projected volume in a pod

In the final exercise of this chapter, you'll modify the `kiada-ss1` pod to use a single projected volume in the `envoy` container. The previous version of the pod used a `configMap` volume mounted in `/etc/envoy` to inject the `envoy.yaml` config file and a `secret` volume mounted in `/etc/certs` to inject the TLS certificate and key files. You'll now replace these two volumes with a single projected volume. This will allow you to keep all three files in the same directory (`/etc/envoy`).

First, you need to change the TLS certificate paths in the `envoy.yaml` configuration file inside the `kiada-envoy-config` config map so that the certificate and key are read from the same directory. After editing, the lines in

the config map should look like this:

```
tls_certificates:
  - certificate_chain:
      filename: "/etc/envoy/example-com.crt"    #A
    private_key:
      filename: "/etc/envoy/example-com.key"    #B
```

You can find the pod manifest with the projected volume in the file `pod.kiada-ssl.projected-volume.yaml`. The relevant parts are shown in the next listing.

Listing 9.22 Using a projected volume instead of a configMap and secret volume

```
apiVersion: v1
kind: Pod
metadata:
  name: kiada-ssl
spec:
  volumes:
    - name: etc-envoy    #A
      projected:    #A
        sources:    #A
          - configMap:    #B
              name: kiada-envoy-config    #B
          - secret:    #C
              name: kiada-tls    #C
              items:    #C
                - key: tls.crt    #C
                  path: example-com.crt    #C
                - key: tls.key    #C
                  path: example-com.key    #C
                  mode: 0600    #D
  containers:
    - name: kiada
      image: luksa/kiada:1.2
      env:
        ...
    - name: envoy
      image: envoyproxy/envoy:v1.14.1
      volumeMounts:    #E
        - name: etc-envoy    #E
          mountPath: /etc/envoy    #E
          readOnly: true    #E
      ports:
```

...

The listing shows that a single projected volume named `etc-envoy` is defined in the pod. Two sources are used for this volume. The first is the `kiada-envoy-config` config map. All entries in this config map become files in the projected volume. The second source is the `kiada-tls` secret. Two of its entries become files in the projected volume - the value of the `tls.crt` key becomes file `example-com.crt`, whereas the value of the `tls.key` key becomes file `example-com.key` in the volume. The volume is mounted in read-only mode in the envoy container at `/etc/envoy`.

As you can see, the source definitions in the projected volume are not much different from the `configMap` and `secret` volumes you created in the previous sections. Therefore, further explanation of the projected volumes is unnecessary. Everything you learned about the other volumes also applies to this new volume type.

9.6 Summary

This wraps up this chapter on how to pass configuration data to containers. You've learned that:

- The default command and arguments defined in the container image can be overridden in the pod manifest.
- Environment variables for each container can also be set in the pod manifest. Their values can be hardcoded in the manifest or can come from other Kubernetes API objects.
- Config maps are Kubernetes API objects used to store configuration data in the form of key/value pairs. Secrets are another similar type of object used to store sensitive data such as credentials, certificates, and authentication keys.
- Entries of both config maps and secrets can be exposed within a container as environment variables or as files via the `configMap` and `secret` volumes.
- Config maps and other API objects can be edited in place using the `kubectl edit` command.
- The Downward API provides a way to expose the pod metadata to the

application running within it. Like config maps and secrets, this data can be injected into environment variables or files.

- Projected volumes can be used to combine multiple volumes of possibly different types into a composite volume that is mounted into a single directory, rather than being forced to mount each individual volume into its own directory.

You've now seen that an application deployed in Kubernetes may require many additional objects. If you are deploying many applications in the same cluster, you need organize them so that everyone can see what fits where. In the next chapter, you'll learn how to do just that.

10 Organizing objects using Namespaces and Labels

This chapter covers

- Using namespaces to split a physical cluster into virtual clusters
- Organizing objects using labels
- Using label selectors to perform operations on subsets of objects
- Using label selectors to schedule pods onto specific nodes
- Using field selectors to filter objects based on their properties
- Annotating objects with additional non-identifying information

A Kubernetes cluster is usually used by many teams. How should these teams deploy objects to the same cluster and organize them so that one team doesn't accidentally modify the objects created by other teams?

And how can a large team deploying hundreds of microservices organize them so that each team member, even if new to the team, can quickly see where each object belongs and what its role in the system is? For example, to which application does a config map or a secret belong.

These are two different problems. Kubernetes solves the first with object namespaces, and the other with object labels. In this chapter, you will learn about both.

NOTE

You'll find the code files for this chapter at <https://github.com/luksa/kubernetes-in-action-2nd-edition/tree/master/Chapter10>.

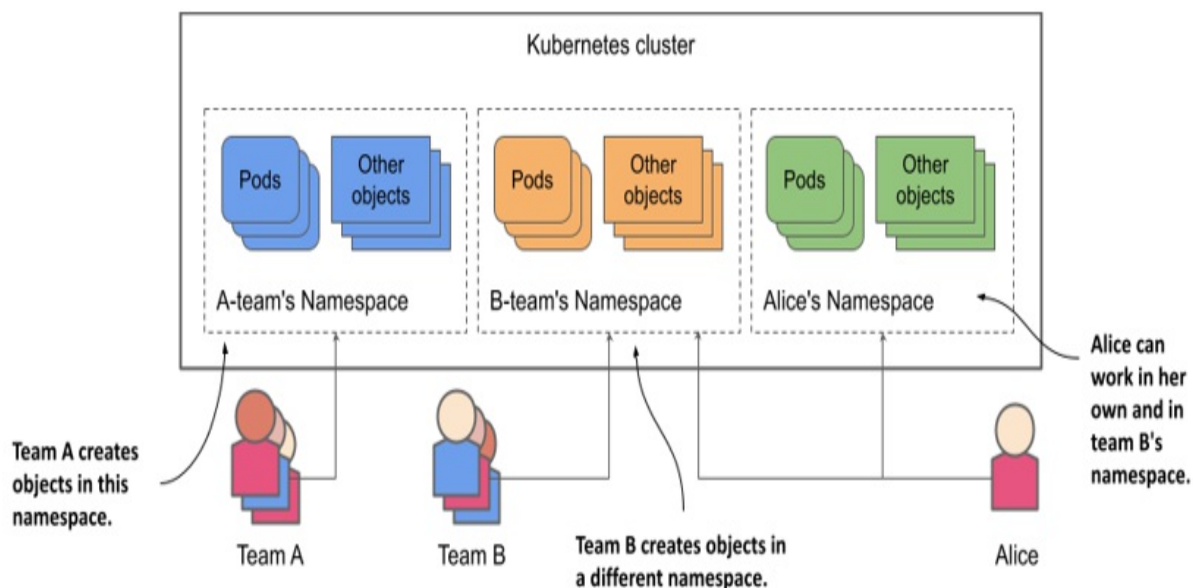
10.1 Organizing objects into Namespaces

Imagine that your organization is running a single Kubernetes cluster that's used by multiple engineering teams. Each of these teams deploys the entire Kiada application suite to develop and test it. You want each team to only deal with their own instance of the application suite - each team only wants to see the objects they've created and not those created by the other teams. This is achieved by creating objects in separate Kubernetes namespaces.

Note

Namespaces in Kubernetes help organize Kubernetes API objects into non-overlapping groups. They have nothing to do with Linux namespaces, which help isolate processes running in one container from those in another, as you learned in chapter 2.

Figure 10.1 Splitting a physical cluster into several virtual clusters by utilizing Kubernetes Namespaces



As shown in the previous figure, you can use namespaces to divide a single physical Kubernetes cluster into many virtual clusters. Instead of everyone creating their objects in a single location, each team gets access to one or more namespaces in which to create their objects. Because namespaces provide a scope for object names, different teams can use the same names for their objects when they create them in their respective namespaces. Some

namespaces can be shared between different teams or individual users.

Understanding when to organize objects into namespaces

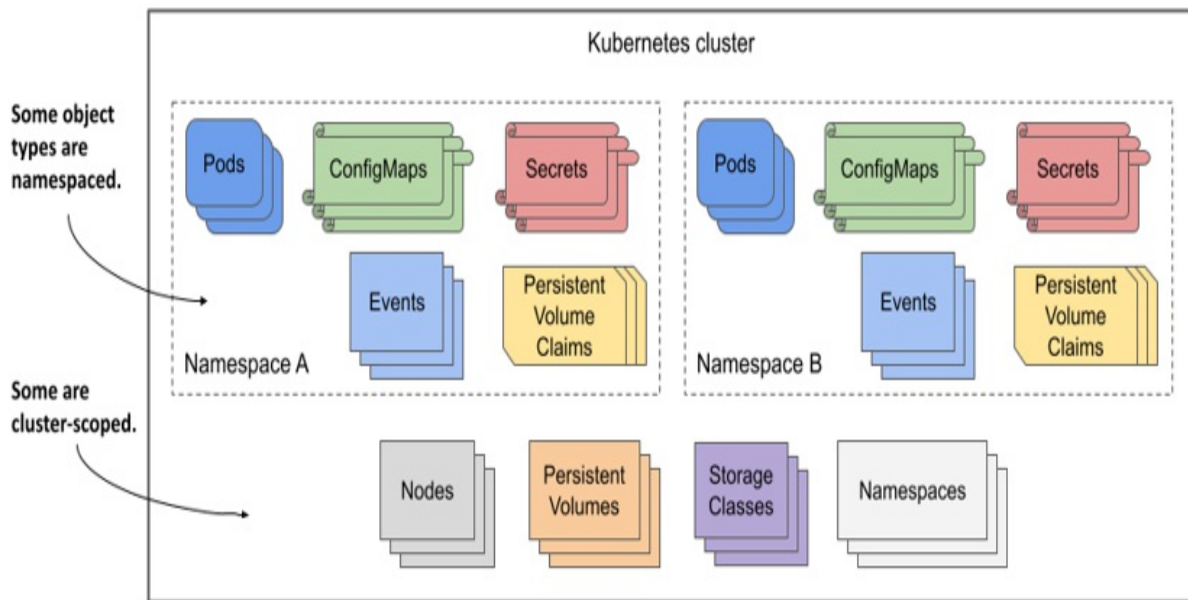
Using multiple namespaces allows you to divide complex systems with numerous components into smaller groups that are managed by different teams. They can also be used to separate objects in a multitenant environment. For example, you can create a separate namespace (or a group of namespaces) for each client and deploy the entire application suite for that client in that namespace (or group).

Note

Most Kubernetes API object types are namespaced, but a few are not. Pods, ConfigMaps, Secrets, PersistentVolumeClaims, and Events are all namespaced. Nodes, PersistentVolumes, StorageClasses, and Namespaces themselves are not. To see if a resource is namespaced or cluster-scoped, check the NAMESPACE column when running `kubectl api-resources`.

Without namespaces, each user of the cluster would have to prefix their object names with a unique prefix or each user would have to use their own Kubernetes cluster.

Figure 10.2 Some Kubernetes API types are namespaced, whereas others are cluster-scoped.



As you'll learn in chapter 23, namespaces also provide a scope for user privileges. A user may have permission to manage objects in one namespace but not in others. Similarly, resource quotas, which are also tied to namespaces, are explained in chapter 20.

10.1.1 Listing namespaces and the objects they contain

Every Kubernetes cluster you create contains a few common namespaces. Let's see what they are.

Listing namespaces

Since each namespace is represented by the *Namespace* object, you can display them with the `kubectl get` command, as you would any other Kubernetes API object. To see the namespaces in your cluster, run the following command:

```
$ kubectl get namespaces
NAME                STATUS    AGE
default             Active   1h
kube-node-lease     Active   1h
kube-public         Active   1h
```

kube-system	Active	1h
local-path-storage	Active	1h

Note

The short form for namespace is `ns`. You can also list namespaces with `kubect1 get ns`.

Up to this point, you've been working in the default namespace. Every time you created an object, it was created in that namespace. Similarly, when you listed objects, such as pods, with the `kubect1 get` command, the command only displayed the objects in that namespace. You may be wondering if there are pods in the other namespaces. Let's take a look.

Note

Namespaces prefixed with `kube-` are reserved for Kubernetes system namespaces.

Listing objects in a specific namespace

To list the pods in the `kube-system` namespace, run `kubect1 get` with the `--namespace` option as follows:

```
$ kubect1 get pods --namespace kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
coredns-558bd4d5db-4n5zg	1/1	Running	0	1h
coredns-558bd4d5db-tnfws	1/1	Running	0	1h
etcd-kind-control-plane	1/1	Running	0	1h
kindnet-54ks9	1/1	Running	0	1h
...				

Tip

You can also use `-n` instead of `--namespace`.

You'll learn more about these pods later in this book. Don't worry if the pods shown here don't exactly match the ones in your cluster. As the namespace name implies, these are the Kubernetes system pods. By having them in this

separate namespace, everything stays neatly nice and clear. If they were all in the default namespace, mixed in with the pods you create yourself, it would be hard to tell what belongs where, and you could accidentally delete system objects.

Listing objects across all namespaces

Instead of listing objects in each namespace individually, you can also tell `kubectl` to list objects in all namespaces. This time, instead of listing pods, let's list all config maps in the cluster:

```
$ kubectl get cm --all-namespaces
```

NAMESPACE	NAME	DATA
default	kiada-envoy-config	2
default	kube-root-ca.crt	1
kube-node-lease	kube-root-ca.crt	1
kube-public	cluster-info	1
kube-public	kube-root-ca.crt	1
...		

Tip

You can also type `-A` instead of `--all-namespaces`.

The `--all-namespaces` option is handy when you want to see all objects in the cluster, regardless of namespace, or when you can't remember which namespace an object is in.

10.1.2 Creating namespaces

Now that you know the other namespaces in your cluster, you'll create two new namespaces.

Creating a namespace with `kubectl create namespace`

The fastest way to create a namespace is to use the `kubectl create namespace` command. Create a namespace named `kiada-test` as follows:

```
$ kubectl create namespace kiada-test1
```

```
namespace/kiada-test1 created
```

Note

The names of most objects must conform to the naming conventions for DNS subdomain names, as specified in RFC 1123, that is, they may contain only lowercase alphanumeric characters, hyphens, and dots, and must start and end with an alphanumeric character. The same applies to namespaces, but they may not contain dots.

You’ve just created the namespace `kiada-test1`. You’ll now create another one using a different method.

Creating a namespace from a manifest file

As mentioned earlier, Kubernetes namespaces are represented by `Namespace` objects. As such, you can list them with the `kubectl get` command, as you’ve already done, but you can also create them from a YAML or JSON manifest file that you post to the Kubernetes API.

Use this method to create another namespace called `kiada-test2`. First, create a file named `ns.kiada-test.yaml` with the contents of the following listing.

Listing 10.1 A YAML definition of a `Namespace` object

```
apiVersion: v1
kind: Namespace    #A
metadata:
  name: kiada-test2    #B
```

Now, use `kubectl apply` to post the file to the Kubernetes API:

```
$ kubectl apply -f ns.kiada-test.yaml
namespace/kiada-test2 created
```

Developers don’t usually create namespaces this way, but operators do. For example, if you want to create a set of manifest files for a suite of applications will be distributed across multiple namespaces, you can add the

necessary Namespace objects to those manifests so that everything can be deployed without having to first create the namespaces with `kubectl create` and then apply the manifests.

Before you continue, you should run `kubectl get ns` to list all namespaces again to see that your cluster now contains the two namespaces you created.

10.1.3 Managing objects in other namespaces

You've now created two new namespaces: `kiada-test1` and `kiada-test2`, but as mentioned earlier, you're still in the default namespace. If you create an object such as a pod without explicitly specifying the namespace, the object is created in the default namespace.

Creating objects in a specific namespace

In section 10.1.1, you learned that you can specify the `--namespace` argument (or the shorter `-n` option) to list objects in a particular namespace. You can use the same argument when applying an object manifest to the API.

To create the `kiada-ssl` pod and its associated config map and secret in the `kiada-test1` namespace, run the following command:

```
$ kubectl apply -f kiada-ssl.yaml -n kiada-test1
pod/kiada-ssl created
configmap/kiada-envoy-config created
secret/kiada-tls created
```

You can now list pods, config maps and secrets in the `kiada-test1` namespace to confirm that these objects were created there and not in the default namespace:

```
$ kubectl -n kiada-test1 get pods
NAME          READY   STATUS    RESTARTS   AGE
kiada-ssl     2/2     Running   0           1m
```

Specifying the namespace in the object manifest

The object manifest can specify the namespace of the object in the namespace field in the manifest's metadata section. When you apply the manifest with the `kubectl apply` command, the object is created in the specified namespace. You don't need to specify the namespace with the `--namespace` option.

The manifest shown in the following listing contains the same three objects as before, but with the namespace specified in the manifest.

Listing 10.2 Specifying the namespace in the object manifest

```
apiVersion: v1
kind: Pod
metadata:
  name: kiada-ssl
  namespace: kiada-test2    #A
spec:
  volumes: ...
...
```

When you apply this manifest with the following command, the pod, config map, and secret are created in the `kiada-test2` namespace:

```
$ kubectl apply -f pod.kiada-ssl.kiada-test2-namespace.yaml
pod/kiada-ssl created
configmap/kiada-envoy-config created
secret/kiada-tls created
```

Notice that you didn't specify the `--namespace` option this time. If you did, the namespace would have to match the namespace specified in the object manifest, or `kubectl` would display an error like in the following example:

```
$ kubectl apply -f kiada-ssl.kiada-test2-namespace.yaml -n kiada-
the namespace from the provided object "kiada-test2" does not mat
```

Making kubectl default to a different namespace

In the previous two examples you learned how to create and manage objects in namespaces other than the namespace that `kubectl` is currently using as the default. You'll use the `--namespace` option frequently - especially when you want to quickly check what's in another namespace. However, you'll do most

of your work in the current namespace.

After you create a new namespace, you'll usually run many commands in it. To make your life easier, you can tell `kubectl` to switch to that namespace. The current namespace is a property of the current `kubectl` context, which is configured in the `kubeconfig` file.

Note

You learned about the `kubeconfig` file in chapter 3.

To switch to a different namespace, update the current context. For example, to switch to the `kiada-test1` namespace, run the following command:

```
$ kubectl config set-context --current --namespace kiada-test1
Context "kind-kind" modified.
```

Every `kubectl` command you run from now on will use the `kiada-test1` namespace. For example, you can now list the pods in this namespace by simply typing `kubectl get pods`.

TIP

To quickly switch to a different namespace, you can set up the following alias: `alias kns='kubectl config set-context --current --namespace '`. You can then switch between namespaces with `kns some-namespace`. Alternatively, you can install a `kubectl` plugin that does the same thing. You can find it at <https://github.com/ahmetb/kubectx>

There's not much more to learn about creating and managing objects in different namespaces. But before you wrap up this section, I need to explain how well Kubernetes isolates workloads running in different namespaces.

10.1.4 Understanding the (lack of) isolation between namespaces

You created several pods in different namespaces so far. You already know how to use the `--all-namespaces` option (or `-A` for short) to list pods across

all namespaces, so please do so now:

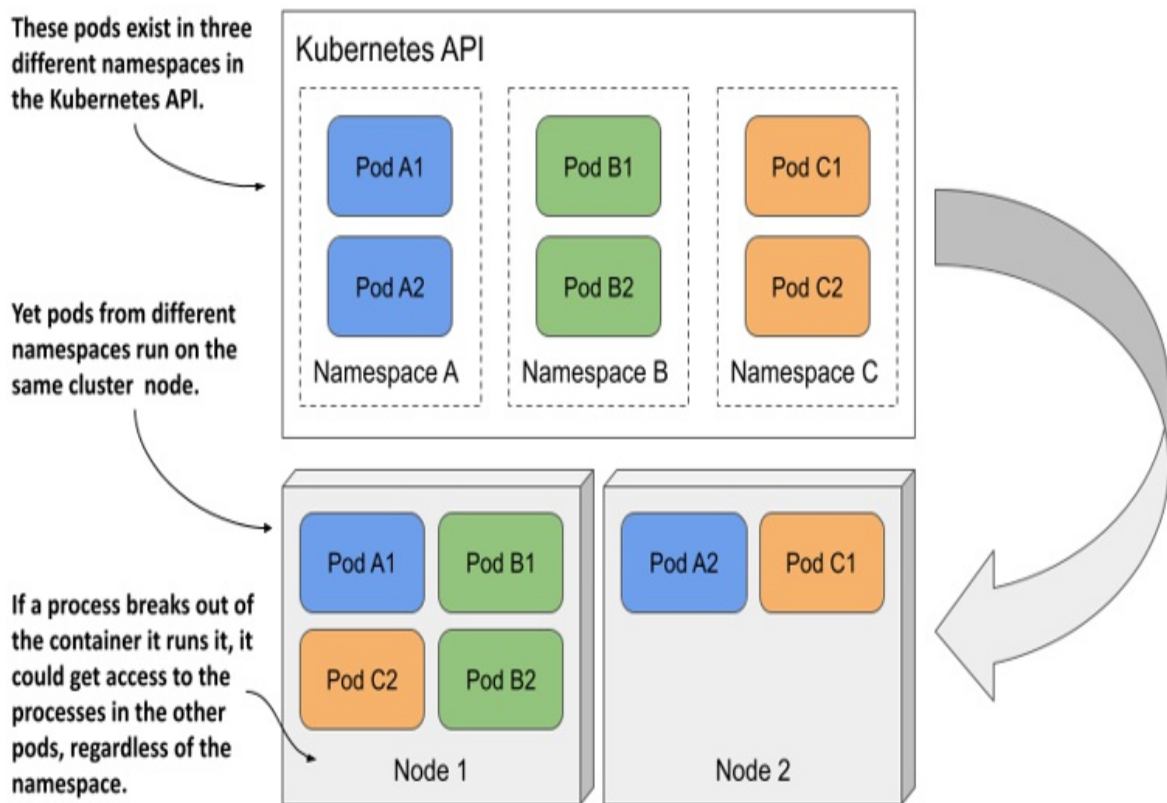
```
$ kubectl get pods -A
NAMESPACE      NAME          READY   STATUS    RESTARTS   AGE   #A
default        kiada-ssl    2/2     Running   0          8h   #A
default        quiz         2/2     Running   0          8h
default        quote       2/2     Running   0          8h
kiada-test1    kiada-ssl    2/2     Running   0          2m   #A
kiada-test2    kiada-ssl    2/2     Running   0          1m   #A
...
```

In the output of the command, you should see at least two pods named `kiada-ssl`. One in the `kiada-test1` namespace and the other in the `kiada-test2` namespace. You may also have another pod named `kiada-ssl` in the `default` namespace from the exercises in the previous chapters. In this case, there are three pods in your cluster with the same name, all of which you were able to create without issue thanks to namespaces. Other users of the same cluster could deploy many more of these pods without stepping on each other's toes.

Understanding the runtime isolation between pods in different namespaces

When users use namespaces in a single physical cluster, it's as if they each use their own virtual cluster. But this is only true up to the point of being able to create objects without running into naming conflicts. The physical cluster nodes are shared by all users in the cluster. This means that the isolation between the their pods is not the same as if they were running on different physical clusters and therefore on different physical nodes.

Figure 10.3 Pods from different namespaces may run on the same cluster node.



When two pods created in different namespaces are scheduled to the same cluster node, they both run in the same OS kernel. Although they are isolated from each other with container technologies, an application that breaks out of its container or consumes too much of the node's resources can affect the operation of the other application. Kubernetes namespaces play no role here.

Understanding network isolation between namespaces

Unless explicitly configured to do so, Kubernetes doesn't provide network isolation between applications running in pods in different namespaces. An application running in one namespace can communicate with applications running in other namespaces. By default, there is no network isolation between namespaces. However, you can use the NetworkPolicy object to configure which applications in which namespaces can connect to which applications in other namespaces. You'll learn about this in chapter 25.

Using namespaces to separate production, staging and development

environments?

Because namespaces don't provide true isolation, you should not use them to split a single physical Kubernetes cluster into the production, staging, and development environments. Hosting each environment on a separate physical cluster is a much safer approach.

10.1.5 Deleting namespaces

Let's conclude this section on namespaces by deleting the two namespaces you created. When you delete the Namespace object, all the objects you created in that namespace are automatically deleted. You don't need to delete them first.

Delete the kiada-test2 namespaces as follows:

```
$ kubectl delete ns kiada-test2
namespace "kiada-test2" deleted
```

The command blocks until everything in the namespace and the namespace itself are deleted. But, if you interrupt the command and list the namespaces before the deletion is complete, you'll see that the namespace's status is Terminating:

```
$ kubectl get ns
NAME                STATUS      AGE
default             Active      2h
kiada-test1         Active      2h
kiada-test2         Terminating 2h
...
```

The reason I show this is because you will eventually run the delete command and it will never finish. You'll probably interrupt the command and check the namespace list, as I show here. Then you'll wonder why the namespace termination doesn't complete.

Diagnosing why namespace termination is stuck

In short, the reason a namespace can't be deleted is because one or more

objects created in it can't be deleted. You may think to yourself, "Oh, I'll list the objects in the namespace with `kubectl get all` to see which object is still there," but that usually doesn't get you any further because `kubectl` doesn't return any results.

Note

Remember that the `kubectl get all` command lists only some types of objects. For example, it doesn't list secrets. Even though the command doesn't return anything, this doesn't mean that the namespace is empty.

In most, if not all, cases where I've seen a namespace get stuck this way, the problem was caused by a custom object and its custom controller not processing the object's deletion and removing a finalizer from the object. You'll learn more about finalizers in chapter 15, and about custom objects and controllers in chapter 29.

Here I just want to show you how to figure out which object is causing the namespace to be stuck. Here's a hint: Namespace objects also have a status field. While the `kubectl describe` command normally also displays the status of the object, at the time of writing this is not the case for Namespaces. I consider this to be a bug that will likely be fixed at some point. Until then, you can check the status of the namespace as follows:

```
$ kubectl get ns kiada-test2 -o yaml
...
status:
  conditions:
  - lastTransitionTime: "2021-10-10T08:35:11Z"
    message: All resources successfully discovered
    reason: ResourcesDiscovered
    status: "False"
    type: NamespaceDeletionDiscoveryFailure
  - lastTransitionTime: "2021-10-10T08:35:11Z"
    message: All legacy kube types successfully parsed
    reason: ParsedGroupVersions
    status: "False"
    type: NamespaceDeletionGroupVersionParsingFailure
  - lastTransitionTime: "2021-10-10T08:35:11Z"      #A
    message: All content successfully deleted, may be waiting on
    reason: ContentDeleted      #A
```

```

    status: "False"      #A
    type: NamespaceDeletionContentFailure      #A
- lastTransitionTime: "2021-10-10T08:35:11Z"    #B
  message: 'Some resources are remaining: pods. has 1 resource
  reason: SomeResourcesRemain      #B
  status: "True"      #B
  type: NamespaceContentRemaining      #B
- lastTransitionTime: "2021-10-10T08:35:11Z"    #C
  message: 'Some content in the namespace has finalizers remain
          xyz.xyz/xyz-finalizer in 1 resource instances'      #
  reason: SomeFinalizersRemain      #C
  status: "True"      #C
  type: NamespaceFinalizersRemaining      #C
phase: Terminating

```

When you delete the kiada-test2 namespace, you won't see the output in this example. The command output in this example is hypothetical. I forced Kubernetes to produce it to demonstrate what happens when the delete process gets stuck. If you look at the output, you'll see that the objects in the namespace were all successfully marked for deletion, but one pod remains in the namespace due to a finalizer that was not removed from the pod. Don't worry about finalizers for now. You'll learn about them soon enough.

Before proceeding to the next section, please also delete the kiada-test1 namespace.

10.2 Organizing pods with labels

In this book, you will build and deploy the full Kiada application suite, which is composed of several services. So far, you've implemented the Kiada, the Quote service, and the Quiz service. These services run in three different pods. Accompanying the pods are other types of objects, like config maps, secrets, persistent volumes, and claims.

As you can imagine, the number of these objects will increase as the book progresses. Before things get out of hand, you need to start organizing these objects so that you and all the other users in your cluster can easily figure out which objects belong to which service.

In other systems that use a microservices architecture, the number of services

can exceed 100 or more. Some of these services are replicated, which means that multiple copies of the same pod are deployed. Also, at certain points in time, multiple versions of a service are running simultaneously. This results in hundreds or even thousands of pods in the system.

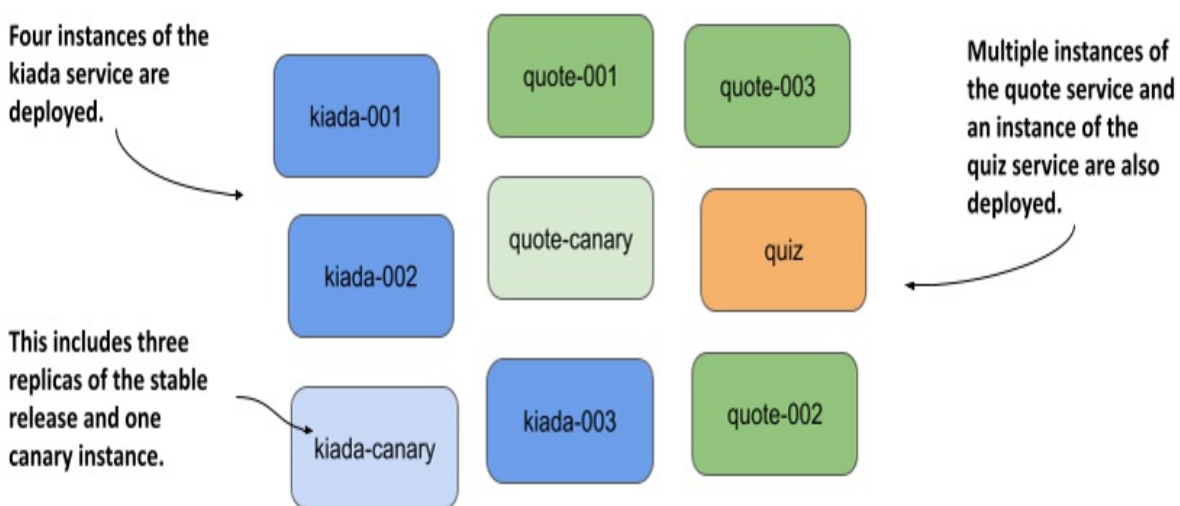
Imagine you, too, start replicating and running multiple releases of the pods in your Kiada suite. For example, suppose you are running both the stable and canary release of the Kiada service.

Definition

A canary release is a deployment pattern where you deploy a new version of an application alongside the stable version, and direct only a small portion of requests to the new version to see how it behaves before rolling it out to all users. This prevents a bad release from being made available to too many users.

You run three replicas of the stable Kiada version, and one canary instance. Similarly, you run three instances of the stable release of the Quote service, along with a canary release of the Quote service. You run a single, stable release of the Quiz service. All these pods are shown in the following figure.

Figure 10.4 Unorganized pods of the Kiada application suite



Even with only nine pods in the system, the system diagram is challenging to

understand. And it doesn't even show any of the other API objects required by the pods. It's obvious that you need to organize them into smaller groups. You could split these three services into three namespaces, but that's not the real purpose of namespaces. A more appropriate mechanism for this case is object *labels*.

10.2.1 Introducing labels

Labels are an incredibly powerful yet simple feature for organizing Kubernetes API objects. A label is a key-value pair you attach to an object that allows any user of the cluster to identify the object's role in the system. Both the key and the value are simple strings that you can specify as you wish. An object can have more than one label, but the label keys must be unique within that object. You normally add labels to objects when you create them, but you can also change an object's labels later.

Using labels to provide additional information about an object

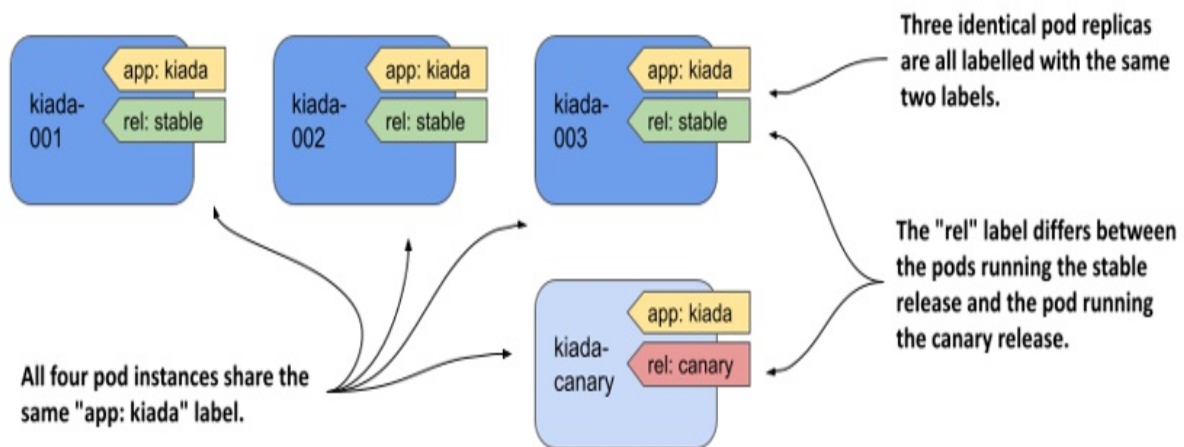
To illustrate the benefits of adding labels to objects, let's take the pods shown in figure 10.4. These pods run three different services - the Kiada service, the Quote, and the Quiz service. Additionally, the pods behind the Kiada and Quote services run different releases of each application. There are three pod instances running a stable release and one running a canary release.

To help identify the application and the release running in each pod, we use pod labels. Kubernetes does not care what labels you add to your objects. You can choose the keys and values however you want. In the case at hand, the following two labels make sense:

- The `app` label indicates to which application the pod belongs.
- The `rel` label indicates whether the pod is running the stable or canary release of the application.

As you can see in the following figure, the value of the `app` label is set to `kiada` in all three `kiada-xxx` and the `kiada-canary` pod, since all these pods are running the Kiada application. The `rel` label differs between the pods running the stable release and the pod running the canary release.

Figure 10.5 Labelling pods with the app and rel label

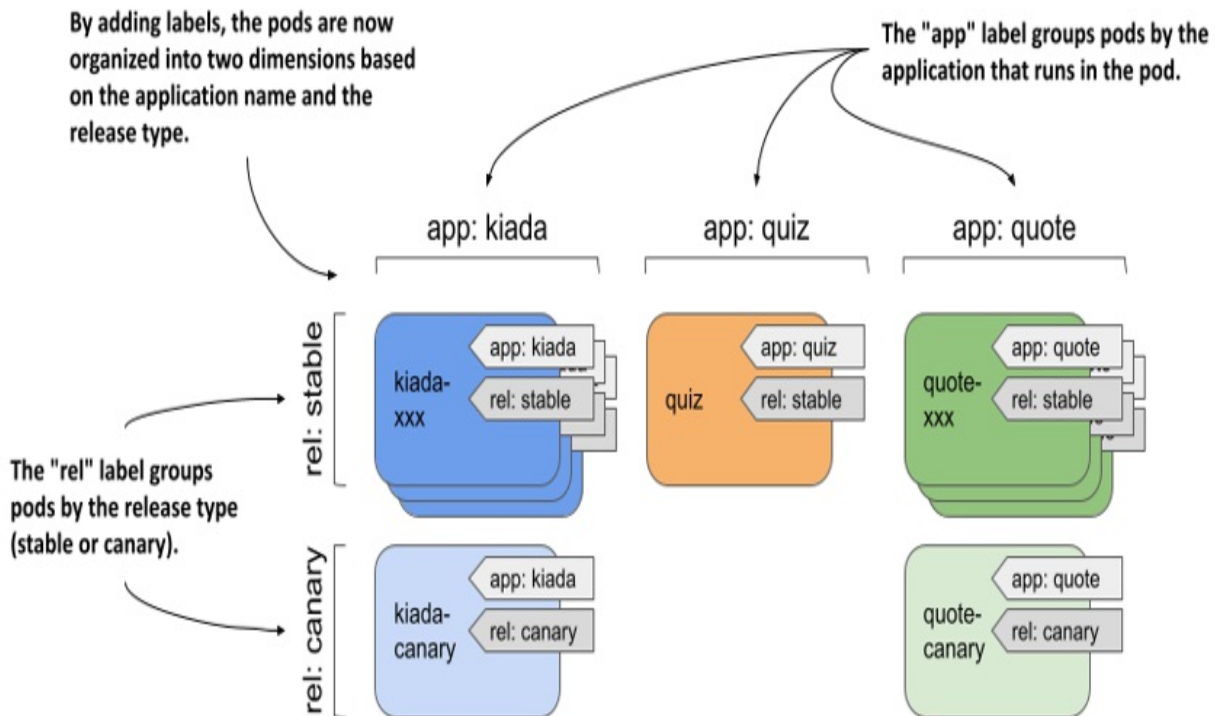


The illustration shows only the kiada pods, but imagine adding the same two labels to the other pods as well. With these labels, users that come across these pods can easily tell what application and what kind of release is running in the pod.

Understanding how labels keep objects organized

If you haven't yet realized the value of adding labels to an object, consider that by adding the `app` and `rel` labels, you've organized your pods in two dimensions (horizontally by application and vertically by release), as shown in the next figure.

Figure 10.6 All the pods of the Kiada suite organized by two criteria



This may seem abstract until you see how these labels make it easier to manage these pods with `kubectl`, so let's get practical.

10.2.2 Attaching labels to pods

The book's code archive contains a set of manifest files with all the pods from the previous example. All the stable pods are already labelled, but the canary pods aren't. You'll label them manually.

Setting up the exercise

To get started, create a new namespace called `kiada` as follows:

```
$ kubectl create namespace kiada
namespace/kiada created
```

Configure `kubectl` to default to this new namespace like this:

```
$ kubectl config set-context --current --namespace kiada
Context "kind-kind" modified.
```

The manifest files are organized into three subdirectories within Chapter10/kiada-suite/. Instead of applying each manifest individually, you can apply them all with the following command:

```
$ kubectl apply -f kiada-suite/ --recursive      #A
configmap/kiada-envoy-config created
pod/kiada-001 created
pod/kiada-002 created
pod/kiada-003 created
pod/kiada-canary created
secret/kiada-tls created
pod/quiz created
persistentvolumeclaim/quiz-data created
pod/quote-001 created
pod/quote-002 created
pod/quote-003 created
pod/quote-canary created
```

You're used to applying a single manifest file, but here you use the `-f` option to specify a directory name. Kubectl will apply all manifest files it finds in that directory. The `--recursive` option causes kubectl to look for manifests in all subdirectories instead of just the specified directory.

As you can see, this command created several objects of different kinds. Labels help keep them organized.

Defining labels in object manifests

Examine the manifest file `kiada-suite/kiada/pod.kiada-001.yaml` shown in the following listing. Look at the metadata section. Besides the `name` field, which you've seen many times before, this manifest also contains the `labels` field. It specifies two labels: `app` and `rel`.

Listing 10.3 A pod with labels

```
apiVersion: v1
kind: Pod
metadata:
  name: kiada-001
  labels:      #A
    app: kiada    #B
    rel: stable   #C
```

```
spec:  
...
```

Labels are supported by all object kinds. Regardless of the kind, you add labels to the object by specifying them in the `metadata.labels` map.

Displaying object labels

You can see the labels of a particular object by running the `kubectl describe` command. View the labels of the pod `kiada-001` as follows:

```
$ kubectl describe pod kiada-001  
Name:          kiada-001  
Namespace:     kiada  
Priority:       0  
Node:          kind-worker2/172.18.0.2  
Start Time:    Sun, 10 Oct 2021 21:58:25 +0200  
Labels:        app=kiada      #A  
               rel=stable     #A  
Annotations:   <none>        #B  
...
```

The `kubectl get pods` command doesn't display labels by default, but you can display them with the `--show-labels` option. Check the labels of all pods in the namespace as follows:

```
$ kubectl get pods --show-labels  
NAME           READY   STATUS    RESTARTS   AGE   LABELS      #A  
kiada-canary   2/2     Running   0           12m   <none>      #B  
kiada-001      2/2     Running   0           12m   app=kiada,rel=s  
kiada-002      2/2     Running   0           12m   app=kiada,rel=s  
kiada-003      2/2     Running   0           12m   app=kiada,rel=s  
quiz           2/2     Running   0           12m   app=quiz,rel=st  
quote-canary   2/2     Running   0           12m   <none>      #B  
quote-001      2/2     Running   0           12m   app=quote,rel=s  
quote-002      2/2     Running   0           12m   app=quote,rel=s  
quote-003      2/2     Running   0           12m   app=quote,rel=s
```

Instead of showing all labels with `--show-labels`, you can also show specific labels with the `--label-columns` option (or the shorter variant `-L`). Each label is displayed in its own column. List all pods along with their `app` and `rel` labels as follows:


```
$ kubectl get pods -L app,rel
NAME          READY   STATUS    RESTARTS   AGE   APP    REL
kiada-canary  2/2     Running   0           14m   kiada  stable
kiada-001     2/2     Running   0           14m   kiada  stable
kiada-002     2/2     Running   0           14m   kiada  stable
kiada-003     2/2     Running   0           14m   kiada  stable
quiz          2/2     Running   0           14m   quiz   stable
quote-canary  2/2     Running   0           14m   quote  stable
quote-001     2/2     Running   0           14m   quote  stable
quote-002     2/2     Running   0           14m   quote  stable
quote-003     2/2     Running   0           14m   quote  stable
```

You can see that the two canary pods have no labels. Let's add them.

Adding labels to an existing object

To add labels to an existing object, you can edit the object's manifest file, add labels to the metadata section, and reapply the manifest using `kubectl apply`. You can also edit the object definition directly in the API using `kubectl edit`. However, the simplest method is to use the `kubectl label` command.

Add the labels `app` and `rel` to the `kiada-canary` pod using the following command:

```
$ kubectl label pod kiada-canary app=kiada rel=canary
pod/kiada-canary labeled
```

Now do the same for the pod `quote-canary`:

```
$ kubectl label pod quote-canary app=kiada rel=canary
pod/quote-canary labeled
```

List the pods and display their labels to confirm that all pods are now labelled. If you didn't notice the error when you entered the previous command, you probably caught it when you listed the pods. The `app` label of the pod `quote-canary` is set to the wrong value (`kiada` instead of `quote`). Let's fix this.

Changing labels of an existing object

You can use the same command to update object labels. To change the label you set incorrectly, run the following command:

```
$ kubectl label pod quote-canary app=quote
error: 'app' already has a value (kiada), and --overwrite is fals
```

To prevent accidentally changing the value of an existing label, you must explicitly tell kubectl to overwrite the label with `--overwrite`. Here's the correct command:

```
$ kubectl label pod quote-canary app=quote --overwrite
pod/quote-canary labeled
```

List the pods again to check that all the labels are now correct.

Labelling all objects of a kind

Now imagine that you want to deploy another application suite in the same namespace. Before doing this, it is useful to add the `suite` label to all existing pods so that you can distinguish which pods belong to one suite and which belong to the other. Run the following command to add the label to all pods in the namespace:

```
$ kubectl label pod --all suite=kiada-suite
pod/kiada-canary labeled
pod/kiada-001 labeled
...
pod/quote-003 labeled
```

List the pods again with the `--show-labels` or the `-L suite` option to confirm that all pods now contain this new label.

Removing a label from an object

Okay, I lied. You will not be setting up another application suite. Therefore, the `suite` label is redundant. To remove the label from an object, run the `kubectl label` command with a minus sign after the label key as follows:

```
$ kubectl label pod kiada-canary suite-      #A
pod/kiada-canary labeled
```

To remove the label from all other pods, specify `--all` instead of the pod name:

```
$ kubectl label pod --all suite-  
label "suite" not found.      #A  
pod/kiada-canary not labeled  #A  
pod/kiada-001 labeled  
...  
pod/quote-003 labeled
```

Note

If you set the label value to an empty string, the label key is not removed. To remove it, you must use the minus sign after the label key.

10.2.3 Label syntax rules

While you can label your objects however you like, there are some restrictions on both the label keys and the values.

Valid label keys

In the examples, you used the label keys `app`, `rel`, and `suite`. These keys have no prefix and are considered private to the user. Common label keys that Kubernetes itself applies or reads always start with a prefix. This also applies to labels used by Kubernetes components outside of the core, as well as other commonly accepted label keys.

An example of a prefixed label key used by Kubernetes is `kubernetes.io/arch`. You can find it on Node objects to identify the architecture type used by the node.

```
$ kubectl get node -L kubernetes.io/arch
```

NAME	STATUS	ROLES	AGE	VERSIO
kind-control-plane	Ready	control-plane,master	31d	v1.21.
kind-worker	Ready	<none>	31d	v1.21.
kind-worker2	Ready	<none>	31d	v1.21.

The label prefixes `kubernetes.io/` and `k8s.io/` are reserved for Kubernetes components. If you want to use a prefix for your labels, use your

organization's domain name to avoid conflicts.

When choosing a key for your labels, some syntax restrictions apply to both the prefix and the name part. The following table provides examples of valid and invalid label keys.

Table 10.1 Examples of valid and invalid label keys

Valid label keys	Invalid label keys
foo	_foo
foo-bar_baz	foo%bar*baz
example/foo	/foo
example/F00	EXAMPLE/foo
example.com/foo	example..com/foo
my_example.com/foo	my@example.com/foo
example.com/foo-bar	example.com/-foo-bar
my.example.com/foo	a.very.long.prefix.over.253.characters/foo

The following syntax rules apply to the prefix:

- Must be a DNS subdomain (must contain only lowercase alphanumeric characters, hyphens, underscores, and dots).
- Must be no more than 253 characters long (not including the slash character).

- Must end with a forward slash.

The prefix must be followed by the label name, which:

- Must begin and end with an alphanumeric character.
- May contain hyphens, underscores, and dots.
- May contain uppercase letters.
- May not be longer than 63 characters.

Valid label values

Remember that labels are used to add identifying information to your objects. As with label keys, there are certain rules you must follow for label values. For example, label values can't contain spaces or special characters. The following table provides examples of valid and invalid label values.

Table 10.2 Examples of valid and invalid label values

Valid label values	Invalid label values
foo	_foo
foo-bar_baz	foo%bar*baz
F00	value.longer.than.63.characters
(empty)	value with spaces

A label value:

- May be empty.
- Must begin with an alphanumeric character if not empty.
- May contain only alphanumeric characters, hyphens, underscores, and

dots.

- Must not contain spaces or other whitespace.
- Must be no more than 63 characters long.

If you need to add values that don't follow these rules, you can add them as annotations instead of labels. You'll learn more about annotations later in this chapter.

10.2.4 Using standard label keys

While you can always choose your own label keys, there are some standard keys you should know. Some of these are used by Kubernetes itself to label system objects, while others have become common for use in user-created objects.

Well-known labels used by Kubernetes

Kubernetes doesn't usually add labels to the objects you create. However, it does use various labels for system objects such as Nodes and PersistentVolumes, especially if the cluster is running in a cloud environment. The following table lists some well-known labels you might find on these objects.

Table 10.3 Well-known labels on Nodes and PersistentVolumes

Label key	Example value	Applied to	Description
kubernetes.io/arch	amd64	Node	The architecture of the node
			The system

kubernetes.io/os	linux	Node	run the
kubernetes.io/hostname	worker-node2	Node	The hos
topology.kubernetes.io/ region	eu-west3	Node PersistentVolume	The wh noc per vol loc
topology.kubernetes.io/ zone	eu-west3-c	Node PersistentVolume	The wh noc per vol loc
node.kubernetes.io/ instance-type	micro-1	Node	The ins Set usi pro infi

Note

You can also find some of these labels under the older prefix `beta.kubernetes.io`, in addition to `kubernetes.io`.

Cloud providers can provide additional labels for nodes and other objects. For example, Google Kubernetes Engine adds the labels `cloud.google.com/gke-nodepool` and `cloud.google.com/gke-os-distribution` to provide further information about each node. You can also find more standard labels on other objects.

Recommended labels for deployed application components

The Kubernetes community has agreed on a set of standard labels that you can add to your objects so that other users and tools can understand them. The following table lists these standard labels.

Table 10.4 Recommended labels used in the Kubernetes community

Label	Example	Description
<code>app.kubernetes.io/name</code>	quotes	The name of the application. If the application consists of multiple components, this is the name of the entire application, not the individual components.
<code>app.kubernetes.io/instance</code>	quotes-foo	The name of this application instance. If you create multiple instances of the same application for different purposes, this label helps you distinguish between them.

<code>app.kubernetes.io/component</code>	database	The role that this component plays in the application architecture.
<code>app.kubernetes.io/part-of</code>	kubia-demo	The name of the application suite to which this application belongs.
<code>app.kubernetes.io/version</code>	1.0.0	The version of the application.
<code>app.kubernetes.io/managed-by</code>	quotes-operator	The tool that manages the deployment and update of this application.

All objects belonging to the same application instance should have the same set of labels. For example, the pod and the persistent volume claim used by that pod should have the same values for the labels listed in the previous table. This way, anyone using the Kubernetes cluster can see which components belong together and which do not. Also, you can manage these components using bulk operations by using label selectors, which are explained in the next section.

10.3 Filtering objects with label selectors

The labels you added to the pods in the previous exercises allow you to identify each object and understand its place in the system. So far, these labels have only provided additional information when you list objects. But

the real power of labels comes when you use *label selectors* to filter objects based on their labels.

Label selectors allow you to select a subset of pods or other objects that contain a particular label and perform an operation on those objects. A label selector is a criterion that filters objects based on whether they contain a particular label key with a particular value.

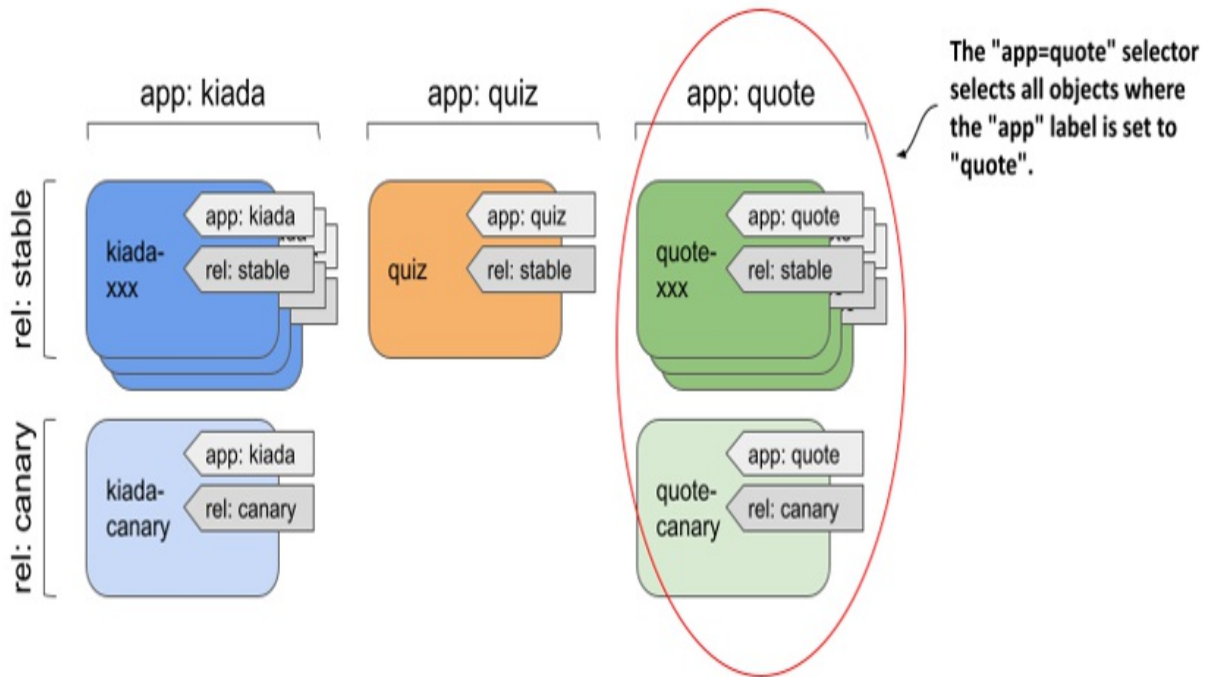
There are two types of label selectors:

- *equality-based* selectors, and
- *set-based* selectors.

Introducing equality-based selectors

An equality-based selector can filter objects based on whether the value of a particular label is equal to or not equal to a particular value. For example, applying the label selector `app=quote` to all pods in our previous example selects all quote pods (all stable instances plus the canary instance), as shown in the following figure.

Figure 10.7 Selecting objects using an equality-based selector



Similarly, the label selector `app!=quote` selects all pods except the quote pods.

Introducing set-based selectors

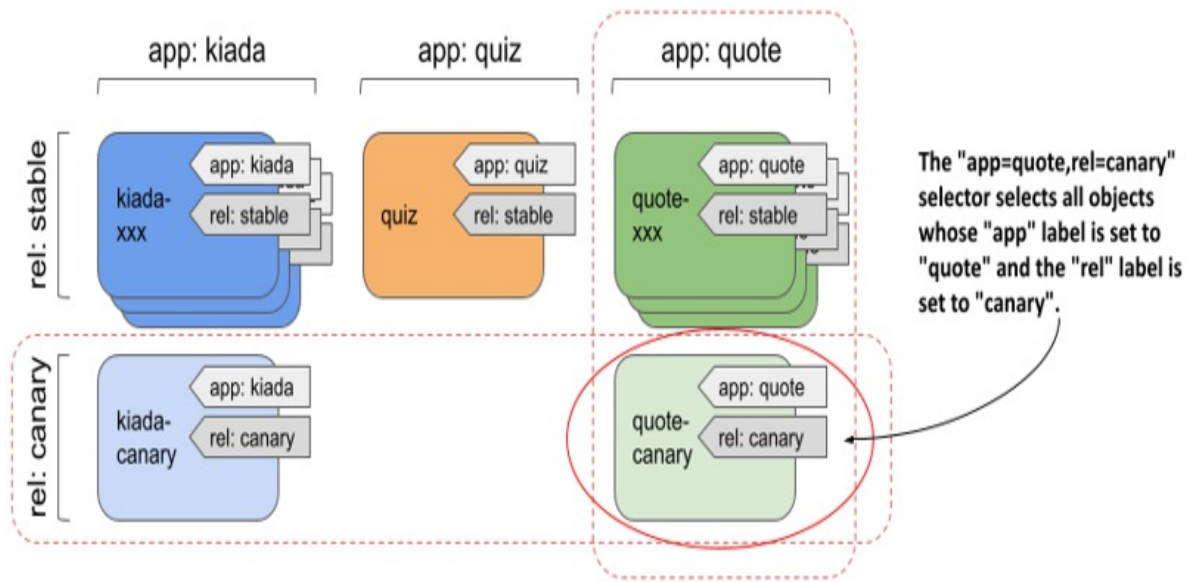
Set-based selectors are more powerful and allow you to specify:

- a set of values that a particular label must have; for example: `app in (quiz, quote)`,
- a set of values that a particular label must not have; for example: `app notin (kiada)`,
- a particular label key that should be present in the object's labels; for example, to select objects that have the app label, the selector is simply `app`,
- a particular label key that should not be present in the object's labels; for example, to select objects that do not have the app label, the selector is `!app`.

Combining multiple selectors

When you filter objects, you can combine multiple selectors. To be selected, an object must match all of the specified selectors. As shown in the following figure, the selector `app=quote,rel=canary` selects the pod `quote-canary`.

Figure 10.8 Combining two label selectors



You use label selectors when managing objects with `kubectl`, but they are also used internally by Kubernetes when an object references a subset of other objects. These scenarios are covered in the next two sections.

10.3.1 Using label selectors for object management with `kubectl`

If you've been following the exercises in this book, you've used the `kubectl get` command many times to list objects in your cluster. When you run this command without specifying a label selector, it prints all the objects of a particular kind. Fortunately, you never had more than a few objects in the namespace, so the list was never too long. In real-world environments, however, you can have hundreds of objects of a particular kind in the namespace. That's when label selectors come in.

Filtering the list of objects using label selectors

You'll use a label selector to list the pods you created in the kiada namespace in the previous section. Let's try the example in figure 10.7, where the selector `app=quote` was used to select only the pods running the quote application. To apply a label selector to `kubectl get`, specify it with the `--selector` argument (or the short equivalent `-l`) as follows:

```
$ kubectl get pods -l app=quote
NAME           READY   STATUS    RESTARTS   AGE
quote-canary   2/2     Running   0           2h
quote-001      2/2     Running   0           2h
quote-002      2/2     Running   0           2h
quote-003      2/2     Running   0           2h
```

Only the quote pods are shown. Other pods are ignored. Now, as another example, try listing all the canary pods:

```
$ kubectl get pods -l rel=canary
NAME           READY   STATUS    RESTARTS   AGE
kiada-canary   2/2     Running   0           2h
quote-canary   2/2     Running   0           2h
```

Let's also try the example from figure 10.8, combining the two selectors `app=quote` and `rel=canary`:

```
$ kubectl get pods -l app=quote,rel=canary
NAME           READY   STATUS    RESTARTS   AGE
quote-canary   2/2     Running   0           2h
```

Only the labels of the quote-canary pod match both label selectors, so only this pod is shown.

As the next example, try using a set-based selector. To display all quiz and quote pods, use the selector `'app in (quiz, quote)'` as follows:

```
$ kubectl get pods -l 'app in (quiz, quote)' -L app
NAME           READY   STATUS    RESTARTS   AGE   APP
quiz           2/2     Running   0           2h   quiz
quote-canary   2/2     Running   0           2h   quote
quote-001      2/2     Running   0           2h   quote
quote-002      2/2     Running   0           2h   quote
quote-003      2/2     Running   0           2h   quote
```

You'd get the same result if you used the equality-based selector

'app!=kiada' or the set-based selector 'app notin (kiada)'. The -L app option in the command displays the value of the app label for each pod (see the APP column in the output).

The only two selectors you haven't tried yet are the ones that only test for the presence (or absence) of a particular label key. If you want to try them, first remove the rel label from the quiz pod with the following command:

```
$ kubectl label pod quiz rel-  
pod/quiz labeled
```

You can now list pods that do not have the rel label like so:

```
$ kubectl get pods -l '!rel'  
NAME      READY   STATUS    RESTARTS   AGE  
quiz      2/2     Running   0           2h
```

NOTE

Make sure to use single quotes around !rel, so your shell doesn't evaluate the exclamation mark.

And to list all pods that *do* have the rel label, run the following command:

```
$ kubectl get pods -l rel
```

The command should show all pods except the quiz pod.

If your Kubernetes cluster is running in the cloud and distributed across multiple regions or zones, you can also try to list nodes of a particular type or list nodes and persistent volumes in a particular region or zone. In table 10.3, you can see which label key to specify in the selector.

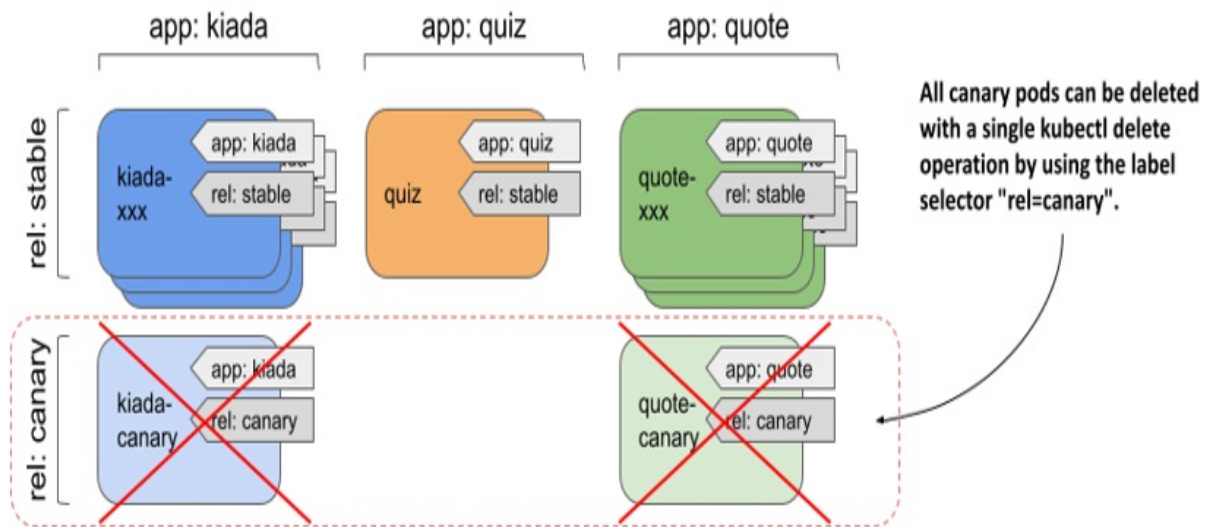
You've now mastered the use of label selectors when listing objects. Do you have the confidence to use them for deleting objects as well?

Deleting objects using a label selector

There are currently two canary releases in use in your system. It turns out that

they aren't behaving as expected and need to be terminated. You could list all canaries in your system and remove them one by one. A faster method is to use a label selector to delete them in a single operation, as illustrated in the following figure.

Figure 10.9 Selecting and deleting all canary pods using the `rel=canary` label selector



Delete the canary pods with the following command:

```
$ kubectl delete pods -l rel=canary
pod "kiada-canary" deleted
pod "quote-canary" deleted
```

The output of the command shows that both the `kiada-canary` and `quote-canary` pods have been deleted. However, because the `kubectl delete` command does not ask for confirmation, you should be very careful when using label selectors to delete objects. Especially in a production environment.

10.3.2 Utilizing label selectors within Kubernetes API objects

You've learned how to use labels and selectors with `kubectl` to organize your objects and filter them, but selectors are also used within Kubernetes API objects.

For example, you can specify a node selector in each Pod object to specify which nodes the pod can be scheduled to. In the next chapter, which explains the Service object, you'll learn that you need to define a pod selector in this object to specify a pod subset to which the service will forward traffic. In the following chapters, you'll see how pod selectors are used by objects such as Deployment, ReplicaSet, DaemonSet, and StatefulSet to define the set of pods that belong to these objects.

Using label selectors to schedule pods to specific nodes

All the pods you've created so far have been randomly distributed across your entire cluster. Normally, it doesn't matter which node a pod is scheduled to, because each pod gets exactly the amount of compute resources it requests (CPU, memory, and so on). Also, other pods can access this pod regardless of which node this and the other pods are running on. However, there are scenarios where you may want to deploy certain pods only on a specific subset of nodes.

A good example is when your hardware infrastructure isn't homogenous. If some of your worker nodes use spinning disks while others use SSDs, you may want to schedule pods that require low-latency storage only to the nodes that can provide it.

Another example is if you want to schedule front-end pods to some nodes and back-end pods to others. Or if you want to deploy a separate set of application instances for each customer and want each set to run on its own set of nodes for security reasons.

In all of these cases, rather than scheduling a pod to a particular node, allow Kubernetes to select a node out from a set of nodes that meet the required criteria. Typically, you'll have more than one node that meets the specified criteria, so that if one node fails, the pods running on it can be moved to the other nodes.

The mechanisms you can use to do this are labels and selectors.

Attaching labels to nodes

The Kiada application suite consists of the Kiada, Quiz, and Quote services. Let's consider the Kiada service as the front-end and the Quiz and Quote services as the back-end services. Imagine that you want the Kiada pods to be scheduled only to the cluster nodes that you reserve for front-end workloads. To do this, you first label some of the nodes as such.

First, list all the nodes in your cluster and select one of the worker nodes. If your cluster consists of only one node, use that one.

```
$ kubectl get node
```

NAME	STATUS	ROLES	AGE	VERSION
kind-control-plane	Ready	control-plane,master	1d	v1.21.
kind-worker	Ready	<none>	1d	v1.21.
kind-worker2	Ready	<none>	1d	v1.21.

In this example, I choose the kind-worker node as the node for the front-end workloads. After selecting your node, add the node-role: front-end label to it as follows:

```
$ kubectl label node kind-worker node-role=front-end
node/kind-worker labeled
```

Now list the nodes with a label selector to confirm that this is the only front-end node:

```
$ kubectl get node -l node-role=front-end
```

NAME	STATUS	ROLES	AGE	VERSION
kind-worker	Ready	<none>	1d	v1.21.1

If your cluster has many nodes, you can label multiple nodes this way.

Scheduling pods to nodes with specific labels

To schedule a pod to the node(s) you designated as front-end nodes, you must add a node selector to the pod's manifest before you create the pod. The following listing shows the contents of the pod.kiada-front-end.yaml manifest file. The node selector is specified in the spec.nodeSelector field.

Listing 3.4 Using a node selector to schedule a pod to a specific node

```
apiVersion: v1
kind: Pod
metadata:
  name: kiada-front-end
spec:
  nodeSelector:      #A
    node-role: front-end    #A
  volumes:
```

In the `nodeSelector` field, you can specify one or more label keys and values that the node must match to be eligible to run the pod. Note that this field only supports specifying an equality-based label selector. The label value must match the value in the selector. You can't use a not-equal or set-based selector in the `nodeSelector` field. However, set-based selectors are supported in other objects.

When you create the pod from the previous listing by applying the manifest with `kubectl apply`, you'll see that the pod is scheduled to the node(s) that you have labelled with the label `node-role: front-end`. You can confirm this by displaying the pod with the `-o wide` option to show the pod's node as follows:

```
$ kubectl get pod kiada-front-end -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP
kiada-front-end     2/2     Running   0           1m    10.244.2.20
```

You can delete and recreate the pod several times to make sure that it always lands on the front-end node(s).

Note

Other mechanisms for affecting pod scheduling are covered in chapter 21.

Using label selectors in persistent volume claims

In chapter 8, you learned about persistent volumes and persistent volume claims. A persistent volume usually represents a network storage volume, and the persistent volume claim allows you to reserve one of the persistent volumes so that you can use it in your pods.

I didn't mention this at the time, but you can specify a label selector in the `PersistentVolumeClaim` object definition to indicate which persistent volumes Kubernetes should consider for binding. Without the label selector, any available persistent volume that matches the capacity and access modes specified in the claim will be bound. If the claim specifies a label selector, Kubernetes also checks the labels of the available persistent volumes and binds the claim to a volume only if its labels match the label selector in the claim.

Unlike the node selector in the `Pod` object, the label selector in the `PersistentVolumeClaim` object supports both equality-based and set-based selectors and uses a slightly different syntax.

The following listing shows a `PersistentVolumeClaim` object definition that uses an equality-based selector to ensure that the bound volume has the label type: `ssd`.

Listing 10.4 A `PersistentVolumeClaim` definition with an equality-based selector

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: ssd-claim
spec:
  selector:      #A
    matchLabels:  #B
      type: ssd   #C
```

The `matchLabels` field behaves just like the `nodeSelector` field in the `Pod` object you learned about in the previous section.

Alternatively, you can use the `matchExpressions` field to specify a more expressive set-based label selector. The following listing shows a selector that matches `PersistentVolumes` with the type label set to anything other than `ssd`, and the age label set to `old` or `very-old`.

Listing 10.5 Using a set-based selector in a `PersistentVolumeClaim`

```
spec:
  selector:
```

```

matchExpressions:    #A
- key: type          #B
  operator: NotIn     #B
  values:             #B
  - ssd               #B
- key: age           #C
  operator: In        #C
  values:             #C
  - old               #C
  - very-old          #C

```

As you can see in the listing, you can specify multiple `matchExpressions` in the selector. To match the selector, the `PersistentVolume`'s labels must match all the expressions.

You must specify the key, operator, and values for each expression. The key is the label key to which the selector is applied. The operator must be one of `In`, `NotIn`, `Exists`, and `DoesNotExist`. When you use the `In` or `NotIn` operators, the values array must not be empty. However, you must omit it when using the `Exists` or `DoesNotExist` operators.

Note

The `NotIn` operator matches objects that don't contain the specified label. A `PersistentVolumeClaim` with the label selector `type NotIn [ssd], age In [old, very-old]` can therefore bind to a `PersistentVolume` with the label `age: old`, even though it has no `type` label. To change this, you must add an additional selector expression with the `Exists` operator.

To see these selectors in action, first create the persistent volumes found in the manifest file `persistent-volumes.yaml`. Then create the two claims in the manifest files `pvc.ssd-claim.yaml` and `pvc.old-non-ssd-claim.yaml`. You can find these files in the `Chapter10/` directory in the book's code archive.

Filtering objects with field selectors

Kubernetes initially only allowed you to filter objects with label selectors. Then it became clear that users want to filter objects by other properties as well. One such example is filtering pods based on the cluster node they are

running on. This can now be accomplished with *field selectors*. Unlike label selectors, you only use field selectors with `kubectl` or other Kubernetes API clients. No object uses field selectors internally.

The set of fields you can use in a field selector depends on the object kind. The `metadata.name` and `metadata.namespace` fields are always supported. Like equality-based label selectors, field selectors support the equal (`=` or `==`) and not equal (`!=`) operator, and you can combine multiple field selectors by separating them with a comma.

Listing pods running on a specific node

As an example of using field selectors, run the following command to list pods on the `kind-worker` node (if your cluster wasn't provisioned with the `kind` tool, you must specify a different node name):

```
$ kubectl get pods --field-selector spec.nodeName=kind-worker
NAME READY STATUS RESTARTS AGE
kiada-front-end 2/2 Running 0 15m
kiada-002 2/2 Running 0 3h
quote-002 2/2 Running 0 3h
```

Instead of displaying all pods in the current namespace, the filter selected only those pods whose `spec.nodeName` field is set to `kind-worker`.

How do you know which field to use in the selector? By looking up the field names with `kubectl explain`, of course. You learned this in chapter 4. For example: `kubectl explain pod.spec` shows the fields in the `spec` section of Pod objects. It doesn't show which fields are supported in field selectors, but you can try to use a field and `kubectl` will tell you if it isn't supported.

Finding pods that aren't running

Another example of using field selectors is to find pods that aren't currently running. You accomplish this by using the `status.phase!=Running` field selector as follows:

```
$ kubectl get pods --field-selector status.phase!=Running
```

Since all pods in your namespace are running, this command won't produce

any results, but it can be useful in practice, especially if you combine it with the `--all-namespaces` option to list non-running pods in all namespaces. The full command is as follows:

```
$ kubectl get pods --field-selector status.phase!=Running --all-n
```

The `--all-namespaces` option is also useful when you use the `metadata.name` or `metadata.namespace` fields in the field selector.

10.4 Annotating objects

Adding labels to your objects makes them easier to manage. In some cases, objects must have labels because Kubernetes uses them to identify which objects belong to the same set. But as you learned in this chapter, you can't just store anything you want in the label value. For example, the maximum length of a label value is only 63 characters, and the value can't contain whitespace at all.

For this reason, Kubernetes provides a feature similar to labels—object *annotations*.

10.4.1 Introducing object annotations

Like labels, annotations are also key-value pairs, but they don't store identifying information and can't be used to filter objects. Unlike labels, an annotation value can be much longer (up to 256 KB at the time of this writing) and can contain any character.

Understanding annotations added by Kubernetes

Tools like `kubectl` and the various controllers that run in Kubernetes may add annotations to your objects if the information can't be stored in one of the object's fields.

Annotations are often used when new features are introduced to Kubernetes. If a feature requires a change to the Kubernetes API (for example, a new field needs to be added to an object's schema), that change is usually deferred for a

few Kubernetes releases until it's clear that the change makes sense. After all, changes to any API should always be made with great care, because after you add a field to the API, you can't just remove it or you'll break everyone that use the API.

Changing the Kubernetes API requires careful consideration, and each change must first be proven in practice. For this reason, instead of adding new fields to the schema, usually a new object annotation is introduced first. The Kubernetes community is given the opportunity to use the feature in practice. After a few releases, when everyone's happy with the feature, a new field is introduced and the annotation is deprecated. Then a few releases later, the annotation is removed.

Adding your own annotations

As with labels, you can add your own annotations to objects. A great use of annotations is to add a description to each pod or other object so that all users of the cluster can quickly see information about an object without having to look it up elsewhere.

For example, storing the name of the person who created the object and their contact information in the object's annotations can greatly facilitate collaboration between cluster users.

Similarly, you can use annotations to provide more details about the application running in a pod. For example, you can attach the URL of the Git repository, the Git commit hash, the build timestamp, and similar information to your pods.

You can also use annotations to add the information that certain tools need to manage or augment your objects. For example, a particular annotation value set to `true` could signal to the tool whether it should process and modify the object.

Understanding annotation keys and values

The same rules that apply to label keys also apply to annotations keys. For

more information, see section 10.2.3. Annotation values, on the other hand, have no special rules. An annotation value can contain any character and can be up to 256 KB long. It must be a string, but can contain plain text, YAML, JSON, or even a Base64-Encoded value.

10.4.2 Adding annotations to objects

Like labels, annotations can be added to existing objects or included in the object manifest file you use to create the object. Let's look at how to add an annotation to an existing object.

Setting object annotations

The simplest way to add an annotation to an existing object is to use the `kubectl annotate` command. Let's add an annotation to one of the pods. You should still have a pod named `kiada-front-end` from one of the previous exercises in this chapter. If not, you can use any other pod or object in your current namespace. Run the following command:

```
$ kubectl annotate pod kiada-front-end created-by='Marko Luksa <m
pod/kiada-front-end annotated
```

This command adds the annotation `created-by` with the value `'Marko Luksa <marko.luksa@xyz.com>'` to the `kiada-front-end` pod.

Specifying annotations in the object manifest

You can also add annotations to your object manifest file before you create the object. The following listing shows an example. You can find the manifest in the `pod.pod-with-annotations.yaml` file.

Listing 10.6 Annotations in an object manifest

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-annotations
  annotations:
```



```
    created-by: Marko Luksa <marko.luksa@xyz.com>      #A
    contact-phone: +1 234 567 890      #B
    managed: 'yes'      #C
    revision: '3'      #D
spec:
  ...
```

Warning

Make sure you enclose the annotation value in quotes if the YAML parser would otherwise treat it as something other than a string. If you don't, a cryptic error will occur when you apply the manifest. For example, if the annotation value is a number like 123 or a value that could be interpreted as a boolean (true, false, but also words like yes and no), enclose the value in quotes (examples: "123", "true", "yes") to avoid the following error: "unable to decode yaml ... ReadString: expects " or n, but found t".

Apply the manifest from the previous listing by executing the following command:

```
$ kubectl apply -f pod.pod-with-annotations.yaml
```

10.4.3 Inspecting an object's annotations

Unlike labels, the `kubectl get` command does not provide an option to display annotations in the object list. To see the annotations of an object, you should use `kubectl describe` or find the annotation in the object's YAML or JSON definition.

Viewing an object's annotations with `kubectl describe`

To see the annotations of the `pod-with-annotations` pod you created, use `kubectl describe`:

```
$ kubectl describe pod pod-with-annotations
Name:          pod-with-annotations
Namespace:     kiada
Priority:       0
Node:          kind-worker/172.18.0.4
Start Time:    Tue, 12 Oct 2021 16:37:50 +0200
```

```
Labels:      <none>
Annotations:  contact-phone: +1 234 567 890      #A
               created-by: Marko Luksa <marko.luksa@xyz.com>      #A
               managed: yes      #A
               revision: 3      #A
Status:      Running

...
```

Displaying the object's annotations in the object's JSON definition

Alternatively, you can use the `jq` command to extract the annotations from the JSON definition of the pod:

```
$ kubectl get pod pod-with-annotations -o json | jq .metadata.annotations
{
  "contact-phone": "+1 234 567 890",
  "created-by": "Marko Luksa <marko.luksa@xyz.com>",
  "kubectl.kubernetes.io/last-applied-configuration": "...",
  "managed": "yes",
  "revision": "3"
}
```

You'll notice that there's an additional annotation in the object with the key `kubectl.kubernetes.io/last-applied-configuration`. It isn't shown by the `kubectl describe` command, because it's only used internally by `kubectl` and would also make the output too long. In the future, this annotation may become deprecated and then be removed. Don't worry if you don't see it when you run the command yourself.

10.4.4 Updating and removing annotations

If you want to use the `kubectl annotate` command to change an existing annotation, you must also specify the `--overwrite` option, just as you would when changing an existing object label. For example, to change the annotation `created-by`, the full command is as follows:

```
$ kubectl annotate pod kiada-front-end created-by='Humpty Dumpty'
```

To remove an annotation from an object, add the minus sign to the end of the annotation key you want to remove:

```
$ kubectl annotate pod kiada-front-end created-by-
```

10.5 Summary

The Kubernetes features described in this chapter will help you keep your cluster organized and make your system easier to understand. In this chapter, you learned that:

- Objects in a Kubernetes cluster are typically divided into many namespaces. Within a namespace, object names must be unique, but you can give two objects the same name if you create them in different namespaces.
- Namespaces allow different users and teams to use the same cluster as if they were using separate Kubernetes clusters.
- Each object can have several labels. Labels are key-value pairs that help identify the object. By adding labels to objects, you can effectively organize objects into groups.
- Label selectors allow you to filter objects based on their labels. You can easily filter pods that belong to a specific application, or by any other criteria if you've previously added the appropriate labels to those pods.
- Field selectors are like label selectors, but they allow you to filter objects based on specific fields in the object manifest. For example, a field selector can be used to list pods that run on a particular node. Unfortunately, you can't use them to filter on annotations.
- Instead of performing an operation on each pod individually, you can use a label selector to perform the same operation on a set of objects that match the label selector.
- Labels and selectors are also used internally by some object types. You can add labels to Node objects and define a node selector in a pod to schedule that pod only to those nodes that meet the specified criteria.
- In addition to labels, you can also add annotations to objects. An annotation can contain a much larger amount of data and can include whitespace and other special characters that aren't allowed in labels. Annotations are typically used to add additional information used by tools and cluster users. They are also used to defer changes to the Kubernetes API.

In the next chapter, you'll learn how to forward traffic to a set of pods using the Service object.

11 Exposing Pods with Services

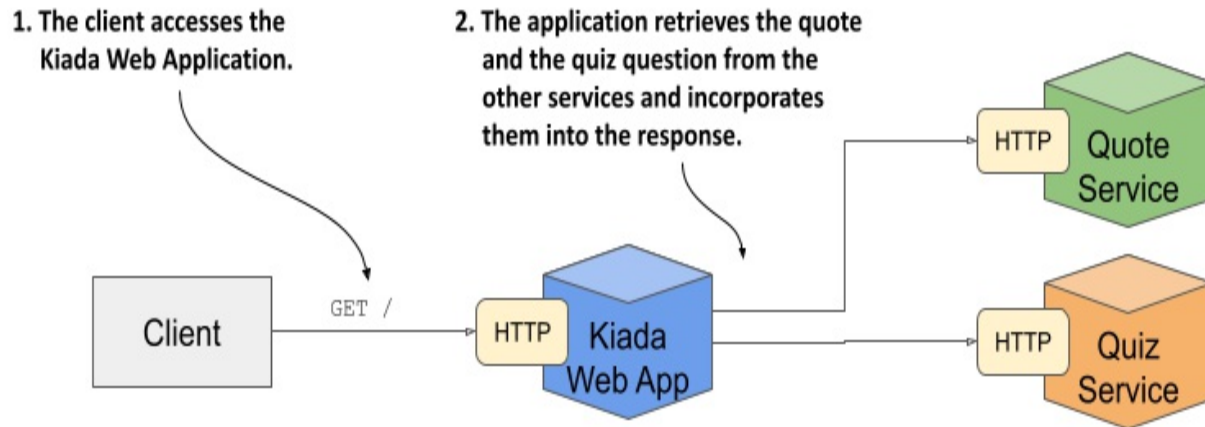
This chapter covers

- Communication between pods
- Distributing client connections over a group of pods providing the same service
- Discovering services in the cluster through DNS and environment variables
- Exposing services to clients outside the cluster
- Using readiness probes to add or remove individual pods from services

Instead of running a single pod to provide a particular service, people nowadays typically run several replicas of the pod so that the load can be distributed across multiple cluster nodes. But that means all pod replicas providing the same service should be reachable at a single address so clients can use that single address, rather than having to keep track of and connect directly to individual pod instances. In Kubernetes, you do that with Service objects.

The Kiada suite you're building in this book consists of three services - the Kiada service, the Quiz service, and the Quote service. So far, these are three isolated services that you interact with individually, but the plan is to connect them, as shown in the following figure.

Figure 11.1 The architecture and operation of the Kiada suite.



The Kiada service will call the other two services and integrate the information they return into the response it sends to the client. Multiple pod replicas will provide each service, so you'll need to use Service objects to expose them.

NOTE

You'll find the code files for this chapter at <https://github.com/luksa/kubernetes-in-action-2nd-edition/tree/master/Chapter11>.

Before you create the Service objects, deploy the pods and the other objects by applying the manifests in the Chapter11/SETUP/ directory as follows:

```
$ kubectl apply -f SETUP/ --recursive
```

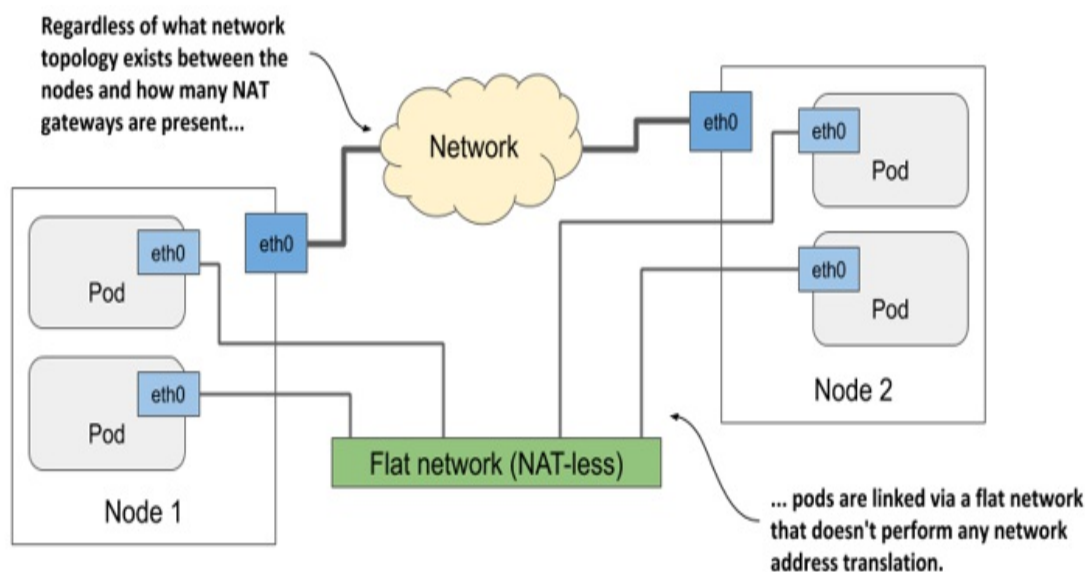
You may recall from the previous chapter that this command applies all manifests in the specified directory and its subdirectories. After applying these manifests, you should have multiple pods in your current Kubernetes namespace.

Understanding how pods communicate

You learned in chapter 5 what pods are, when to combine multiple containers into a pod, and how those containers communicate. But how do containers from different pods communicate?

Each pod has its own network interface with its own IP address. All pods in the cluster are connected by a single private network with a flat address space. As shown in the following figure, even if the nodes hosting the pods are geographically dispersed with many network routers in between, the pods can communicate over their own flat network where no *NAT* (Network Address Translation) is required. This pod network is typically a software-defined network that's layered on top of the actual network that connects the nodes.

Figure 11.2 Pods communicate via their own computer network



When a pod sends a network packet to another pod, neither SNAT (Source NAT) nor DNAT (Destination NAT) is performed on the packet. This means that the source IP and port, and the destination IP and port, of packets exchanged directly between pods are never changed. If the sending pod knows the IP address of the receiving pod, it can send packets to it. The receiving pod can see the sender's IP as the source IP address of the packet.

Although there are many Kubernetes network plugins, they must all behave as described above. Therefore, the communication between two pods is always the same, regardless of whether the pods are running on the same node or on nodes located in different geographic regions. The containers in the pods can communicate with each other over the flat NAT-less network, like computers on a local area network (LAN) connected to a single network

switch. From the perspective of the applications, the actual network topology between the nodes isn't important.

11.1 Exposing pods via services

If an application running in one pod needs to connect to another application running in a different pod, it needs to know the address of the other pod. This is easier said than done for the following reasons:

- Pods are *ephemeral*. A pod can be removed and replaced with a new one at any time. This happens when the pod is evicted from a node to make room for other pods, when the node fails, when the pod is no longer needed because a smaller number of pod replicas can handle the load, and for many other reasons.
- A pod gets its IP address when it's assigned to a node. You don't know the IP address of the pod in advance, so you can't provide it to the pods that will connect to it.
- In horizontal scaling, multiple pod replicas provide the same service. Each of these replicas has its own IP address. If another pod needs to connect to these replicas, it should be able to do so using a single IP or DNS name that points to a load balancer that distributes the load across all replicas.

Also, some pods need to be exposed to clients outside the cluster. Until now, whenever you wanted to connect to an application running in a pod, you used port forwarding, which is for development only. The right way to make a group of pods externally accessible is to use a Kubernetes Service.

11.1.1 Introducing services

A Kubernetes Service is an object you create to provide a single, stable access point to a set of pods that provide the same service. Each service has a stable IP address that doesn't change for as long as the service exists. Clients open connections to that IP address on one of the exposed network ports, and those connections are then forwarded to one of the pods that back that service. In this way, clients don't need to know the addresses of the individual pods providing the service, so those pods can be scaled out or in

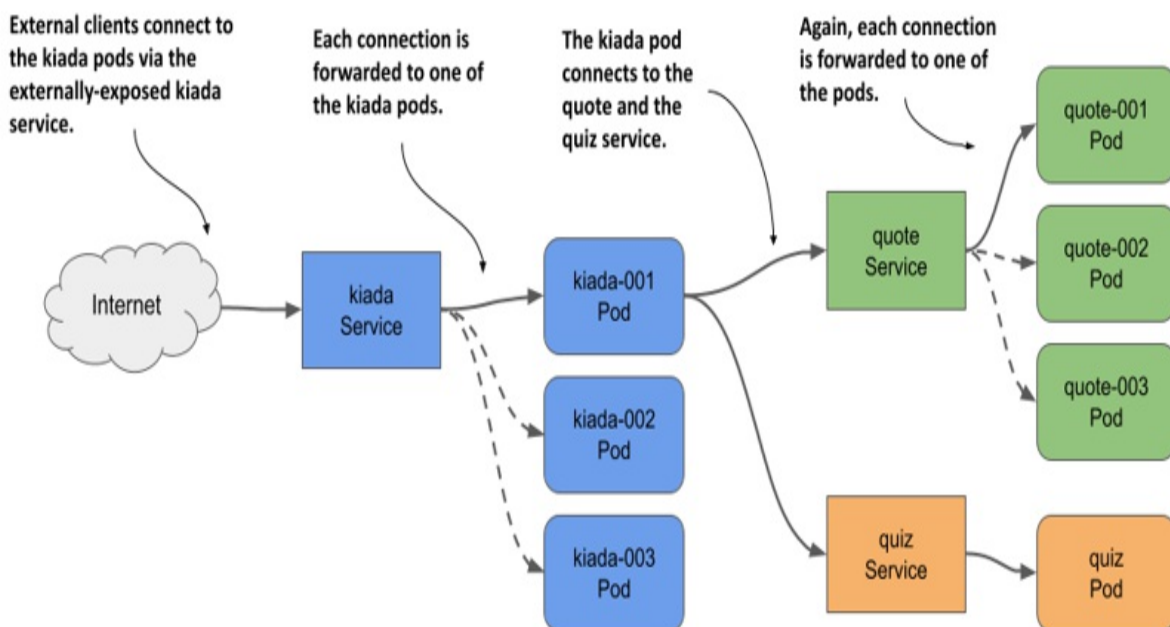
and moved from one cluster node to the other at will. A service acts as a load balancer in front of those pods.

Understanding why you need services

The Kiada suite is an excellent example to explain services. It contains three sets of pods that provide three different services. The Kiada service calls the Quote service to retrieve a quote from the book, and the Quiz service to retrieve a quiz question.

I've made the necessary changes to the Kiada application in version 0.5. You can find the updated source code in the `Chapter11/` directory in the book's code repository. You'll use this new version throughout this chapter. You'll learn how to configure the Kiada application to connect to the other two services, and you'll make it visible to the outside world. Since both the number of pods in each service and their IP addresses can change, you'll expose them via Service objects, as shown in the following figure.

Figure 11.3 Exposing pods with Service objects



By creating a service for the Kiada pods and configuring it to be reachable

from outside the cluster, you create a single, constant IP address through which external clients can connect to the pods. Each connection is forwarded to one of the kiada pods.

By creating a service for the Quote pods, you create a stable IP address through which the Kiada pods can reach the Quote pods, regardless of the number of pod instances behind the service and their location at any given time.

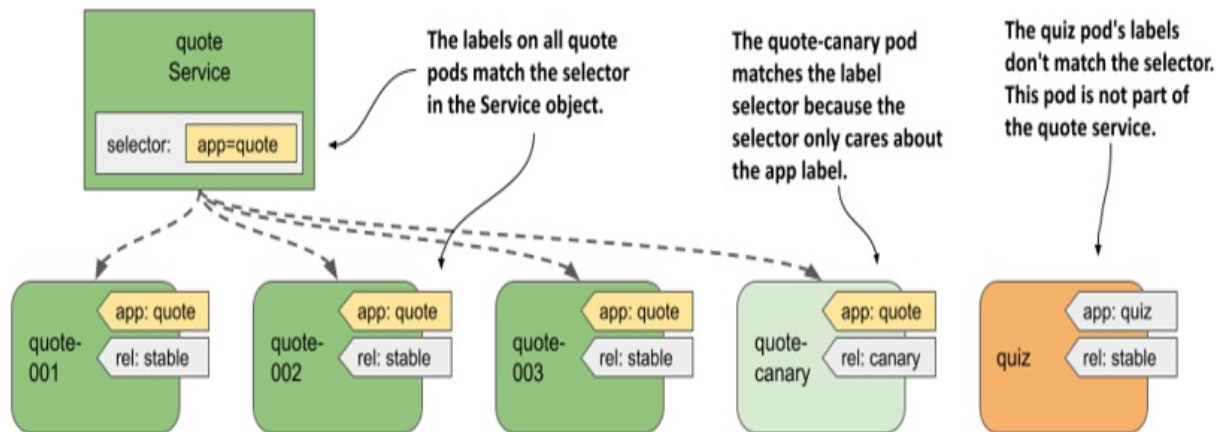
Although there's only one instance of the Quiz pod, it too must be exposed through a service, since the pod's IP address changes every time the pod is deleted and recreated. Without a service, you'd have to reconfigure the Kiada pods each time or make the pods get the Quiz pod's IP from the Kubernetes API. If you use a service, you don't have to do that because its IP address never changes.

Understanding how pods become part of a service

A service can be backed by more than one pod. When you connect to a service, the connection is passed to one of the backing pods. But how do you define which pods are part of the service and which aren't?

In the previous chapter, you learned about labels and label selectors and how they're used to organize a set of objects into subsets. Services use the same mechanism. As shown in the next figure, you add labels to Pod objects and specify the label selector in the Service object. The pods whose labels match the selector are part of the service.

Figure 11.4 Label selectors determine which pods are part of the Service.



The label selector defined in the quote service is `app=quote`, which means that it selects all quote pods, both stable and canary instances, since they all contain the label key `app` with the value `quote`. Other labels on the pods don't matter.

11.1.2 Creating and updating services

Kubernetes supports several types of services: `ClusterIP`, `NodePort`, `LoadBalancer`, and `ExternalName`. The `ClusterIP` type, which you'll learn about first, is only used internally, within the cluster. If you create a `Service` object without specifying its type, that's the type of service you get. The services for the Quiz and Quote pods are of this type because they're used by the Kiada pods within the cluster. The service for the Kiada pods, on the other hand, must also be accessible to the outside world, so the `ClusterIP` type isn't sufficient.

Creating a service YAML manifest

The following listing shows the minimal YAML manifest for the quote `Service` object.

Listing 11.1 YAML manifest for the quote service

```
apiVersion: v1      #A
kind: Service       #A
metadata:
```

```
  name: quote      #B
spec:
  type: ClusterIP   #C
  selector:         #D
    app: quote      #D
  ports:           #E
  - name: http      #E
    port: 80        #E
    targetPort: 80   #E
    protocol: TCP    #E
```

Note

Since the quote Service object is one of the objects that make up the Quote application, you could also add the `app: quote` label to this object. However, because this label isn't required for the service to function, it's omitted in this example.

Note

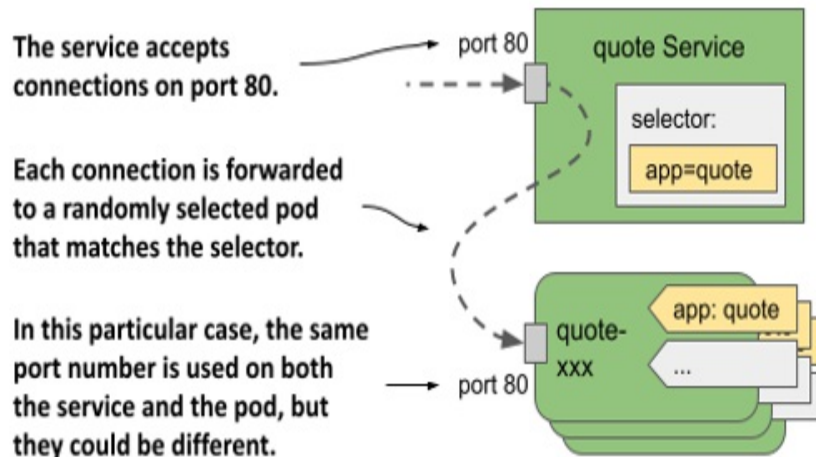
If you create a service with multiple ports, you must specify a name for each port. It's best to do the same for services with a single port.

Note

Instead of specifying the port number in the `targetPort` field, you can also specify the name of the port as defined in the container's port list in the pod definition. This allows the service to use the correct target port number even if the pods behind the service use different port numbers.

The manifest defines a `ClusterIP` Service named `quote`. The service accepts connections on port 80 and forwards each connection to port 80 of a randomly selected pod matching the `app=quote` label selector, as shown in the following figure.

Figure 11.5 The quote service and the pods that it forwards traffic to



To create the service, apply the manifest file to the Kubernetes API using `kubectl apply`.

Creating a service with `kubectl expose`

Normally, you create services like you create other objects, by applying an object manifest using `kubectl apply`. However, you can also create services using the `kubectl expose` command, as you did in chapter 3 of this book.

Create the service for the Quiz pod as follows:

```
$ kubectl expose pod quiz --name quiz  
service/quiz exposed
```

This command creates a service named `quiz` that exposes the `quiz` pod. To do this, it checks the pod's labels and creates a Service object with a label selector that matches all the pod's labels.

Note

In chapter 3, you used the `kubectl expose` command to expose a Deployment object. In this case, the command took the selector from the Deployment and used it in the Service object to expose all its pods. You'll learn about Deployments in chapter 13.

You've now created two services. You'll learn how to connect to them in section 11.1.3, but first let's see if they're configured correctly.

Listing services

When you create a service, it's assigned an internal IP address that any workload running in the cluster can use to connect to the pods that are part of that service. This is the cluster IP address of the service. You can see it by listing services with the `kubectl get services` command. If you want to see the label selector of each service, use the `-o wide` option as follows:

```
$ kubectl get svc -o wide
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
quiz	ClusterIP	10.96.136.190	<none>	8080/TCP	15s
quote	ClusterIP	10.96.74.151	<none>	80/TCP	23s

Note

The shorthand for services is `svc`.

The output of the command shows the two services you created. For each service, the type, IP addresses, exposed ports, and label selector are printed.

Note

You can also view the details of each service with the `kubectl describe svc` command.

You'll notice that the `quiz` service uses a label selector that selects pods with the labels `app: quiz` and `rel: stable`. This is because these are the labels of the `quiz` pod from which the service was created using the `kubectl expose` command.

Let's think about this. Do you want the `quiz` service to include only the stable pods? Probably not. Maybe later you decide to deploy a canary release of the `quiz` service in parallel with the stable version. In that case, you want traffic to be directed to both pods.

Another thing I don't like about the quiz service is the port number. Since the service uses HTTP, I'd prefer it to use port 80 instead of 8080. Fortunately, you can change the service after you create it.

Changing the service's label selector

To change the label selector of a service, you can use the `kubectl set selector` command. To fix the selector of the quiz service, run the following command:

```
$ kubectl set selector service quiz app=quiz
service/quiz selector updated
```

List the services again with the `-o wide` option to confirm the selector change. This method of changing the selector is useful if you're deploying multiple versions of an application and want to redirect clients from one version to another.

Changing the ports exposed by the service

To change the ports that the service forwards to pods, you can edit the Service object with the `kubectl edit` command or update the manifest file and then apply it to the cluster.

Before continuing, run `kubectl edit svc quiz` and change the port from 8080 to 80, making sure to only change the port field and leaving the `targetPort` set to 8080, as this is the port that the quiz pod listens on.

Configuring basic service properties

The following table lists the basic fields you can set in the Service object.

Table 11.1 Fields in the Service object's spec for configuring the service's basic properties

Field	Field type	Description
-------	------------	-------------

<code>type</code>	<code>string</code>	Specifies the type of this Service object. Allowed values are <code>ClusterIP</code> , <code>NodePort</code> , <code>LoadBalancer</code> , and <code>ExternalName</code> . The default value is <code>ClusterIP</code> . The differences between these types are explained in the following sections of this chapter.
<code>clusterIP</code>	<code>string</code>	The internal IP address within the cluster where the service is available. Normally, you leave this field blank and let Kubernetes assign the IP. If you set it to <code>None</code> , the service is a headless service. These are explained in section 11.4.
<code>selector</code>	<code>map[string]string</code>	Specifies the label keys and values that the pod must have in order for this service to forward traffic to it. If you don't set this field, you are responsible for managing the service endpoints. This is explained in section 11.3.
<code>ports</code>	<code>[]Object</code>	List of ports exposed by this service. Each entry can specify the name, protocol, appProtocol, port, nodePort, and targetPort.

Other fields are explained throughout the remainder of this chapter.

IPv4/IPv6 dual-stack support

Kubernetes supports both IPv4 and IPv6. Whether dual-stack networking is supported in your cluster depends on whether the `IPv6DualStack` feature gate is enabled for the cluster components to which it applies.

When you create a Service object, you can specify whether you want the service to be a single- or dual-stack service through the `ipFamilyPolicy` field. The default value is `SingleStack`, which means that only a single IP family is assigned to the service, regardless of whether the cluster is configured for single-stack or dual-stack networking. Set the value to `PreferDualStack` if you want the service to receive both IP families when the cluster supports dual-stack, and one IP family when it supports single-stack networking. If your service requires both an IPv4 and an IPv6 address, set the value to `RequireDualStack`. The creation of the service will be successful only on dual-stack clusters.

After you create the Service object, its `spec.ipFamilies` array indicates which IP families have been assigned to it. The two valid values are `IPv4` and `IPv6`. You can also set this field yourself to specify which IP family to assign to the service in clusters that provide dual-stack networking. The `ipFamilyPolicy` must be set accordingly or the creation will fail.

For dual-stack services, the `spec.clusterIP` field contains only one of the IP addresses, but the `spec.clusterIPs` field contains both the IPv4 and IPv6 addresses. The order of the IPs in the `clusterIPs` field corresponds to the order in the `ipFamilies` field.

11.1.3 Accessing cluster-internal services

The `ClusterIP` services you created in the previous section are accessible only within the cluster, from other pods and from the cluster nodes. You can't access them from your own machine. To see if a service is actually working, you must either log in to one of the nodes with `ssh` and connect to the service from there, or use the `kubectl exec` command to run a command like `curl` in an existing pod and get it to connect to the service.

Note

You can also use the `kubectl port-forward svc/my-service` command to connect to one of the pods backing the service. However, this command doesn't connect to the service. It only uses the Service object to find a pod to connect to. The connection is then made directly to the pod, bypassing the

service.

Connecting to services from pods

To use the service from a pod, run a shell in the `quote-001` pod as follows:

```
$ kubectl exec -it quote-001 -c nginx -- sh
/ #
```

Now check if you can access the two services. Use the cluster IP addresses of the services that `kubectl get services` displays. In my case, the `quiz` service uses cluster IP `10.96.136.190`, whereas the `quote` service uses IP `10.96.74.151`. From the `quote-001` pod, I can connect to the two services as follows:

```
/ # curl http://10.96.136.190      #A
This is the quiz service running in pod quiz

/ # curl http://10.96.74.151      #B
This is the quote service running in pod quote-canary
```

Note

You don't need to specify the port in the `curl` command, because you set the service port to 80, which is the default for HTTP.

If you repeat the last command several times, you'll see that the service forwards the request to a different pod each time:

```
/ # while true; do curl http://10.96.74.151; done
This is the quote service running in pod quote-canary
This is the quote service running in pod quote-003
This is the quote service running in pod quote-001
...
```

The service acts as a load balancer. It distributes requests to all the pods that are behind it.

Configuring session affinity on services

You can configure whether the service should forward each connection to a different pod, or whether it should forward all connections from the same client to the same pod. You do this via the `spec.sessionAffinity` field in the Service object. Only two types of service session affinity are supported: `None` and `ClientIP`.

The default type is `None`, which means there's no guarantee to which pod each connection will be forwarded. However, if you set the value to `ClientIP`, all connections originating from the same IP will be forwarded to the same pod. In the `spec.sessionAffinityConfig.clientIP.timeoutSeconds` field, you can specify how long the session will persist. The default value is 3 hours.

It may surprise you to learn that Kubernetes doesn't provide cookie-based session affinity. However, considering that Kubernetes services operate at the transport layer of the OSI network model (UDP and TCP) not at the application layer (HTTP), they don't understand HTTP cookies at all.

Resolving services via DNS

Kubernetes clusters typically run an internal DNS server that all pods in the cluster are configured to use. In most clusters, this internal DNS service is provided by CoreDNS, whereas some clusters use kube-dns. You can see which one is deployed in your cluster by listing the pods in the `kube-system` namespace.

No matter which implementation runs in your cluster, it allows pods to resolve the cluster IP address of a service by name. Using the cluster DNS, pods can therefore connect to the quiz service like so:

```
/ # curl http://quiz      #A
This is the quiz service running in pod quiz
```

A pod can resolve any service defined in the same namespace as the pod by simply pointing to the name of the service in the URL. If a pod needs to connect to a service in a different namespace, it must append the namespace of the Service object to the URL. For example, to connect to the quiz service in the `kiada` namespace, a pod can use the URL `http://quiz.kiada/`

regardless of which namespace it's in.

From the `quote-001` pod where you ran the shell command, you can also connect to the service as follows:

```
/ # curl http://quiz.kiada    #A
This is the quiz service running in pod quiz
```

A service is resolvable under the following DNS names:

- `<service-name>`, if the service is in the same namespace as the pod performing the DNS lookup,
- `<service-name>.<service-namespace>` from any namespace, but also under
- `<service-name>.<service-namespace>.svc`, and
- `<service-name>.<service-namespace>.svc.cluster.local`.

Note

The default domain suffix is `cluster.local` but can be changed at the cluster level.

The reason you don't need to specify the fully qualified domain name (FQDN) when resolving the service through DNS is because of the search line in the pod's `/etc/resolv.conf` file. For the `quote-001` pod, the file looks like this:

```
/ # cat /etc/resolv.conf
search kiada.svc.cluster.local svc.cluster.local cluster.local lo
nameserver 10.96.0.10
options ndots:5
```

When you try to resolve a service, the domain names specified in the search field are appended to the name until a match is found. If you're wondering what the IP address is in the `nameserver` line, you can list all the services in your cluster to find out:

```
$ kubectl get svc -A
```

NAMESPACE	NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
default	kubernetes	ClusterIP	10.96.0.1	<none>

kiada	quiz	ClusterIP	10.96.136.190	<none>
kiada	quote	ClusterIP	10.96.74.151	<none>
kube-system	kube-dns	ClusterIP	10.96.0.10	<none>

The nameserver in the pod's `resolv.conf` file points to the kube-dns service in the kube-system namespace. This is the cluster DNS service that the pods use. As an exercise, try to figure out which pod(s) this service forwards traffic to.

Configuring the pod's DNS policy

Whether or not a pod uses the internal DNS server can be configured using the `dnsPolicy` field in the pod's spec. The default value is `ClusterFirst`, which means that the pod uses the internal DNS first and then the DNS configured for the cluster node. Other valid values are `Default` (uses the DNS configured for the node), `None` (no DNS configuration is provided by Kubernetes; you must configure the pod's DNS settings using the `dnsConfig` field explained in the next paragraph), and `ClusterFirstWithHostNet` (for special pods that use the host's network instead of their own - this is explained later in the book).

Setting the `dnsPolicy` field affects how Kubernetes configures the pod's `resolv.conf` file. You can further customize this file through the pod's `dnsConfig` field. The `pod-with-dns-options.yaml` file in the book's code repository demonstrates the use of this field.

Discovering services through environment variables

Nowadays, virtually every Kubernetes cluster offers the cluster DNS service. In the early days, this wasn't the case. Back then, the pods found the IP addresses of the services using environment variables. These variables still exist today.

When a container is started, Kubernetes initializes a set of environment variables for each service that exists in the pod's namespace. Let's see what these environment variables look like by looking at the environment of one of your running pods.

Since you created your pods before the services, you won't see any environment variables related to the services except those for the kubernetes service, which exists in the default namespace.

Note

The kubernetes service forwards traffic to the API server. You'll use it in chapter 16.

To see the environment variables for the two services that you created, you must restart the container as follows:

```
$ kubectl exec quote-001 -c nginx -- kill 1
```

When the container is restarted, its environment variables contain the entries for the quiz and quote services. Display them with the following command:

```
$ kubectl exec -it quote-001 -c nginx -- env | sort
...
QUIZ_PORT_80_TCP_ADDR=10.96.136.190      #A
QUIZ_PORT_80_TCP_PORT=80                 #A
QUIZ_PORT_80_TCP_PROTO=tcp               #A
QUIZ_PORT_80_TCP=tcp://10.96.136.190:80  #A
QUIZ_PORT=tcp://10.96.136.190:80         #A
QUIZ_SERVICE_HOST=10.96.136.190          #A
QUIZ_SERVICE_PORT=80                     #A
QUOTE_PORT_80_TCP_ADDR=10.96.74.151      #B
QUOTE_PORT_80_TCP_PORT=80                #B
QUOTE_PORT_80_TCP_PROTO=tcp              #B
QUOTE_PORT_80_TCP=tcp://10.96.74.151:80  #B
QUOTE_PORT=tcp://10.96.74.151:80         #B
QUOTE_SERVICE_HOST=10.96.74.151          #B
QUOTE_SERVICE_PORT=80                    #B
```

Quite a handful of environment variables, wouldn't you say? For services with multiple ports, the number of variables is even larger. An application running in a container can use these variables to find the IP address and port(s) of a particular service.

NOTE

In the environment variable names, the hyphens in the service name are

converted to underscores and all letters are uppercased.

Nowadays, applications usually get this information through DNS, so these environment variables aren't as useful as in the early days. They can even cause problems. If the number of services in a namespace is too large, any pod you create in that namespace will fail to start. The container exits with exit code 1 and you see the following error message in the container's log:

```
standard_init_linux.go:228: exec user process caused: argument li
```

To prevent this, you can disable the injection of service information into the environment by setting the `enableServiceLinks` field in the pod's spec to `false`.

Understanding why you can't ping a service IP

You've learned how to verify that a service is forwarding traffic to your pods. But what if it doesn't? In that case, you might want to try pinging the service's IP. Why don't you try that right now? Ping the quiz service from the `quote-001` pod as follows:

```
$ kubectl exec -it quote-001 -c nginx -- ping quiz
PING quiz (10.96.136.190): 56 data bytes
^C
--- quiz ping statistics ---
15 packets transmitted, 0 packets received, 100% packet loss
command terminated with exit code 1
```

Wait a few seconds and then interrupt the process by pressing Control-C. As you can see, the IP address was resolved correctly, but none of the packets got through. This is because the IP address of the service is virtual and has meaning only in conjunction with one of the ports defined in the service. This is explained in chapter 18, which explains the internal workings of services. For now, remember that you can't ping services.

Using services in a pod

Now that you know that the Quiz and Quote services are accessible from pods, you can deploy the Kiada pods and configure them to use the two

services. The application expects the URLs of these services in the environment variables `QUIZ_URL` and `QUOTE_URL`. These aren't environment variables that Kubernetes adds on its own, but variables that you set manually so that the application knows where to find the two services. Therefore, the `env` field of the `kiada` container must be configured as in the following listing.

Listing 11.2 Configuring the service URLs in the `kiada` pod

```
...
  env:
  - name: QUOTE_URL      #A
    value: http://quote/quote    #A
  - name: QUIZ_URL      #B
    value: http://quiz    #B
  - name: POD_NAME
    ...
```

The environment variable `QUOTE_URL` is set to `http://quote/quote`. The hostname is the same as the name of the service you created in the previous section. Similarly, `QUIZ_URL` is set to `http://quiz`, where `quiz` is the name of the other service you created.

Deploy the Kiada pods by applying the manifest file `kiada-stable-and-canary.yaml` to your cluster using `kubectl apply`. Then run the following command to open a tunnel to one of the pods you just created:

```
$ kubectl port-forward kiada-001 8080 8443
```

You can now test the application at <http://localhost:8080> or <https://localhost:8443>. If you use `curl`, you should see a response like the following:

```
$ curl http://localhost:8080
==== TIP OF THE MINUTE
Kubectl options that take a value can be specified with an equal

==== POP QUIZ
First question
0) First answer
1) Second answer
2) Third answer
```


Submit your answer to `/question/1/answers/<index of answer>` using

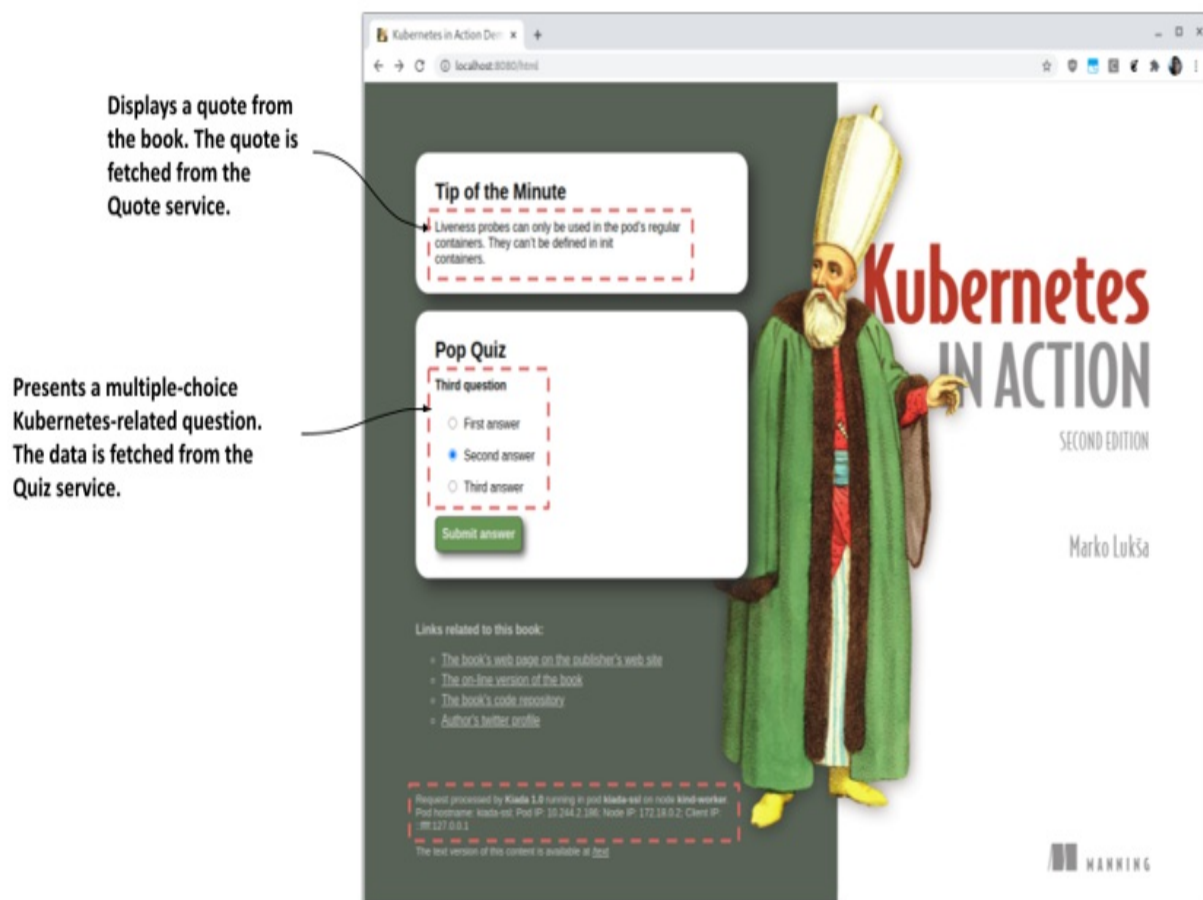
==== REQUEST INFO

Request processed by Kubiya 1.0 running in pod "kiada-001" on node
Pod hostname: kiada-001; Pod IP: 10.244.1.90; Node IP: 172.18.0.2

HTML version of this content is available at `/html`

If you open the URL in your web browser, you get the web page shown in the following figure.

Figure 11.6 The Kiada application when accessed with a web browser



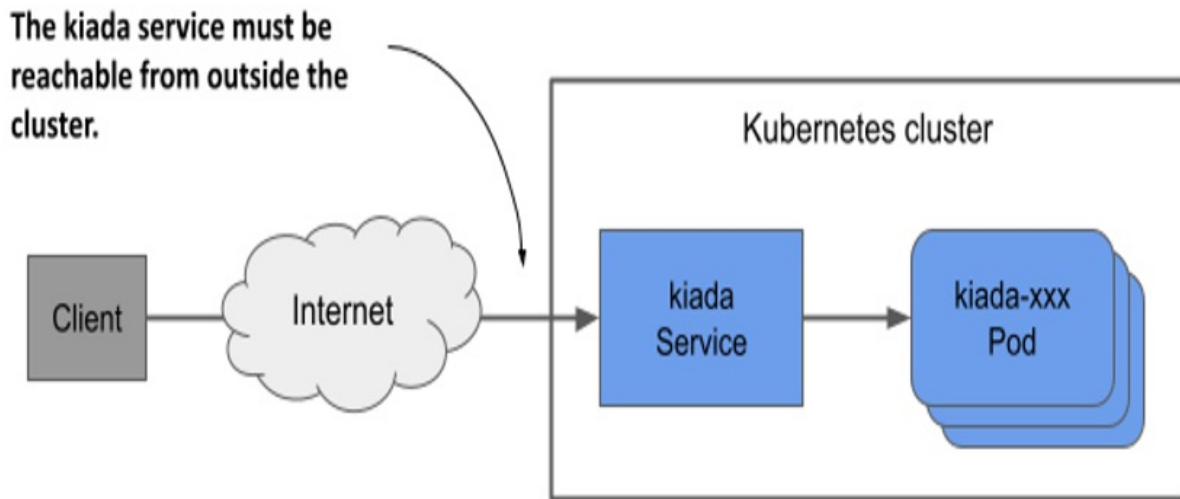
If you can see the quote and quiz question, it means that the `kiada-001` pod is able to communicate with the quote and quiz services. If you check the logs of the pods that back these services, you'll see that they are receiving requests. In the case of the quote service, which is backed by multiple pods,

you'll see that each request is sent to a different pod.

11.2 Exposing services externally

ClusterIP services like the ones you created in the previous section are only accessible within the cluster. Because clients must be able to access the Kiada service from outside the cluster, as shown in the next figure, creating a ClusterIP service won't suffice.

Figure 11.7 Exposing a service externally



If you need to make a service available to the outside world, you can do one of the following:

- assign an additional IP to a node and set it as one of the service's externalIPs,
- set the service's type to NodePort and access the service through the node's port(s),
- ask Kubernetes to provision a load balancer by setting the type to LoadBalancer, or
- expose the service through an Ingress object.

A rarely used method is to specify an additional IP in the `spec.externalIPs` field of the Service object. By doing this, you're telling Kubernetes to treat

any traffic directed to that IP address as traffic to be processed by the service. When you ensure that this traffic arrives at a node with the service's external IP as its destination, Kubernetes forwards it to one of the pods that back the service.

A more common way to make a service available externally is to set its type to `NodePort`. Kubernetes makes the service available on a network port on all cluster nodes (the so-called node port, from which this service type gets its name). Like `ClusterIP` services, the service gets an internal cluster IP, but is also accessible through the node port on each of the cluster nodes. Usually, you then provision an external load balancer that redirects traffic to these node ports. The clients can connect to your service via the load balancer's IP address.

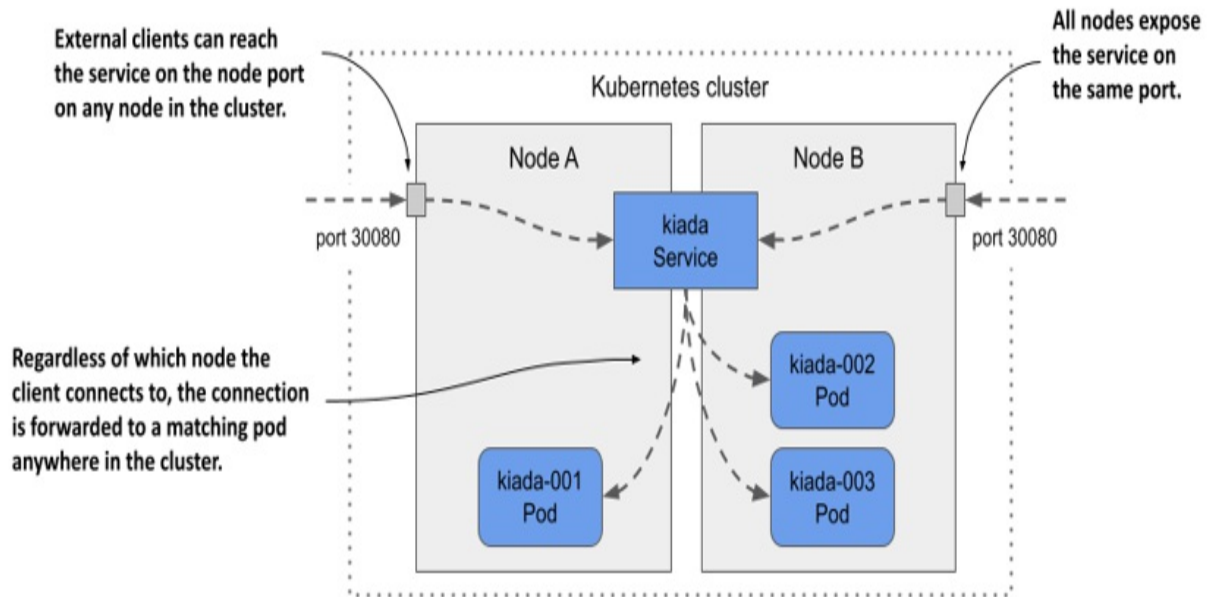
Instead of using a `NodePort` service and manually setting up the load balancer, Kubernetes can also do this for you if you set the service type to `LoadBalancer`. However, not all clusters support this service type, as the provisioning of the load balancer depends on the infrastructure the cluster is running on. Most cloud providers support `LoadBalancer` services in their clusters, whereas clusters deployed on premises require an add-on such as `MetalLB`, a load-balancer implementation for bare-metal Kubernetes clusters.

The final way to expose a group of pods externally is radically different. Instead of exposing the service externally via node ports and load balancers, you can use an `Ingress` object. How this object exposes the service depends on the underlying ingress controller, but it allows you to expose many services through a single externally reachable IP address. You'll learn more about this in the next chapter.

11.2.1 Exposing pods through a `NodePort` service

One way to make pods accessible to external clients is to expose them through a `NodePort` service. When you create such a service, the pods that match its selector are accessible through a specific port on all nodes in the cluster, as shown in the following figure. Because this port is open on the nodes, it's called a node port.

Figure 11.8 Exposing pods through a NodePort service



Like a ClusterIP service, a NodePort service is accessible through its internal cluster IP, but also through the node port on each of the cluster nodes. In the example shown in the figure, the pods are accessible through port 30080. As you can see, this port is open on both cluster nodes.

It doesn't matter which node a client connects to because all the nodes will forward the connection to a pod that belongs to the service, regardless of which node is running the pod. When the client connects to node A, a pod on either node A or B can receive the connection. The same is true when the client connects to the port on node B.

Creating a NodePort service

To expose the kiada pods through a NodePort service, you create the service from the manifest shown in the following listing.

Listing 11.3 A NodePort service exposing the kiada pods on two ports

```
apiVersion: v1
kind: Service
metadata:
```

```

name: kiada
spec:
  type: NodePort      #A
  selector:
    app: kiada
  ports:
    - name: http      #B
      port: 80        #C
      nodePort: 30080  #D
      targetPort: 8080 #E
    - name: https     #F
      port: 443        #F
      nodePort: 30443  #F
      targetPort: 8443 #F

```

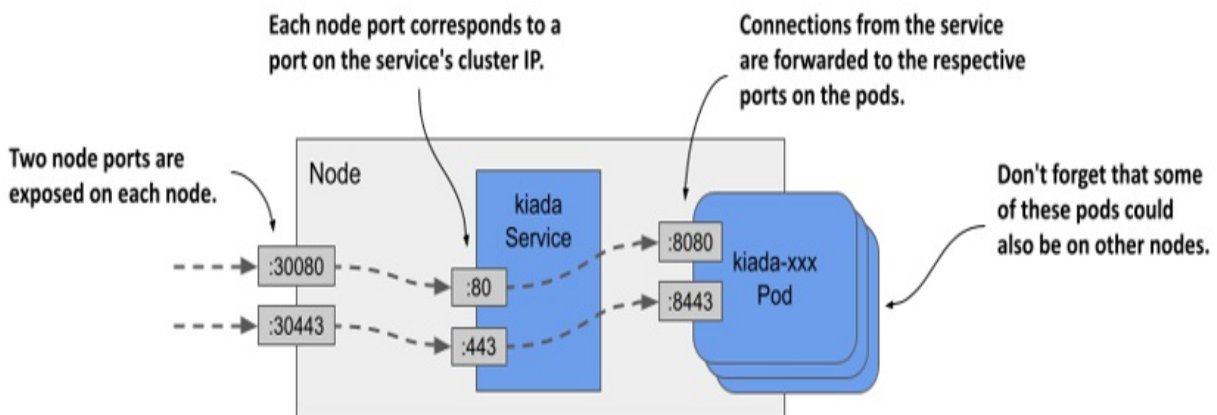
Compared to the `ClusterIP` services you created earlier the type of service in the listing is `NodePort`. Unlike the previous services, this service exposes two ports and defines the `nodePort` numbers for each of those ports.

Note

You can omit the `nodePort` field to allow Kubernetes to assign the port number. This prevents port conflicts between different `NodePort` services.

The service specifies six different port numbers, which might make it difficult to understand, but the following figure should help you make sense of it.

Figure 11.9 Exposing multiple ports through with a `NodePort` service



Examining your NodePort service

After you create the service, inspect it with the `kubectl get` command as follows:

```
$ kubectl get svc
NAME      TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)
kiada     NodePort    10.96.226.212    <none>           80:30080/TCP, 44
quiz      ClusterIP   10.96.173.186    <none>           80/TCP
quote     ClusterIP   10.96.161.97     <none>           80/TCP
```

Compare the `TYPE` and `PORT(S)` columns of the services you've created so far. Unlike the two `ClusterIP` services, the `kiada` service is a `NodePort` service that exposes node ports 30080 and 30443 in addition to ports 80 and 443 available on the service's cluster IP.

Accessing a NodePort service

To find out all `IP:port` combinations over which the service is available, you need not only the node port number(s), but also the IPs of the nodes. You can get these by running `kubectl get nodes -o wide` and looking at the `INTERNAL-IP` and `EXTERNAL-IP` columns. Clusters running in the cloud usually have the external IP set for the nodes, whereas clusters running on bare metal may set only the internal IP of the nodes. You should be able to reach the node ports using these IPs, if there are no firewalls in the way.

Note

To allow traffic to node ports when using GKE, run `gcloud compute firewall-rules create gke-allow-nodeports --allow=tcp:30000-32767`. If your cluster is running on a different cloud provider, check the provider's documentation on how to configure the firewall to allow access to node ports.

In the cluster I provisioned with the `kind` tool, the internal IPs of the nodes are as follows:

```
$ kubectl get nodes -o wide
NAME                STATUS    ROLES    ...    INTERN
```

kind-control-plane	Ready	control-plane,master	...	172.18
kind-worker	Ready	<none>	...	172.18
kind-worker2	Ready	<none>	...	172.18

The kiada service is available on all these IPs, even the IP of the node running the Kubernetes control plane. I can access the service at any of the following URLs:

- 10.96.226.212:80 within the cluster (this is the cluster IP and the internal port),
- 172.18.0.3:30080 from wherever the node kind-control-plane is reachable, as this is the node's IP address; the port is one of the node ports of the kiada service,
- 172.18.0.4:30080 (the second node's IP address and the node port), and
- 172.18.0.2:30080 (the third node's IP address and the node port).

The service is also accessible via HTTPS on port 443 within the cluster and via node port 30443. If my nodes also had external IPs, the service would also be available through the two node ports on those IPs. If you're using Minikube or another single-node cluster, you should use the IP of that node.

Tip

If you're using Minikube, you can easily access your NodePort services through your browser by running `minikube service <service-name> [-n <namespace>]`.

Use `curl` or your web browser to access the service. Select one of the nodes and find its IP address. Send the HTTP request to port 30080 of this IP. Check the end of the response to see which pod handled the request and which node the pod is running on. For example, here's the response I received to one of my requests:

```
$ curl 172.18.0.4:30080
...
==== REQUEST INFO
Request processed by KUBIA 1.0 running in pod "kiada-001" on node
Pod hostname: kiada-001; Pod IP: 10.244.1.90; Node IP: 172.18.0.2
```

Notice that I sent the request to the 172.18.0.4, which is the IP of the kind-worker node, but the pod that handled the request was running on the node kind-worker2. The first node forwarded the connection to the second node, as explained in the introduction to NodePort services.

Did you also notice where the pod thought the request came from? Look at the `Client IP` at the end of the response. That's not the IP of the computer from which I sent the request. You may have noticed that it's the IP of the node I sent the request to. I explain why this is and how you can prevent it in section 11.2.3.

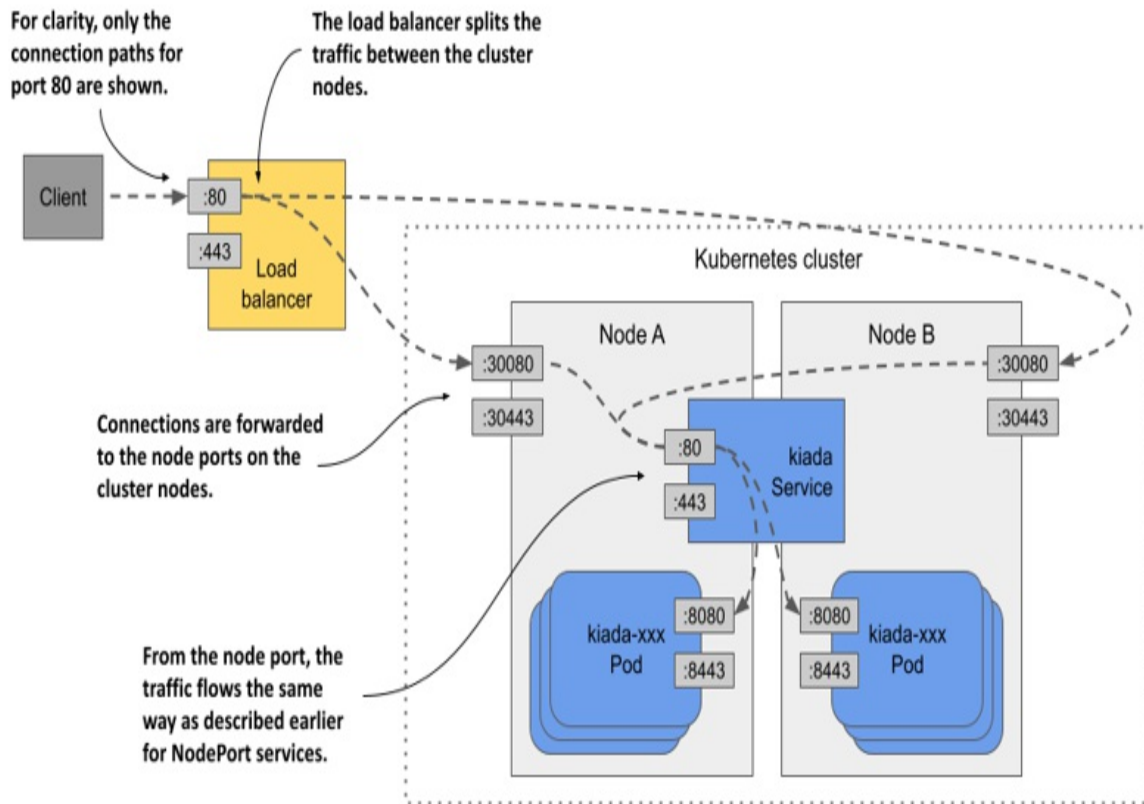
Try sending the request to the other nodes as well. You'll see that they all forward the requests to a random kiada pod. If your nodes are reachable from the internet, the application is now accessible to users all over the world. You could use round robin DNS to distribute incoming connections across the nodes or put a proper Layer 4 load balancer in front of the nodes and point the clients to it. Or you could just let Kubernetes do this, as explained in the next section.

11.2.2 Exposing a service through an external load balancer

In the previous section, you created a service of type `NodePort`. Another service type is `LoadBalancer`. As the name suggests, this service type makes your application accessible through a load balancer. While all services act as load balancers, creating a `LoadBalancer` service causes an actual load balancer to be provisioned.

As shown in the following figure, this load balancer stands in front of the nodes and handles the connections coming from the clients. It routes each connection to the service by forwarding it to the node port on one of the nodes. This is possible because the `LoadBalancer` service type is an extension of the `NodePort` type, which makes the service accessible through these node ports. By pointing clients to the load balancer rather than directly to the node port of a particular node, the client never attempts to connect to an unavailable node because the load balancer forwards traffic only to healthy nodes. In addition, the load balancer ensures that connections are distributed evenly across all nodes in the cluster.

Figure 11.10 Exposing a LoadBalancer service



Not all Kubernetes clusters support this type of service, but if your cluster runs in the cloud, it almost certainly does. If your cluster runs on premises, it'll support LoadBalancer services if you install an add-on. If the cluster doesn't support this type of service, you can still create services of this type, but the service is only accessible through its node ports.

Creating a LoadBalancer service

The manifest in the following listing contains the definition of a LoadBalancer service.

Listing 11.4 A LoadBalancer-type service

```
apiVersion: v1
kind: Service
metadata:
```

```

name: kiada
spec:
  type: LoadBalancer    #A
  selector:
    app: kiada
  ports:
    - name: http
      port: 80
      nodePort: 30080
      targetPort: 8080
    - name: https
      port: 443
      nodePort: 30443
      targetPort: 8443

```

This manifest differs from the manifest of the NodePort service you deployed earlier in only one line - the line that specifies the service type. The selector and ports are the same as before. The node ports are only specified so that they aren't randomly selected by Kubernetes. If you don't care about the node port numbers, you can omit the nodePort fields.

Apply the manifest with `kubectl apply`. You don't have to delete the existing kiada service first. This ensures that the internal cluster IP of the service remains unchanged.

Connecting to the service through the load balancer

After you create the service, it may take a few minutes for the cloud infrastructure to create the load balancer and update its IP address in the Service object. This IP address will then appear as the external IP address of your service:

```

$ kubectl get svc kiada
NAME      TYPE           CLUSTER-IP      EXTERNAL-IP      PORT(S)
kiada     LoadBalancer   10.96.226.212    172.18.255.200   80:30080/

```

In my case, the IP address of the load balancer is 172.18.255.200 and I can reach the service through port 80 and 443 of this IP. Until the load balancer is created, <pending> is displayed in the EXTERNAL-IP column instead of an IP address. This could be because the provisioning process isn't yet complete or because the cluster doesn't support LoadBalancer services.

Adding support for LoadBalancer services with MetalLB

If your cluster runs on bare metal, you can install MetalLB to support LoadBalancer services. You can find it at metallb.universe.tf. If you created your cluster with the kind tool, you can install MetalLB using the `install-metallb-kind.sh` script from the book's code repository. If you created your cluster with another tool, you can check the MetalLB documentation for how to install it.

Adding support for LoadBalancer services is optional. You can always use the node ports directly. The load balancer is just an additional layer.

Tweaking LoadBalancer services

LoadBalancer services are easy to create. You just set the type to LoadBalancer. However, if you need more control over the load balancer, you can configure it with the additional fields in the Service object's spec explained in the following table.

Table 11.2 Fields in the service spec that you can use to configure LoadBalancer services

Field	Field type	Description
loadBalancerClass	string	If the cluster supports multiple classes of load balancers, you can specify which one to use for this service. The possible values depend on the load balancer controllers installed in the cluster.
		If supported by the cloud provider, this field can be

<code>loadBalancerIP</code>	<code>string</code>	used to specify the desired IP for the load balancer.
<code>loadBalancerSourceRanges</code>	<code>[]string</code>	Restricts the client IPs that are allowed to access the service through the load balancer. Not supported by all load balancer controllers.
<code>allocateLoadBalancerNodePorts</code>	<code>boolean</code>	Specifies whether to allocate node ports for this LoadBalancer-type service. Some load balancer implementations can forward traffic to pods without relying on node ports.

11.2.3 Configuring the external traffic policy for a service

You've already learned that when an external client connects to a service through the node port, either directly or through the load balancer, the connection may be forwarded to a pod that's on a different node than the one that received the connection. In this case, an additional network hop must be made to reach the pod. This results in increased latency.

Also, as mentioned earlier, when forwarding the connection from one node to another in this manner, the source IP must be replaced with the IP of the node that originally received the connection. This obscures the IP address of the client. Thus, the application running in the pod can't see where the connection is coming from. For example, a web server running in a pod can't record the true client IP in its access log.

The reason the node needs to change the source IP is to ensure that the

returned packets are sent back to the node that originally received the connection so that it can return them to the client.

Pros and cons of the Local external traffic policy

Both the additional network hop problem and the source IP obfuscation problem can be solved by preventing nodes from forwarding traffic to pods that aren't running on the same node. This is done by setting the `externalTrafficPolicy` field in the Service object's `spec` field to `Local`. This way, a node forwards external traffic only to pods running on the node that received the connection.

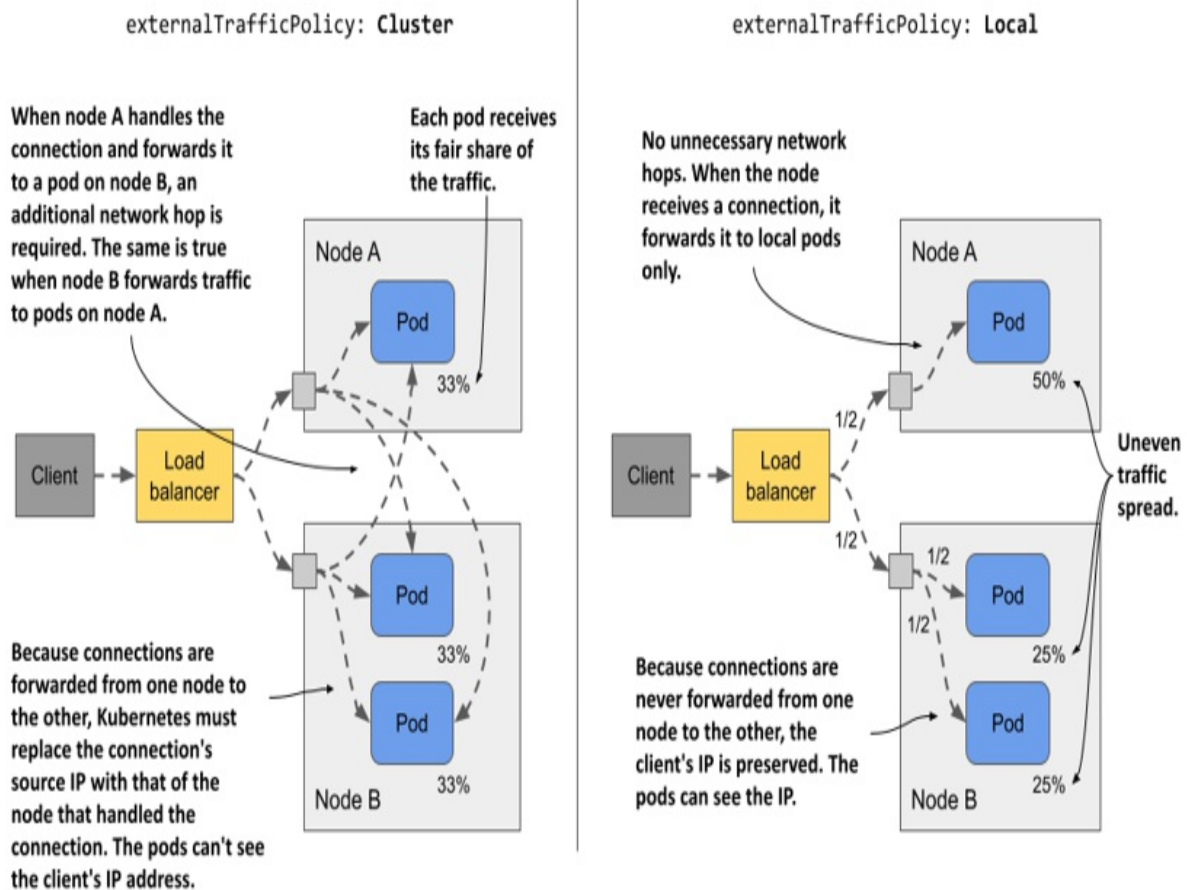
However, setting the external traffic policy to `Local` leads to other problems. First, if there are no local pods on the node that received the connection, the connection hangs. You must therefore ensure that the load balancer forwards connections only to nodes that have at least one such pod. This is done using the `healthCheckNodePort` field. The external load balancer uses this node port to check whether a node contains endpoints for the service or not. This allows the load balancer to forward traffic only to nodes that have such a pod.

The second problem you run into when the external traffic policy is set to `Local` is the uneven distribution of traffic across pods. If the load balancers distribute traffic evenly among the nodes, but each node runs a different number of pods, the pods on the nodes with fewer pods will receive a higher amount of traffic.

Comparing the Cluster and the Local external traffic policies

Consider the case presented in the following figure. There's one pod running on node A and two on node B. The load balancer routes half of the traffic to node A and the other half to node B.

Figure 11.11 Understanding the two external traffic policies for NodePort and LoadBalancer services



When `externalTrafficPolicy` is set to `Cluster`, each node forwards traffic to all pods in the system. Traffic is split evenly between the pods. Additional network hops are required, and the client IP is obfuscated.

When the `externalTrafficPolicy` is set to `Local`, all traffic arriving at node A is forwarded to the single pod on that node. This means that this pod receives 50% of all traffic. Traffic arriving at node B is split between two pods. Each pod receives 25% of the total traffic processed by the load balancer. There are no unnecessary network hops, and the source IP is that of the client.

As with most decisions you make as an engineer, which external traffic policy to use in each service depends on what tradeoffs you're willing to make.

11.3 Managing service endpoints

So far you've learned that services are backed by pods, but that's not always the case. The endpoints to which a service forwards traffic can be anything that has an IP address.

11.3.1 Introducing the Endpoints object

A service is typically backed by a set of pods whose labels match the label selector defined in the Service object. Apart from the label selector, the Service object's spec or status section doesn't contain the list of pods that are part of the service. However, if you use `kubectl describe` to inspect the service, you'll see the IPs of the pods under Endpoints, as follows:

```
$ kubectl describe svc kiada
Name:                kiada
...
Port:                http 80/TCP
TargetPort:          8080/TCP
NodePort:            http 30080/TCP
Endpoints:           10.244.1.7:8080,10.244.1.8:8080,10.244.1.9:8080
...
```

The `kubectl describe` command collects this data not from the Service object, but from an Endpoints object whose name matches that of the service. The endpoints of the `kiada` service are specified in the `kiada` Endpoints object.

Listing Endpoints objects

You can retrieve Endpoints objects in the current namespace as follows:

```
$ kubectl get endpoints
NAME      ENDPOINTS
kiada     10.244.1.7:8443,10.244.1.8:8443,10.244.1.9:8443 + 5 more.
quiz      10.244.1.11:8080
quote     10.244.1.10:80,10.244.2.10:80,10.244.2.8:80 + 1 more...
```

Note

The shorthand for endpoints is `ep`. Also, the object kind is `Endpoints` (plural form) not `Endpoint`. Running `kubectl get endpoint` fails with an error.

As you can see, there are three `Endpoints` objects in the namespace. One for each service. Each `Endpoints` object contains a list of IP and port combinations that represent the endpoints for the service.

Inspecting an `Endpoints` object more closely

To see which pods represent these endpoints, use `kubectl get -o yaml` to retrieve the full manifest of the `Endpoints` object as follows:

```
$ kubectl get ep kiada -o yaml
apiVersion: v1
kind: Endpoints
metadata:
  name: kiada      #A
  namespace: kiada  #A
...
subsets:
- addresses:
  - ip: 10.244.1.7    #B
    nodeName: kind-worker  #C
    targetRef:
      kind: Pod
      name: kiada-002    #D
      namespace: kiada  #D
      resourceVersion: "2950"
      uid: 18cea623-0818-4ff1-9fb2-cddcf5d138c3
    ...          #E
  ports:      #F
  - name: https    #F
    port: 8443     #F
    protocol: TCP  #F
  - name: http     #F
    port: 8080     #F
    protocol: TCP  #F
```

As you can see, each pod is listed as an element of the `addresses` array. In the `kiada` `Endpoints` object, all endpoints are in the same endpoint subset, because they all use the same port numbers. However, if one group of pods uses port 8080, for example, and another uses port 8088, the `Endpoints` object would contain two subsets, each with its own ports.

Understanding who manages the Endpoints object

You didn't create any of the three Endpoints objects. They were created by Kubernetes when you created the associated Service objects. These objects are fully managed by Kubernetes. Each time a new pod appears or disappears that matches the Service's label selector, Kubernetes updates the Endpoints object to add or remove the endpoint associated with the pod. You can also manage a service's endpoints manually. You'll learn how to do that later.

11.3.2 Introducing the EndpointSlice object

As you can imagine, the size of an Endpoints object becomes an issue when a service contains a very large number of endpoints. Kubernetes control plane components need to send the entire object to all cluster nodes every time a change is made. In large clusters, this leads to noticeable performance issues. To counter this, the EndpointSlice object was introduced, which splits the endpoints of a single service into multiple slices.

While an Endpoints object contains multiple endpoint subsets, each EndpointSlice contains only one. If two groups of pods expose the service on different ports, they appear in two different EndpointSlice objects. Also, an EndpointSlice object supports a maximum of 1000 endpoints, but by default Kubernetes only adds up to 100 endpoints to each slice. The number of ports in a slice is also limited to 100. Therefore, a service with hundreds of endpoints or many ports can have multiple EndpointSlices objects associated with it.

Like Endpoints, EndpointSlices are created and managed automatically.

Listing EndpointSlice objects

In addition to the Endpoints objects, Kubernetes creates the EndpointSlice objects for your three services. You can see them with the `kubectl get endpointslices` command:

```
$ kubectl get endpointslices
NAME                ADDRESSTYPE  PORTS          ENDPOINTS
kiada-m24zq         IPv4         8080,8443      10.244.1.7,10.244.1.8,10.
```

quiz-qbckq	IPv4	8080	10.244.1.11
quote-5dqhx	IPv4	80	10.244.2.8,10.244.1.10,10.

Note

As of this writing, there is no shorthand for endpointslices.

You'll notice that unlike Endpoints objects, whose names match the names of their respective Service objects, each EndpointSlice object contains a randomly generated suffix after the service name. This way, many EndpointSlice objects can exist for each service.

Listing EndpointSlices for a particular service

To see only the EndpointSlice objects associated with a particular service, you can specify a label selector in the `kubectl get` command. To list the EndpointSlice objects associated with the `kiada` service, use the label selector `kubernetes.io/service-name=kiada` as follows:

```
$ kubectl get endpointslices -l kubernetes.io/service-name=kiada
NAME          ADDRESSTYPE  PORTS          ENDPOINTS
kiada-m24zq   IPv4         8080,8443     10.244.1.7,10.244.1.8,10.
```

Inspecting an EndpointSlice

To examine an EndpointSlice object in more detail, you use `kubectl describe`. Since the `describe` command doesn't require the full object name, and all EndpointSlice objects associated with a service begin with the service name, you can see them all by specifying only the service name, as shown here:

```
$ kubectl describe endpointslice kiada
Name:          kiada-m24zq
Namespace:     kiada
Labels:        endpointslice.kubernetes.io/managed-by=endpointslic
               kubernetes.io/service-name=kiada
Annotations:   endpoints.kubernetes.io/last-change-trigger-time: 2
AddressType:   IPv4
Ports:         #A
  Name      Port  Protocol  #A
```

```

-----      #A
http    8080   TCP    #A
https   8443   TCP    #A
Endpoints:
- Addresses: 10.244.1.7      #B
  Conditions:
    Ready:    true
    Hostname:  <unset>
    TargetRef: Pod/kiada-002    #C
    Topology:  kubernetes.io/hostname=kind-worker    #D
...

```

Note

If multiple EndpointSlices match the name you provide to `kubectl describe`, the command will print all of them.

The information in the output of the `kubectl describe` command isn't much different from the information in the Endpoint object you saw earlier. The EndpointSlice object contains a list of ports and endpoint addresses, as well as information about the pods that represent those endpoints. This includes the pod's topology information, which is used for topology-aware traffic routing. You'll learn about it later in this chapter.

11.3.3 Managing service endpoints manually

When you create a Service object with a label selector, Kubernetes automatically creates and manages the Endpoints and EndpointSlice objects and uses the selector to determine the service endpoints. However, you can also manage endpoints manually by creating the Service object without a label selector. In this case, you must create the Endpoints object yourself. You don't need to create the EndpointSlice objects because Kubernetes mirrors the Endpoints object to create corresponding EndpointSlices.

Typically, you manage service endpoints this way when you want to make an existing external service accessible to pods in your cluster under a different name. This way, the service can be found through the cluster DNS and environment variables.

Creating a service without a label selector

The following listing shows an example of a Service object manifest that doesn't define a label selector. You'll manually configure the endpoints for this service.

Listing 11.5 A service with no pod selector

```
apiVersion: v1
kind: Service
metadata:
  name: external-service      #A
spec:      #B
  ports:   #B
  - name: http      #B
    port: 80      #B
```

The manifest in the listing defines a service named `external-service` that accepts incoming connections on port 80. As explained in the first part of this chapter, pods in the cluster can use the service either through its cluster IP address, which is assigned when you create the service, or through its DNS name.

Creating an Endpoints object

If a service doesn't define a pod selector, no Endpoints object is automatically created for it. You must do this yourself. The following listing shows the manifest of the Endpoints object for the service you created in the previous section.

Listing 11.6 An Endpoints object created by hand

```
apiVersion: v1
kind: Endpoints
metadata:
  name: external-service      #A
subsets:
- addresses:
  - ip: 1.1.1.1      #B
  - ip: 2.2.2.2      #B
  ports:
  - name: http      #C
    port: 88      #C
```

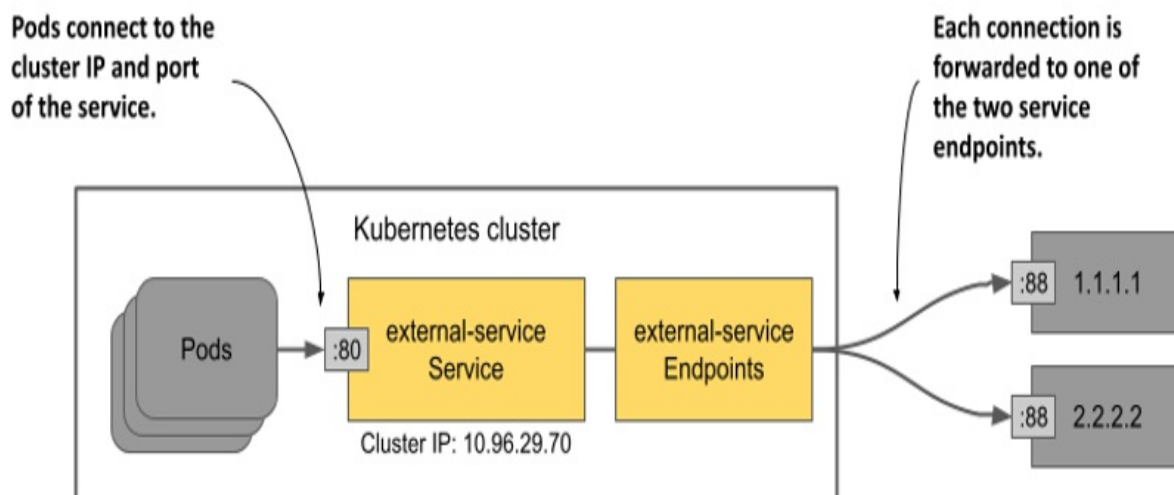
The Endpoints object must have the same name as the service and contain the list of destination addresses and ports. In the listing, IP addresses 1.1.1.1 and 2.2.2.2 represent the endpoints for the service.

Note

You don't have to create the EndpointSlice object. Kubernetes creates it from the Endpoints object.

The creation of the Service and its associated Endpoints object allows pods to use this service in the same way as other services defined in the cluster. As shown in the following figure, traffic sent to the service's cluster IP is distributed to the service's endpoints. These endpoints are outside the cluster but could also be internal.

Figure 11.12 Pods consuming a service with two external endpoints.



If you later decide to migrate the external service to pods running inside the Kubernetes cluster, you can add a selector to the service to redirect traffic to those pods instead of the endpoints you configured by hand. This is because Kubernetes immediately starts managing the Endpoints object after you add the selector to the service.

You can also do the opposite: If you want to migrate an existing service from the cluster to an external location, remove the selector from the Service

object so that Kubernetes no longer updates the associated Endpoints object. From then on, you can manage the service's endpoints manually.

You don't have to delete the service to do this. By changing the existing Service object, the cluster IP address of the service remains constant. The clients using the service won't even notice that you've relocated the service.

11.4 Understanding DNS records for Service objects

An important aspect of Kubernetes services is the ability to look them up via DNS. This is something that deserves to be looked at more closely.

You know that a service is assigned an internal cluster IP address that pods can resolve through the cluster DNS. This is because each service gets an A record in DNS (or an AAAA record for IPv6). However, a service also receives an SRV record for each of the ports it makes available.

Let's take a closer look at these DNS records. First, run a one-off pod like this:

```
$ kubectl run -it --rm dns-test --image=giantswarm/tiny-tools  
/ #
```

This command runs a pod named `dns-test` with a container based on the container image `giantswarm/tiny-tools`. This image contains the `host`, `nslookup`, and `dig` tools that you can use to examine DNS records. When you run the `kubectl run` command, your terminal will be attached to the shell process running in the container (the `-it` option does this). When you exit the shell, the pod will be removed (by the `--rm` option).

11.4.1 Inspecting a service's A and SRV records in DNS

You start by inspecting the A and SRV records associated with your services.

Looking up a service's A record

To determine the IP address of the `quote` service, you run the `nslookup`

command in the shell running in the container of the dns-test pod like so:

```
/ # nslookup quote
Server:      10.96.0.10
Address:     10.96.0.10#53 //

Name:   quote.kiada.svc.cluster.local    #A
Address: 10.96.161.97                    #B
```

Note

You can use dig instead of nslookup, but you must either use the +search option or specify the fully qualified domain name of the service for the DNS lookup to succeed (run either dig +search quote or dig quote.kiada.svc.cluster.local).

Now look up the IP address of the kiada service. Although this service is of type LoadBalancer and thus has both an internal cluster IP and an external IP (that of the load balancer), the DNS returns only the cluster IP. This is to be expected since the DNS server is internal and is only used within the cluster.

Looking up SRV records

A service provides one or more ports. Each port is given an SRV record in DNS. Use the following command to retrieve the SRV records for the kiada service:

```
/ # nslookup -query=SRV kiada
Server:      10.96.0.10
Address:     10.96.0.10#53 // //

kiada.kiada.svc.cluster.local    service = 0 50 80 kiada.kiada.svc
kiada.kiada.svc.cluster.local    service = 0 50 443 kiada.kiada.sv
```

Note

As of this writing, GKE still runs kube-dns instead of CoreDNS. Kube-dns doesn't support all the DNS queries shown in this section.

A smart client running in a pod could look up the SRV records of a service to

find out what ports are provided by the service. If you define the names for those ports in the Service object, they can even be looked up by name. The SRV record has the following form:

```
_port-name._port-protocol.service-name.namespace.svc.cluster.local
```

The names of the two ports in the kiada service are http and https, and both define TCP as the protocol. To get the SRV record for the http port, run the following command:

```
/ # nslookup -query=SRV _http._tcp.kiada
Server:      10.96.0.10
Address:     10.96.0.10#53 //
```

```
_http._tcp.kiada.kiada.svc.cluster.local      service = 0 100 8
```

Tip

To list all services and the ports they expose in the kiada namespace, you can run the command `nslookup -query=SRV any.kiada.svc.cluster.local`. To list all services in the cluster, use the name `any.any.svc.cluster.local`.

You'll probably never need to look for SRV records, but some Internet protocols, such as SIP and XMPP, depend on them to work.

Note

Please leave the shell in the `dns-test` pod running, because you'll need it in the exercises in the next section when you learn about headless services.

11.4.2 Using headless services to connect to pods directly

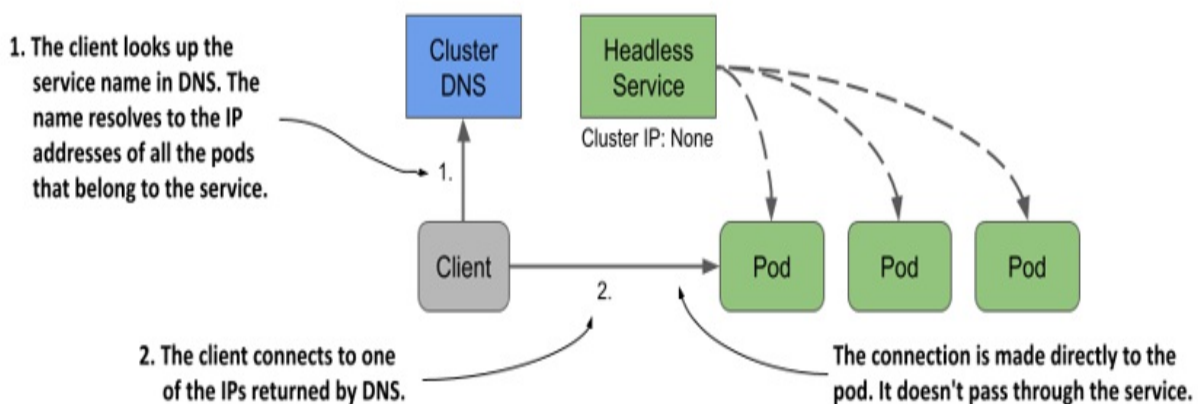
Services expose a set of pods at a stable IP address. Each connection to that IP address is forwarded to a random pod or other endpoint that backs the service. Connections to the service are automatically distributed across its endpoints. But what if you want the client to do the load balancing? What if the client needs to decide which pod to connect to? Or what if it needs to connect to all pods that back the service? What if the pods that are part of a service all need to connect directly to each other? Connecting via the

service's cluster IP clearly isn't the way to do this. What then?

Instead of connecting to the service IP, clients could get the pod IPs from the Kubernetes API, but it's better to keep them Kubernetes-agnostic and use standard mechanisms like DNS. Fortunately, you can configure the internal DNS to return the pod IPs instead of the service's cluster IP by creating a *headless* service.

For headless services, the cluster DNS returns not just a single A record pointing to the service's cluster IP, but multiple A records, one for each pod that's part of the service. Clients can therefore query the DNS to get the IPs of all the pods in the service. With this information, the client can then connect directly to the pods, as shown in the next figure.

Figure 11.13 With headless services, clients connect directly to the pods



Creating a headless service

To create a headless service, you set the `clusterIP` field to `None`. Create another service for the quote pods but make this one headless. The following listing shows its manifest:

Listing 11.7 A headless service

```
apiVersion: v1
kind: Service
```

```

metadata:
  name: quote-headless
spec:
  clusterIP: None      #A
  selector:
    app: quote
  ports:
    - name: http
      port: 80
      targetPort: 80
      protocol: TCP

```

After you create the service with `kubectl apply`, you can check it with `kubectl get`. You'll see that it has no cluster IP:

```

$ kubectl get svc quote-headless -o wide
NAME                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)
quote-headless      ClusterIP     None          <none>         80/TCP

```

Because the service doesn't have a cluster IP, the DNS server can't return it when you try to resolve the service name. Instead, it returns the IP addresses of the pods. Before you continue, list the IPs of the pods that match the service's label selector as follows:

```

$ kubectl get po -l app=quote -o wide
NAME                READY    STATUS    RESTARTS    AGE    IP                N
quote-canary        2/2     Running    0           3h     10.244.2.9        k
quote-001           2/2     Running    0           3h     10.244.2.10       k
quote-002           2/2     Running    0           3h     10.244.2.8        k
quote-003           2/2     Running    0           3h     10.244.1.10       k

```

Note the IP addresses of these pods.

Understanding DNS A records returned for a headless service

To see what the DNS returns when you resolve the service, run the following command in the `dns-test` pod you created in the previous section:

```

/ # nslookup quote-headless
Server:      10.96.0.10
Address:     10.96.0.10#53 //

Name:       quote-headless.kiada.svc.cluster.local

```

```
Address: 10.244.2.9      #A
Name:    quote-headless.kiada.svc.cluster.local
Address: 10.244.2.8      #B
Name:    quote-headless.kiada.svc.cluster.local
Address: 10.244.2.10     #C
Name:    quote-headless.kiada.svc.cluster.local
Address: 10.244.1.10     #D
```

The DNS server returns the IP addresses of the four pods that match the service's label selector. This is different from what DNS returns for regular (non-headless) services such as the quote service, where the name resolves to the cluster IP of the service:

```
/ # nslookup quote
Server:      10.96.0.10
Address:     10.96.0.10#53 //

Name:    quote.kiada.svc.cluster.local
Address: 10.96.161.97      #A
```

Understanding how clients use headless services

Clients that wish to connect directly to pods that are part of a service, can do so by retrieving the A (or AAAA) records from the DNS. The client can then connect to one, some, or all the returned IP addresses.

Clients that don't perform the DNS lookup themselves, can use the service as they'd use a regular, non-headless service. Because the DNS server rotates the list of IP addresses it returns, a client that simply uses the service's FQDN in the connection URL will get a different pod IP each time. Therefore, client requests are distributed across all pods.

You can try this by sending multiple requests the quote-headless service with `curl` from the `dns-test` pod as follows:

```
/ # while true; do curl http://quote-headless; done
This is the quote service running in pod quote-002
This is the quote service running in pod quote-001
This is the quote service running in pod quote-002
This is the quote service running in pod quote-canary
...
```

Each request is handled by a different pod, just like when you use the regular service. The difference is that with a headless service you connect directly to the pod IP, while with regular services you connect to the cluster IP of the service, and your connection is forwarded to one of the pods. You can see this by running `curl` with the `--verbose` option and examining the IP it connects to:

```
/ # curl --verbose http://quote-headless    #A
*   Trying 10.244.1.10:80...                #A
* Connected to quote-headless (10.244.1.10) port 80 (#0)
...

/ # curl --verbose http://quote            #B
*   Trying 10.96.161.97:80...              #B
* Connected to quote (10.96.161.97) port 80 (#0)
...
```

Headless services with no label selector

To conclude this section on headless services, I'd like to mention that services with manually configured endpoints (services without a label selector) can also be headless. If you omit the label selector and set the `clusterIP` to `None`, the DNS will return an `A/AAAA` record for each endpoint, just as it does when the service endpoints are pods. To test this yourself, apply the manifest in the `svc.external-service-headless.yaml` file and run the following command in the `dns-test` pod:

```
/ # nslookup external-service-headless
```

11.4.3 Creating a CNAME alias for an existing service

In the previous sections, you learned how to create `A` and `AAAA` records in the cluster DNS. To do this, you create `Service` objects that either specify a label selector to find the service endpoints, or you define them manually using the `Endpoints` and `EndpointSlice` objects.

There's also a way to add `CNAME` records to the cluster DNS. In Kubernetes, you add `CNAME` records to DNS by creating a `Service` object, just as you do for `A` and `AAAA` records.

Note

A CNAME record is a DNS record that maps an alias to an existing DNS name instead of an IP address.

Creating an ExternalName service

To create a service that serves as an alias for an existing service, whether it exists inside or outside the cluster, you create a Service object whose type field is set to ExternalName. The following listing shows an example of this type of service.

Listing 11.8 An ExternalName-type service

```
apiVersion: v1
kind: Service
metadata:
  name: time-api
spec:
  type: ExternalName      #A
  externalName: worldtimeapi.org    #B
```

In addition to setting the type to ExternalName, the service manifest must also specify in the externalName field external name to which this service resolves. No Endpoints or EndpointSlice object is required for ExternalName services.

Connecting to an ExternalName service from a pod

After the service is created, pods can connect to the external service using the domain name `time-api.<namespace>.svc.cluster.local` (or `time-api` if they're in the same namespace as the service) instead of using the actual FQDN of the external service, as shown in the following example:

```
$ kubectl exec -it kiada-001 -c kiada -- curl http://time-api/api
```

Resolving ExternalName services in DNS

Because ExternalName services are implemented at the DNS level (only a CNAME record is created for the service), clients don't connect to the service through the cluster IP, as is the case with non-headless ClusterIP services. They connect directly to the external service. Like headless services, ExternalName services have no cluster IP, as the following output shows:

```
$ kubectl get svc time-api
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
time-api	ExternalName	<none>	worldtimeapi.org	80/TCP

As a final exercise in this section on DNS, you can try resolving the time-api service in the dns-test pod as follows:

```
/ # nslookup time-api
Server:      10.96.0.10
Address:     10.96.0.10#53 //

time-api.kiada.svc.cluster.local      canonical name = worldtim
Name:   worldtimeapi.org              #B
Address: 213.188.196.246              #B
Name:   worldtimeapi.org              #B
Address: 2a09:8280:1::3:e             #B
```

You can see that time-api.kiada.svc.cluster.local points to worldtimeapi.org. This concludes this section on DNS records for Kubernetes services. You can now exit the shell in the dns-test pod by typing exit or pressing Control-D. The pod is deleted automatically.

11.5 Configuring services to route traffic to nearby endpoints

When you deploy pods, they are distributed across the nodes in the cluster. If cluster nodes span different availability zones or regions and the pods deployed on those nodes exchange traffic with each other, network performance and traffic costs can become an issue. In this case, it makes sense for services to forward traffic to pods that aren't far from the pod where the traffic originates.

In other cases, a pod may need to communicate only with service endpoints

on the same node as the pod. Not for performance or cost reasons, but because only the node-local endpoints can provide the service in the proper context. Let me explain what I mean.

11.5.1 Forwarding traffic only within the same node with `internalTrafficPolicy`

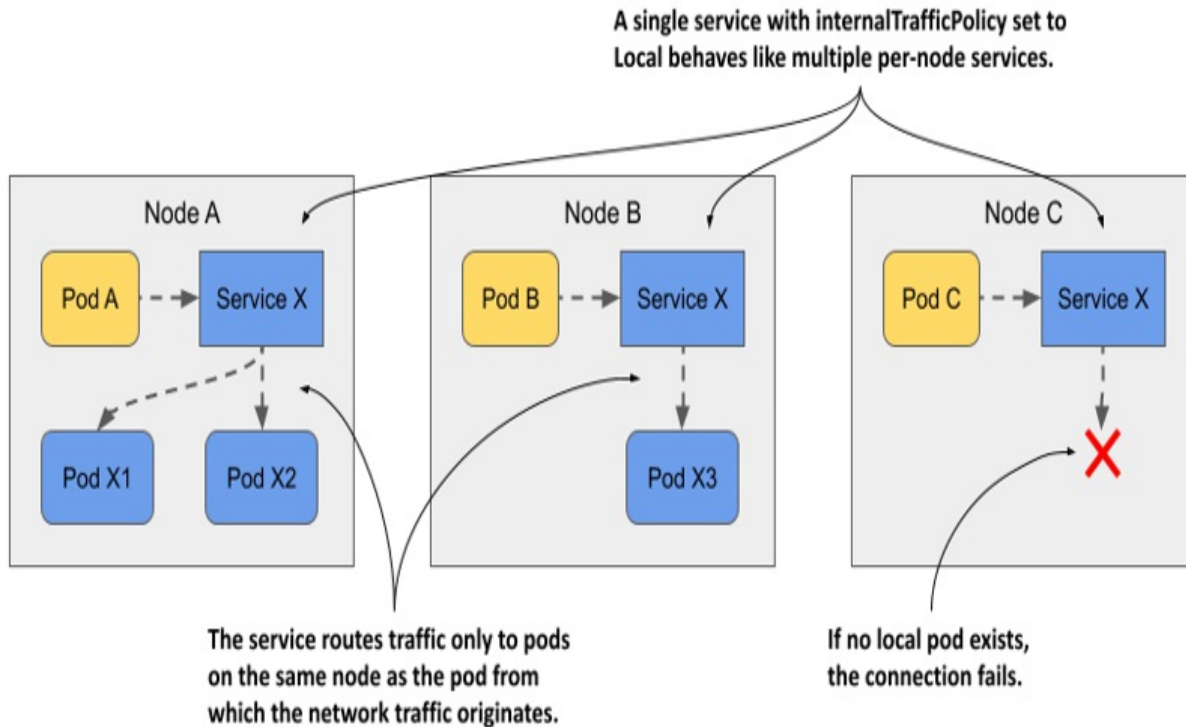
If pods provide a service that's tied in some way to the node on which the pod is running, you must ensure that client pods running on a particular node connect only to the endpoints on the same node. You can do this by creating a Service with the `internalTrafficPolicy` set to `Local`.

Note

You previously learned about the `externalTrafficPolicy` field, which is used to prevent unnecessary network hops between nodes when external traffic arrives in the cluster. The service's `internalTrafficPolicy` field is similar, but serves a different purpose.

As shown in the following figure, if the service is configured with the `Local` internal traffic policy, traffic from pods on a given node is forwarded only to pods on the same node. If there are no node-local service endpoints, the connection fails.

Figure 11.14 The behavior of a service with `internalTrafficPolicy` set to `Local`



Imagine a system pod running on each cluster node that manages communication with a device attached to the node. The pods don't use the device directly, but communicate with the system pod. Since pod IPs are fungible, while service IPs are stable, pods connect to the system pod through a Service. To ensure that pods connect only to the local system pod and not to those on other nodes, the service is configured to forward traffic only to local endpoints. You don't have any such pods in your cluster, but you can use the quote pods to try this feature.

Creating a service with a local internal traffic policy

The following listing shows the manifest for a service named `quote-local`, which forwards traffic only to pods running on the same node as the client pod.

Listing 11.9 A service that only forwards traffic to local endpoints

```
apiVersion: v1
kind: Service
metadata:
```



```

    name: quote-local
spec:
  internalTrafficPolicy: Local    #A
  selector:
    app: quote
  ports:
  - name: http
    port: 80
    targetPort: 80
    protocol: TCP

```

As you can see in the manifest, the service will forward traffic to all pods with the label `app: quote`, but since `internalTrafficPolicy` is set to `Local`, it won't forward traffic to all quote pods in the cluster, only to the pods that are on the same node as the client pod. Create the service by applying the manifest with `kubectl apply`.

Observing node-local traffic routing

Before you can see how the service routes traffic, you need to figure out where the client pods and the pods that are the endpoints of the service are located. List the pods with the `-o wide` option to see which node each pod is running on.

Select one of the kiada pods and note its cluster node. Use `curl` to connect to the `quote-local` service from that pod. For example, my `kiada-001` pod runs on the `kind-worker` node. If I run `curl` in it multiple times, all requests are handled by the quote pods on the same node:

```

$ kubectl exec kiada-001 -c kiada -- sh -c "while :; do curl -s q
This is the quote service running in pod quote-002 on node kind-w
This is the quote service running in pod quote-canary on node kin
This is the quote service running in pod quote-canary on node kin
This is the quote service running in pod quote-002 on node kind-w

```

No request is forwarded to the pods on the other node(s). If I delete the two pods on the `kind-worker` node, the next connection attempt will fail:

```

$ kubectl exec -it kiada-001 -c kiada -- curl http://quote-local
curl: (7) Failed to connect to quote-local port 80: Connection re

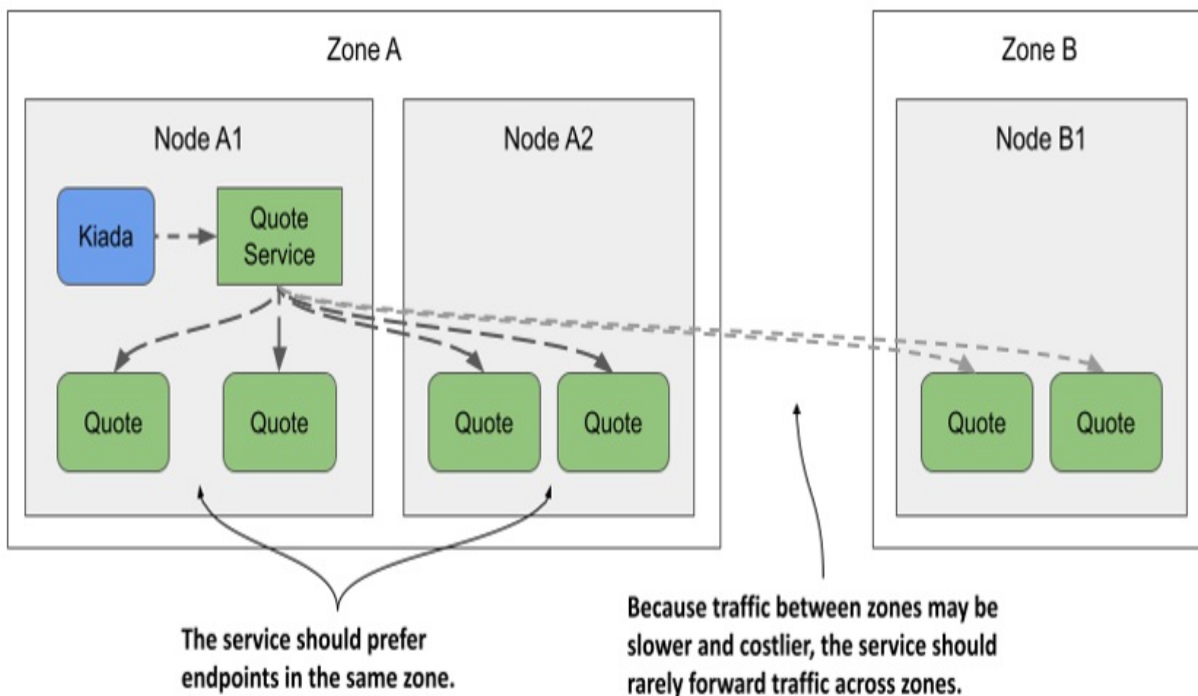
```

In this section, you learned how to forward traffic only to node-local endpoints when the semantics of the service require it. In other cases, you may want traffic to be forwarded preferentially to endpoints near the client pod, and only to more distant pods when needed. You'll learn how to do this in the next section.

11.5.2 Topology-aware hints

Imagine the Kiada suite running in a cluster with nodes spread across multiple data centers in different zones and regions, as shown in the following figure. You don't want a Kiada pod running in one zone to connect to Quote pods in another zone, unless there are no Quote pods in the local zone. Ideally, you want connections to be made within the same zone to reduce network traffic and associated costs.

Figure 11.15 Routing serviced traffic across availability zones



What was just described and illustrated in the figure is called *topology-aware traffic routing*. Kubernetes supports it by adding topology-aware hints to each endpoint in the EndpointSlice object.

Note

As of this writing, topology-aware hints are an alpha-level feature, so this could still change or be removed in the future.

Since this feature is still in alpha, it isn't enabled by default. Instead of explaining how to try it, I'll just explain how it works.

Understanding how topology aware hints are calculated

First, all your cluster nodes must contain the `kubernetes.io/zone` label to indicate which zone each node is located in. To indicate that a service should use topology-aware hints, you must set the `service.kubernetes.io/topology-aware-hints` annotation to `Auto`. If the service has a sufficient number of endpoints, Kubernetes adds the hints to each endpoint in the `EndpointSlice` object(s). As you can see in the following listing, the `hints` field specifies the zones from which this endpoint is to be consumed.

Listing 11.10 EndpointSlice with topology aware hints

```
apiVersion: discovery.k8s.io/v1
kind: EndpointSlice
endpoints:
- addresses:
  - 10.244.2.2
  conditions:
    ready: true
  hints:    #A
    forZones:    #A
    - name: zoneA    #A
  nodeName: kind-worker
  targetRef:
    kind: Pod
    name: quote-002
    namespace: default
    resourceVersion: "944"
    uid: 03343161-971d-403c-89ae-9632e7cd0d8d
  zone: zoneA    #B
...
```

The listing shows only a single endpoint. The endpoint represents the pod `quote-002` running on node `kind-worker`, which is located in `zoneA`. For this reason, the hints for this endpoint indicate that it is to be consumed by pods in `zoneA`. In this particular case, only `zoneA` should use this endpoint, but the `forZones` array could contain multiple zones.

These hints are computed by the `EndpointSlice` controller, which is part of the Kubernetes control plane. It assigns endpoints to each zone based on the number of CPU cores that can be allocated in the zone. If a zone has a higher number of CPU cores, it'll be assigned a higher number of endpoints than a zone with fewer CPU cores. In most cases, the hints ensure that traffic is kept within a zone, but to ensure a more even distribution, this isn't always the case.

Understanding where topology aware hints are used

Each node ensures that traffic sent to the service's cluster IP is forwarded to one of the service's endpoints. If there are no topology-aware hints in the `EndpointSlice` object, all endpoints, regardless of the node on which they reside, will receive traffic originating from a particular node. However, if all endpoints in the `EndpointSlice` object contain hints, each node processes only the endpoints that contain the node's zone in the hints and ignores the rest. Traffic originating from a pod on the node is therefore forwarded to only some endpoints.

Currently, you can't influence topology-aware routing except to turn it on or off, but that may change in the future.

11.6 Managing the inclusion of a pod in service endpoints

There's one more thing about services and endpoints that wasn't covered yet. You learned that a pod is included as an endpoint of a service if its labels match the service's label selector. Once a new pod with matching labels shows up, it becomes part of the service and connections are forwarded to the pod. But what if the application in the pod isn't immediately ready to accept

connections?

It may be that the application needs time to load either the configuration or the data, or that it needs to warm up so that the first client connection can be processed as quickly as possible without unnecessary latency caused by the fact that the application has just started. In such cases, you don't want the pod to receive traffic immediately, especially if the existing pod instances can handle the traffic. It makes sense not to forward requests to a pod that's just starting up until it becomes ready.

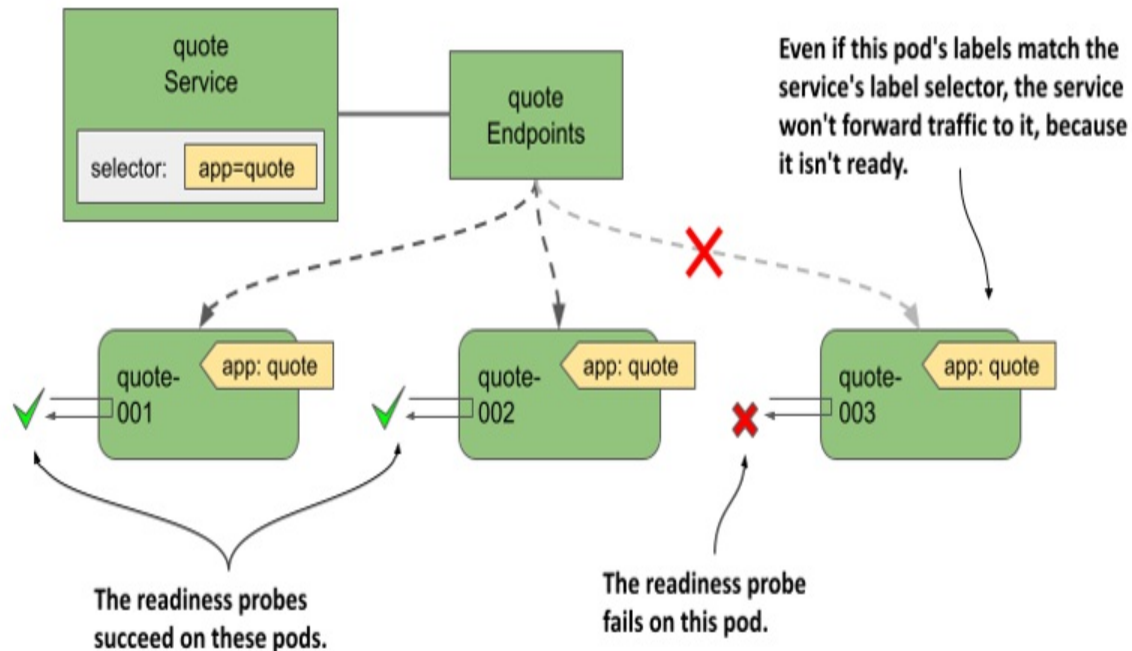
11.6.1 Introducing readiness probes

In chapter 6, you learned how to keep your applications healthy by letting Kubernetes restart containers that fail their liveness probes. A similar mechanism called *readiness probes* allows an application to signal that it's ready to accept connections.

Like liveness probes, the Kubelet also calls the readiness probe periodically to determine the readiness status of the pod. If the probe is successful, the pod is considered ready. The opposite is true if it fails. Unlike liveness probes, a container whose readiness probe fails isn't restarted; it's only removed as an endpoint from the services to which it belongs.

As you can see in the following figure, if a pod fails its readiness probe, the service doesn't forward connections to the pod even though its labels match the label selector defined in the service.

Figure 11.16 Pods that fail the readiness probe are removed from the service



The notion of being ready is specific to each application. The application developer decides what readiness means in the context of their application. To do this, they expose an endpoint through which Kubernetes asks the application whether it's ready or not. Depending on the type of endpoint, the correct readiness probe type must be used.

Understanding readiness probe types

As with liveness probes, Kubernetes supports three types of readiness probes:

- An `exec` probe executes a process in the container. The exit code used to terminate the process determines whether the container is ready or not.
- An `httpGet` probe sends a GET request to the container via HTTP or HTTPS. The response code determines the container's readiness status.
- A `tcpSocket` probe opens a TCP connection to a specified port on the container. If the connection is established, the container is considered ready.

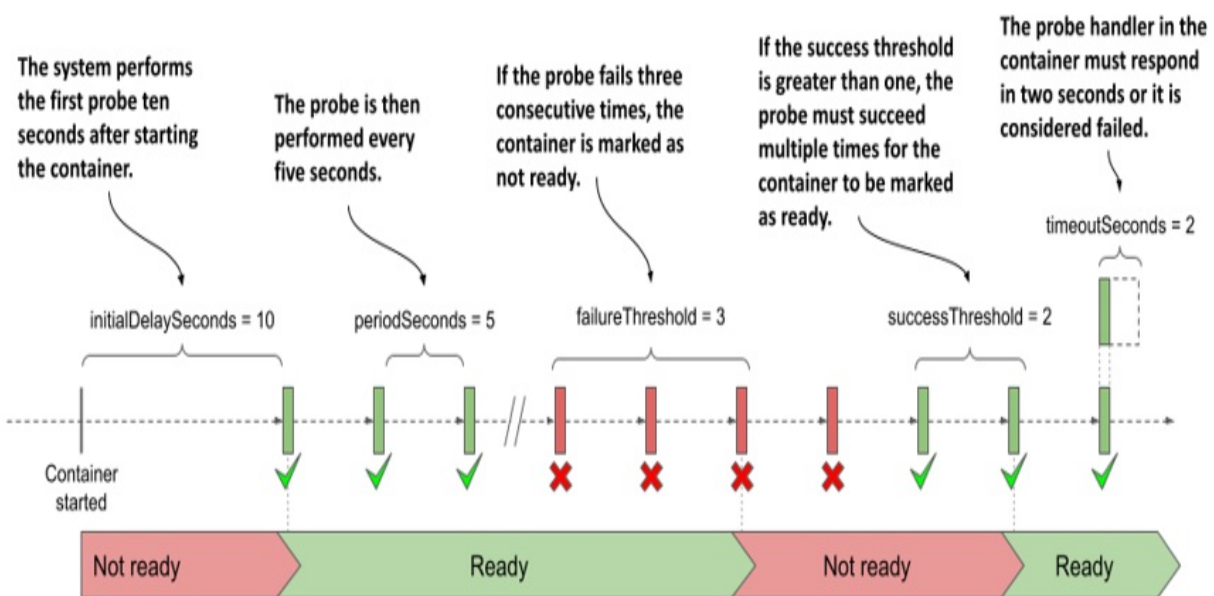
Configuring how often the probe is executed

You may recall that you can configure when and how often the liveness

probe runs for a given container using the following properties: `initialDelaySeconds`, `periodSeconds`, `failureThreshold`, and `timeoutSeconds`. These properties also apply to readiness probes, but they also support the additional `successThreshold` property, which specifies how many times the probe must succeed for the container to be considered ready.

These settings are best explained graphically. The following figure shows how the individual properties affect the execution of the readiness probe and the resulting readiness status of the container.

Figure 11.17 Readiness probe execution and resulting readiness status of the container



Note

If the container defines a startup probe, the initial delay for the readiness probe begins when the startup probe succeeds. Startup probes are explained in chapter 6.

When the container is ready, the pod becomes an endpoint of the services whose label selector it matches. When it's no longer ready, it's removed from those services.

11.6.2 Adding a readiness probe to a pod

To see readiness probes in action, create a new pod with a probe that you can switch from success to failure at will. This isn't a real-world example of how to configure a readiness probe, but it allows you to see how the outcome of the probe affects the pod's inclusion in the service.

The following listing shows the relevant part of the pod manifest file `pod.kiada-mock-readiness.yaml`, which you can find in the book's code repository.

Listing 11.11 A readiness probe definition in a pod

```
apiVersion: v1
kind: Pod
...
spec:
  containers:
  - name: kiada
    ...
    readinessProbe:      #A
      exec:              #B
        command:         #B
        - ls              #B
        - /var/ready      #B
      initialDelaySeconds: 10    #C
      periodSeconds: 5         #C
      failureThreshold: 3      #C
      successThreshold: 2      #C
      timeoutSeconds: 2        #C
    ...
```

The readiness probe periodically runs the `ls /var/ready` command in the `kiada` container. The `ls` command returns the exit code zero if the file exists, otherwise it's nonzero. Since zero is considered a success, the readiness probe succeeds if the file is present.

The reason to define such a strange readiness probe is so that you can change its outcome by creating or removing the file in question. When you create the pod, the file doesn't exist yet, so the pod isn't ready. Before you create the pod, delete all other `kiada` pods except `kiada-001`. This makes it easier to see

the service endpoints change.

Observing the pods' readiness status

After you create the pod from the manifest file, check its status as follows:

```
$ kubectl get po kiada-mock-readiness
NAME                READY   STATUS    RESTARTS   AGE   #A
kiada-mock-readiness 1/2     Running   0           1m    #A
```

The READY column shows that only one of the pod's containers is ready. This is the envoy container, which doesn't define a readiness probe. Containers without a readiness probe are considered ready as soon as they're started.

Since the pod's containers aren't all ready, the pod shouldn't receive traffic sent to the service. You can check this by sending several requests to the kiada service. You'll notice that all requests are handled by the kiada-001 pod, which is the only active endpoint of the service. This is evident from the Endpoints and EndpointSlice objects associated with the service. For example, the kiada-mock-readiness pod appears in the notReadyAddresses instead of the addresses array in the Endpoints object:

```
$ kubectl get endpoints kiada -o yaml
apiVersion: v1
kind: Endpoints
metadata:
  name: kiada
  ...
subsets:
- addresses:
  - ...
  notReadyAddresses:    #A
  - ip: 10.244.1.36    #A
    nodeName: kind-worker2    #A
    targetRef:          #A
      kind: Pod          #A
      name: kiada-mock-readiness    #A
      namespace: default    #A
    ...
```

In the EndpointSlice object, the endpoint's ready condition is false:

```
$ kubectl get endpointslices -l kubernetes.io/service-name=kiada
apiVersion: v1
items:
- addressType: IPv4
  apiVersion: discovery.k8s.io/v1
  endpoints:
  - addresses:
    - 10.244.1.36
    conditions:      #A
      ready: false    #A
    nodeName: kind-worker2
    targetRef:
      kind: Pod
      name: kiada-mock-readiness
      namespace: default
    ...
```

Note

In some cases, you may want to disregard the readiness status of pods. This may be the case if you want all pods in a group to get A, AAAA, and SRV records even though they aren't ready. If you set the `publishNotReadyAddresses` field in the Service object's spec to `true`, non-ready pods are marked as ready in both the Endpoints and EndpointSlice objects. Components like the cluster DNS treat them as ready.

For the readiness probe to succeed, create the `/var/ready` file in the container as follows:

```
$ kubectl exec kiada-mock-readiness -c kiada -- touch /var/ready
```

The `kubectl exec` command runs the `touch` command in the `kiada` container of the `kiada-mock-readiness` pod. The `touch` command creates the specified file. The container's readiness probe will now be successful. All the pod's containers should now show as ready. Verify that this is the case as follows:

```
$ kubectl get po kiada-mock-readiness
```

NAME	READY	STATUS	RESTARTS	AGE
kiada-mock-readiness	1/2	Running	0	10m

Surprisingly, the pod is still not ready. Is something wrong or is this the expected result? Take a closer look at the pod with `kubectl describe`. In the

output you'll find the following line:

```
Readiness:    exec [ls /var/ready] delay=10s timeout=2s period=5s
```

The readiness probe defined in the pod is configured to check the status of the container every 5 seconds. However, it's also configured to require two consecutive probe attempts to be successful before setting the status of the container to ready. Therefore, it takes about 10 seconds for the pod to be ready after you create the `/var/ready` file.

When this happens, the pod should become an active endpoint of the service. You can verify this is the case by examining the `Endpoints` or `EndpointSlice` objects associated with the service, or by simply accessing the service a few times and checking to see if the `kiada-mock-readiness` pod receives any of the requests you send.

If you want to remove the pod from the service again, run the following command to remove the `/var/ready` file from the container:

```
$ kubectl exec kiada-mock-readiness -c kiada -- rm /var/ready
```

This mockup of a readiness probe is just to show how readiness probes work. In the real world, the readiness probe shouldn't be implemented in this way. If you want to manually remove pods from a service, you can do so by either deleting the pod or changing the pod's labels rather than manipulating the readiness probe outcome.

Tip

If you want to manually control whether or not a pod is included in a service, add a label key such as `enabled` to the pod and set its value to `true`. Then add the label selector `enabled=true` to your service. Remove the label from the pod to remove the pod from the service.

11.6.3 Implementing real-world readiness probes

If you don't define a readiness probe in your pod, it becomes a service endpoint as soon as it's created. This means that every time you create a new

pod instance, connections forwarded by the service to that new instance will fail until the application in the pod is ready to accept them. To prevent this, you should always define a readiness probe for the pod.

In the previous section, you learned how to add a mock readiness probe to a container to manually control whether the pod is a service endpoint or not. In the real world, the readiness probe result should reflect the ability of the application running in the container to accept connections.

Defining a minimal readiness probe

For containers running an HTTP server, it's much better to define a simple readiness probe that checks whether the server responds to a simple GET / request, such as the one in the following snippet, than to have no readiness probe at all.

```
readinessProbe:
  httpGet:    #A
    port: 8080    #A
    path: /      #B
    scheme: HTTP  #B
```

When Kubernetes invokes this readiness probe, it sends the GET / request to port 8080 of the container and checks the returned HTTP response code. If the response code is greater than or equal to 200 and less than 400, the probe is successful, and the pod is considered ready. If the response code is anything else (for example, 404 or 500) or the connection attempt fails, the readiness probe is considered failed and the pod is marked as not ready.

This simple probe ensures that the pod only becomes part of the service when it can actually handle HTTP requests, rather than immediately when the pod is started.

Defining a better readiness probe

A simple readiness probe like the one shown in the previous section isn't always sufficient. Take the Quote pod, for example. You may recall that it runs two containers. The quote-writer container selects a random quote

from this book and writes it to a file called `quote` in the volume shared by the two containers. The `nginx` container serves files from this shared volume. Thus, the quote itself is available at the URL path `/quote`.

The purpose of the Quote pod is clearly to provide a random quote from the book. Therefore, it shouldn't be marked ready until it can serve this quote. If you direct the readiness probe to the URL path `/`, it'll succeed even if the `quote-writer` container hasn't yet created the quote file. Therefore, the readiness probe in the Quote pod should be configured as shown in the following snippet from the `pod.quote-readiness.yaml` file:

```
readinessProbe:
  httpGet:
    port: 80
    path: /quote      #A
    scheme: HTTP
  failureThreshold: 1  #B
```

If you add this readiness probe to your Quote pod, you'll see that the pod is only ready when the quote file exists. Try deleting the file from the pod as follows:

```
$ kubectl exec quote-readiness -c quote-writer -- rm /var/local/o
```

Now check the pod's readiness status with `kubectl get pod` and you'll see that one of the containers is no longer ready. When the `quote-writer` recreates the file, the container becomes ready again. You can also inspect the endpoints of the quote service with `kubectl get endpoints quote` to see that the pod is removed and then re-added.

Implementing a dedicated readiness endpoint

As you saw in the previous example, it may be sufficient to point the readiness probe to an existing path served by the HTTP server, but it's also common for an application to provide a dedicated endpoint, such as `/healthz/ready` or `/readyz`, through which it reports its readiness status. When the application receives a request on this endpoint, it can perform a series of internal checks to determine its readiness status.

Let's take the Quiz service as an example. The Quiz pod runs both an HTTP server and a MongoDB container. As you can see in the following listing, the quiz-api server implements the /healthz/ready endpoint. When it receives a request, it checks if it can successfully connect to MongoDB in the other container. If so, it responds with a 200 OK. If not, it returns 500 Internal Server Error.

Listing 11.12: The readiness endpoint in the quiz-api application

```
func (s *HTTPServer) ListenAndServe(listenAddress string) {
    router := mux.NewRouter()
    router.Methods("GET").Path("/").HandlerFunc(s.handleRoot)
    router.Methods("GET").Path("/healthz/ready").HandlerFunc(s.handleReadiness)
    ...
}

func (s *HTTPServer) handleReadiness(res http.ResponseWriter, req
conn, err := s.db.Connect()      #B
if err != nil {                  #C
    res.WriteHeader(http.StatusInternalServerError)      #C
    _, _ = fmt.Fprintf(res, "ERROR: %v\n", err.Error())  #C
    return          #C
}
defer conn.Close()

res.WriteHeader(http.StatusOK)   #D
_, _ = res.Write([]byte("Readiness check successful"))  #D
}
```

The readiness probe defined in the Quiz pod ensures that everything the pod needs to provide its services is present and working. As additional components are added to the quiz-api application, further checks can be added to the readiness check code. An example of this is the addition of an internal cache. The readiness endpoint could check to see if the cache is warmed up, so that only then is the pod exposed to clients.

Checking dependencies in the readiness probe

In the Quiz pod, the MongoDB database is an internal dependency of the quiz-api container. The Kiada pod, on the other hand, depends on the Quiz and Quote services, which are external dependencies. What should the

readiness probe check in the Kiada pod? Should it check whether it can reach the Quote and Quiz services?

The answer to this question is debatable, but any time you check dependencies in a readiness probe, you must consider what happens if a transient problem, such as a temporary increase in network latency, causes the probe to fail.

Note that the `timeoutSeconds` field in the readiness probe definition limits the time the probe has to respond. The default timeout is only one second. The container must respond to the readiness probe in this time.

If the Kiada pod calls the other two services in its readiness check, but their responses are only slightly delayed due to a transient network disruption, its readiness probe fails and the pod is removed from the service endpoints. If this happens to all Kiada pods at the same time, there will be no pods left to handle client requests. The disruption may only last a second, but the pods may not be added back to the service until dozens of seconds later, depending on how the `periodSeconds` and `successThreshold` properties are configured.

When you check external dependencies in your readiness probes, you should consider what happens when these types of transient network problems occur. Then you should set your periods, timeouts, and thresholds accordingly.

Tip

Readiness probes that try to be too smart can cause more problems than they solve. As a rule of thumb, readiness probes shouldn't test external dependencies, but can test dependencies within the same pod.

The Kiada application also implements the `/healthz/ready` endpoint instead of having the readiness probe use the `/` endpoint to check its status. This endpoint simply responds with the HTTP response code `200 OK` and the word `Ready` in the response body. This ensures that the readiness probe only checks that the application itself is responding, without also connecting to the Quiz or Quote services. You can find the pod manifest in the `pod.kiada-readiness.yaml` file.

Understanding readiness probes in the context of pod shutdown

One last note before you close this chapter. As you know, readiness probes are most important when the pod starts, but they also ensure that the pod is taken out of service when something causes it to no longer be ready during normal operation. But what about when the pod is terminating? A pod that's in the process of shutting down shouldn't be part of any services. Do you need to consider that when implementing the readiness probe?

Fortunately, when you delete a pod, Kubernetes not only sends the termination signal to the pod's containers, but also removes the pod from all services. This means you don't have to make any special provisions for terminating pods in your readiness probes. You don't have to make sure that the probe fails when your application receives the termination signal.

11.7 Summary

In this chapter, you finally connected the Kiada pods to the Quiz and Service pods. Now you can use the Kiada suite to test the knowledge you've acquired so far and refresh your memory with quotes from this book. In this chapter, you learned that:

- Pods communicate over a flat network that allows any pod to reach any other pod in the cluster, regardless of the actual network topology connecting the cluster nodes.
- A Kubernetes service makes a group of pods available under a single IP address. While the IPs of the pods may change, the IP of the service remains constant.
- The cluster IP of the service is reachable from inside the cluster, but NodePort and LoadBalancer services are also accessible from outside the cluster.
- Service endpoints are either determined by a label selector specified in the Service object or configured manually. These endpoints are stored in the Endpoints and EndpointSlice objects.
- Client pods can find services using the cluster DNS or environment variables. Depending on the type of Service, the following DNS records may be created: A, AAAA, SRV, and CNAME.

- Services can be configured to forward external traffic only to pods on the same node that received the external traffic, or to pods anywhere in the cluster. They can also be configured to route internal traffic only to pods on the same node as the pod from which the traffic originates from. Topology-aware routing ensures that traffic isn't routed across availability zones when a local pod can provide the requested service.
- Pods don't become service endpoints until they're ready. By implementing a readiness probe handler in an application, you can define what readiness means in the context of that particular application.

In the next chapter, you'll learn how to use Ingress objects to make multiple services accessible through a single external IP address.

12 Exposing Services with Ingress

This chapter covers

- Creating Ingress objects
- Deploying and understanding Ingress controllers
- Securing ingresses with TLS
- Adding additional configuration to an Ingress
- Using IngressClasses when multiple controllers are installed
- Using Ingresses with non-service backends

In the previous chapter, you learned how to use the Service object to expose a group of pods at a stable IP address. If you use the LoadBalancer service type, the service is made available to clients outside the cluster through a load balancer. This approach is fine if you only need to expose a single service externally, but it becomes problematic with large numbers of services, since each service needs its own public IP address.

Fortunately, by exposing these services through an *Ingress* object instead, you only need a single IP address. Additionally, the Ingress provides other features such as HTTP authentication, cookie-based session affinity, URL rewriting, and others that Service objects can't.

NOTE

You'll find the code files for this chapter at <https://github.com/luksa/kubernetes-in-action-2nd-edition/tree/master/Chapter12>.

12.1 Introducing Ingresses

Before I explain what an Ingress is in the context of Kubernetes, it may help readers for whom English isn't their first language to define what the term *ingress* means.

Definition

Ingress (noun)—The act of going in or entering; the right to enter; a means or place of entering; entryway.

In Kubernetes, an Ingress is a way for external clients to access the services of applications running in the cluster. The Ingress function consists of the following three components:

- The *Ingress API object*, which is used to define and configure an ingress.
- An *L7 load balancer* or *reverse proxy* that routes traffic to the backend services.
- The *ingress controller*, which monitors the Kubernetes API for Ingress objects and deploys and configures the load balancer or reverse proxy.

Note

L4 and L7 refer to layer 4 (Transport Layer; TCP, UDP) and layer 7 (Application Layer; HTTP) of the Open Systems Interconnection Model (OSI Model).

Note

Unlike a forward proxy, which routes and filters outgoing traffic and is typically located in the same location as the clients it serves, a reverse proxy handles incoming traffic and routes it to one or more backend servers. A reverse proxy is located near those servers.

In most online content, the term *ingress controller* is often used to refer to the load balancer/reverse proxy and the actual controller as one entity, but they're two different components. For this reason, I refer to them separately in this chapter.

I also use the term *proxy* for the L7 load balancer, so you don't confuse it with the L4 load balancer that handles the traffic for LoadBalancer-type services.

12.1.1 Introducing the Ingress object kind

When you want to expose a set of services externally, you create an Ingress object and reference the Service objects in it. Kubernetes uses this Ingress object to configure an L7 load balancer (an HTTP reverse proxy) that makes the services accessible to external clients through a common endpoint.

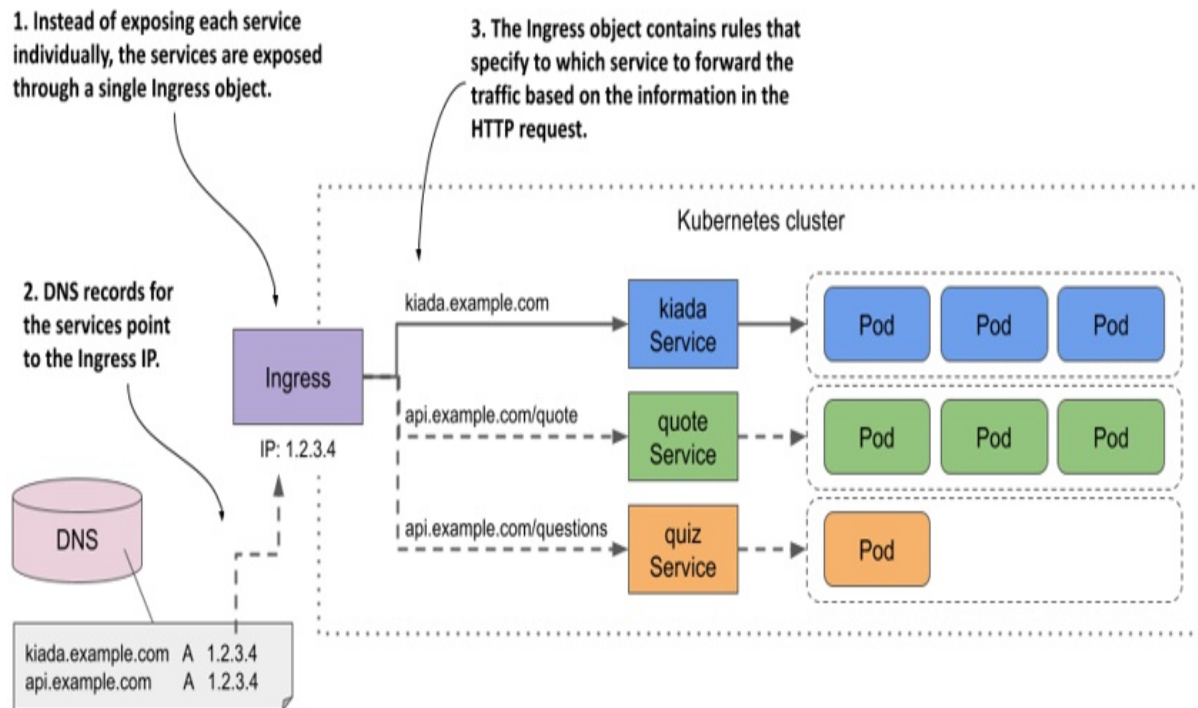
Note

If you expose a Service through an Ingress, you can usually leave the Service type set to `ClusterIP`. However, some ingress implementations require the Service type to be `NodePort`. Refer to the ingress controller's documentation to see if this is the case.

Exposing services through an Ingress object

While an Ingress object can be used to expose a single Service, it's typically used in combination with multiple Service objects, as shown in the following figure. The figure shows how a single Ingress object makes all three services in the Kiada suite accessible to external clients.

Figure 12.1 An Ingress forwards external traffic to multiple services



The Ingress object contains rules for routing traffic to the three services based on the information in the HTTP request. The public DNS entries for the services all point to the same Ingress. The Ingress determines which service should receive the request from the request itself. If the client request specifies the host `kiada.example.com`, the Ingress forwards it to the pods that belong to the kiada service, whereas requests that specify the host `api.example.com` are forwarded to the quote or quiz services, depending on which path is requested.

Using multiple Ingress objects in a cluster

An Ingress object typically handles traffic for all Service objects in a particular Kubernetes namespace, but multiple Ingresses are also an option. Normally, each Ingress object gets its own IP address, but some ingress implementations use a shared endpoint for all Ingress objects you create in the cluster.

12.1.2 Introducing the Ingress controller and the reverse proxy

Not all Kubernetes clusters support Ingresses out of the box. This functionality is provided by a cluster add-on component called Ingress controller. This controller is the link between the Ingress object and the actual physical ingress (the reverse proxy). Often the controller and the proxy run as two processes in the same container or as two containers in the same pod. That's why people use the term ingress controller to mean both.

Sometimes the controller or the proxy is located outside the cluster. For example, the Google Kubernetes Engine provides its own Ingress controller that uses Google Cloud Platform's L7 load balancer to provide the Ingress functionality to the cluster.

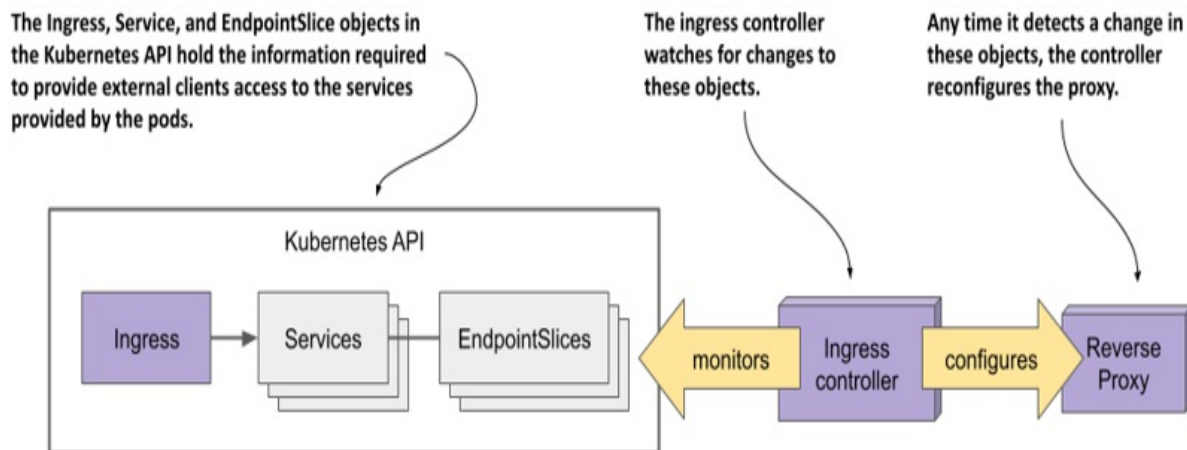
If your cluster is deployed in multiple availability zones, a single ingress can handle traffic for all of them. It forwards each HTTP request to the best zone depending on where the client is located, for example.

There's a wide range of ingress controllers to choose from. The Kubernetes community maintains a list at <https://kubernetes.io/docs/concepts/services-networking/ingress-controllers/>. Among the most popular are the Nginx ingress controller, Ambassador, Contour, and Traefik. Most of these ingress controllers use Nginx, HAProxy, or Envoy as the reverse proxy, but some use their own proxy implementation.

Understanding the role of the ingress controller

The ingress controller is the software component that brings the Ingress object to life. As shown in the following figure, the controller connects to the Kubernetes API server and monitors the Ingress, Service, and Endpoints or EndpointSlice objects. Whenever you create, modify, or delete these objects, the controller is notified. It uses the information in these objects to provision and configure the reverse proxy for the ingress, as shown in the following figure.

Figure 12.2 The role of an ingress controller



When you create the Ingress object, the controller reads its spec section and combines it with the information in the Service and EndpointSlice objects it references. The controller converts this information into the configuration for the reverse proxy. It then sets up a new proxy with this configuration and performs additional steps to ensure that the proxy is reachable from outside the cluster. If the proxy is running in a pod inside the cluster, this usually means that a LoadBalancer type service is created to expose the proxy externally.

When you make changes to the Ingress object, the controller updates the configuration of the proxy, and when you delete it, the controller stops and removes the proxy and any other objects it created alongside it.

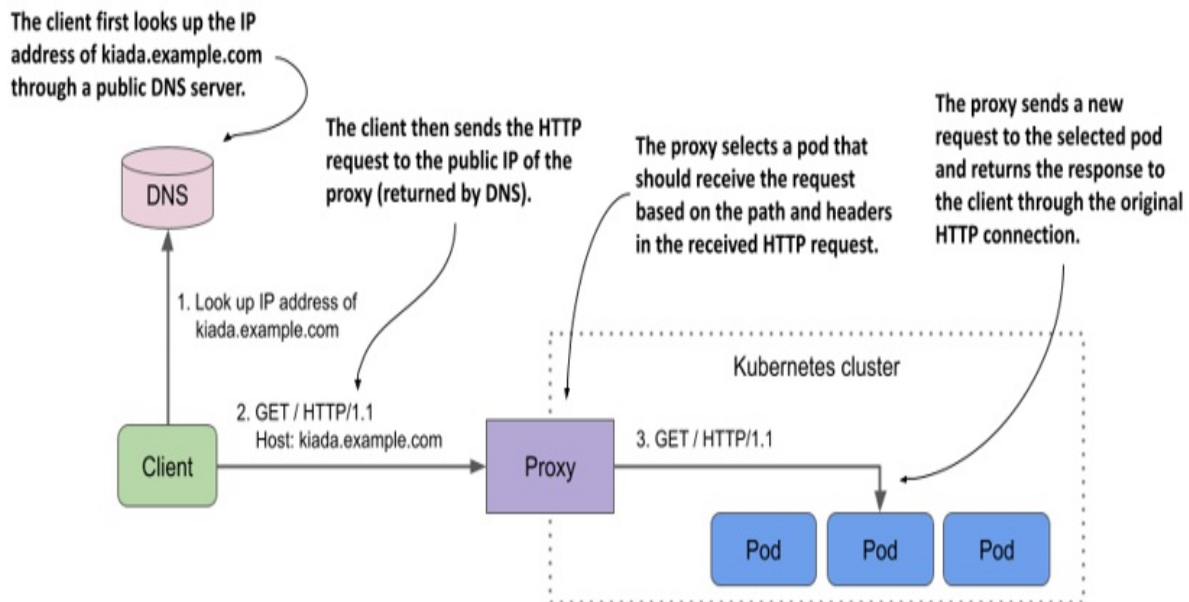
Understanding how the proxy forwards traffic to the services

The reverse proxy (or L7 load balancer) is the component that handles incoming HTTP requests and forwards it to the services. The proxy configuration typically contains a list of virtual hosts and, for each, a list of endpoint IPs. This information is obtained from the Ingress, Service, and Endpoints/EndpointSlice objects. When clients connect to the proxy, the proxy uses this information to route the request to an endpoint such as a pod based on the request path and headers.

The following figure shows how a client accesses the Kiada service through the proxy. The client first performs a DNS lookup of `kiada.example.com`.

The DNS server returns the public IP address of the reverse proxy. Then the client sends an HTTP request to the proxy where the Host header contains the value `kiada.example.com`. The proxy maps this host to the IP address of one of the Kiada pods and forwards the HTTP request to it. Note that the proxy doesn't send the request to the service IP, but directly to the pod. This is how most ingress implementations work.

Figure 12.3 Accessing pods through an Ingress



12.1.3 Installing an ingress controller

Before you start creating Ingresses, you need to make sure that an ingress controller runs in your cluster. As you learned in the previous section, not all Kubernetes clusters have one.

If you're using a managed cluster with one of the major cloud providers, an ingress controller is already in place. In Google Kubernetes Engine, the ingress controller is GLBC (GCE L7 Load Balancer), in AWS the Ingress functionality is provided by the AWS Load Balancer Controller, while Azure provides AGIC (Application Gateway Ingress Controller). Check your cloud provider's documentation to see if an ingress controller is provided and

whether you need to enable it. Alternatively, you can install the ingress controller yourself.

As you already know, there are many different ingress implementations to choose from. They all provide the type of traffic routing explained in the previous section, but each provides different additional features. In all the examples in this chapter, I used the Nginx ingress controller. I suggest that you use it as well unless your cluster provides a different one. To install the Nginx ingress controller in your cluster, see the sidebar.

Note

There are two implementations of the Nginx ingress controller. One is provided by the Kubernetes maintainers and the other is provided by the authors of Nginx itself. If you're new to Kubernetes, you should start with the former. That's the one I used.

Installing the Nginx ingress controller

Regardless of how you run your Kubernetes cluster, you should be able to install the Nginx ingress controller by following the instructions at <https://kubernetes.github.io/ingress-nginx/deploy/>.

If you created your cluster using the kind tool, you can install the controller by running the following command:

```
$ kubectl apply -f https://raw.githubusercontent.com/kubernetes/i
```

If you run your cluster with Minikube, you can install the controller as follows:

```
$ minikube addons enable ingress
```

12.2 Creating and using Ingress objects

The previous section explained the basics of Ingress objects and controllers, and how to install the Nginx ingress controller. In this section, you'll learn how to use an Ingress to expose the services of the Kiada suite.

Before you create your first Ingress object, you must deploy the pods and services of the Kiada suite. If you followed the exercises in the previous chapter, they should already be there. If not, you can create them by creating the kiada namespace and then applying all manifests in the the Chapter12/SETUP/ directory with the following command:

```
$ kubectl apply -f SETUP/ --recursive
```

12.2.1 Exposing a service through an Ingress

An Ingress object references one or more Service objects. Your first Ingress object exposes the kiada service, which you created in the previous chapter. Before you create the Ingress, refresh your memory by looking at the service manifest in the following listing.

Listing 12.1 The kiada service manifest

```
apiVersion: v1
kind: Service
metadata:
  name: kiada      #A
spec:
  type: ClusterIP   #B
  selector:
    app: kiada
  ports:
    - name: http     #C
      port: 80        #C
      targetPort: 8080 #C
    - name: https
      port: 443
      targetPort: 8443
```

The Service type is `ClusterIP` because the service itself doesn't need to be directly accessible to clients outside the cluster, since the Ingress will take care of that. Although the service exposes ports 80 and 443, the Ingress will forward traffic only to port 80.

Creating the Ingress object

The Ingress object manifest is shown in the following listing. You can find it in the file `Chapter12/ing.kiada-example-com.yaml` in the book's code repository.

Listing 12.2 An Ingress object exposing the kiada service at kiada.example.com

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: kiada-example-com      #A
spec:
  rules:
  - host: kiada.example.com    #B
    http:
      paths:
      - path: /                #C
        pathType: Prefix      #C
        backend:
          service:             #D
            name: kiada        #D
            port:               #D
              number: 80       #D
```

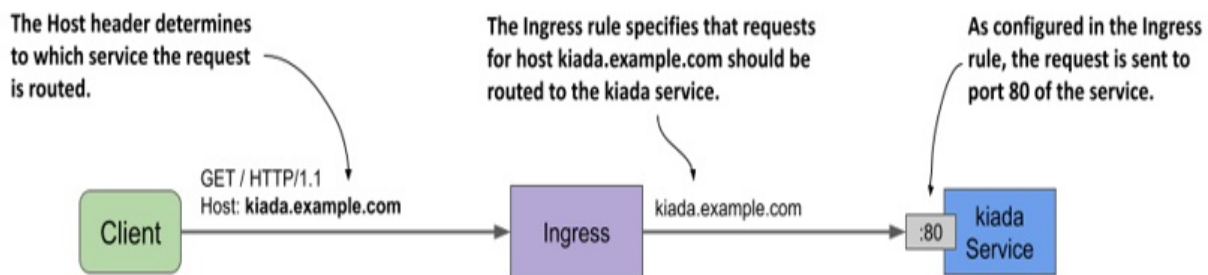
The manifest in the listing defines an Ingress object named `kiada-example-com`. While you can give the object any name you want, it's recommended that the name reflect the host and/or path(s) specified in the ingress rules.

Warning

In Google Kubernetes Engine, the Ingress name mustn't contain dots, otherwise the following error message will be displayed in the events associated with the Ingress object: `Error syncing to GCP: error running load balancer syncing routine: invalid loadbalancer name.`

The Ingress object in the listing defines a single rule. The rule states that all requests for the host `kiada.example.com` should be forwarded to port 80 of the `kiada` service, regardless of the requested path (as indicated by the `path` and `pathType` fields). This is illustrated in the following figure.

Figure 12.4 How the `kiada-example-com` Ingress object configures external traffic routing



Inspecting an Ingress object to get its public IP address

After creating the Ingress object with `kubectl apply`, you can see its basic information by listing Ingress objects in the current namespace with `kubectl get ingresses` as follows:

```
$ kubectl get ingresses
```

NAME	CLASS	HOSTS	ADDRESS	PORT
kiada-example-com	nginx	kiada.example.com	11.22.33.44	80

Note

You can use `ing` as a shorthand for `ingress`.

To see the Ingress object in detail, use the `kubectl describe` command as follows:

```
$ kubectl describe ing kiada-example-com
```

```

Name:          kiada-example-com      #A
Namespace:     default                #A
Address:       11.22.33.44           #B
Default backend: default-http-backend:80 (172.17.0.15:8080)  #
Rules:         #D
  Host          Path    Backends    #D
  ----          -
  kiada.example.com  #D
                  /    kiada:80 (172.17.0.4:8080,172.17.0.5:808
Annotations:    <none>
Events:
  Type    Reason    Age                From
  ----    -
  Normal  Sync      5m6s (x2 over 5m28s)  nginx-ingress-controller
  
```

As you can see, the `kubectl describe` command lists all the rules in the Ingress object. For each rule, not only is the name of the target service shown, but also its endpoints. If you see an error message related to the default backend, ignore it for now. You'll fix it later.

Both `kubectl get` and `kubectl describe` display the IP address of the ingress. This is the IP address of the L7 load balancer or reverse proxy to which clients should send requests. In the example output, the IP address is `11.22.33.44` and the port is `80`.

Note

The address may not be displayed immediately. This is very common when the cluster is running in the cloud. If the address isn't displayed after several minutes, it means that no ingress controller has processed the Ingress object. Check if the controller is running. Since a cluster can run multiple ingress controllers, it's possible that they'll all ignore your Ingress object if you don't specify which of them should process it. Check the documentation of your chosen ingress controller to find out if you need to add the `kubernetes.io/ingress.class` annotation or set the `spec.ingressClassName` field in the Ingress object. You'll learn more about this field later.

You can also find the IP address in the Ingress object's status field as follows:

```
$ kubectl get ing kiada -o yaml
...
status:
  loadBalancer:
    ingress:
      - ip: 11.22.33.44      #A
```

Note

Sometimes the displayed address can be misleading. For example, if you use Minikube and start the cluster in a VM, the ingress address will show up as `localhost`, but that's only true from the VM's perspective. The actual ingress address is the IP address of the VM, which you can get with the

minikube ip command.

Adding the ingress IP to the DNS

After you add an Ingress to a production cluster, the next step is to add a record to your Internet domain's DNS server. In these examples, we assume that you own the domain `example.com`. To allow external clients to access your service through the ingress, you configure the DNS server to resolve the domain name `kiada.example.com` to the ingress IP `11.22.33.44`.

In a local development cluster, you don't have to deal with DNS servers. Since you're only accessing the service from your own computer, you can get it to resolve the address by other means. This is explained next, along with instructions on how to access the service through the ingress.

Accessing services through the ingress

Since ingresses use virtual hosting to figure out where to forward the request, you won't get the desired result by simply sending an HTTP request to the Ingress' IP address and port. You need to make sure that the Host header in the HTTP request matches one of the rules in the Ingress object.

To achieve this, you must tell the HTTP client to send the request to the host `kiada.example.com`. However, this requires resolving the host to the Ingress IP. If you use `curl`, you can do this without having to configure your DNS server or your local `/etc/hosts` file. Let's take `11.22.33.44` as the ingress IP. You can access the `kiada` service through the ingress with the following command:

```
$ curl --resolve kiada.example.com:80:11.22.33.44 http://kiada.ex
* Added kiada.example.com:80:11.22.33.44 to DNS cache      #A
* Hostname kiada.example.com was found in DNS cache        #B
*   Trying 11.22.33.44:80...                                #B
* Connected to kiada.example.com (11.22.33.44) port 80 (#0)  #B
> GET / HTTP/1.1
> Host: kiada.example.com      #C
> User-Agent: curl/7.76.1
> Accept: */*
...
```

The `--resolve` option adds the hostname `kiada.example.com` to the DNS cache. This ensures that `kiada.example.com` resolves to the ingress IP. Curl then opens the connection to the ingress and sends the HTTP request. The Host header in the request is set to `kiada.example.com` and this allows the ingress to forward the request to the correct service.

Of course, if you want to use your web browser instead, you can't use the `--resolve` option. Instead, you can add the following entry to your `/etc/hosts` file.

```
11.22.33.44    kiada.example.com    #A
```

Note

On Windows, the hosts file is usually located at `C:\Windows\System32\Drivers\etc\hosts`.

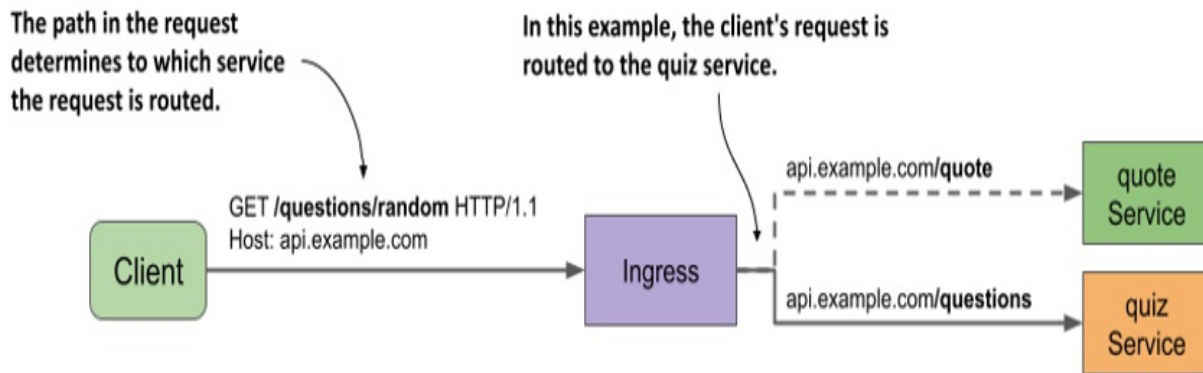
You can now access the service at <http://kiada.example.com> with your web browser or `curl` without having to use the `--resolve` option to map the hostname to the IP.

12.2.2 Path-based ingress traffic routing

An Ingress object can contain many rules and therefore map multiple hosts and paths to multiple services. You've already created an Ingress for the `kiada` service. Now you'll create one for the `quote` and `quiz` services.

The Ingress object for these two services makes them available through the same host: `api.example.com`. The path in the HTTP request determines which service receives each request. As you can see in the following figure, all requests with the path `/quote` are forwarded to the `quote` service, and all requests whose path starts with `/questions` are forwarded to the `quiz` service.

Figure 12.5 Path-based ingress traffic routing



The following listing shows the Ingress manifest.

Listing 12.3 Ingress mapping request paths to different services

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api-example-com
spec:
  rules:
  - host: api.example.com      #A
    http:
      paths:
      - path: /quote          #B
        pathType: Exact      #B
        backend:             #B
          service:           #B
            name: quote      #B
            port:            #B
            name: http       #B
      - path: /questions      #C
        pathType: Prefix     #C
        backend:             #C
          service:           #C
            name: quiz       #C
            port:            #C
            name: http       #C
```

In the Ingress object shown in the listing, a single rule with two paths is defined. The rule matches HTTP requests with the host `api.example.com`. In this rule, the paths array contains two entries. The first matches requests that ask for the `/quote` path and forwards them to the port named `http` in the

quote Service object. The second entry matches all requests whose first path element is /questions and forwards them to the port http of the quiz service.

Note

By default, no URL rewriting is performed by the ingress proxy. If the client requests the path /quote, the path in the request that the proxy makes to the backend service is also /quote. In some ingress implementations, you can change this by specifying a URL rewrite rule in the Ingress object.

After you create the Ingress object from the manifest in the previous listing, you can access the two services it exposes as follows (replace the IP with that of your ingress):

```
$ curl --resolve api.example.com:80:11.22.33.44 api.example.com/q
$ curl --resolve api.example.com:80:11.22.33.44 api.example.com/q
```

If you want to access these services with your web browser, add api.example.com to the line you added earlier to your /etc/hosts file. It should now look like this:

```
11.22.33.44      kiada.example.com api.example.com      #A
```

Understanding how the path is matched

Did you notice the difference between the pathType fields in the two entries in the previous listing? The pathType field specifies how the path in the request is matched with the paths in the ingress rule. The three supported values are summarized in the following table.

Table 12.1 Supported values in the pathType field

PathType	Description
	The requested URL path must exactly match the

Exact	path specified in the ingress rule.
Prefix	The requested URL path must begin with the path specified in the ingress rule, element by element.
ImplementationSpecific	Path matching depends on the implementation of the ingress controller.

If multiple paths are specified in the ingress rule and the path in the request matches more than one path in the rule, priority is given to paths with the Exact path type.

Matching paths using the Exact path type

The following table shows examples of how matching works when pathType is set to Exact.

Table 12.2 Request paths matched when pathType is Exact

Path in rule	Matches request path	Doesn't match
/	/	/foo /bar
/foo	/foo	/foo/ /bar
		/foo

/foo/	/foo/	/foo/bar /bar
/F00	/F00	/foo

As you can see from the examples in the table, the matching works as you'd expect. It's case sensitive, and the path in the request must exactly match the path specified in the ingress rule.

Matching paths using the Prefix path type

When pathType is set to Prefix, things aren't as you might expect. Consider the examples in the following table.

Table 12.3 Request paths matched when pathType is Prefix

Path in rule	Matches request paths	Doesn't match
/	All paths; for example: / /foo /foo/	
/foo or /foo/	/foo /foo/ /foo/bar	/foobar /bar
/F00	/F00	/foo

The request path isn't treated as a string and checked to see if it begins with the specified prefix. Instead, both the path in the rule and the request path are split by / and then each element of the request path is compared to the corresponding element of the prefix. Take the path /foo, for example. It matches the request path /foo/bar, but not /foobar. It also doesn't match the request path /fooxyz/bar.

When matching, it doesn't matter if the path in the rule or the one in the request ends with a forward slash. As with the `Exact` path type, matching is case sensitive.

Matching paths using the `ImplementationSpecific` path type

The `ImplementationSpecific` path type is, as the name implies, dependent on the implementation of the ingress controller. With this path type, each controller can set its own rules for matching the request path. For example, in GKE you can use wildcards in the path. Instead of using the `Prefix` type and setting the path to /foo, you can set the type to `ImplementationSpecific` and the path to /foo/*.

12.2.3 Using multiple rules in an Ingress object

In the previous sections you created two Ingress objects to access the Kiada suite services. In most Ingress implementations, each Ingress object requires its own public IP address, so you're now probably using two public IP addresses. Since this is potentially costly, it's better to consolidate the Ingress objects into one.

Creating an Ingress object with multiple rules

Because an Ingress object can contain multiple rules, it's trivial to combine multiple objects into one. All you have to do is take the rules and put them into the same Ingress object, as shown in the following listing. You can find the manifest in the file `ing.kiada.yaml`.

Listing 12.4 Ingress exposing multiple services on different hosts

```

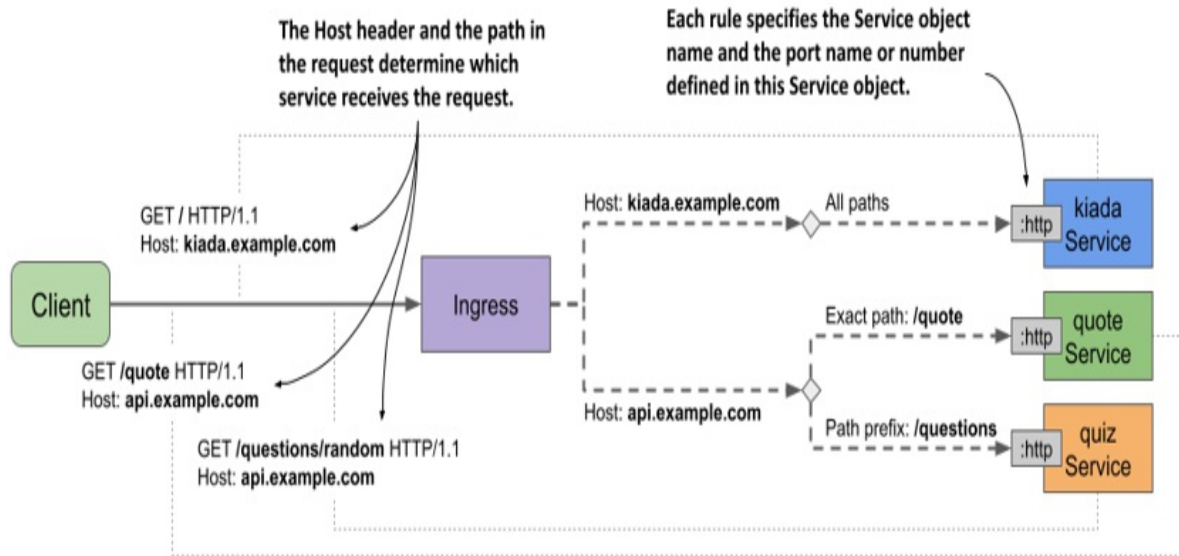
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: kiada
spec:
  rules:
  - host: kiada.example.com      #A
    http: #A
      paths: #A
      - path: /                  #A
        pathType: Prefix        #A
        backend: #A
          service: #A
            name: kiada         #A
            port: #A
            name: http          #A
  - host: api.example.com      #B
    http: #B
      paths: #B
      - path: /quote            #B
        pathType: Exact         #B
        backend: #B
          service: #B
            name: quote         #B
            port: #B
            name: http          #B
      - path: /questions        #B
        pathType: Prefix        #B
        backend: #B
          service: #B
            name: quiz          #B
            port: #B
            name: http          #B

```

This single Ingress object handles all traffic for all services in the Kiada suite yet only requires a single public IP address.

The Ingress object uses virtual hosts to route traffic to the backend services. If the value of the Host header in the request is `kiada.example.com`, the request is forwarded to the `kiada` service. If the header value is `api.example.com`, the request is routed to one of the other two services, depending on the requested path. The Ingress and the associated Service objects are shown in the next figure.

Figure 12.6 An Ingress object covering all services of the Kiada suite



You can delete the two Ingress objects you created earlier and replace them with the one in the previous listing. Then you can try to access all three services through this ingress. Since this is a new Ingress object, its IP address is most likely not the same as before. So you need to update the DNS, the `/etc/hosts` file, or the `--resolve` option when you run the `curl` command again.

Using wildcards in the host field

The host field in the ingress rules supports the use of wildcards. This allows you to capture all requests sent to a host that matches `*.example.com` and forward them to your services. The following table shows how wildcard matching works.

Table 12.4 Examples of using wildcards in the ingress rule's host field

Host	Matches request hosts	Doesn't match
kiada.example.com	kiada.example.com	example.com api.example.com

		foo.kiada.example.com
*.example.com	kiada.example.com	example.com
	api.example.com	foo.kiada.example.com
	foo.example.com	

Look at the example with the wildcard. As you can see, `*.example.com` matches `kiada.example.com`, but it doesn't match `foo.kiada.example.com` or `example.com`. This is because a wildcard only covers a single element of the DNS name.

As with rule paths, a rule that exactly matches the host in the request takes precedence over rules with host wildcards.

Note

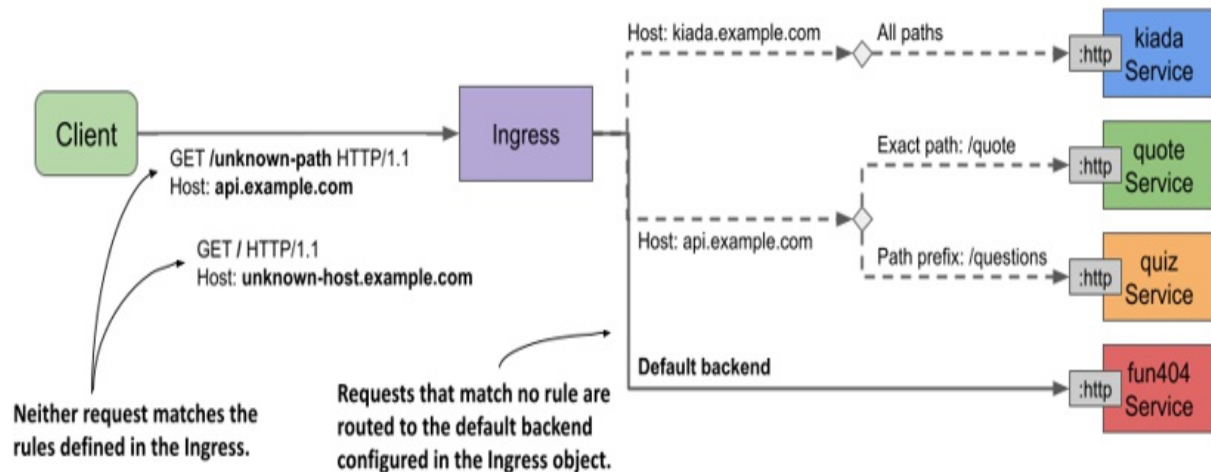
You can also omit the host field altogether to make the rule match any host.

12.2.4 Setting the default backend

If the client request doesn't match any rules defined in the Ingress object, the response `404 Not Found` is normally returned. However, you can also define a default backend to which the ingress should forward the request if no rules are matched. The default backend serves as a catch-all rule.

The following figure shows the default backend in the context of the other rules in the Ingress object.

Figure 12.7 The default backend handles requests that match no Ingress rule



As you can see in the figure, a service named fun404 is used as the default backend. Let's add it to the kiada Ingress object.

Specifying the default backend in an Ingress object

You specify the default backend in the `spec.defaultBackend` field, as shown in the following listing (the full manifest can be found in the `ing.kiada.defaultBackend.yaml` file).

Listing 12.5 Specifying the default backend in the Ingress object

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: kiada
spec:
  defaultBackend:
    service:
      name: fun404
      port:
        name: http
  rules:
    ...
```

In the listing, you can see that setting the default backend isn't much different from setting the backend in the rules. Just as you specify the name and port of the backend service in each rule, you also specify the name and port of the default backend service in the `service` field under `spec.defaultBackend`.

Creating the service and pod for the default backend

The kiada Ingress object is configured to forward requests that don't match any rules to a service called fun404. You need to create this service and the underlying pod. You can find an object manifest with both object definitions in the file `all.my-default-backend.yaml`. The contents of the file are shown in the following listing.

Listing 12.6 The Pod and Service object manifests for the default ingress backend

```
apiVersion: v1
kind: Pod
metadata:
  name: fun404      #A
  labels:
    app: fun404     #B
spec:
  containers:
    - name: server
      image: luksa/static-http-server      #C
      args:      #D
        - --listen-port=8080      #D
        - --response-code=404     #D
        - --text=This isn't the URL you're looking for.      #D
      ports:
        - name: http      #E
          containerPort: 8080      #E
---
apiVersion: v1
kind: Service
metadata:
  name: fun404      #F
  labels:
    app: fun404
spec:
  selector:      #G
    app: fun404      #G
  ports:
    - name: http      #H
      port: 80      #H
      targetPort: http      #I
```

After applying both the Ingress object manifest and the Pod and Service object manifest, you can test the default backend by sending a request that

doesn't match any of the rules in the ingress. For example:

```
$ curl api.example.com/unknown-path --resolve api.example.com:80:  
This isn't the URL you're looking for.      #B
```

As expected, the response text matches what you configured in the fun404 pod. Of course, instead of using the default backend to return a custom 404 status, you can use it to forward all requests to default to a service of your choice.

You can even create an Ingress object with only a default backend and no rules to forward all external traffic to a single service. If you're wondering why you'd do this using an Ingress object and not by simply setting the service type to LoadBalancer, it's because ingresses can provide additional HTTP features that services can't. One example is securing the communication between the client and the service with Transport Layer Security (TLS), which is explained next.

12.3 Configuring TLS for an Ingress

So far in this chapter, you've used the Ingress object to allow external HTTP traffic to your services. These days, however, you usually want to secure at least all external traffic with SSL/TLS.

You may recall that the kiada service provides both an HTTP and an HTTPS port. When you created the Ingress, you only configured it to forward HTTP traffic to the service, but not HTTPS. You'll do this now.

There are two ways to add HTTPS support. You can either allow HTTPS to pass through the ingress proxy and have the backend pod terminate the TLS connection, or have the proxy terminate and connect to the backend pod through HTTP.

12.3.1 Configuring the Ingress for TLS passthrough

You may be surprised to learn that Kubernetes doesn't provide a standard way to configure TLS passthrough in Ingress objects. If the ingress controller supports TLS passthrough, you can usually configure it by adding

annotations to the Ingress object. In the case of the Nginx ingress controller, you add the annotation shown in the following listing.

Listing 12.7 Enabling SSL passthrough in an Ingress when using the Nginx Ingress Controller

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: kiada-ssl-passthrough
  annotations:
    nginx.ingress.kubernetes.io/ssl-passthrough: "true"    #A
spec:
  ...
```

SSL passthrough support in the Nginx ingress controller isn't enabled by default. To enable it, the controller must be started with the `--enable-ssl-passthrough` flag.

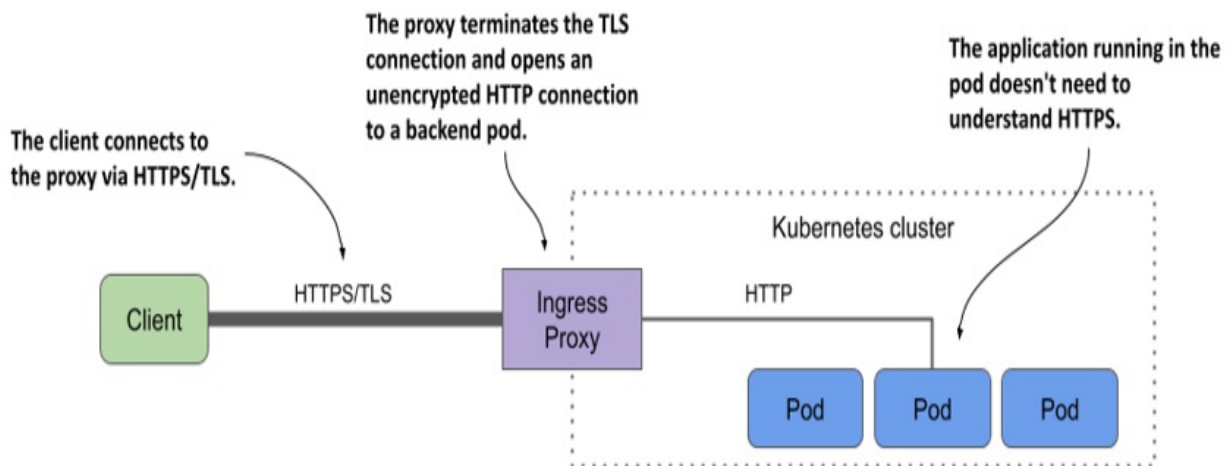
Since this is a non-standard feature that depends heavily on which ingress controller you're using, let's not delve into it any further. For more information on how to enable passthrough in your case, see the documentation of the controller you're using.

Instead, let's focus on terminating the TLS connection at the ingress proxy. This is a standard feature provided by most Ingress controllers and therefore deserves a closer look.

12.3.2 Terminating TLS at the ingress

Most, if not all, ingress controller implementations support TLS termination at the ingress proxy. The proxy terminates the TLS connection between the client and itself and forwards the HTTP request unencrypted to the backend pod, as shown in the next figure.

Figure 12.8 Securing connections to the ingress using TLS



To terminate the TLS connection, the proxy needs a TLS certificate and a private key. You provide them via a Secret that you reference in the Ingress object.

Creating a TLS secret for the Ingress

For the kiada Ingress, you can either create the Secret from the manifest file `secret.tls-example-com.yaml` in the book's code repository or generate the private key, certificate, and Secret with the following commands:

```
$ openssl req -x509 -newkey rsa:4096 -keyout example.key -out example.crt -sha256 -days 7300 -nodes \ #A
-subj '/CN=*.example.com' \ #A
-addext 'subjectAltName = DNS:*.example.com' #A
```

```
$ kubectl create secret tls tls-example-com --cert=example.crt --key=example.key --secret/tls-example-com created #B
```

The certificate and the private key are now stored in a Secret named `tls-example-com` under the keys `tls.crt` and `tls.key`, respectively.

Adding the TLS secret to the Ingress

To add the Secret to the Ingress object, either edit the object with `kubectl edit` and add the lines highlighted in the next listing or apply the

ing.kiada.tls.yaml file with `kubectl apply`.

Listing 12.8 Adding a TLS secret to an Ingress

```
on: networking.k8s.io/v1
kind: Ingress
metadata:
  name: kiada
spec:
  tls:      #A
  - secretName: tls-example-com      #B
    hosts:      #C
    - "*.example.com"      #C
  rules:
  ...
```

As you can see in the listing, the `tls` field can contain one or more entries. Each entry specifies the `secretName` where the TLS certificate/key pair is stored and a list of hosts to which the pair applies.

Warning

The hosts specified in `tls.hosts` must match the names used in the certificate in the secret.

Accessing the Ingress through TLS

After you update the Ingress object, you can access the service via HTTPS as follows:

```
$ curl https://kiada.example.com --resolve kiada.example.com:443:
* Added kiada.example.com:443:11.22.33.44 to DNS cache
* Hostname kiada.example.com was found in DNS cache
*   Trying 11.22.33.44:443...
* Connected to kiada.example.com (11.22.33.44) port 443 (#0)
...
* Server certificate:      #A
*   subject: CN=*.example.com      #A
*   start date: Dec  5 09:48:10 2021 GMT      #A
*   expire date: Nov 30 09:48:10 2041 GMT      #A
*   issuer: CN=*.example.com      #A
...
```

```
> GET / HTTP/2
> Host: kiada.example.com
...
```

The command's output shows that the server certificate matches the one you configured the Ingress with.

By adding the TLS secret to the Ingress, you've not only secured the `kiada` service, but also the `quote` and `quiz` services, since they're all included in the Ingress object. Try to access them through the Ingress using HTTPS. Remember that the pods that provide these two services don't provide HTTPS themselves. The Ingress does that for them.

12.4 Additional Ingress configuration options

I hope you haven't forgotten that you can use the `kubectl explain` command to learn more about a particular API object type, and that you use it regularly. If not, now is a good time to use it to see what else you can configure in an Ingress object's `spec` field. Inspect the output of the following command:

```
$ kubectl explain ingress.spec
```

Look at the list of fields displayed by this command. You may be surprised to see that in addition to the `defaultBackend`, `rules`, and `tls` fields explained in the previous sections, only one other field is supported, namely `ingressClassName`. This field is used to specify which ingress controller should process the Ingress object. You'll learn more about it later. For now, I want to focus on the lack of additional configuration options that HTTP proxies normally provide.

The reason you don't see any other fields for specifying these options is that it would be nearly impossible to include all possible configuration options for every possible ingress implementation in the Ingress object's schema. Instead, these custom options are configured via annotations or in separate custom Kubernetes API objects.

Each ingress controller implementation supports its own set of annotations or

objects. I mentioned earlier that the Nginx ingress controller uses annotations to configure TLS passthrough. Annotations are also used to configure HTTP authentication, session affinity, URL rewriting, redirects, Cross-Origin Resource Sharing (CORS), and more. The list of supported annotations can be found at <https://kubernetes.github.io/ingress-nginx/user-guide/nginx-configuration/annotations/>.

I don't want to go into each of these annotations, since they're implementation specific, but I do want to show you an example of how you can use them.

12.4.1 Configuring the Ingress using annotations

You learned in the previous chapter that Kubernetes services only support client IP-based session affinity. Cookie-based session affinity isn't supported because services operate at Layer 4 of the OSI network model, whereas cookies are part of Layer 7 (HTTP). However, because Ingresses operate at L7, they can support cookie-based session affinity. This is the case with the Nginx ingress controller that I use in the following example.

Using annotations to enable cookie-based session affinity in Nginx ingresses

The following listing shows an example of using Nginx-ingress-specific annotations to enable cookie-based session affinity and configure the session cookie name. The manifest shown in the listing can be found in the `ing.kiada.nginx-affinity.yaml` file.

Listing 12.9 Using annotations to configure session affinity in an Nginx ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: kiada
  annotations:
    nginx.ingress.kubernetes.io/affinity: cookie      #A
    nginx.ingress.kubernetes.io/session-cookie-name: SESSION_COOK
spec:
  ...
```

In the listing, you can see the annotations `nginx.ingress.kubernetes.io/affinity` and `nginx.ingress.kubernetes.io/session-cookie-name`. The first annotation enables cookie-based session affinity, and the second sets the cookie name. The annotation key prefix indicates that these annotations are specific to the Nginx ingress controller and are ignored by other implementations.

Testing the cookie-based session affinity

If you want to see session affinity in action, first apply the manifest file, wait until the Nginx configuration is updated, and then retrieve the cookie as follows:

```
$ curl -I http://kiada.example.com --resolve kiada.example.com:80
HTTP/1.1 200 OK
Date: Mon, 06 Dec 2021 08:58:10 GMT
Content-Type: text/plain
Connection: keep-alive
Set-Cookie: SESSION_COOKIE=1638781091; Path=/; HttpOnly    #A
```

You can now include this cookie in your request by specifying the `Cookie` header:

```
$ curl -H "Cookie: SESSION_COOKIE=1638781091" http://kiada.exempl
--resolve kiada.example.com:80:11.22.33.44
```

If you run this command several times, you'll notice that the HTTP request is always forwarded to the same pod, which indicates that the session affinity is using the cookie.

12.4.2 Configuring the Ingress using additional API objects

Some ingress implementations don't use annotations for additional ingress configuration, but instead provide their own object kinds. In the previous section, you saw how to use annotations to configure session affinity when using the Nginx ingress controller. In the current section, you'll learn how to do the same in Google Kubernetes Engine.

Note

You'll learn how to create your own custom object kinds via the CustomResourceDefinition object in chapter 29.

Using the BackendConfig object type to enable cookie-based session affinity in GKE

In clusters running on GKE, a custom object of type BackendConfig can be found in the Kubernetes API. You create an instance of this object and reference it by name in the Service object to which you want to apply the object. You reference the object using the `cloud.google.com/backend-config` annotations, as shown in the following listing.

Listing 12.10 Referring to a BackendConfig in a Service object in GKE

```
apiVersion: v1
kind: Service
metadata:
  name: kiada
  annotations:
    cloud.google.com/backend-config: '{"default": "kiada-backend-spec":
```

You can use the BackendConfig object to configure many things. Since this object is beyond the scope of this book, use `kubectl explain backendconfig.spec` to learn more about it, or see the GKE documentation.

As a quick example of how custom objects are used to configure ingresses, I'll show you how to configure cookie-based session affinity using the BackendConfig object. You can see the object manifest in the following listing.

Listing 12.11 Using GKE-specific BackendConfig object to configure session affinity

```
apiVersion: cloud.google.com/v1      #A
kind: BackendConfig                  #A
metadata:
  name: kiada-backend-config
spec:
  sessionAffinity:                   #B
    affinityType: GENERATED_COOKIE #B
```

In the listing, the session affinity type is set to `GENERATED_COOKIE`. Since this object is referenced in the `kiada` service, whenever a client accesses the service through the ingress, the request is always routed to the same backend pod.

In this and the previous section, you saw two ways to add custom configuration to an Ingress object. Since the method depends on which ingress controller you're using, see its documentation for more information.

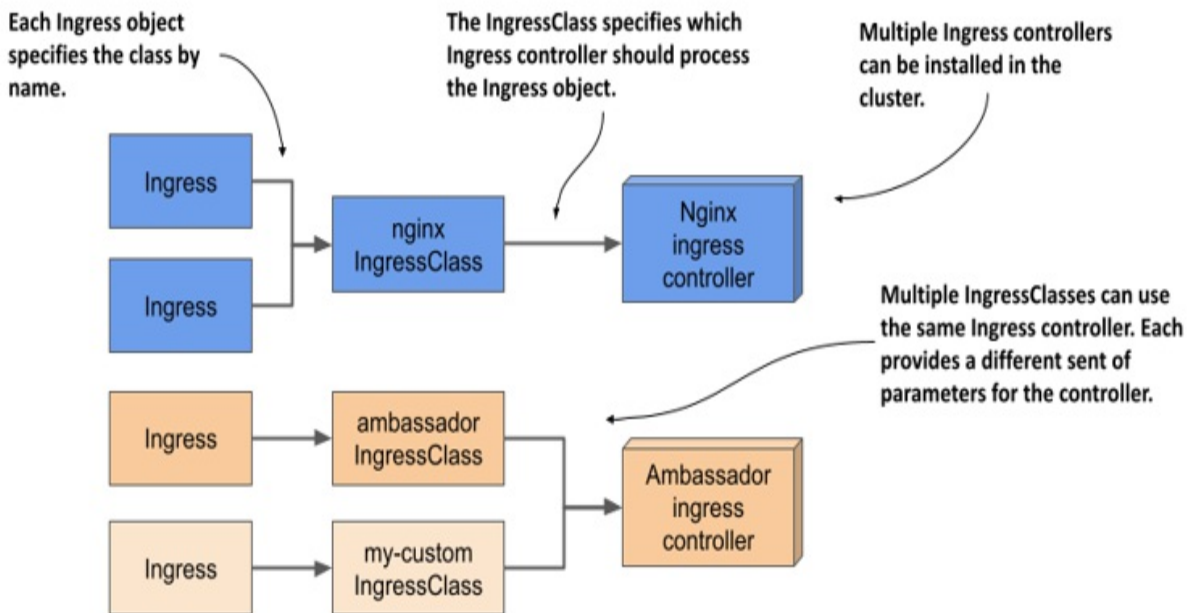
12.5 Using multiple ingress controllers

Since different ingress implementations provide different additional functionality, you may want to install multiple ingress controllers in a cluster. In this case, each Ingress object needs to indicate which ingress controller should process it. Originally, this was accomplished by specifying the controller name in the `kubernetes.io/ingress.class` annotation of the Ingress object. This method is now deprecated, but some controllers still use it.

Instead of using the annotation, the correct way to specify the controller to use is through IngressClass objects. One or more IngressClass objects are usually created when you install an ingress controller.

When you create an Ingress object, you specify the ingress class by specifying the name of the IngressClass object in the Ingress object's `spec` field. Each IngressClass specifies the name of the controller and optional parameters. Thus, the class you reference in your Ingress object determines which ingress proxy is provisioned and how it's configured. As you can see in the next figure, different Ingress objects can reference different IngressClasses, which in turn reference different ingress controllers.

Figure 12.9 The relationship between Ingresses, IngressClasses, and Ingress controllers



12.5.1 Introducing the IngressClass object kind

If the Nginx ingress controller is running in your cluster, an IngressClass object named `nginx` was created when you installed the controller. If other ingress controllers are deployed in your cluster, you may also find other IngressClasses.

Finding IngressClasses in your cluster

To see which ingress classes your cluster offers, you can list them with `kubectl get`:

```
$ kubectl get ingressclasses
NAME          CONTROLLER          PARAMETERS    AGE    #A
nginx         k8s.io/ingress-nginx <none>       10h
```

The output of the command shows that a single IngressClass named `nginx` exists in the cluster. Ingresses that use this class are processed by the `k8s.io/ingress-nginx` controller. You can also see that this class doesn't specify any controller parameters.

Inspecting the YAML manifest of an IngressClass object

Let's take a closer look at the nginx IngressClass object by examining its YAML definition:

```
$ kubectl get ingressclasses nginx -o yaml
apiVersion: networking.k8s.io/v1      #A
kind: IngressClass                    #A
metadata:
  name: nginx                          #B
spec:
  controller: k8s.io/ingress-nginx    #C
```

As you can see, this IngressClass object specifies nothing more than the name of the controller. Later you'll see how you can also add parameters for the controller to the object.

12.5.2 Specifying the IngressClass in the Ingress object

When you create an Ingress object, you can specify the class of the ingress using the `ingressClassName` field in the `spec` section of the Ingress object, as in the following listing.

Listing 12.12 Ingress object referencing a specific IngressClass

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: kiada
spec:
  ingressClassName: nginx      #A
  rules:
  ...
```

The Ingress object in the listing indicates that its class should be `nginx`. Since this IngressClass specifies `k8s.io/ingress-nginx` as the controller, the Ingress from this listing is processed by the Nginx ingress controller.

Setting the default IngressClass

If multiple ingress controllers are installed in the cluster, there should be multiple IngressClass objects. If an Ingress object doesn't specify the class,

Kubernetes applies the default IngressClass, marked as such by setting the `ingressclass.kubernetes.io/is-default-class` annotation to "true".

12.5.3 Adding parameters to an IngressClass

In addition to using IngressClasses to specify which ingress controller to use for a particular Ingress object, IngressClasses can also be used with a single ingress controller if it can provide different ingress flavors. This is achieved by specifying different parameters in each IngressClass.

Specifying parameters in the IngressClass object

The IngressClass object doesn't provide any fields for you to set the parameters within the object itself, as each ingress controller has its own specifics and would require a different set of fields. Instead, the custom configuration of an IngressClass is typically stored in a separate custom Kubernetes object type that's specific to each ingress controller implementation. You create an instance of this custom object type and reference it in the IngressClass object.

For example, AWS provides an object with kind `IngressClassParams` in API group `elbv2.k8s.aws`, version `v1beta1`. To configure the parameters in an IngressClass object, you reference the `IngressClassParams` object instance as shown in the following listing.

Listing 12.13 Referring to a custom parameters object in the IngressClass

```
apiVersion: networking.k8s.io/v1
kind: IngressClass      #A
metadata:
  name: custom-ingress-class
spec:
  controller: ingress.k8s.aws/alb      #B
  parameters:      #C
    apiGroup: elbv2.k8s.aws      #C
    kind: IngressClassParams      #C
    name: custom-ingress-params      #C
```

In the listing, the `IngressClassParams` object instance that contains the

parameters for this IngressClass is named `custom-ingress-params`. The object kind and `apiGroup` are also specified.

Example of a custom API object type used to hold parameters for the IngressClass

The following listing shows an example of an `IngressClassParams` object.

Listing 12.14 Example IngressClassParams object manifest

```
apiVersion: elbv2.k8s.aws/v1beta1      #A
kind: IngressClassParams                #A
metadata:
  name: custom-ingress-params           #B
spec:
  scheme: internal                      #C
  ipAddressType: dualstack              #C
  tags:                                  #C
  - key: org                            #C
    value: my-org                       #C
```

With the `IngressClass` and `IngressClassParams` objects in place, cluster users can create `Ingress` objects with the `ingressClassName` set to `custom-ingress-class`. The objects are processed by the `ingress.k8s.aws/alb` controller (the AWS Load Balancer controller). The controller reads the parameters from the `IngressClassParams` object and uses them to configure the load balancer.

Kubernetes doesn't care about the contents of the `IngressClassParams` object. They're only used by the ingress controller. Since each implementation uses its own object type, you should refer to the controller's documentation or use `kubectl explain` to learn more about each type.

12.6 Using custom resources instead of services as backends

In this chapter, the backends referenced in the `Ingress` have always been `Service` objects. However, some ingress controllers allow you to use other

resources as backends.

Theoretically, an ingress controller could allow using an Ingress object to expose the contents of a ConfigMap or PersistentVolume, but it's more typical for controllers to use resource backends to provide an option for configuring advanced Ingress routing rules through a custom resource.

12.6.1 Using a custom object to configure Ingress routing

The Citrix ingress controller provides the HTTPRoute custom object type, which allows you to configure where the ingress should route HTTP requests. As you can see in the following manifest, you don't specify a Service object as the backend, but you instead specify the kind, apiGroup, and name of the HTTPRoute object that contains the routing rules.

Listing 12.15 Example Ingress object using a resource backend

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
spec:
  ingressClassName: citrix
  rules:
  - host: example.com
    http:
      paths:
      - pathType: ImplementationSpecific
        backend:
          resource:
            apiGroup: citrix.com      #B
            kind: HTTPRoute          #B
            name: my-example-route   #C
```

The Ingress object in the listing specifies a single rule. It states that the ingress controller should forward traffic destined for the host `example.com` according to the configuration specified in the object of the kind `HTTPRoute` (from the API group `citrix.com`) named `my-example-route`. Since the `HTTPRoute` object isn't part of the Kubernetes API, its contents are beyond the scope of this book, but you can probably guess that it contains rules like those in the Ingress object but specified differently and with additional

configuration options.

At the time of writing, ingress controllers that support custom resource backends are rare, but maybe you might want to implement one yourself. By the time you finish reading this book, you'll know how.

12.7 Summary

In this chapter, you learned how to create Ingress objects to make one or more services accessible to external clients. You learned that:

- An Ingress controller configures an L7 load balancer or reverse proxy based on the configuration in the Ingress object.
- While a Service is an abstraction over a set of Pods, an Ingress is an abstraction over a set of Services.
- An Ingress requires a single public IP regardless of the number of services it exposes, whereas each LoadBalancer service requires its own public IP.
- External clients must resolve the hostnames specified in the Ingress object to the IP address of the ingress proxy. To accomplish this, you must add the necessary records to the DNS server responsible for the domain to which the host belongs. Alternatively, for development purposes, you can modify the `/etc/hosts` file on your local machine.
- An Ingress operates at Layer 7 of the OSI model and can therefore provide HTTP-related functionality that Services operating at Layer 4 cannot.
- An Ingress proxy usually forwards HTTP requests directly to the backend pod without going through the service IP, but this depends on the ingress implementation.
- The Ingress object contains rules that specify to which service the HTTP request received by the ingress proxy should be forwarded based on the host and path in the request. Each rule can specify an exact host or one with a wildcard and either an exact path or path prefix.
- The default backend is a catch-all rule that determines which service should handle requests that don't match any rule.
- An Ingress can be configured to expose services over TLS. The Ingress proxy can terminate the TLS connection and forward the HTTP request

to the backend pod unencrypted. Some ingress implementations support TLS passthrough.

- Ingress configuration options that are specific to a particular ingress implementation are set via annotations of the Ingress object or through custom Kubernetes object kinds that the controller provides.
- A Kubernetes cluster can run multiple ingress controller implementations simultaneously. When you create an Ingress object, you specify the IngressClass. The IngressClass object specifies which controller should process the Ingress object. Optionally, the IngressClass can also specify parameters for the controller.

You now understand how to expose groups of pods both internally and externally. In the next chapter, you'll learn how to manage these pods as a unit and replicate them via a Deployment object.

13 Replicating Pods with ReplicaSets

This chapter covers

- Replicating Pods with the ReplicaSet object
- Keeping Pods running when cluster nodes fail
- The reconciliation control loop in Kubernetes controllers
- API Object ownership and garbage collection

So far in this book, you've deployed workloads by creating Pod objects directly. In a production cluster, you might need to deploy dozens or even hundreds of copies of the same Pod, so creating and managing those Pods would be difficult. Fortunately, in Kubernetes, you can automate the creation and management of Pod replicas with the ReplicaSet object.

Note

Before ReplicaSets were introduced, similar functionality was provided by the ReplicationController object type, which is now deprecated. A ReplicationController behaves exactly like a ReplicaSet, so everything that's explained in this chapter also applies to ReplicationControllers.

Before you begin, make sure that the Pods, Services, and other objects of the Kiada suite are present in your cluster. If you followed the exercises in the previous chapter, they should already be there. If not, you can create them by creating the kiada namespace and applying all the manifests in the Chapter13/SETUP/ directory with the following command:

```
$ kubectl apply -f SETUP -R
```

NOTE

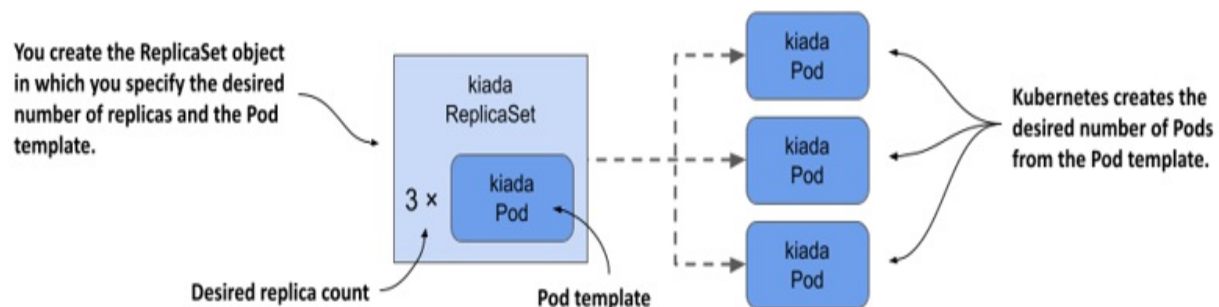
You can find the code files for this chapter at

<https://github.com/luksa/kubernetes-in-action-2nd-edition/tree/master/Chapter13>.

13.1 Introducing ReplicaSets

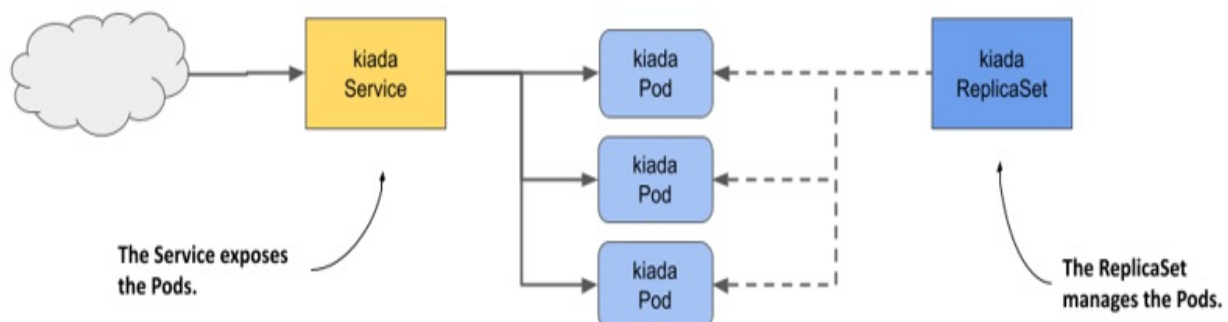
A ReplicaSet represents a group of Pod replicas (exact copies of a Pod). Instead of creating Pods one by one, you can create a ReplicaSet object in which you specify a Pod template and the desired number of replicas, and then have Kubernetes create the Pods, as shown in the following figure.

Figure 13.1 ReplicaSets in a nutshell



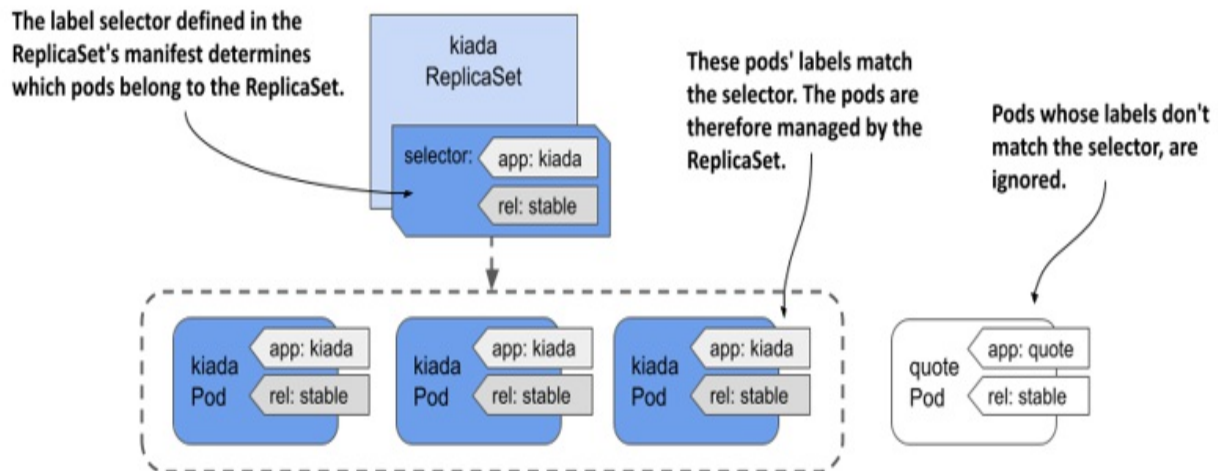
The ReplicaSet allows you to manage the Pods as a single unit, but that's about it. If you want to expose these Pods as one, you still need a Service object. As you can see in the following figure, each set of Pods that provides a particular service usually needs both a ReplicaSet and a Service object.

Figure 13.2 The relationship between Services, ReplicaSets, and Pods.



And just as with Services, the ReplicaSet's label selector and Pod labels determine which Pods belong to the ReplicaSet. As shown in the following figure, a ReplicaSet only cares about the Pods that match its label selector and ignores the rest.

Figure 13.3 A ReplicaSet only cares about Pods that match its label selector



Based on the information so far, you might think that you only use a ReplicaSet if you want to create multiple copies of a Pod, but that's not the case. Even if you only need to create a single Pod, it's better to do it through a ReplicaSet than to create it directly, because the ReplicaSet ensures that the Pod is always there to do its job.

Imagine creating a Pod directly for an important service, and then the node running the Pod fails when you're not there. Your service is down until you recreate the Pod. If you'd deployed the Pod via a ReplicaSet, it would automatically recreate the Pod. It's clearly better to create Pods via a ReplicaSet than directly.

However, as useful as ReplicaSets can be, they don't provide everything you need to run a workload long-term. At some point, you'll want to upgrade the workload to a newer version, and that's where ReplicaSets fall short. For this reason, applications are typically deployed not through ReplicaSets, but through Deployments that let you update them declaratively. This begs the question of why you need to learn about ReplicaSets if you're not going to

use them. The reason is that most of the functionality that a Deployment provides is provided by the ReplicaSets that Kubernetes creates underneath it. Deployments take care of updates, but everything else is handled by the underlying ReplicaSets. Therefore, it's important to understand what they do and how.

13.1.1 Creating a ReplicaSet

Let's start by creating the ReplicaSet object for the Kiada service. The service currently runs in three Pods that you created directly from three separate Pod manifests, which you'll now replace with a single ReplicaSet manifest. Before you create the manifest, let's look at what fields you need to specify in the spec section.

Introducing the ReplicaSet spec

A ReplicaSet is a relatively simple object. The following table explains the three key fields you specify in the ReplicaSet's spec section.

Table 13.1 The main fields in the ReplicaSet specification

Field name	Description
replicas	The desired number of replicas. When you create the ReplicaSet object, Kubernetes creates this many Pods from the Pod template. It keeps this number of Pods until you delete the ReplicaSet.
selector	The label selector contains either a map of labels in the matchLabels subfield or a list of label selector requirements in the matchExpressions subfield. Pods that match the label selector are considered part of this ReplicaSet.

template	The Pod template for the ReplicaSet's Pods. When a new Pod needs to be created, the object is created using this template.
----------	--

The selector and template fields are required, but you can omit the replicas field. If you do, a single replica is created.

Creating a ReplicaSet object manifest

Create a ReplicaSet object manifest for the Kiada Pods. The following listing shows what it looks like. You can find the manifest in the file `rs.kiada.yaml`.

Listing 13.1 The kiada ReplicaSet object manifest

```
apiVersion: apps/v1      #A
kind: ReplicaSet        #A
metadata:
  name: kiada           #B
spec:
  replicas: 5           #C
  selector:             #D
    matchLabels:        #D
      app: kiada        #D
      rel: stable       #D
  template:             #E
    metadata:           #E
      labels:           #E
        app: kiada      #E
        rel: stable     #E
    spec:               #E
      containers:       #E
        - name: kiada    #E
          image: luksa/kiada:0.5    #E
          ...            #E
      volumes:          #E
        - ...           #E
```

ReplicaSets are part of the apps API group, version v1. As explained in the previous table, the replicas field specifies that this ReplicaSet should create three copies of the Pod using the template in the template field.

You'll notice that the `labels` in the Pod template match those in the `selector` field. If they don't, the Kubernetes API will reject the `ReplicaSet` because the Pods created with the template won't count against the desired number of replicas, which would result in the creation of an infinite number of Pods.

Did you notice that there's no Pod name in the template? That's because the Pod names are generated from the `ReplicaSet` name.

The rest of the template exactly matches the manifests of the `kiada` Pods you created in the previous chapters. To create the `ReplicaSet`, you use the `kubectl apply` command that you've used many times before. The command is as follows:

```
$ kubectl apply -f rs.kiada.yaml
replicaset.apps/kiada created
```

13.1.2 Inspecting a ReplicaSet and its Pods

To display basic information about the `ReplicaSet` you just created, use the `kubectl get` command like so:

```
$ kubectl get rs kiada
NAME      DESIRED   CURRENT   READY   AGE
kiada     5         5         5       1m
```

Note

The shorthand for `replicaset` is `rs`.

The output of the command shows the desired number of replicas, the current number of replicas, and the number of replicas that are considered ready as reported by their readiness probes. This information is read from the `replicas`, `fullyLabeledReplicas`, and `readyReplicas` status fields of the `ReplicaSet` object, respectively. Another status field called `availableReplicas` indicates how many replicas are available, but its value isn't displayed by the `kubectl get` command.

If you run the `kubectl get replicaset` command with the `-o wide` option,

some additional very useful information is displayed. Run the following command to find out what:

```
$ kubectl get rs -o wide
NAME      ...   CONTAINERS   IMAGES
kiada ... kiada,envoy luksa/kiada:0.5,envoyproxy/envoy:v1.14.1 ap
```

In addition to the columns displayed previously, this expanded output shows not only the label selector, but also the container names and images used in the Pod template. Considering how important this information is, it's surprising that it's not displayed when listing the Pods with `kubectl get pods`.

Tip

To see container and image names, list ReplicaSets with the `-o wide` option instead of trying to get this information from the Pods.

To see all the information about a ReplicaSet, use the `kubectl describe` command:

```
$ kubectl describe rs kiada
```

The output shows the label selector used in the ReplicaSet, the number of Pods and their status, and the full template used to create those Pods.

Listing the Pods in a ReplicaSet

Kubectl doesn't provide a direct way to list the Pods in a ReplicaSet, but you can take the ReplicaSet's label selector and use it in the `kubectl get pods` command as follows:

```
$ kubectl get po -l app=kiada,rel=stable
NAME          READY   STATUS    RESTARTS   AGE   #A
kiada-001     2/2     Running   0           12m   #A
kiada-002     2/2     Running   0           12m   #A
kiada-003     2/2     Running   0           12m   #A
kiada-86wzp   2/2     Running   0            8s   #B
kiada-k9hn2   2/2     Running   0            8s   #B
```


Before you created the ReplicaSet, you had three kiada Pods from the previous chapters and now you have five, which is the desired number of replicas defined in the ReplicaSet. The labels of the three existing Pods matched the ReplicaSet's label selector and were adopted by the ReplicaSet. Two additional Pods were created to ensure that the number of Pods in the set matched the desired number of replicas.

Understanding how Pods in a ReplicaSet are named

As you can see, the names of the two new Pods contain five random alphanumeric characters instead of continuing the sequence of numbers you used in your Pod names. It's typical for Kubernetes to assign random names to the objects it creates.

There's even a special metadata field that lets you create objects without giving the full name. Instead of the name field, you specify the name prefix in the generateName field. You first used this field in chapter 8, when you ran the `kubectl create` command several times to create multiple copies of a Pod and give each a unique name. The same approach is used when Kubernetes creates Pods for a ReplicaSet.

When Kubernetes creates Pods for a ReplicaSet, it sets the generateName field to match the ReplicaSet name. The Kubernetes API server then generates the full name and puts it in the name field. To see this, select one of the two additional Pods that were created and check its metadata section as follows:

```
$ kubectl get po kiada-86wzp -o yaml
apiVersion: v1
kind: Pod
metadata:
  generateName: kiada-      #A
  labels:
    ...
  name: kiada-86wzp      #B
  ...
```

In the case of ReplicaSet Pods, giving the Pods random names makes sense because these Pods are exact copies of each other and therefore fungible.

There's also no concept of order between these Pods, so the use of sequential numbers is nonsensical. Even though the Pod names look reasonable now, imagine what happens if you delete some of them. If you delete them out of order, the numbers are no longer consecutive. However, for stateful workloads, it may make sense to number the Pods sequentially. That's what happens when you use a StatefulSet object to create the Pods. You'll learn more about StatefulSets in chapter 16.

Displaying the logs of the ReplicaSet's Pods

The random names of ReplicaSet Pods make them somewhat difficult to work with. For example, to view the logs of one of these Pods, it's relatively tedious to type the name of the Pod when you run the `kubectl logs` command. If the ReplicaSet contains only a single Pod, entering the full name seems unnecessary. Fortunately, in this case, you can print the Pod's logs as follows:

```
$ kubectl logs rs/kiada -c kiada
```

So instead of specifying the Pod name, you type `rs/kiada`, where `rs` is the abbreviation for ReplicaSet and `kiada` is the name of the ReplicaSet object. The `-c kiada` option tells `kubectl` to print the log of the `kiada` container. You need to use this option only if the Pod has more than one container. If the ReplicaSet has multiple Pods, as in your case, only the logs of one of the Pods will be displayed.

If you want to see the logs of all the Pods, you can run the `kubectl logs` command with a label selector instead. For example, to stream the logs of the `envoy` containers in all `kiada` Pods, run the following command:

```
$ kubectl logs -l app=kiada -c envoy
```

To display the logs of all containers, use the `--all-containers` option instead of specifying the container name. Of course, if you're displaying the logs of multiple Pods or containers, you can't tell where each line came from. Use the `--prefix` option to prefix each log line with the name of the Pod and container it came from, like this:

```
$ kubectl logs -l app=kiada --all-containers --prefix
```

Viewing logs from multiple Pods is very useful when traffic is split between Pods and you want to view every request received, regardless of which Pod handled it. For example, try streaming the logs with the following command:

```
$ kubectl logs -l app=kiada -c kiada --prefix -f
```

Now open the application in your web browser or with `curl`. Use the Ingress, LoadBalancer, or NodePort service as explained in the previous two chapters.

13.1.3 Understanding Pod ownership

Kubernetes created the two new Pods from the template you specified in the ReplicaSet object. They're owned and controlled by the ReplicaSet, just like the three Pods you created manually. You can see this when you use the `kubectl describe` command to inspect the Pods. For example, check the `kiada-001` Pod as follows:

```
$ kubectl describe po kiada-001
Name:          kiada-001
Namespace:     kiada
...
Controlled By: ReplicaSet/kiada    #A
...
```

The `kubectl describe` command gets this information from the metadata section of the Pod's manifest. Let's take a closer look. Run the following command:

```
$ kubectl get po kiada-001 -o yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    app: kiada
    rel: stable
  name: kiada-001
  namespace: kiada
  ownerReferences:    #A
  - apiVersion: apps/v1    #A
    blockOwnerDeletion: true    #A
```

```
    controller: true      #A
    kind: ReplicaSet      #A
    name: kiada           #A
    uid: 8e19d9b3-bbf1-4830-b0b4-da81dd0e6e22      #A
    resourceVersion: "527511"
    uid: d87afa5c-297d-4ccb-bb0a-9eb48670673f
spec:
  ...
```

The metadata section in an object manifest sometimes contains the `ownerReferences` field, which contains references to the owner(s) of the object. This field can contain multiple owners, but most objects have only a single owner, just like the `kiada-001` Pod. In the case of this Pod, the `kiada` ReplicaSet is the *owner*, and the Pod is the so-called *dependent*.

Kubernetes has a garbage collector that automatically deletes dependent objects when their owner is deleted. If an object has multiple owners, the object is deleted when all its owners are gone. If you delete the ReplicaSet object that owns the `kiada-001` and the other Pods, the garbage collector would also delete the Pods.

An owner reference can also indicate which owner is the controller of the object. The `kiada-001` Pod is controlled by the `kiada` ReplicaSet, as indicated by the `controller: true` line in the manifest. This means that you should no longer control the three Pods directly, but through the ReplicaSet object.

13.2 Updating a ReplicaSet

In a ReplicaSet, you specify the desired number of replicas, a Pod template, and a label selector. The selector is immutable, but you can update the other two properties. By changing the desired number of replicas, you scale the ReplicaSet. Let's see what happens when you do that.

13.2.1 Scaling a ReplicaSet

In the ReplicaSet, you've set the desired number of replicas to five, and that's the number of Pods currently owned by the ReplicaSet. However, you can now update the ReplicaSet object to change this number. You can do this

either by changing the value in the manifest file and reapplying it, or by editing the object directly with the `kubectl edit` command. However, the easiest way to scale a ReplicaSet is to use the `kubectl scale` command.

Scaling a ReplicaSet using the `kubectl scale` command

Let's increase the number of kiada Pods to six. To do this, execute the following command:

```
$ kubectl scale rs kiada --replicas 6
replicaset.apps/kiada scaled
```

Now check the ReplicaSet again to confirm that it now has six Pods:

```
$ kubectl get rs kiada
```

NAME	DESIRED	CURRENT	READY	AGE
kiada	6	6	5	10m

The columns indicate that the ReplicaSet is now configured with six Pods, and this is also the current number of Pods. One of the Pods isn't yet ready, but only because it was just created. List the Pods again to confirm that an additional Pod instance has been created:

```
$ kubectl get po -l app=kiada,rel=stable
```

NAME	READY	STATUS	RESTARTS	AGE	
kiada-001	2/2	Running	0	22m	
kiada-002	2/2	Running	0	22m	
kiada-003	2/2	Running	0	22m	
kiada-86wzp	2/2	Running	0	10m	
kiada-dmskr	2/2	Running	0	11s	#A
kiada-k9hn2	2/2	Running	0	10m	

As expected, a new Pod was created, bringing the total number of Pods to the desired six. If this application served actual users and you needed to scale to a hundred Pods or more due to increased traffic, you could do so in a snap with the same command. However, your cluster may not be able to handle that many Pods.

Scaling down

Just as you scale up a ReplicaSet, you can also scale it down with the same command. You can also scale a ReplicaSet by editing its manifest with `kubectl edit`. Let's scale it to four replicas using this method. Run the following command:

```
$ kubectl edit rs kiada
```

This should open the ReplicaSet object manifest in your text editor. Find the `replicas` field and change the value to 4. Save the file and close the editor so `kubectl` can post the updated manifest to the Kubernetes API. Verify that you now have four Pods:

```
$ kubectl get pods -l app=kiada,rel=stable
```

NAME	READY	STATUS	RESTARTS	AGE	
kiada-001	2/2	Running	0	28m	
kiada-002	2/2	Running	0	28m	
kiada-003	2/2	Running	0	28m	
kiada-86wzp	0/2	Terminating	0	16m	#A
kiada-dmslr	2/2	Terminating	0	125m	#A
kiada-k9hn2	2/2	Running	0	16m	

As expected, two of the Pods are being terminated and should disappear when the processes in their containers stop running. But how does Kubernetes decide which Pods to remove? Does it just select them randomly?

Understanding which Pods are deleted first when a ReplicaSet is scaled down

When you scale down a ReplicaSet, Kubernetes follows some well thought out rules to decide which Pod(s) to delete first. It deletes Pods in the following order:

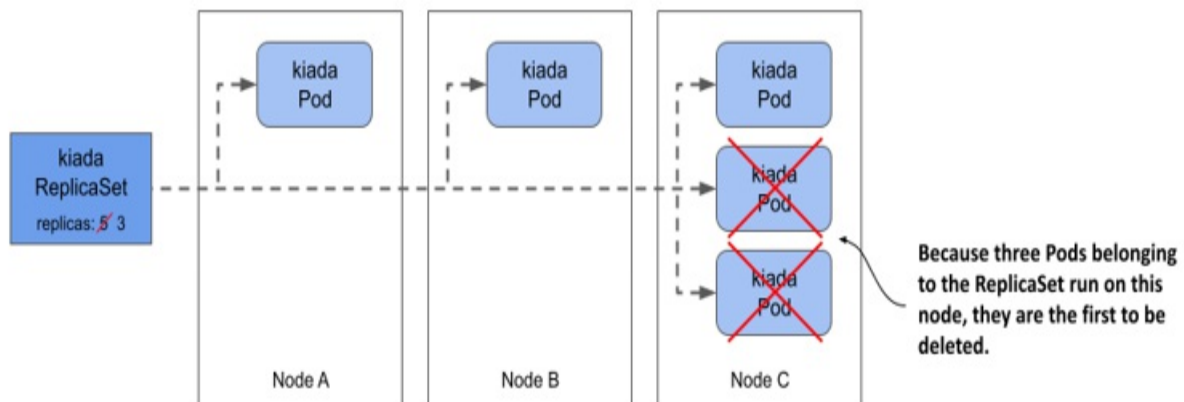
1. Pods that aren't yet assigned to a node.
2. Pods whose phase is unknown.
3. Pods that aren't ready.
4. Pods that have a lower deletion cost.
5. Pods that are collocated with a greater number of related replicas.
6. Pods that have been ready for a shorter time.
7. Pods with a greater number of container restarts.

8. Pods that were created later than the other Pods.

These rules ensure that Pods that haven't been scheduled yet, and defective Pods are deleted first, while the well-functioning ones are left alone. You can also influence which Pod is deleted first by setting the annotation `controller.kubernetes.io/pod-deletion-cost` on your Pods. The value of the annotation must be a string that can be parsed into a 32-bit integer. Pods without this annotation and those with a lower value will be deleted before Pods with higher values.

Kubernetes also tries to keep the Pods evenly distributed across the cluster nodes. The following figure shows an example where the ReplicaSet is scaled from five to three replicas. Because the third node runs two collocated replicas more than the other two nodes, the Pods on the third node are deleted first. If this rule didn't exist, you could end up with three replicas on a single node.

Figure 13.4 Kubernetes keeps related Pods evenly distributed across the cluster nodes.



Scaling down to zero

In some cases, it's useful to scale the number of replicas down to zero. All Pods managed by the ReplicaSet will be deleted, but the ReplicaSet object itself will remain and can be scaled back up at will. You can try this now by running the following commands:

```
$ kubectl scale rs kiada --replicas 0    #A
```

```

replicaset.apps/kiada scaled

$ kubectl get po -l app=kiada      #B
No resources found in kiada namespace.      #B

$ kubectl scale rs kiada --replicas 2      #C
replicaset.apps/kiada scaled

$ kubectl get po -l app=kiada
NAME          READY   STATUS    RESTARTS   AGE   #D
kiada-dl7vz   2/2     Running   0           6s    #D
kiada-dn9fb   2/2     Running   0           6s    #D

```

As you'll see in the next chapter, a ReplicaSet scaled to zero is very common when the ReplicaSet is owned by a Deployment object.

Tip

If you need to temporarily shut down all instances of your workload, set the desired number of replicas to zero instead of deleting the ReplicaSet object.

13.2.2 Updating the Pod template

In the next chapter, you'll learn about the Deployment object, which differs from ReplicaSets in how it handles Pod template updates. This difference is why you usually manage Pods with Deployments and not ReplicaSets. Therefore, it's important to see what ReplicaSets don't do.

Editing a ReplicaSet's Pod template

The kiada Pods currently have labels that indicate the name of the application and the release type (whether it's a stable release or something else). It would be great if a label indicated the exact version number, so you can easily distinguish between them when you run different versions simultaneously.

To add a label to the Pods that the ReplicaSet creates, you must add the label to its Pod template. You can't add the label with the `kubectl label` command, because then it would be added to the ReplicaSet itself and not to the Pod template. There's no `kubectl` command that does this, so you must edit the manifest with `kubectl edit` as you did before. Find the `template`

field and add the label key `ver` with value `0.5` to the `metadata.labels` field in the template, as shown in the following listing.

Listing 13.2 Adding a label to the Pod template

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  ...
spec:
  replicas: 2
  selector:      #A
    matchLabels:  #A
      app: kiada   #A
      rel: stable  #A
  template:
    metadata:
      labels:
        app: kiada
        rel: stable
        ver: '0.5'   #B
    spec:
      ...
```

Make sure you add the label in the right place. Don't add it to the selector, as this would cause the Kubernetes API to reject your update, since the selector is immutable. The version number must be enclosed in quotes, otherwise the YAML parser will interpret it as a decimal number and the update will fail, since label values must be strings. Save the file and close the editor so that `kubect1` can post the updated manifest to the API server.

Note

Did you notice that the labels in the Pod template and those in the selector aren't identical? They don't have to be identical, but the labels in the selector must be a subset of the labels in the template.

Understanding how the ReplicaSet's Pod template is used

You updated the Pod template, now check if the change is reflected in the Pods. List the Pods and their labels as follows:

```
$ kubectl get pods -l app=kiada --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS
kiada-dl7vz	2/2	Running	0	10m	app=kiada,rel=st
kiada-dn9fb	2/2	Running	0	10m	app=kiada,rel=st

Since the Pods still only have the two labels from the original Pod template, it's clear that Kubernetes didn't update the Pods. However, if you now scale the ReplicaSet up by one, the new Pod should contain the label you added, as shown here:

```
$ kubectl scale rs kiada --replicas 3
replicaset.apps/kiada scaled
```

```
$ kubectl get pods -l app=kiada --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS
kiada-dl7vz	2/2	Running	0	14m	app=kiada,rel=st
kiada-dn9fb	2/2	Running	0	14m	app=kiada,rel=st
kiada-z9dp2	2/2	Running	0	47s	app=kiada,rel=st

You should think of the Pod template as a cookie cutter that Kubernetes uses to cut out new Pods. When you change the Pod template, only the cookie cutter changes and that only affects the Pods that are created afterwards.

13.3 Understanding the operation of the ReplicaSet controller

In the previous sections, you saw how changing the replicas and template within the ReplicaSet object causes Kubernetes to do something with the Pods that belong to the ReplicaSet. The Kubernetes component that performs these actions is called the controller. Most of the object types you create through your cluster's API have an associated controller. For example, in the previous chapter you learned about the Ingress controller, which manages Ingress objects. There's also the Endpoints controller for the Endpoints objects, the Namespace controller for the Namespace objects, and so on.

Not surprisingly, ReplicaSets are managed by the ReplicaSet controller. Any change you make to a ReplicaSet object is detected and processed by this controller. When you scale the ReplicaSet, the controller is the one that creates or deletes the Pods. Each time it does this, it also creates an Event

object that informs you of what it's done. As you learned in chapter 4, you can see the events associated with an object at the bottom of the output of the `kubectl describe` command as shown in the next snippet, or by using the `kubectl get events` command to specifically list the Event objects.

```
$ kubectl describe rs kiada
```

```
...
```

```
Events:
```

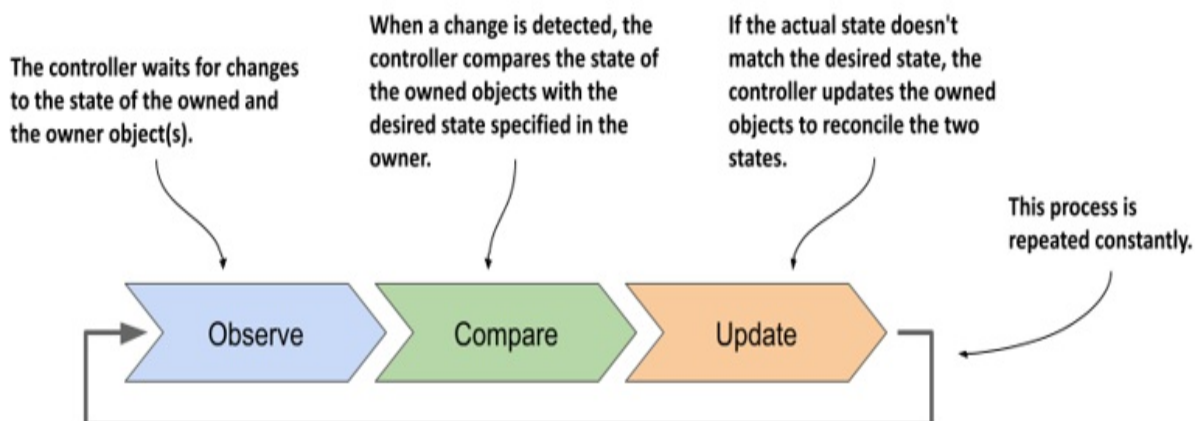
Type	Reason	Age	From	Message
----	-----	----	----	-----
Normal	SuccessfulDelete	34m	replicaset-controller	Deleted
Normal	SuccessfulCreate	30m	replicaset-controller	Created
Normal	SuccessfulCreate	30m	replicaset-controller	Created
Normal	SuccessfulCreate	16m	replicaset-controller	Created

To understand ReplicaSets, you must understand the operation of their controller.

13.3.1 Introducing the reconciliation control loop

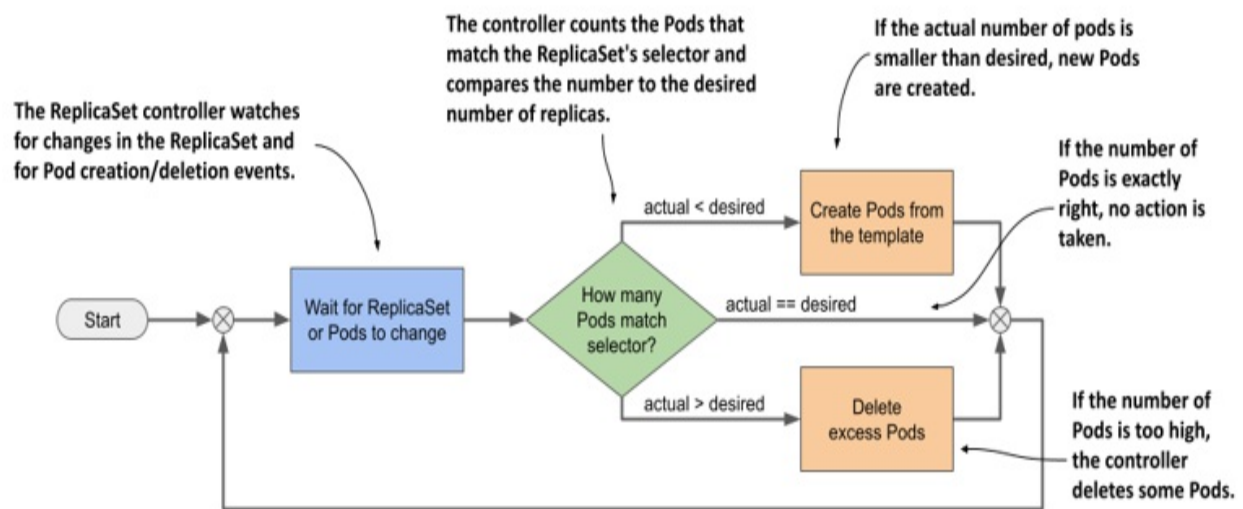
As shown in the following figure, a controller observes the state of both the owner and the dependent objects. After each change in this state, the controller compares the state of the dependent objects with the desired state specified in the owning object. If these two states differ, the controller makes changes to the dependent object(s) to reconcile the two states. This is the so-called reconciliation control loop that you'll find in all controllers.

Figure 13.5 A controller's reconciliation control loop



The ReplicaSet controller's reconciliation control loop consists of observing ReplicaSets and Pods. Each time a ReplicaSet or Pod changes, the controller checks the list of Pods associated with the ReplicaSet and ensures that the actual number of Pods matches the desired number specified in the ReplicaSet. If the actual number of Pods is lower than the desired number, it creates new replicas from the Pod template. If the number of Pods is higher than desired, it deletes the excess replicas. The flowchart in the following figure explains the entire process.

Figure 13.6 The ReplicaSet controller's reconciliation loop



13.3.2 Understanding how the ReplicaSet controller reacts to Pod changes

You've seen how the controller responds immediately to changes in the ReplicaSet's replicas field. However, that's not the only way the desired number and the actual number of Pods can differ. What if no one touches the ReplicaSet, but the actual number of Pods changes? The ReplicaSet controller's job is to make sure that the number of Pods always matches the specified number. Therefore, it should also come into action in this case.

Deleting a Pod managed by a ReplicaSet

Let's look at what happens if you delete one of the Pods managed by the ReplicaSet. Select one and delete it with `kubectl delete`:

```
$ kubectl delete pod kiada-z9dp2      #A
pod "kiada-z9dp2" deleted
```

Now list the Pods again:

```
$ kubectl get pods -l app=kiada
NAME          READY   STATUS    RESTARTS   AGE
kiada-dl7vz    2/2     Running   0           34m
kiada-dn9fb    2/2     Running   0           34m
kiada-rfkqb    2/2     Running   0           47s      #A
```

The Pod you deleted is gone, but a new Pod has appeared to replace the missing Pod. The number of Pods again matches the desired number of replicas set in the ReplicaSet object. Again, the ReplicaSet controller reacted immediately and reconciled the actual state with the desired state.

Even if you delete all kiada Pods, three new ones will appear immediately so that they can serve your users. You can see this by running the following command:

```
$ kubectl delete pod -l app=kiada
```

Creating a Pod that matches the ReplicaSet's label selector

Just as the ReplicaSet controller creates new Pods when it finds that there are fewer Pods than needed, it also deletes Pods when it finds too many. You've already seen this happen when you reduced the desired number of replicas, but what if you manually create a Pod that matches the ReplicaSet's label selector? From the controller's point of view, one of the Pods must disappear.

Let's create a Pod called `one-kiada-too-many`. The name doesn't match the prefix that the controller assigns to the ReplicaSet's Pods, but the Pod's labels match the ReplicaSet's label selector. You can find the Pod manifest in the file `pod.one-kiada-too-many.yaml`. Apply the manifest with `kubectl apply` to create the Pod, and then immediately list the kiada Pods as follows:

```
$ kubectl get po -l app=kiada
```

NAME	READY	STATUS	RESTARTS	AGE	
kiada-jp4vh	2/2	Running	0	11m	
kiada-r4k9f	2/2	Running	0	11m	
kiada-shfgj	2/2	Running	0	11m	
one-kiada-too-many	0/2	Terminating	0	3s	#A

As expected, the ReplicaSet controller deletes the Pod as soon as it detects it. The controller doesn't like it when you create Pods that match the label selector of a ReplicaSet. As shown, the name of the Pod doesn't matter. Only the Pod's labels matter.

What happens when a node that runs a ReplicaSet's Pod fails?

In the previous examples, you saw how a ReplicaSet controller reacts when someone tampers with the Pods of a ReplicaSet. Although these examples do a good job of illustrating how the ReplicaSet controller works, they don't really show the true benefit of using a ReplicaSet to run Pods. The best reason to create Pods via a ReplicaSet instead of directly is that the Pods are automatically replaced when your cluster nodes fail.

Warning

In the next example, a cluster node is caused to fail. In a poorly configured cluster, this can cause the entire cluster to fail. Therefore, you should only perform this exercise if you're willing to rebuild the cluster from scratch if necessary.

To see what happens when a node stops responding, you can disable its network interface. If you created your cluster with the kind tool, you can disable the network interface of the kind-worker2 node with the following command:

```
$ docker exec kind-worker2 ip link set eth0 down
```

Note

Pick a node that has at least one of your kiada Pods running on it. List the Pods with the `-o wide` option to see which node each Pod runs on.

Note

If you're using GKE, you can log into the node with the `gcloud compute ssh` command and shut down its network interface with the `sudo ifconfig eth0 down` command. The ssh session will stop responding, so you'll need to close it by pressing Enter, followed by “~.” (tilde and dot, without the quotes).

Soon, the status of the Node object representing the cluster node changes to `NotReady`:

```
$ kubectl get node
```

NAME	STATUS	ROLES	AGE	VER
kind-control-plane	Ready	control-plane,master	2d3h	v1.
kind-worker	Ready	<none>	2d3h	v1.
kind-worker2	NotReady	<none>	2d3h	v1.

This status indicates that the Kubelet running on the node hasn't contacted the API server for some time. Since this isn't a clear sign that the node is down, as it could just be a temporary network glitch, this doesn't immediately affect the status of the Pods running on the node. They'll continue to show as `Running`. However, after a few minutes, Kubernetes realizes that the node is down and marks the Pods for deletion.

Note

The time that elapses between a node becoming unavailable and its Pods being deleted can be configured using the *Taints and Tolerations* mechanism, which is explained in chapter 23.

Once the Pods are marked for deletion, the ReplicaSet controller creates new Pods to replace them. You can see this in the following output.

```
$ kubectl get pods -l app=kiada -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP
kiada-ffstj	2/2	Running	0	35s	10.244.1.150
kiada-l2r85	2/2	Terminating	0	37m	10.244.2.173
kiada-n98df	2/2	Terminating	0	37m	10.244.2.174
kiada-vnc4b	2/2	Running	0	37m	10.244.1.148
kiada-wkpsn	2/2	Running	0	35s	10.244.1.151

As you can see in the output, the two Pods on the `kind-worker2` node are marked as `Terminating` and have been replaced by two new Pods scheduled to the healthy node `kind-worker`. Again, three Pod replicas are running as specified in the `ReplicaSet`.

The two Pods that are being deleted remain in the `Terminating` state until the node comes back online. In reality, the containers in those Pods are still running because the Kubelet on the node can't communicate with the API server and therefore doesn't know that they should be terminated. However, when the node's network interface comes back online, the Kubelet terminates the containers, and the Pod objects are deleted. The following commands restore the node's network interface:

```
$ docker exec kind-worker2 ip link set eth0 up
$ docker exec kind-worker2 ip route add default via 172.18.0.1
```

Your cluster may be using a gateway IP other than `172.18.0.1`. To find it, run the following command:

```
$ docker network inspect kind -f '{{ (index .IPAM.Config 0).Gateway
```

Note

If you're using GKE, you must remotely reset the node with the `gcloud compute instances reset <node-name>` command.

When do Pods not get replaced?

The previous sections have demonstrated that the `ReplicaSet` controller ensures that there are always as many healthy Pods as specified in the `ReplicaSet` object. But is this always the case? Is it possible to get into a state where the number of Pods matches the desired replica count, but the Pods can't provide the service to their clients?

Remember the liveness and readiness probes? If a container's liveness probe fails, the container is restarted. If the probe fails multiple times, there's a significant time delay before the container is restarted. This is due to the exponential backoff mechanism explained in chapter 6. During the backoff

delay, the container isn't in operation. However, it's assumed that the container will eventually be back in service. If the container fails the readiness rather than the liveness probe, it's also assumed that the problem will eventually be fixed.

For this reason, Pods whose containers continually crash or fail their probes are never automatically deleted, even though the ReplicaSet controller could easily replace them with Pods that might run properly. Therefore, be aware that a ReplicaSet doesn't guarantee that you'll always have as many healthy replicas as you specify in the ReplicaSet object.

You can see this for yourself by failing one of the Pods' readiness probes with the following command:

```
$ kubectl exec rs/kiada -c kiada -- curl -X POST localhost:9901/h
```

Note

If you specify the ReplicaSet instead of the Pod name when running the `kubectl exec` command, the specified command is run in one of the Pods, not all of them, just as with `kubectl logs`.

After about thirty seconds, the `kubectl get pods` command indicates that one of the Pod's containers is no longer ready:

```
$ kubectl get pods -l app=kiada
```

NAME	READY	STATUS	RESTARTS	AGE	
kiada-78j7m	1/2	Running	0	21m	#A
kiada-98lmx	2/2	Running	0	21m	
kiada-wk99p	2/2	Running	0	21m	

The Pod no longer receives any traffic from the clients, but the ReplicaSet controller doesn't delete and replace it, even though it's aware that only two of the three Pods are ready and accessible, as indicated by the ReplicaSet status:

```
$ kubectl get rs
```

NAME	DESIRED	CURRENT	READY	AGE	
kiada	3	3	2	2h	#A

IMPORTANT

A ReplicaSet only ensures that the desired number of Pods are present. It doesn't ensure that their containers are actually running and ready to handle traffic.

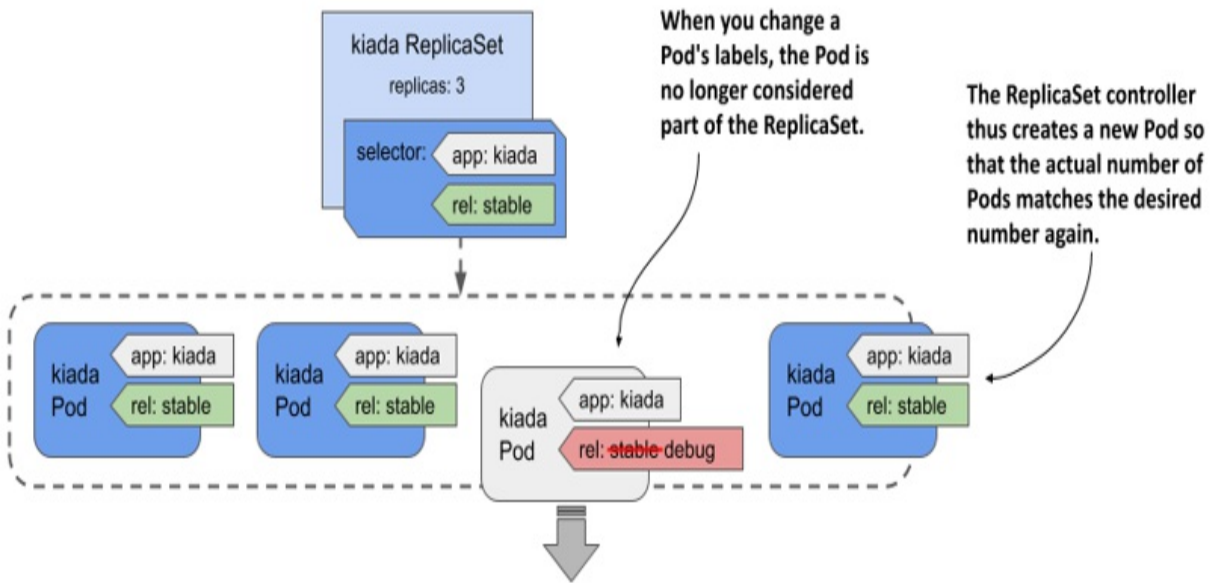
If this happens in a real production cluster and the remaining Pods can't handle all the traffic, you'll have to delete the bad Pod yourself. But what if you want to find out what's wrong with the Pod first? How can you quickly replace the faulty Pod without deleting it so you can debug it?

You could scale the ReplicaSet up by one replica, but then you'll have to scale back down when you finish debugging the faulty Pod. Fortunately, there's a better way. It'll be explained in the next section.

13.3.3 Removing a Pod from the ReplicaSet's control

You already know that the ReplicaSet controller is constantly making sure that the number of Pods that match the ReplicaSet's label selector matches the desired number of replicas. So, if you remove a Pod from the set of Pods that match the selector, the controller replaces it. To do this, you simply change the labels of the faulty Pod, as shown in the following figure.

Figure 13.7 Changing a Pod's labels to remove it from the ReplicaSet



The ReplicaSet controller replaces the Pod with a new one, and from that point on, no longer pays attention to the faulty Pod. You can calmly figure out what's wrong with it while the new Pod takes over the traffic.

Let's try this with the Pod whose readiness probe you failed in the previous section. For a Pod to match the ReplicaSet's label selector, it must have the labels `app=kiada` and `rel=stable`. Pods without these labels aren't considered part of the ReplicaSet. So, to remove the broken Pod from the ReplicaSet, you need to remove or change at least one of these two labels. One way is to change the value of the `rel` label to `debug` as follows:

```
$ kubectl label po kiada-78j7m rel=debug --overwrite
```

Since only two Pods now match the label selector, one less than the desired number of replicas, the controller immediately creates another Pod, as shown in the following output:

```
$ kubectl get pods -l app=kiada -L app,rel
NAME          READY   STATUS    RESTARTS   AGE   APP    REL
kiada-78j7m   1/2     Running   0           60m   kiada  debug
kiada-98lmx   2/2     Running   0           60m   kiada  stable
kiada-wk99p   2/2     Running   0           60m   kiada  stable
kiada-xtxcl   2/2     Running   0           9s    kiada  stable
```

As you can see from the values in the APP and REL columns, three Pods match

the selector, while the broken Pod doesn't. This Pod is no longer managed by the ReplicaSet. Therefore, when you're done inspecting the Pod, you need to delete it manually.

Note

When you remove a Pod from a ReplicaSet, the reference to the ReplicaSet object is removed from the Pod's ownerReferences field.

Now that you've seen how the ReplicaSet controller responds to all the events shown in this and previous sections, you understand everything you need to know about this controller.

13.4 Deleting a ReplicaSet

A ReplicaSet represents a group of Pod replicas that you manage as a unit. By creating a ReplicaSet object, you indicate that you want a specific number of Pod replicas based on a specific Pod template in your cluster. By deleting the ReplicaSet object, you indicate that you no longer want those Pods. So when you delete a ReplicaSet, all the Pods that belong to it are also deleted. This is done by the garbage collector, as explained earlier in this chapter.

13.4.1 Deleting a ReplicaSet and all associated Pods

To delete a ReplicaSet and all Pods it controls, run the following command:

```
$ kubectl delete rs kiada
replicaset.apps "kiada" deleted
```

As expected, this also deletes the Pods:

```
$ kubectl get pods -l app=kiada
```

NAME	READY	STATUS	RESTARTS	AGE
kiada-2dq4f	0/2	Terminating	0	7m29s
kiada-f5nff	0/2	Terminating	0	7m29s
kiada-khmj5	0/2	Terminating	0	7m29s

But in some cases, you don't want that. So how can you prevent the garbage collector from removing the Pods? Before we get to that, recreate the

ReplicaSet by reapplying the `rs.kiada.versionLabel.yaml` file.

13.4.2 Deleting a ReplicaSet while preserving the Pods

At the beginning of this chapter you learned that the label selector in a ReplicaSet is immutable. If you want to change the label selector, you have to delete the ReplicaSet object and create a new one. In doing so, however, you may not want the Pods to be deleted, because that would cause your service to become unavailable. Fortunately, you can tell Kubernetes to orphan the Pods instead of deleting them.

To preserve the Pods when you delete the ReplicaSet object, use the following command:

```
$ kubectl delete rs kiada --cascade=orphan      #A
replicaset.apps "kiada" deleted
```

Now, if you list the Pods, you'll find that they've been preserved. If you look at their manifests, you'll notice that the ReplicaSet object has been removed from `ownerReferences`. These Pods are now orphaned, but if you create a new ReplicaSet with the same label selector, it'll take these Pods under its wing. Apply the `rs.kiada.versionLabel.yaml` file again to see this for yourself.

13.5 Summary

In this chapter, you learned that:

- A ReplicaSet represents a group of identical Pods that you manage as a unit. In the ReplicaSet, you specify a Pod template, the desired number of replicas, and a label selector.
- Almost all Kubernetes API object types have an associated controller that processes objects of that type. In each controller, a reconciliation control loop runs that constantly reconciles the actual state with the desired state.
- The ReplicaSet controller ensures that the actual number of Pods always matches the desired number specified in the ReplicaSet. When these two

numbers diverge, the controller immediately reconciles them by creating or deleting Pod objects.

- You can change the number of replicas you want at any time and the controller will take the necessary steps to honor your request. However, when you update the Pod template, the controller won't update the existing Pods.
- Pods created by a ReplicaSet are owned by that ReplicaSet. If you delete the owner, the dependents are deleted by the garbage collector, but you can tell `kubectl` to orphan them instead.

In the next chapter, you'll replace the ReplicaSet with a Deployment object.

14 Managing Pods with Deployments

This chapter covers

- Deploying stateless workloads with the Deployment object
- Horizontally scaling Deployments
- Updating workloads declaratively
- Preventing rollouts of faulty workloads
- Implementing various deployment strategies

In the previous chapter, you learned how to deploy Pods via ReplicaSets. However, workloads are rarely deployed this way because ReplicaSets don't provide the functionality necessary to easily update these Pods. This functionality is provided by the Deployment object type. By the end of this chapter, each of the three services in the Kiada suite will have its own Deployment object.

Before you begin, make sure that the Pods, Services, and other objects of the Kiada suite are present in your cluster. If you followed the exercises in the previous chapter, they should already be there. If not, you can create them by creating the kiada namespace and applying all the manifests in the Chapter14/SETUP/ directory with the following command:

```
$ kubectl apply -f SETUP -R
```

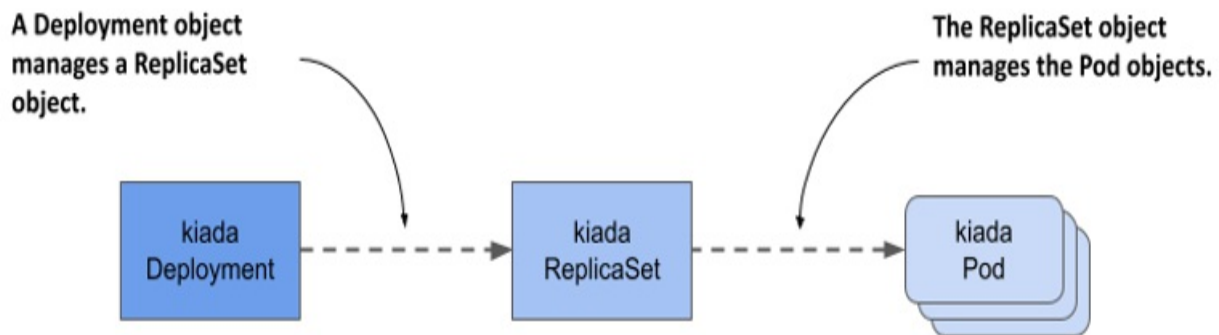
NOTE

You can find the code files for this chapter at <https://github.com/luksa/kubernetes-in-action-2nd-edition/tree/master/Chapter14>.

14.1 Introducing Deployments

When you deploy a workload to Kubernetes, you typically do so by creating a Deployment object. A Deployment object doesn't directly manage the Pod objects, but manages them through a ReplicaSet object that's automatically created when you create the Deployment object. As shown in the next figure, the Deployment controls the ReplicaSet, which in turn controls the individual Pods.

Figure 14.1 The relationship between Deployments, ReplicaSets and Pods.



A Deployment allows you to update the application declaratively. This means that rather than manually performing a series of operations to replace a set of Pods with ones running an updated version of your application, you just update the configuration in the Deployment object and let Kubernetes automate the update.

As with ReplicaSets, you specify a Pod template, the desired number of replicas, and a label selector in a Deployment. The Pods created based on this Deployment are exact replicas of each other and are fungible. For this and other reasons, Deployments are mainly used for stateless workloads, but you can also use them to run a single instance of a stateful workload. However, because there's no built-in way to prevent users from scaling the Deployment to multiple instances, the application itself must ensure that only a single instance is active when multiple replicas are running simultaneously.

Note

To run replicated stateful workloads, a *StatefulSet* is the better option. You'll learn about them in the next chapter.

14.1.1 Creating a Deployment

In this section, you'll replace the kiada ReplicaSet with a Deployment. Delete the ReplicaSet without deleting the Pods as follows:

```
$ kubectl delete rs kiada --cascade=orphan
```

Let's see what you need to specify in the spec section of a Deployment and how it compares to that of the ReplicaSet.

Introducing the Deployment spec

The spec section of a Deployment object isn't much different from a ReplicaSet's. As you can see in the following table, the main fields are the same as the ones in a ReplicaSet, with only one additional field.

Table 14.1 The main fields you specify in a Deployment's spec section

Field name	Description
replicas	The desired number of replicas. When you create the Deployment object, Kubernetes creates this many Pods from the Pod template. It keeps this number of Pods until you delete the Deployment.
selector	The label selector contains either a map of labels in the matchLabels subfield or a list of label selector requirements in the matchExpressions subfield. Pods that match the label selector are considered part of this Deployment.
template	The Pod template for the Deployment's Pods. When a new Pod needs to be created, the object is created using this template.

strategy	The update strategy defines how Pods in this Deployment are replaced when you update the Pod template.
----------	--

The replicas, selector, and template fields serve the same purpose as those in ReplicaSets. In the additional strategy field, you can configure the update strategy to be used when you update this Deployment.

Creating a Deployment manifest from scratch

When we need to create a new Deployment manifest, most of us usually copy an existing manifest file and modify it. However, if you don't have an existing manifest handy, there's a clever way to create the manifest file from scratch.

You may remember that you first created a Deployment in chapter 3 of this book. This is the command you used then:

```
$ kubectl create deployment kiada --image=luksa/kiada:0.1
```

But since this command creates the object directly instead of creating the manifest file, it's not quite what you want. However, you may recall that you learned in chapter 5 that you can pass the `--dry-run=client` and `-o yaml` options to the `kubectl create` command if you want to create an object manifest without posting it to the API. So, to create a rough version of a Deployment manifest file, you can use the following command:

```
$ kubectl create deployment my-app --image=my-image --dry-run=cli
```

You can then edit the manifest file to make your final changes, such as adding additional containers and volumes or changing the existing container definition. However, since you already have a manifest file for the `kiada` ReplicaSet, the fastest option is to turn it into a Deployment manifest.

Creating a Deployment object manifest

Creating a Deployment manifest is trivial if you already have the ReplicaSet

manifest. You just need to copy the `rs.kiada.versionLabel.yaml` file to `deploy.kiada.yaml`, for example, and then edit it to change the `kind` field from `ReplicaSet` to `Deployment`. While you're at it, please also change the number of replicas from two to three. Your Deployment manifest should look like the following listing.

Listing 14.1 The kiada Deployment object manifest

```
apiVersion: apps/v1
kind: Deployment      #A
metadata:
  name: kiada
spec:
  replicas: 3         #B
  selector:           #C
    matchLabels:      #C
      app: kiada      #C
      rel: stable      #C
  template:           #D
    metadata:         #D
      labels:         #D
        app: kiada    #D
        rel: stable   #D
        ver: '0.5'    #D
    spec:             #D
      ...             #D
```

Creating and inspecting the Deployment object

To create the Deployment object from the manifest file, use the `kubectl apply` command. You can use the usual commands like `kubectl get deployment` and `kubectl describe deployment` to get information about the Deployment you created. For example:

```
$ kubectl get deploy kiada
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
kiada     3/3     3            3           25s
```

Note

The shorthand for deployment is `deploy`.

The Pod number information that the `kubectl get` command displays is read from the `readyReplicas`, `replicas`, `updatedReplicas`, and `availableReplicas` fields in the status section of the Deployment object. Use the `-o yaml` option to see the full status.

Note

Use the wide output option (`-o wide`) with `kubectl get deploy` to display the label selector and the container names and images used in the Pod template.

If you just want to know if the Deployment rollout was successful, you can also use the following command:

```
$ kubectl rollout status deployment kiada
Waiting for deployment "kiada" rollout to finish: 0 of 3 updated
Waiting for deployment "kiada" rollout to finish: 1 of 3 updated
Waiting for deployment "kiada" rollout to finish: 2 of 3 updated
deployment "kiada" successfully rolled out
```

If you run this command immediately after creating the Deployment, you can track how the deployment of Pods is progressing. According to the output of the command, the Deployment has successfully rolled out the three Pod replicas.

Now list the Pods that belong to the Deployment. It uses the same selector as the ReplicaSet from the previous chapter, so you should see three Pods, right? To check, list the Pods with the label selector `app=kiada,rel=stable` as follows:

```
$ kubectl get pods -l app=kiada,rel=stable
```

NAME	READY	STATUS	RESTARTS	AGE	
kiada-4t87s	2/2	Running	0	16h	#A
kiada-5lg8b	2/2	Running	0	16h	#A
kiada-7bffb9bf96-4knb6	2/2	Running	0	6m	#B
kiada-7bffb9bf96-7g2md	2/2	Running	0	6m	#B
kiada-7bffb9bf96-qf4t7	2/2	Running	0	6m	#B

Surprisingly, there are five Pods that match the selector. The first two are those created by the ReplicaSet from the previous chapter, while the last three were created by the Deployment. Although the label selector in the

Deployment matches the two existing Pods, they weren't picked up like you would expect. How come?

At the beginning of this chapter, I explained that the Deployment doesn't directly control the Pods but delegates this task to an underlying ReplicaSet. Let's take a quick look at this ReplicaSet:

```
$ kubectl get rs
NAME                                DESIRED    CURRENT    READY    AGE
kiada-7bffb9bf96                   3          3          3        17m
```

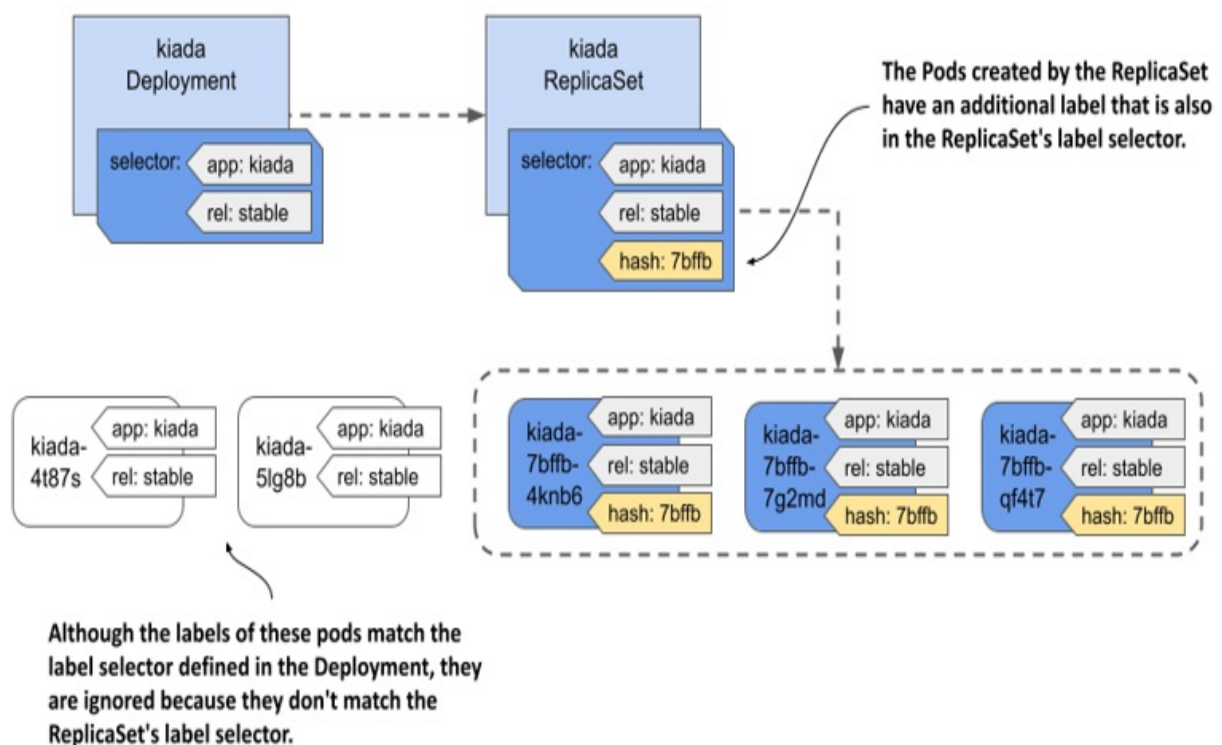
You'll notice that the name of the ReplicaSet isn't simply kiada, but also contains an alphanumeric suffix (-7bffb9bf96) that seems to be randomly generated like the names of the Pods. Let's find out what it is. Take a closer look at the ReplicaSet as follows:

```
$ kubectl describe rs kiada      #A
Name:                            kiada-7bffb9bf96
Namespace:                       kiada
Selector:                        app=kiada,pod-template-hash=7bffb9bf96,rel=stable
Labels:                          app=kiada
                                pod-template-hash=7bffb9bf96      #C
                                rel=stable
                                ver=0.5
Annotations:                     deployment.kubernetes.io/desired-replicas: 3
                                deployment.kubernetes.io/max-replicas: 4
                                deployment.kubernetes.io/revision: 1
Controlled By:                   Deployment/kiada      #D
Replicas:                        3 current / 3 desired
Pods Status:                     3 Running / 0 Waiting / 0 Succeeded / 0 Failed
Pod Template:
  Labels:  app=kiada
          pod-template-hash=7bffb9bf96      #C
          rel=stable
          ver=0.5
  Containers:
  ...
```

The `Controlled By` line indicates that this ReplicaSet has been created and is owned and controlled by the kiada Deployment. You'll notice that the Pod template, selector, and the ReplicaSet itself contain an additional label key `pod-template-hash` that you never defined in the Deployment object. The value of this label matches the last part of the ReplicaSet's name. This

additional label is why the two existing Pods weren't acquired by this ReplicaSet. List the Pods with all their labels to see how they differ:

As you can see in the following figure, when the ReplicaSet was created, the ReplicaSet controller couldn't find any Pods that matched the label selector, so it created three new Pods. If you had added this label to the two existing Pods before creating the Deployment, they'd have been picked up by the ReplicaSet.



the contents of the Pod template. Because the same value is used for the ReplicaSet name, the name depends on the contents of the Pod template. It follows that every time you change the Pod template, a new ReplicaSet is created. You'll learn more about this in section 14.2, which explains Deployment updates.

You can now delete the two kiada Pods that aren't part of the Deployment. To do this, you use the `kubectl delete` command with a label selector that selects only the Pods that have the labels `app=kiada` and `rel=stable` and don't have the label `pod-template-hash`. This is what the full command looks like:

```
$ kubectl delete po -l 'app=kiada,rel=stable,!pod-template-hash'
```

Troubleshooting Deployments that fail to produce any Pods

Under certain circumstances, when creating a Deployment, Pods may not appear. Troubleshooting in this case is easy if you know where to look. To try this out for yourself, apply the manifest file `deploy.where-are-the-pods.yaml`. This will create a Deployment object called `where-are-the-pods`. You'll notice that no Pods are created for this Deployment, even though the desired number of replicas is three. To troubleshoot, you can inspect the Deployment object with `kubectl describe`. The Deployment's events don't show anything useful, but its Conditions do:

```
$ kubectl describe deploy where-are-the-pods
...
Conditions:
Type Status Reason
-----
Progressing True NewReplicaSetCreated
Available False MinimumReplicasUnavailable
ReplicaFailure True FailedCreate #A
```

The `ReplicaFailure` condition is `True`, indicating an error. The reason for the error is `FailedCreate`, which doesn't mean much. However, if you look at the conditions in the status section of the Deployment's YAML manifest, you'll notice that the `message` field of the `ReplicaFailure` condition tells you exactly what happened. Alternatively, you can examine the ReplicaSet and its events to see the same message as follows:

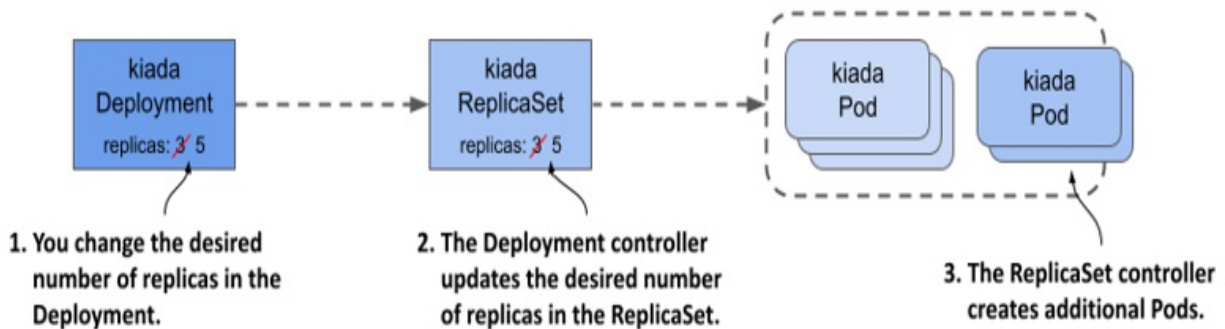
```
$ kubectl describe rs where-are-the-pods-67cbc77f88
...
Events:
Type Reason Age From Message
-----
Warning FailedCreate 61s (x18 over 11m) replicaset-controller Err
```

There are many possible reasons why the ReplicaSet controller can't create a Pod, but they're usually related to user privileges. In this example, the ReplicaSet controller couldn't create the Pod because a service account is missing. You'll learn more about service accounts in chapter 25. The most important conclusion from this exercise is that if Pods don't appear after you create (or update) a Deployment, you should look for the cause in the underlying ReplicaSet.

14.1.2 Scaling a Deployment

Scaling a Deployment is no different from scaling a ReplicaSet. When you scale a Deployment, the Deployment controller does nothing but scale the underlying ReplicaSet, leaving the ReplicaSet controller to do the rest, as shown in the following figure.

Figure 14.3 Scaling a Deployment



Scaling a Deployment

You can scale a Deployment by editing the object with the `kubectl edit` command and changing the value of the `replicas` field, by changing the

value in the manifest file and reapplying it, or by using the `kubectl scale` command. For example, scale the `kiada` Deployment to 5 replicas as follows:

```
$ kubectl scale deploy kiada --replicas 5
deployment.apps/kiada scaled
```

If you list the Pods, you'll see that there are now five `kiada` Pods. If you check the events associated with the Deployment using the `kubectl describe` command, you'll see that the Deployment controller has scaled the `ReplicaSet`.

```
$ kubectl describe deploy kiada
...
Events:
  Type      Reason              Age   From                      Message
  ----      -
  Normal    ScalingReplicaSet   4s    deployment-controller     Scaled
                               7bffb9b
```

If you check the events associated with the `ReplicaSet` using `kubectl describe rs kiada`, you'll see that it was indeed the `ReplicaSet` controller that created the Pods.

Everything you learned about `ReplicaSet` scaling and how the `ReplicaSet` controller ensures that the actual number of Pods always matches the desired number of replicas also applies to Pods deployed via a Deployment.

Scaling a `ReplicaSet` owned by a Deployment

You might wonder what happens when you scale a `ReplicaSet` object owned and controlled by a Deployment. Let's find out. First, start watching `ReplicaSets` by running the following command:

```
$ kubectl get rs -w
```

Now scale the `kiada-7bffb9bf96` `ReplicaSet` by running the following command in another terminal:

```
$ kubectl scale rs kiada-7bffb9bf96 --replicas 7
replicaset.apps/kiada-7bffb9bf96 scaled
```

If you look at the output of the first command, you'll see that the desired number of replicas goes up to seven but is soon reverted to five. This happens because the Deployment controller detects that the desired number of replicas in the ReplicaSet no longer matches the number in the Deployment object and so it changes it back.

Important

If you make changes to an object that is owned by another object, you should expect that your changes will be undone by the controller that manages the object.

Depending on whether the ReplicaSet controller noticed the change before the Deployment controller undid it, it may have created two new Pods. But when the Deployment controller then reset the desired number of replicas back to five, the ReplicaSet controller deleted the Pods.

As you might expect, the Deployment controller will undo any changes you make to the ReplicaSet, not just when you scale it. Even if you delete the ReplicaSet object, the Deployment controller will recreate it. Feel free to try this now.

Inadvertently scaling a Deployment

To conclude this section on Deployment scaling, I should warn you about a scenario in which you might accidentally scale a Deployment without meaning to.

In the Deployment manifest you applied to the cluster, the desired number of replicas was three. Then you changed it to five with the `kubectl scale` command. Imagine doing the same thing in a production cluster. For example, because you need five replicas to handle all the traffic that the application is receiving.

Then you notice that forgot to add the `app` and `re1` labels to the Deployment object. You added them to the Pod template inside the Deployment object, but not to the object itself. This doesn't affect the operation of the

Deployment, but you want all your objects to be nicely labelled, so you decide to add the labels now. You could use the `kubectl label` command, but you'd rather fix the original manifest file and reapply it. This way, when you use the file to create the Deployment in the future, it'll contain the labels you want.

To see what happens in this case, apply the manifest file `deploy.kiada.labelled.yaml`. The only difference between from the original manifest file `deploy.kiada.yaml` are the labels added to the Deployment. If you list the Pods after applying the manifest, you'll see that you no longer have five Pods in your Deployment. Two of the Pods have been deleted:

```
$ kubectl get pods -l app=kiada
```

NAME	READY	STATUS	RESTARTS	AGE
kiada-7bffb9bf96-4knb6	2/2	Running	0	46m
kiada-7bffb9bf96-7g2md	2/2	Running	0	46m
kiada-7bffb9bf96-lkgmx	2/2	Terminating	0	5m
kiada-7bffb9bf96-qf4t7	2/2	Running	0	46m
kiada-7bffb9bf96-z6skm	2/2	Terminating	0	5m

To see why two Pods were removed, check the Deployment object:

```
$ kubectl get deploy
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
kiada	3/3	3	3	46m

The Deployment is now configured to have only three replicas, instead of the five it had before you applied the manifest. However, you never intended to change the number of replicas, only to add labels to the Deployment object. So, what happened?

The reason that applying the manifest changed the desired number of replicas is that the `replicas` field in the manifest file is set to 3. You might think that removing this field from the updated manifest would have prevented the problem, but in fact it would make the problem worse. Try applying the `deploy.kiada.noReplicas.yaml` manifest file that doesn't contain the `replicas` field to see what happens.

If you apply the file, you'll only have one replica left. That's because the

Kubernetes API sets the value to 1 when the `replicas` field is omitted. Even if you explicitly set the value to `null`, the effect is the same.

Imagine this happening in your production cluster when the load on your application is so high that dozens or hundreds of replicas are needed to handle the load. An innocuous update like the one in this example would severely disrupt the service.

You can prevent this by not specifying the `replicas` field in the original manifest when you create the Deployment object. If you forget to do this, you can still repair the existing Deployment object by running the following command:

```
$ kubectl apply edit-last-applied deploy kiada
```

This opens the contents of the `kubectl.kubernetes.io/last-applied-configuration` annotation of the Deployment object in a text editor and allows you to remove the `replicas` field. When you save the file and close the editor, the annotation in the Deployment object is updated. From that point on, updating the Deployment with `kubectl apply` no longer overwrites the desired number of replicas, as long as you don't include the `replicas` field.

Note

When you `kubectl apply`, the value of the `kubectl.kubernetes.io/last-applied-configuration` is used to calculate the changes needed to be made to the API object.

Tip

To avoid accidentally scaling a Deployment each time you reapply its manifest file, omit the `replicas` field in the manifest when you create the object. You initially only get one replica, but you can easily scale the Deployment to suit your needs.

14.1.3 Deleting a Deployment

Before we get to Deployment updates, which are the most important aspect of Deployments, let's take a quick look at what happens when you delete a Deployment. After learning what happens when you delete a ReplicaSet, you probably already know that when you delete a Deployment object, the underlying ReplicaSet and Pods are also deleted.

Preserving the ReplicaSet and Pods when deleting a Deployment

If you want to keep the Pods, you can run the `kubectl delete` command with the `--cascade=orphan` option, as you can with a ReplicaSet. If you use this approach with a Deployment, you'll find that this not only preserves the Pods, but also the ReplicaSets. The Pods still belong to and are controlled by that ReplicaSet.

Adopting an existing ReplicaSet and Pods

If you recreate the Deployment, it picks up the existing ReplicaSet, assuming you haven't changed the Deployment's Pod template in the meantime. This happens because the Deployment controller finds an existing ReplicaSet with a name that matches the ReplicaSet that the controller would otherwise create.

14.2 Updating a Deployment

In the previous section where you learned about the basics of Deployments, you probably didn't see any advantage in using a Deployment instead of a ReplicaSet. The advantage only becomes clear when you update the Pod template in the Deployment. You may recall that this has no immediate effect with a ReplicaSet. The updated template is only used when the ReplicaSet controller creates a new Pod. However, when you update the Pod template in a Deployment, the Pods are replaced immediately.

The kiada Pods are currently running version 0.5 of the application, which you'll now update to version 0.6. You can find the files for this new version in the directory `Chapter14/kiada-0.6`. You can build the container image yourself or use the image `luksa/kiada:0.6` that I created.

Introducing the available update strategies

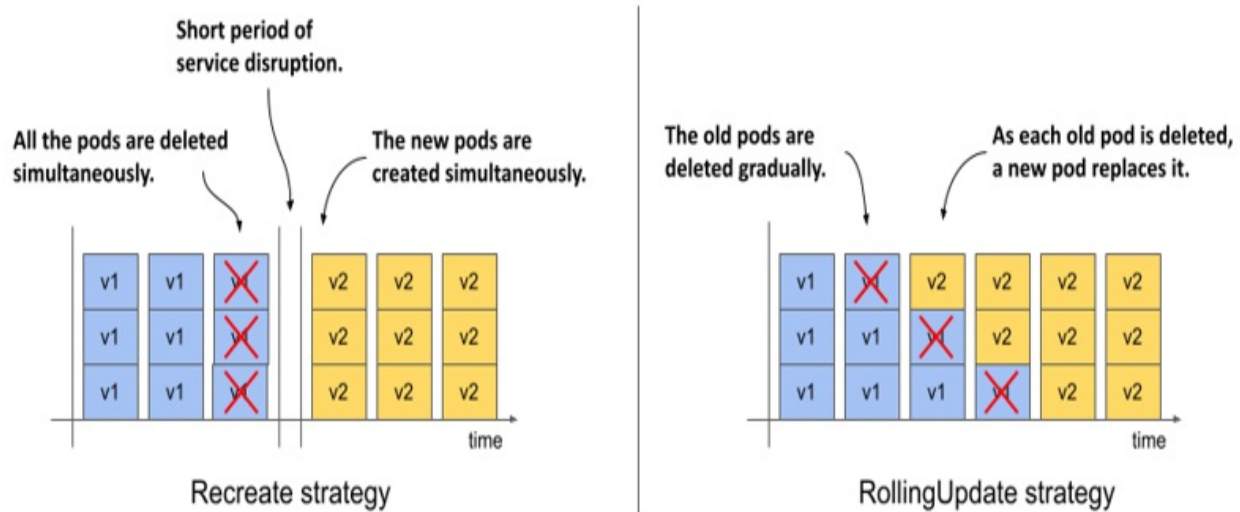
When you update the Pod template to use the new container image, the Deployment controller stops the Pods running with the old image and replaces them with the new Pods. The way the Pods are replaced depends on the update strategy configured in the Deployment. At the time of writing, Kubernetes supports the two strategies described in the following table.

Table 14.2 Update strategies supported by Deployments

Strategy type	Description
Recreate	In the Recreate strategy, all Pods are deleted at the same time, and then, when all their containers are finished, the new Pods are created at the same time. For a short time, while the old Pods are being terminated and before the new Pods are ready, the service is unavailable. Use this strategy if your application doesn't allow you to run the old and new versions at the same time and service downtime isn't an issue.
RollingUpdate	The RollingUpdate strategy causes old Pods to be gradually removed and replaced with new ones. When a Pod is removed, Kubernetes waits until the new Pod is ready before removing the next Pod. This way, the service provided by the Pods remains available throughout the upgrade process. This is the default strategy.

The following figure illustrates the difference between the two strategies. It shows how the Pods are replaced over time for each of the strategies.

Figure 14.4 The difference between the Recreate and the RollingUpdate strategies



The Recreate strategy has no configuration options, while the RollingUpdate strategy lets you configure how many Pods Kubernetes replaces at a time. You’ll learn more about this later.

14.2.1 The Recreate strategy

The Recreate strategy is much simpler than RollingUpdate, so I’ll cover it first. Since you didn’t specify the strategy in the Deployment object, it defaults to RollingUpdate, so you need to change it before triggering the update.

Configuring the Deployment to use the Recreate strategy

To configure a Deployment to use the Recreate update strategy, you must include the lines highlighted in the following listing in your Deployment manifest. You can find this manifest in the `deploy.kiada.recreate.yaml` file.

Listing 14.2 Enabling the Recreate update strategy in a Deployment

```
...
spec:
  strategy:      #A
  type: Recreate  #A
  replicas: 3
```

...

You can add these lines to the Deployment object by editing it with the `kubectl edit` command or by applying the updated manifest file with `kubectl apply`. Since this change doesn't affect the Pod template, it doesn't trigger an update. Changing the Deployment's labels, annotations, or the desired number of replicas also doesn't trigger it.

Updating the container image with `kubectl set image`

To update the Pods to the new version of the Kiada container image, you need to update the `image` field in the `kiada` container definition within the Pod template. You can do this by updating the manifest with `kubectl edit` or `kubectl apply`, but for a simple image change you can also use the `kubectl set image` command. With this command, you can change the image name of any container of any API object that contains containers. This includes Deployments, ReplicaSets, and even Pods. For example, you could use the following command to update the `kiada` container in your `kiada` Deployment to use version `0.6` of the `luksa/kiada` container image like so:

```
$ kubectl set image deployment kiada kiada=luksa/kiada:0.6
```

However, since the Pod template in your Deployment also specifies the application version in the Pod labels, changing the image without also changing the label value would result in an inconsistency.

Updating the container image and labels using `kubectl patch`

To change the image name and label value at the same time, you can use the `kubectl patch` command, which allows you to update multiple manifest fields without having to manually edit the manifest or apply an entire manifest file. To update both the image name and the label value, you could run the following command:

```
$ kubectl patch deploy kiada --patch '{"spec": {"template": {"met
```

This command may be hard for you to parse because the patch is given as a single-line JSON string. In this string, you'll find a partial Deployment

manifest that contains only the fields you want to change. If you specify the patch in a multi-line YAML string, it'll be much clearer. The complete command then looks as follows:

```
$ kubectl patch deploy kiada --patch '      #A
spec:      #B
  template:      #B
    metadata:      #B
    labels:      #B
    ver: "0.6"      #B
  spec:      #B
    containers:      #B
    - name: kiada      #B
      image: luksa/kiada:0.6'      #B
```

Note

You can also write this partial manifest to a file and use `--patch-file` instead of `--patch`.

Now run one of the `kubectl patch` commands to update the Deployment, or apply the manifest file `deploy.kiada.0.6.recreate.yaml` to get the same result.

Observing the Pod state changes during an update

Immediately after you update the Deployment, run the following command repeatedly to observe what happens to the Pods:

```
$ kubectl get po -l app=kiada -L ver
```

This command lists the kiada Pods and displays the value of their version label in the `VER` column. You'll notice that the status of all these Pods changes to `Terminating` at the same time, as shown here:

NAME	READY	STATUS	RESTARTS	AGE
kiada-7bffb9bf96-7w92k	0/2	Terminating	0	3m38s
kiada-7bffb9bf96-h8wnv	0/2	Terminating	0	3m38s
kiada-7bffb9bf96-xgb6d	0/2	Terminating	0	3m38s

The Pods soon disappear, but are immediately replaced by Pods that run the

new version:

NAME	READY	STATUS	RESTARTS	A
kiada-5d5c5f9d76-5pghx	0/2	ContainerCreating	0	1
kiada-5d5c5f9d76-qfkts	0/2	ContainerCreating	0	1
kiada-5d5c5f9d76-vkdr1	0/2	ContainerCreating	0	1

After a few seconds, all new Pods are ready. The whole process is very fast, but you can repeat it as many times as you want. Revert the Deployment by applying the previous version of the manifest in the `deploy.kiada.recreate.yaml` file, wait until the Pods are replaced, and then update to version 0.6 by applying the `deploy.kiada.0.6.recreate.yaml` file again.

Understanding how an update using the Recreate strategy affects service availability

In addition to watching the Pod list, try to access the service via Ingress in your web browser, as described in chapter 12, while the update is in progress.

You'll notice the short time interval when the Ingress proxy returns the status 503 Service Temporarily Unavailable. If you try to access the service directly using the internal cluster IP, you'll find that the connection is rejected during this time.

Understanding the relationship between a Deployment and its ReplicaSets

When you list the Pods, you'll notice that the names of the Pods that ran version 0.5 are different from those that run version 0.6. The names of the old Pods start with `kiada-7bffb9bf96`, while the names of the new Pods start with `kiada-5d5c5f9d76`. You may recall that Pods created by a ReplicaSet get their names from that ReplicaSet. The name change indicates that these new Pods belong to a different ReplicaSet. List the ReplicaSets to confirm this as follows:

```
$ kubectl get rs -L ver
```

NAME	DESIRED	CURRENT	READY	AGE	VER
------	---------	---------	-------	-----	-----

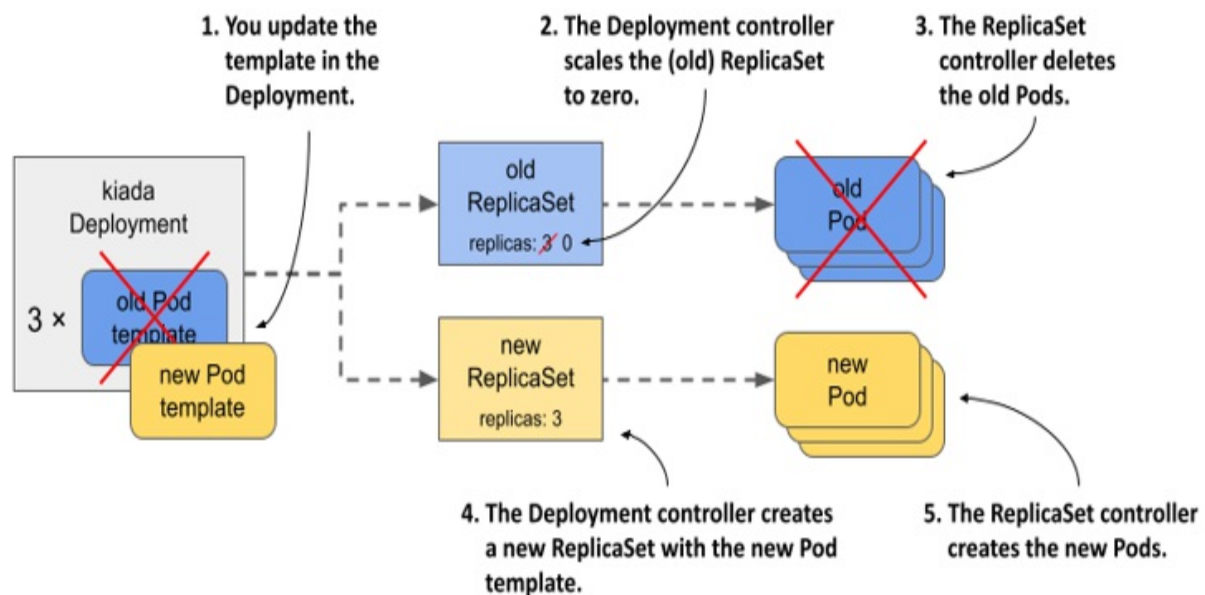
kiada-5d5c5f9d76	3	3	3	13m	0.6	#A
kiada-7bffb9bf96	0	0	0	16m	0.5	#B

Note

The labels you specify in the Pod template in a Deployment are also applied to the ReplicaSet. So if you add a label with the version number of the application, you can see the version when you list the ReplicaSets. This way you can easily distinguish between different ReplicaSets since you can't do that by looking at their names.

When you originally created the Deployment, only one ReplicaSet was created and all Pods belonged to it. When you updated the Deployment, a new ReplicaSet was created. Now all the Pods of this Deployment are controlled by this ReplicaSet, as shown in the following figure.

Figure 14.5 Updating a Deployment



Understanding how the Deployment's Pods transitioned from one ReplicaSet to the other

If you'd been watching the ReplicaSets when you triggered the update, you'd have seen the following progression. At the beginning, only the old

ReplicaSet was present:

NAME	DESIRED	CURRENT	READY	AGE	VER	
kiada-7bffb9bf96	3	3	3	16m	0.5	#A

The Deployment controller then scaled the ReplicaSet to zero replicas, causing the ReplicaSet controller to delete all the Pods:

NAME	DESIRED	CURRENT	READY	AGE	VER	
kiada-7bffb9bf96	0	0	0	16m	0.5	#A

Next, the Deployment controller created the new ReplicaSet and configured it with three replicas.

NAME	DESIRED	CURRENT	READY	AGE	VER	
kiada-5d5c5f9d76	3	0	0	0s	0.6	#A
kiada-7bffb9bf96	0	0	0	16m	0.5	#B

The ReplicaSet controller creates the three new Pods, as indicated by the number in the CURRENT column. When the containers in these Pods start and begin accepting connections, the value in the READY column also changes to three.

NAME	DESIRED	CURRENT	READY	AGE	VER	
kiada-5d5c5f9d76	3	3	0	1s	0.6	#A
kiada-7bffb9bf96	0	0	0	16m	0.5	

Note

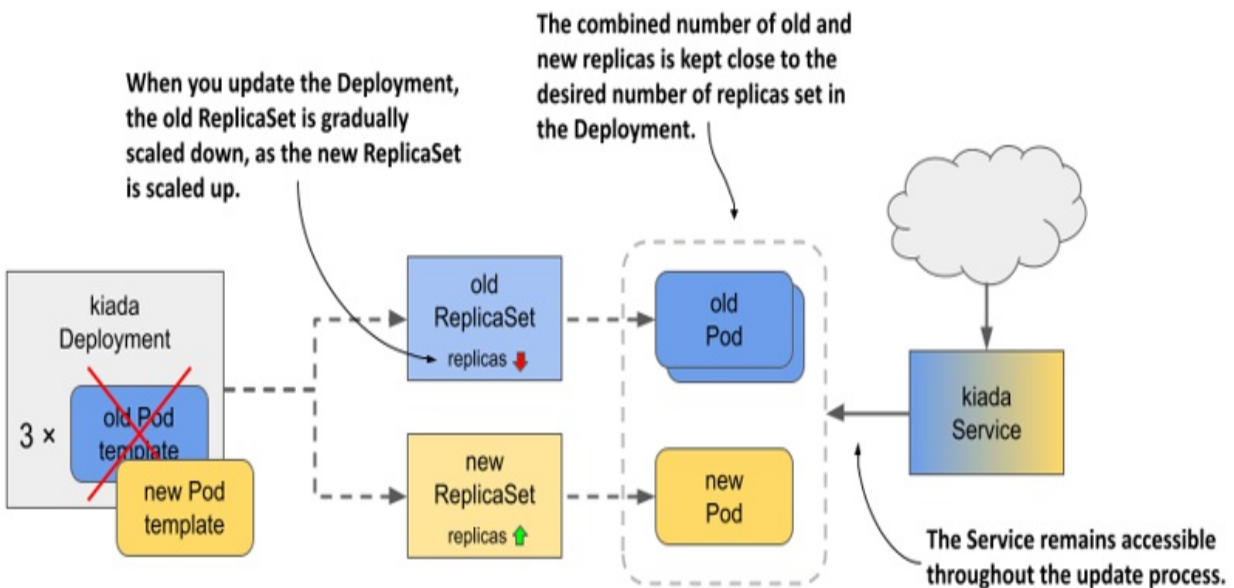
You can see what the Deployment controller and the ReplicaSet controller did by looking at the events associated with the Deployment object and the two ReplicaSets.

The update is now complete. If you open the Kiada service in your web browser, you should see the updated version. In the lower right corner you'll see four boxes indicating the version of the Pod that processed the browser's request for each of the HTML, CSS, JavaScript, and the main image file. These boxes will be useful when you perform a rolling update to version 0.7 in the next section.

14.2.2 The RollingUpdate strategy

The service disruption associated with the Recreate strategy is usually not acceptable. That's why the default strategy in Deployments is RollingUpdate. When you use this strategy, the Pods are replaced gradually, by scaling down the old ReplicaSet and simultaneously scaling up the new ReplicaSet by the same number of replicas. The Service is never left with no Pods to which to forward traffic.

Figure 14.6 What happens with the ReplicaSets, Pods, and the Service during a rolling update.



Configuring the Deployment to use the RollingUpdate strategy

To configure a Deployment to use the RollingUpdate update strategy, you must set its strategy field as shown in the following listing. You can find this manifest in the file `deploy.kiada.0.7.rollingUpdate.yaml`.

Listing 14.3 Enabling the Recreate update strategy in a Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kiada
```

```
spec:
  strategy:
    type: RollingUpdate      #A
    rollingUpdate:          #B
      maxSurge: 0           #B
      maxUnavailable: 1     #B
  minReadySeconds: 10
  replicas: 3
  selector:
    ...
```

In the strategy section, the type field sets the strategy to RollingUpdate, while the maxSurge and maxUnavailable parameters in the rollingUpdate subsection configure how the update should be performed. You could omit this entire subsection and set only the type, but since the default values of the maxSurge and maxUnavailable parameters make it difficult to explain the update process, you set them to the values shown in the listing to make it easier to follow the update process. Don't worry about these two parameters for now, because they'll be explained later.

You may have noticed that the Deployment's spec in the listing also includes the minReadySeconds field. Although this field isn't part of the update strategy, it affects how fast the update progresses. By setting this field to 10, you'll be able to follow the progression of the rolling update even better. You'll learn what this attribute does by the end of this chapter.

Updating the image name in the manifest

In addition to setting the strategy and minReadySeconds in the Deployment manifest, let's also set the image name to luksa/kiada:0.7 and update the version label, so that when you apply this manifest file, you immediately trigger the update. This is to show that you can change the strategy and trigger an update in a single `kubectl apply` operation. You don't have to change the strategy beforehand for it to be used in the update.

Triggering the update and Observing the rollout of the new version

To start the rolling update, apply the manifest file `deploy.kiada.0.7.rollingUpdate.yaml`. You can track the progress of the

rollout with the `kubectl rollout status` command, but it only shows the following:

```
$ kubectl rollout status deploy kiada
Waiting for deploy "kiada" rollout to finish: 1 out of 3 new repl
Waiting for deploy "kiada" rollout to finish: 2 out of 3 new repl
Waiting for deploy "kiada" rollout to finish: 2 of 3 updated repl
deployment "kiada" successfully rolled out
```

To see exactly how the Deployment controller performs the update, it's best to look at how the state of the underlying ReplicaSets changes. First, the ReplicaSet with version 0.6 runs all three Pods. The ReplicaSet for version 0.7 doesn't exist yet. The ReplicaSet for the previous version 0.5 is also there, but let's ignore it, as it's not involved in this update. The initial state of 0.6 ReplicaSet is as follows:

NAME	DESIRED	CURRENT	READY	AGE	VER	
kiada-5d5c5f9d76	3	3	3	53m	0.6	#A

When the update begins, the ReplicaSet running version 0.6 is scaled down by one Pod, while the ReplicaSet for version 0.7 is created and configured to run a single replica:

NAME	DESIRED	CURRENT	READY	AGE	VER	
kiada-58df67c6f6	1	1	0	2s	0.7	#A
kiada-5d5c5f9d76	2	2	2	53m	0.6	#B

Because the old ReplicaSet has been scaled down, the ReplicaSet controller has marked one of the old Pods for deletion. This Pod is now terminating and is no longer considered ready, while the other two old Pods take over all the service traffic. The Pod that's part of the new ReplicaSet is just starting up and therefore isn't ready. The Deployment controller waits until this new Pod is ready before resuming the update process. When this happens, the state of the ReplicaSets is as follows:

NAME	DESIRED	CURRENT	READY	AGE	VER	
kiada-58df67c6f6	1	1	1	6s	0.7	#A
kiada-5d5c5f9d76	2	2	2	53m	0.6	

At this point, traffic is again handled by three Pods. Two are still running version 0.6 and one is running version 0.7. Because you set `minReadySeconds`

to 10, the Deployment controller waits that many seconds before proceeding with the update. It then scales the old ReplicaSet down by one replica, while scaling the new ReplicaSet up by one replica. The ReplicaSets now look as follows:

NAME	DESIRED	CURRENT	READY	AGE	VER	
kiada-58df67c6f6	2	2	1	16s	0.7	#A
kiada-5d5c5f9d76	1	1	1	53m	0.6	#B

The service load is now handled by one old and one new Pod. The second new Pod isn't yet ready, so it's not yet receiving traffic. Ten seconds after the Pod is ready, the Deployment controller makes the final changes to the two ReplicaSets. Again, the old ReplicaSet is scaled down by one, bringing the desired number of replicas to zero. The new ReplicaSet is scaled up so that the desired number of replicas is three, as shown here:

NAME	DESIRED	CURRENT	READY	AGE	VER	
kiada-58df67c6f6	3	3	2	29s	0.7	#A
kiada-5d5c5f9d76	0	0	0	54m	0.6	#B

The last remaining old Pod is terminated and no longer receives traffic. All client traffic is now handled by the new version of the application. When the third new Pod is ready, the rolling update is complete.

At no time during the update was the service unavailable. There were always at least two replicas handling the traffic. You can see for yourself by reverting to the old version and triggering the update again. To do this, reapply the `deploy.kiada.0.6.recreate.yaml` manifest file. Because this manifest uses the Recreate strategy, all the Pods are deleted immediately and then the Pods with the version 0.6 are started simultaneously.

Before you trigger the update to 0.7 again, run the following command to track the update process from the clients' point of view:

```
$ kubectl run -it --rm --restart=Never kiada-client --image curli  
'while true; do curl -s http://kiada | grep "Request processed
```

When you run this command, you create a Pod called `kiada-client` that uses `curl` to continuously send requests to the `kiada` service. Instead of printing the entire response, it prints only the line with the version number and the

Pod and node names.

While the client is sending requests to the service, trigger another update by reapplying the manifest file `deploy.kiada.0.7.rollingUpdate.yaml`. Observe how the output of the `curl` command changes during the rolling update. Here's a short summary:

```
Request processed by Kiada 0.6 running in pod "kiada-5d5c5f9d76-q
Request processed by Kiada 0.6 running in pod "kiada-5d5c5f9d76-2
...
Request processed by Kiada 0.6 running in pod "kiada-5d5c5f9d76-2
Request processed by Kiada 0.7 running in pod "kiada-58df67c6f6-4
Request processed by Kiada 0.6 running in pod "kiada-5d5c5f9d76-6
Request processed by Kiada 0.7 running in pod "kiada-58df67c6f6-4
Request processed by Kiada 0.7 running in pod "kiada-58df67c6f6-4
...
Request processed by Kiada 0.7 running in pod "kiada-58df67c6f6-4
Request processed by Kiada 0.7 running in pod "kiada-58df67c6f6-f
Request processed by Kiada 0.7 running in pod "kiada-58df67c6f6-1
```

During the rolling update, some client requests are handled by the new Pods that run version 0.6, while others are handled by the Pods with version 0.6. Due to the increasing share of the new Pods, more and more responses come from the new version of the application. When the update is complete, the responses come only from the new version.

14.2.3 Configuring how many Pods are replaced at a time

In the rolling update shown in the previous section, the Pods were replaced one by one. You can change this by changing the parameters of the rolling update strategy.

Introducing the `maxSurge` and `maxUnavailable` configuration options

The two parameters that affect how fast Pods are replaced during a rolling update are `maxSurge` and `maxUnavailable`, which I mentioned briefly when I introduced the `RollingUpdate` strategy. You can set these parameters in the `rollingUpdate` subsection of the Deployment's `strategy` field, as shown in the following listing.

Listing 14.4 Specifying parameters for the rollingUpdate strategy

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 0      #A
      maxUnavailable: 1  #A
```

The following table explains the effect of each parameter.

Table 14.3 About the maxSurge and maxUnavailable configuration options

Property	Description
maxSurge	The maximum number of Pods above the desired number of replicas that the Deployment can have during the rolling update. The value can be an absolute number or a percentage of the desired number of replicas.
maxUnavailable	The maximum number of Pods relative to the desired replica count that can be unavailable during the rolling update. The value can be an absolute number or a percentage of the desired number of replicas.

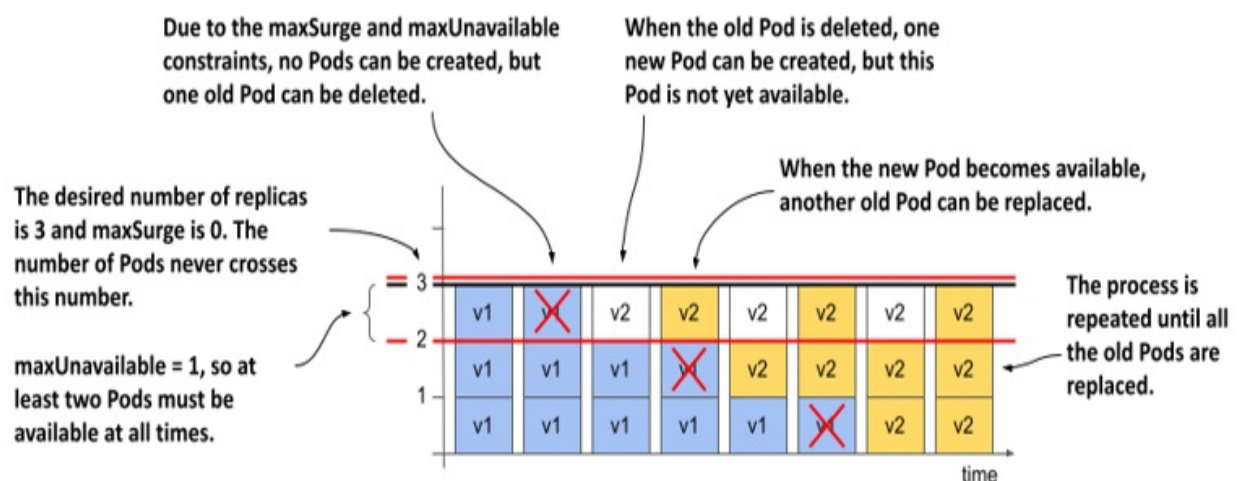
The most important thing about these two parameters is that their values are relative to the desired number of replicas. For example, if the desired number of replicas is three, maxUnavailable is one, and the current number of Pods is five, the number of Pods that must be available is two, not four.

Let's look at how these two parameters affect how the Deployment controller performs the update. This is best explained by going through the possible combinations one by one.

MaxSurge=0, maxUnavailable=1

When you performed the rolling update in the previous section, the desired number of replicas was three, maxSurge was zero and maxUnavailable was one. The following figure shows how the Pods were updated over time.

Figure 14.7 How Pods are replaced when maxSurge is 0 and maxUnavailable is 1

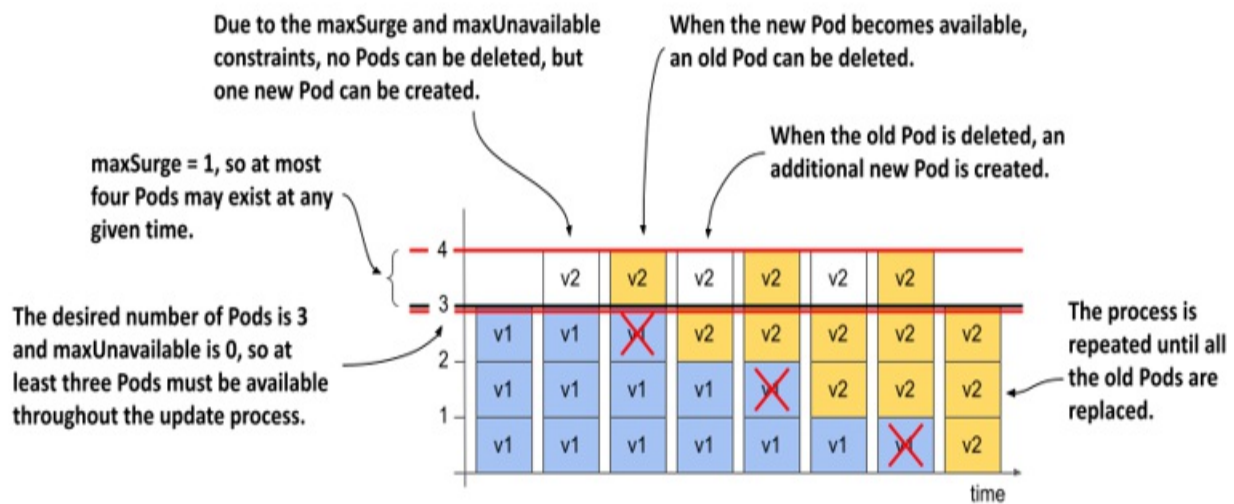


Because maxSurge was set to 0, the Deployment controller wasn't allowed to add Pods beyond the desired number of replicas. Therefore, there were never more than 3 Pods associated with the Deployment. Because maxUnavailable was set to 1, the Deployment controller had to keep the number of available replicas above two and therefore could only delete one old Pod at a time. It couldn't delete the next Pod until the new Pod that replaced the deleted Pod became available.

MaxSurge=1, maxUnavailable=0

What happens if you reverse the two parameters and set maxSurge to 1 and maxUnavailable to 0? If the desired number of replicas is three, there must be at least three replicas available throughout the process. Because the maxSurge parameter is set to 1, there should never be more than four Pods total. The following figure shows how the update unfolds.

Figure 14.8 How Pods are replaced when maxSurge is 1 and maxUnavailable is 0



First, the Deployment controller can't scale the old ReplicaSet down because that would cause the number of available Pods to fall below the desired number of replicas. But the controller can scale the new ReplicaSet up by one Pod, because the maxSurge parameter allows the Deployment to have one Pod above the desired number of replicas.

At this point, the Deployment has three old Pods and one new Pod. When the new Pod is available, the traffic is handled by all four Pods for a moment. The Deployment controller can now scale down the old ReplicaSet by one Pod, since there would still be three Pods available. The controller can then scale up the new ReplicaSet. This process is repeated until the new ReplicaSet has three Pods and the old ReplicaSet has none.

At all times during the update, the desired number of Pods was available and the total number of Pods never exceeded one over the desired replica count.

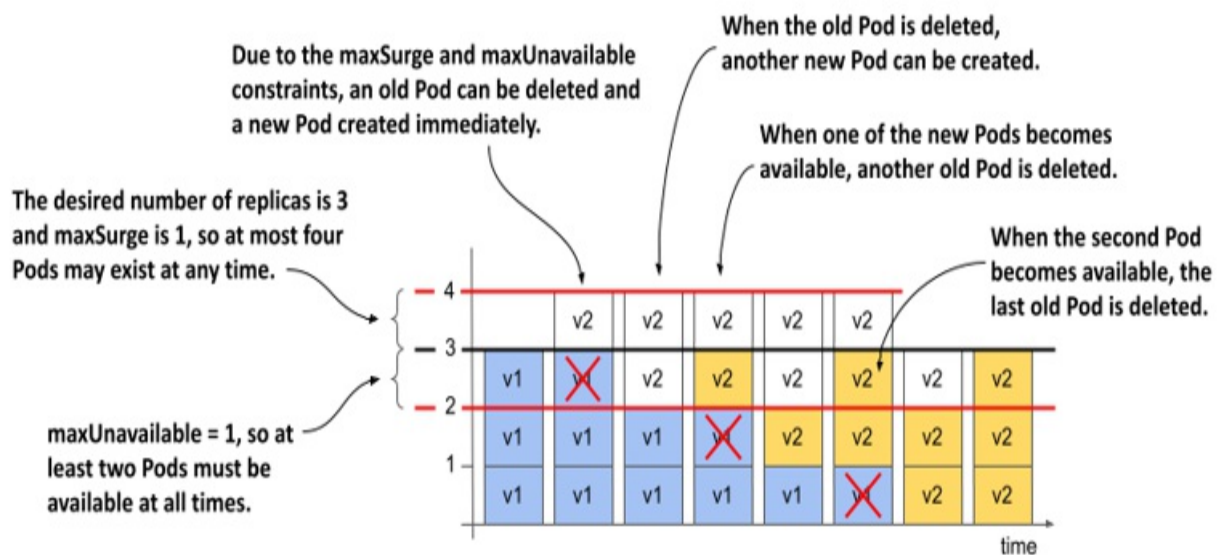
Note

You can't set both maxSurge and maxUnavailable to zero, as this wouldn't allow the Deployment to exceed the desired number of replicas or remove Pods, as one Pod would then be unavailable.

maxSurge=1, maxUnavailable=1

If you set both `maxSurge` and `maxUnavailable` to 1, the total number of replicas in the Deployment can be up to four, and two must always be available. The following figure shows the progression over time.

Figure 14.9 How Pods are replaced when both `maxSurge` and `maxUnavailable` are 1



The Deployment controller immediately scales the new ReplicaSet up by one replica and the old ReplicaSet down the same amount. As soon as the old ReplicaSet reports that it has marked one of the old Pods for deletion, the Deployment controller scales the new ReplicaSet up by another Pod.

Each ReplicaSet is now configured with two replicas. The two Pods in the old ReplicaSet are still running and available, while the two new Pods are starting. When one of the new Pods is available, another old Pod is deleted and another new Pod is created. This continues until all the old Pods are replaced. The total number of Pods never exceeds four, and at least two Pods are available at any given time.

Note

Because the Deployment controller doesn't count the Pods itself, but gets the information about the number of Pods from the status of the underlying ReplicaSets, and because the ReplicaSet never counts the Pods that are being terminated, the total number of Pods may actually exceed 4 if you count the

Pods that are being terminated.

Using higher values of maxSurge and maxUnavailable

If maxSurge is set to a value higher than one, the Deployment controller is allowed to add even more Pods at a time. If maxUnavailable is higher than one, the controller is allowed to remove more Pods.

Using percentages

Instead of setting maxSurge and maxUnavailable to an absolute number, you can set them to a percentage of the desired number of replicas. The controller calculates the absolute maxSurge number by rounding up, and maxUnavailable by rounding down.

Consider a case where replicas is set to 10 and maxSurge and maxUnavailable are set to 25%. If you calculate the absolute values, maxSurge becomes 3, and maxUnavailable becomes 2. So, during the update process, the Deployment may have up to 13 Pods, at least 8 of which are always available and handling the traffic.

Note

The default value for maxSurge and maxUnavailable is 25%.

14.2.4 Pausing the rollout process

The rolling update process is fully automated. Once you update the Pod template in the Deployment object, the rollout process begins and doesn't end until all Pods are replaced with the new version. However, you can pause the rolling update at any time. You may want to do this to check the behavior of the system while both versions of the application are running, or to see if the first new Pod behaves as expected before replacing the other Pods.

Pausing the rollout

To pause an update in the middle of the rolling update process, use the following command:

```
$ kubectl rollout pause deployment kiada  
deployment.apps/kiada paused
```

This command sets the value of the `paused` field in the Deployment's `spec` section to `true`. The Deployment controller checks this field before any change to the underlying ReplicaSets.

Try the update from version 0.6 to version 0.7 again and pause the Deployment when the first Pod is replaced. Open the application in your web browser and observe its behavior. Read the sidebar to learn what to look for.

Be careful when using rolling updates with a web application

If you pause the update while the Deployment is running both the old and new versions of the application and access it through your web browser, you'll notice an issue that can occur when using this strategy with web applications.

Refresh the page in your browser several times and watch the colors and version numbers displayed in the four boxes in the lower right corner. You'll notice that you get version 0.6 for some resources and version 0.7 for others. This is because some requests sent by your browser are routed to Pods running version 0.6 and some are routed to those running version 0.7. For the Kiada application, this doesn't matter, because there aren't any major changes in the CSS, JavaScript, and image files between the two versions. However, if this were the case, the HTML could be rendered incorrectly.

To prevent this, you could use session affinity or update the application in two steps. First, you'd add the new features to the CSS and other resources, but maintain backwards compatibility. After you've fully rolled out this version, you can then roll out the version with the changes to the HTML. Alternatively, you can use the blue-green deployment strategy, explained later in this chapter.

Resuming the rollout

To resume a paused rollout, execute the following command:

```
$ kubectl rollout resume deployment kiada
deployment.apps/kiada resumed
```

Using the pause feature to block rollouts

Pausing a Deployment can also be used to prevent updates to the Deployment from immediately triggering the update process. This allows you to make multiple changes to the Deployment and not start the rollout until you've made all the necessary changes. Once you're ready for the changes to take effect, you resume the Deployment and the rollout process begins.

14.2.5 Updating to a faulty version

When you roll out a new version of an application, you can use the `kubectl rollout pause` command to verify that the Pods running the new version are working as expected before you resume the rollout. You can also let Kubernetes do this for you automatically.

Understanding Pod availability

In chapter 11, you learned what it means for a Pod and its containers to be considered ready. However, when you list Deployments with `kubectl get deployments`, you see both how many Pods are ready and how many are available. For example, during a rolling update, you might see the following output:

```
$ kubectl get deploy kiada
NAME      READY   UP-TO-DATE   AVAILABLE   AGE   #A
kiada     3/3     1            2           50m   #A
```

Although three Pods are ready, not all three are available. For a Pod to be available, it must be ready for a certain amount of time. This time is configurable via the `minReadySeconds` field that I mentioned briefly when I introduced the `RollingUpdate` strategy.

Note

A Pod that's ready but not yet available is included in your services and thus receives client requests.

Delaying Pod availability with `minReadySeconds`

When a new Pod is created in a rolling update, the Deployment controller waits until the Pod is available before continuing the rollout process. By default, the Pod is considered available when it's ready (as indicated by the Pod's readiness probe). If you specify `minReadySeconds`, the Pod isn't considered available until the specified amount of time has elapsed after the Pod is ready. If the Pod's containers crash or fail their readiness probe during this time, the timer is reset.

In one of the previous sections, you set `minReadySeconds` to 10 to slow down the rollout so you could track it more easily. In practice, you can set this property to a much higher value to automatically pause the rollout for a longer period after the new Pods are created. For example, if you set `minReadySeconds` to 3600, you ensure that the update won't continue until the first Pods with the new version prove that they can operate for a full hour without problems.

Although you should obviously test your application in both a test and staging environment before moving it to production, using `minReadySeconds` is like an airbag that helps avoid disaster if a faulty version slips through all the tests. The downside is that it slows down the entire rollout, not just the first stage.

Deploying a broken application version

To see how the combination of a readiness probe and `minReadySeconds` can save you from rolling out a faulty application version, you'll deploy version 0.8 of the Kiada service. This is a special version that returns 500 `Internal Server Error` responses a while after the process starts. This time is configurable via the `FAIL_AFTER_SECONDS` environment variable.

To deploy this version, apply the `deploy.kiada.0.8.minReadySeconds60.yaml` manifest file. The relevant

parts of the manifest are shown in the following listing.

Listing 14.5 Deployment manifest with a readiness probe and minReadySeconds

```
apiVersion: apps/v1
kind: Deployment
...
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 0
      maxUnavailable: 1
  minReadySeconds: 60    #A
  ...
  template:
    ...
    spec:
      containers:
        - name: kiada
          image: luksa/kiada:0.8    #B
          env:
            - name: FAIL_AFTER_SECONDS    #C
              value: "30"    #C
            ...
          readinessProbe:    #D
            initialDelaySeconds: 0    #D
            periodSeconds: 10    #D
            failureThreshold: 1    #D
            httpGet:    #D
              port: 8080    #D
              path: /healthz/ready    #D
              scheme: HTTP    #D
            ...
      ...
```

As you can see in the listing, `minReadySeconds` is set to 60, whereas `FAIL_AFTER_SECONDS` is set to 30. The readiness probe runs every 10 seconds. The first Pod created in the rolling update process runs smoothly for the first thirty seconds. It's marked ready and therefore receives client requests. But after the 30 seconds, those requests and the requests made as part of the readiness probe fail. The Pod is marked as not ready and is never considered available due to the `minReadySeconds` setting. This causes the rolling update to stop.

Initially, some responses that clients receive will be sent by the new version. Then, some requests will fail, but soon afterward, all responses will come from the old version again.

Setting `minReadySeconds` to 60 minimizes the negative impact of the faulty version. Had you not set `minReadySeconds`, the new Pod would have been considered available immediately and the rollout would have replaced all the old Pods with the new version. All these new Pods would soon fail, resulting in a complete service outage. If you'd like to see this yourself, you can try applying the `deploy.kiada.0.8.minReadySeconds0.yaml` manifest file later. But first, let's see what happens when the rollout gets stuck for a long time.

Checking whether the rollout is progressing

The Deployment object indicates whether the rollout process is progressing via the Progressing condition, which you can find in the object's `status.conditions` list. If no progress is made for 10 minutes, the status of this condition changes to `false` and the reason changes to `ProgressDeadlineExceeded`. You can see this by running the `kubectl describe` command as follows:

```
$ kubectl describe deploy kiada
...
Conditions:
  Type           Status  Reason
  ----           -
  Available      True    MinimumReplicasAvailable
  Progressing    False   ProgressDeadlineExceeded   #A
```

Note

You can configure a different progress deadline by setting the `spec.progressDeadlineSeconds` field in the Deployment object. If you increase `minReadySeconds` to more than 600, you must set the `progressDeadlineSeconds` field accordingly.

If you run the `kubectl rollout status` command after you trigger the update, it prints a message that the progress deadline has been exceeded, and terminates.

```
$ kubectl rollout status deploy kiada
Waiting for "kiada" rollout to finish: 1 out of 3 new replicas ha
error: deployment "kiada" exceeded its progress deadline
```

Other than reporting that the rollout has stalled, Kubernetes takes no further action. The rollout process never stops completely. If the Pod becomes ready and remains so for the duration of `minReadySeconds`, the rollout process continues. If the Pod never becomes ready again, the rollout process simply doesn't continue. You can cancel the rollout as explained in the next section.

14.2.6 Rolling back a Deployment

If you update a Deployment and the update fails, you can use the `kubectl apply` command to reapply the previous version of the Deployment manifest or tell Kubernetes to roll back the last update.

Rolling back a Deployment

You can rollback the Deployment to the previous version by running the `kubectl rollout undo` command as follows:

```
$ kubectl rollout undo deployment kiada
deployment.apps/kiada rolled back
```

Running this command has a similar effect to applying the previous version of the object manifest file. The undo process follows the same steps as a normal update. It does so by respecting the update strategy specified in the Deployment object. Thus, if the `RollingUpdate` strategy is used, the Pods are rolled back gradually.

TIP

The `kubectl rollout undo` command can be used while the rollout process is running to cancel the rollout, or after the rollout is complete to undo it.

Note

When a Deployment is paused with the `kubectl pause` command, the `kubectl rollout undo` command does nothing until you resume the Deployment with `kubectl rollout resume`.

Displaying a Deployment's rollout history

Not only can you use the `kubectl rollout undo` command to revert to the previous version, but you can also revert to one of the previous versions. Of course, you may want to see what those versions looked like first. You can do that with the `kubectl rollout history` command. Unfortunately, as I write this, this command is almost useless. You'll understand what I mean when you see its output:

```
$ kubectl rollout history deploy kiada
deployment.apps/kiada
REVISION  CHANGE-CAUSE
1          <none>
2          <none>
11         <none>
```

The only information we can glean from this command is that the Deployment has gone through two revisions. The column `CHANGE-CAUSE` is empty, so we can't see what the reason for each change was.

The values in this column are populated if you use the `--record` option when you run `kubectl` commands that modify the Deployment. However, this option is now deprecated and will be removed. Hopefully, another mechanism will then be introduced that will allow the `rollout history` command to display more information about each change.

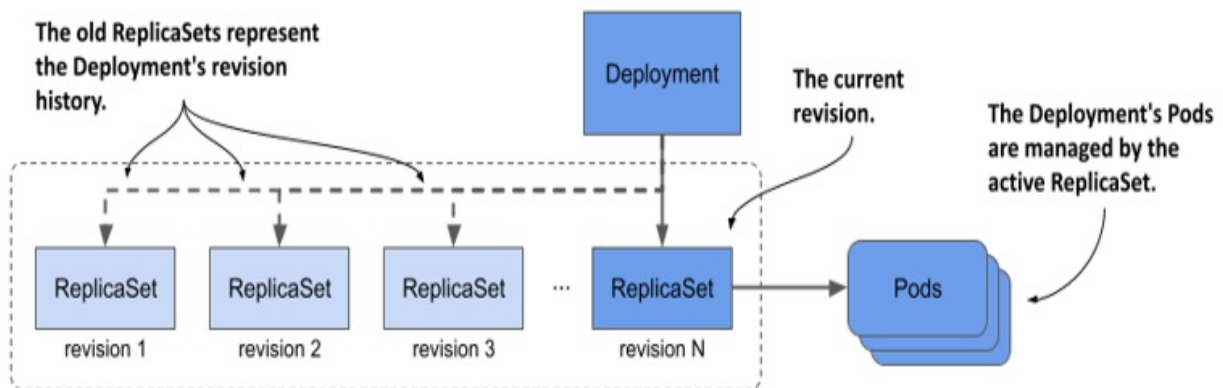
Currently, you can inspect each revision individually by running the `kubectl rollout history` command with the `--revision` option. For example, to inspect the second revision, run the following command:

```
$ kubectl rollout history deploy kiada --revision 2
deployment.apps/kiada with revision #2
Pod Template:
  Labels:      app=kiada
               pod-template-hash=7bffb9bf96
               rel=stable
```

```
Containers:
  kiada:
    Image:      luksa/kiada:0.6
    ...
```

You may wonder where the revision history is stored. You won't find it in the Deployment object. Instead, the history of a Deployment is represented by the ReplicaSets associated with the Deployment, as shown in the following figure. Each ReplicaSet represents one revision. This is the reason why the Deployment controller doesn't delete the old ReplicaSet object after the update process is complete.

Figure 14.10 A Deployment's revision history



Note

The size of the revision history, and thus the number of ReplicaSets that the Deployment controller keeps for a given Deployment, is determined by the `revisionHistoryLimit` field in the Deployment's spec. The default value is 10.

As an exercise, try to find the revision number in which version 0.6 of the Kiada service was deployed. You'll need this revision number in the next section.

Tip

Instead of using `kubectl rollout history` to view the history of a Deployment, listing ReplicaSets with `-o wide` is a better option, because it shows the image tags used in the Pod. To find the revision number for each ReplicaSet, look at the ReplicaSet's annotations.

Rolling back to a specific Deployment revision

You used the `kubectl rollout undo` command to revert from the faulty version 0.8 to version 0.7. But the yellow background for the “Tip of the day” and “Pop quiz” sections of the user interface doesn't look as nice as the white background in version 0.6, so let's roll back to this version.

You can revert to a specific revision by specifying the revision number in the `kubectl rollout undo` command. For example, if you want to revert to the first revision, run the following command:

```
$ kubectl rollout undo deployment kiada --to-revision=1
```

If you found the revision number that contains version 0.6 of the Kiada service, please use the `kubectl rollout undo` command to revert to it.

Understanding the difference between rolling back and applying an older version of the manifest file

You might think that using `kubectl rollout undo` to revert to the previous version of the Deployment manifest is equivalent to applying the previous manifest file, but that's not the case. The `kubectl rollout undo` command reverts only the Pod template and preserves any other changes you made to the Deployment manifest. This includes changes to the update strategy and the desired number of replicas. The `kubectl apply` command, on the other hand, overwrites these changes.

Restarting Pods with `kubectl rollout restart`

In addition to the `kubectl rollout` commands explained in this and previous sections, there's one more command I should mention.

At some point, you may want to restart all the Pods that belong to a Deployment. You can do that with the `kubectl rollout restart` command. This command deletes and replaces the Pods using the same strategy used for updates.

If the Deployment is configured with the `RollingUpdate` strategy, the Pods are recreated gradually so that service availability is maintained throughout the process. If the `Recreate` strategy is used, all Pods are deleted and recreated simultaneously.

14.3 Implementing other deployment strategies

In the previous sections, you learned how the `Recreate` and `RollingUpdate` strategies work. Although these are the only strategies supported by the Deployment controller, you can also implement other well-known strategies, but with a little more effort. You can do this manually or have a higher-level controller automate the process. At the time of writing, Kubernetes doesn't provide such controllers, but you can find them in projects like Flagger (github.com/fluxcd/flagger) and Argo Rollouts (argoproj.github.io/argo-rollouts).

In this section, I'll just give you an overview of how the most common deployment strategies are implemented. The following table explains these strategies, while the subsequent sections explain how they're implemented in Kubernetes.

Table 14.4 Common deployment strategies

Strategy	Description
Recreate	Stop all Pods running the previous version, then create all Pods with the new version.
Rolling	Gradually replace the old Pods with the new ones, either one by

update	one or multiple at the same time. This strategy is also known as <i>Ramped</i> or <i>Incremental</i> .
Canary	Create one or a very small number of new Pods, redirect a small amount of traffic to those Pods to make sure they behave as expected. Then replace all the remaining Pods.
A/B testing	Create a small number of new Pods and redirect a subset of users to those Pods based on some condition. A single user is always redirected to the same version of the application. Typically, you use this strategy to collect data on how effective each version is at achieving certain goals.
Blue/Green	Deploy the new version of the Pods in parallel with the old version. Wait until the new Pods are ready, and then switch all traffic to the new Pods. Then delete the old Pods.
Shadowing	Deploy the new version of the Pods alongside the old version. Forward each request to both versions, but return only the old version's response to the user, while discarding the new version's response. This way, you can see how the new version behaves without affecting users. This strategy is also known as <i>Traffic mirroring</i> or <i>Dark launch</i> .

As you know, the Recreate and RollingUpdate strategies are directly supported by Kubernetes, but you could also consider the Canary strategy as partially supported. Let me explain.

14.3.1 The Canary deployment strategy

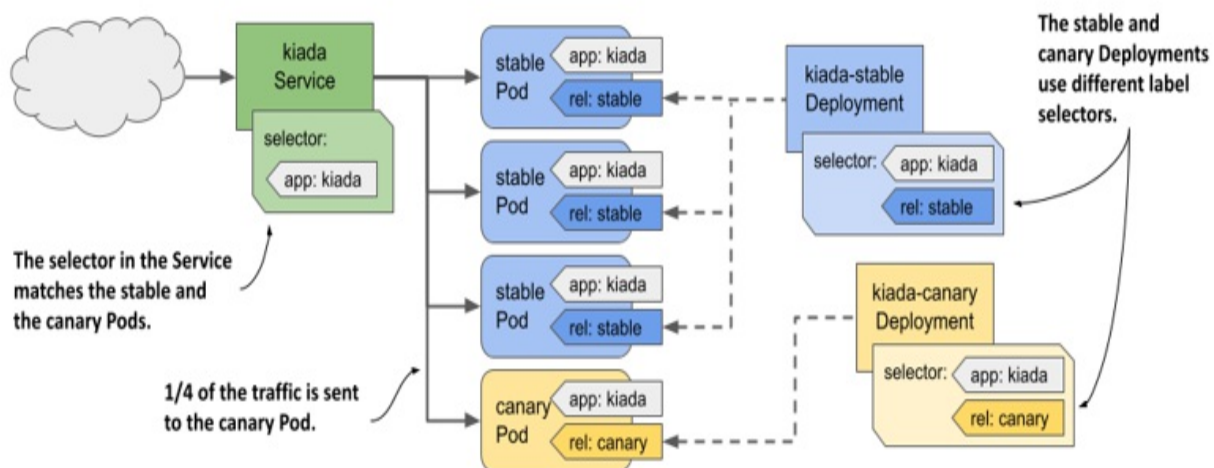
If you set the `minReadySeconds` parameter to a high enough value, the update process resembles a Canary deployment in that the process is paused until the

first new Pods prove their worthiness. The difference with a true Canary deployment is that this pause applies not only to the first Pod(s), but to every step of the update process.

Alternatively, you can use the `kubectl rollout pause` command immediately after creating the first Pod(s) and manually check those canary Pods. When you're sure that the new version is working as expected, you continue the update with the `kubectl rollout resume` command.

Another way to accomplish the same thing is to create a separate Deployment for the canary Pods and set the desired number of replicas to a much lower number than in the Deployment for the stable version. You configure the Service to forward traffic to the Pods in both Deployments. Because the Service spreads the traffic evenly across the Pods and because the canary Deployment has much fewer Pods than the stable Deployment, only a small amount of traffic is sent to the canary Pods, while the majority is sent to the stable Pods. This approach is illustrated in the following figure.

Figure 14.11 Implementing the Canary deployment strategy using two Deployments

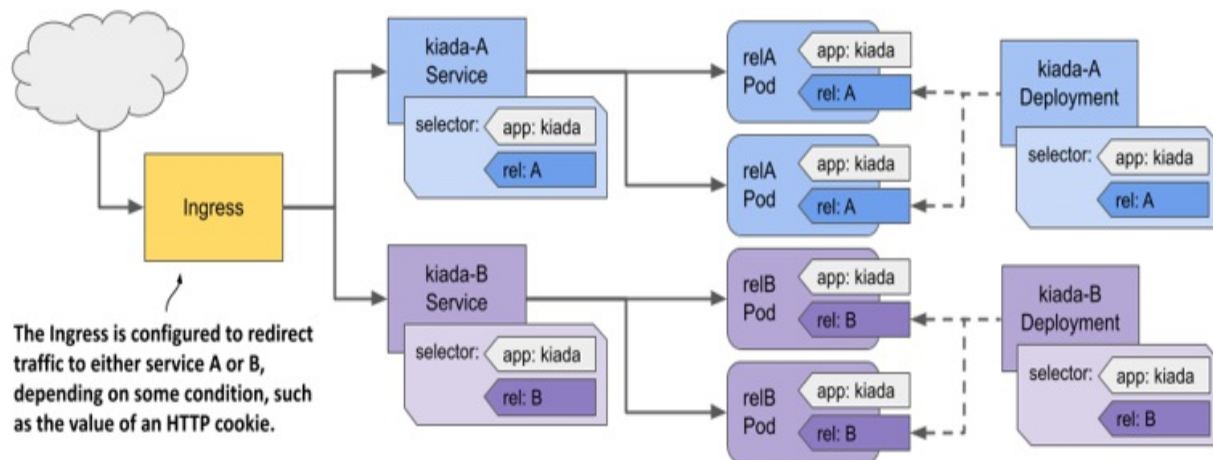


When you're ready to update the other Pods, you can perform a regular rolling update of the old Deployment and delete the canary Deployment.

14.3.2 The A/B strategy

If you want to implement the A/B deployment strategy to roll out a new version only to specific users based on a specific condition such as location, language, user agent, HTTP cookie, or header, you create two Deployments and two Services. You configure the Ingress object to route traffic to one Service or the other based on the selected condition, as shown in the following figure.

Figure 14.12 Implementing the A/B strategy using two Deployments, Services, and an Ingress

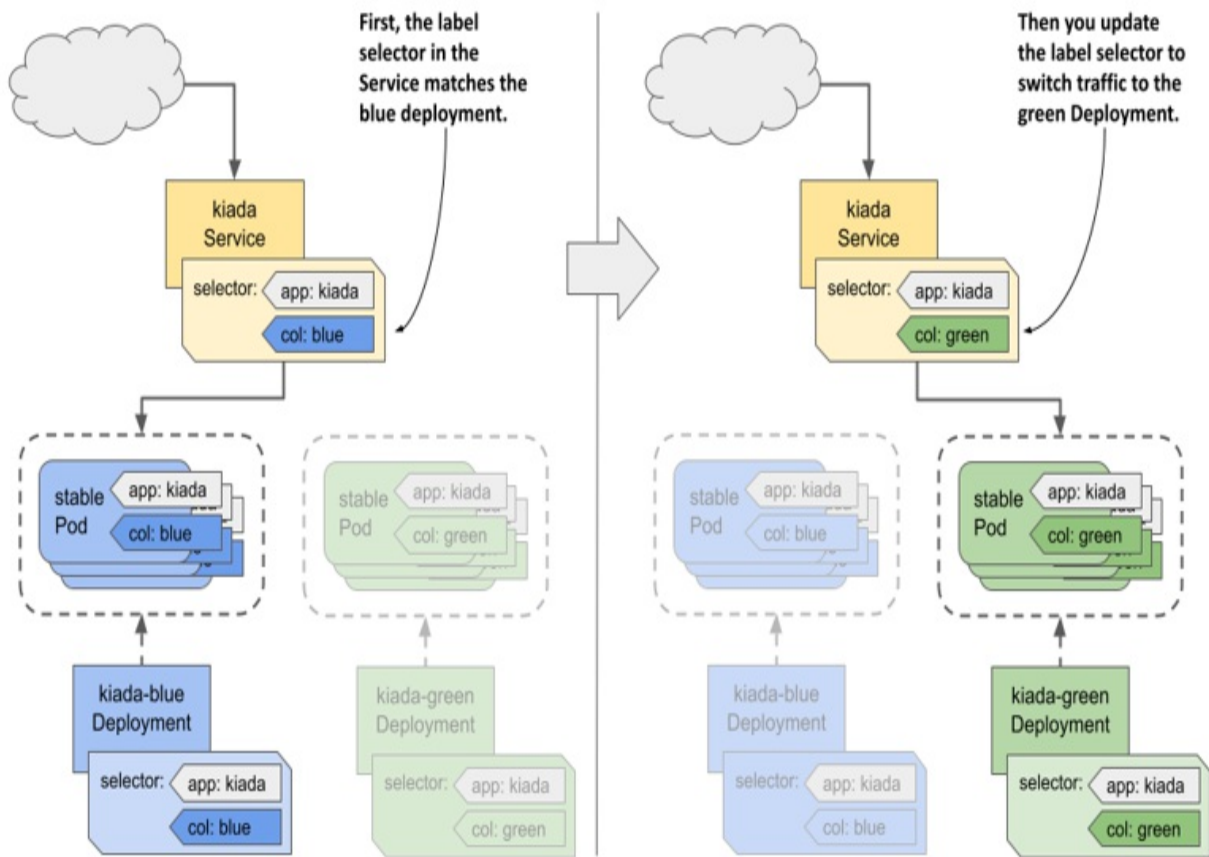


As of this writing, Kubernetes doesn't provide a native way to implement this deployment strategy, but some Ingress implementations do. See the documentation for your chosen Ingress implementation for more information.

14.3.3 The Blue/Green strategy

In the Blue/Green strategy, another Deployment, called the Green Deployment, is created alongside the first Deployment, called the Blue Deployment. The Service is configured to forward traffic only to the Blue Deployment until you decide to switch all traffic to the Green Deployment. The two groups of Pods thus use different labels, and the label selector in the Service matches one group at a time. You switch the traffic from one group to the other by updating the label selector in the Service, as shown in the following figure.

Figure 14.13 Implementing a Blue/Green deployment with labels and selectors



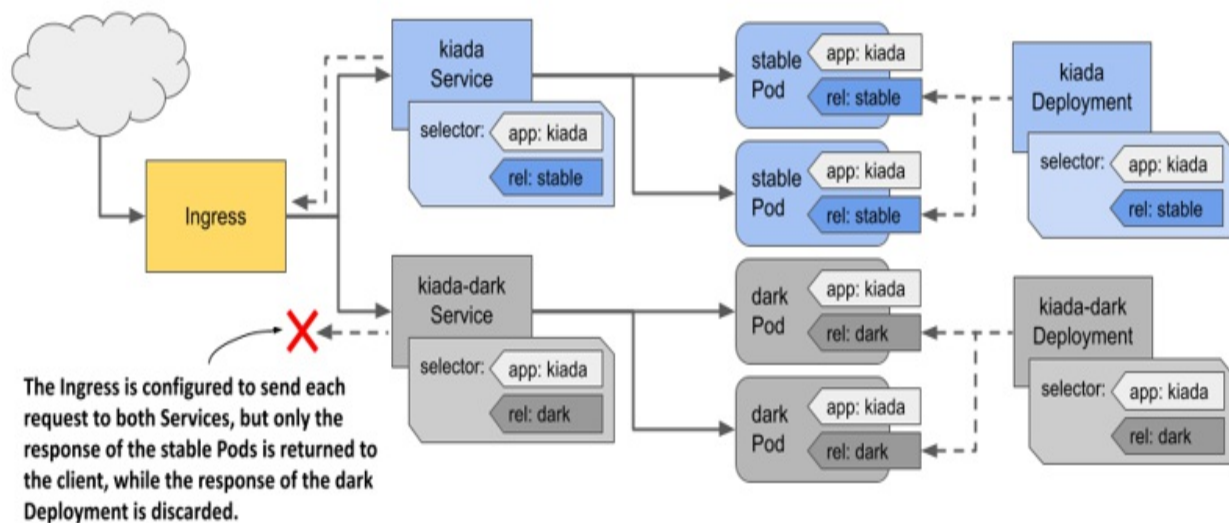
As you know, Kubernetes provides everything you need to implement this strategy. No additional tools are needed.

14.3.4 Traffic shadowing

Sometimes you're not quite sure if the new version of your application will work properly in the actual production environment, or if it can handle the load. In this case, you can deploy the new version alongside the existing version by creating another Deployment object and configuring the Pod labels so that the Pods of this Deployment don't match the label selector in the Service.

You configure the Ingress or proxy that sits in front of the Pods to send traffic to the existing Pods, but also mirror it to the new Pods. The proxy sends the response from the existing Pods to the client and discards the response from the new Pods, as shown in the following figure.

Figure 14.14 Implementing Traffic shadowing



As with A/B testing, Kubernetes doesn't natively provide the necessary functionality to implement traffic shadowing, but some Ingress implementations do.

14.4 Summary

In this chapter, you created a Deployment for the Kiada service, now do the same for the Quote and Quiz services. If you need help, you can find the `deploy.quote.yaml` and `deploy.quiz.yaml` files in the book's code repository.

Here's a summary of what you learned in this chapter:

- A Deployment is an abstraction layer over ReplicaSets. In addition to all the functionality that a ReplicaSet provides, Deployments also allow you to update Pods declaratively. When you update the Pod template, the old Pods are replaced with new Pods created using the updated template.
- During an update, the Deployment controller replaces Pods based on the strategy configured in the Deployment. In the `Recreate` strategy, all Pods are replaced at once, while in the `RollingUpdate` strategy, they're replaced gradually.

- Pods created by a ReplicaSet are owned by that ReplicaSet. The ReplicaSet is usually owned by a Deployment. If you delete the owner, the dependents are deleted by the garbage collector, but you can tell `kubectl` to orphan them instead.
- Other deployment strategies aren't directly supported by Kubernetes, but can be implemented by appropriately configuring Deployments, Services, and the Ingress.

You also learned that Deployments are typically used to run stateless applications. In the next chapter, you'll learn about StatefulSets, which are tailored to run stateful applications.

15 Deploying stateful workloads with StatefulSets

This chapter covers

- Managing stateful workloads via StatefulSet objects
- Exposing individual Pods via headless Services
- Understanding the difference between Deployments and StatefulSets
- Automating stateful workload management with Kubernetes Operators

Each of the three services in your Kiada suite is now deployed via a Deployment object. The Kiada and Quote services each have three replicas, while the Quiz service has only one because its data doesn't allow it to scale easily. In this chapter, you'll learn how to properly deploy and scale stateful workloads like the Quiz service with a *StatefulSet*.

Before you begin, create the kiada Namespace, change to the Chapter15/ directory and apply all manifests in the SETUP/ directory with the following command:

```
$ kubectl apply -n kiada -f SETUP -R
```

IMPORTANT

The examples in this chapter assume that the objects are created in the kiada Namespace. If you create them in a different location, you must update the DNS domain names in several places.

NOTE

You can find the code files for this chapter at <https://github.com/luksa/kubernetes-in-action-2nd-edition/tree/master/Chapter15>.

15.1 Introducing StatefulSets

Before you learn about StatefulSets and how they differ from Deployments, it's good to know how the requirements of stateful workloads differ from those of their stateless counterparts.

15.1.1 Understanding stateful workload requirements

A stateful workload is a piece of software that must store and maintain state in order to function. This state must be maintained when the workload is restarted or relocated. This makes stateful workloads much more difficult to operate.

Stateful workloads are also much harder to scale because you can't simply add and remove replicas without considering their state, as you can with stateless workloads. If the replicas can share state by reading and writing the same files, adding new replicas isn't a problem. However, for this to be possible, the underlying storage technology must support it. On the other hand, if each replica stores its state in its own files, you'll need to allocate a separate volume for each replica. With the Kubernetes resources you've encountered so far, this is easier said than done. Let's look at these two options to understand the issues associated with both.

Sharing state across multiple Pod replicas

In Kubernetes, you can use PersistentVolumes with the `ReadWriteMany` access mode to share data across multiple Pods. However, in most cloud environments, the underlying storage technology typically only supports the `ReadWriteOnce` and `ReadOnlyMany` access modes, not `ReadWriteMany`, meaning you can't mount the volume on multiple nodes in read/write mode. Therefore, Pods on different nodes can't read and write to the same PersistentVolume.

Let's demonstrate this problem using the Quiz service. Can you scale the quiz Deployment to, say, three replicas? Let's see what happens. The `kubectl scale` command is as follows:


```
$ kubectl scale deploy quiz --replicas 3
deployment.apps/quiz scaled
```

Now check the Pods like so:

```
$ kubectl get pods -l app=quiz
```

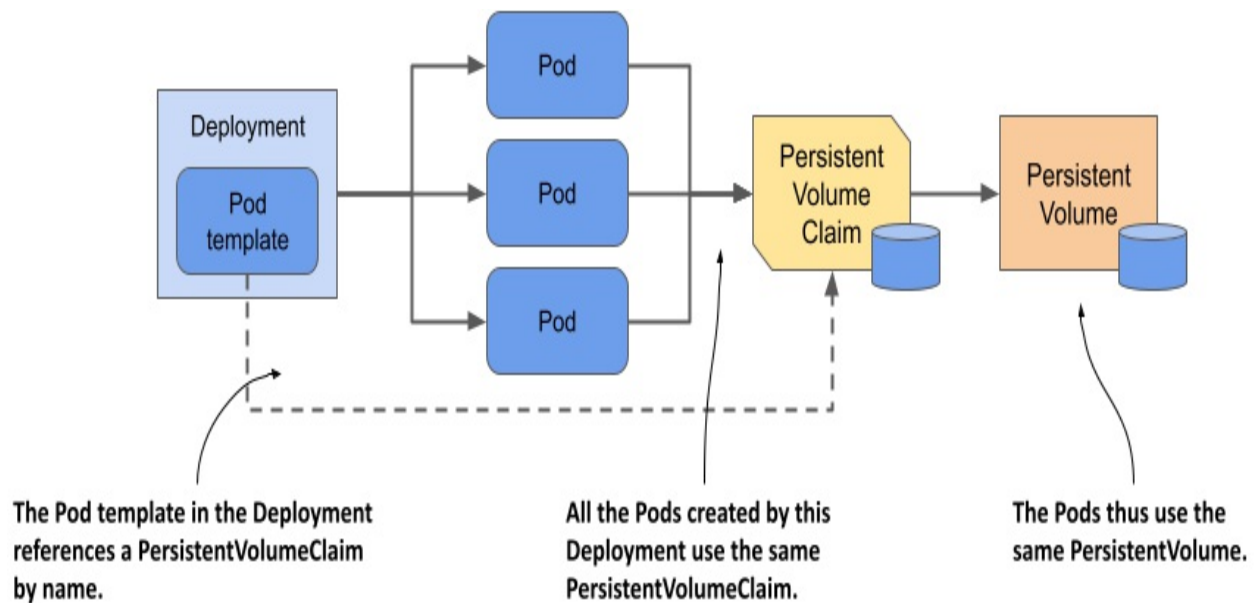
NAME	READY	STATUS	RESTARTS	A
quiz-6f4968457-2c8ws	2/2	Running	0	1
quiz-6f4968457-cdw97	0/2	CrashLoopBackOff	1 (14s ago)	2
quiz-6f4968457-qdn29	0/2	Error	2 (16s ago)	2

As you can see, only the Pod that existed before the scale-up is running, while the two new Pods aren't. Depending on the type of cluster you're using, these two Pods may not start at all, or they may start but immediately terminate with an error message. For example, in a cluster on Google Kubernetes Engine, the containers in the Pods don't start because the PersistentVolume can't be attached to the new Pods because its access mode is ReadWriteOnce and the volume can't be attached to multiple nodes at once. In kind-provisioned clusters, the containers start, but the mongo container fails with an error message, which you can see as follows:

```
$ kubectl logs quiz-6f4968457-cdw97 -c mongo #A
... "msg": "DBException in initAndListen, terminating", "attr": {"err
```

The error message indicates that you can't use the same data directory in multiple instances of MongoDB. The three quiz Pods use the same directory because they all use the same PersistentVolumeClaim and therefore the same PersistentVolume, as illustrated in the next figure.

Figure 15.1 All Pods from a Deployment use the same PersistentVolumeClaim and PersistentVolume.



Since this approach doesn't work, the alternative is to use a separate PersistentVolume for each Pod replica. Let's look at what this means and whether you can do it with a single Deployment object.

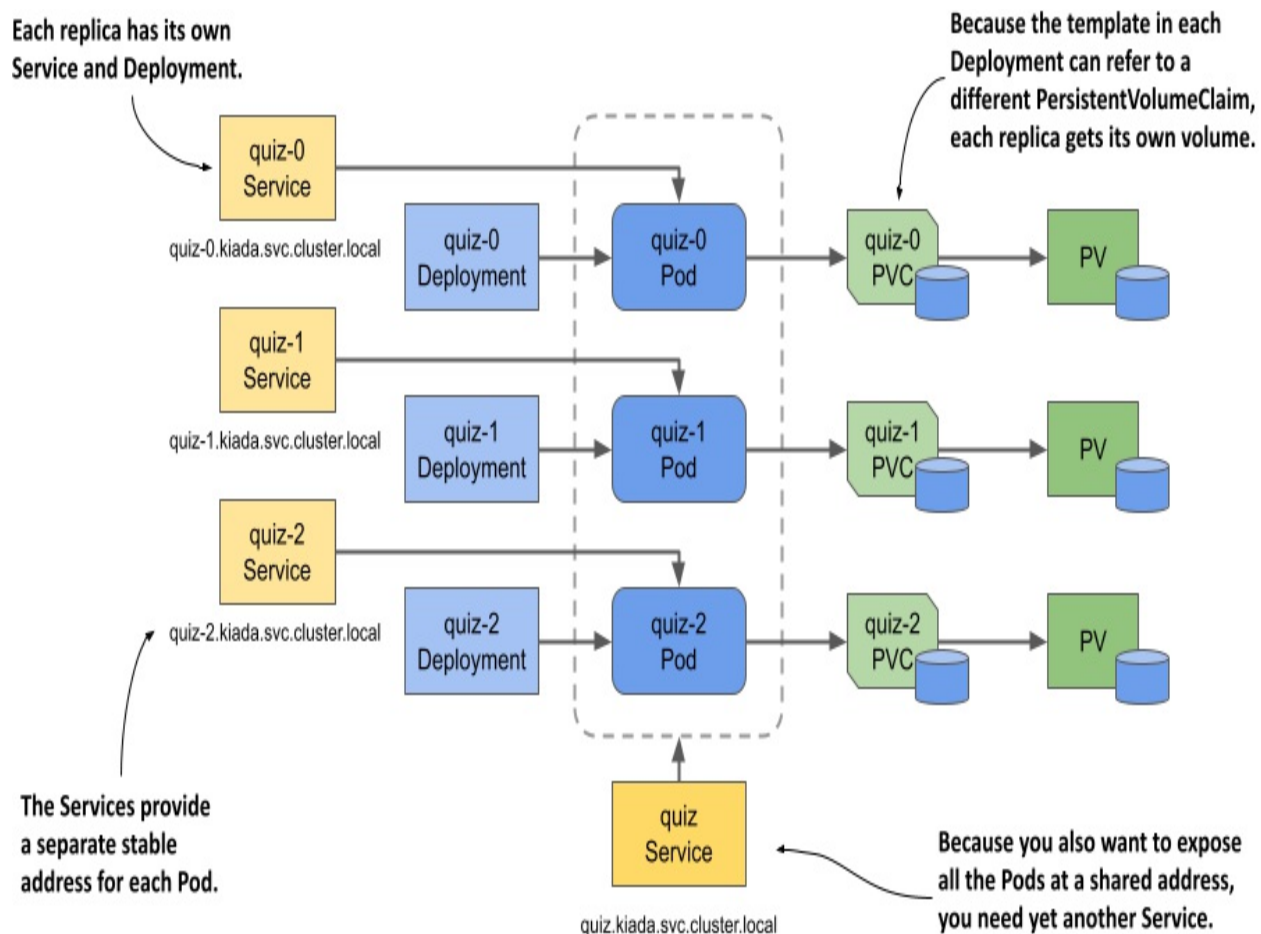
Using a dedicated PersistentVolume for each replica

As you learned in the previous section, MongoDB only supports a single instance by default. If you want to deploy multiple MongoDB instances with the same data, you must create a MongoDB *replica set* that replicates the data across those instances (here the term “replica set” is a MongoDB-specific term and doesn't refer to the Kubernetes ReplicaSet resource). Each instance needs its own storage volume and a stable address that other replicas and clients can use to connect to it. Therefore, to deploy a MongoDB replica set in Kubernetes, you need to ensure that:

- each Pod has its own PersistentVolume,
- each Pod is addressable by its own unique address,
- when a Pod is deleted and replaced, the new Pod is assigned the same address and PersistentVolume.

You can't do this with a single Deployment and Service, but you can do it by creating a separate Deployment, Service, and PersistentVolumeClaim for each replica, as shown in the following figure.

Figure 15.2 Providing each replica with its own volume and address.



Each Pod has its own Deployment, so the Pod can use its own PersistentVolumeClaim and PersistentVolume. The Service associated with each replica gives it a stable address that always resolves to the IP address of the Pod, even if the Pod is deleted and recreated elsewhere. This is necessary because with MongoDB, as with many other distributed systems, you must specify the address of each replica when you initialize the replica set. In addition to these per-replica Services, you may need yet another Service to make all Pods accessible to clients at a single address. So, the whole system looks daunting.

It gets worse from here. If you need to increase the number of replicas, you can't use the `kubectl scale` command; you have to create additional Deployments, Services, and PersistentVolumeClaims, which adds to the complexity.

Even though this approach is feasible, it's complex and it would be difficult to operate this system. Fortunately, Kubernetes provides a better way to do this with a single Service and a single StatefulSet object.

Note

You don't need the quiz Deployment and the quiz-data PersistentVolumeClaim anymore, so please delete them as follows: `kubectl delete deploy/quiz pvc/quiz-data`.

15.1.2 Comparing StatefulSets with Deployments

A StatefulSet is similar to a Deployment, but is specifically tailored to stateful workloads. However, there are significant differences in the behavior of these two objects. This difference is best explained with the *Pets vs. Cattle* analogy that you may have heard of. If not, let me explain.

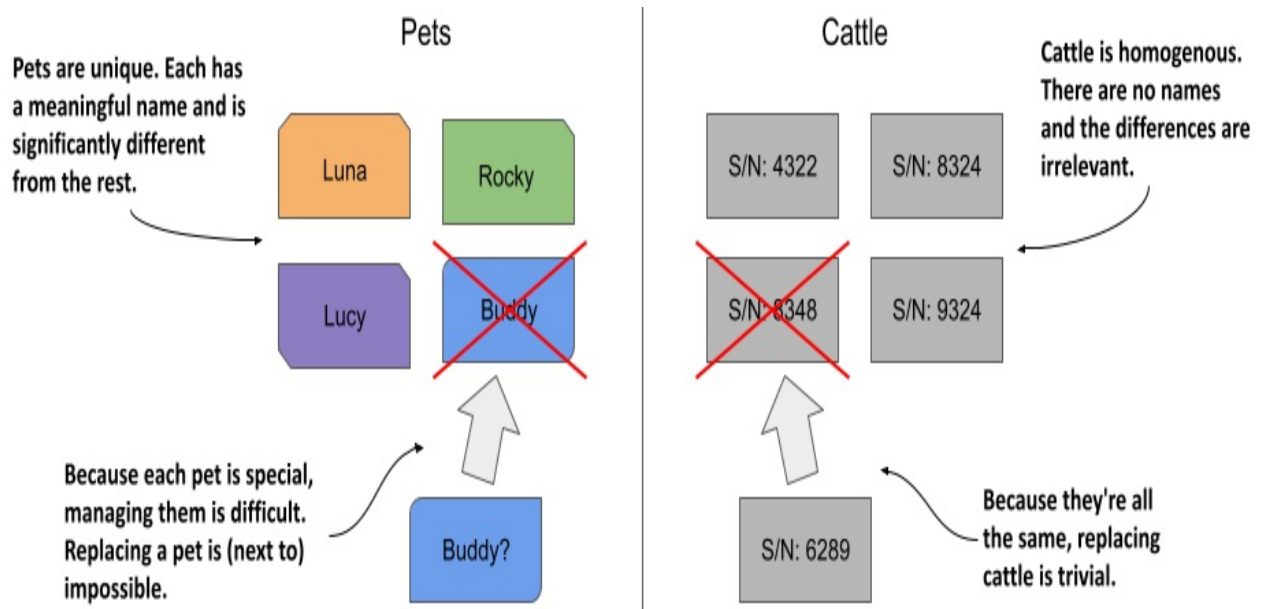
Note

StatefulSets were originally called PetSets. The name comes from this *Pets vs. Cattle* analogy.

The Pets vs. Cattle analogy

We used to treat our hardware infrastructure and workloads like pets. We gave each server a name and took care of each workload instance individually. However, it turns out that it's much easier to manage hardware and software if you treat them like cattle and think of them as indistinguishable entities. That makes it easy to replace each unit without worrying that the replacement isn't exactly the unit that was there before, much like a farmer treats cattle.

Figure 15.3 Treating entities as pets vs. as cattle



Stateless workloads deployed via Deployments are like cattle. If a Pod is replaced, you probably won't even notice. Stateful workloads, on the other hand, are like pets. If a pet gets lost, you can't just replace it with a new one. Even if you give the replacement pet the same name, it won't behave exactly like the original. However, in the hardware/software world, this is possible if you can give the replacement the same network identity and state as the replaced instance. And this is exactly what happens when you deploy an application with a StatefulSet.

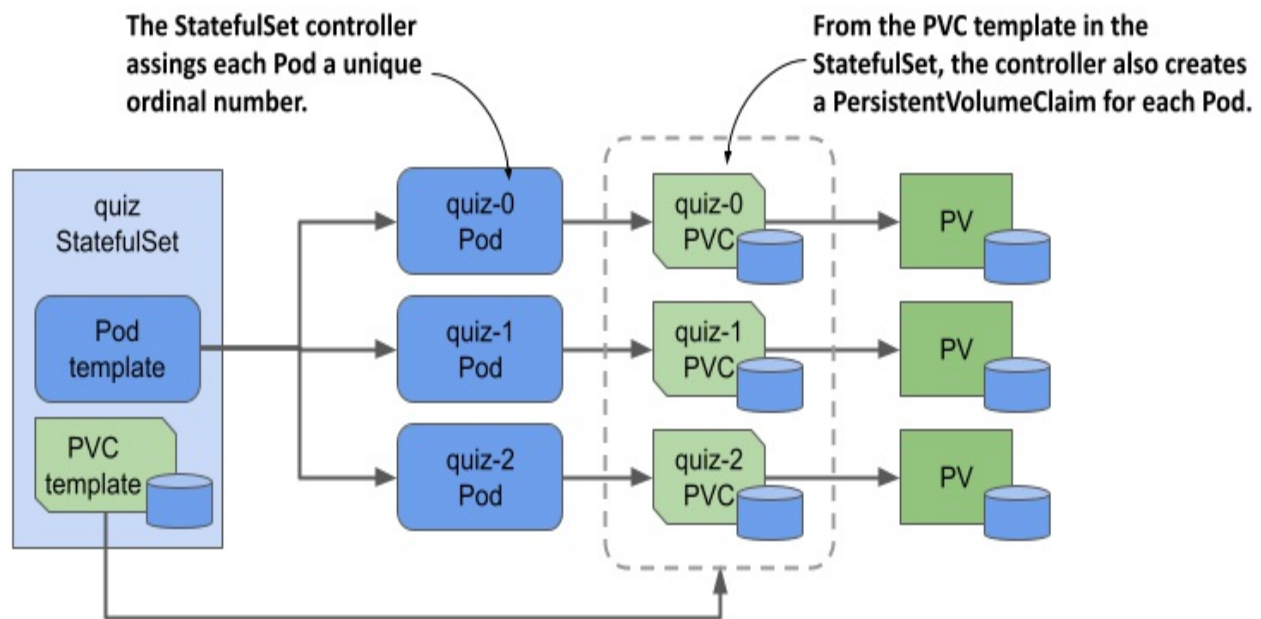
Deploying Pods with a StatefulSet

As with Deployments, in a StatefulSet you specify a Pod template, the desired number of replicas, and a label selector. However, you can also specify a PersistentVolumeClaim template. Each time the StatefulSet controller creates a new replica, it creates not only a new Pod object, but also one or more PersistentVolumeClaim objects.

The Pods created from a StatefulSet aren't exact copies of each other, as is the case with Deployments, because each Pod points to a different set of PersistentVolumeClaims. In addition, the names of the Pods aren't random. Instead, each Pod is given a unique ordinal number, as is each PersistentVolumeClaim. When a StatefulSet Pod is deleted and recreated, it's given the same name as the Pod it replaced. Also, a Pod with a particular

ordinal number is always associated with PersistentVolumeClaims with the same number. This means that the state associated with a particular replica is always the same, no matter how often the Pod is recreated.

Figure 15.4 A StatefulSet, its Pods, and PersistentVolumeClaims



Another notable difference between Deployments and StatefulSets is that, by default, the Pods of a StatefulSet aren't created concurrently. Instead, they're created one at a time, similar to a rolling update of a Deployment. When you create a StatefulSet, only the first Pod is created initially. Then the StatefulSet controller waits until the Pod is ready before creating the next one.

A StatefulSet can be scaled just like a Deployment. When you scale a StatefulSet up, new Pods and PersistentVolumeClaims are created from their respective templates. When you scale down the StatefulSet, the Pods are deleted, but the PersistentVolumeClaims are either retained or deleted, depending on the policy you configure in the StatefulSet.

15.1.3 Creating a StatefulSet

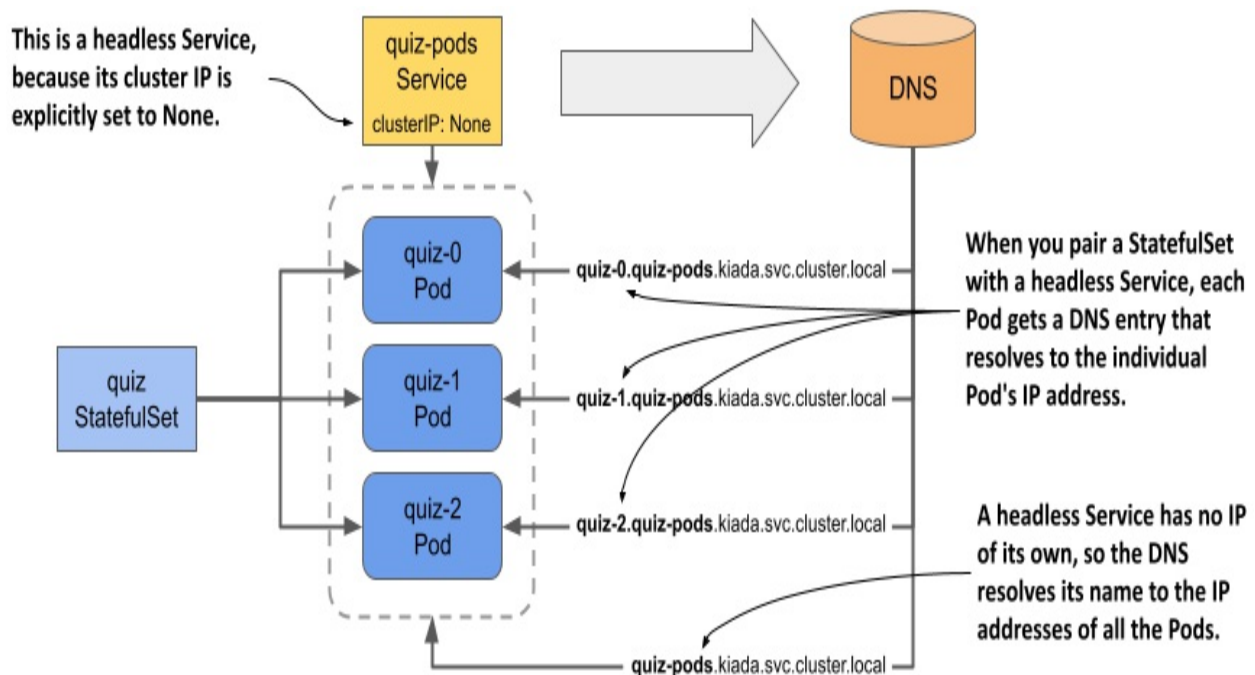
In this section, you'll replace the quiz Deployment with a StatefulSet. Each StatefulSet must have an associated headless Service that exposes the Pods individually, so the first thing you must do is create this Service.

Creating the governing Service

The headless Service associated with a StatefulSet gives the Pods their network identity. You may recall from chapter 11 that a headless Service doesn't have a cluster IP address, but you can still use it to communicate with the Pods that match its label selector. Instead of a single A or AAAA DNS record pointing to the Service's IP, the DNS record for a headless Service points to the IPs of all the Pods that are part of the Service.

As you can see in the following figure, when using a headless Service with a StatefulSet, an additional DNS record is created for each Pod so that the IP address of each Pod can be looked up by its name. This is how stateful Pods maintain their stable network identity. These DNS records don't exist when the headless Service isn't associated with a StatefulSet.

Figure 15.5 A headless Service used in combination with a StatefulSet



You already have a Service called `quiz` that you created in the previous chapters. You could change it into a headless Service, but let's create an additional Service instead, because the new Service will expose all `quiz` Pods, whether they're ready or not.

This headless Service will allow you to resolve individual Pods, so let's call it `quiz-pods`. Create the service with the `kubectl apply` command. You can find the Service manifest in the `svc.quiz-pods.yaml` file, whose contents are shown in the following listing.

Listing 15.1 Headless Service for the quiz StatefulSet

```
apiVersion: v1
kind: Service
metadata:
  name: quiz-pods      #A
spec:
  clusterIP: None      #B
  publishNotReadyAddresses: true    #C
  selector:            #D
    app: quiz          #D
  ports:               #E
    - name: mongodb    #E
      port: 27017      #E
```

In the listing, the `clusterIP` field is set to `None`, which makes this a headless Service. If you set `publishNotReadyAddresses` to `true`, the DNS records for each Pod are created immediately when the Pod is created, rather than only when the Pod is ready. This way, the `quiz-pods` Service will include all `quiz` Pods, regardless of their readiness status.

Creating the StatefulSet

After you create the headless Service, you can create the StatefulSet. You can find the object manifest in the `sts.quiz.yaml` file. The most important parts of the manifest are shown in the following listing.

Listing 15.2 The object manifest for a StatefulSet

```
apiVersion: apps/v1      #A
kind: StatefulSet        #A
metadata:
  name: quiz
spec:
  serviceName: quiz-pods  #B
  podManagementPolicy: Parallel    #C
```



```

replicas: 3      #D
selector:      #E
  matchLabels:  #E
    app: quiz   #E
template:      #F
  metadata:
    labels:     #E
      app: quiz  #E
      ver: "0.1" #E
  spec:
    volumes:    #G
      - name: db-data      #G
        persistentVolumeClaim: #G
          claimName: db-data  #G
    containers:
      - name: quiz-api
        ...
      - name: mongo
        image: mongo:5
        command: #H
          - mongod #H
          - --bind_ip #H
          - 0.0.0.0 #H
          - --replSet #H
          - quiz #H
        volumeMounts: #I
          - name: db-data #I
            mountPath: /data/db #I
volumeClaimTemplates: #J
- metadata: #J
  name: db-data #J
  labels: #J
    app: quiz #J
  spec: #J
    resources: #J
      requests: #J
        storage: 1Gi #J
    accessModes: #J
      - ReadWriteOnce #J

```

The manifest defines an object of kind `StatefulSet` from the `API` group `apps`, version `v1`. The name of the `StatefulSet` is `quiz`. In the `StatefulSet` spec, you'll find some fields you know from `Deployments` and `ReplicaSets`, such as `replicas`, `selector`, and `template`, explained in the previous chapter, but this manifest contains other fields that are specific to `StatefulSets`. In the `serviceName` field, for example, you specify the name of the headless `Service`

that governs this StatefulSet.

By setting `podManagementPolicy` to `Parallel`, the StatefulSet controller creates all Pods simultaneously. Since some distributed applications can't handle multiple instances being launched at the same time, the default behavior of the controller is to create one Pod at a time. However, in this example, the `Parallel` option makes the initial scale-up less involved.

In the `volumeClaimTemplates` field, you specify the templates for the PersistentVolumeClaims that the controller creates for each replica. Unlike the Pod templates, where you omit the name field, you must specify the name in the PersistentVolumeClaim template. This name must match the name in the `volumes` section of the Pod template.

Create the StatefulSet by applying the manifest file as follows:

```
$ kubectl apply -f sts.quiz.yaml
statefulset.apps/quiz created
```

15.1.4 Inspecting the StatefulSet, Pods, and PersistentVolumeClaims

After you create the StatefulSet, you can use the `kubectl rollout status` command to see its status like so:

```
$ kubectl rollout status sts quiz
Waiting for 3 pods to be ready...
```

Note

The shorthand for StatefulSets is `sts`.

After `kubectl` prints this message, it doesn't continue. Interrupt its execution by pressing Control-C and check the StatefulSet status with the `kubectl get` command to investigate why.

```
$ kubectl get sts
NAME      READY   AGE
quiz      0/3     22s
```

Note

As with Deployments and ReplicaSets, you can use the `-o wide` option to display the names of the containers and images used in the StatefulSet.

The value in the `READY` column shows that none of the replicas are ready. List the Pods with `kubectl get pods` as follows:

```
$ kubectl get pods -l app=quiz
NAME      READY   STATUS    RESTARTS   AGE
quiz-0    1/2     Running   0           56s
quiz-1    1/2     Running   0           56s
quiz-2    1/2     Running   0           56s
```

Note

Did you notice the Pod names? They don't contain a template hash or random characters. the name of each Pod is composed of the StatefulSet name and an ordinal number, as explained in the introduction.

You'll notice that only one of the two containers in each Pod is ready. If you examine a Pod with the `kubectl describe` command, you'll see that the `mongo` container is ready, but the `quiz-api` container isn't, because its readiness check fails. This is because the endpoint called by the readiness probe (`/healthz/ready`) checks whether the `quiz-api` process can query the MongoDB server. The failed readiness probe indicates that this isn't possible. If you check the logs of the `quiz-api` container as follows, you'll see why:

```
$ kubectl logs quiz-0 -c quiz-api
... INTERNAL ERROR: connected to mongo, but couldn't execute the
```

As indicated in the error message, the connection to MongoDB has been established, but the server doesn't allow the `ping` command to be executed. The reason is that the server was started with the `--replSet` option configuring it to use replication, but the MongoDB replica set hasn't been initiated yet. To do this, run the following command:

```
$ kubectl exec -it quiz-0 -c mongo -- mongosh --quiet --eval 'rs.
  _id: "quiz",
  members: ['
```

```
{_id: 0, host: "quiz-0.quiz-pods.kiada.svc.cluster.local:2701
{_id: 1, host: "quiz-1.quiz-pods.kiada.svc.cluster.local:2701
{_id: 2, host: "quiz-2.quiz-pods.kiada.svc.cluster.local:2701
```

Note

Instead of typing this long command, you can also run the `initiate-mongo-replicaset.sh` shell script, which you can find in this chapter's code directory.

If the MongoDB shell gives the following error message, you probably forgot to create the `quiz-pods` Service beforehand:

```
MongoServerError: replSetInitiate quorum check failed because not
```

If the initiation of the replica set is successful, the command prints the following message:

```
{ ok: 1 }
```

All three `quiz` Pods should be ready shortly after the replica set is initiated. If you run the `kubectl rollout status sts quiz` command again, you'll see the following output:

```
$ kubectl rollout status sts quiz
partitioned roll out complete: 3 new pods have been updated...
```

Inspecting the StatefulSet with `kubectl describe`

As you know, you can examine an object in detail with the `kubectl describe` command. Here you can see what it displays for the `quiz` StatefulSet:

```
$ kubectl describe sts quiz
Name:          quiz
Namespace:     kiada
CreationTimestamp:  Sat, 12 Mar 2022 18:05:43 +0100
Selector:      app=quiz #A
Labels:        app=quiz
Annotations:    <none>
Replicas:      3 desired | 3 total #B
```

```

Update Strategy:    RollingUpdate
  Partition:        0
Pods Status:        3 Running / 0 Waiting / 0 Succeeded / 0 Failed
Pod Template:       #D
...                #D
Volume Claims:      #E
  Name:             db-data      #E
  StorageClass:     #E
  Labels:           app=quiz      #E
  Annotations:      <none>       #E
  Capacity:         1Gi          #E
  Access Modes:     [ReadWriteOnce] #E
Events:             #F
  Type      Reason          Age      From                      Message
  ----      -
  Normal    SuccessfulCreate 10m      statefulset-controller    create
                                Pod quiz su
                                quiz su
  Normal    SuccessfulCreate 10m      statefulset-controller    create
                                Statefu
...          #F

```

As you can see, the output is very similar to that of a ReplicaSet and Deployment. The most noticeable difference is the presence of the PersistentVolumeClaim template, which you won't find in the other two object types. The events at the bottom of the output show you exactly what the StatefulSet controller did. Whenever it creates a Pod or a PersistentVolumeClaim, it also creates an Event that tells you what it did.

Inspecting the Pods

Let's take a closer look at the manifest of the first Pod to see how it compares to Pods created by a ReplicaSet. Use the `kubectl get` command to print the Pod manifest like so:

```

$ kubectl get pod quiz-0 -o yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    app: quiz      #A
    controller-revision-hash: quiz-7576f64fbc      #A
    statefulset.kubernetes.io/pod-name: quiz-0      #A
    ver: "0.1"     #A

```

```

name: quiz-0
namespace: kiada
ownerReferences:      #B
- apiVersion: apps/v1      #B
  blockOwnerDeletion: true  #B
  controller: true         #B
  kind: StatefulSet        #B
  name: quiz               #B
spec:
  containers:      #C
  ...              #C
  volumes:
  - name: db-data
    persistentVolumeClaim:      #D
      claimName: db-data-quiz-0  #D
status:
  ...

```

The only label you defined in the Pod template in the StatefulSet manifest was `app`, but the StatefulSet controller added two additional labels to the Pod:

- The label `controller-revision-hash` serves the same purpose as the label `pod-template-hash` on the Pods of a ReplicaSet. It allows the controller to determine to which revision of the StatefulSet a particular Pod belongs.
- The label `statefulset.kubernetes.io/pod-name` specifies the Pod name and allows you to create a Service for a specific Pod instance by using this label in the Service's label selector.

Since this Pod object is managed by the StatefulSet, the `ownerReferences` field indicates this fact. Unlike Deployments, where Pods are owned by ReplicaSets, which in turn are owned by the Deployment, StatefulSets own the Pods directly. The StatefulSet takes care of both replication and updating of the Pods.

The Pod's containers match the containers defined in the StatefulSet's Pod template, but that's not the case for the Pod's volumes. In the template you specified the `claimName` as `db-data`, but here in the Pod it's been changed to `db-data-quiz-0`. This is because each Pod instance gets its own `PersistentVolumeClaim`. The name of the claim is made up of the `claimName` and the name of the Pod.

Inspecting the PersistentVolumeClaims

Along with the Pods, the StatefulSet controller creates a PersistentVolumeClaim for each Pod. List them as follows:

```
$ kubectl get pvc -l app=quiz
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODE
db-data-quiz-0	Bound	pvc...1bf8ccaf	1Gi	RWO
db-data-quiz-1	Bound	pvc...c8f860c2	1Gi	RWO
db-data-quiz-2	Bound	pvc...2cc494d6	1Gi	RWO

You can check the manifest of these PersistentVolumeClaims to make sure they match the template specified in the StatefulSet. Each claim is bound to a PersistentVolume that's been dynamically provisioned for it. These volumes don't yet contain any data, so the Quiz service doesn't currently return anything. You'll import the data next.

15.1.5 Understanding the role of the headless Service

An important requirement of distributed applications is peer discovery—the ability for each cluster member to find the other members. If an application deployed via a StatefulSet needs to find all other Pods in the StatefulSet, it could do so by retrieving the list of Pods from the Kubernetes API. However, since we want applications to remain Kubernetes-agnostic, it's better for the application to use DNS and not talk to Kubernetes directly.

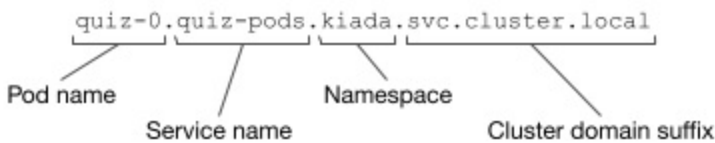
For example, a client connecting to a MongoDB replica set must know the addresses of all the replicas, so it can find the primary replica when it needs to write data. You must specify the addresses in the connection string you pass to the MongoDB client. For your three quiz Pods, the following connection URI can be used:

```
mongodb://quiz-0.quiz-pods.kiada.svc.cluster.local:27017,quiz-1.q  
cluster.local:27017,quiz-2.quiz-pods.kiada.svc.cluster.local:2701
```

If the StatefulSet was configured with additional replicas, you'd need to add their addresses to the connection string, too. Fortunately, there's a better way.

Exposing stateful Pods through DNS individually

In chapter 11 you learned that a Service object not only exposes a set of Pods at a stable IP address but also makes the cluster DNS resolve the Service name to this IP address. With a headless Service, on the other hand, the name resolves to the IPs of the Pods that belong to the Service. However, when a headless Service is associated with a StatefulSet, each Pod also gets its own A or AAAA record that resolves directly to the individual Pod's IP. For example, because you combined the quiz StatefulSet with the quiz-pods headless Service, the IP of the quiz-0 Pod is resolvable at the following address:



All the other replicas created by the StatefulSet are resolvable in the same way.

Exposing stateful Pods via SRV records

In addition to the A and AAAA records, each stateful Pod also gets SRV records. These can be used by the MongoDB client to look up the addresses and port numbers used by each Pod so you don't have to specify them manually. However, you must ensure that the SRV record has the correct name. MongoDB expects the SRV record to start with `_mongodb`. To ensure that's the case, you must set the port name in the Service definition to `mongodb` like you did in listing 15.1. This ensures that the SRV record is as follows:



Using SRV records allows the MongoDB connection string to be much simpler. Regardless of the number of replicas in the set, the connection string is always as follows:

```
mongodb+srv://quiz-pods.kiada.svc.cluster.local
```

Instead of specifying the addresses individually, the `mongodb+srv` scheme

tells the client to find the addresses by performing an SRV lookup for the domain name `_mongodb._tcp.quiz-pods.kiada.svc.cluster.local`. You'll use this connection string to import the quiz data into MongoDB, as explained next.

Importing quiz data into MongoDB

In the previous chapters, an init container was used to import the quiz data into the MongoDB store. The init container approach is no longer valid since the data is now replicated, so if you were to use it, the data would be imported multiple times. Instead, let's move the import to a dedicated Pod.

You can find the Pod manifest in the file `pod.quiz-data-importer.yaml`. The file also contains a ConfigMap that contains the data to be imported. The following listing shows the contents of the manifest file.

Listing 15.3 The manifest of the quiz-data-importer Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: quiz-data-importer
spec:
  restartPolicy: OnFailure      #A
  volumes:
    - name: quiz-questions
      configMap:
        name: quiz-questions
  containers:
    - name: mongoimport
      image: mongo:5
      command:
        - mongoimport
        - mongodb+srv://quiz-pods.kiada.svc.cluster.local/kiada?tls=f
        - --collection
        - questions
        - --file
        - /questions.json
        - --drop
      volumeMounts:
        - name: quiz-questions
          mountPath: /questions.json
```

```

        subPath: questions.json
        readOnly: true
    ---
apiVersion: v1
kind: ConfigMap
metadata:
  name: quiz-questions
  labels:
    app: quiz
data:
  questions.json: ...

```

The quiz-questions ConfigMap is mounted into the quiz-data-importer Pod through a configMap volume. When the Pod's container starts, it runs the mongoimport command, which connects to the primary MongoDB replica and imports the data from the file in the volume. The data is then replicated to the secondary replicas.

Since the mongoimport container only needs to run once, the Pod's restartPolicy is set to OnFailure. If the import fails, the container will be restarted as many times as necessary until the import succeeds. Deploy the Pod using the kubectl apply command and verify that it completed successfully. You can do this by checking the status of the Pod as follows:

```

$ kubectl get pod quiz-data-importer
NAME                READY   STATUS    RESTARTS   AGE
quiz-data-importer  0/1     Completed 0           50s

```

If the STATUS column displays the value Completed, it means that the container exited without errors. The logs of the container will show the number of imported documents. You should now be able to access the Kiada suite via curl or your web browser and see that the Quiz service returns the questions you imported. You can delete the quiz-data-importer Pod and the quiz-questions ConfigMap at will.

Now answer a few quiz questions and use the following command to check if your answers are stored in MongoDB:

```

$ kubectl exec quiz-0 -c mongo -- mongosh kiada --quiet --eval 'd

```

When you run this command, the mongosh shell in pod quiz-0 connects to

the kiada database and displays all the documents stored in the responses collection in JSON form. Each of these documents represents an answer that you submitted.

Note

This command assumes that `quiz-0` is the primary MongoDB replica, which should be the case unless you deviated from the instructions for creating the StatefulSet. If the command fails, try running it in the `quiz-1` and `quiz-2` Pods. You can also find the primary replica by running the MongoDB command `rs.status().primary` in any quiz Pod.

15.2 Understanding StatefulSet behavior

In the previous section, you created the StatefulSet and saw how the controller created the Pods. You used the cluster DNS records that were created for the headless Service to import data into the MongoDB replica set. Now you'll put the StatefulSet to the test and learn about its behavior. First, you'll see how it handles missing Pods and node failures.

15.2.1 Understanding how a StatefulSet replaces missing Pods

Unlike the Pods created by a ReplicaSet, the Pods of a StatefulSet are named differently and each has its own PersistentVolumeClaim (or set of PersistentVolumeClaims if the StatefulSet contains multiple claim templates). As mentioned in the introduction, if a StatefulSet Pod is deleted and replaced by the controller with a new instance, the replica retains the same identity and is associated with the same PersistentVolumeClaim. Try deleting the `quiz-1` Pod as follows:

```
$ kubectl delete po quiz-1
pod "quiz-1" deleted
```

The pod that's created in its place has the same name, as you can see here:

```
$ kubectl get po -l app=quiz
```

NAME	READY	STATUS	RESTARTS	AGE
quiz-0	2/2	Running	0	94m

quiz-1	2/2	Running	0	5s	#A
quiz-2	2/2	Running	0	94m	

The IP address of the new Pod might be different, but that doesn't matter because the DNS records have been updated to point to the new address. Clients using the Pod's hostname to communicate with it won't notice any difference.

In general, this new Pod can be scheduled to any cluster node if the PersistentVolume bound to the PersistentVolumeClaim represents a network-attached volume and not a local volume. If the volume is local to the node, the Pod is always scheduled to this node.

Like the ReplicaSet controller, its StatefulSet counterpart ensures that there are always the desired number of Pods configured in the `replicas` field. However, there's an important difference in the guarantees that a StatefulSet provides compared to a ReplicaSet. This difference is explained next.

15.2.2 Understanding how a StatefulSet handles node failures

StatefulSets provide much stricter concurrent Pod execution guarantees than ReplicaSets. This affects how the StatefulSet controller handles node failures and should therefore be explained first.

Understanding the at-most-one semantics of StatefulSets

A StatefulSet guarantees at-most-one semantics for its Pods. Since two Pods with the same name can't be in the same namespace at the same time, the ordinal-based naming scheme of StatefulSets is sufficient to prevent two Pods with the same identity from running at the same time.

Remember what happens when you run a group of Pods via a ReplicaSet and one of the nodes stops reporting to the Kubernetes control plane? A few minutes later, the ReplicaSet controller determines that the node and the Pods are gone and creates replacement Pods that run on the remaining nodes, even though the Pods on the original node may still be running. If the StatefulSet controller also replaces the Pods in this scenario, you'd have two replicas with the same identity running concurrently. Let's see if that happens.

Disconnecting a node from the network

As in the chapter 13, you'll cause the network interface of one of the nodes to fail. You can try this exercise if your cluster has more than one node. Find the name of the node running the quiz-1 Pod. Suppose it's the node kind-worker2. If you use a kind-provisioned cluster, turn off the node's network interface as follows:

```
$ docker exec kind-worker2 ip link set eth0 down    #A
```

If you're using a GKE cluster, use the following command to connect to the node:

```
$ gcloud compute ssh gke-kiada-default-pool-35644f7e-3001    #A
```

Run the following command on the node to shut down its network interface:

```
$ sudo ifconfig eth0 down
```

Note

Shutting down the network interface will hang the ssh session. You can end the session by pressing Enter followed by “~.” (tilde and dot, without the quotes).

Because the node's network interface is down, the Kubelet running on the node can no longer contact the Kubernetes API server and tell it that the node and all its Pods are still running. The Kubernetes control plane soon marks the node as NotReady, as seen here:

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERS
kind-control-plane	Ready	control-plane,master	10h	v1.2
kind-worker	Ready	<none>	10h	v1.2
kind-worker2	NotReady	<none>	10h	v1.2

After a few minutes, the status of the quiz-1 Pod that was running on this node changes to Terminating, as you can see in the Pod list:

```
$ kubectl get pods -l app=quiz
```

NAME	READY	STATUS	RESTARTS	AGE	
quiz-0	2/2	Running	0	12m	
quiz-1	2/2	Terminating	0	7m39s	#A
quiz-2	2/2	Running	0	12m	

When you inspect the Pod with the `kubectl describe` command, you see a warning event with the message “Node is not ready” as shown here:

```
$ kubectl describe po quiz-1
```

```
...
```

```
Events:
```

Type	Reason	Age	From
----	-----	----	----
Warning	NodeNotReady	11m	node-controller

Understanding why the StatefulSet controller doesn’t replace the Pod

At this point I’d like to point out that the Pod’s containers are still running. The node isn’t down, it only lost network connectivity. The same thing happens if the Kubelet process running on the node fails, but the containers keep running.

This is an important fact because it explains why the StatefulSet controller shouldn’t delete and recreate the Pod. If the StatefulSet controller deletes and recreates the Pod while the Kubelet is down, the new Pod would be scheduled to another node and the Pod’s containers would start. There would then be two instances of the same workload running with the same identity. That’s why the StatefulSet controller doesn’t do that.

Manually deleting the Pod

If you want the Pod to be recreated elsewhere, manual intervention is required. A cluster operator must confirm that the node has indeed failed and manually delete the Pod object. However, the Pod object is already marked for deletion, as indicated by its status, which shows the Pod as Terminating. Deleting the Pod with the usual `kubectl delete pod` command has no effect.

The Kubernetes control plane waits for the Kubelet to report that the Pod’s

containers have terminated. Only then is the deletion of the Pod object complete. However, since the Kubelet responsible for this Pod isn't working, this never happens. To delete the Pod without waiting for confirmation, you must delete it as follows:

```
$ kubectl delete pod quiz-1 --force --grace-period 0
warning: Immediate deletion does not wait for confirmation that t
pod "quiz-0" force deleted
```

Note the warning that the Pod's containers may keep running. That's the reason why you must make sure that the node has really failed before deleting the Pod in this way.

Recreating the Pod

After you delete the Pod, it's replaced by the StatefulSet controller, but the Pod may not start. There are two possible scenarios. Which one occurs depends on whether the replica's PersistentVolume is a local volume, as in *kind*, or a network-attached volume, as in GKE.

If the PersistentVolume is a local volume on the failed node, the Pod can't be scheduled and its STATUS remains Pending, as shown here:

```
$ kubectl get pod quiz-1 -o wide
NAME      READY   STATUS    RESTARTS   AGE     IP          NODE      N
quiz-1    0/2     Pending   0           2m38s   <none>     <none>    <
```

The Pod's events show why the Pod can't be scheduled. Use the `kubectl describe` command to display them as follows.

```
$ kubectl describe pod quiz-1
```

```
...
```

```
Events:
```

Type	Reason	Age	From	Message
----	-----	----	----	-----
Warning	FailedScheduling	21s	default-scheduler	0/3 nodes a
1 node had taint {node-role.kubernetes.io/master: },				that the pod
1 node had taint {node.kubernetes.io/unreachable: },				that the pod
1 node had volume node affinity conflict.			#D	

The event message mentions taints, which you'll learn about in chapter 23.

Without going into detail here, I'll just say that the Pod can't be scheduled to any of the three nodes because one node is a control plane node, another node is unreachable (duh, you just made it so), but the most important part of the warning message is the part about the affinity conflict. The new quiz-1 Pod can only be scheduled to the same node as the previous Pod instance, because that's where its volume is located. And since this node isn't reachable, the Pod can't be scheduled.

If you're running this exercise on GKE or other cluster that uses network-attached volumes, the Pod will be scheduled to another node but may not be able to run if the volume can't be detached from the failed node and attached to that other node. In this case, the STATUS of the Pod is as follows:

```
$ kubectl get pod quiz-1 -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	N
quiz-1	0/2	ContainerCreating	0	38s	1.2.3.4	g

The Pod's events indicate that the PersistentVolume can't be detached. Use `kubectl describe` as follows to display them:

```
$ kubectl describe pod quiz-1
```

```
...
```

Events:

Type	Reason	Age	From	Mes
Warning	FailedAttachVolume	77s	attachdetach-controller	Multi-Atta

Deleting the PersistentVolumeClaim to get the new Pod to run

What do you do if the Pod can't be attached to the same volume? If the workload running in the Pod can rebuild its data from scratch, for example by replicating the data from the other replicas, you can delete the PersistentVolumeClaim so that a new one can be created and bound to a new PersistentVolume. However, since the StatefulSet controller only creates the PersistentVolumeClaims when it creates the Pod, you must also delete the Pod object. You can delete both objects as follows:

```
$ kubectl delete pvc/db-data-quiz-1 pod/quiz-1
persistentvolumeclaim "db-data-quiz-1" deleted
pod "quiz-1" deleted
```


A new PersistentVolumeClaim and a new Pod are created. The PersistentVolume bound to the claim is empty, but MongoDB replicates the data automatically.

Fixing the node

Of course, you can save yourself all that trouble if you can fix the node. If you're running this example on GKE, the system does it automatically by restarting the node a few minutes after it goes offline. To restore the node when using the *kind* tool, run the following commands:

```
$ docker exec kind-worker2 ip link set eth0 up
$ docker exec kind-worker2 ip route add default via 172.18.0.1
```

When the node is back online, the deletion of the Pod is complete, and the new quiz-1 Pod is created. In a *kind* cluster, the Pod is scheduled to the same node because the volume is local.

15.2.3 Scaling a StatefulSet

Just like ReplicaSets and Deployments, you can also scale StatefulSets. When you scale up a StatefulSet, the controller creates both a new Pod and a new PersistentVolumeClaim. But what happens when you scale it down? Are the PersistentVolumeClaims deleted along with the Pods?

Scaling down

To scale a StatefulSet, you can use the `kubectl scale` command or change the value of the `replicas` field in the manifest of the StatefulSet object. Using the first approach, scale the quiz StatefulSet down to a single replica as follows:

```
$ kubectl scale sts quiz --replicas 1
statefulset.apps/quiz scaled
```

As expected, two Pods are now in the process of termination:

```
$ kubectl get pods -l app=quiz
```

NAME	READY	STATUS	RESTARTS	AGE	
quiz-0	2/2	Running	0	1h	
quiz-1	2/2	Terminating	0	14m	#A
quiz-2	2/2	Terminating	0	1h	#A

Unlike ReplicaSets, when you scale down a StatefulSet, the Pod with the highest ordinal number is deleted first. You scaled down the quiz StatefulSet from three replicas to one, so the two Pods with the highest ordinal numbers, quiz-2 and quiz-1, were deleted. This scaling method ensures that the ordinal numbers of the Pods always start at zero and end at a number less than the number of replicas.

But what happens to the PersistentVolumeClaims? List them as follows:

```
$ kubectl get pvc -l app=quiz
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	AGE
db-data-quiz-0	Bound	pvc...1bf8ccaf	1Gi			RWO
db-data-quiz-1	Bound	pvc...c8f860c2	1Gi			RWO
db-data-quiz-2	Bound	pvc...2cc494d6	1Gi			RWO

Unlike Pods, their PersistentVolumeClaims are preserved. This is because deleting a claim would cause the bound PersistentVolume to be recycled or deleted, resulting in data loss. Retaining PersistentVolumeClaims is the default behavior, but you can configure the StatefulSet to delete them via the `persistentVolumeClaimRetentionPolicy` field, as you'll learn later. The other option is to delete the claims manually.

It's worth noting that if you scale the quiz StatefulSet to just one replica, the quiz Service is no longer available, but this has nothing to do with Kubernetes. It's because you configured the MongoDB replica set with three replicas, so at least two replicas are needed to have quorum. A single replica has no quorum and therefore must deny both reads and writes. This causes the readiness probe in the quiz-api container to fail, which in turn causes the Pod to be removed from the Service and the Service to be left with no Endpoints. To confirm, list the Endpoints as follows:

```
$ kubectl get endpoints -l app=quiz
```

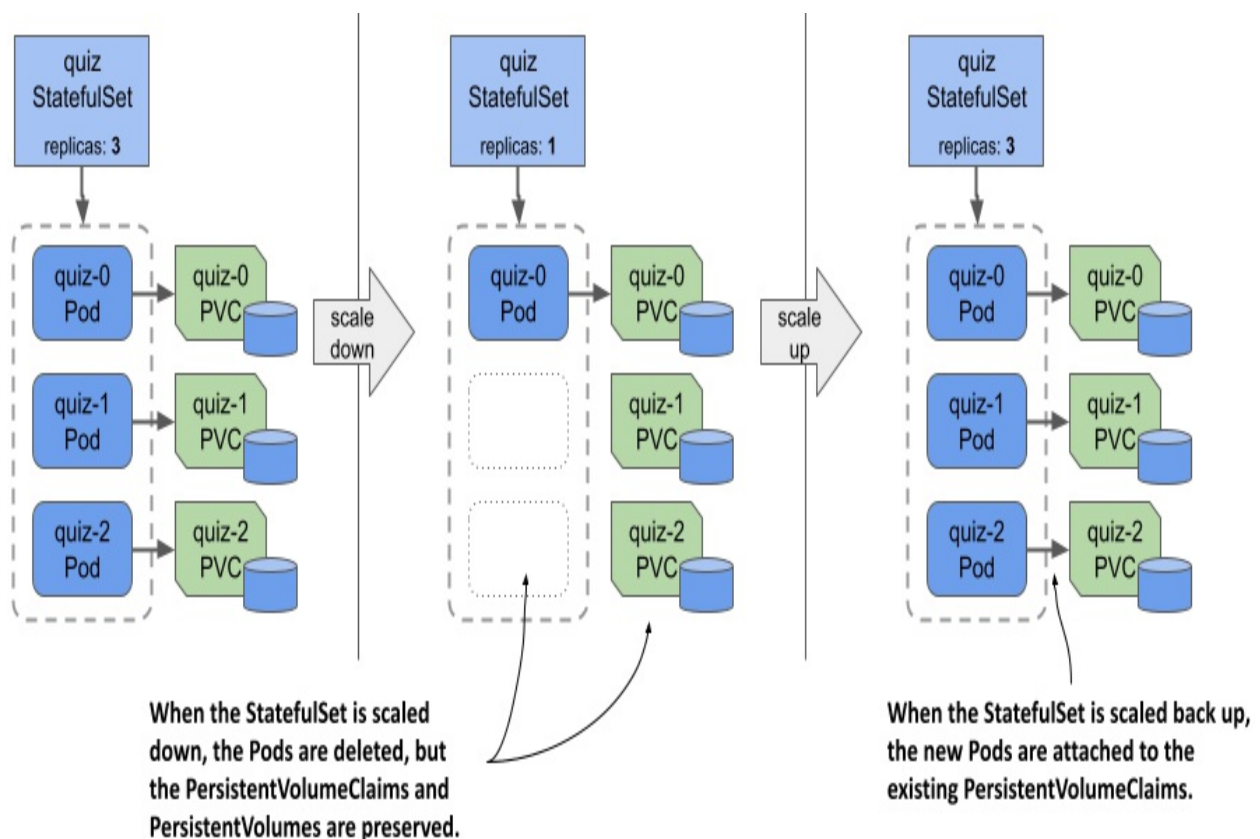
NAME	ENDPOINTS	AGE	
quiz		1h	#A
quiz-pods	10.244.1.9:27017	1h	#B

After you scale down the StatefulSet, you need to reconfigure the MongoDB replica set to work with the new number of replicas, but that's beyond the scope of this book. Instead, let's scale the StatefulSet back up to get the quorum again.

Scaling up

Since PersistentVolumeClaims are preserved when you scale down a StatefulSet, they can be reattached when you scale back up, as shown in the following figure. Each Pod is associated with the same PersistentVolumeClaim as before, based on the Pod's ordinal number.

Figure 15.6 StatefulSets don't delete PersistentVolumeClaims when scaling down; then they reattach them when scaling back up.



Scale the quiz StatefulSet back up to three replicas as follows:

```
$ kubectl scale sts quiz --replicas 3
statefulset.apps/quiz scaled
```

Now check each Pod to see if it's associated with the correct PersistentVolumeClaim. The quorum is restored, all Pods are ready, and the Service is available again. Use your web browser to confirm.

Now scale the StatefulSet to five replicas. The controller creates two additional Pods and PersistentVolumeClaims, but the Pods aren't ready. Confirm this as follows:

```
$ kubectl get pods quiz-3 quiz-4
```

NAME	READY	STATUS	RESTARTS	AGE	
quiz-3	1/2	Running	0	4m55s	#A
quiz-4	1/2	Running	0	4m55s	#A

As you can see, only one of the two containers is ready in each replica. There's nothing wrong with these replicas except that they haven't been added to the MongoDB replica set. You could add them by reconfiguring the replica set, but that's beyond the scope of this book, as mentioned earlier.

You're probably starting to realize that managing stateful applications in Kubernetes involves more than just creating and managing a StatefulSet object. That's why you usually use a Kubernetes Operator for this, as explained in the last part of this chapter.

Before I conclude this section on StatefulSet scaling, I want to point out one more thing. The quiz Pods are exposed by two Services: the regular quiz Service, which addresses only Pods that are ready, and the headless quiz-pods Service, which includes all Pods, regardless of their readiness status. The kiada Pods connect to the quiz Service, and therefore all the requests sent to the Service are successful, as the requests are forwarded only to the three healthy Pods.

Instead of adding the quiz-pods Service, you could've made the quiz Service headless, but then you'd have had to choose whether or not the Service should publish the addresses of unready Pods. From the clients' point of view, Pods that aren't ready shouldn't be part of the Service. From MongoDB's perspective, all Pods must be included because that's how the replicas find each other. Using two Services solves this problem. For this reason, it's common for a StatefulSet to be associated with both a regular Service and a headless Service.

15.2.4 Changing the PersistentVolumeClaim retention policy

In the previous section, you learned that StatefulSets preserve the PersistentVolumeClaims by default when you scale them down. However, if the workload managed by the StatefulSet never requires data to be preserved, you can configure the StatefulSet to automatically delete the PersistentVolumeClaim by setting the `persistentVolumeClaimRetentionPolicy` field. In this field, you specify the retention policy to be used during scaledown and when the StatefulSet is deleted.

For example, to configure the `quiz` StatefulSet to delete the PersistentVolumeClaims when the StatefulSet is scaled but retain them when it's deleted, you must set the policy as shown in the following listing, which shows part of the `sts.quiz.pvcRetentionPolicy.yaml` manifest file.

Listing 15.4 Configuring the PersistentVolumeClaim retention policy in a StatefulSet

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: quiz
spec:
  persistentVolumeClaimRetentionPolicy:
    whenScaled: Delete      #A
    whenDeleted: Retain     #B
  ...
```

The `whenScaled` and `whenDeleted` fields are self-explanatory. Each field can either have the value `Retain`, which is the default, or `Delete`. Apply this manifest file using `kubectl apply` to change the PersistentVolumeClaim retention policy in the `quiz` StatefulSet as follows:

```
$ kubectl apply -f sts.quiz.pvcRetentionPolicy.yaml
```

Note

At the time of writing, this is still an alpha-level feature. For the policy to be honored by the StatefulSet controller, you must enable the feature gate `StatefulSetAutoDeletePVC` when you create the cluster. To do this in the

kind tool, use the `create-kind-cluster.sh` and `kind-multi-node.yaml` files in the `Chapter15/` directory in the book's code archive.

Scaling the StatefulSet

The `whenScaled` policy in the `quiz` StatefulSet is now set to `Delete`. Scale the StatefulSet to three replicas, to remove the two unhealthy Pods and their PersistentVolumeClaims.

```
$ kubectl scale sts quiz --replicas 3
statefulset.apps/quiz scaled
```

List the PersistentVolumeClaims to confirm that there are only three left.

Deleting the StatefulSet

Now let's see if the `whenDeleted` policy is followed. Your aim is to delete the Pods, but not the PersistentVolumeClaims. You've already set the `whenDeleted` policy to `Retain`, so you can delete the StatefulSet as follows:

```
$ kubectl delete sts quiz
statefulset.apps "quiz" deleted
```

List the PersistentVolumeClaims to confirm that all three are present. The MongoDB data files are therefore preserved.

Note

If you want to delete a StatefulSet but keep the Pods and the PersistentVolumeClaims, you can use the `--cascade=orphan` option. In this case, the PersistentVolumeClaims will be preserved even if the retention policy is set to `Delete`.

Ensuring data is never lost

To conclude this section, I want to caution you against setting either retention policy to `Delete`. Consider the example just shown. You set the `whenDeleted` policy to `Retain` so that the data is preserved if the StatefulSet is accidentally

deleted, but since the `whenScaled` policy is set to `Delete`, the data would still be lost if the `StatefulSet` is scaled to zero before it's deleted.

TIP

Set the `persistentVolumeClaimRetentionPolicy` to `Delete` only if the data stored in the `PersistentVolumes` associated with the `StatefulSet` is retained elsewhere or doesn't need to be retained. You can always delete the `PersistentVolumeClaims` manually. Another way to ensure data retention is to set the `reclaimPolicy` in the `StorageClass` referenced in the `PersistentVolumeClaim` template to `Retain`.

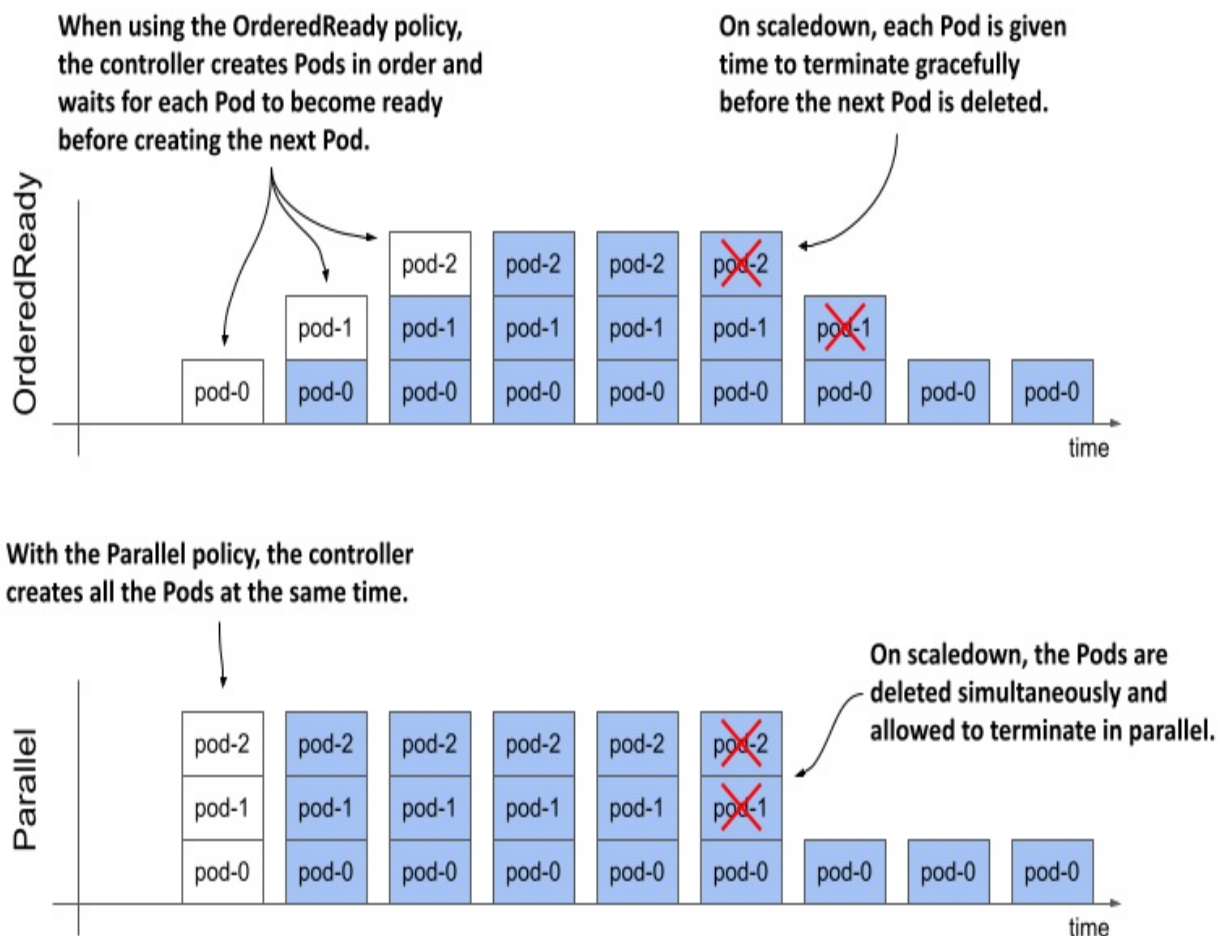
15.2.5 Using the OrderedReady Pod management policy

Working with the quiz `StatefulSet` has been easy. However, you may recall that in the `StatefulSet` manifest, you set the `podManagementPolicy` field to `Parallel`, which instructs the controller to create all Pods at the same time rather than one at a time. While MongoDB has no problem starting all replicas simultaneously, some stateful workloads do.

Introducing the two Pod management policies

When `StatefulSets` were introduced, the Pod management policy wasn't configurable, and the controller always deployed the Pods sequentially. To maintain backward compatibility, this way of working had to be maintained when this field was introduced. Therefore, the default `podManagementPolicy` is `OrderedReady`, but you can relax the `StatefulSet` ordering guarantees by changing the policy to `Parallel`. The following figure shows how Pods are created and deleted over time with each policy.

Figure 15.7 Comparison between the OrderedReady and Parallel Pod management policy



The following table explains the differences between the two policies in more detail.

Table 15.1 The supported podManagementPolicy values

Value	Description
OrderedReady	Pods are created one at a time in ascending order. After creating each Pod, the controller waits until the Pod is ready before creating the next Pod. The same process is used when scaling up and replacing Pods when they're deleted or their nodes fail. When scaling down, the Pods are deleted in reverse order. The controller waits until each deleted Pod is

	finished before deleting the next one.
Parallel	All Pods are created and deleted at the same time. The controller doesn't wait for individual Pods to be ready.

The `OrderedReady` policy is convenient when the workload requires that each replica be fully started before the next one is created and/or fully shut down before the next replica is asked to quit. However, this policy has its drawbacks. Let's look at what happens when we use it in the quiz `StatefulSet`.

Understanding the drawbacks of the `OrderedReady` Pod management policy

Recreate the `StatefulSet` by applying the manifest file `sts.quiz.orderedReady.yaml` with the `podManagementPolicy` set to `OrderedReady`, as shown in the following listing:

Listing 15.5 Specifying the `podManagementPolicy` in the `StatefulSet`

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: quiz
spec:
  podManagementPolicy: OrderedReady      #A
  minReadySeconds: 10                    #B
  serviceName: quiz-pods
  replicas: 3
  ...
```

In addition to setting the `podManagementPolicy`, the `minReadySeconds` field is also set to 10 so you can better see the effects of the `OrderedReady` policy. This field has the same role as in a `Deployment`, but is used not only for `StatefulSet` updates, but also when the `StatefulSet` is scaled.

Note

At the time of writing, the `podManagementPolicy` field is immutable. If you want to change the policy of an existing `StatefulSet`, you must delete and recreate it, like you just did. You can use the `--cascade=orphan` option to prevent Pods from being deleted during this operation.

Observe the `quiz` Pods with the `--watch` option to see how they're created. Run the `kubectl get` command as follows:

```
$ kubectl get pods -l app=quiz --watch
NAME      READY   STATUS    RESTARTS   AGE
quiz-0    1/2     Running   0           22s
```

As you may recall from the previous chapters, the `--watch` option tells `kubectl` to watch for changes to the specified objects. The command first lists the objects and then waits. When the state of an existing object is updated or a new object appears, the command prints the updated information about the object.

Note

When you run `kubectl` with the `--watch` option, it uses the same API mechanism that controllers use to wait for changes to the objects they're observing.

You'll be surprised to see that only a single replica is created when you recreate the `StatefulSet` with the `OrderedReady` policy, even though the `StatefulSet` is configured with three replicas. The next Pod, `quiz-1`, doesn't show up no matter how long you wait. The reason is that the `quiz-api` container in Pod `quiz-0` never becomes ready, as was the case when you scaled the `StatefulSet` to a single replica. Since the first Pod is never ready, the controller never creates the next Pod. It can't do that because of the configured policy.

As before, the `quiz-api` container isn't ready because the MongoDB instance running alongside it doesn't have quorum. Since the readiness probe defined in the `quiz-api` container depends on the availability of MongoDB, which needs at least two Pods for quorum, and since the `StatefulSet` controller doesn't start the next Pod until the first one's ready, the `StatefulSet` is now

stuck in a deadlock.

One could argue that the readiness probe in the `quiz-api` container shouldn't depend on MongoDB. This is debatable, but perhaps the problem lies in the use of the `OrderedReady` policy. Let's stick with this policy anyway, since you've already seen how the `Parallel` policy behaves. Instead, let's reconfigure the readiness probe to call the root URI rather than the `/healthz/ready` endpoint. This way, the probe only checks if the HTTP server is running in the `quiz-api` container, without connecting to MongoDB.

Updating a stuck StatefulSet with the OrderedReady policy

Use the `kubectl edit sts quiz` command to change the path in the readiness probe definition, or use the `kubectl apply` command to apply the updated manifest file `sts.quiz.orderedReady.readinessProbe.yaml`. The following listing shows how the readiness probe should be configured:

Listing 15.6 Setting the readiness probe in the `quiz-api` container

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: quiz
spec:
  ...
  template:
    ...
    spec:
      containers:
        - name: quiz-api
          ...
          readinessProbe:
            httpGet:
              port: 8080
              path: /      #A
              scheme: HTTP
          ...
```

After you update the Pod template in the StatefulSet, you expect the `quiz-0` Pod to be deleted and recreated with the new Pod template, right? List the

Pods as follows to check if this happens.

```
$ kubectl get pods -l app=quiz
NAME      READY   STATUS    RESTARTS   AGE   #A
quiz-0    1/2     Running   0          5m
```

As you can see from the age of the Pod, it's still the same Pod. Why hasn't the Pod been updated? When you update the Pod template in a ReplicaSet or Deployment, the Pods are deleted and recreated, so why not here?

The reason for this is probably the biggest drawback of using StatefulSets with the default Pod management policy `OrderedReady`. When you use this policy, the StatefulSet does nothing until the Pod is ready. If your StatefulSet gets into the same state as shown here, you'll have to manually delete the unhealthy Pod.

Now delete the `quiz-0` Pod and watch the StatefulSet controller create the three pods one by one as follows:

```
$ kubectl get pods -l app=quiz --watch
NAME      READY   STATUS             RESTARTS   AGE   #A
quiz-0    0/2     Terminating      0          20m   #A
quiz-0    0/2     Pending            0          0s    #B
quiz-0    0/2     ContainerCreating  0          0s    #B
quiz-0    1/2     Running            0          3s    #B
quiz-0    2/2     Running            0          3s    #B
quiz-1    0/2     Pending            0          0s    #C
quiz-1    0/2     ContainerCreating  0          0s    #C
quiz-1    2/2     Running            0          3s    #C
quiz-2    0/2     Pending            0          0s    #D
quiz-2    0/2     ContainerCreating  0          1s    #D
quiz-2    2/2     Running            0          4s    #D
```

As you can see, the Pods are created in ascending order, one at a time. You can see that Pod `quiz-1` isn't created until both containers in Pod `quiz-0` are ready. What you can't see is that because of the `minReadySeconds` setting, the controller waits an additional 10 seconds before creating Pod `quiz-1`. Similarly, Pod `quiz-2` is created 10 seconds after the containers in Pod `quiz-1` are ready. During the entire process, at most one Pod was being started. For some workloads, this is necessary to avoid race conditions.

Scaling a StatefulSet with the OrderedReady policy

When you scale the StatefulSet configured with the OrderedReady Pod management policy, the Pods are created/deleted one by one. Scale the quiz StatefulSet to a single replica and watch as the Pods are removed. First, the Pod with the highest ordinal, quiz-2, is marked for deletion, while Pod quiz-1 remains untouched. When the termination of Pod quiz-2 is complete, Pod quiz-1 is deleted. The minReadySeconds setting isn't used during scale-down, so there's no additional delay.

Just as with concurrent startup, some stateful workloads don't like it when you remove multiple replicas at once. With the OrderedReady policy, you let each replica finish its shutdown procedure before the shutdown of the next replica is triggered.

Blocked scale-downs

Another feature of the OrderedReady Pod management policy is that the controller blocks the scale-down operation if not all replicas are ready. To see this for yourself, create a new StatefulSet by applying the manifest file `sts.demo-ordered.yaml`. This StatefulSet deploys three replicas using the OrderedReady policy. After the Pods are created, fail the readiness probe in the Pod `demo-ordered-0` by running the following command:

```
$ kubectl exec demo-ordered-0 -- rm /tmp/ready
```

Running this command removes the `/tmp/ready` file that the readiness probe checks for. The probe is successful if the file exists. After you run this command, the `demo-ordered-0` Pod is no longer ready. Now scale the StatefulSet to two replicas as follows:

```
$ kubectl scale sts demo-ordered --replicas 2
statefulset.apps/demo-ordered scaled
```

If you list the pods with the `app=demo-ordered` label selector, you'll see that the StatefulSet controller does nothing. Unfortunately, the controller doesn't generate any Events or update the status of the StatefulSet object to tell you why it didn't perform the scale-down.

The controller completes the scale operation when the Pod is ready. You can make the readiness probe of the demo-ordered-0 Pod succeed by recreating the /tmp/ready file as follows:

```
$ kubectl exec demo-ordered-0 -- touch /tmp/ready
```

I suggest you investigate the behavior of this StatefulSet further and compare it to the StatefulSet in the manifest file `sts.demo-parallel.yaml`, which uses the `Parallel` Pod management policy. Use the `rm` and `touch` commands as shown to affect the outcome of the readiness probe in different replicas and see how it affects the two StatefulSets.

Ordered removal of Pods when deleting the StatefulSet

The `OrderedReady` Pod management policy affects the initial rollout of StatefulSet Pods, their scaling, and how Pods are replaced when a node fails. However, the policy doesn't apply when you delete the StatefulSet. If you want to terminate the Pods in order, you should first scale the StatefulSet to zero, wait until the last Pod finishes, and only then delete the StatefulSet.

15.3 Updating a StatefulSet

In addition to declarative scaling, StatefulSets also provide declarative updates, similar to Deployments. When you update the Pod template in a StatefulSet, the controller recreates the Pods with the updated template.

You may recall that the Deployment controller can perform the update in two ways, depending on the strategy specified in the Deployment object. You can also specify the update strategy in the `updateStrategy` field in the `spec` section of the StatefulSet manifest, but the available strategies are different from those in a Deployment, as you can see in the following table.

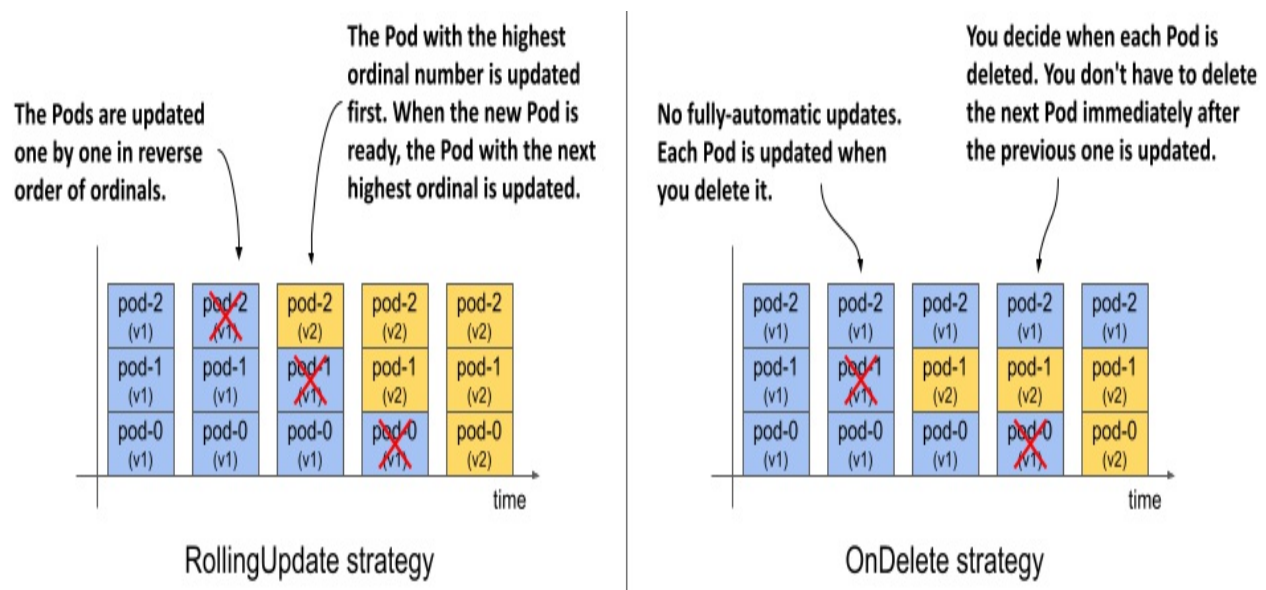
Table 15.2 The supported StatefulSet update strategies

Value	Description

RollingUpdate	In this update strategy, the Pods are replaced one by one. The Pod with the highest ordinal number is deleted first and replaced with a Pod created with the new template. When this new Pod is ready, the Pod with the next highest ordinal number is replaced. The process continues until all Pods have been replaced. This is the default strategy.
OnDelete	The StatefulSet controller waits for each Pod to be manually deleted. When you delete the Pod, the controller replaces it with a Pod created with the new template. With this strategy, you can replace Pods in any order and at any rate.

The following figure shows how the Pods are updated over time for each update strategy.

Figure 15.8 How the Pods are updated over time with different update strategies



The RollingUpdate strategy, which you can find in both Deployments and StatefulSets, is similar between the two objects, but differs in the parameters you can set. The OnDelete strategy lets you replace Pods at your own pace and in any order. It's different from the Recreate strategy found in

Deployments, which automatically deletes and replaces all Pods at once.

15.3.1 Using the RollingUpdate strategy

The RollingUpdate strategy in a StatefulSet behaves similarly to the RollingUpdate strategy in Deployments, but only one Pod is replaced at a time. You may recall that you can configure the Deployment to replace multiple Pods at once using the `maxSurge` and `maxUnavailable` parameters. The rolling update strategy in StatefulSets has no such parameters.

You may also recall that you can slow down the rollout in a Deployment by setting the `minReadySeconds` field, which causes the controller to wait a certain amount of time after the new Pods are ready before replacing the other Pods. You’ve already learned that StatefulSets also provide this field and that it affects the scaling of StatefulSets in addition to the updates.

Let’s update the `quiz-api` container in the `quiz` StatefulSet to version `0.2`. Since `RollingUpdate` is the default update strategy type, you can omit the `updateStrategy` field in the manifest. To trigger the update, use `kubectl edit` to change the value of the `ver` label and the image tag in the `quiz-api` container to `0.2`. You can also apply the manifest file `sts.quiz.0.2.yaml` with `kubectl apply` instead.

You can track the rollout with the `kubectl rollout status` command as in the previous chapter. The full command and its output are as follows:

```
$ kubectl rollout status sts quiz
Waiting for partitioned roll out to finish: 0 out of 3 new pods h
Waiting for 1 pods to be ready...
Waiting for partitioned roll out to finish: 1 out of 3 new pods h
Waiting for 1 pods to be ready...
...
```

Because the Pods are replaced one at a time and the controller waits until each replica is ready before moving on to the next, the `quiz` Service remains accessible throughout the process. If you list the Pods as they’re updated, you’ll see that the Pod with the highest ordinal number, `quiz-2`, is updated first, followed by `quiz-1`, as shown here:


```
$ kubectl get pods -l app=quiz -L controller-revision-hash,ver
NAME      READY   STATUS    RESTARTS   AGE      CONTROLLER-REVISI
quiz-0    2/2     Running   0           50m      quiz-6c48bdd8df
quiz-1    2/2     Terminating 0           10m      quiz-6c48bdd8df
quiz-2    2/2     Running   0           20s      quiz-6945968d9
```

The update process is complete when the Pod with the lowest ordinal number, quiz-0, is updated. At this point, the `kubectl rollout status` command reports the following status:

```
$ kubectl rollout status sts quiz
partitioned roll out complete: 3 new pods have been updated...
```

Updates with Pods that aren't ready

If the StatefulSet is configured with the `RollingUpdate` strategy and you trigger the update when not all Pods are ready, the rollout is held back. The `kubectl rollout status` indicates that the controller is waiting for one or more Pods to be ready.

If a new Pod fails to become ready during the update, the update is also paused, just like a Deployment update. The rollout will resume when the Pod is ready again. So, if you deploy a faulty version whose readiness probe never succeeds, the update will be blocked after the first Pod is replaced. If the number of replicas in the StatefulSet is sufficient, the service provided by the Pods in the StatefulSet is unaffected.

Displaying the revision history

You may recall that Deployments keep a history of recent revisions. Each revision is represented by the ReplicaSet that the Deployment controller created when that revision was active. StatefulSets also keep a revision history. You can use the `kubectl rollout history` command to display it as follows.

```
$ kubectl rollout history sts quiz
statefulset.apps/quiz
REVISION  CHANGE-CAUSE
1          <none>
2          <none>
```

You may wonder where this history is stored, because unlike Deployments, a StatefulSet manages Pods directly. And if you look at the object manifest of the quiz StatefulSet, you'll notice that it only contains the current Pod template and no previous revisions. So where is the revision history of the StatefulSet stored?

The revision history of StatefulSets and DaemonSets, which you'll learn about in the next chapter, is stored in ControllerRevision objects. A ControllerRevision is a generic object that represents an immutable snapshot of the state of an object at a particular point in time. You can list ControllerRevision objects as follows:

```
$ kubectl get controllerrevisions
```

NAME	CONTROLLER	REVISION	AGE
quiz-6945968d9	statefulset.apps/quiz	2	1m
quiz-6c48bdd8df	statefulset.apps/quiz	1	50m

Since these objects are used internally, you don't need to know anything more about them. However, if you want to learn more, you can use the `kubectl explain` command.

Rolling back to a previous revision

If you're updating the StatefulSet and the rollout hangs, or if the rollout was successful, but you want to revert to the previous revision, you can use the `kubectl rollout undo` command, as described in the previous chapter. You'll update the quiz StatefulSet again in the next section, so please reset it to the previous version as follows:

```
$ kubectl rollout undo sts quiz
statefulset.apps/quiz rolled back
```

You can also use the `--to-revision` option to return to a specific revision. As with Deployments, Pods are rolled back using the update strategy configured in the StatefulSet. If the strategy is `RollingUpdate`, the Pods are reverted one at a time.

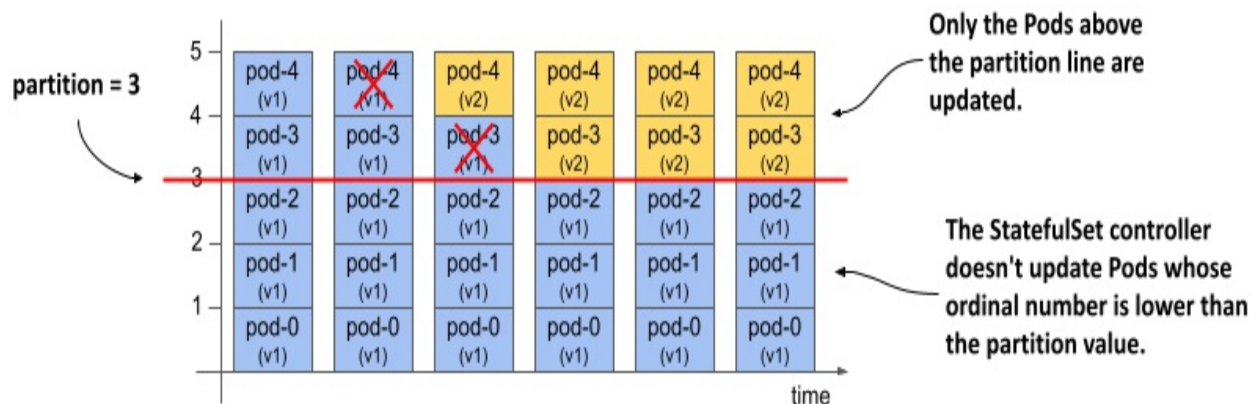
15.3.2 RollingUpdate with partition

StatefulSets don't have a pause field that you can use to prevent a Deployment rollout from being triggered, or to pause it halfway. If you try to pause the StatefulSet with the `kubectl rollout pause sts quiz` command, you receive the following error message:

```
$ kubectl rollout pause sts quiz
error: statefulsets.apps "quiz" pausing is not supported
```

In a StatefulSet you can achieve the same result and more with the `partition` parameter of the `RollingUpdate` strategy. The value of this field specifies the ordinal number at which the StatefulSet should be partitioned. As shown in the following figure, pods with an ordinal number lower than the partition value aren't updated.

Figure 15.9 Partitioning a rolling update



If you set the `partition` value appropriately, you can implement a Canary deployment, control the rollout manually, or stage an update instead of triggering it immediately.

Staging an update

To stage a StatefulSet update without actually triggering it, set the `partition` value to the number of replicas or higher, as in the manifest file `sts.quiz.0.2.partition.yaml` shown in the following listing.

Listing 15.7 Staging a StatefulSet update with the `partition` field

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: quiz
spec:
  updateStrategy:
    type: RollingUpdate
    rollingUpdate:
      partition: 3      #A
  replicas: 3          #A
  ...

```

Apply this manifest file and confirm that the rollout doesn't start even though the Pod template has been updated. If you set the partition value this way, you can make several changes to the StatefulSet without triggering the rollout. Now let's look at how you can trigger the update of a single Pod.

Deploying a canary

To deploy a canary, set the partition value to the number of replicas minus one. Since the quiz StatefulSet has three replicas, you set the partition to 2. You can do this with the `kubectl patch` command as follows:

```
$ kubectl patch sts quiz -p '{"spec": {"updateStrategy": {"rollin
statefulset.apps/quiz patched
```

If you now look at the list of quiz Pods, you'll see that only the Pod quiz-2 has been updated to version 0.2 because only its ordinal number is greater than or equal to the partition value.

```
$ kubectl get pods -l app=quiz -L controller-revision-hash,ver
```

NAME	READY	STATUS	RESTARTS	AGE	CONTROLLER-REVISION-H
quiz-0	2/2	Running	0	8m	quiz-6c48bdd8df
quiz-1	2/2	Running	0	8m	quiz-6c48bdd8df
quiz-2	2/2	Running	0	20s	quiz-6945968d9

The Pod quiz-2 is the canary that you use to check if the new version behaves as expected before rolling out the changes to the remaining Pods.

At this point I'd like to draw your attention to the status section of the StatefulSet object. It contains information about the total number of replicas,

the number of replicas that are ready and available, the number of current and updated replicas, and their revision hashes. To display the status, run the following command:

```
$ kubectl get sts quiz -o yaml
...
status:
  availableReplicas: 3      #A
  collisionCount: 0
  currentReplicas: 2       #B
  currentRevision: quiz-6c48bdd8df      #B
  observedGeneration: 8
  readyReplicas: 3        #A
  replicas: 3             #A
  updateRevision: quiz-6945968d9      #C
  updatedReplicas: 1      #C
```

As you can see from the status, the StatefulSet is now split into two partitions. If a Pod is deleted at this time, the StatefulSet controller will create it with the correct template. For example, if you delete one of the Pods with version 0.1, the replacement Pod will be created with the previous template and will run again with version 0.1. If you delete the Pod that's already been updated, it'll be recreated with the new template. Feel free to try this out for yourself. You can't break anything.

Completing a partitioned update

When you're confident the canary is fine, you can let the StatefulSet update the remaining pods by setting the partition value to zero as follows:

```
$ kubectl patch sts quiz -p '{"spec": {"updateStrategy": {"rollin
statefulset.apps/quiz patched
```

When the partition field is set to zero, the StatefulSet updates all Pods. First, the pod quiz-1 is updated, followed by quiz-0. If you had more Pods, you could also use the partition field to update the StatefulSet in phases. In each phase, you decide how many Pods you want to update and set the partition value accordingly.

At the time of writing, partition is the only parameter of the RollingUpdate

strategy. You've seen how you can use it to control the rollout. If you want even more control, you can use the `onDelete` strategy, which I'll try next. Before you continue, please reset the StatefulSet to the previous revision as follows:

```
$ kubectl rollout undo sts quiz
statefulset.apps/quiz rolled back
```

15.3.3 OnDelete strategy

If you want to have full control over the rollout process, you can use the `onDelete` update strategy. To configure the StatefulSet with this strategy, use `kubectl apply` to apply the manifest file `sts.quiz.0.2.onDelete.yaml`. The following listing shows how the update strategy is set.

Listing 15.8 Setting the `onDelete` update strategy

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: quiz
spec:
  updateStrategy: #A
    type: OnDelete #A
  ...
```

This manifest updates the `quiz-api` container in the Pod template to use the `:0.2` image tag. However, because it sets the update strategy to `onDelete`, nothing happens when you apply the manifest.

If you use the `onDelete` strategy, the rollout is semi-automatic. You manually delete each Pod, and the StatefulSet controller then creates the replacement Pod with the new template. With this strategy, you can decide which Pod to update and when. You don't necessarily have to delete the Pod with the highest ordinal number first. Try deleting the Pod `quiz-0`. When its containers exit, a new `quiz-0` Pod with version `0.2` appears:

```
$ kubectl get pods -l app=quiz -L controller-revision-hash,ver
NAME      READY   STATUS    RESTARTS   AGE      CONTROLLER-REVISION-H
quiz-0    2/2     Running   0           53s     quiz-6945968d9
quiz-1    2/2     Running   0           11m     quiz-6c48bdd8df
```

To complete the rollout, you need to delete the remaining Pods. You can do this in the order that the workloads require, or in the order that you want.

Rolling back with the OnDelete strategy

Since the update strategy also applies when you use the `kubectl rollout undo` command, the rollback process is also semi-automatic. You must delete each Pod yourself if you want to roll it back to the previous revision.

Updates with Pods that aren't ready

Since you control the rollout and the controller replaces any Pod you delete, the Pod's readiness status is irrelevant. If you delete a Pod that's not ready, the controller updates it.

If you delete a Pod and the new Pod isn't ready, but you still delete the next Pod, the controller will update that second Pod as well. It's your responsibility to consider Pod readiness.

15.4 Managing stateful applications with Kubernetes Operators

In this chapter, you saw that managing a stateful application can involve more than what Kubernetes provides with the StatefulSet object. In the case of MongoDB, you need to reconfigure the MongoDB replica set every time you scale the StatefulSet. If you don't, the replica set may lose quorum and stop working. Also, if a cluster node fails, manual intervention is required to move the Pods to the remaining nodes.

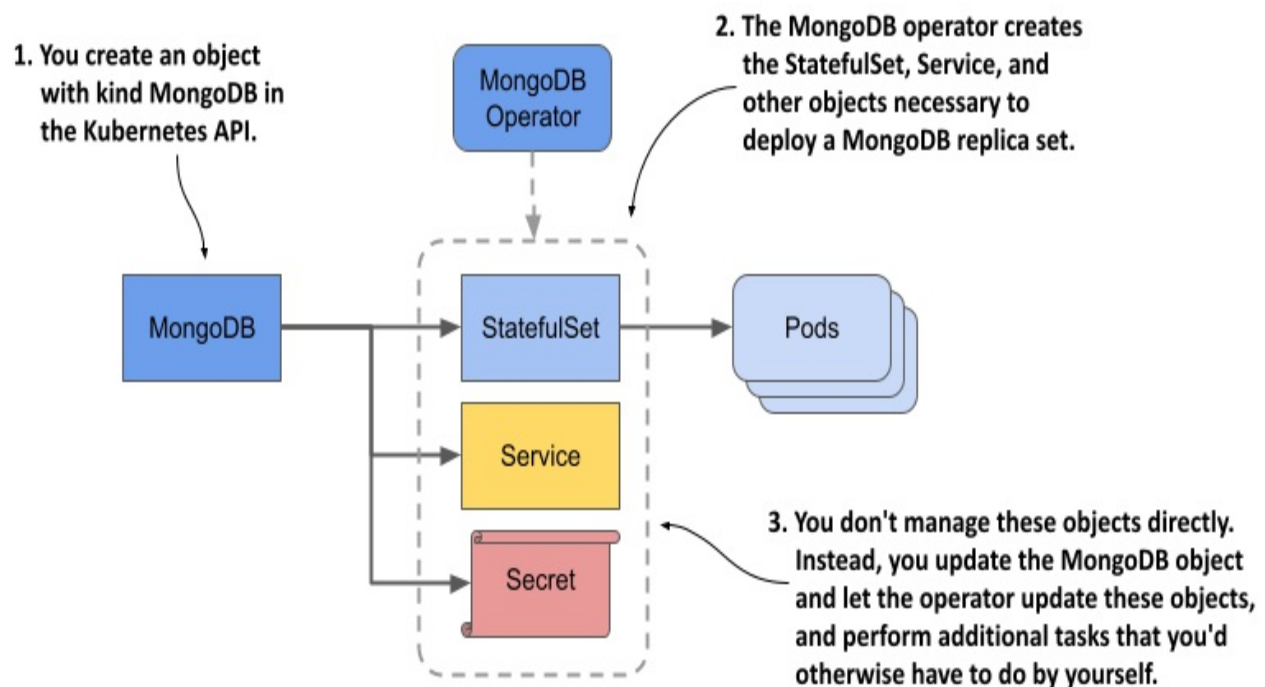
Managing stateful applications is difficult. StatefulSets do a good job of automating some basic tasks, but much of the work still has to be done manually. If you want to deploy a fully automated stateful application, you need more than what StatefulSets can provide. This is where Kubernetes *operators* come into play. I'm not referring to the people running Kubernetes

clusters, but the software that does it for them.

A *Kubernetes operator* is an application-specific controller that automates the deployment and management of an application running on Kubernetes. An operator is typically developed by the same organization that builds the application, as they know best how to manage it. Kubernetes doesn't ship with operators. Instead, you must install them separately.

Each operator extends the Kubernetes API with its own set of custom object types that you use to deploy and configure the application. You create an instance of this custom object type using the Kubernetes API and leave it to the operator to create the Deployments or StatefulSets that create the Pods in which the application runs, as shown in the following figure.

Figure 15.10 Managing an application through custom resources and operators



In this section, you'll learn how to use the MongoDB Community Operator to deploy MongoDB. Since I don't know how the operator will change after the book is published, I won't go into too much detail, but I'll list all the steps that were necessary to install the Operator and deploy MongoDB at the time I wrote the book so you can get a feel for what's required even if you don't try

it yourself.

If you do want to try this yourself, please follow the documentation in the GitHub repository of the MongoDB community operator at <https://github.com/mongodb/mongodb-kubernetes-operator>.

15.4.1 Deploying the MongoDB community operator

An operator is itself an application that you typically deploy in the same Kubernetes cluster as the application that the operator is to manage. At the time of writing, the MongoDB operator documentation instructs you to first clone the GitHub repository as follows:

```
$ git clone https://github.com/mongodb/mongodb-kubernetes-operator
```

Then you go to the `mongodb-kubernetes-operator` directory, where you find the source code of the operator and some Kubernetes object manifests. You can ignore the source code. You're only interested in the manifest files.

You can decide if you want to deploy the operator and MongoDB in the same namespace, or if you want to deploy the operator so that each user in the cluster can deploy their own MongoDB instance(s). For simplicity, I'll use a single namespace.

Extending the API with the MongoDBCommunity object kind

First, you create a CustomResourceDefinition object that extends your cluster's Kubernetes API with an additional object type. To do this, you apply the object manifest as follows:

```
$ kubectl apply -f config/crd/bases/mongodbcommunity.mongodb.com_customresourcedefinition/mongodbcommunity.mongodb
```

Using your cluster's API, you can now create objects of kind `MongoDBCommunity`. You'll create this object later.

Note

Unfortunately, the object kind is MongoDBCommunity, which makes it hard to understand that this object represents a MongoDB deployment and not a community. The reason it's called MongoDBCommunity is because you're using the community version of the operator. If you use the Enterprise version, the naming is more appropriate. There the object kind is MongoDB, which clearly indicates that the object represents a MongoDB deployment.

Creating supporting objects

Next, you create various other security-related objects by applying their manifests. Here you need to specify the namespace in which these objects should be created. Let's use the namespace `mongodb`. Apply the manifests as follows:

```
$ kubectl apply -k config/rbac/ -n mongodb
serviceaccount/mongodb-database created
serviceaccount/mongodb-kubernetes-operator created
role.rbac.authorization.k8s.io/mongodb-database created
role.rbac.authorization.k8s.io/mongodb-kubernetes-operator created
rolebinding.rbac.authorization.k8s.io/mongodb-database created
rolebinding.rbac.authorization.k8s.io/mongodb-kubernetes-operator created
```

Note

You'll learn more about these object types and CustomResourceDefinitions in the remaining chapters of this book.

Installing the operator

The last step is to install the operator by creating a Deployment as follows:

```
$ kubectl create -f config/manager/manager.yaml -n mongodb
deployment.apps/mongodb-kubernetes-operator created
```

Verify that the operator Pod exists and is running by listing the Pods in the `mongodb` namespace:

```
$ kubectl get pods -n mongodb
```

NAME	READY	STATUS
<code>mongodb-kubernetes-operator-648bf8cc59-wzvvhx</code>	<code>1/1</code>	<code>Running</code>

That wasn't so hard, was it? The operator is running now, but you haven't deployed MongoDB yet. The operator is just the tool you use to do that.

15.4.2 Deploying MongoDB via the operator

To deploy a MongoDB replica set, you create an instance of the `MongoDBCommunity` object type instead of creating `StatefulSets` and the other objects.

Creating an instance of the `MongoDBCommunity` object type

First edit the file

`config/samples/mongodb.com_v1_mongodbcommunity_cr.yaml` to replace the string `<your-password-here>` with the password of your choice.

The file contains manifests for a `MongoDBCommunity` and a `Secret` object. The following listing shows the manifest of the former.

Listing 15.9 The `MongoDBCommunity` custom object manifest

```
apiVersion: mongodbcommunity.mongodb.com/v1      #A
kind: MongoDBCommunity      #A
metadata:
  name: example-mongodb      #B
spec:
  members: 3      #C
  type: ReplicaSet      #C
  version: "4.2.6"      #D
  security:      #E
    authentication:      #E
      modes: ["SCRAM"]      #E
  users:      #E
    - name: my-user      #E
      db: admin      #E
      passwordSecretRef:      #E
        name: my-user-password      #E
      roles:      #E
        - name: clusterAdmin      #E
          db: admin      #E
        - name: userAdminAnyDatabase      #E
          db: admin      #E
      scramCredentialsSecretName: my-scram      #E
```

```
additionalMongodConfig:      #E
  storage.wiredTiger.engineConfig.journalCompressor: zlib      #
```

As you can see, this custom object has the same structure as the Kubernetes API core objects. The `apiVersion` and `kind` fields specify the object type, the `name` field in the `metadata` section specifies the object name, and the `spec` section specifies the configuration for the MongoDB deployment, including type and version, the desired number of replica set members, and the security-related configuration.

Note

If the custom resource definition is well done, as in this case, you can use the `kubectl explain` command to learn more about the fields supported in this object type.

To deploy MongoDB, you apply this manifest file with `kubectl apply` as follows:

```
$ kubectl apply -f config/samples/mongodb.com_v1_mongodbcommunity
mongodbcommunity.mongodbcommunity.mongodb.com/example-mongodb cre
secret/my-user-password created
```

Inspecting the MongoDBCommunity object

You can then see the object you created with the `kubectl get` command as follows:

```
$ kubectl get mongodbcommunity
NAME                PHASE      VERSION
example-mongodb     Running    4.2.6
```

Just like the other Kubernetes controllers, the object you created is now processed in the reconciliation loop running in the operator. Based on the `MongoDBCommunity` object, the operator creates several objects: a `StatefulSet`, two `Services`, and some `Secrets`. If you check the `ownerReferences` field in these objects, you'll see that they're all owned by the `example-mongodb` `MongoDBCommunity` object. If you make direct changes to these objects, such as scaling the `StatefulSet`, the operator will

immediately undo your changes.

After the operator creates the Kubernetes core objects, the core controllers do their part. For example, the StatefulSet controller creates the Pods. Use `kubectl get` to list them as follows:

```
$ kubectl get pods -l app=example-mongodb-svc
NAME                READY   STATUS    RESTARTS   AGE
example-mongodb-0    2/2     Running   0           3m
example-mongodb-1    2/2     Running   0           2m
example-mongodb-2    2/2     Running   0           1m
```

The MongoDB operator not only creates the StatefulSet, but also makes sure that the MongoDB replica set is initiated automatically. You can use it right away. No additional manual configuration is required.

Managing the MongoDB deployment

You control the MongoDB deployment through the MongoDBCommunity object. The operator updates the configuration every time you update this object. For example, if you want to resize the MongoDB replica set, you change the value of the `members` field in the `example-mongodb` object. The operator then scales the underlying StatefulSet and reconfigures the MongoDB replica set. This makes scaling MongoDB trivial.

Note

At the time of writing, you can't use the `kubectl scale` command to scale the MongoDBCommunity object, but I'm sure the MongoDB operator developers will fix this soon.

15.4.3 Cleaning up

To uninstall MongoDB, delete the MongoDBCommunity object as follows:

```
$ kubectl delete mongodbcommunity example-mongodb
mongodbcommunity.mongodbcommunity.mongodb.com "example-mongodb" d
```

As you might expect, this orphans the underlying StatefulSet, Services, and

other objects. The garbage collector then deletes them. To remove the operator, you can delete the entire mongodb Namespace as follows:

```
$ kubectl delete ns mongodb
namespace "mongodb" deleted
```

As a last step, you also need to delete the CustomResourceDefinition to remove the custom object type from the API as follows:

```
$ kubectl delete crd mongodbcommunity.mongodbcommunity.mongodb.co
customresourcedefinition "mongodbcommunity.mongodbcommunity.mongo
```

15.5 Summary

In this chapter, you learned how to run stateful applications in Kubernetes. You learned that:

- Stateful workloads are harder to manage than their stateless counterparts because managing state is difficult. However, with StatefulSets, managing stateful workloads becomes much easier because the StatefulSet controller automates most of the work.
- With StatefulSets you can manage a group of Pods as pets, whereas Deployments treat the Pods like cattle. The Pods in a StatefulSet use ordinal numbers instead of having random names.
- A StatefulSet ensures that each replica gets its own stable identity and its own PersistentVolumeClaim(s). These claims are always associated with the same Pods.
- In combination with a StatefulSet, a headless Service ensures that each Pod receives a DNS record that always resolves to the Pod's IP address, even if the Pod is moved to another node and receives a new IP address.
- StatefulSet Pods are created in the order of ascending ordinal numbers, and deleted in reverse order.
- The Pod management policy configured in the StatefulSet determines whether Pods are created and deleted sequentially or simultaneously.
- The PersistentVolumeClaim retention policy determines whether claims are deleted or retained when you scale down or delete a StatefulSet.
- When you update the Pod template in a StatefulSet, the controller updates the underlying Pods. This happens on a rolling basis, from

highest to lowest ordinal number. Alternatively, you can use a semi-automatic update strategy, where you delete a Pod and the controller then replaces it.

- Since StatefulSets don't provide everything needed to fully manage a stateful workload, these types of workloads are typically managed via custom API object types and Kubernetes Operators. You create an instance of the custom object, and the Operator then creates the StatefulSet and supporting objects.

In this chapter, you also created the `quiz-data-importer` Pod, which, unlike all the other Pods you've created so far, performs a single task and then exits. In the next chapter, you'll learn how to run these types of workloads using the Job and CronJob object types. You'll also learn how to use a DaemonSet to run a system Pod on each node.

16 Deploying node agents and daemons with DaemonSets

This chapter covers

- Running an agent Pod on each cluster node
- Running agent Pods on a subset of nodes
- Allowing Pods to access the host node's resources
- Assigning a priority class to a Pod
- Communicating with the local agent Pod

In the previous chapters, you learned how to use Deployments or StatefulSets to distribute multiple replicas of a workload across the nodes of your cluster. But what if you want to run exactly one replica on each node? For example, you might want each node to run an agent or daemon that provides a system service such as metrics collection or log aggregation for that node. To deploy these types of workloads in Kubernetes, you use a DaemonSet.

Before you begin, create the kiada Namespace, change to the Chapter16/ directory, and apply all manifests in the SETUP/ directory by running the following commands:

```
$ kubectl create ns kiada
$ kubectl config set-context --current --namespace kiada
$ kubectl apply -f SETUP -R
```

NOTE

You can find the code files for this chapter at <https://github.com/luksa/kubernetes-in-action-2nd-edition/tree/master/Chapter16>.

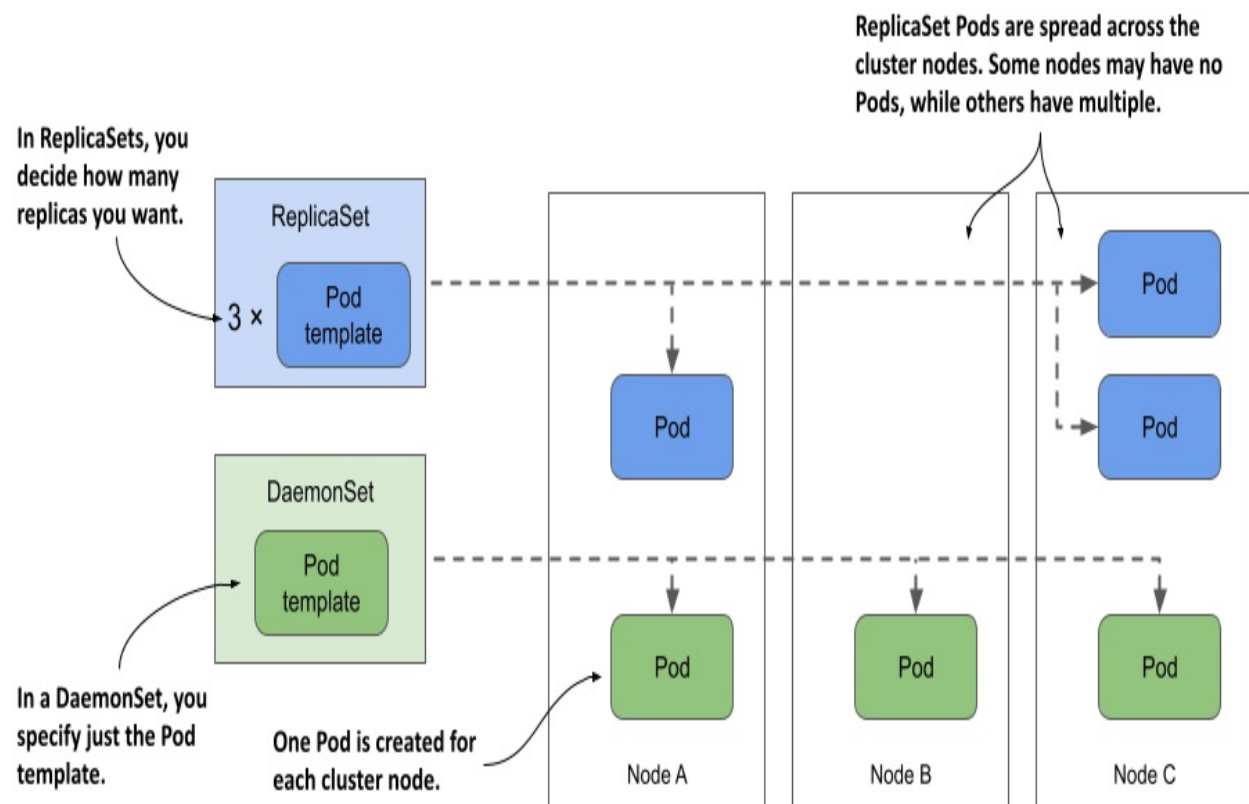
16.1 Introducing DaemonSets

A DaemonSet is an API object that ensures that exactly one replica of a Pod is running on each cluster node. By default, daemon Pods are deployed on every node, but you can use a node selector to restrict deployment to some of the nodes.

16.1.1 Understanding the DaemonSet object

A DaemonSet contains a Pod template and uses it to create multiple Pod replicas, just like Deployments, ReplicaSets, and StatefulSets. However, with a DaemonSet, you don't specify the desired number of replicas as you do with the other objects. Instead, the DaemonSet controller creates as many Pods as there are nodes in the cluster. It ensures that each Pod is scheduled to a different Node, unlike Pods deployed by a ReplicaSet, where multiple Pods can be scheduled to the same Node, as shown in the following figure.

Figure 16.1 DaemonSets run a Pod replica on each node, whereas ReplicaSets scatter them around the cluster.



What type of workloads are deployed via DaemonSets and why

A DaemonSet is typically used to deploy infrastructure Pods that provide some sort of system-level service to each cluster node. This includes the log collection for the node's system processes, as well as its Pods, daemons to monitor these processes, tools that provide the cluster's network and storage, manage the installation and update of software packages, and services that provide interfaces to the various devices attached to the node.

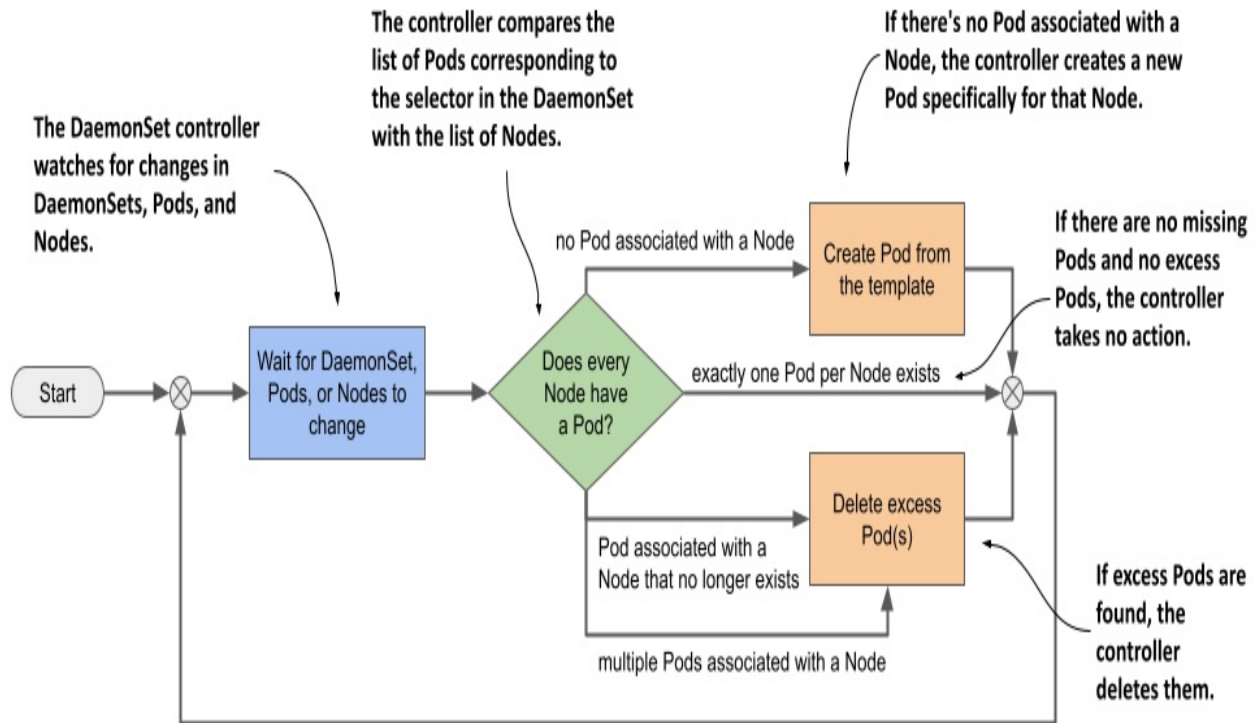
The Kube Proxy component, which is responsible for routing traffic for the Service objects you create in your cluster, is usually deployed via a DaemonSet in the kube-system Namespace. The Container Network Interface (CNI) plugin that provides the network over which the Pods communicate is also typically deployed via a DaemonSet.

Although you could run system software on your cluster nodes using standard methods such as init scripts or systemd, using a DaemonSet ensures that you manage all workloads in your cluster in the same way.

Understanding the operation of the DaemonSet controller

Just like ReplicaSets and StatefulSets, a DaemonSet contains a Pod template and a label selector that determines which Pods belong to the DaemonSet. In each pass of its reconciliation loop, the DaemonSet controller finds the Pods that match the label selector, checks that each node has exactly one matching Pod, and creates or removes Pods to ensure that this is the case. This is illustrated in the next figure.

Figure 16.2 The DaemonSet controller's reconciliation loop



When you add a Node to the cluster, the DaemonSet controller creates a new Pod and associates it with that Node. When you remove a Node, the DaemonSet deletes the Pod object associated with it. If one of these daemon Pods disappears, for example, because it was deleted manually, the controller immediately recreates it. If an additional Pod appears, for example, if you create a Pod that matches the label selector in the DaemonSet, the controller immediately deletes it.

16.1.2 Deploying Pods with a DaemonSet

A DaemonSet object manifest looks very similar to that of a ReplicaSet, Deployment, or StatefulSet. Let's look at a DaemonSet example called demo, which you can find in the book's code repository in the file `ds.demo.yaml`. The following listing shows the full manifest.

Listing 16.1 A DaemonSet manifest example

```

apiVersion: apps/v1      #A
kind: DaemonSet          #A
metadata:
  name: demo             #B

```

```
spec:
  selector:      #C
    matchLabels:  #C
      app: demo   #C
  template:      #D
    metadata:     #D
      labels:     #D
        app: demo   #D
    spec:         #D
      containers:  #D
        - name: demo   #D
          image: busybox  #D
          command:     #D
            - sleep    #D
            - infinity  #D
```

The DaemonSet object kind is part of the apps/v1 API group/version. In the object's spec, you specify the label selector and a Pod template, just like a ReplicaSet for example. The metadata section within the template must contain labels that match the selector.

Note

The selector is immutable, but you can change the labels as long as they still match the selector. If you need to change the selector, you must delete the DaemonSet and recreate it. You can use the `--cascade=orphan` option to preserve the Pods while replacing the DaemonSet.

As you can see in the listing, the demo DaemonSet deploys Pods that do nothing but execute the `sleep` command. That's because the goal of this exercise is to observe the behavior of the DaemonSet itself, not its Pods. Later in this chapter, you'll create a DaemonSet whose Pods actually do something.

Quickly inspecting a DaemonSet

Create the DaemonSet by applying the `ds.demo.yaml` manifest file with `kubectl apply` and then list all DaemonSets in the current Namespace as follows:

```
$ kubectl get ds
```

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE
demo	2	2	2	2	2	<none>

Note

The shorthand for DaemonSet is ds.

The command's output shows that two Pods were created by this DaemonSet. In your case, the number may be different because it depends on the number and type of Nodes in your cluster, as I'll explain later in this section.

Just as with ReplicaSets, Deployments, and StatefulSets, you can run `kubectl get` with the `-o wide` option to also display the names and images of the containers and the label selector.

```
$ kubectl get ds -o wide
```

NAME	DESIRED	CURRENT	...	CONTAINERS	IMAGES	SELECTOR
Demo	2	2	...	demo	busybox	app=demo

Inspecting a DaemonSet in detail

The `-o wide` option is the fastest way to see what's running in the Pods created by each DaemonSet. But if you want to see even more details about the DaemonSet, you can use the `kubectl describe` command, which gives the following output:

```
$ kubectl describe ds demo
```

```
Name: demo #A
Selector: app=demo #B
Node-Selector: <none> #C
Labels: <none> #D
Annotations: deprecated.daemonset.template.generation: 1 #E
Desired Number of Nodes Scheduled: 2 #F
Current Number of Nodes Scheduled: 2 #F
Number of Nodes Scheduled with Up-to-date Pods: 2 #F
Number of Nodes Scheduled with Available Pods: 2 #F
Number of Nodes Misscheduled: 0 #F
Pods Status: 2 Running / 0 Waiting / 0 Succeeded / 0 Failed #
Pod Template: #G
  Labels: app=demo #G
  Containers: #G
    demo: #G
```

```

Image:      busybox      #G
Port:       <none>       #G
Host Port:  <none>       #G
Command:    #G
  sleep     #G
  infinity  #G
Environment: <none>      #G
Mounts:     <none>      #G
Volumes:    <none>      #G
Events:      #H
  Type      Reason              Age    From                      Message
  ----      -
Normal      SuccessfulCreate    40m    daemonset-controller      Created p
Normal      SuccessfulCreate    40m    daemonset-controller      Created p

```

The output of the `kubectl describe` commands includes information about the object's labels and annotations, the label selector used to find the Pods of this DaemonSet, the number and state of these Pods, the template used to create them, and the Events associated with this DaemonSet.

Understanding a DaemonSet's status

During each reconciliation, the DaemonSet controller reports the state of the DaemonSet in the object's status section. Let's look at the demo DaemonSet's status. Run the following command to print the object's YAML manifest:

```

$ kubectl get ds demo -o yaml
...
status:
  currentNumberScheduled: 2
  desiredNumberScheduled: 2
  numberAvailable: 2
  numberMisscheduled: 0
  numberReady: 2
  observedGeneration: 1
  updatedNumberScheduled: 2

```

As you can see, the status of a DaemonSet consists of several integer fields. The following table explains what the numbers in those fields mean.

Table 16.1 DaemonSet status fields

Value	Description
currentNumberScheduled	The number of Nodes that run at least one Pod associated with this DaemonSet.
desiredNumberScheduled	The number of Nodes that should run the daemon Pod, regardless of whether they actually run it.
numberAvailable	The number of Nodes that run at least one daemon Pod that's available.
numberMisscheduled	The number of Nodes that are running a daemon Pod but shouldn't be running it.
numberReady	The number of Nodes that have at least one daemon Pod running and ready
updatedNumberScheduled	The number of Nodes whose daemon Pod is current with respect to the Pod template in the DaemonSet.

The status also contains the `observedGeneration` field, which has nothing to do with DaemonSet Pods. You can find this field in virtually all other objects that have a spec and a status. You'll learn about this field in chapter 20, so ignore it for now.

You'll notice that all the status fields explained in the previous table indicate the number of Nodes, not Pods. Some field descriptions also imply

that more than one daemon Pod could be running on a Node, even though a DaemonSet is supposed to run exactly one Pod on each Node. The reason for this is that when you update the DaemonSet's Pod template, the controller runs a new Pod alongside the old Pod until the new Pod is available. When you observe the status of a DaemonSet, you aren't interested in the total number of Pods in the cluster, but in the number of Nodes that the DaemonSet serves.

Understanding why there are fewer daemon Pods than Nodes

In the previous section, you saw that the DaemonSet status indicates that two Pods are associated with the demo DaemonSet. This is unexpected because my cluster has three Nodes, not just two.

I mentioned that you can use a node selector to restrict the Pods of a DaemonSet to some of the Nodes. However, the demo DaemonSet doesn't specify a node selector, so you'd expect three Pods to be created in a cluster with three Nodes. What's going on here? Let's get to the bottom of this mystery by listing the daemon Pods with the same label selector defined in the DaemonSet.

Note

Don't confuse the label selector with the node selector; the former is used to associate Pods with the DaemonSet, while the latter is used to associate Pods with Nodes.

The label selector in the DaemonSet is `app=demo`. Pass it to the `kubectl get` command with the `-l` (or `--selector`) option. Additionally, use the `-o wide` option to display the Node for each Pod. The full command and its output are as follows:

```
$ kubectl get pods -l app=demo -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
demo-w8tgm	1/1	Running	0	80s	10.244.2.42	kin
demo-wqd22	1/1	Running	0	80s	10.244.1.64	kin

Now list the Nodes in the cluster and compare the two lists:


```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
kind-control-plane	Ready	control-plane,master	22h	v1.23.
kind-worker	Ready	<none>	22h	v1.23.
kind-worker2	Ready	<none>	22h	v1.23.

It looks like the DaemonSet controller has only deployed Pods on the worker Nodes, but not on the master Node running the cluster's control plane components. Why is that?

In fact, if you're using a multi-node cluster, it's very likely that none of the Pods you deployed in the previous chapters were scheduled to the Node hosting the control plane, such as the `kind-control-plane` Node in a cluster created with the `kind` tool. As the name implies, this Node is meant to only run the Kubernetes components that control the cluster. In chapter 2, you learned that containers help isolate workloads, but this isolation isn't as good as when you use multiple separate virtual or physical machines. A misbehaving workload running on the control plane Node can negatively affect the operation of the entire cluster. For this reason, Kubernetes only schedules workloads to control plane Nodes if you explicitly allow it. This rule also applies to workloads deployed through a DaemonSet.

Deploying daemon Pods on control plane Nodes

The mechanism that prevents regular Pods from being scheduled to control plane Nodes is called Taints and Tolerations. You'll learn more about it in chapter 23. Here, you'll only learn how to get a DaemonSet to deploy Pods to all Nodes. This may be necessary if the daemon Pods provide a critical service that needs to run on all nodes in the cluster. Kubernetes itself has at least one such service—the Kube Proxy. In most clusters today, the Kube Proxy is deployed via a DaemonSet. You can check if this is the case in your cluster by listing DaemonSets in the `kube-system` namespace as follows:

```
$ kubectl get ds -n kube-system
```

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE
kindnet	3	3	3	3	3
kube-proxy	3	3	3	3	3

If, like me, you use the `kind` tool to run your cluster, you'll see two

DaemonSets. Besides the kube-proxy DaemonSet, you'll also find a DaemonSet called kindnet. This DaemonSet deploys the Pods that provide the network between all the Pods in the cluster via CNI, the Container Network Interface, which you'll learn more about in chapter 19.

The numbers in the output of the previous command indicate that the Pods of these DaemonSets are deployed on all cluster nodes. Their manifests reveal how they do this. Display the manifest of the kube-proxy DaemonSet as follows and look for the lines I've highlighted:

```
$ kubectl get ds kube-proxy -n kube-system -o yaml
apiVersion: apps/v1
kind: DaemonSet
...
spec:
  template:
    spec:
      ...
      tolerations:      #A
      - operator: Exists    #A
      volumes:
      ...
```

The highlighted lines aren't self-explanatory and it's hard to explain them without going into the details of taints and tolerations. In short, some Nodes may specify taints, and a Pod must tolerate a Node's taints to be scheduled to that Node. The two lines in the previous example allow the Pod to tolerate all possible taints, so consider them a way to deploy daemon Pods on absolutely all Nodes.

As you can see, these lines are part of the Pod template and not direct properties of the DaemonSet. Nevertheless, they're considered by the DaemonSet controller, because it wouldn't make sense to create a Pod that the Node rejects.

Inspecting a daemon Pod

Now let's turn back to the demo DaemonSet to learn more about the Pods that it creates. Take one of these Pods and display its manifest as follows:

```

$ kubectl get po demo-w8tgm -o yaml      #A
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: "2022-03-23T19:50:35Z"
  generateName: demo-
  labels:      #B
    app: demo      #B
    controller-revision-hash: 8669474b5b      #B
    pod-template-generation: "1"      #B
  name: demo-w8tgm
  namespace: bookinfo
  ownerReferences:      #C
  - apiVersion: apps/v1      #C
    blockOwnerDeletion: true      #C
    controller: true      #C
    kind: DaemonSet      #C
    name: demo      #C
    uid: 7e1da779-248b-4ff1-9bdb-5637dc6b5b86      #C
  resourceVersion: "67969"
  uid: 2d044e7f-a237-44ee-aa4d-1fe42c39da4e
spec:
  affinity:      #D
    nodeAffinity:      #D
      requiredDuringSchedulingIgnoredDuringExecution:      #D
        nodeSelectorTerms:      #D
        - matchFields:      #D
          - key: metadata.name      #D
            operator: In      #D
            values:      #D
          - kind-worker      #D
  containers:
    ...

```

Each Pod in a DaemonSet gets the labels you define in the Pod template, plus some additional labels that the DaemonSet controller itself adds. You can ignore the pod-template-generation label because it's obsolete. It's been replaced by the label controller-revision-hash. You may remember seeing this label in StatefulSet Pods in the previous chapter. It serves the same purpose—it allows the controller to distinguish between Pods created with the old and the new Pod template during updates.

The ownerReferences field indicates that daemon Pods belong directly to the DaemonSet object, just as stateful Pods belong to the StatefulSet object. There's no object between the DaemonSet and the Pods, as is the case with

Deployments and their Pods.

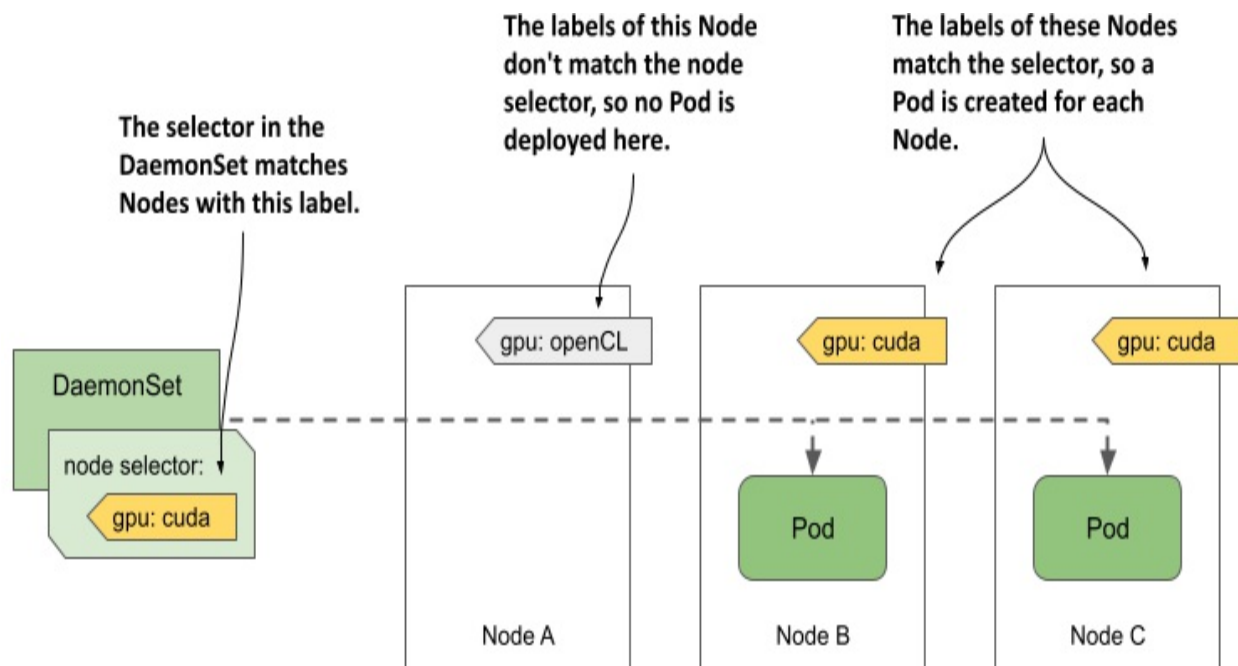
The last item in the manifest of a daemon Pod I want you to draw your attention to is the `spec.affinity` section. You'll learn more about Pod affinity in chapter 23, where I explain Pod scheduling in detail, but you should be able to tell that the `nodeAffinity` field indicates that this particular Pod needs to be scheduled to the Node kind-`worker`. This part of the manifest isn't included in the DaemonSet's Pod template, but is added by the DaemonSet controller to each Pod it creates. The node affinity of each Pod is configured differently to ensure that the Pod is scheduled to a specific Node.

In older versions of Kubernetes, the DaemonSet controller specified the target node in the Pod's `spec.nodeName` field, which meant that the DaemonSet controller scheduled the Pod directly without involving the Kubernetes Scheduler. Now, the DaemonSet controller sets the `nodeAffinity` field and leaves the `nodeName` field empty. This leaves scheduling to the Scheduler, which also takes into account the Pod's resource requirements and other properties.

16.1.3 Deploying to a subset of Nodes with a node selector

A DaemonSet deploys Pods to all cluster nodes that don't have taints that the Pod doesn't tolerate, but you may want a particular workload to run only on a subset of those nodes. For example, if only some of the nodes have special hardware, you might want to run the associated software only on those nodes and not on all of them. With a DaemonSet, you can do this by specifying a node selector in the Pod template. Note the difference between a node selector and a pod selector. The DaemonSet controller uses the former to filter eligible Nodes, whereas it uses the latter to know which Pods belong to the DaemonSet. As shown in the following figure, the DaemonSet creates a Pod for a particular Node only if the Node's labels match the node selector.

Figure 16.3 A node selector is used to deploy DaemonSet Pods on a subset of cluster nodes.



The figure shows a DaemonSet that deploys Pods only on Nodes that contain a CUDA-enabled GPU and are labelled with the label `gpu: cuda`. The DaemonSet controller deploys the Pods only on Nodes B and C, but ignores node A, because its label doesn't match the node selector specified in the DaemonSet.

Note

CUDA or Compute Unified Device Architecture is a parallel computing platform and API that allows software to use compatible Graphics Processing Units (GPUs) for general purpose processing.

Specifying a node selector in the DaemonSet

You specify the node selector in the `spec.nodeSelector` field in the Pod template. The following listing shows the same demo DaemonSet you created earlier, but with a `nodeSelector` configured so that the DaemonSet only deploys Pods to Nodes with the label `gpu: cuda`. You can find this manifest in the file `ds.demo.nodeSelector.yaml`.

Listing 16.2 A DaemonSet with a node selector

```

apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: demo
  labels:
    app: demo
spec:
  selector:
    matchLabels:
      app: demo
  template:
    metadata:
      labels:
        app: demo
    spec:
      nodeSelector:      #A
      gpu: cuda          #A
      containers:
      - name: demo
        image: busybox
        command:
        - sleep
        - infinity

```

Use the `kubectl apply` command to update the demo DaemonSet with this manifest file. Use the `kubectl get` command to see the status of the DaemonSet:

```

$ kubectl get ds
NAME      DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE
demo      0         0         0       0            0          gpu=c

```

As you can see, there are now no Pods deployed by the demo DaemonSet because no nodes match the node selector specified in the DaemonSet. You can confirm this by listing the Nodes with the node selector as follows:

```

$ kubectl get nodes -l gpu=cuda
No resources found

```

Moving Nodes in and out of scope of a DaemonSet by changing their labels

Now imagine you just installed a CUDA-enabled GPU to the Node kind-

worker2. You add the label to the Node as follows:

```
$ kubectl label node kind-worker2 gpu=cuda
node/kind-worker2 labeled
```

The DaemonSet controller watches not just DaemonSet and Pod, but also Node objects. When it detects a change in the labels of the kind-worker2 Node, it runs its reconciliation loop and creates a Pod for this Node, since it now matches the node selector. List the Pods to confirm:

```
$ kubectl get pods -l app=demo -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE
demo-jbhqg    1/1     Running   0           16s   10.244.1.65   kin
```

When you remove the label from the Node, the controller deletes the Pod:

```
$ kubectl label node kind-worker2 gpu-      #A
node/kind-worker2 unlabeled
```

```
$ kubectl get pods -l app=demo
NAME          READY   STATUS    RESTARTS   AGE   #B
demo-jbhqg    1/1     Terminating   0           71s
```

Using standard Node labels in DaemonSets

Kubernetes automatically adds some standard labels to each Node. Use the `kubectl describe` command to see them. For example, the labels of my kind-worker2 node are as follows:

```
$ kubectl describe node kind-worker2
Name:          kind-worker2
Roles:         <none>
Labels:        gpu=cuda
               kubernetes.io/arch=amd64
               kubernetes.io/hostname=kind-worker2
               kubernetes.io/os=linux
```

You can use these labels in your DaemonSets to deploy Pods based on the properties of each Node. For example, if your cluster consists of heterogeneous Nodes that use different operating systems or architectures, you configure a DaemonSet to target a specific OS and/or architecture by using the `kubernetes.io/arch` and `kubernetes.io/os` labels in its node

selector.

Suppose your cluster consists of AMD- and ARM-based Nodes. You have two versions of your node agent container image. One is compiled for AMD CPUs and the other is compiled for ARM CPUs. You can create a DaemonSet to deploy the AMD-based image to the AMD nodes, and a separate DaemonSet to deploy the ARM-based image to the other nodes. The first DaemonSet would use the following node selector:

```
nodeSelector:  
  kubernetes.io/arch: amd64
```

The other DaemonSet would use the following node selector:

```
nodeSelector:  
  kubernetes.io/arch: arm
```

This multiple DaemonSets approach is ideal if the configuration of the two Pod types differs not only in the container image, but also in the amount of compute resources you want to provide to each container. You can read more about this in chapter 22.

Note

You don't need multiple DaemonSets if you just want each node to run the correct variant of your container image for the node's architecture and there are no other differences between the Pods. In this case, using a single DaemonSet with multi-arch container images is the better option.

Updating the node selector

Unlike the Pod label selector, the node selector is mutable. You can change it whenever you want to change the set of Nodes that the DaemonSet should target. One way to change the selector is to use the `kubectl patch` command. In chapter 14, you learned how to patch an object by specifying the part of the manifest that you want to update. However, you can also update an object by specifying a list of patch operations using the JSON patch format. You can learn more about this format at jsonpatch.com. Here I

show you an example of how to use JSON patch to remove the `nodeSelector` field from the object manifest of the demo `DaemonSet`:

```
$ kubectl patch ds demo --type='json' -p='[{ "op": "remove", "pat
```

Instead of providing an updated portion of the object manifest, the JSON patch in this command specifies that the `spec.template.spec.nodeSelector` field should be removed.

16.1.4 Updating a `DaemonSet`

As with `Deployments` and `StatefulSets`, when you update the Pod template in a `DaemonSet`, the controller automatically deletes the Pods that belong to the `DaemonSet` and replaces them with Pods created with the new template.

You can configure the update strategy to use in the `spec.updateStrategy` field in the `DaemonSet` object's manifest, but the `spec.minReadySeconds` field also plays a role, just as it does for `Deployments` and `StatefulSets`. At the time of writing, `DaemonSets` support the strategies listed in the following table.

Table 16.2 The supported `DaemonSet` update strategies

Value	Description
<code>RollingUpdate</code>	In this update strategy, Pods are replaced one by one. When a Pod is deleted and recreated, the controller waits until the new Pod is ready. Then it waits an additional amount of time, specified in the <code>spec.minReadySeconds</code> field of the <code>DaemonSet</code> , before updating the Pods on the other Nodes. This is the default strategy.
<code>OnDelete</code>	The <code>DaemonSet</code> controller performs the update in a semi-automatic way. It waits for you to manually delete each Pod, and then replaces it with a new Pod from the updated

template. With this strategy, you can replace Pods at your own pace.

The RollingUpdate strategy is similar to that in Deployments, and the onDelete strategy is just like that in StatefulSets. As in Deployments, you can configure the RollingUpdate strategy with the maxSurge and maxUnavailable parameters, but the default values for these parameters in DaemonSets are different. The next section explains why.

The RollingUpdate strategy

To update the Pods of the demo DaemonSet, use the `kubectl apply` command to apply the manifest file `ds.demo.v2.rollingUpdate.yaml`. Its contents are shown in the following listing.

Listing 16.3 Specifying the RollingUpdate strategy in a DaemonSet

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: demo
spec:
  minReadySeconds: 30      #A
  updateStrategy:          #B
    type: RollingUpdate    #B
    rollingUpdate:         #B
      maxSurge: 0          #B
      maxUnavailable: 1    #B
  selector:
    matchLabels:
      app: demo
  template:
    metadata:
      labels:
        app: demo
        ver: v2            #C
    spec:
      ...
```

In the listing, the type of updateStrategy is RollingUpdate, with maxSurge set to 0 and maxUnavailable set to 1.

Note

These are the default values, so you can also remove the `updateStrategy` field completely and the update is performed the same way.

When you apply this manifest, the Pods are replaced as follows:

```
$ kubectl get pods -l app=demo -L ver
```

NAME	READY	STATUS	RESTARTS	AGE	VER	
demo-5nrz4	1/1	Terminating	0	10m		#A
demo-vx27t	1/1	Running	0	11m		#A

```
$ kubectl get pods -l app=demo -L ver
```

NAME	READY	STATUS	RESTARTS	AGE	VER	
demo-k2d6k	1/1	Running	0	36s	v2	#B
demo-vx27t	1/1	Terminating	0	11m		#B

```
$ kubectl get pods -l app=demo -L ver
```

NAME	READY	STATUS	RESTARTS	AGE	VER	
demo-k2d6k	1/1	Running	0	126s	v2	#C
demo-s7hsc	1/1	Running	0	62s	v2	#C

Since `maxSurge` is set to zero, the DaemonSet controller first stops the existing daemon Pod before creating a new one. Coincidentally, zero is also the default value for `maxSurge`, since this is the most reasonable behavior for daemon Pods, considering that the workloads in these Pods are usually node agents and daemons, of which only a single instance should run at a time.

If you set `maxSurge` above zero, two instances of the Pod run on the Node during an update for the time specified in the `minReadySeconds` field. Most daemons don't support this mode because they use locks to prevent multiple instances from running simultaneously. If you tried to update such a daemon in this way, the new Pod would never be ready because it couldn't obtain the lock, and the update would fail.

The `maxUnavailable` parameter is set to one, which means that the DaemonSet controller updates only one Node at a time. It doesn't start updating the Pod on the next Node until the Pod on the previous node is ready and available. This way, only one Node is affected if the new version of the workload running in the new Pod can't be started.

If you want the Pods to update at a higher rate, increase the `maxUnavailable` parameter. If you set it to a value higher than the number of Nodes in your cluster, the daemon Pods will be updated on all Nodes simultaneously, like the Recreate strategy in Deployments.

Tip

To implement the Recreate update strategy in a DaemonSet, set the `maxSurge` parameter to 0 and `maxUnavailable` to 10000 or more, so that this value is always higher than the number of Nodes in your cluster.

An important caveat to rolling DaemonSet updates is that if the readiness probe of an existing daemon Pod fails, the DaemonSet controller immediately deletes the Pod and replaces it with a Pod with the updated template. In this case, the `maxSurge` and `maxUnavailable` parameters are ignored.

Likewise, if you delete an existing Pod during a rolling update, it's replaced with a new Pod. The same thing happens if you configure the DaemonSet with the `onDelete` update strategy. Let's take a quick look at this strategy as well.

The OnDelete update strategy

An alternative to the `RollingUpdate` strategy is `onDelete`. As you know from the previous chapter on StatefulSets, this is a semi-automatic strategy that allows you to work with the DaemonSet controller to replace the Pods at your discretion, as shown in the next exercise. The following listing shows the contents of the manifest file `ds.demo.v3.onDelete.yaml`.

Listing 16.4 Setting the DaemonSet update strategy

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: demo
spec:
  updateStrategy:      #A
    type: OnDelete     #A
  selector:
```

```

    matchLabels:
      app: demo
  template:
    metadata:
      labels:
        app: demo
        ver: v3      #B
    spec:
      ...

```

The `onDelete` strategy has no parameters you can set to affect how it works, since the controller only updates the Pods you manually delete. Apply this manifest file with `kubectl apply` and then check the DaemonSet as follows to see that no action is taken by the DaemonSet controller:

```
$ kubectl get ds
```

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE
demo	2	2	2	0	2	<none

The output of the `kubectl get ds` command shows that neither Pod in this DaemonSet is up to date. This is to be expected since you updated the Pod template in the DaemonSet, but the Pods haven't yet been updated, as you can see when you list them:

```
$ kubectl get pods -l app=demo -L ver
```

NAME	READY	STATUS	RESTARTS	AGE	VER	
demo-k2d6k	1/1	Running	0	10m	v2	#A
demo-s7hsc	1/1	Running	0	10m	v2	#A

To update the Pods, you must delete them manually. You can delete as many Pod as you want and in any order, but let's delete only one for now. Select a Pod and delete it as follows:

```
$ kubectl delete po demo-k2d6k --wait=false      #A
pod "demo-k2d6k" deleted
```

You may recall that, by default, the `kubectl delete` command doesn't exit until the deletion of the object is complete. If you use the `--wait=false` option, the command marks the object for deletion and exits without waiting for the Pod to actually be deleted. This way, you can keep track of what happens behind the scenes by listing Pods several times as follows:

```
$ kubectl get pods -l app=demo -L ver
```

NAME	READY	STATUS	RESTARTS	AGE	VER	
demo-k2d6k	1/1	Terminating	0	10m	v2	#A
demo-s7hsc	1/1	Running	0	10m	v2	#A

```
$ kubectl get pods -l app=demo -L ver
```

NAME	READY	STATUS	RESTARTS	AGE	VER	
demo-4gf5h	1/1	Running	0	15s	v3	#B
demo-s7hsc	1/1	Running	0	11m	v2	#B

If you list the DaemonSets with the `kubectl get` command as follows, you'll see that only one Pod has been updated:

```
$ kubectl get ds
```

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE
demo	2	2	2	1	2	<none

Delete the remaining Pod(s) to complete the update.

Considering the use of the OnDelete strategy for critical daemon Pods

With this strategy, you can update cluster-critical Pods with much more control, albeit with more effort. This way, you can be sure that the update won't break your entire cluster, as might happen with a fully automated update if the readiness probe in the daemon Pod can't detect all possible problems.

For example, the readiness probe defined in the DaemonSet probably doesn't check if the other Pods on the same Node are still working properly. If the updated daemon Pod is ready for `minReadySeconds`, the controller will proceed with the update on the next Node, even if the update on the first Node caused all other Pods on the Node to fail. The cascade of failures could bring down your entire cluster. However, if you perform the update using the `OnDelete` strategy, you can verify the operation of the other Pods after updating each daemon Pod and before deleting the next one.

16.1.5 Deleting the DaemonSet

To finish this introduction to DaemonSets, delete the demo DaemonSet as follows:

```
$ kubectl delete ds demo
daemonset.apps "demo" deleted
```

As you'd expect, doing so will also delete all demo Pods. To confirm, list the Pods as follows:

```
$ kubectl get pods -l app=demo
```

NAME	READY	STATUS	RESTARTS	AGE
demo-4gf5h	1/1	Terminating	0	2m22s
demo-s7hsc	1/1	Terminating	0	6m53s

This concludes the explanation of DaemonSets themselves, but Pods deployed via DaemonSets differ from Pods deployed via Deployments and StatefulSets in that they often access the host node's file system, its network interface(s), or other hardware. You'll learn about this in the next section.

16.2 Special features in Pods running node agents and daemons

Unlike regular workloads, which are usually isolated from the node they run on, node agents and daemons typically require greater access to the node. As you know, the containers running in a Pod can't access the devices and files of the node, or all the system calls to the node's kernel because they live in their own Linux namespaces (see chapter 2). If you want a daemon, agent, or other workload running in a Pod to be exempt from this restriction, you must specify this in the Pod manifest.

To explain how you can do this, look at the DaemonSets in the kube-system namespace. If you run Kubernetes via kind, your cluster should contain the two DaemonSets as follows:

```
$ kubectl get ds -n kube-system
```

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE
kindnet	3	3	3	3	3
kube-proxy	3	3	3	3	3

If you don't use kind, the list of DaemonSets in kube-system may look different, but you should find the kube-proxy DaemonSet in most clusters, so I'll focus on this one.

16.2.1 Giving containers access to the OS kernel

The operating system kernel provides system calls that programs can use to interact with the operating system and hardware. Some of these calls are harmless, while others could negatively affect the operation of the node or the other containers running on it. For this reason, containers are not allowed to execute these calls unless explicitly allowed to do so. This can be achieved in two ways. You can give the container full access to the kernel or to groups of system calls by specifying the capabilities to be given to the container.

Running a privileged container

If you want to give a process running in a container full access to the operating system kernel, you can mark the container as privileged. You can see how to do this by inspecting the Pod template in the kube-proxy DaemonSet as follows:

```
$ kubectl -n kube-system get ds kube-proxy -o yaml
apiVersion: apps/v1
kind: DaemonSet
spec:
  template:
    spec:
      containers:
      - name: kube-proxy
        securityContext:      #A
          privileged: true    #A
      ...
```

The kube-proxy DaemonSet runs Pods with a single container, also called kube-proxy. In the securityContext section of this container's definition, the privileged flag is set to true. This gives the process running in the kube-proxy container root access to the host's kernel and allows it to modify the node's network packet filtering rules. As you'll learn in chapter 19, Kubernetes Services are implemented this way.

Giving a container access to specific capabilities

A privileged container bypasses all kernel permission checks and thus has

full access to the kernel, whereas a node agent or daemon typically only needs access to a subset of the system calls provided by the kernel. From a security perspective, running such workloads as privileged is far from ideal. Instead, you should grant the workload access to only the minimum set of system calls it needs to do its job. You achieve this by specifying the capabilities that it needs in the container definition.

The kube-proxy DaemonSet doesn't use capabilities, but other DaemonSets in the kube-system namespace may do so. An example is the kindnet DaemonSet, which sets up the pod network in a kind-provisioned cluster. The capabilities listed in the Pod template are as follows:

```
$ kubectl -n kube-system get ds kindnet -o yaml
apiVersion: apps/v1
kind: DaemonSet
spec:
  template:
    spec:
      containers:
      - name: kindnet-cni
        securityContext:      #A
          capabilities:      #A
            add:              #A
              - NET_RAW      #A
              - NET_ADMIN    #A
            privileged: false #B
```

Instead of being fully privileged, the capabilities NET_RAW and NET_ADMIN are added to the container. According to the capabilities man pages, which you can display with the `man capabilities` command on a Linux system, the NET_RAW capability allows the container to use special socket types and bind to any address, while the NET_ADMIN capability allows various privileged network-related operations such as interface configuration, firewall management, changing routing tables, and so on. Things you'd expect from a container that sets up the networking for all other Pods on a Node.

16.2.2 Accessing the node's filesystem

A node agent or daemon may need to access the host node's file system. For example, a node agent deployed through a DaemonSet could be used to

install software packages on all cluster nodes.

You already learned in chapter 7 how to give a Pod's container access to the host node's file system via the `hostPath` volume, so I won't go into it again, but it's interesting to see how this volume type is used in the context of a daemon pod.

Let's take another look at the `kube-proxy` DaemonSet. In the Pod template, you'll find two `hostPath` volumes, as shown here:

```
$ kubectl -n kube-system get ds kube-proxy -o yaml
apiVersion: apps/v1
kind: DaemonSet
spec:
  template:
    spec:
      volumes:
      - hostPath:      #A
        path: /run/xtables.lock      #A
        type: FileOrCreate      #A
        name: xtables-lock      #A
      - hostPath:      #B
        path: /lib/modules      #B
        type: ""      #B
        name: lib-modules      #B
```

The first volume allows the process in the `kube-proxy` daemon Pod to access the node's `xtables.lock` file, which is used by the `iptables` or `nftables` tools that the process uses to manipulate the node's IP packet filtering. The other `hostPath` volume allows the process to access the kernel modules that are installed on the node.

16.2.3 Using the node's network and other namespaces

As you know, each Pod gets its own network interface. However, you may want some of your Pods, especially those deployed through a DaemonSet, to use the node's network interface(s) instead of having their own. The Pods deployed through the `kube-proxy` DaemonSet use this approach. You can see this by examining the Pod template as follows:

```
$ kubectl -n kube-system get ds kube-proxy -o yaml
```

```
apiVersion: apps/v1
kind: DaemonSet
spec:
  template:
    spec:
      dnsPolicy: ClusterFirst
      hostNetwork: true      #A
```

In the Pod's spec, the `hostNetwork` field is set to `true`. This causes the Pod to use the host Node's network environment (devices, stacks, and ports) instead of having its own, just like all other processes that run directly on the node and not in a container. This means that the Pod won't even get its own IP address but will use the Node's address(es). If you list the Pods in the `kube-system` Namespace with the `-o wide` option as follows, you'll see that the IPs of the `kube-proxy` Pods match the IPs of their respective host Nodes.

```
$ kubectl -n kube-system get po -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP
kube-proxy-gj9pd	1/1	Running	0	90m	172.18.0.4
kube-proxy-rhjqr	1/1	Running	0	90m	172.18.0.2
kube-proxy-vq5g8	1/1	Running	0	90m	172.18.0.3

Configuring daemon Pods to use the host node's network is useful when the process running in the Pod needs to be accessible through a network port at the node's IP address.

Note

Another option is for the Pod to use its own network, but forward one or more host ports to the container by using the `hostPort` field in the container's port list. You'll learn how to do this later.

Containers in a Pod configured with `hostNetwork: true` continue to use the other namespace types, so they remain isolated from the node in other respects. For example, they use their own IPC and PID namespaces, so they can't see the other processes or communicate with them via inter-process communication. If you want a daemon Pod to use the node's IPC and PID namespaces, you can configure this using the `hostIPC` and `hostPID` properties in the Pod's spec.

16.2.4 Marking daemon Pods as critical

A node can run a few system Pods and many Pods with regular workloads. You don't want Kubernetes to treat these two groups of Pods the same, as the system Pods are probably more important than the non-system Pods. For example, if a system Pod can't be scheduled to a Node because the Node is already full, Kubernetes should evict some of the non-system Pods to make room for the system Pod.

Introducing Priority Classes

By default, Pods deployed via a DaemonSet are no more important than Pods deployed via Deployments or StatefulSets. To mark your daemon Pods as more or less important, you use Pod priority classes. These are represented by the PriorityClass object. You can list them as follows:

```
$ kubectl get priorityclasses
```

NAME	VALUE	GLOBAL-DEFAULT	AGE
system-cluster-critical	2000000000	false	9h
system-node-critical	2000001000	false	9h

Each cluster usually comes with two priority classes: `system-cluster-critical` and `system-node-critical`, but you can also create your own. As the name implies, Pods in the `system-cluster-critical` class are critical to the operation of the cluster. Pods in the `system-node-critical` class are critical to the operation of individual nodes, meaning they can't be moved to a different node.

You can learn more about the priority classes defined in your cluster by using the `kubectl describe priorityclasses` command as follows:

```
$ kubectl describe priorityclasses
```

Name:	system-cluster-critical
Value:	2000000000
GlobalDefault:	false
Description:	Used for system critical pods that must run in th
Annotations:	<none>
Events:	<none>

Name:	system-node-critical
-------	----------------------

```
Value:          2000001000
GlobalDefault:  false
Description:    Used for system critical pods that must not be mo
Annotations:    <none>
Events:        <none>
```

As you can see, each priority class has a value. The higher the value, the higher the priority. The preemption policy in each class determines whether or not Pods with lower priority should be evicted when a Pod with that class is scheduled to an overbooked Node.

You specify which priority class a Pod belongs to by specifying the class name in the `priorityClassName` field of the Pod's `spec` section. For example, the kube-proxy DaemonSet sets the priority class of its Pods to `system-node-critical`. You can see this as follows:

```
$ kubectl -n kube-system get ds kube-proxy -o yaml
apiVersion: apps/v1
kind: DaemonSet
spec:
  template:
    spec:
      priorityClassName: system-node-critical    #A
```

The priority class of the kube-proxy Pods ensures that the kube-proxy Pods have a higher priority than the other Pods, since a node can't function properly without a kube-proxy Pod (Pods on the Node can't use Kubernetes Services).

When you create your own DaemonSets to run other node agents that are critical to the operation of a node, remember to set the `priorityClassName` appropriately.

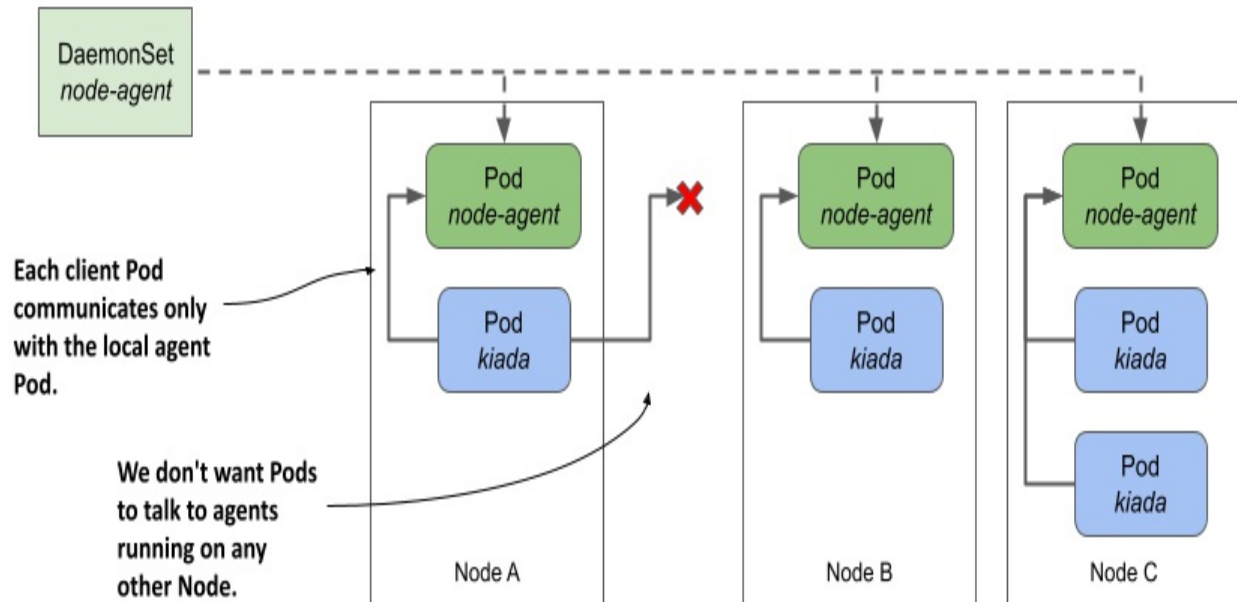
16.3 Communicating with the local daemon Pod

A daemon Pod often provides a service to the other Pods running on the same node. The workloads running in these Pods must connect to the locally running daemon, not one running on another node. In chapter 11, you learned that Pods communicate via Services. However, when a Service receives traffic from a client Pod, it forwards it to a random Pod that may or may not

be running on the same Node as the client.

How do you ensure that a Pod always connects to a daemon Pod running on the same Node, as shown in the next figure? In this section, you'll learn several ways to do that.

Figure 16.4 How do we get client pods to only talk to the locally-running daemon Pod?



In the following examples, you'll use a demo node agent written in Go that allows clients to retrieve system information such as uptime and average node utilization over HTTP. This allows Pods like Kiada to retrieve information from the agent instead of retrieving it directly from the node.

The source code for the node agent can be found in the `Chapter16/node-agent-0.1/` directory. Either build the container image yourself or use the prebuilt image at `luksa/node-agent:0.1`.

In `Chapter16/kiada-0.9` you'll find version 0.9 of the Kiada application. This version connects to the node agent, retrieves the node information, and displays it along with the other pod and node information that was displayed in earlier versions.

16.3.1 Binding directly to a host port

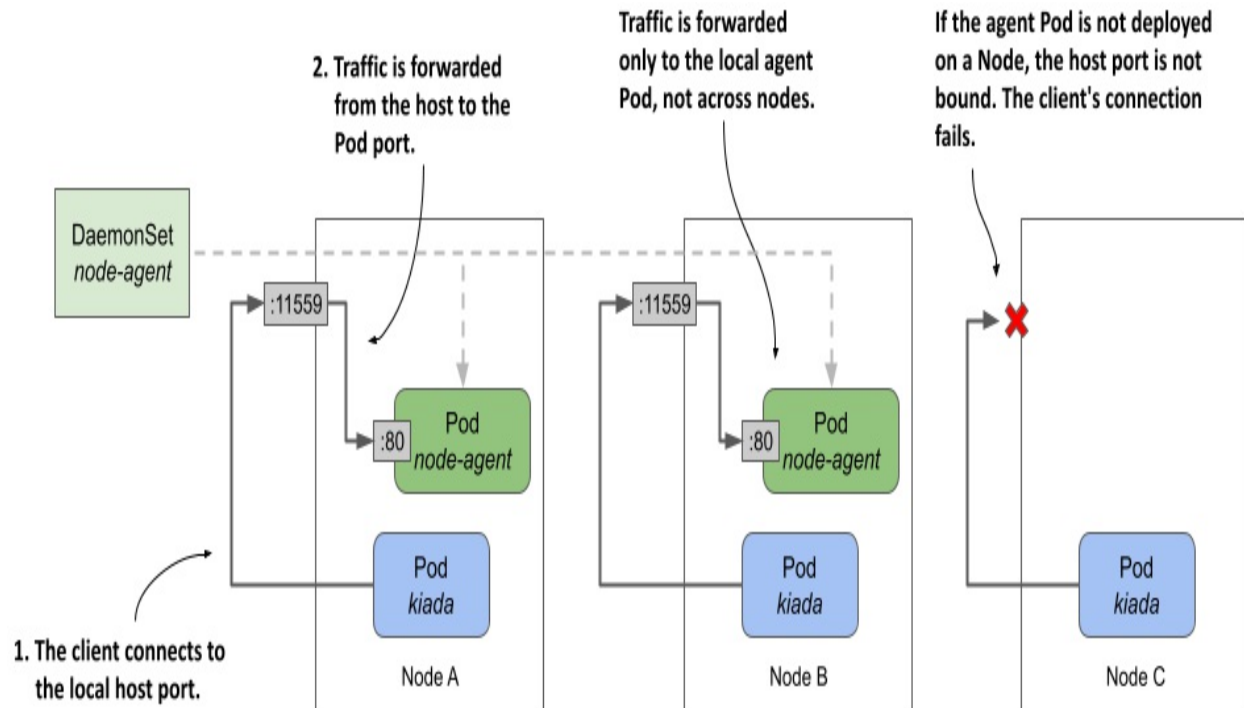
One way to ensure that clients can connect to the local daemon Pod on a given Node is to forward a network port on the host node to a port on the daemon Pod and configure the client to connect to it. To do this, you specify the desired port number of the host node in the list of ports in the Pod manifest using the `hostPort` field, as shown in the following listing. You can find this example in the file `ds.node-agent.hostPort.yaml`.

Listing 16.5 Forwarding a host port to a container

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-agent
  ...
spec:
  template:
    spec:
      containers:
      - name: node-agent
        image: luksa/node-agent:0.1
        args:      #B
        - --listen-address      #B
        - :80      #B
        ...
        ports:      #A
        - name: http
          containerPort: 80      #B
          hostPort: 11559      #C
```

The manifest defines a DaemonSet that deploys node agent Pods listening on port 80 of the Pod's network interface. However, in the list of ports, the container's port 80 is also accessible through port 11559 of the host Node. The process in the container binds only to port 80, but Kubernetes ensures that traffic received by the host Node on port 11559 is forwarded to port 80 within the node-agent container, as shown in the following figure.

Figure 16.5 Exposing a daemon Pod via a host port



As you can see in the figure, each Node forwards traffic from the host port only to the local agent Pod. This is different from the NodePort Service explained in chapter 11, where a client connection to the node port is forwarded to a random Pod in the cluster, possibly one running on another Node. It also means that if no agent Pod is deployed on a Node, the attempt to connect to the host port will fail.

Deploying the agent and checking its connectivity

Deploy the node-agent DaemonSet by applying the `ds.node-agent.hostPort.yaml` manifest. Verify that the number of Pods matches the number of Nodes in your cluster and that all Pods are running.

Check if the node agent Pod responds to requests. Select one of the Nodes, find its IP address, and send a `GET /` request to its port 11559. For example, if you're using `kind` to provision your cluster, you can find the IP of the `kind-worker` node as follows:

```
$ kubectl get node kind-worker -o wide
NAME           STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXT
kind-worker    Ready    <none>   26m   v1.23.4   172.18.0.2    <no
```


In my case, the IP of the Node is 172.18.0.2. To send the GET request, I run `curl` as follows:

```
$ curl 172.18.0.2:11559
kind-worker uptime: 5h58m10s, load average: 1.62, 1.83, 2.25, act
```

If access to the Node is obstructed by a firewall, you may need to connect to the Node via SSH and access the port via `localhost`, as follows:

```
root@kind-worker:/# curl localhost:11559
kind-worker uptime: 5h59m20s, load average: 1.53, 1.77, 2.20, act
```

The HTTP response shows that the node-agent Pod is working. You can now deploy the Kiada app and let it connect to the agent. But how do you tell Kiada where to find the local node-agent Pod?

Pointing the Kiada application to the agent via the Node's IP address

Kiada searches for the node agent URL using the environment variable `NODE_AGENT_URL`. For the application to connect to the local agent, you must pass the IP of the host node and port 11559 in this variable. Of course, this IP depends on which Node the individual Kiada Pod is scheduled, so you can't just specify a fixed IP address in the Pod manifest. Instead, you use the Downward API to get the local Node IP, as you learned in chapter 9. The following listing shows the part of the `deploy.kiada.0.9.hostPort.yaml` manifest where the `NODE_AGENT_URL` environment variable is set.

Listing 16.6 Using the DownwardAPI to set the `NODE_AGENT_URL` variable

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kiada
spec:
  template:
    spec:
      containers:
      - name: kiada
        image: luksa/kiada:0.9
        imagePullPolicy: Always
        env:
```

```

...
- name: NODE_IP      #A
  valueFrom:         #A
    fieldRef:        #A
      fieldPath: status.hostIP      #A
- name: NODE_AGENT_URL      #B
  value: http://$(NODE_IP):11559      #B
...

```

As you can see in the listing, the environment variable `NODE_AGENT_URL` references the variable `NODE_IP`, which is initialized via the Downward API. The host port 11559 that the agent is bound to is hardcoded.

Apply the `deploy.kiada.0.9.hostPort.yaml` manifest and call the Kiada application to see if it retrieves and displays the node information from the local node agent, as shown here:

```

$ curl http://kiada.example.com
...
Request processed by Kiada 0.9 running in pod "kiada-68fbb5fcb9-r
...
Node info: kind-worker2 uptime: 6h17m48s, load average: 0.87, 1.2
           active/total threads: 5/4283      #A
...

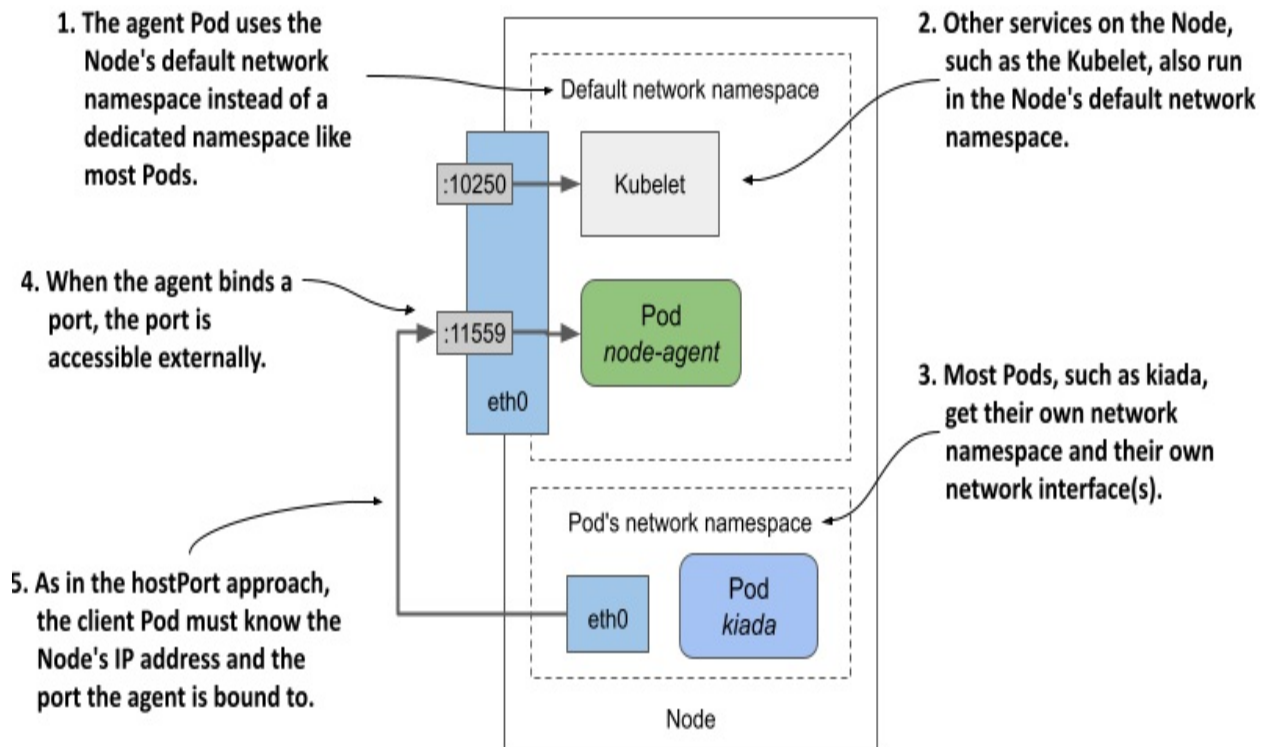
```

The response shows that the request was processed by a Kiada Pod running on the node `kind-worker2`. The `Node info` line indicates that the node information was retrieved from the agent on the same node. Every time you press refresh in your browser or run the `curl` command, the node name in the `Node info` line should always match the node in the `Request processed by` line. This shows that each Kiada pod gets the node information from its local agent and never from an agent on another node.

16.3.2 Using the node's network stack

A similar approach to the previous section is for the agent Pod to directly use the Node's network environment instead of having its own, as described in section 16.2.3. In this case, the agent is reachable through the node's IP address via the port to which it binds. When the agent binds to port 11559, client Pods can connect to the agent through this port on the node's network interface, as shown in the following figure.

Figure 16.6 Exposing a daemon Pod by using the host node's network namespace



The following listing shows the `ds.node-agent.hostNetwork.yaml` manifest, in which the Pod is configured to use the host node's network environment instead of its own. The agent is configured to listen on port 11559.

Listing 16.7 Exposing a node agent by letting the Pod use the host node's network

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-agent
  ...
spec:
  template:
    spec:
      hostNetwork: true    #A
      ...
      containers:
      - name: node-agent
        image: luksa/node-agent:0.1
        imagePullPolicy: Always
```

```

args:
- --listen-address    #B
- :11559              #B
...
ports:                #C
- name: http          #C
  containerPort: 11559 #C
readinessProbe:
  failureThreshold: 1
  httpGet:
    port: 11559
    scheme: HTTP

```

Since the node agent is configured to bind to port 11559 via the `--listen-address` argument, the agent is reachable via this port on the node's network interface(s). From the client's point of view, this is exactly like using the `hostPort` field in the previous section, but from the agent's point of view, it's different because the agent was previously bound to port 80 and traffic from the node's port 11559 was forwarded to the container's port 80, whereas now it's bound directly to port 11559.

Use the `kubectl apply` command to update the `DaemonSet` to see this in action. Since nothing has changed from the client's point of view, the Kiada application you used in the previous section should still be able to get the node information from the agent. You can check this by reloading the application in your browser or making a new request with the `curl` command.

16.3.3 Using a local Service

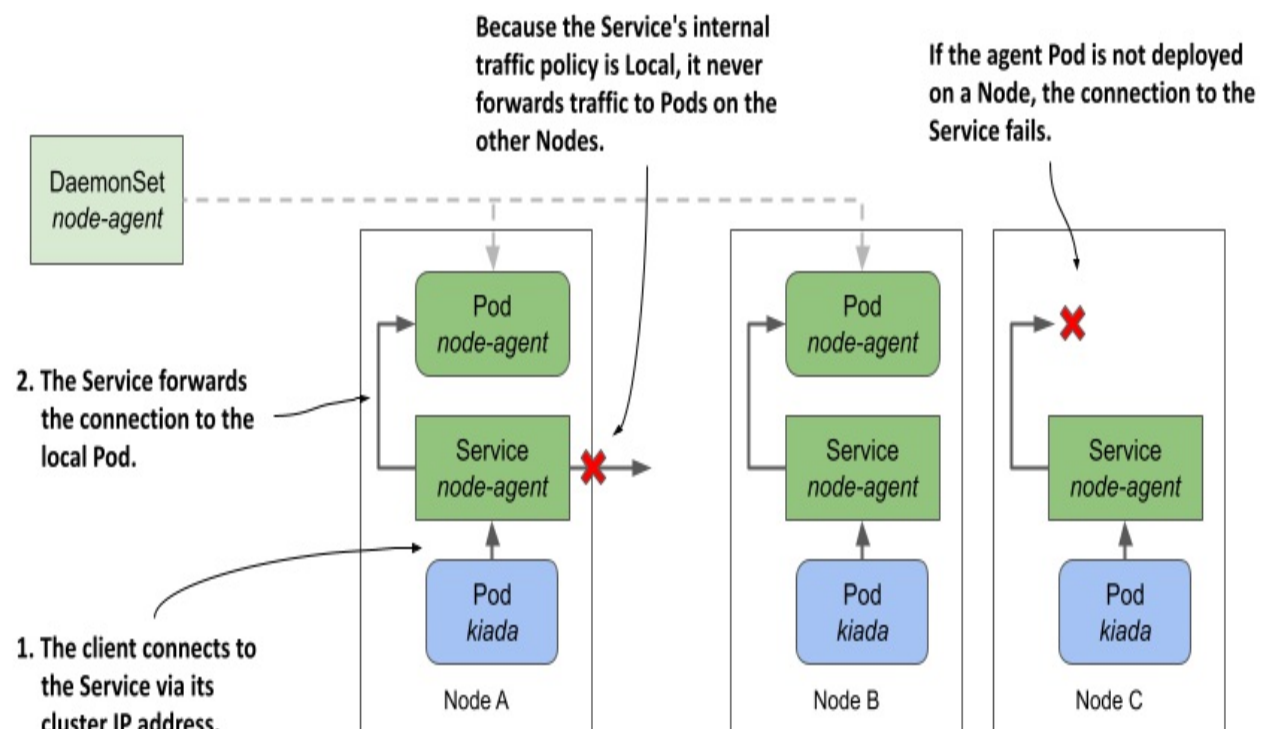
The two approaches to connecting to a local daemon Pod described in the previous sections aren't ideal because they require that the daemon Pod be reachable through the Node's network interface, which means that client pods must look up the Node's IP address. These approaches also don't prevent external clients from accessing the agent.

If you don't want the daemon to be visible to the outside world, or if you want client Pods to access the daemon the same way they access other Pods in the cluster, you can make the daemon Pods accessible through a Kubernetes Service. However, as you know, this results in connections being

forwarded to a random daemon Pod that's not necessarily running on the same Node as the client. Fortunately, as you learned in chapter 11, you can configure a Service to forward traffic only within the same node by setting the `internalTrafficPolicy` in the Service manifest to `Local`.

The following figure shows how this type of Service is used to expose the node-agent Pods so that their clients always connect to the agent running on the same Node as the client.

Figure 16.7 Exposing daemon Pods via a Service with internal traffic policy set to Local



As explained in chapter 11, a Service whose `internalTrafficPolicy` is set to `Local` behaves like multiple per-Node Services, each backed only by the Pods running on that Node. For example, when clients on Node A connect to the Service, the connection is forwarded only to the Pods on Node A. Clients on Node B only connect to Pods on Node B. In the case of the node-agent Service, there's only one such Pod on each Node.

Note

If the DaemonSet through which agent Pods are deployed uses a Node selector, some Nodes may not have an agent running. If a Service with `internalTrafficPolicy` set to `Local` is used to expose the local agent, a client's connection to the Service on that Node will fail.

To try this approach, update your node-agent DaemonSet, create the Service, and configure the Kiada application to use it, as explained next.

Updating the node-agent DaemonSet

In the `ds.node-agent.yaml` file, you'll find a DaemonSet manifest that deploys ordinary Pods that don't use the `hostPort` or `hostNetwork` fields. The agent in the Pod simply binds to port 80 of the container's IP address.

When you apply this manifest to your cluster, the Kiada application can no longer access the node agent because it's no longer bound to port 11559 of the node. To fix this, you need to create a Service called `node-agent` and reconfigure the Kiada application to access the agent through this Service.

Creating the Service with internal traffic policy set to Local

The following listing shows the Service manifest, which you can find in the file `svc.node-agent.yaml`.

Listing 16.8 Exposing daemon Pods via a Service using the Local internal traffic policy

```
apiVersion: v1
kind: Service
metadata:
  name: node-agent
  labels:
    app: node-agent
spec:
  internalTrafficPolicy: Local    #A
  selector:                    #B
    app: node-agent            #B
  ports:                       #C
  - name: http                 #C
    port: 80                   #C
```

The selector in the Service manifest is configured to match Pods with the label `app: node-agent`. This corresponds to the label assigned to agent Pods in the DaemonSet Pod template. Since the Service's `internalTrafficPolicy` is set to `Local`, the Service forwards traffic only to Pods with this label on the same Node. Pods on the other nodes are ignored even if their label matches the selector.

Configuring Kiada to connect to the node-agent Service

Once you've created the Service, you can reconfigure the Kiada application to use it, as shown in the following listing. The full manifest can be found in the `deploy.kiada.0.9.yaml` file.

Listing 16.9 Configuring the Kiada app to access the node agent via the local Service

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kiada
spec:
  template:
    spec:
      containers:
      - name: kiada
        image: luksa/kiada:0.9
        env:
          ...
          - name: NODE_AGENT_URL      #A
            value: http://node-agent  #A
          ...
```

The environment variable `NODE_AGENT_URL` is now set to `http://node-agent`. This is the name of the Service defined in the `svc.node-agent.local.yaml` manifest file earlier.

Apply the Service and the updated Deployment manifest and confirm that each Kiada Pod uses the local agent to display the node information, just as in the previous approaches.

Deciding which approach to use

You may be wondering which of these three approaches to use. The approach described in this section, using a local Service, is the cleanest and least invasive because it doesn't affect the node's network and doesn't require special permissions. Use the `hostPort` or `hostNetwork` approach only if you need to reach the agent from outside the cluster.

If the agent exposes multiple ports, you may think it's easier to use `hostNetwork` instead of `hostPort` so you don't have to forward each port individually, but that's not ideal from a security perspective. If the Pod is configured to use the host network, an attacker can use the Pod to bind to any port on the Node, potentially enabling man-in-the-middle attacks.

16.4 Summary

In this chapter, you learned how to run daemons and node agents. You learned that:

- A `DaemonSet` object represents a set of daemon Pods distributed across the cluster Nodes so that exactly one daemon Pod instance runs on each node.
- A `DaemonSet` is used to deploy daemons and agents that provide system-level services such as log collection, process monitoring, node configuration, and other services required by each cluster Node.
- When you add a node selector to a `DaemonSet`, the daemon Pods are deployed only on a subset of all cluster Nodes.
- A `DaemonSet` doesn't deploy Pods to control plane Nodes unless you configure the Pod to tolerate the Nodes' taints.
- The `DaemonSet` controller ensures that a new daemon Pod is created when a new Node is added to the cluster, and that it's removed when a Node is removed.
- Daemon Pods are updated according to the update strategy specified in the `DaemonSet`. The `RollingUpdate` strategy updates Pods automatically and in a rolling fashion, whereas the `OnDelete` strategy requires you to manually delete each Pod for it to be updated.
- If Pods deployed through a `DaemonSet` require extended access to the Node's resources, such as the file system, network environment, or privileged system calls, you configure this in the Pod template in the

DaemonSet.

- Daemon Pods should generally have a higher priority than Pods deployed via Deployments. This is achieved by setting a higher `PriorityClass` for the Pod.
- Client Pods can communicate with local daemon Pods through a Service with `internalTrafficPolicy` set to `Local`, or through the Node's IP address if the daemon Pod is configured to use the node's network environment (`hostNetwork`) or a host port is forwarded to the Pod (`hostPort`).

In the next chapter, you'll learn how to run batch workloads with the `Job` and `CronJob` object types.

17 Running finite workloads with Jobs and CronJobs

This chapter covers

- Running finite tasks with Jobs
- Handling Job failures
- Parameterizing Pods created through a Job
- Processing items in a work queue
- Enabling communication between a Job's Pods
- Using CronJobs to run Jobs at a specific time or at regular intervals

As you learned in the previous chapters, a Pod created via a Deployment, StatefulSet, or DaemonSet, runs continuously. When the process running in one of the Pod's containers terminates, the Kubelet restarts the container. The Pod never stops on its own, but only when you delete the Pod object. Although this is ideal for running web servers, databases, system services, and similar workloads, it's not suitable for finite workloads that only need to perform a single task.

A finite workload doesn't run continuously, but lets a task run to completion. In Kubernetes, you run this type of workload using the *Job* resource. However, a Job always runs its Pods immediately, so you can't use it for scheduling tasks. For that, you need to wrap the Job in a *CronJob* object. This allows you to schedule the task to run at a specific time in the future or at regular intervals.

In this chapter you'll learn everything about Jobs and CronJobs. Before you begin, create the kiada Namespace, change to the Chapter17/ directory, and apply all the manifests in the SETUP/ directory by running the following commands:

```
$ kubectl create ns kiada
$ kubectl config set-context --current --namespace kiada
$ kubectl apply -f SETUP -R
```

NOTE

You can find the code files for this chapter at <https://github.com/luksa/kubernetes-in-action-2nd-edition/tree/master/Chapter17>.

Don't be alarmed if you find that one of the containers in each quiz Pod fails to become ready. This is to be expected since the MongoDB database running in those Pods hasn't yet been initialized. You'll create a Job resource to do just that.

17.1 Running tasks with the Job resource

Before you create your first Pod via the Job resource, let's think about the Pods in the kiada Namespace. They're all meant to run continuously. When a container in one of these pods terminates, it's automatically restarted. When the Pod is deleted, it's recreated by the controller that created the original Pod. For example, if you delete one of the kiada pods, it's quickly recreated by the Deployment controller because the `replicas` field in the kiada Deployment specifies that three Pods should always exist.

Now consider a Pod whose job is to initialize the MongoDB database. You don't want it to run continuously; you want it to perform one task and then exit. Although you want the Pod's containers to restart if they fail, you don't want them to restart when they finish successfully. You also don't want a new Pod to be created after you delete the Pod that completed its task.

You may recall that you already created such a Pod in chapter 15, namely the `quiz-data-importer` Pod. It was configured with the `OnFailure` restart policy to ensure that the container would restart only if it failed. When the container terminated successfully, the Pod was finished, and you could delete it. Since you created this Pod directly and not through a Deployment, StatefulSet or DaemonSet, it wasn't recreated. So, what's wrong with this approach and why would you create the Pod via a Job instead?

To answer this question, consider what happens if someone accidentally deletes the Pod prematurely or if the Node running the Pod fails. In these

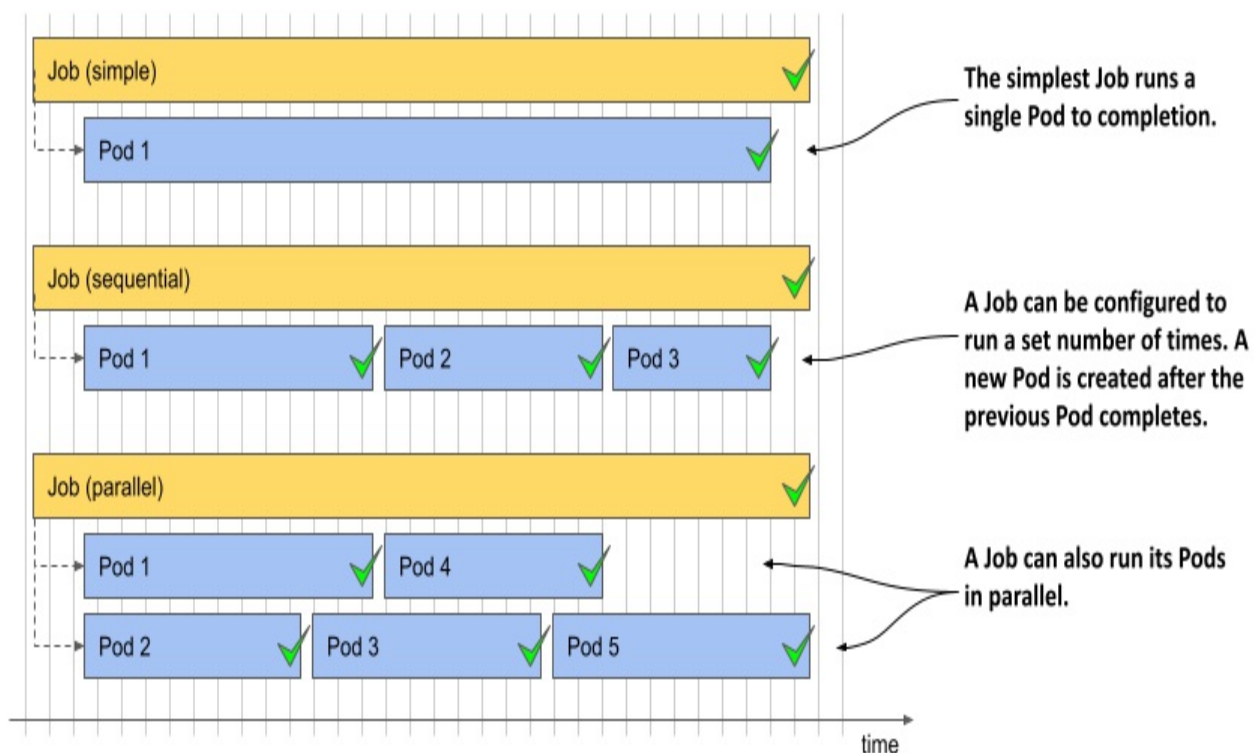
cases, Kubernetes wouldn't automatically recreate the Pod. You'd have to do that yourself. And you'd have to watch that Pod from creation to completion. That might be fine for a Pod that completes its task in seconds, but you probably don't want to be stuck watching a Pod for hours. So, it's better to create a Job object and let Kubernetes do the rest.

17.1.1 Introducing the Job resource

The Job resource resembles a Deployment in that it creates one or more Pods, but instead of ensuring that those Pods run indefinitely, it only ensures that a certain number of them complete successfully.

As you can see in the following figure, the simplest Job runs a single Pod to completion, whereas more complex Jobs run multiple Pods, either sequentially or concurrently. When all containers in a Pod terminate with success, the Pod is considered completed. When all the Pods have completed, the Job itself is also completed.

Figure 17.1 Three different Job examples. Each Job is completed once its Pods have completed successfully.



As you might expect, a Job resource defines a Pod template and the number of Pods that must be successfully completed. It also defines the number of Pods that may run in parallel.

Note

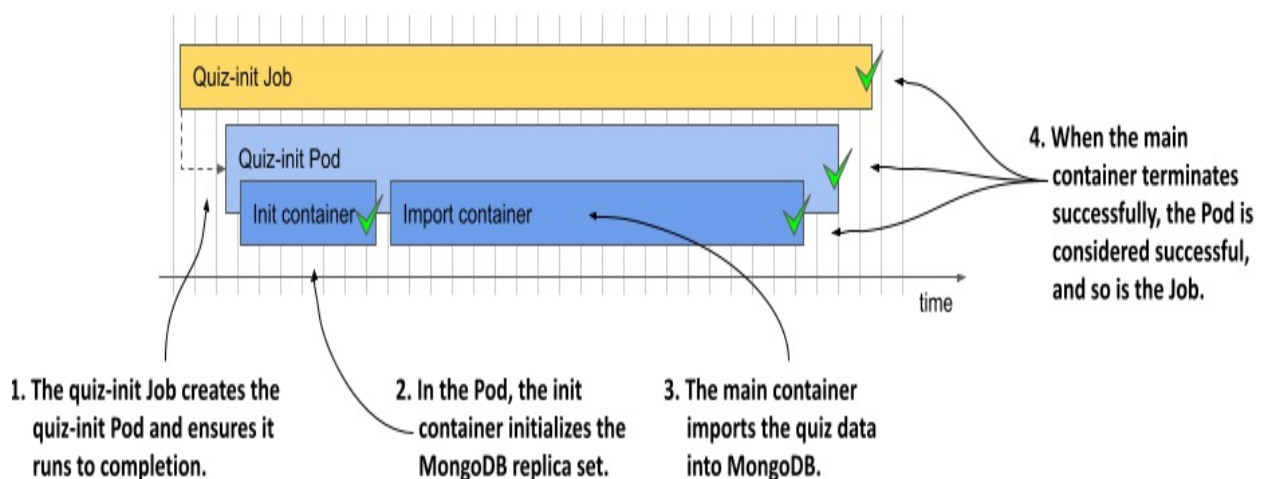
Unlike Deployments and other resources that contain a Pod template, you can't modify the template in a Job object after creating the object.

Let's look at what the simplest Job object looks like.

Defining a Job resource

In this section, you take the `quiz-data-importer` Pod from chapter 15 and turn it into a Job. This Pod imports the data into the Quiz MongoDB database. You may recall that before running this Pod, you had to initiate the MongoDB replica set by issuing a command in one of the quiz Pods. You can do that in this Job as well, using an `init` container. The Job and the Pod it creates are visualized in the following figure.

Figure 17.2 An overview of the `quiz-init` Job



The following listing shows the Job manifest, which you can find in the file `job.quiz-init.yaml`.

Note

The manifest file also contains a ConfigMap in which the quiz questions are stored but this ConfigMap is not shown in the listing.

Listing 17.1 A Job manifest for running a single task

```
apiVersion: batch/v1      #A
kind: Job      #A
metadata:
  name: quiz-init
  labels:
    app: quiz
    task: init
spec:
  template:      #B
    metadata:      #C
    labels:      #C
      app: quiz      #C
      task: init      #C
    spec:
      restartPolicy: OnFailure      #D
      initContainers:      #E
      - name: init      #E
        image: mongo:5      #E
        command:      #E
          - sh      #E
          - -c      #E
          - |      #E
            mongosh mongodb://quiz-0.quiz-pods.kiada.svc.cluster.lo
--quiet --file /dev/stdin <<EOF      #E
#E
      # MongoDB code that initializes the replica set      #E
      # Refer to the job.quiz-init.yaml file to see the actua
#E
      EOF      #E
    containers:      #F
      - name: import      #F
        image: mongo:5      #F
        command:      #F
          - mongoimport      #F
          - mongodb+srv://quiz-pods.kiada.svc.cluster.local/kiada?t
          - --collection      #F
          - questions      #F
          - --file      #F
          - /questions.json      #F
          - --drop      #F
        volumeMounts:      #F
```

```

- name: quiz-data      #F
  mountPath: /questions.json    #F
  subPath: questions.json      #F
  readOnly: true      #F
volumes:
- name: quiz-data
  configMap:
    name: quiz-data

```

The manifest in the listing defines a Job object that runs a single Pod to completion. Jobs belong to the batch API group, and you're using API version v1 to define the object. The Pod that this Job creates consists of two containers that execute in sequence, as one is an init and the other a normal container. The init container makes sure that the MongoDB replica set is initialized, then the main container imports the quiz questions from the quiz-data ConfigMap that's mounted into the container through a volume.

The Pod's `restartPolicy` is set to `OnFailure`. A Pod defined within a Job can't use the default policy of `Always`, as that would prevent the Pod from completing.

Note

In a Job's pod template, you must explicitly set the restart policy to either `OnFailure` or `Never`.

You'll notice that unlike Deployments, the Job manifest in the listing doesn't define a selector. While you can specify it, you don't have to, as Kubernetes sets it automatically. The Pod template in the listing does contain two labels, but they're there only for your convenience.

Running a Job

The Job controller creates the Pods immediately after you create the Job object. To run the quiz-init Job, apply the `job.quiz-init.yaml` manifest with `kubectl apply`.

Displaying a brief Job status

To get a brief overview of the Job's status, list the Jobs in the current Namespace as follows:

```
$ kubectl get jobs
NAME          COMPLETIONS  DURATION  AGE
quiz-init     0/1           3s        3s
```

The COMPLETIONS column indicates how many times the Job has run and how many times it's configured to complete. The DURATION column shows how long the Job has been running. Since the task the quiz-init Job performs is relatively short, its status should change within a few seconds. List the Jobs again to confirm this:

```
$ kubectl get jobs
NAME          COMPLETIONS  DURATION  AGE
quiz-init     1/1           6s        42s
```

The output shows that the Job is now complete, which took 6 seconds.

Displaying the detailed Job status

To see more details about the Job, use the `kubectl describe` command as follows:

```
$ kubectl describe job quiz-init
Name:          quiz-init
Namespace:     kiada
Selector:      controller-uid=98f0fe52-12ec-4c76-a185-4ccee9ba
Labels:        app=quiz
               task=init
Annotations:   batch.kubernetes.io/job-tracking:
Parallelism:   1
Completions:   1
Completion Mode: NonIndexed
Start Time:    Sun, 02 Oct 2022 12:17:59 +0200
Completed At:  Sun, 02 Oct 2022 12:18:05 +0200
Duration:      6s
Pods Statuses: 0 Active / 1 Succeeded / 0 Failed    #B
Pod Template:
  Labels:  app=quiz      #C
          controller-uid=98f0fe52-12ec-4c76-a185-4ccee9bae1ef
          job-name=quiz-init    #C
          task=init      #C
```



```

Init Containers:
  init: ...
Containers:
  import: ...
Volumes:
  quiz-data: ...
Events:
  Type            Reason              Age   From                  Message
  ----            -
  Normal          SuccessfulCreate    7m33s  job-controller        Created pod: q
  Normal          Completed           7m27s  job-controller        Job completed

```

In addition to the Job name, namespace, labels, annotations, and other properties, the output of the `kubectl describe` command also shows the selector that was automatically assigned. The `controller-uid` label used in the selector was also automatically added to the Job's Pod template. The `job-name` label was also added to the template. As you'll see in the next section, you can easily use this label to list the Pods that belong to a particular Job.

At the end of the `kubectl describe` output, you see the Events associated with this Job object. Only two events were generated for this Job: the creation of the Pod and the successful completion of the Job.

Examining the Pods that belong to a Job

To list the Pods created for a particular Job, you can use the `job-name` label that's automatically added to those Pods. To list the Pods of the `quiz-init` job, run the following command:

```

$ kubectl get pods -l job-name=quiz-init
NAME                READY   STATUS    RESTARTS   AGE
quiz-init-xpl8d     0/1     Completed 0           25m

```

The pod shown in the output has finished its task. The Job controller doesn't delete the Pod, so you can see its status and view its logs.

Examining the logs of a Job Pod

The fastest way to see the logs of a Job is to pass the Job name instead of the Pod name to the `kubectl logs` command. To see the logs of the `quiz-init`

Job, you could do something like the following:

```
$ kubectl logs job/quiz-init --all-containers --prefix #A
[pod/quiz-init-xpl8d/init] Replica set initialized successfully!
[pod/quiz-init-xpl8d/import] 2022-10-02T10:51:01.967+0000 connec
[pod/quiz-init-xpl8d/import] 2022-10-02T10:51:01.969+0000 droppi
[pod/quiz-init-xpl8d/import] 2022-10-02T10:51:03.811+0000 6 docu
```

The `--all-containers` option tells `kubectl` to print the logs of all the Pod's containers, and the `--prefix` option ensures that each line is prefixed with the source, that is, the pod and container names.

The output contains both the `init` and the `import` container logs. These logs indicate that the MongoDB replica set has been successfully initialized and that the question database has been populated with data.

Suspending active Jobs and creating Jobs in a suspended state

When you created the `quiz-init` Job, the Job controller created the Pod as soon as you created the Job object. However, you can also create Jobs in a suspended state. Let's try this out by creating another Job. As you can see in the following listing, you suspend it by setting the `suspend` field to `true`. You can find this manifest in the file `job.demo-suspend.yaml`.

Listing 17.2 The manifest of a suspended Job

```
apiVersion: batch/v1
kind: Job
metadata:
  name: demo-suspend
spec:
  suspend: true      #A
  template:
    spec:
      restartPolicy: OnFailure
      containers:
      - name: demo
        image: busybox
        command:
        - sleep
        - "60"
```

Apply the manifest in the listing to create the Job. List the Pods as follows to confirm that none have been created yet:

```
$ kubectl get po -l job-name=demo-suspend
No resources found in kiada namespace.
```

The Job controller generates an Event indicating the suspension of the Job. You can see it when you run `kubectl get events` or when you describe the Job with `kubectl describe`:

```
$ kubectl describe job demo-suspend
...
Events:
  Type      Reason      Age      From          Message
  ----      -
  Normal    Suspended   3m37s    job-controller  Job suspended
```

When you're ready to run the Job, you unsuspend it by patching the object as follows:

```
$ kubectl patch job demo-suspend -p '{"spec":{"suspend": false}}'
job.batch/demo-suspend patched
```

The Job controller creates the Pod and generates an Event indicating that the Job has resumed.

You can also suspend a running Job, whether you created it in a suspended state or not. To suspend a Job, set `suspend` to `true` with the following `kubectl patch` command:

```
$ kubectl patch job demo-suspend -p '{"spec":{"suspend": true}}'
job.batch/demo-suspend patched
```

The Job controller immediately deletes the Pod associated with the Job and generates an Event indicating that the Job has been suspended. The Pod's containers are shut down gracefully, as they are every time you delete a Pod, regardless of how it was created. You can resume the Job at your discretion by resetting the `suspend` field to `false`.

Deleting Jobs and their Pods

You can delete a Job any time. Regardless of whether its Pods are still running or not, they're deleted in the same way as when you delete a Deployment, StatefulSet, or DaemonSet.

You don't need the quiz-init Job anymore, so delete it as follows:

```
$ kubectl delete job quiz-init
job.batch "quiz-init" deleted
```

Confirm that the Pod has also been deleted by listing the Pods as follows:

```
$ kubectl get po -l job-name=quiz-init
No resources found in kiada namespace.
```

You may recall that Pods are deleted by the garbage collector because they're orphaned when their owner, in this case the Job object named quiz-init, is deleted. If you want to delete only the Job, but keep the Pods, you delete the Job with the `--cascade=orphan` option. You can try this method with the demo-suspend Job as follows:

```
$ kubectl delete job demo-suspend --cascade=orphan
job.batch "demo-suspend" deleted
```

If you now list Pods, you'll see that the Pod still exists. Since it's now a standalone Pod, it's up to you to delete it when you no longer need it.

Automatically deleting Jobs

By default, you must delete Job objects manually. However, you can flag the Job for automatic deletion by setting the `ttlSecondsAfterFinished` field in the Job's spec. As the name implies, this field specifies how long the Job and its Pods are kept after the Job is finished.

To see this setting in action, try creating the Job in the `job.demo-ttl.yaml` manifest. The Job will run a single Pod that will complete successfully after 20 seconds. Since `ttlSecondsAfterFinished` is set to 10, the Job and its Pod are deleted ten seconds later.

Warning

If you set the `ttlSecondsAfterFinished` field in a Job, the Job and its pods are deleted whether the Job completes successfully or not. If this happens before you can check the logs of the failed Pods, it's hard to determine what caused the Job to fail.

17.1.2 Running a task multiple times

In the previous section, you learned how to execute a task once. However, you can also configure the Job to execute the same task several times, either in parallel or sequentially. This may be necessary because the container running the task can only process a single item, so you need to run the container multiple times to process the entire input, or you may simply want to run the processing on multiple cluster nodes to improve performance.

You'll now create a Job that inserts fake responses into the Quiz database, simulating a large number of users. Instead of having only one Pod that inserts data into the database, as in the previous example, you'll configure the Job to create five such Pods. However, instead of running all five Pods simultaneously, you'll configure the Job to run at most two Pods at a time. The following listing shows the Job manifest. You can find it in the file `job.generate-responses.yaml`.

Listing 17.3 A Job for running a task multiple times

```
apiVersion: batch/v1      #A
kind: Job                 #A
metadata:                  #A
  name: generate-responses #A
  labels:
    app: quiz
spec:
  completions: 5          #B
  parallelism: 2          #C
  template:
    metadata:
      labels:
        app: quiz
    spec:
      restartPolicy: OnFailure
```

```
containers:
- name: mongo
  image: mongo:5
  command:
  ...
```

In addition to the Pod template, the Job manifest in the listing defines two new properties, `completions` and `parallelism`, which are explained next.

Understanding Job completions and parallelism

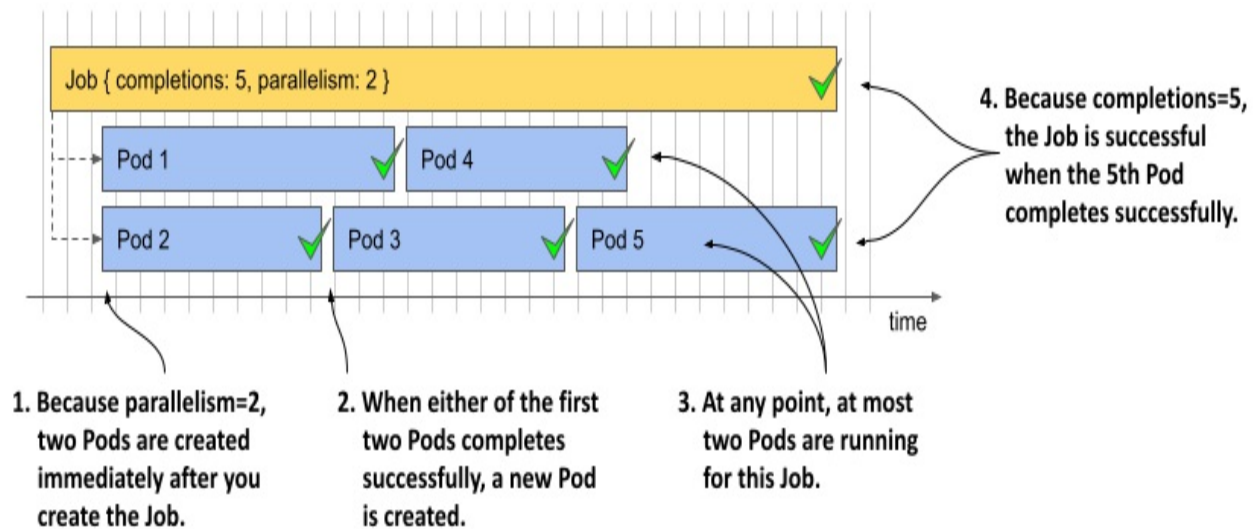
The `completions` field specifies the number of Pods that must be successfully completed for this Job to be complete. The `parallelism` field specifies how many of these Pods may run in parallel. There is no upper limit to these values, but your cluster may only be able to run so many Pods in parallel.

You can set neither of these fields, one or the other, or both. If you don't set either field, both values are set to one by default. If you set only `completions`, this is the number of Pods that run one after the other. If you set only `parallelism`, this is the number of Pods that run, but only one must complete successfully for the Job to be complete.

If you set `parallelism` higher than `completions`, the Job controller creates only as many Pods as you specified in the `completions` field.

If `parallelism` is lower than `completions`, the Job controller runs at most `parallelism` Pods in parallel but creates additional Pods when those first Pods complete. It keeps creating new Pods until the number of successfully completed Pods matches `completions`. The following figure shows what happens when `completions` is 5 and `parallelism` is 2.

Figure 17.3 Running a parallel Job with `completion=5` and `parallelism=2`



As shown in the figure, the Job controller first creates two Pods and waits until one of them completes. In the figure, Pod 2 is the first to finish. The controller immediately creates the next Pod (Pod 3), bringing the number of running Pods back to two. The controller repeats this process until five Pods complete successfully.

The following table explains the behavior for different examples of completions and parallelism.

Table 17.1 Completions and parallelism combinations

Completions	Parallelism	Job behavior
Not set	Not set	A single Pod is created. Same as when completions and parallelism is 1.
1	1	A single Pod is created. If the Pod completes successfully, the Job is complete. If the Pod is deleted before completing, it's replaced by a new Pod.

2	5	Only three Pods are created. The same as if parallelism was 2.
5	2	Two Pods are created initially. When one of them completes, the third Pod is created. There are again two Pods running. When one of the two completes, the fourth Pod is created. There are again two Pods running. When another one completes, the fifth and last Pod is created.
5	5	Five Pods run simultaneously. If one of them is deleted before it completes, a replacement is created. The Job is complete when five Pods complete successfully.
5	Not set	Five Pods are created sequentially. A new Pod is created only when the previous Pod completes (or fails).
Not set	5	Five Pods are created simultaneously, but only one needs to complete successfully for the Job to complete.

In the generate-responses Job that you're about to create, the number of completions is set to 5 and parallelism is set to 2, so at most two Pods will run in parallel. The Job isn't complete until five Pods complete successfully. The total number of Pods may end up being higher if some of the Pods fail. More on this in the next section.

Running the Job

Use `kubectl apply` to create the Job by applying the manifest file

job.generate-responses.yaml. List the Pods while running the Job as follows:

```
$ kubectl get po -l job-name=generate-responses
```

NAME	READY	STATUS	RESTARTS	AGE
generate-responses-7kqw4	1/1	Running	2 (20s ago)	27s
generate-responses-98mh8	0/1	Completed	0	27s
generate-responses-tbgns	1/1	Running	0	3s

List the Pods several times to observe the number Pods whose STATUS is shown as Running or Completed. As you can see, at any given time, at most two Pods run simultaneously. After some time, the Job completes. You can see this by displaying the Job status with the `kubectl get` command as follows:

```
$ kubectl get job generate-responses
```

NAME	COMPLETIONS	DURATION	AGE	
generate-responses	5/5	110s	115s	#A

The COMPLETIONS column shows that this Job completed five out of the desired five times, which took 110 seconds. If you list the Pods again, you should see five completed Pods, as follows:

```
$ kubectl get po -l job-name=generate-responses
```

NAME	READY	STATUS	RESTARTS	AGE	
generate-responses-5xtlk	0/1	Completed	0	82s	#
generate-responses-7kqw4	0/1	Completed	3	2m46s	
generate-responses-98mh8	0/1	Completed	0	2m46s	
generate-responses-tbgns	0/1	Completed	1	2m22s	
generate-responses-vbvq8	0/1	Completed	1	111s	

As indicated in the Job status earlier, you should see five Completed Pods. However, if you look closely at the RESTARTS column, you'll notice that some of these Pods had to be restarted. The reason for this is that I hard-coded a 25% failure rate into the code running in those Pods. I did this to show what happens when an error occurs.

17.1.3 Understanding how Job failures are handled

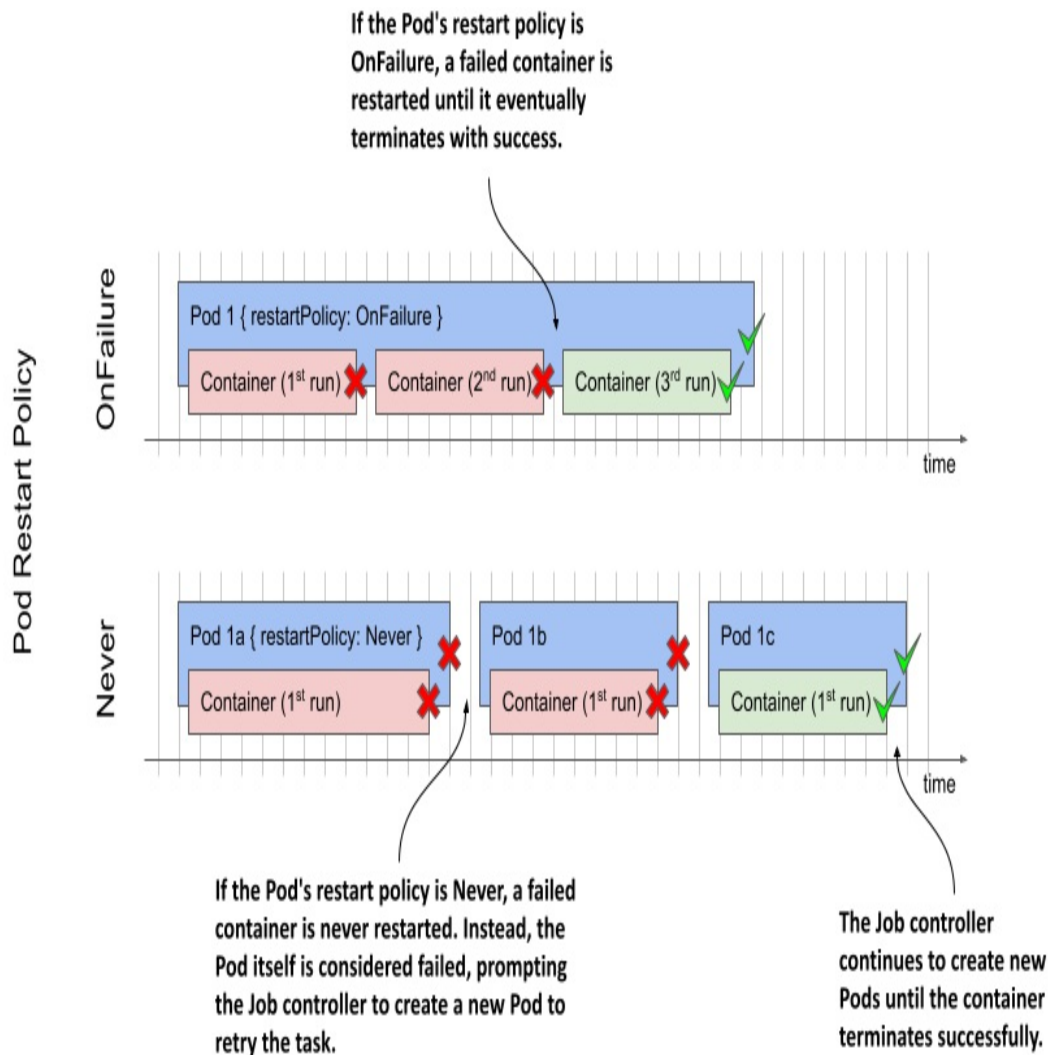
As explained earlier, the reason for running tasks through a Job rather than directly through Pods is that Kubernetes ensures that the task is completed

even if the individual Pods or their Nodes fail. However, there are two levels at which such failures are handled:

- At the Pod level.
- At the Job level.

When a container in the Pod fails, the Pod's `restartPolicy` determines whether the failure is handled at the Pod level by the Kubelet or at the Job level by the Job controller. As you can see in the following figure, if the `restartPolicy` is `OnFailure`, the failed container is restarted within the same Pod. However, if the policy is `Never`, the entire Pod is marked as failed and the Job controller creates a new Pod.

Figure 17.4 How failures are handled depending on the Pod's restart policy



Let's examine the difference between these two scenarios.

Handling failures at the Pod level

In the generate-responses Job you created in the previous section, the Pod's restartPolicy was set to OnFailure. As discussed earlier, whenever the container is executed, there is a 25% chance that it'll fail. In these cases, the container terminates with a non-zero exit code. The Kubelet notices the failure and restarts the container.

The new container runs in the same Pod on the same Node and therefore allows for a quick turnaround. The container may fail again and get restarted several times but will eventually terminate successfully and the Pod will be

marked complete.

Note

As you learned in one of the previous chapters, the Kubelet doesn't restart the container immediately if it crashes multiple times, but adds a delay after each crash and doubles it after each restart.

Handling failures at the Job level

When the Pod template in a Job manifest sets the Pod's `restartPolicy` to `Never`, the Kubelet doesn't restart its containers. Instead, the entire Pod is marked as failed and the Job controller must create a new Pod. This new Pod might be scheduled on a different Node.

Note

If the Pod is scheduled to run on a different Node, the container images may need to be downloaded before the container can run.

If you want to see the Job controller handle the failures in the `generate-responses` Job, delete the existing Job and recreate it from the manifest file `job.generate-responses.restartPolicyNever.yaml`. In this manifest, the Pod's `restartPolicy` is set to `Never`.

The Job completes in about a minute or two. If you list the Pods as follows, you'll notice that it has now taken more than five Pods to get the job done.

```
$ kubectl get po -l job-name=generate-responses
```

NAME	READY	STATUS	RESTARTS	AGE
generate-responses-2dbrn	0/1	Error	0	2m43s
generate-responses-4pckt	0/1	Error	0	2m39s
generate-responses-8c8wz	0/1	Completed	0	2m43s
generate-responses-bnm4t	0/1	Completed	0	3m10s
generate-responses-kn55w	0/1	Completed	0	2m16s
generate-responses-t2r67	0/1	Completed	0	3m10s
generate-responses-xpbnr	0/1	Completed	0	2m34s

You should see five Completed Pods and a few Pods whose status is Error.

The number of those Pods should match the number of successful and failed Pods when you inspect the Job object using the `kubectl describe job` command as follows:

```
$ kubectl describe job generate-responses
...
Pods Statuses:      0 Active / 5 Succeeded / 2 Failed
...
```

Note

It's possible that the number of Pods is different in your case. It's also possible that the Job isn't completed. This is explained in the next section.

To conclude this section, delete the `generate-responses` Job.

Preventing Jobs from failing indefinitely

The two Jobs you created in the previous sections may not have completed because they failed too many times. When that happens, the Job controller gives up. Let's demonstrate this by creating a Job that always fails. You can find the manifest in the file `job.demo-always-fails.yaml`. Its contents are shown in the following listing.

Listing 17.4 A Job that always fails

```
apiVersion: batch/v1
kind: Job
metadata:
  name: demo-always-fails
spec:
  completions: 10
  parallelism: 3
  template:
    spec:
      restartPolicy: OnFailure
      containers:
      - name: demo
        image: busybox
        command:
        - 'false'      #A
```

When you create the Job in this manifest, the Job controller creates three Pods. The container in these Pods terminates with a non-zero exit code, which causes the Kubelet to restart it. After a few restarts, the Job controller notices that these Pods are failing, so it deletes them and marks the Job as failed.

Unfortunately, you won't see that the controller has given up if you simply check the Job status with `kubectl get job`. When you run this command, you only see the following:

```
$ kubectl get job
NAME                COMPLETIONS  DURATION  AGE
demo-always-fails   0/10          2m48s     2m48s
```

The output of the command indicates that the Job has zero completions, but it doesn't indicate whether the controller is still trying to complete the Job or has given up. You can, however, see this in the events associated with the Job. To see the events, run `kubectl describe` as follows:

```
$ kubectl describe job demo-always-fails
...
Events:
Type      Reason              Age    From           Message
----      -
Normal    SuccessfulCreate    5m6s   job-controller Created pod
Normal    SuccessfulCreate    5m6s   job-controller Created pod
Normal    SuccessfulCreate    5m6s   job-controller Created pod
Normal    SuccessfulDelete    4m43s  job-controller Deleted pod
Normal    SuccessfulDelete    4m43s  job-controller Deleted pod
Normal    SuccessfulDelete    4m43s  job-controller Deleted pod
Warning   BackoffLimitExceeded 4m43s  job-controller Job has reached backoff limit
```

The warning event at the bottom indicates that the backoff limit of the Job has been reached, which means that the Job has failed. You can confirm this by checking the Job status as follows:

```
$ kubectl get job demo-always-fails -o yaml
...
status:
  conditions:
  - lastProbeTime: "2022-10-02T15:42:39Z"
    lastTransitionTime: "2022-10-02T15:42:39Z"
```

```
    message: Job has reached the specified backoff limit    #A
    reason: BackoffLimitExceeded    #A
    status: "True"    #B
    type: Failed    #B
failed: 3
startTime: "2022-10-02T15:42:16Z"
uncountedTerminatedPods: {}
```

It's almost impossible to see this, but the Job ended after 6 retries, which is the default backoff limit. You can set this limit for each Job in the `spec.backoffLimit` field in its manifest.

Once a Job exceeds this limit, the Job controller deletes all running Pods and no longer creates new Pods for it. To restart a failed Job, you must delete and recreate it.

Limiting the time allowed for a Job to complete

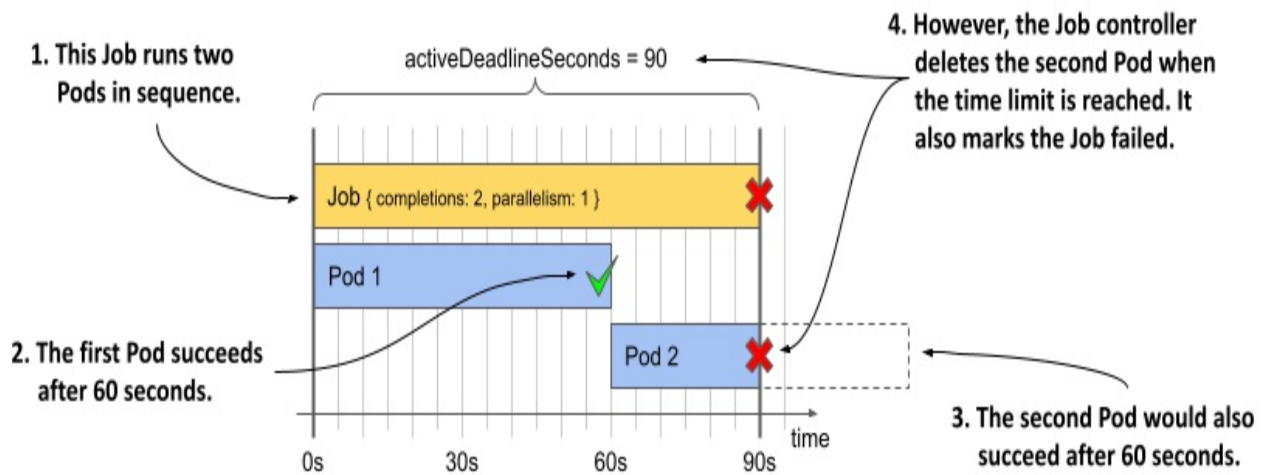
Another way a Job can fail is if it doesn't finish on time. By default, this time isn't limited, but you can set the maximum time using the `activeDeadlineSeconds` field in the Job's spec, as shown in the following listing (see the manifest file `job.demo-deadline.yaml`):

Listing 17.5 A Job with a time limit

```
apiVersion: batch/v1
kind: Job
metadata:
  name: demo-deadline
spec:
  completions: 2    #A
  parallelism: 1    #B
  activeDeadlineSeconds: 90    #C
  template:
    spec:
      restartPolicy: OnFailure
      containers:
      - name: demo-suspend
        image: busybox
        command:
        - sleep    #D
        - "60"    #D
```

From the completions field shown in the listing, you can see that the Job requires two completions to be completed. Since `parallelism` is set to 1, the two Pods run one after the other. Given the sequential execution of these two Pods and the fact that each Pod needs 60 seconds to complete, the execution of the entire Job takes just over 120 seconds. However, since `activeDeadlineSeconds` for this Job is set to 90, the Job can't be successful. The following figure illustrates this scenario.

Figure 17.5 Setting a time limit for a Job



To see this for yourself, create this Job by applying the manifest and wait for it to fail. When it does, the following Event is generated by the Job controller:

```
$ kubectl describe job demo-deadline
...
Events:
  Type      Reason            Age   From           Message
  ----      -
  Warning   DeadlineExceeded  1m    job-controller  Job was active
                                     deadline
```

Note

Remember that the `activeDeadlineSeconds` in a Job applies to the Job as a whole, not to the individual Pods created in the context of that Job.

17.1.4 Parameterizing Pods in a Job

Until now, the tasks you performed in each Job were identical to each other. For example, the Pods in the `generate-responses` Job all did the same thing: they inserted a series of responses into the database. But what if you want to run a series of related tasks that aren't identical? Maybe you want each Pod to process only a subset of the data? That's where the Job's `completionMode` field comes in.

At the time of writing, two completion modes are supported: `Indexed` and `NonIndexed`. The Jobs you created so far in this chapter were `NonIndexed`, as this is the default mode. All Pods created by such a Job are indistinguishable from each other. However, if you set the Job's `completionMode` to `Indexed`, each Pod is given an index number that you can use to distinguish the Pods. This allows each Pod to perform only a portion of the entire task. See the following table for a comparison between the two completion modes.

Table 17.2 Supported Job completion modes

Value	Description
<code>NonIndexed</code>	The Job is considered complete when the number of successfully completed Pods created by this Job equals the value of the <code>spec.completions</code> field in the Job manifest. All Pods are equal to each other. This is the default mode.
<code>Indexed</code>	Each Pod is given a completion index (starting at 0) to distinguish the Pods from each other. The Job is considered complete when there is one successfully completed Pod for each index. If a Pod with a particular index fails, the Job controller creates a new Pod with the same index. The completion index assigned to each Pod is specified in the Pod annotation <code>batch.kubernetes.io/job-completion-index</code>

and in the <code>JOB_COMPLETION_INDEX</code> environment variable in the Pod's containers.
--

Note

In the future, Kubernetes may support additional modes for Job processing, either through the built-in Job controller or through additional controllers.

To better understand these completion modes, you'll create a Job that reads the responses in the Quiz database, calculates the number of valid and invalid responses for each day, and stores those results back in the database. You'll do this in two ways, using both completion modes so you understand the difference.

Implementing the aggregation script

As you can imagine, the Quiz database can get very large if many users are using the application. Therefore, you don't want a single Pod to process all the responses, but rather you want each Pod to process only a specific month.

I've prepared a script that does this. The Pods will obtain this script from a ConfigMap. You can find its manifest in the file `cm.aggregate-responses.yaml`. The actual code is unimportant, but what is important is that it accepts two parameters: the *year* and *month* to process. The code reads these parameters via the environment variables `YEAR` and `MONTH`, as you can see in the following listing.

Listing 17.6 The ConfigMap with the MongoDB script for processing Quiz responses

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: aggregate-responses
  labels:
    app: aggregate-responses
data:
  script.js: |
    var year = parseInt(process.env["YEAR"]);    #A
```

```
var month = parseInt(process.env["MONTH"]);    #A
...
```

Apply this ConfigMap manifest to your cluster with the following command:

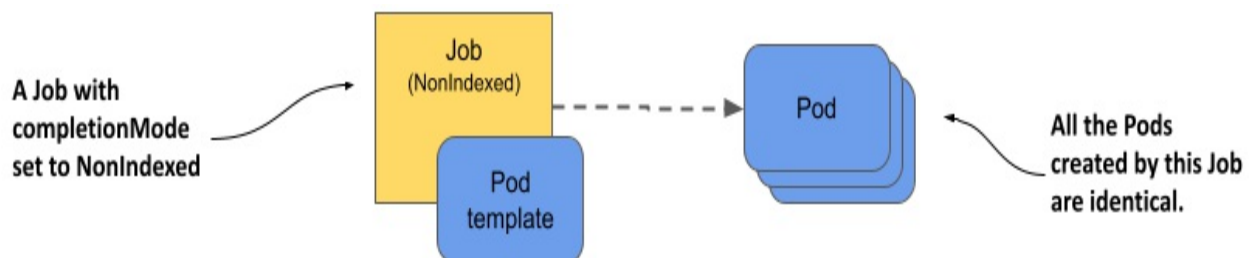
```
$ kubectl apply -f cm.aggregate-responses.yaml
configmap/aggregate-responses created
```

Now imagine you want to calculate the totals for each month of 2020. Since the script only processes a single month, you need 12 Pods to process the whole year. How should you create the Job to generate these Pods, since you need to pass a different month to each Pod?

The NonIndexed completion mode

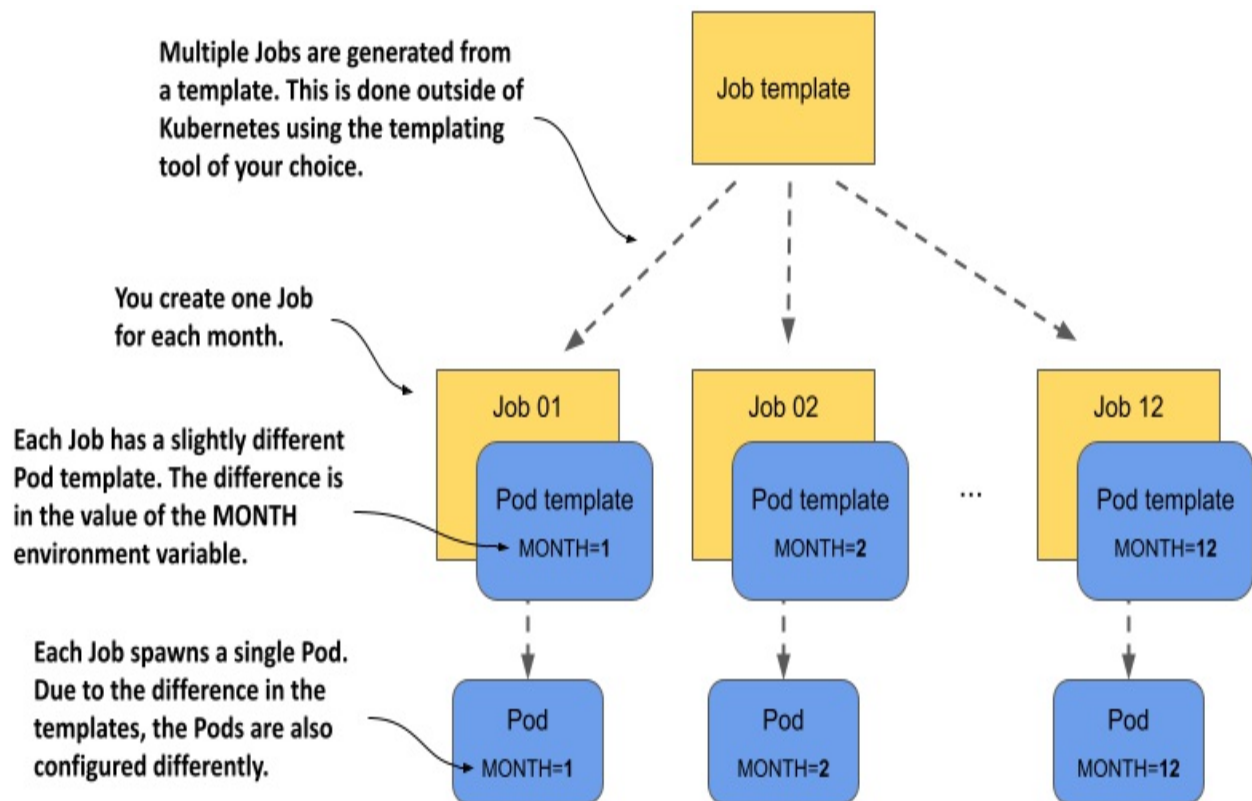
Before `completionMode` support was added to the Job resource, all Jobs operated in the so called `NonIndexed` mode. The problem with this mode is that all generated Pods are identical.

Figure 17.6 Jobs using the NonIndexed completionMode spawn identical Pods



So, if you use this completion mode, you can't pass a different `MONTH` value to each Pod. You must create a separate Job object for each month. This way, each Job can set the `MONTH` environment variable in the Pod template to a different value, as shown in the following figure.

Figure 17.7 Creating similar Jobs from a template



To create these different Jobs, you need to create separate Job manifests. You can do this manually or using an external templating system. Kubernetes itself doesn't provide any functionality for creating Jobs from templates.

Let's return to our example with the aggregate-responses Job. To process the entire year 2020, you need to create twelve Job manifests. You could use a full-blown template engine for this, but you can also do it with a relatively simple shell command.

First you must create the template. You can find it in the file `job.aggregate-responses-2020.tmp1.yaml`. The following listing shows how it looks.

Listing 17.7 A template for creating Job manifests for the aggregate-responses Job

```
apiVersion: batch/v1
kind: Job
metadata:
  name: aggregate-responses-2020-__MONTH__      #A
spec:
  completionMode: NonIndexed
```

```

template:
  spec:
    restartPolicy: OnFailure
    containers:
    - name: updater
      image: mongo:5
      env:
      - name: YEAR
        value: "2020"
      - name: MONTH
        value: "__MONTH__"      #B
    ...

```

If you use Bash, you can create the manifests from this template and apply them directly to the cluster with the following command:

```

$ for month in {1..12}; do \      #A
    sed -e "s/__MONTH__/$month/g" job.aggregate-responses-2020.tm
    | kubectl apply -f - ; \      #C
done
job.batch/aggregate-responses-2020-1 created      #D
job.batch/aggregate-responses-2020-2 created      #D
...      #D
job.batch/aggregate-responses-2020-12 created      #D

```

This command uses a for loop to render the template twelve times. Rendering the template simply means replacing the string `__MONTH__` in the template with the actual month number. The resulting manifest is applied to the cluster using `kubectl apply`.

Note

If you want to run this example but don't use Linux, you can use the manifests I created for you. Use the following command to apply them to your cluster: `kubectl apply -f job.aggregate-responses-2020.generated.yaml`.

The twelve Jobs you just created are now running in your cluster. Each Job creates a single Pod that processes a specific month. To see the generated statistics, use the following command:

```

$ kubectl exec quiz-0 -c mongo -- mongosh kiada --quiet --eval 'd
[

```

```
{      #A
  _id: ISODate("2020-02-28T00:00:00.000Z"),      #A
  totalCount: 120,      #A
  correctCount: 25,      #A
  incorrectCount: 95      #A
},      #A
...
```

If all twelve Jobs processed their respective months, you should see many entries like the one shown here. You can now delete all twelve aggregate-responses Jobs as follows:

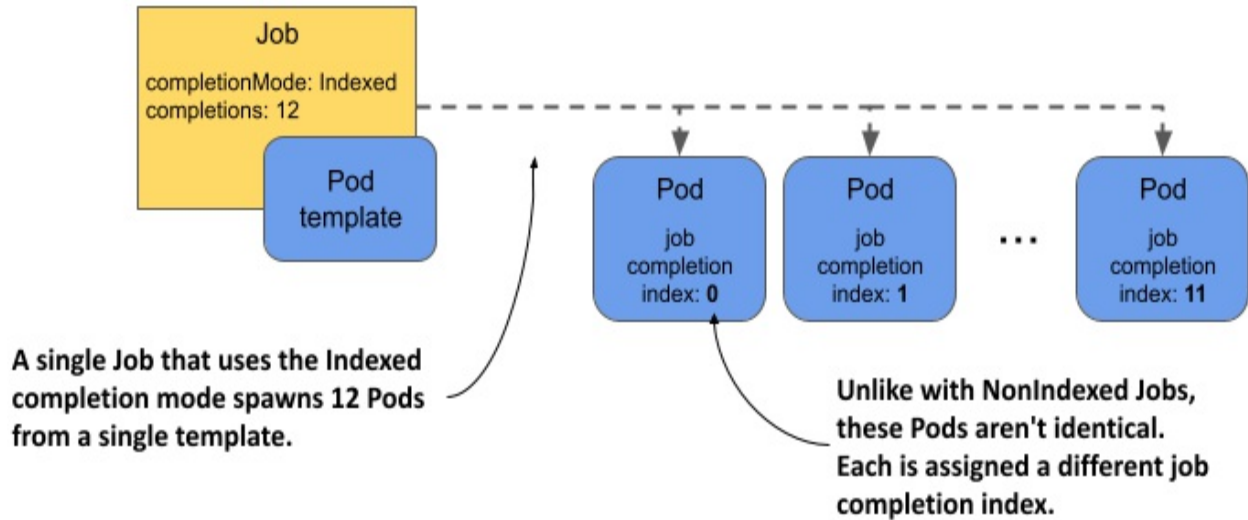
```
$ kubectl delete jobs -l app=aggregate-responses
```

In this example, the parameter passed to each Job was a simple integer, but the real advantage of this approach is that you can pass any value or set of values to each Job and its Pod. The disadvantage, of course, is that you end up with more than one Job, which means more work compared to managing a single Job object. And if you create those Job objects at the same time, they will all run at the same time. That's why creating a single Job using the Indexed completion mode is the better option, as you'll see next.

Introducing the Indexed completion mode

As mentioned earlier, when a Job is configured with the Indexed completion mode, each Pod is assigned a completion index (starting at 0) that distinguishes the Pod from the other Pods in the same Job, as shown in the following figure.

Figure 17.8 Pods spawned by a Job with the Indexed completion mode each get their own index number



The number of Pods is determined by the completions field in the Job's spec. The Job is considered completed when there is one successfully completed Pod for each index.

The following listing shows a Job manifest that uses the Indexed completion mode to run twelve Pods, one for each month. Note that the MONTH environment variable isn't set. This is because the script, as you'll see later, uses the completion index to determine the month to process.

Listing 17.8 A Job manifest using the Indexed completionMode

```
apiVersion: batch/v1
kind: Job
metadata:
  name: aggregate-responses-2021
  labels:
    app: aggregate-responses
    year: "2021"
spec:
  completionMode: Indexed      #A
  completions: 12             #B
  parallelism: 3               #C
  template:
    metadata:
      labels:
        app: aggregate-responses
        year: "2021"
    spec:
      restartPolicy: OnFailure
```

```

containers:
- name: updater
  image: mongo:5
  env:
    - name: YEAR      #D
      value: "2021"    #D
  command:
    - mongosh
    - mongodb+srv://quiz-pods.kiada.svc.cluster.local/kiada?t
    - --quiet
    - --file
    - /script.js
  volumeMounts:
    - name: script
      subPath: script.js
      mountPath: /script.js
volumes:
- name: script
  configMap:      #E
    name: aggregate-responses-indexed      #E

```

In the listing, the completionMode is Indexed and the number of completions is 12, as you might expect. To run three Pods in parallel, parallelism is set to 3.

The JOB_COMPLETION_INDEX environment variable

Unlike in the aggregate-responses-2020 example, in which you passed in both the YEAR and MONTH environment variables, here you pass in only the YEAR variable. To determine which month the Pod should process, the script looks up the environment variable JOB_COMPLETION_INDEX, as shown in the following listing.

Listing 17.9 Using the JOB_COMPLETION_INDEX environment variable in your code

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: aggregate-responses-indexed
  labels:
    app: aggregate-responses-indexed
data:
  script.js: |

```



```
var year = parseInt(process.env["YEAR"]);
var month = parseInt(process.env["JOB_COMPLETION_INDEX"]) + 1
...
```

This environment variable isn't specified in the Pod template but is added to each Pod by the Job controller. The workload running in the Pod can use this variable to determine which part of a dataset to process.

In the aggregate-responses example, the value of the variable represents the month number. However, because the environment variable is zero-based, the script must increment the value by 1 to get the month.

The job-completion-index annotation

In addition to setting the environment variable, the Job controller also sets the job completion index in the `batch.kubernetes.io/job-completion-index` annotation of the Pod. Instead of using the `JOB_COMPLETION_INDEX` environment variable, you can pass the index via any environment variable by using the Downward API, as explained in chapter 9. For example, to pass the value of this annotation to the `MONTH` environment variable, the `env` entry in the Pod template would look like this:

```
env:
- name: MONTH      #A
  valueFrom:        #B
    fieldRef:       #B
      fieldPath: metadata.annotations['batch.kubernetes.io/job-co
```

You might think that with this approach you could just use the same script as in the `aggregate-responses-2020` example, but that's not the case. Since you can't do math when using the Downward API, you'd have to modify the script to properly handle the `MONTH` environment variable, which starts at 0 instead of 1.

Running an Indexed Job

To run this indexed variant of the `aggregate-responses` Job, apply the manifest file `job.aggregate-responses-2021-indexed.yaml`. You can then track the created Pods by running the following command:

```
$ kubectl get pods -l job-name=aggregate-responses-2021
```

NAME	READY	STATUS	RESTARTS	A
aggregate-responses-2021-0-kptfr	1/1	Running	0	2
aggregate-responses-2021-1-r4vfq	1/1	Running	0	2
aggregate-responses-2021-2-snz4m	1/1	Running	0	2

Did you notice that the Pod names contain the job completion index? The Job name is `aggregate-responses-2021`, but the Pod names are in the form `aggregate-responses-2021-<index>-<random string>`.

Note

The completion index also appears in the Pod hostname. The hostname is of the form `<job-name>-<index>`. This facilitates communication between Pods of an indexed Job, as you'll see in a later section.

Now check the Job status with the following command:

```
$ kubectl get jobs
```

NAME	COMPLETIONS	DURATION	AGE
aggregate-responses-2021	7/12	2m17s	2m17s

Unlike the example where you used multiple Jobs with the `NonIndexed` completion mode, all the work is done with a single Job object, which makes things much more manageable. Although there are still twelve Pods, you don't have to care about them unless the Job fails. When you see that the Job is completed, you can be sure that the task is done, and you can delete the Job to clean everything up.

Using the job completion index in more advanced use-cases

In the previous example, the code in the workload used the job completion index directly as input. But what about tasks where the input isn't a simple number?

For example, imagine a container image that accepts an input file and processes it in some way. It expects the file to be in a certain location and have a certain name. Suppose the file is called `/var/input/file.bin`. You want to use this image to process 1000 files. Can you do that with an indexed

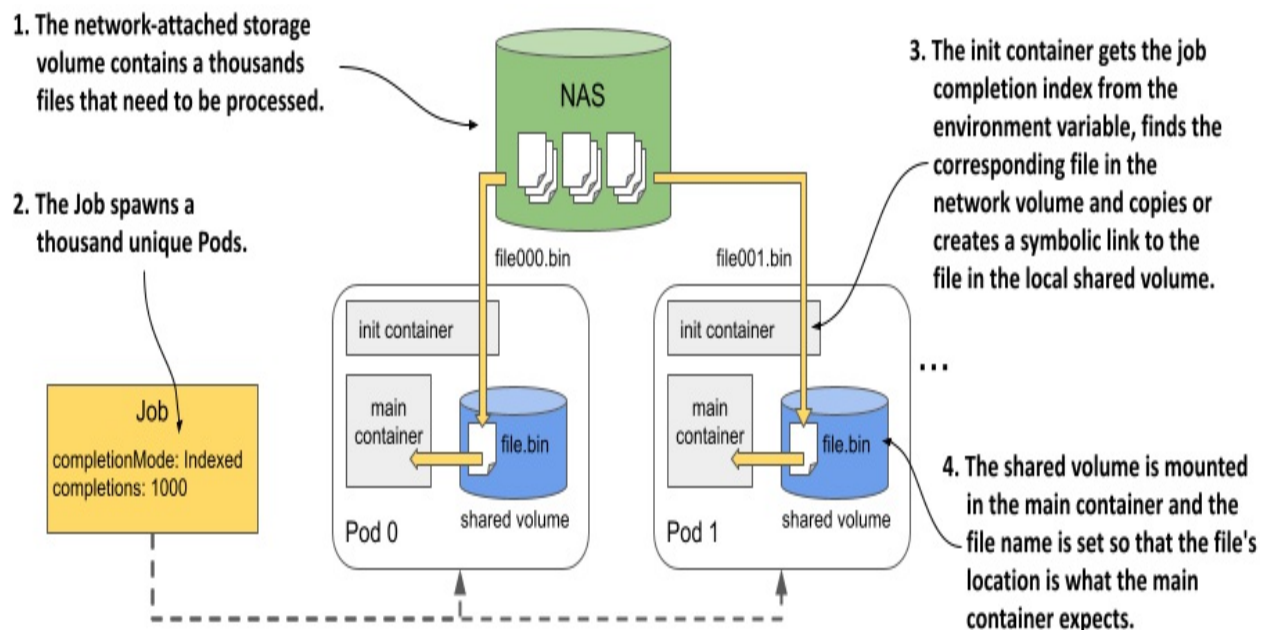
job without changing the code in the image?

Yes, you can! By adding an init container and a volume to the Pod template. You create a Job with `completionMode` set to `Indexed` and `completions` set to `1000`. In the Job's Pod template, you add two containers and a volume that is shared by these two containers. One container runs the image that processes the file. Let's call this the main container. The other container is an init container that reads the completion index from the environment variable and prepares the input file on the shared volume.

If the thousand files you need to process are on a network volume, you can also mount that volume in the Pod and have the init container create a symbolic link named `file.bin` in the Pod's shared internal volume to one of the files in the network volume. The init container must make sure that each completion index corresponds to a different file in the network volume.

If the internal volume is mounted in the main container at `/var/input`, the main container can process the file without knowing anything about the completion index or the fact that there are a thousand files being processed. The following figure shows how all this would look.

Figure 17.9 An init container providing the input file to the main container based on the job completion index



As you can see, even though an indexed Job provides only a simple integer to each Pod, there is a way to use that integer to prepare much more complex input data for the workload. All you need is an init container that transforms the integer into this input data.

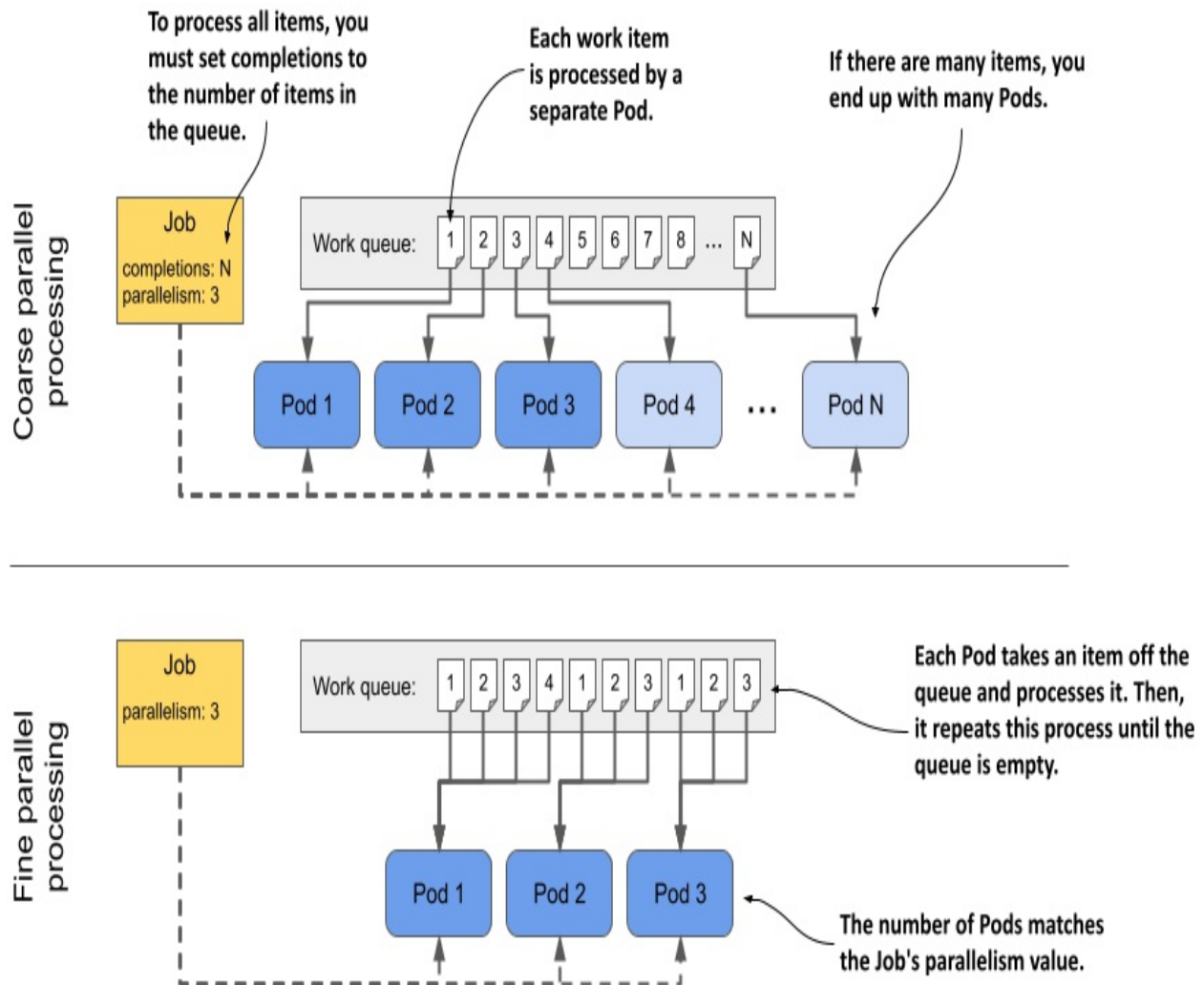
17.1.5 Running Jobs with a work queue

The Jobs in the previous section were assigned static work. However, often the work to be performed is assigned dynamically using a work queue. Instead of specifying the input data in the Job itself, the Pod retrieves that data from the queue. In this section, you'll learn two methods for processing a work queue in a Job.

The previous paragraph may have given the impression that Kubernetes itself provides some kind of queue-based processing, but that isn't the case. When we talk about Jobs that use a queue, the queue and the component that retrieves the work items from that queue need to be implemented in your containers. Then you create a Job that runs those containers in one or more Pods. To learn how to do this, you'll now implement another variant of the aggregate-responses Job. This one uses a queue as the source of the work to be executed.

There are two ways to process a work queue: *coarse* or *fine*. The following figure illustrates the difference between these two methods.

Figure 17.10 The difference between coarse and fine parallel processing



In *coarse* parallel processing, each Pod takes an item from the queue, processes it, and then terminates. Therefore, you end up with one Pod per work item. In contrast, in *fine* parallel processing, typically only a handful of Pods are created and each Pod processes multiple work items. They all work in parallel until the entire queue is processed. In both methods, you can run as many Pods in parallel as you want, if your cluster can accommodate them.

Creating the work queue

The Job you'll create for this exercise will process the Quiz responses from 2022. Before you create this Job, you must first set up the work queue. To keep things simple, you implement the queue in the existing MongoDB

database. To create the queue, you run the following command:

```
$ kubectl exec -it quiz-0 -c mongo -- mongosh kiada --eval '
db.monthsToProcess.insertMany([
  {_id: "2022-01", year: 2022, month: 1},
  {_id: "2022-02", year: 2022, month: 2},
  {_id: "2022-03", year: 2022, month: 3},
  {_id: "2022-04", year: 2022, month: 4},
  {_id: "2022-05", year: 2022, month: 5},
  {_id: "2022-06", year: 2022, month: 6},
  {_id: "2022-07", year: 2022, month: 7},
  {_id: "2022-08", year: 2022, month: 8},
  {_id: "2022-09", year: 2022, month: 9},
  {_id: "2022-10", year: 2022, month: 10},
  {_id: "2022-11", year: 2022, month: 11},
  {_id: "2022-12", year: 2022, month: 12}])'
```

NOTE

This command assumes that quiz-0 is the primary MongoDB replica. If the command fails with the error message “not primary”, try running the command in all three Pods, or you can ask MongoDB which of the three is the primary replica with the following command: `kubectl exec quiz-0 -c mongo -- mongosh --eval 'rs.hello().primary'`.

The command inserts 12 work items into the MongoDB collection named `monthsToProcess`. Each work item represents a particular month that needs to be processed.

Processing a work queue using coarse parallel processing

Let’s start with an example of coarse parallel processing, where each Pod processes only a single work item. You can find the Job manifest in the file `job.aggregate-responses-queue-coarse.yaml` and is shown in the following listing.

Listing 17.10 Processing a work queue using coarse parallel processing

```
apiVersion: batch/v1
kind: Job
metadata:
```

```

    name: aggregate-responses-queue-coarse
spec:
  completions: 6      #A
  parallelism: 3      #B
  template:
    spec:
      restartPolicy: OnFailure
      containers:
      - name: processor
        image: mongo:5
        command:
        - mongosh      #C
        - mongodb+srv://quiz-pods.kiada.svc.cluster.local/kiada?t
        - --quiet      #C
        - --file      #C
        - /script.js    #C
        volumeMounts:    #D
        - name: script    #D
          subPath: script.js    #D
          mountPath: /script.js    #D
      volumes:    #D
      - name: script    #D
        configMap:    #D
          name: aggregate-responses-queue-coarse    #D

```

The Job creates Pods that run a script in MongoDB that takes a single item from the queue and processes it. Note that `completions` is 6, meaning that this Job only processes 6 of the 12 items you added to the queue. The reason for this is that I want to leave a few items for the fine parallel processing example that comes after this one.

The `parallelism` setting for this Job is 3, which means that three work items are processed in parallel by three different Pods.

The script that each Pod executes is defined in the `aggregate-responses-queue-coarse` ConfigMap. The manifest for this ConfigMap is in the same file as the Job manifest. A rough outline of the script can be seen in the following listing.

Listing 17.11 A MongoDB script processing a single work item

```
print("Fetching one work item from queue...");
```

```

var workItem = db.monthsToProcess.findOneAndDelete({});      #A
if (workItem == null) {      #B
    print("No work item found. Processing is complete.");      #B
    quit(0);      #B
}      #B

print("Found work item:");      #C
print("  Year:  " + workItem.year);      #C
print("  Month: " + workItem.month);      #C

var year = parseInt(workItem.year);      #C
var month = parseInt(workItem.month) + 1;      #C
// code that processes the item      #C

print("Done.");      #D
quit(0);      #D

```

The script takes an item from the work queue. As you know, each item represents a single month. The script performs an aggregation query on the Quiz responses for that month that calculates the number of correct, incorrect, and total responses, and stores the result back in MongoDB.

To run the Job, apply `job.aggregate-responses-queue-coarse.yaml` with `kubectl apply` and observe the status of the Job with `kubectl get jobs`. You can also check the Pods to make sure that three Pods are running in parallel, and that the total number of Pods is six after the Job is complete.

If all goes well, your work queue should now only contain the 6 months that haven't been processed by the Job. You can confirm this by running the following command:

```

$ kubectl exec quiz-0 -c mongo -- mongosh kiada --quiet --eval 'd
[
  { _id: '2022-07', year: 2022, month: 7 },
  { _id: '2022-08', year: 2022, month: 8 },
  { _id: '2022-09', year: 2022, month: 9 },
  { _id: '2022-10', year: 2022, month: 10 },
  { _id: '2022-11', year: 2022, month: 11 },
  { _id: '2022-12', year: 2022, month: 12 }
]

```

You can check the logs of the six Pods to see if they have processed the exact months for which the items were removed from the queue. You'll process the

remaining items with fine parallel processing. Before you continue, please delete the aggregate-responses-queue-coarse Job with `kubectl delete`. This also removes the six Pods.

Processing a work queue using fine parallel processing

In fine parallel processing, each Pod handles multiple work items. It takes an item from the queue, processes it, takes the next item, and repeats this process until there are no items left in the queue. As before, multiple Pods can work in parallel.

The Job manifest is in the file `job.aggregate-responses-queue-fine.yaml`. The Pod template is virtually the same as in the previous example, but it doesn't contain the `completions` field, as you can see in the following listing.

Listing 17.12 Processing a work queue using the fine parallel processing approach

```
apiVersion: batch/v1
kind: Job
metadata:
  name: aggregate-responses-queue-fine
spec:
  parallelism: 3      #A
  template:
    ...
```

A Job that uses fine parallel processing doesn't set the `completions` field because a single successful completion indicates that all the items in the queue have been processed. This is because the Pod terminates with success when it has processed the last work item.

You may wonder what happens if some Pods are still processing their items when another Pod reports success. Fortunately, the Job controller lets the other Pods finish their work. It doesn't kill them.

As before, the manifest file also contains a ConfigMap that contains the MongoDB script. Unlike the previous script, this script processes one work item after the other until the queue is empty, as shown in the following listing.

Listing 17.13 A MongoDB script that processes the entire queue

```
print("Processing quiz responses - queue - all work items");
print("=====");
print();
print("Fetching work items from queue...");
print();

while (true) {      #A
    var workItem = db.monthsToProcess.findOneAndDelete({});    #B
    if (workItem == null) {      #C
        print("No work item found. Processing is complete.");
        quit(0);      #C
    }      #C
    print("Found work item:");      #D
    print("  Year:  " + workItem.year);      #D
    print("  Month: " + workItem.month);      #D
    // process the item      #D
    ...      #D

    print("Done processing item.");      #E
    print("-----");      #E
    print();      #E
}      #E
```

To run this Job, apply the manifest file `job.aggregate-responses-queue-fine.yaml`. You should see three Pods associated with it. When they finish processing the items in the queue, their containers terminate, and the Pods show as Completed:

```
$ kubectl get pods -l job-name=aggregate-responses-queue-fine
NAME                                READY    STATUS    RESTART
aggregate-responses-queue-fine-9slk1 0/1      Completed 0
aggregate-responses-queue-fine-hxqbw 0/1      Completed 0
aggregate-responses-queue-fine-szqks 0/1      Completed 0
```

The status of the Job also indicates that all three Pods have completed:

```
$ kubectl get jobs
NAME                                COMPLETIONS    DURATION    AGE
aggregate-responses-queue-fine     3/1 of 3        3m19s       5m34s
```

The last thing you need to do is check if the work queue is actually empty. You can do that with the following command:

```
$ kubectl exec quiz-1 -c mongo -- mongosh kiada --quiet --eval 'd
0    #A
```

As you can see, the queue is zero, so the Job is completed.

Continuous processing of work queues

To conclude this section on Jobs with work queues, let's see what happens when you add items to the queue after the Job is complete. Add a work item for January 2023 as follows:

```
$ kubectl exec -it quiz-0 -c mongo -- mongosh kiada --quiet --eva
{ acknowledged: true, insertedId: '2023-01' }
```

Do you think the Job will create another Pod to handle this work item? The answer is obvious when you consider that Kubernetes doesn't know anything about the queue, as I explained earlier. Only the containers running in the Pods know about the existence of the queue. So, of course, if you add a new item after the Job finishes, it won't be processed unless you recreate the Job.

Remember that Jobs are designed to run tasks to completion, not continuously. To implement a worker Pod that continuously monitors a queue, you should run the Pod with a Deployment instead. However, if you want to run the Job at regular intervals rather than continuously, you can also use a CronJob, as explained in the second part of this chapter.

17.1.6 Communication between Job Pods

Most Pods running in the context of a Job run independently, unaware of the other Pods running in the same context. However, some tasks require that these Pods communicate with each other.

In most cases, each Pod needs to communicate with a specific Pod or with all its peers, not just with a random Pod in the group. Fortunately, it's trivial to enable this kind of communication. You only have to do three things:

- Set the `completionMode` of the Job to `Indexed`.
- Create a headless Service.

- Configure this Service as a subdomain in the Pod template.

Let me explain this with an example.

Creating the headless Service manifest

Let's first look at how the headless Service must be configured. Its manifest is shown in the following listing.

Listing 17.14 Headless Service for communication between Job Pods

```
apiVersion: v1
kind: Service
metadata:
  name: demo-service
spec:
  clusterIP: none      #A
  selector:
    job-name: comm-demo  #B
  ports:
    - name: http
      port: 80
```

As you learned in chapter 11, you must set `clusterIP` to `none` to make the Service headless. You also need to make sure that the label selector matches the Pods that the Job creates. The easiest way to do this is to use the `job-name` label in the selector. You learned at the beginning of this chapter that this label is automatically added to the Pods. The value of the label is set to the name of the Job object, so you need to make sure that the value you use in the selector matches the Job name.

Creating the Job manifest

Now let's see how the Job manifest must be configured. Examine the following listing.

Listing 17.15 A Job manifest enabling pod-to-pod communication

```
apiVersion: batch/v1
kind: Job
```

```

metadata:
  name: comm-demo      #A
spec:
  completionMode: Indexed      #B
  completions: 2      #C
  parallelism: 2      #C
  template:
    spec:
      subdomain: demo-service      #D
      restartPolicy: Never
      containers:
      - name: comm-demo
        image: busybox
        command:      #E
        - sleep      #E
        - "600"      #E

```

As mentioned earlier, the completion mode must be set to Indexed. This Job is configured to run two Pods in parallel so you can experiment with them. In order for the Pods to find each other via DNS, you need to set their subdomain to the name of the headless Service.

You can find both the Job and the Service manifest in the `job.comm-demo.yaml` file. Create the two objects by applying the file and then list the Pods as follows:

```

$ kubectl get pods -l job-name=comm-demo

```

NAME	READY	STATUS	RESTARTS	AGE
comm-demo-0-mrvlp	1/1	Running	0	34s
comm-demo-1-kvpb4	1/1	Running	0	34s

Note the names of the two Pods. You need them to execute commands in their containers.

Connecting to Pods from other Pods

Check the hostname of the first Pod with the following command. Use the name of your Pod.

```

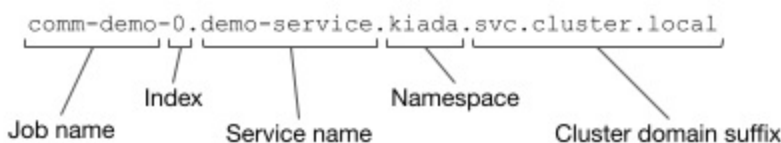
$ kubectl exec comm-demo-0-mrvlp -- hostname -f
comm-demo-0.demo-service.kiada.svc.cluster.local

```

The second Pod can communicate with the first Pod at this address. To confirm this, try pinging the first Pod from the second Pod using the following command (this time, pass the name of your second Pod to the `kubectl exec` command):

```
$ kubectl exec comm-demo-1-kvpb4 -- ping comm-demo-0.demo-service
PING comm-demo-0.demo-service.kiada.svc.cluster.local (10.244.2.7
64 bytes from 10.244.2.71: seq=0 ttl=63 time=0.060 ms
64 bytes from 10.244.2.71: seq=1 ttl=63 time=0.062 ms
...
```

As you can see, the second Pod can communicate with the first Pod without knowing its exact name, which is known to be random. A pod running in the context of a Job can determine the names of its peers according to the following pattern:



But you can simplify the address even further. As you may recall, when resolving DNS records for objects in the same Namespace, you don't have to use the fully qualified domain name. You can omit the Namespace and the cluster domain suffix. So the second Pod can connect to the first Pod using the address `comm-demo-0.demo-service`, as shown in the following example:

```
$ kubectl exec comm-demo-1-kvpb4 -- ping comm-demo-0.demo-service
PING comm-demo-0.demo-service (10.244.2.71): 56 data bytes
64 bytes from 10.244.2.71: seq=0 ttl=63 time=0.040 ms
64 bytes from 10.244.2.71: seq=1 ttl=63 time=0.067 ms
...
```

As long as the Pods know how many Pods belong to the same Job (in other words, what the value of the `completions` field is), they can easily find all their peers via DNS. They don't need to ask the Kubernetes API server for their names or IP addresses.

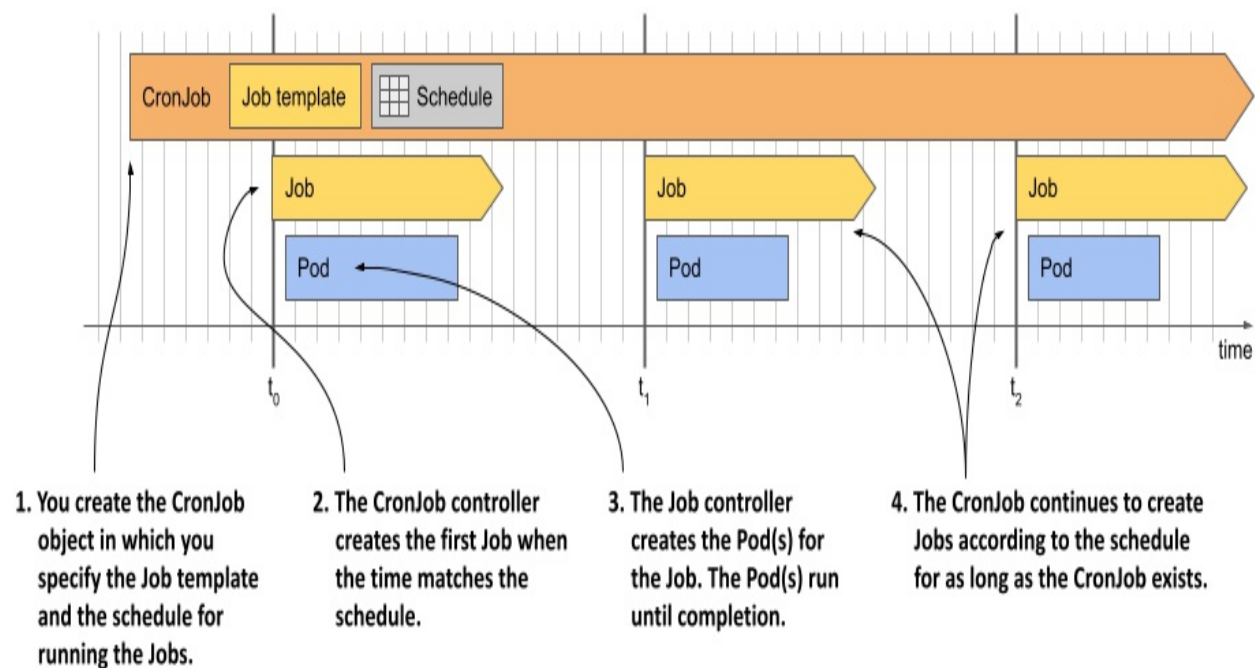
This concludes the first part of this chapter. Please delete any remaining Jobs before continuing.

17.2 Scheduling Jobs with CronJobs

When you create a Job object, it starts executing immediately. Although you can create the Job in a suspended state and later un-suspend it, you cannot configure it to run at a specific time. To achieve this, you can wrap the Job in a CronJob object.

In the CronJob object you specify a Job template and a schedule. According to this schedule, the CronJob controller creates a new Job object from the template. You can set the schedule to do this several times a day, at a specific time of day, or on specific days of the week or month. The controller will continue to create Jobs according to the schedule until you delete the CronJob object. The following figure illustrates how a CronJob works.

Figure 17.11 The operation of a CronJob



As you can see in the figure, each time the CronJob controller creates a Job, the Job controller subsequently creates the Pod(s), just like when you manually create the Job object. Let's see this process in action.

17.2.1 Creating a CronJob

The following listing shows a CronJob manifest that runs a Job every minute. This Job aggregates the Quiz responses received today and updates the daily quiz statistics. You can find the manifest in the `cj.aggregate-responses-every-minute.yaml` file.

Listing 17.16 A CronJob that runs a Job every minute

```
apiVersion: batch/v1      #A
kind: CronJob             #A
metadata:
  name: aggregate-responses-every-minute
spec:
  schedule: "* * * * *"   #B
  jobTemplate:            #C
    metadata:             #C
      labels:             #C
        app: aggregate-responses-today    #C
    spec:                 #C
      template:          #C
        metadata:        #C
          labels:        #C
            app: aggregate-responses-today    #C
        spec:            #C
          restartPolicy: OnFailure            #C
          containers:    #C
            - name: updater    #C
              image: mongo:5    #C
              command:         #C
                - mongosh    #C
                - mongodb+srv://quiz-pods.kiada.svc.cluster.local/kia
                - --quiet    #C
                - --file    #C
                - /script.js    #C
              volumeMounts:    #C
                - name: script    #C
                  subPath: script.js    #C
                  mountPath: /script.js    #C
            volumes:          #C
              - name: script    #C
                configMap:      #C
                  name: aggregate-responses-today    #C
```

As you can see in the listing, a CronJob is just a thin wrapper around a Job. There are only two parts in the CronJob spec: the `schedule` and the `jobTemplate`. You learned how to write a Job manifest in the previous

sections, so that part should be clear. If you know the crontab format, you should also understand how the schedule field works. If not, I explain it in section 17.2.2. First, let's create the CronJob object from the manifest and see it in action.

Running a CronJob

Apply the manifest file to create the CronJob. Use the `kubectl get cj` command to check the object:

```
$ kubectl get cj
NAME                                SCHEDULE    SUSPEND    ACTIVE
aggregate-responses-every-minute    * * * * *   False     0
```

Note

The shorthand for CronJob is `cj`.

Note

When you list CronJobs with the `-o wide` option, the command also shows the container names and images used in the Pod, so you can easily see what the CronJob does.

The command output shows the list of CronJobs in the current Namespace. For each CronJob, the name, schedule, whether the CronJob is suspended, the number of currently active Jobs, the last time a Job was scheduled, and the age of the object are displayed.

As indicated by the information in the columns `ACTIVE` and `LAST SCHEDULE`, no Job has yet been created for this CronJob. The CronJob is configured to create a new Job every minute. The first Job is created when the next minute starts, and the output of the `kubectl get cj` command then looks like this:

```
$ kubectl get cj
NAME                                SCHEDULE    SUSPEND    ACTIVE
aggregate-responses-every-minute    * * * * *   False     1
```

The command output now shows an active Job that was created 2 seconds

ago. Unlike the Job controller, which adds the job-name label to the Pods so you can easily list Pods associated with a Job, the CronJob controller doesn't add labels to the Job. So, if you want to list Jobs created by a specific CronJob, you need to add your own labels to the Job template.

In the manifest for the aggregate-responses-every-minute CronJob, you added the label "app: aggregate-responses-today" to both the Job template and the Pod template within that Job template. This allows you to easily list the Jobs and Pods associated with this CronJob. List the associated Jobs as follows:

```
$ kubectl get jobs -l app=aggregate-responses-today
```

NAME	COMPLETIONS	DURATION
aggregate-responses-every-minute-27755219	1/1	36s

The CronJob has created only one Job so far. As you can see, the Job name is generated from the CronJob name. The number at the end of the name is the scheduled time of the Job in Unix Epoch Time, converted to minutes.

When the CronJob controller creates the Job object, the Job controller creates one or more Pods, depending on the Job template. To list the Pods, you use the same label selector as before. The command looks like this:

```
$ kubectl get pods -l app=aggregate-responses-today
```

NAME	READY	STATUS
aggregate-responses-every-minute-27755219-4s197	0/1	Completed

The status shows that this Pod has completed successfully, but you already knew that from the Job status.

Inspecting the CronJob status in detail

The `kubectl get cronjobs` command only shows the number of currently active Jobs and when the last Job was scheduled. Unfortunately, it doesn't show whether the last Job was successful. To get this information, you can either list the Jobs directly or check the CronJob status in YAML form as follows:

```
$ kubectl get cj aggregate-responses-every-minute -o yaml
```

```

...
status:
  active:      #A
  - apiVersion: batch/v1      #A
    kind: Job      #A
    name: aggregate-responses-every-minute-27755221      #A
    namespace: kiada      #A
    resourceVersion: "5299"      #A
    uid: 430a0064-098f-4b46-b1af-eea690597353      #A
  lastScheduleTime: "2022-10-09T11:01:00Z"      #B
  lastSuccessfulTime: "2022-10-09T11:00:41Z"      #C

```

As you can see, the status section of a CronJob object shows a list with references to the currently running Jobs (field `active`), the last time the Job was scheduled (field `lastScheduleTime`), and the last time the Job completed successfully (field `lastSuccessfulTime`). From the last two fields you can deduce whether the last run was successful.

Inspecting Events associated with a CronJob

To see the full details of a CronJob and all Events associated with the object, use the `kubectl describe` command as follows:

```

$ kubectl describe cj aggregate-responses-every-minute
Name:                aggregate-responses-every-minute
Namespace:           kiada
Labels:              <none>
Annotations:         <none>
Schedule:            * * * * *
Concurrency Policy:   Allow
Suspend:             False
Successful Job History Limit: 3
Failed Job History Limit: 1
Starting Deadline Seconds: <unset>
Selector:            <unset>
Parallelism:         <unset>
Completions:         <unset>
Pod Template:
...
Last Schedule Time:  Sun, 09 Oct 2022 11:01:00 +0200
Active Jobs:         aggregate-responses-every-minute-27755221
Events:
  Type    Reason          Age    From    Message
  ----    -

```

```

Normal SuccessfulCreate 98s cronjob-controller Created job
Normal SawCompletedJob 41s cronjob-controller Saw complet
responses-e
status: Com
...

```

As can be seen in the command output, the CronJob controller generates a `SuccessfulCreate` Event when it creates a Job, and a `SawCompletedJob` Event when the Job completes.

17.2.2 Configuring the schedule

The schedule in the CronJob spec is written in crontab format. If you're not familiar with this syntax, you can find tutorials and explanations online, but the following section is meant as a short introduction.

Understanding the crontab format

A schedule in crontab format consists of five fields and looks as follows:

```

schedule: " * * * * *"
           |  |  |  |  |
           |  |  |  |  |
Minute   Hour Day of Month Month Day of Week
(0-59)   (0-23) (1-31) (1-12 or JAN-DEC) (0-6 or SUN-SAT)

```

From left to right, the fields are the minute, hour, day of the month, month, and day of the week when the schedule should be triggered. In the example, an asterisk (*) appears in each field, meaning that each field matches any value.

If you've never seen a cron schedule before, it may not be obvious that the schedule in this example triggers every minute. But don't worry, this will become clear to you as you learn what values to use instead of asterisks and as you see other examples. In each field, you can specify a specific value, range of values, or group of values instead of the asterisk, as explained in the following table.

Table 17.3 Understanding the patterns in a CronJob's schedule field

Value	Description
5	A single value. For example, if the value 5 is used in the Month field, the schedule will trigger if the current month is May.
MAY	In the Month and Day of week fields, you can use three-letter names instead of numeric values.
1-5	A range of values. The specified range includes both limits. For the Month field, 1-5 corresponds to JAN-MAY, in which case the schedule triggers if the current month is between January and May (inclusive).
1, 2, 5-8	A list of numbers or ranges. In the Month field, 1, 2, 5-8 stands for January, February, May, June, July, and August.
*	Matches the entire range of values. For example, * in the Month field is equivalent to 1-12 or JAN-DEC.
*/3	Every Nth value, starting with the first value. For example, if */3 is used in the Month field, it means that every third month is included in the schedule, while the others aren't. A CronJob using this schedule will be executed in January, April, July, and October.
5/2	Every Nth value, starting with the specified value. In the Month field, 5/2 causes the schedule to trigger every other month, starting in May. In other words, this schedule is triggered if the month is May, July, September, or November.

3-10/2	The /N pattern can also be applied to ranges. In the Month field, 3-10/2 indicates that between March and October, only every other month is included in the schedule. Thus, the schedule includes the months of March, May, July, and September.
--------	---

Of course, these values can appear in different time fields and together they define the exact times at which this schedule is triggered. The following table shows examples of different schedules and their explanations.

Table 17.4 Cron examples

Schedule	Explanation
* * * * *	Every minute (at every minute of every hour, regardless of month, day of the month, or day of the week).
15 * * * *	Fifteen minutes after every hour.
0 0 * 1-3 *	Every day at midnight, but only from January to March.
*/5 18 * * *	Every five minutes between 18:00 (6 PM) and 18:59 (6:59 PM).
* * 7 5 *	Every minute on May 7.
0,30 3 7 5 *	At 3:00AM and 3:30AM on May 7.
0 0 * * 1-5	At 0:00 AM every weekday (Monday through Friday).

Warning

A CronJob creates a new Job when all fields in the crontab match the current date and time, except for the *Day of month* and *Day of week* fields. The CronJob will run if *either* of these fields match. You might expect the schedule “* * 13 * 5” to only trigger on Friday the 13th, but it’ll trigger on every 13th of the Month *as well as* every Friday.

Fortunately, simple schedules don’t have to be specified this way. Instead, you can use one of the following special values:

- @hourly, to run the Job every hour (at the top of the hour),
- @daily, to run it every day at midnight,
- @weekly, to run it every Sunday at midnight,
- @monthly, to run it at 0:00 on the first day of each month,
- @yearly or @annually to run it at 0:00 on January 1st of each year.

Setting the Time Zone to use for scheduling

The CronJob controller, like most other controllers in Kubernetes, runs within the Controller Manager component of the Kubernetes Control Plane. By default, the CronJob controller schedules CronJobs based on the time zone used by the Controller Manager. This can cause your CronJobs to run at times you didn’t intend, especially if the Control Plane is running in another location that uses a different time zone.

By default, the time zone isn’t specified. However, you can specify it using the `timeZone` field in the `spec` section of the CronJob manifest. For example, if you want your CronJob to run Jobs at 3 AM Central European Time (CET time zone), the CronJob manifest should look like the following listing:

Listing 17.17 Setting a time zone for the CronJob schedule

```
apiVersion: batch/v1      #A
kind: CronJob             #A
metadata:
```

```

    name: runs-at-3am-cet
spec:
  schedule: "0 3 * * *"      #A
  timeZone: CET              #A
  jobTemplate:
    ...

```

17.2.3 Suspending and resuming a CronJob

Just as you can suspend a Job, you can suspend a CronJob. At the time of writing, there is no specific `kubectl` command to suspend a CronJob, so you must do so using the `kubectl patch` command as follows:

```
$ kubectl patch cj aggregate-responses-every-minute -p '{"spec":{"cronjob.batch/aggregate-responses-every-minute patched
```

While a CronJob is suspended, the controller doesn't start any new Jobs for it, but allows all Jobs already running to finish, as the following output shows:

```
$ kubectl get cj
NAME                                SCHEDULE      SUSPEND      ACTIVE
aggregate-responses-every-minute    * * * * *     True         1
```

The output shows that the CronJob is suspended, but that a Job is still active. When that Job is finished, no new Jobs will be created until you resume the CronJob. You can do this as follows:

```
$ kubectl patch cj aggregate-responses-every-minute -p '{"spec":{"cronjob.batch/aggregate-responses-every-minute patched
```

As with Jobs, you can create CronJobs in a suspended state and resume them later.

17.2.4 Automatically removing finished Jobs

Your `aggregate-responses-every-minute` CronJob has been active for several minutes, so several Job objects have been created in that time. In my case, the CronJob has been in existence for over ten minutes, which means that more than ten Jobs have been created. However, when I list the Jobs, I

see only see four, as you can see in the following output:

```
$ kubectl get job -l app=aggregate-responses-today
NAME                                     COMPLETIONS   DURATION
aggregate-responses-every-minute-27755408 1/1            57s
aggregate-responses-every-minute-27755409 1/1            61s
aggregate-responses-every-minute-27755410 1/1            53s
aggregate-responses-every-minute-27755411 0/1            5s
```

Why don't I see more Jobs? This is because the CronJob controller automatically deletes completed Jobs. However, not all of them are deleted. In the CronJob's spec, you can use the fields `successfulJobsHistoryLimit` and `failedJobsHistoryLimit` to specify how many successful and failed Jobs to keep. By default, CronJobs keeps 3 successful and 1 failed Job. The Pods associated with each kept Job are also preserved, so you can view their logs.

As an exercise, you can try setting the `successfulJobsHistoryLimit` in the `aggregate-responses-every-minute` CronJob to 1. You can do that by modifying the existing CronJob object with the `kubectl edit` command. After you have updated the CronJob, list the Jobs again to verify that all but one Job has been deleted.

17.2.5 Setting a start deadline

The CronJob controller creates the Job objects at approximately the scheduled time. If the cluster is working normally, there is at most a delay of a few seconds. However, if the cluster's Control Plane is overloaded or if the Controller Manager component running the CronJob controller is offline, this delay may be longer.

If it's crucial that the Job shouldn't start too far after its scheduled time, you can set a deadline in the `startingDeadlineSeconds` field, as shown in the following listing.

Listing 17.18 Specifying a starting deadline in a CronJob

```
apiVersion: batch/v1
kind: CronJob
```

```
spec:
  schedule: "* * * * *"
  startingDeadlineSeconds: 30    #A
  ...
```

If the CronJob controller can't create the Job within 30 seconds of the scheduled time, it won't create it. Instead, a `MissSchedule` event will be generated to inform you why the Job wasn't created.

What happens when the CronJob controller is offline for a long time

If the `startingDeadlineSeconds` field isn't set and the CronJob controller is offline for an extended period of time, undesirable behavior may occur when the controller comes back online. This is because the controller will immediately create all the Jobs that should have been created while it was offline.

However, this will only happen if the number of missing jobs is less than 100. If the controller detects that more than 100 Jobs were missed, it doesn't create any Jobs. Instead, it generates a `TooManyMissedTimes` event. By setting the start deadline, you can prevent this from happening.

17.2.6 Handling Job concurrency

The `aggregate-responses-every-minute` CronJob creates a new Job every minute. What happens if a Job run takes longer than one minute? Does the CronJob controller create another Job even if the previous Job is still running?

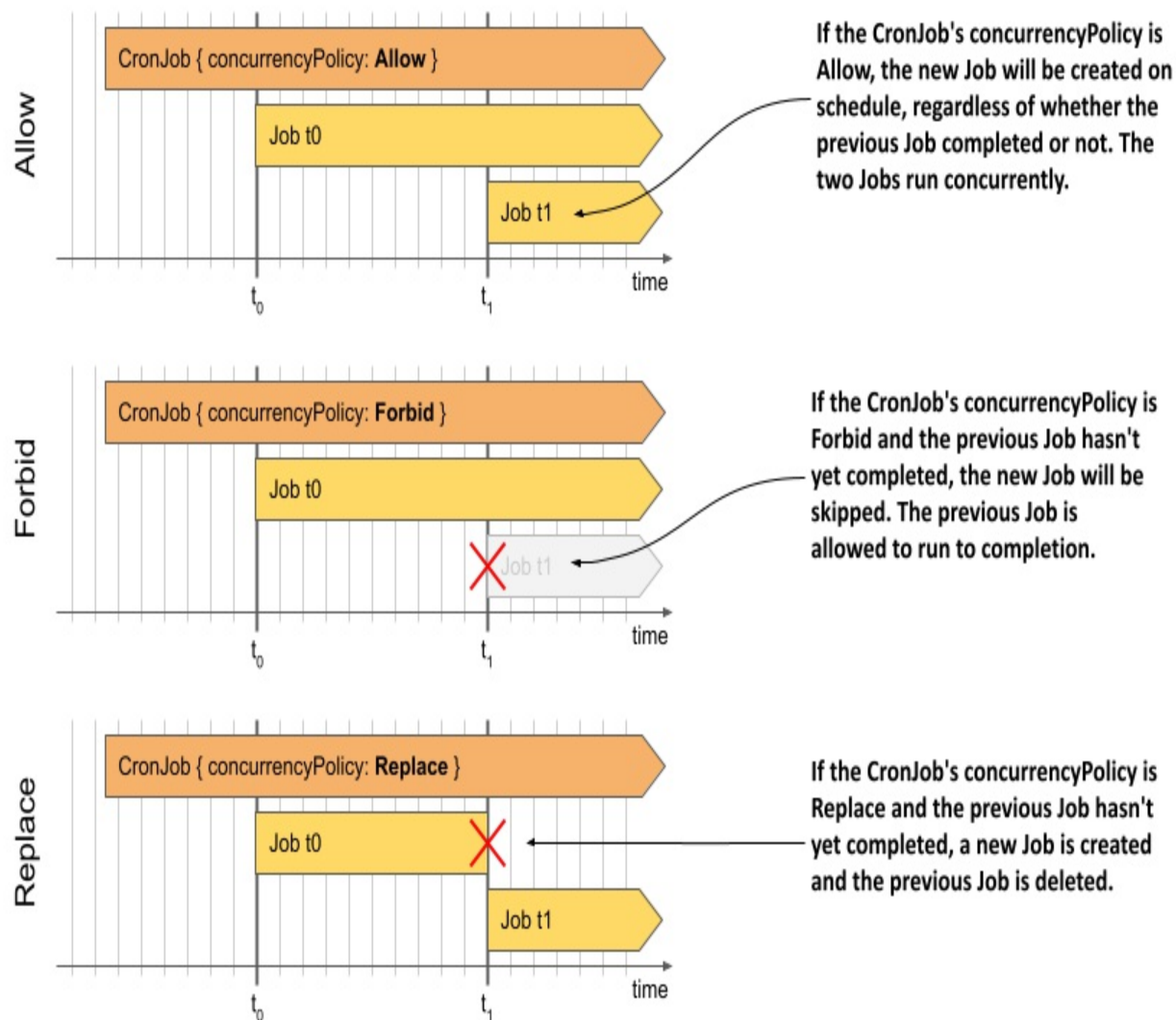
Yes! If you keep an eye on the CronJob status, you may eventually see the following status:

```
$ kubectl get cj
NAME                                SCHEDULE      SUSPEND    ACTIVE
aggregate-responses-every-minute    * * * * *     True       2
```

The `ACTIVE` column indicates that two Jobs are active at the same time. By default, the CronJob controller creates new Jobs regardless of how many previous Jobs are still active. However, you can change this behavior by

setting the concurrencyPolicy in the CronJob spec. The following figure shows the three supported concurrency policies.

Figure 17.12 Comparing the behavior of the three CronJob concurrency policies



For easier reference, the supported concurrency policies are also explained in the following table.

Table 17.5 Supported concurrency policies

Value	Description

Allow	Multiple Jobs are allowed to run at the same time. This is the default setting.
Forbid	Concurrent runs are prohibited. If the previous run is still active when a new run is to be scheduled, the CronJob controller records a JobAlreadyActive event and skips creating a new Job.
Replace	The active Job is canceled and replaced by a new one. The CronJob controller cancels the active Job by deleting the Job object. The Job controller then deletes the Pods, but they're allowed to terminate gracefully. This means that two Jobs are still running at the same time, but one of them is being terminated.

If you want to see how the concurrency policy affects the execution of CronJob, you can try deploying the CronJobs in the following manifest files:

- `cj.concurrency-allow.yaml`,
- `cj.concurrency-forbid.yaml`,
- `cj.concurrency-replace.yaml`.

17.2.7 Deleting a CronJob and its Jobs

To temporarily suspend a CronJob, you can suspend it as described in one of the previous sections. If you want to cancel a CronJob completely, delete the CronJob object as follows:

```
$ kubectl delete cj aggregate-responses-every-minute
cronjob.batch "aggregate-responses-every-minute" deleted
```

When you delete the CronJob, all the Jobs it created will also be deleted. When they're deleted, the Pods are deleted as well, which causes their containers to shut down gracefully.

Deleting the CronJob while preserving the Jobs and their Pods

If you want to delete the CronJob but keep the Jobs and the underlying Pods, you should use the `--cascade=orphan` option when deleting the CronJob, as in the following example:

```
$ kubectl delete cj aggregate-responses-every-minute --cascade=or
```

Note

If you delete a CronJob with the option `--cascade=orphan` while a Job is active, the active Job will be preserved and allowed to complete the task it's executing.

17.3 Summary

In this chapter, you learned about Jobs and CronJobs. You learned that:

- A Job object is used to run workloads that execute a task to completion instead of running indefinitely.
- Running a task with the Job object ensures that the Pod running the task is rescheduled in the event of a node failure.
- A Job can be configured to repeat the same task several times if you set the `completions` field. You can specify the number of tasks that are executed in parallel using the `parallelism` field.
- When a container running a task fails, the failure is handled either at the Pod level by the Kubelet or at the Job level by the Job controller.
- By default, the Pods created by a Job are identical unless you set the Job's `completionMode` to `Indexed`. In that case, each Pod gets its own completion index. This index allows each Pod to process only a certain portion of the data.
- You can use a work queue in a Job, but you must provide your own queue and implement work item retrieval in your container.
- Pods running in a Job can communicate with each other, but you need to define a headless Service so they can find each other via DNS.
- If you want to run a Job at a specific time or at regular intervals, you wrap it in a CronJob. In the CronJob you define the schedule in the well-known crontab format.

This brings us to the end of the second part of this book. You now know how to run all kinds of workloads in Kubernetes. In the next part, you'll learn more about the Kubernetes Control Plane and how it works.